

YTBLND

1.0

Generated by Doxygen 1.14.0

1 Namespace Index	1
1.1 Namespace List	1
2 Hierarchical Index	3
2.1 Class Hierarchy	3
3 Class Index	5
3.1 Class List	5
4 File Index	9
4.1 File List	9
5 Namespace Documentation	13
5.1 ModelJson Namespace Reference	13
5.1.1 Detailed Description	13
5.1.2 Function Documentation	13
5.1.2.1 escape()	13
5.1.2.2 quote()	14
5.1.2.3 tolso8601()	14
6 Class Documentation	15
6.1 ActiveBlendsPanel Class Reference	15
6.1.1 Detailed Description	17
6.1.2 Constructor & Destructor Documentation	17
6.1.2.1 ActiveBlendsPanel()	17
6.1.3 Member Function Documentation	17
6.1.3.1 OnLeaveBlend()	17
6.1.3.2 OnSelectBlend()	17
6.1.3.3 RebuildList()	17
6.1.3.4 Refresh()	17
6.1.4 Member Data Documentation	18
6.1.4.1 m_controller	18
6.1.4.2 m_emptyLabel	18
6.1.4.3 m_listInner	18
6.1.4.4 m_listScroll	18
6.1.4.5 m_listSizer	18
6.1.4.6 m_nav	18
6.2 AppController Class Reference	19
6.2.1 Detailed Description	21
6.2.2 Constructor & Destructor Documentation	21
6.2.2.1 AppController() [1/2]	21
6.2.2.2 AppController() [2/2]	21
6.2.2.3 ~AppController()	21
6.2.3 Member Function Documentation	22

6.2.3.1 clearProgressReporter()	22
6.2.3.2 enrichActiveBlendFeedIfMissingMetadata()	22
6.2.3.3 enrichIfMissingMetadata()	22
6.2.3.4 enrichMissingMetadataSubsetMultithreaded()	22
6.2.3.5 fetchUserBlends()	23
6.2.3.6 get_current_blend_id()	23
6.2.3.7 get_current_blend_title()	23
6.2.3.8 get_current_email()	24
6.2.3.9 get_current_user()	24
6.2.3.10 get_current_username()	24
6.2.3.11 getEventRouter()	24
6.2.3.12 handleCreateBlend()	24
6.2.3.13 handleDeleteAccount()	25
6.2.3.14 handleLeaveBlend()	25
6.2.3.15 handleLogin()	25
6.2.3.16 handleLogout()	25
6.2.3.17 handleOpenChat()	26
6.2.3.18 handlePlayVideo()	26
6.2.3.19 handleRefresh()	26
6.2.3.20 handleRegister()	26
6.2.3.21 handleSelectBlend()	27
6.2.3.22 handleSendMessage()	27
6.2.3.23 handleUploadData()	27
6.2.3.24 loadParticipantsWithWatchLater()	27
6.2.3.25 loadParticipantsWithWatchLaterParallel()	28
6.2.3.26 lookupUser()	28
6.2.3.27 parseBlendID()	28
6.2.3.28 parseChatHistory()	29
6.2.3.29 parseParticipantIDs()	29
6.2.3.30 parseUserFromAuthResponse()	29
6.2.3.31 parseWatchLaterVideos()	30
6.2.3.32 registerBlendOnServer()	30
6.2.3.33 registerEvents()	30
6.2.3.34 reportProgress()	31
6.2.3.35 setProgressReporter()	31
6.2.4 Member Data Documentation	31
6.2.4.1 appState	31
6.2.4.2 currentBlend	31
6.2.4.3 eventRouter	31
6.2.4.4 http	31
6.2.4.5 isConnected	32
6.2.4.6 pendingAccountCreated_	32

6.2.4.7 pendingRegistration_	32
6.2.4.8 progressReporter	32
6.2.4.9 server_url	32
6.3 AppState Class Reference	33
6.3.1 Detailed Description	34
6.3.2 Constructor & Destructor Documentation	34
6.3.2.1 AppState()	34
6.3.3 Member Function Documentation	34
6.3.3.1 addSessionUser()	34
6.3.3.2 clearSession()	35
6.3.3.3 clearSessionUsers()	35
6.3.3.4 createChatRoom()	35
6.3.3.5 getActiveBlend()	35
6.3.3.6 getActiveChatRoom()	35
6.3.3.7 getChatRoom()	35
6.3.3.8 getCurrentUser()	36
6.3.3.9 getInstance()	36
6.3.3.10 getSessionUsers()	36
6.3.3.11 getUsersWithoutData()	36
6.3.3.12 isGeneratingBlend()	37
6.3.3.13 setActiveBlend()	37
6.3.3.14 setCurrentUser()	37
6.3.3.15 setIsBlendGenerating()	37
6.3.3.16 setPendingBlendMessage()	37
6.3.3.17 setPendingUploadError()	38
6.3.3.18 setUsersWithoutData()	38
6.3.3.19 takePendingBlendMessage()	38
6.3.3.20 takePendingUploadError()	39
6.3.4 Member Data Documentation	39
6.3.4.1 activeBlend	39
6.3.4.2 chatRooms	39
6.3.4.3 currentUser	39
6.3.4.4 isBlendGenerating	39
6.3.4.5 pendingBlendMessage	40
6.3.4.6 pendingUploadError	40
6.3.4.7 sessionUsers	40
6.3.4.8 usersWithoutData	40
6.4 Blend Class Reference	40
6.4.1 Detailed Description	41
6.4.2 Constructor & Destructor Documentation	41
6.4.2.1 Blend()	41
6.4.3 Member Function Documentation	42

6.4.3.1	getAlgorithmUsed()	42
6.4.3.2	getBlendID()	42
6.4.3.3	getParticipants()	42
6.4.3.4	getTitle()	42
6.4.3.5	getVideo()	42
6.4.3.6	getVideoList()	43
6.4.3.7	setBlendID()	43
6.4.3.8	setVideoList()	43
6.4.3.9	size()	44
6.4.3.10	toString()	44
6.4.4	Member Data Documentation	44
6.4.4.1	algorithmUsed	44
6.4.4.2	blendID	44
6.4.4.3	participants	44
6.4.4.4	title	44
6.4.4.5	videoList	45
6.5	BlendChatPanel Class Reference	45
6.5.1	Detailed Description	47
6.5.2	Constructor & Destructor Documentation	47
6.5.2.1	BlendChatPanel()	47
6.5.3	Member Function Documentation	47
6.5.3.1	ConnectWebSocket()	47
6.5.3.2	DoSend()	47
6.5.3.3	FormatTimestamp()	48
6.5.3.4	OnInputEnter()	48
6.5.3.5	OnParticipantsToggle()	48
6.5.3.6	OnPollTimer()	48
6.5.3.7	OnSend()	49
6.5.3.8	Refresh()	49
6.5.3.9	Reload()	49
6.5.4	Member Data Documentation	49
6.5.4.1	m_controller	49
6.5.4.2	m_incoming	49
6.5.4.3	m_incomingMx	49
6.5.4.4	m_inputCtrl	50
6.5.4.5	m_inputPanel	50
6.5.4.6	m_msgInner	50
6.5.4.7	m_msgScroll	50
6.5.4.8	m_msgSizer	50
6.5.4.9	m_nav	50
6.5.4.10	m_noBlendLabel	50
6.5.4.11	m_participantsBtn	50

6.5.4.12 m_participantsDropdown	51
6.5.4.13 m_participantsSizer	51
6.5.4.14 m_pollTimer	51
6.5.4.15 m_ws	51
6.5.4.16 m_wsBlendID	51
6.6 BlendCreationPanel Class Reference	52
6.6.1 Detailed Description	53
6.6.2 Constructor & Destructor Documentation	53
6.6.2.1 BlendCreationPanel()	53
6.6.3 Member Function Documentation	53
6.6.3.1 OnAdd()	53
6.6.3.2 OnCreate()	54
6.6.3.3 OnRemoveUser()	54
6.6.3.4 RebuildUserList()	54
6.6.3.5 Refresh()	54
6.6.3.6 Reload()	54
6.6.3.7 UpdateCountLabel()	54
6.6.4 Member Data Documentation	54
6.6.4.1 m_addedUsers	54
6.6.4.2 m_blendNameCtrl	55
6.6.4.3 m_controller	55
6.6.4.4 m_countLabel	55
6.6.4.5 m_createBtn	55
6.6.4.6 m_emptyLabel	55
6.6.4.7 m_nav	55
6.6.4.8 m_userListInner	55
6.6.4.9 m_userListScroll	55
6.6.4.10 m_userListSizer	56
6.7 BlendFeedPanel Class Reference	56
6.7.1 Detailed Description	58
6.7.2 Constructor & Destructor Documentation	58
6.7.2.1 BlendFeedPanel()	58
6.7.3 Member Function Documentation	58
6.7.3.1 LoadFromBlend()	58
6.7.3.2 NextPage()	58
6.7.3.3 Refresh()	58
6.7.4 Member Data Documentation	59
6.7.4.1 kPageSize	59
6.7.4.2 m_cards	59
6.7.4.3 m_controller	59
6.7.4.4 m_offset	59
6.8 AppController::BlendSummary Struct Reference	59

6.8.1 Detailed Description	59
6.8.2 Member Data Documentation	60
6.8.2.1 blendID	60
6.8.2.2 title	60
6.9 BtnConfig Struct Reference	60
6.9.1 Detailed Description	60
6.9.2 Member Data Documentation	60
6.9.2.1 maxHeight	60
6.9.2.2 maxWidth	61
6.9.2.3 minHeight	61
6.9.2.4 minWidth	61
6.10 Channel Class Reference	61
6.10.1 Detailed Description	61
6.10.2 Constructor & Destructor Documentation	61
6.10.2.1 Channel()	61
6.10.3 Member Function Documentation	62
6.10.3.1 getCategories()	62
6.10.3.2 getChannelID()	62
6.10.3.3 getDisplayName()	62
6.10.3.4 toString()	62
6.10.4 Member Data Documentation	63
6.10.4.1 categories	63
6.10.4.2 channelID	63
6.10.4.3 displayName	63
6.11 ChatRoom Class Reference	63
6.11.1 Detailed Description	64
6.11.2 Constructor & Destructor Documentation	64
6.11.2.1 ChatRoom()	64
6.11.3 Member Function Documentation	64
6.11.3.1 addMessage() [1/2]	64
6.11.3.2 addMessage() [2/2]	64
6.11.3.3 clearMessages()	65
6.11.3.4 getBlendID()	65
6.11.3.5 getMessages()	65
6.11.3.6 getParticipantIDs()	65
6.11.3.7 isParticipant()	65
6.11.3.8 toString()	66
6.11.4 Member Data Documentation	66
6.11.4.1 blendID	66
6.11.4.2 messages	66
6.11.4.3 participantIDs	66
6.12 ChatWebSocket Class Reference	67

6.12.1 Detailed Description	67
6.12.2 Constructor & Destructor Documentation	67
6.12.2.1 ChatWebSocket()	67
6.12.2.2 ~ChatWebSocket()	68
6.12.3 Member Function Documentation	68
6.12.3.1 build_fullUrl()	68
6.12.3.2 connect()	68
6.12.3.3 content_formatter()	68
6.12.3.4 get_fullUrl()	69
6.12.3.5 send_message()	69
6.12.3.6 set_blendID()	69
6.12.3.7 set_userID()	69
6.12.3.8 setOnMessage()	69
6.12.4 Member Data Documentation	70
6.12.4.1 blendID	70
6.12.4.2 fullUrl	70
6.12.4.3 isStarted	70
6.12.4.4 m_onMessage	70
6.12.4.5 url	70
6.12.4.6 userID	70
6.12.4.7 ws	71
6.13 ConfirmationDialog Class Reference	71
6.13.1 Detailed Description	72
6.13.2 Constructor & Destructor Documentation	72
6.13.2.1 ConfirmationDialog()	72
6.13.3 Member Function Documentation	72
6.13.3.1 Confirm()	72
6.14 CsvParser Class Reference	73
6.14.1 Detailed Description	74
6.14.2 Constructor & Destructor Documentation	74
6.14.2.1 CsvParser() [1/2]	74
6.14.2.2 CsvParser() [2/2]	74
6.14.2.3 ~CsvParser()	74
6.14.3 Member Function Documentation	74
6.14.3.1 getParsed()	74
6.14.3.2 parse()	75
6.14.3.3 setData()	75
6.14.4 Member Data Documentation	75
6.14.4.1 parse_id	75
6.14.4.2 raw_data	75
6.15 CsvSource Class Reference	76
6.15.1 Detailed Description	77

6.15.2 Constructor & Destructor Documentation	77
6.15.2.1 CsvSource()	77
6.15.3 Member Function Documentation	77
6.15.3.1 getSourceId()	77
6.15.3.2 read()	77
6.15.3.3 setSource()	77
6.15.4 Member Data Documentation	78
6.15.4.1 source_id	78
6.15.4.2 src	78
6.16 DataExtractor Class Reference	78
6.16.1 Detailed Description	79
6.16.2 Constructor & Destructor Documentation	79
6.16.2.1 DataExtractor()	79
6.16.2.2 ~DataExtractor()	79
6.16.3 Member Function Documentation	79
6.16.3.1 extract()	79
6.16.3.2 setParser()	80
6.16.3.3 setSource()	80
6.16.4 Member Data Documentation	80
6.16.4.1 par_format	80
6.16.4.2 src_format	80
6.17 DataInstructionsPanel Class Reference	81
6.17.1 Detailed Description	82
6.17.2 Constructor & Destructor Documentation	82
6.17.2.1 DataInstructionsPanel()	82
6.17.3 Member Function Documentation	82
6.17.3.1 OnBrowse()	82
6.17.4 Member Data Documentation	82
6.17.4.1 m_controller	82
6.17.4.2 m_nav	83
6.18 DataManager Class Reference	83
6.18.1 Detailed Description	83
6.18.2 Member Function Documentation	84
6.18.2.1 createUser()	84
6.18.2.2 findBlendForUser()	84
6.18.2.3 findUserById()	84
6.18.2.4 loadBlendParticipants()	84
6.18.2.5 loadWatchLater()	85
6.18.2.6 saveBlend()	85
6.18.2.7 saveWatchLater()	85
6.18.2.8 validatePassword()	86
6.19 DataSaver Class Reference	86

6.19.1 Detailed Description	86
6.19.2 Constructor & Destructor Documentation	86
6.19.2.1 DataSaver()	86
6.19.3 Member Function Documentation	87
6.19.3.1 save()	87
6.19.3.2 setFileBin()	87
6.19.3.3 setFileFormatter()	87
6.19.4 Member Data Documentation	88
6.19.4.1 bins	88
6.19.4.2 formatters	88
6.20 EventRouter Class Reference	88
6.20.1 Detailed Description	88
6.20.2 Constructor & Destructor Documentation	89
6.20.2.1 EventRouter()	89
6.20.3 Member Function Documentation	89
6.20.3.1 deregisterAll()	89
6.20.3.2 dispatch()	89
6.20.3.3 registerListener()	89
6.20.4 Member Data Documentation	90
6.20.4.1 listeners	90
6.21 File_ID Class Reference	90
6.21.1 Detailed Description	90
6.21.2 Member Data Documentation	90
6.21.2.1 CSV	90
6.21.2.2 HTML	90
6.21.2.3 JSON	91
6.22 FileBin Class Reference	91
6.22.1 Detailed Description	91
6.22.2 Constructor & Destructor Documentation	91
6.22.2.1 ~FileBin()	91
6.22.3 Member Function Documentation	91
6.22.3.1 setFileName()	91
6.22.3.2 setOutputFile()	92
6.22.3.3 setRawData()	92
6.22.3.4 write()	92
6.23 FileFormatter Class Reference	92
6.23.1 Detailed Description	93
6.23.2 Constructor & Destructor Documentation	93
6.23.2.1 ~FileFormatter()	93
6.23.3 Member Function Documentation	93
6.23.3.1 format()	93
6.23.3.2 setData()	93

6.24 FileSource Class Reference	94
6.24.1 Detailed Description	94
6.24.2 Member Function Documentation	94
6.24.2.1 getSourceId()	94
6.24.2.2 read()	95
6.24.2.3 setSource()	95
6.25 Friend Class Reference	95
6.25.1 Detailed Description	96
6.25.2 Constructor & Destructor Documentation	96
6.25.2.1 Friend()	96
6.25.3 Member Function Documentation	96
6.25.3.1 getDisplayName()	96
6.25.3.2 getEmail()	97
6.25.3.3 getUserID()	97
6.25.3.4 setDisplayName()	97
6.25.3.5 setEmail()	97
6.25.3.6 toString()	98
6.25.4 Member Data Documentation	98
6.25.4.1 displayName	98
6.25.4.2 email	98
6.25.4.3 userID	98
6.26 HtmlParser Class Reference	99
6.26.1 Detailed Description	100
6.26.2 Constructor & Destructor Documentation	100
6.26.2.1 HtmlParser() [1/2]	100
6.26.2.2 HtmlParser() [2/2]	100
6.26.3 Member Function Documentation	100
6.26.3.1 extract_links()	100
6.26.3.2 get_text()	100
6.26.3.3 getParsed()	101
6.26.3.4 parse()	101
6.26.3.5 setData()	101
6.26.4 Member Data Documentation	101
6.26.4.1 parse_id	101
6.26.4.2 raw_data	102
6.27 HtmlSource Class Reference	102
6.27.1 Detailed Description	103
6.27.2 Constructor & Destructor Documentation	103
6.27.2.1 HtmlSource()	103
6.27.3 Member Function Documentation	103
6.27.3.1 getSourceId()	103
6.27.3.2 read()	103

6.27.3.3 setSource()	104
6.27.4 Member Data Documentation	104
6.27.4.1 source_id	104
6.27.4.2 src_path	104
6.28 HttpClient Class Reference	104
6.28.1 Detailed Description	105
6.28.2 Constructor & Destructor Documentation	105
6.28.2.1 HttpClient()	105
6.28.3 Member Function Documentation	106
6.28.3.1 build_auth_user_lookup_endpoint()	106
6.28.3.2 build_blend_participant_endpoint()	106
6.28.3.3 build_chat_history_endpoint()	106
6.28.3.4 build_chatroom_detail_endpoint()	107
6.28.3.5 build_delete_user_endpoint()	107
6.28.3.6 build_latest_blend_endpoint()	107
6.28.3.7 build_leave_blend_endpoint()	108
6.28.3.8 build_user_blends_endpoint()	108
6.28.3.9 build_watch_later_endpoint()	108
6.28.3.10 del()	109
6.28.3.11 get()	109
6.28.3.12 getLastStatusCode()	110
6.28.3.13 isRequestSuccessful()	110
6.28.3.14 post()	110
6.28.3.15 request()	111
6.28.3.16 wasLastRequestSuccessful()	111
6.28.3.17 write_callback()	112
6.28.4 Member Data Documentation	112
6.28.4.1 ALT_SUCC	112
6.28.4.2 AUTH_USER_PREFIX	112
6.28.4.3 baseUrl	112
6.28.4.4 BLEND	113
6.28.4.5 D	113
6.28.4.6 DELETE_USER_PREFIX	113
6.28.4.7 DUPLICATE	113
6.28.4.8 G	113
6.28.4.9 hdr	113
6.28.4.10 INV_USR_LOG_ERR	113
6.28.4.11 lastStatusCode	113
6.28.4.12 LOGIN	114
6.28.4.13 MAL_REQ_ERR	114
6.28.4.14 MISS_RM_CNTXT	114
6.28.4.15 P	114

6.28.4.16 REG_USER	114
6.28.4.17 SERVER_ERR	114
6.28.4.18 SUCC	114
6.28.4.19 UNAUTH_RES_REQ_ERR	115
6.29 IBlendAlgorithm Class Reference	115
6.29.1 Detailed Description	115
6.29.2 Member Function Documentation	115
6.29.2.1 generateBlend()	115
6.30 ImageBackgroundPanel Class Reference	116
6.30.1 Detailed Description	117
6.30.2 Constructor & Destructor Documentation	117
6.30.2.1 ImageBackgroundPanel()	117
6.30.3 Member Function Documentation	117
6.30.3.1 OnPaint()	117
6.30.4 Member Data Documentation	117
6.30.4.1 m_imageStore	117
6.30.4.2 m_key	118
6.31 IRefreshablePanel Class Reference	118
6.31.1 Detailed Description	118
6.31.2 Member Function Documentation	119
6.31.2.1 Refresh()	119
6.32 IThemedElement Class Reference	119
6.32.1 Detailed Description	119
6.33 LoginPanel Class Reference	120
6.33.1 Detailed Description	121
6.33.2 Constructor & Destructor Documentation	121
6.33.2.1 LoginPanel()	121
6.33.3 Member Function Documentation	122
6.33.3.1 BuildRegisterForm()	122
6.33.3.2 BuildSignInForm()	122
6.33.3.3 LoadThemedBackground()	122
6.33.3.4 OnPaint()	122
6.33.3.5 OnRegister()	122
6.33.3.6 OnSignIn()	122
6.33.3.7 ProceedAfterLogin()	123
6.33.3.8 Refresh()	123
6.33.3.9 Reset()	123
6.33.3.10 ShowTab()	123
6.33.4 Member Data Documentation	123
6.33.4.1 m_bgImage	123
6.33.4.2 m_controller	123
6.33.4.3 m_formBook	124

6.33.4.4 m_nav	124
6.33.4.5 m_regConfirm	124
6.33.4.6 m_regEmail	124
6.33.4.7 m_regError	124
6.33.4.8 m_registerTab	124
6.33.4.9 m_regPassword	124
6.33.4.10 m_regUsername	124
6.33.4.11 m_siError	125
6.33.4.12 m_signinTab	125
6.33.4.13 m_siPassword	125
6.33.4.14 m_siUsername	125
6.34 MainFrame Class Reference	125
6.34.1 Detailed Description	127
6.34.2 Member Enumeration Documentation	127
6.34.2.1 IconBase	127
6.34.2.2 ImageKey	128
6.34.3 Constructor & Destructor Documentation	128
6.34.3.1 MainFrame()	128
6.34.4 Member Function Documentation	128
6.34.4.1 ApplyApplcons()	128
6.34.4.2 BuildHomePage()	128
6.34.4.3 LoadImage()	128
6.34.4.4 LoadThemedIcon()	128
6.34.4.5 MakeIconButton()	129
6.34.4.6 NavigateTo()	129
6.34.4.7 ReloadThemedIcons()	130
6.34.4.8 ShowSettingsDropdown()	130
6.34.4.9 ShowUserDropdown()	130
6.34.4.10 TriggerFeedRefresh()	130
6.34.4.11 UpdateBlendIndicatorLabel()	130
6.34.5 Member Data Documentation	130
6.34.5.1 activeBlendsPanel	130
6.34.5.2 blendIndicatorLabel	131
6.34.5.3 blendsIconBtn	131
6.34.5.4 book	131
6.34.5.5 chatIconBtn	131
6.34.5.6 chatPanel	131
6.34.5.7 controller	131
6.34.5.8 creationPanel	131
6.34.5.9 dataInstrPanel	131
6.34.5.10 feedPanel	132
6.34.5.11 images	132

6.34.5.12 loginPanel	132
6.34.5.13 refreshBtn	132
6.34.5.14 refreshInProgress	132
6.34.5.15 settingsIconBtn	132
6.34.5.16 settingsPanel	132
6.34.5.17 userIconBtn	132
6.34.5.18 userPanel	133
6.35 Message Struct Reference	133
6.35.1 Detailed Description	133
6.35.2 Constructor & Destructor Documentation	133
6.35.2.1 Message()	133
6.35.3 Member Function Documentation	134
6.35.3.1 toString()	134
6.35.4 Member Data Documentation	134
6.35.4.1 text	134
6.35.4.2 timestamp	134
6.35.4.3 userID	134
6.36 OperationProgressDialog Class Reference	135
6.36.1 Detailed Description	136
6.36.2 Constructor & Destructor Documentation	136
6.36.2.1 OperationProgressDialog()	136
6.36.2.2 ~OperationProgressDialog()	136
6.36.3 Member Function Documentation	136
6.36.3.1 ShowModal()	136
6.36.3.2 UpdateProgress()	136
6.36.3.3 UpdateProgressFill()	137
6.36.4 Member Data Documentation	137
6.36.4.1 currentProgress	137
6.36.4.2 messageLabel	137
6.36.4.3 percentLabel	137
6.36.4.4 progressFill	137
6.36.4.5 progressTrack	138
6.37 Palette Struct Reference	138
6.37.1 Detailed Description	138
6.37.2 Member Data Documentation	138
6.37.2.1 Accent	138
6.37.2.2 AccentHover	139
6.37.2.3 Background	139
6.37.2.4 Danger	139
6.37.2.5 Name	139
6.37.2.6 Separator	139
6.37.2.7 Surface	139

6.37.2.8 SurfaceRaised	139
6.37.2.9 TextMuted	139
6.37.2.10 TextPrimary	140
6.37.2.11 TextSecondary	140
6.38 Parser Class Reference	140
6.38.1 Detailed Description	140
6.38.2 Member Function Documentation	141
6.38.2.1 getParsed()	141
6.38.2.2 parse()	141
6.38.2.3 setData()	141
6.39 RandomBlendAlgorithm Class Reference	142
6.39.1 Detailed Description	142
6.39.2 Constructor & Destructor Documentation	143
6.39.2.1 RandomBlendAlgorithm()	143
6.39.3 Member Function Documentation	143
6.39.3.1 generateBlend()	143
6.39.4 Member Data Documentation	143
6.39.4.1 videosPerUser	143
6.40 SettingsPanel Class Reference	144
6.40.1 Detailed Description	145
6.40.2 Constructor & Destructor Documentation	145
6.40.2.1 SettingsPanel()	145
6.40.3 Member Function Documentation	145
6.40.3.1 OnLogout()	145
6.40.3.2 OnShow()	145
6.40.3.3 OnThemeChanged()	146
6.40.3.4 RefreshUserInfo()	146
6.40.4 Member Data Documentation	146
6.40.4.1 m_controller	146
6.40.4.2 m_nav	146
6.40.4.3 m_themeChoice	146
6.41 SqliteDataManager Class Reference	147
6.41.1 Detailed Description	148
6.41.2 Constructor & Destructor Documentation	148
6.41.2.1 SqliteDataManager()	148
6.41.2.2 ~SqliteDataManager()	148
6.41.3 Member Function Documentation	148
6.41.3.1 createUser()	148
6.41.3.2 findBlendForUser()	149
6.41.3.3 findUserById()	149
6.41.3.4 initSchema()	149
6.41.3.5 loadBlendParticipants()	149

6.41.3.6 loadWatchLater()	150
6.41.3.7 saveBlend()	150
6.41.3.8 saveWatchLater()	150
6.41.3.9 validatePassword()	151
6.41.4 Member Data Documentation	151
6.41.4.1 db	151
6.42 ThumbnailEvent Class Reference	151
6.42.1 Detailed Description	152
6.42.2 Constructor & Destructor Documentation	152
6.42.2.1 ThumbnailEvent()	152
6.42.3 Member Function Documentation	153
6.42.3.1 Clone()	153
6.42.4 Member Data Documentation	153
6.42.4.1 generation	153
6.42.4.2 image	153
6.42.4.3 success	153
6.43 TopBar Class Reference	154
6.43.1 Detailed Description	155
6.43.2 Constructor & Destructor Documentation	155
6.43.2.1 TopBar()	155
6.43.3 Member Function Documentation	155
6.43.3.1 OnBack()	155
6.43.3.2 OnThemeUpdate()	156
6.43.4 Member Data Documentation	156
6.43.4.1 m_backBtn	156
6.43.4.2 m_backDest	156
6.43.4.3 m_nav	156
6.44 UIColors Class Reference	156
6.44.1 Detailed Description	157
6.44.2 Member Function Documentation	157
6.44.2.1 Accent()	157
6.44.2.2 AccentHover()	158
6.44.2.3 Background()	158
6.44.2.4 Danger()	158
6.44.2.5 GetIcon()	158
6.44.2.6 GetRoleFromColour()	158
6.44.2.7 LoadThemesFromFile()	158
6.44.2.8 MakeIconButton()	159
6.44.2.9 Separator()	159
6.44.2.10 SetTheme()	159
6.44.2.11 Surface()	160
6.44.2.12 SurfaceRaised()	160

6.44.2.13 TextMuted()	160
6.44.2.14 TextPrimary()	160
6.44.2.15 TextSecondary()	160
6.44.2.16 UpdateUI()	160
6.44.3 Member Data Documentation	161
6.44.3.1 Current	161
6.44.3.2 ThemeMap	161
6.44.3.3 ThemeOrder	161
6.45 User Class Reference	161
6.45.1 Detailed Description	162
6.45.2 Constructor & Destructor Documentation	162
6.45.2.1 User()	162
6.45.3 Member Function Documentation	163
6.45.3.1 addFriend()	163
6.45.3.2 getEmail()	163
6.45.3.3 getFriends()	163
6.45.3.4 getPassword()	164
6.45.3.5 getUserID()	164
6.45.3.6 getUsername()	164
6.45.3.7 getYouTubeData() [1/2]	164
6.45.3.8 getYouTubeData() [2/2]	164
6.45.3.9 removeFriend()	164
6.45.3.10 setEmail()	165
6.45.3.11 setPassword()	165
6.45.3.12 setUsername()	165
6.45.3.13 setYouTubeData()	165
6.45.3.14 toString()	166
6.45.4 Member Data Documentation	166
6.45.4.1 email	166
6.45.4.2 friends	166
6.45.4.3 password	166
6.45.4.4 userID	166
6.45.4.5 username	167
6.45.4.6 youTubeData	167
6.46 UserPanel Class Reference	167
6.46.1 Detailed Description	169
6.46.2 Constructor & Destructor Documentation	169
6.46.2.1 UserPanel()	169
6.46.3 Member Function Documentation	169
6.46.3.1 OnConfirmDelete()	169
6.46.3.2 OnDeleteRequest()	169
6.46.3.3 OnLogout()	170

6.46.3.4 OnReuploadCSV()	170
6.46.3.5 OnShow()	170
6.46.3.6 Refresh()	170
6.46.3.7 RefreshUserInfo()	170
6.46.3.8 SetDeleteReauthVisible()	171
6.46.4 Member Data Documentation	171
6.46.4.1 m_confirmDeleteBtn	171
6.46.4.2 m_controller	171
6.46.4.3 m_deleteErrorLabel	171
6.46.4.4 m_deletePasswordField	171
6.46.4.5 m_emailLabel	171
6.46.4.6 m_nav	171
6.46.4.7 m_usernameLabel	172
6.47 Video Class Reference	172
6.47.1 Detailed Description	172
6.47.2 Constructor & Destructor Documentation	173
6.47.2.1 Video()	173
6.47.3 Member Function Documentation	173
6.47.3.1 getChannelID()	173
6.47.3.2 getChannelLogoURL()	173
6.47.3.3 getChannelName()	174
6.47.3.4 getDuration()	174
6.47.3.5 getTags()	174
6.47.3.6 getThumbnailURL()	174
6.47.3.7 getTitle()	174
6.47.3.8 getVideoID()	175
6.47.3.9 toString()	175
6.47.4 Member Data Documentation	175
6.47.4.1 channelID	175
6.47.4.2 channelLogoURL	175
6.47.4.3 channelName	175
6.47.4.4 duration	175
6.47.4.5 tags	176
6.47.4.6 thumbnailURL	176
6.47.4.7 title	176
6.47.4.8 videoID	176
6.48 VideoCard Class Reference	176
6.48.1 Detailed Description	178
6.48.2 Constructor & Destructor Documentation	178
6.48.2.1 VideoCard()	178
6.48.3 Member Function Documentation	179
6.48.3.1 Clear()	179

6.48.3.2 DoGetBestSize()	179
6.48.3.3 DrawWrappedText()	179
6.48.3.4 FetchChannelLogoAsync()	179
6.48.3.5 FetchThumbnailAsync()	180
6.48.3.6 OnChannelLogoLoaded()	180
6.48.3.7 OnClick()	180
6.48.3.8 OnMouseEnter()	180
6.48.3.9 OnMouseLeave()	180
6.48.3.10 OnPaint()	180
6.48.3.11 OnThumbnailLoaded()	180
6.48.3.12 SetVideo()	180
6.48.4 Member Data Documentation	181
6.48.4.1 m_channelID	181
6.48.4.2 m_channelLogo	181
6.48.4.3 m_channelLogoURL	181
6.48.4.4 m_channelName	181
6.48.4.5 m_controller	181
6.48.4.6 m_hovered	181
6.48.4.7 m_logoFailed	182
6.48.4.8 m_logoGen	182
6.48.4.9 m_logoLoading	182
6.48.4.10 m_thumbFailed	182
6.48.4.11 m_thumbGen	182
6.48.4.12 m_thumbLoading	182
6.48.4.13 m_thumbnail	182
6.48.4.14 m_thumbnailURL	183
6.48.4.15 m_title	183
6.48.4.16 m_videoID	183
6.49 VideoEntry Class Reference	183
6.49.1 Detailed Description	184
6.49.2 Constructor & Destructor Documentation	184
6.49.2.1 VideoEntry()	184
6.49.3 Member Function Documentation	184
6.49.3.1 getLastWatched()	184
6.49.3.2 getVideo()	185
6.49.3.3 getWatchCount()	185
6.49.3.4 setLastWatched()	185
6.49.3.5 setWatchCount()	185
6.49.3.6 toString()	186
6.49.4 Member Data Documentation	186
6.49.4.1 lastWatched	186
6.49.4.2 video	186

6.49.4.3 watchCount	186
6.50 YouTubeMetadataFetcher::VideoMeta Struct Reference	186
6.50.1 Detailed Description	187
6.50.2 Member Data Documentation	187
6.50.2.1 channelId	187
6.50.2.2 channelName	187
6.50.2.3 thumbnailUrl	187
6.50.2.4 title	187
6.51 WatchHistoryParser Class Reference	187
6.51.1 Detailed Description	188
6.51.2 Constructor & Destructor Documentation	188
6.51.2.1 WatchHistoryParser()	188
6.51.3 Member Function Documentation	188
6.51.3.1 extractVideoIdFromHref()	188
6.51.3.2 getParserId()	189
6.51.3.3 parse()	189
6.51.3.4 parseMultiThreaded()	189
6.51.3.5 parseRange()	189
6.51.4 Member Data Documentation	190
6.51.4.1 filePath	190
6.51.4.2 parser_id	190
6.52 WatchLaterParser Class Reference	190
6.52.1 Detailed Description	190
6.52.2 Constructor & Destructor Documentation	190
6.52.2.1 WatchLaterParser()	190
6.52.3 Member Function Documentation	191
6.52.3.1 getParserId()	191
6.52.3.2 parse()	191
6.52.3.3 parseMultiThreaded()	191
6.52.4 Member Data Documentation	192
6.52.4.1 filePath	192
6.52.4.2 parser_id	192
6.53 WeightedBlendAlgorithm Class Reference	192
6.53.1 Detailed Description	193
6.53.2 Constructor & Destructor Documentation	193
6.53.2.1 WeightedBlendAlgorithm() [1/2]	193
6.53.2.2 WeightedBlendAlgorithm() [2/2]	193
6.53.3 Member Function Documentation	194
6.53.3.1 generateBlend()	194
6.53.3.2 normalizeWeights()	194
6.53.3.3 selectWeightedVideos()	194
6.53.4 Member Data Documentation	195

6.53.4.1 participantWeights	195
6.53.4.2 videosPerUser	195
6.54 YouTubeData Class Reference	195
6.54.1 Detailed Description	196
6.54.2 Constructor & Destructor Documentation	196
6.54.2.1 YouTubeData()	196
6.54.3 Member Function Documentation	196
6.54.3.1 getLikedVideos()	196
6.54.3.2 getMostWatchedVideos()	196
6.54.3.3 getSubscribedChannels()	197
6.54.3.4 getTopChannels()	197
6.54.3.5 getWatchHistory()	197
6.54.3.6 getWatchLaterVideos()	197
6.54.3.7 setLikedVideos()	197
6.54.3.8 setSubscribedChannels()	198
6.54.3.9 setWatchHistory()	198
6.54.3.10 setWatchLaterVideos()	198
6.54.3.11 toString()	198
6.54.4 Member Data Documentation	199
6.54.4.1 likedVideos	199
6.54.4.2 subscribedChannels	199
6.54.4.3 watchHistory	199
6.54.4.4 watchLaterVideos	199
6.55 YouTubeDataImporter Class Reference	199
6.55.1 Detailed Description	200
6.55.2 Member Function Documentation	200
6.55.2.1 import()	200
6.55.2.2 importMultiThreaded()	200
6.55.2.3 lowerExtension()	200
6.56 YouTubeMetadataFetcher Class Reference	201
6.56.1 Detailed Description	202
6.56.2 Constructor & Destructor Documentation	202
6.56.2.1 YouTubeMetadataFetcher()	202
6.56.3 Member Function Documentation	202
6.56.3.1 enrich()	202
6.56.3.2 fetchChannelLogos()	202
6.56.3.3 fetchVideoBatch()	202
6.56.3.4 httpGet()	203
6.56.3.5 writeCallback()	203
6.56.4 Member Data Documentation	203
6.56.4.1 apiKey	203
6.56.4.2 kBatchSize	203

6.57 YTLNDApp Class Reference	204
6.57.1 Detailed Description	205
6.57.2 Member Function Documentation	205
6.57.2.1 OnExit()	205
6.57.2.2 OnInit()	205
6.57.3 Member Data Documentation	205
6.57.3.1 controller	205
7 File Documentation	207
7.1 IBlendAlgorithm.hpp File Reference	207
7.1.1 Detailed Description	208
7.2 IBlendAlgorithm.hpp	208
7.3 RandomBlendAlgorithm.cpp File Reference	209
7.3.1 Detailed Description	209
7.4 RandomBlendAlgorithm.cpp	210
7.5 RandomBlendAlgorithm.hpp File Reference	210
7.5.1 Detailed Description	211
7.6 RandomBlendAlgorithm.hpp	212
7.7 WeightedBlendAlgorithm.hpp File Reference	212
7.7.1 Detailed Description	213
7.8 WeightedBlendAlgorithm.hpp	213
7.9 CsvParser.cpp File Reference	213
7.9.1 Detailed Description	214
7.10 CsvParser.cpp	214
7.11 CsvParser.hpp File Reference	215
7.11.1 Detailed Description	216
7.12 CsvParser.hpp	216
7.13 CsvSource.cpp File Reference	217
7.13.1 Detailed Description	217
7.14 CsvSource.cpp	217
7.15 CsvSource.hpp File Reference	218
7.15.1 Detailed Description	219
7.16 CsvSource.hpp	219
7.17 DataExtractor.cpp File Reference	219
7.17.1 Detailed Description	220
7.18 DataExtractor.cpp	220
7.19 DataExtractor.hpp File Reference	221
7.19.1 Detailed Description	221
7.20 DataExtractor.hpp	222
7.21 DataSaver.hpp File Reference	222
7.21.1 Detailed Description	223
7.22 DataSaver.hpp	223

7.23 File_ID.hpp File Reference	224
7.23.1 Detailed Description	224
7.24 File_ID.hpp	224
7.25 FileBin.hpp File Reference	225
7.25.1 Detailed Description	225
7.26 FileBin.hpp	226
7.27 FileFormatter.hpp File Reference	226
7.27.1 Detailed Description	227
7.28 FileFormatter.hpp	227
7.29 FileSource.hpp File Reference	228
7.29.1 Detailed Description	229
7.30 FileSource.hpp	229
7.31 HtmlParser.cpp File Reference	229
7.31.1 Detailed Description	230
7.32 HtmlParser.cpp	230
7.33 HtmlSource.cpp File Reference	231
7.33.1 Detailed Description	232
7.34 HtmlSource.cpp	232
7.35 Parser.hpp File Reference	232
7.35.1 Detailed Description	233
7.36 Parser.hpp	234
7.37 WatchHistoryParser.cpp File Reference	234
7.37.1 Detailed Description	234
7.38 WatchHistoryParser.cpp	235
7.39 WatchHistoryParser.hpp File Reference	237
7.39.1 Detailed Description	238
7.40 WatchHistoryParser.hpp	239
7.41 WatchLaterParser.cpp File Reference	239
7.41.1 Detailed Description	239
7.42 WatchLaterParser.cpp	240
7.43 WatchLaterParser.hpp File Reference	242
7.43.1 Detailed Description	243
7.44 WatchLaterParser.hpp	243
7.45 YouTubeDataImporter.cpp File Reference	243
7.45.1 Detailed Description	244
7.46 YouTubeDataImporter.cpp	244
7.47 YouTubeDataImporter.hpp File Reference	245
7.47.1 Detailed Description	246
7.48 YouTubeDataImporter.hpp	246
7.49 AppController.cpp File Reference	247
7.49.1 Detailed Description	247
7.49.2 Typedef Documentation	248

7.49.2.1 Json	248
7.49.3 Variable Documentation	248
7.49.3.1 kYouTubeAPIKey	248
7.50 ApplicationController.cpp	248
7.51 ApplicationController.hpp File Reference	267
7.51.1 Detailed Description	268
7.52 ApplicationController.hpp	269
7.53 AppState.cpp File Reference	270
7.53.1 Detailed Description	271
7.54 AppState.cpp	271
7.55 AppState.hpp File Reference	273
7.55.1 Detailed Description	274
7.56 AppState.hpp	274
7.57 EventRouter.cpp File Reference	275
7.57.1 Detailed Description	275
7.58 EventRouter.cpp	276
7.59 EventRouter.hpp File Reference	276
7.59.1 Detailed Description	277
7.59.2 Typedef Documentation	277
7.59.2.1 EventHandler	277
7.59.2.2 EventPayload	278
7.60 EventRouter.hpp	278
7.61 main.cpp File Reference	278
7.61.1 Detailed Description	279
7.62 main.cpp	279
7.63 Blend.cpp File Reference	280
7.63.1 Detailed Description	280
7.64 Blend.cpp	280
7.65 Blend.hpp File Reference	282
7.65.1 Detailed Description	282
7.66 Blend.hpp	283
7.67 Channel.cpp File Reference	283
7.67.1 Detailed Description	284
7.68 Channel.cpp	284
7.69 Channel.hpp File Reference	285
7.69.1 Detailed Description	286
7.70 Channel.hpp	286
7.71 ChatRoom.cpp File Reference	286
7.71.1 Detailed Description	287
7.72 ChatRoom.cpp	287
7.73 ChatRoom.hpp File Reference	288
7.73.1 Detailed Description	289

7.74 ChatRoom.hpp	289
7.75 Friend.cpp File Reference	290
7.75.1 Detailed Description	290
7.76 Friend.cpp	290
7.77 Friend.hpp File Reference	291
7.77.1 Detailed Description	292
7.78 Friend.hpp	292
7.79 JsonUtils.hpp File Reference	292
7.79.1 Detailed Description	293
7.80 JsonUtils.hpp	294
7.81 Message.cpp File Reference	294
7.81.1 Detailed Description	295
7.82 Message.cpp	295
7.83 Message.hpp File Reference	295
7.83.1 Detailed Description	296
7.84 Message.hpp	297
7.85 User.cpp File Reference	297
7.85.1 Detailed Description	297
7.86 User.cpp	298
7.87 User.hpp File Reference	298
7.87.1 Detailed Description	299
7.88 User.hpp	300
7.89 Video.cpp File Reference	300
7.89.1 Detailed Description	301
7.90 Video.cpp	301
7.91 Video.hpp File Reference	302
7.91.1 Detailed Description	303
7.92 Video.hpp	303
7.93 VideoEntry.cpp File Reference	304
7.93.1 Detailed Description	304
7.94 VideoEntry.cpp	305
7.95 VideoEntry.hpp File Reference	305
7.95.1 Detailed Description	306
7.96 VideoEntry.hpp	306
7.97 YouTubeData.cpp File Reference	307
7.97.1 Detailed Description	307
7.98 YouTubeData.cpp	307
7.99 YouTubeData.hpp File Reference	309
7.99.1 Detailed Description	310
7.100 YouTubeData.hpp	311
7.101 ServerConfig.hpp File Reference	311
7.101.1 Detailed Description	312

7.101.2 Variable Documentation	312
7.101.2.1 kAppBackendBaseUrl	312
7.101.2.2 kAppBackendWsPrefix	312
7.101.2.3 kTestBackendBaseUrl	313
7.101.2.4 kTestBackendWsPrefix	313
7.102 ServerConfig.hpp	313
7.103 ChatWebSocket.cpp File Reference	313
7.103.1 Detailed Description	314
7.103.2 Function Documentation	314
7.103.2.1 urlEncode()	314
7.104 ChatWebSocket.cpp	314
7.105 ChatWebSocket.hpp File Reference	315
7.105.1 Detailed Description	316
7.106 ChatWebSocket.hpp	317
7.107 DataManager.hpp File Reference	317
7.107.1 Detailed Description	318
7.108 DataManager.hpp	319
7.109 HttpClient.cpp File Reference	319
7.109.1 Detailed Description	320
7.110 HttpClient.cpp	320
7.111 HttpClient.hpp File Reference	322
7.111.1 Detailed Description	323
7.112 HttpClient.hpp	323
7.113 JsonRequestBuilder.cpp File Reference	324
7.113.1 Detailed Description	325
7.113.2 Function Documentation	325
7.113.2.1 buildBlendJson()	325
7.113.2.2 buildDeleteAccountJson()	326
7.113.2.3 buildLoginJson()	326
7.113.2.4 buildRegisterJson()	327
7.113.2.5 buildWatchLaterJson()	327
7.114 JsonRequestBuilder.cpp	328
7.115 JsonRequestBuilder.hpp File Reference	329
7.115.1 Detailed Description	330
7.115.2 Function Documentation	330
7.115.2.1 buildBlendJson()	330
7.115.2.2 buildDeleteAccountJson()	331
7.115.2.3 buildLoginJson()	331
7.115.2.4 buildRegisterJson()	332
7.115.2.5 buildWatchLaterJson()	332
7.116 JsonRequestBuilder.hpp	333
7.117 SqliteDataManager.cpp File Reference	333

7.117.1 Detailed Description	334
7.118 SqliteDataManager.cpp	334
7.119 SqliteDataManager.hpp File Reference	338
7.119.1 Detailed Description	339
7.120 SqliteDataManager.hpp	340
7.121 YouTubeMetadataFetcher.cpp File Reference	340
7.121.1 Detailed Description	341
7.121.2 Typedef Documentation	341
7.121.2.1 json	341
7.122 YouTubeMetadataFetcher.cpp	341
7.123 YouTubeMetadataFetcher.hpp File Reference	344
7.123.1 Detailed Description	345
7.124 YouTubeMetadataFetcher.hpp	345
7.125 ActiveBlendsPanel.cpp File Reference	346
7.125.1 Detailed Description	347
7.126 ActiveBlendsPanel.cpp	347
7.127 ActiveBlendsPanel.hpp File Reference	349
7.127.1 Detailed Description	350
7.128 ActiveBlendsPanel.hpp	351
7.129 BlendChatPanel.cpp File Reference	351
7.129.1 Detailed Description	352
7.129.2 Function Documentation	352
7.129.2.1 GetLocalTime()	352
7.129.2.2 IsSameLocalDay()	352
7.130 BlendChatPanel.cpp	352
7.131 BlendChatPanel.hpp File Reference	358
7.131.1 Detailed Description	359
7.131.1.1 LAYOUT	359
7.131.1.2 MESSAGE LIST	359
7.131.1.3 SENDING	359
7.132 BlendChatPanel.hpp	359
7.133 BlendCreationPanel.cpp File Reference	360
7.133.1 Detailed Description	361
7.134 BlendCreationPanel.cpp	361
7.135 BlendCreationPanel.hpp File Reference	366
7.135.1 Detailed Description	367
7.135.1.1 LAYOUT	367
7.135.1.2 USER-ADD FLOW	368
7.135.1.3 BLEND CREATION	368
7.135.1.4 RELOAD	368
7.136 BlendCreationPanel.hpp	368
7.137 BlendFeedPanel.cpp File Reference	369

7.137.1 Detailed Description	370
7.138 BlendFeedPanel.cpp	370
7.139 BlendFeedPanel.hpp File Reference	371
7.139.1 Detailed Description	372
7.139.1.1 PAGING	372
7.139.1.2 LOADING	373
7.140 BlendFeedPanel.hpp	373
7.141 ButtonsConfig.cpp File Reference	373
7.141.1 Detailed Description	374
7.141.2 Function Documentation	374
7.141.2.1 ApplySizeBounds()	374
7.141.2.2 GetConfig()	374
7.141.2.3 GetMaxSize()	375
7.141.2.4 GetMinSize()	375
7.141.2.5 GetSize()	375
7.142 ButtonsConfig.cpp	376
7.143 ButtonsConfig.hpp File Reference	377
7.143.1 Detailed Description	378
7.143.2 Enumeration Type Documentation	378
7.143.2.1 ButtonType	378
7.143.3 Function Documentation	379
7.143.3.1 ApplySizeBounds()	379
7.143.3.2 GetConfig()	379
7.143.3.3 GetMaxSize()	379
7.143.3.4 GetMinSize()	379
7.143.3.5 GetSize()	380
7.144 ButtonsConfig.hpp	380
7.145 ConfirmationDialog.cpp File Reference	381
7.145.1 Detailed Description	381
7.146 ConfirmationDialog.cpp	381
7.147 ConfirmationDialog.hpp File Reference	383
7.147.1 Detailed Description	383
7.148 ConfirmationDialog.hpp	384
7.149 DataInstructionsPanel.cpp File Reference	384
7.149.1 Detailed Description	385
7.150 DataInstructionsPanel.cpp	385
7.151 DataInstructionsPanel.hpp File Reference	387
7.151.1 Detailed Description	388
7.152 DataInstructionsPanel.hpp	388
7.153 IRefreshablePanel.hpp File Reference	389
7.153.1 Detailed Description	389
7.154 IRefreshablePanel.hpp	389

7.155 LoginPanel.cpp File Reference	390
7.155.1 Detailed Description	390
7.155.2 Function Documentation	390
7.155.2.1 MakeError()	390
7.155.2.2 MakeField()	391
7.156 LoginPanel.cpp	391
7.157 LoginPanel.hpp File Reference	395
7.157.1 Detailed Description	396
7.157.1.1 SIGN IN	397
7.157.1.2 REGISTER	397
7.157.1.3 POST-LOGIN ROUTING	397
7.157.1.4 RESET	397
7.158 LoginPanel.hpp	397
7.159 MainFrame.cpp File Reference	398
7.159.1 Detailed Description	399
7.160 MainFrame.cpp	399
7.161 MainFrame.hpp File Reference	408
7.161.1 Detailed Description	408
7.162 MainFrame.hpp	409
7.163 OperationProgressDialog.cpp File Reference	410
7.163.1 Detailed Description	411
7.164 OperationProgressDialog.cpp	411
7.165 OperationProgressDialog.hpp File Reference	412
7.165.1 Detailed Description	413
7.166 OperationProgressDialog.hpp	414
7.167 ResourcePathUtils.cpp File Reference	414
7.167.1 Detailed Description	415
7.167.2 Function Documentation	415
7.167.2.1 FindResourcePath()	415
7.167.2.2 GetIconFolder()	415
7.168 ResourcePathUtils.cpp	416
7.169 ResourcePathUtils.hpp File Reference	416
7.169.1 Detailed Description	417
7.169.2 Function Documentation	417
7.169.2.1 FindResourcePath()	417
7.169.2.2 GetIconFolder()	418
7.170 ResourcePathUtils.hpp	418
7.171 SettingsPanel.cpp File Reference	418
7.171.1 Detailed Description	419
7.172 SettingsPanel.cpp	419
7.173 SettingsPanel.hpp File Reference	421
7.173.1 Detailed Description	422

7.174 SettingsPanel.hpp	422
7.175 TopBar.cpp File Reference	422
7.175.1 Detailed Description	423
7.176 TopBar.cpp	423
7.177 TopBar.hpp File Reference	424
7.177.1 Detailed Description	425
7.178 TopBar.hpp	425
7.179 UIColors.cpp File Reference	425
7.179.1 Detailed Description	426
7.180 UIColors.cpp	426
7.181 UIColors.hpp File Reference	429
7.181.1 Detailed Description	430
7.181.2 Enumeration Type Documentation	430
7.181.2.1 ColorRole	430
7.182 UIColors.hpp	430
7.183 UIPages.hpp File Reference	431
7.183.1 Detailed Description	432
7.183.2 Typedef Documentation	433
7.183.2.1 NavigateFn	433
7.183.3 Enumeration Type Documentation	433
7.183.3.1 Page	433
7.184 UIPages.hpp	434
7.185 UserPanel.cpp File Reference	434
7.185.1 Detailed Description	434
7.186 UserPanel.cpp	435
7.187 UserPanel.hpp File Reference	438
7.187.1 Detailed Description	439
7.187.1.1 LAYOUT	439
7.187.1.2 REFRESH ON SHOW	439
7.187.1.3 LOGOUT	439
7.187.1.4 FUTURE FEATURES	439
7.188 UserPanel.hpp	440
7.189 VideoCard.cpp File Reference	440
7.189.1 Detailed Description	441
7.189.2 Function Documentation	441
7.189.2.1 LineHeight()	441
7.190 VideoCard.cpp	442
7.191 VideoCard.hpp File Reference	446
7.191.1 Detailed Description	447
7.192 VideoCard.hpp	448

Chapter 1

Namespace Index

1.1 Namespace List

Here is a list of all documented namespaces with brief descriptions:

ModelJson	
JSON formatting utilities for model serialization	13

Chapter 2

Hierarchical Index

2.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

AppController	19
AppState	33
Blend	40
AppController::BlendSummary	59
BtnConfig	60
Channel	61
ChatRoom	63
ChatWebSocket	67
DataExtractor	78
DataManager	83
SqliteDataManager	147
DataSaver	86
EventRouter	88
File_ID	90
FileBin	91
FileFormatter	92
FileSource	94
CsvSource	76
HtmlSource	102
Friend	95
HttpClient	104
IBlendAlgorithm	115
RandomBlendAlgorithm	142
WeightedBlendAlgorithm	192
IRefreshablePanel	118
ActiveBlendsPanel	15
BlendChatPanel	45
BlendCreationPanel	52
BlendFeedPanel	56
LoginPanel	120
UserPanel	167
IThemedElement	119
TopBar	154
Message	133

Palette	138
Parser	140
CsvParser	73
HtmlParser	99
UIColors	156
User	161
Video	172
VideoEntry	183
YouTubeMetadataFetcher::VideoMeta	186
WatchHistoryParser	187
WatchLaterParser	190
wxApp	
YTBLNDApp	204
wxDialog	
ConfirmationDialog	71
OperationProgressDialog	135
wxEvent	
ThumbnailEvent	151
wxFrame	
MainFrame	125
wxPanel	
ActiveBlendsPanel	15
BlendChatPanel	45
BlendCreationPanel	52
BlendFeedPanel	56
DataInstructionsPanel	81
ImageBackgroundPanel	116
LoginPanel	120
SettingsPanel	144
TopBar	154
UserPanel	167
VideoCard	176
YouTubeData	195
YouTubeDataImporter	199
YouTubeMetadataFetcher	201

Chapter 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

ActiveBlendsPanel	
ActiveBlendsPanel class declaration	15
AppController	
Stub for future use	19
AppState	
AppState class declaration	33
Blend	
Blend class declaration	40
BlendChatPanel	
BlendChatPanel class declaration	45
BlendCreationPanel	
BlendCreationPanel class declaration	52
BlendFeedPanel	
Panel that renders paged blend videos in a fixed 3x2 card layout	56
AppController::BlendSummary	
BlendSummary class declaration	59
BtnConfig	
Width/height dimensions for a button preset	60
Channel	
Channel class declaration	61
ChatRoom	
ChatRoom class declaration	63
ChatWebSocket	
ChatWebSocket class declaration	67
ConfirmationDialog	
ConfirmationDialog class declaration	71
CsvParser	
CsvParser class declaration	73
CsvSource	
CsvSource class declaration	76
DataExtractor	
DataExtractor class declaration	78
DataInstructionsPanel	
DataInstructionsPanel class declaration	81
DataManager	
DataManager class declaration	83

DataSaver	
DataSaver class declaration	86
EventRouter	
EventRouter class declaration	88
File_ID	
File_ID class declaration	90
FileBin	
FileBin class declaration	91
FileFormatter	
FileFormatter class declaration	92
FileSource	
FileSource class declaration	94
Friend	
Friend class declaration	95
HtmlParser	
HTML file parser implementation using Gumbo	99
HtmlSource	
HTML file source/reader implementation	102
HttpClient	
HttpClient class declaration	104
IBlendAlgorithm	
IBlendAlgorithm class declaration	115
ImageBackgroundPanel	116
IRefreshablePanel	
Interface for panels that support refresh operations	118
IThemedElement	
Interface for any UI component that needs custom refresh logic (like icons)	119
LoginPanel	
LoginPanel class declaration	120
MainFrame	
Root application window containing the full-window page switcher	125
Message	
A single chat message sent within a ChatRoom	133
OperationProgressDialog	
Non-modal progress dialog showing upload/processing progress	135
Palette	
Palette class declaration	138
Parser	
Parser class declaration	140
RandomBlendAlgorithm	
RandomBlendAlgorithm class declaration	142
SettingsPanel	
SettingsPanel class declaration	144
SqliteDataManager	
SqliteDataManager type declaration	147
ThumbnailEvent	
Event fired on the VideoCard when an async image fetch completes	151
TopBar	
Reusable horizontal top bar for interior pages	154
UIColors	156
User	
User class declaration	161
UserPanel	
Panel for user account display and account actions	167
Video	
Video class declaration	172
VideoCard	
Custom wxPanel widget that displays a single YouTube video preview	176

VideoEntry	
VideoEntry class declaration	183
YouTubeMetadataFetcher::VideoMeta	
VideoMeta class declaration	186
WatchHistoryParser	
WatchHistoryParser class declaration	187
WatchLaterParser	
WatchLaterParser class declaration	190
WeightedBlendAlgorithm	
WeightedBlendAlgorithm class declaration	192
YouTubeData	
YouTubeData class declaration	195
YouTubeDataImporter	
YouTubeDataImporter class declaration	199
YouTubeMetadataFetcher	
YouTubeMetadataFetcher class declaration	201
YTBLNDApp	204

Chapter 4

File Index

4.1 File List

Here is a list of all documented files with brief descriptions:

IBlendAlgorithm.hpp	Abstract interface for blend-generation algorithms	207
RandomBlendAlgorithm.cpp	Implementation for RandomBlendAlgorithm	209
RandomBlendAlgorithm.hpp	Random Blend implementation of IBlendAlgorithm	210
WeightedBlendAlgorithm.hpp	Weighted Blend implementation of IBlendAlgorithm	212
CsvParser.cpp	Implementation for CsvParser	213
CsvParser.hpp	CSV file parser implementing the Parser interface	215
CsvSource.cpp	Implementation for CsvSource	217
CsvSource.hpp	CSV file reader implementing the FileSource interface	218
DataExtractor.cpp	DataExtractor class implementation	219
DataExtractor.hpp	Extracts raw file data and parses it into structured format	221
DataSaver.hpp	Coordinates FileFormatter and FileBin to persist structured data to files	222
File_ID.hpp	File format ID constants	224
FileBin.hpp	Abstract base class for writing data to files	225
FileFormatter.hpp	Abstract base class for formatting structured data to file format	226
FileSource.hpp	Abstract base class for reading file source data	228
HtmlParser.cpp	HTML file parser implementation using Gumbo	229
HtmlSource.cpp	HTML file source/reader implementation	231
Parser.hpp	Abstract base class for parsing file data into structured format	232

WatchHistoryParser.cpp	Implementation for WatchHistoryParser	234
WatchHistoryParser.hpp	Specialized parser for YouTube watch-history HTML exports	237
WatchLaterParser.cpp	Implementation for WatchLaterParser	239
WatchLaterParser.hpp	Specialized parser for YouTube Takeout Watch Later CSV files	242
YouTubeDataImporter.cpp	Implementation for YouTubeDataImporter	243
YouTubeDataImporter.hpp	Entry-point importer that routes YouTube exports to the right parser	245
AppController.cpp	Implementation for AppController	247
AppController.hpp	Central application coordinator	267
AppState.cpp	Implementation for AppState	270
AppState.hpp	Session-scoped singleton holding all runtime application state	273
EventRouter.cpp	Implementation for EventRouter	275
EventRouter.hpp	Lightweight event-dispatch system connecting the UI layer to AppController	276
main.cpp	Implementation for main	278
Blend.cpp	Implementation for Blend	280
Blend.hpp	Model representing a generated video feed shared by a group of users	282
Channel.cpp	Implementation for Channel	283
Channel.hpp	Model representing a YouTube channel	285
ChatRoom.cpp	Implementation for ChatRoom	286
ChatRoom.hpp	In-memory chat session tied to a specific Blend	288
Friend.cpp	Implementation for Friend	290
Friend.hpp	Model representing a friend/contact entry on a User 's list	291
JsonUtils.hpp	Utility functions for JSON string formatting and serialization	292
Message.cpp	Implementation for Message	294
Message.hpp	A single chat message sent within a ChatRoom	295
User.cpp	Implementation for User	297
User.hpp	Model representing a registered application user	298
Video.cpp	Implementation for Video	300
Video.hpp	Immutable data model for a single YouTube video	302
VideoEntry.cpp	Implementation for VideoEntry	304

VideoEntry.hpp	
A Video decorated with watch-history metadata	305
YouTubeData.cpp	
Implementation for YouTubeData	307
YouTubeData.hpp	
Container for all YouTube data associated with a User	309
ServerConfig.hpp	
Centralized backend server addresses	311
ChatWebSocket.cpp	
Implementation for ChatWebSocket	313
ChatWebSocket.hpp	
Simple WebSocket wrapper	315
DataManager.hpp	
Interface for all user-persistence operations	317
HttpClient.cpp	
Implementation for HttpClient	319
HttpClient.hpp	
Simple HTTP client wrapper	322
RequestJsonBuilder.cpp	
Implementation for RequestJsonBuilder	324
RequestJsonBuilder.hpp	
JSON request body builders used by ServiceLayer HTTP calls	329
SqliteDataManager.cpp	
Implementation for SqliteDataManager	333
SqliteDataManager.hpp	
SQLite-backed implementation of DataManager	338
YouTubeMetadataFetcher.cpp	
Implementation for YouTubeMetadataFetcher	340
YouTubeMetadataFetcher.hpp	
Enriches a list of Video objects (video-ID only) with full snippet metadata from the YouTube Data API v3	344
ActiveBlendsPanel.cpp	
Implementation for ActiveBlendsPanel	346
ActiveBlendsPanel.hpp	
Panel for viewing and managing active blends	349
BlendChatPanel.cpp	
Implementation for BlendChatPanel	351
BlendChatPanel.hpp	
Blend chat screen	358
BlendCreationPanel.cpp	
Implementation for BlendCreationPanel	360
BlendCreationPanel.hpp	
New-blend setup screen	366
BlendFeedPanel.cpp	
Implementation for BlendFeedPanel	369
BlendFeedPanel.hpp	
Home-page video feed panel that displays paged blend results	371
ButtonsConfig.cpp	
Implements shared button sizing presets used across UILayer	373
ButtonsConfig.hpp	
Centralized button size presets for UI layer widgets	377
ConfirmationDialog.cpp	
Implementation for ConfirmationDialog	381
ConfirmationDialog.hpp	
Dark-themed modal confirmation dialog	383
DataInstructionsPanel.cpp	
Implementation for DataInstructionsPanel	384

DataInstructionsPanel.hpp	
Setup instructions panel for importing Google Takeout CSV data	387
IRefreshablePanel.hpp	
Interface for panels that support refresh operations	389
LoginPanel.cpp	
Implementation for LoginPanel	390
LoginPanel.hpp	
Sign In / Register screen	395
MainFrame.cpp	
Implementation for MainFrame	398
MainFrame.hpp	
Root application window containing the full-window page switcher	408
OperationProgressDialog.cpp	
Implementation of non-modal progress dialog	410
OperationProgressDialog.hpp	
Dark-themed non-modal progress dialog for file upload/processing	412
ResourcePathUtils.cpp	
Shared resource path resolution helpers for UI components	414
ResourcePathUtils.hpp	
Shared resource path resolution helpers for UI components	416
SettingsPanel.cpp	
Implementation for SettingsPanel	418
SettingsPanel.hpp	
Settings Panel controls the theme choice for the user's GUI	421
TopBar.cpp	
Implementation for TopBar	422
TopBar.hpp	
Reusable interior-page header bar	424
UIColors.cpp	
Logic for dynamic theme switching and UI recursion	425
UIColors.hpp	
Colour palletes to use throughout the GUI	429
UIPages.hpp	
Page enum and NavigateFn callback type for the MainFrame page switcher	431
UserPanel.cpp	
Implementation for UserPanel	434
UserPanel.hpp	
Account info and logout panel	438
VideoCard.cpp	
Implementation for VideoCard	440
VideoCard.hpp	
Custom wxPanel widget that displays a single YouTube video preview	446

Chapter 5

Namespace Documentation

5.1 ModelJson Namespace Reference

JSON formatting utilities for model serialization.

Functions

- `std::string escape` (`const std::string &input`)
- `std::string quote` (`const std::string &input`)
- `std::string toIso8601` (`std::time_t timestamp`)

5.1.1 Detailed Description

JSON formatting utilities for model serialization.

5.1.2 Function Documentation

5.1.2.1 escape()

```
std::string ModelJson::escape (  
    const std::string & input) [inline]
```

Escapes special characters in a string for use in JSON values.

Converts escape sequences such as quotes, backslashes, newlines, tabs, and control characters into their JSON-safe representations. Characters below 0x20 are converted to Unicode escape sequences.

Parameters

<i>input</i>	The input string to escape.
--------------	-----------------------------

Returns

A new string with all special characters properly escaped for JSON.

Definition at line 35 of file [JsonUtils.hpp](#).

5.1.2.2 quote()

```
std::string ModelJson::quote (
    const std::string & input) [inline]
```

Wraps a string in double quotes after escaping special characters.

Combines [escape\(\)](#) and quoting to produce a JSON-safe string value. The returned string includes the surrounding double quotes.

Parameters

<i>input</i>	The input string to quote and escape.
--------------	---------------------------------------

Returns

A JSON-safe quoted string (e.g., "hello world").

Definition at line 69 of file [JsonUtils.hpp](#).

5.1.2.3 toIso8601()

```
std::string ModelJson::toIso8601 (
    std::time_t timestamp) [inline]
```

Converts a UNIX timestamp to an ISO 8601 formatted string.

Formats the given timestamp (seconds since epoch) as an ISO 8601 string in UTC timezone. The format is: YYYY-MM-DDTHH:MM:SSZ

Parameters

<i>timestamp</i>	UNIX timestamp (seconds since epoch).
------------------	---------------------------------------

Returns

ISO 8601

formatted timestamp string in UTC (e.g., "2026-04-03T12:30:45Z").

Definition at line 82 of file [JsonUtils.hpp](#).

Chapter 6

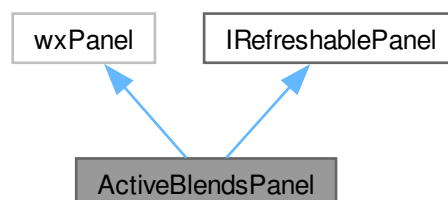
Class Documentation

6.1 ActiveBlendsPanel Class Reference

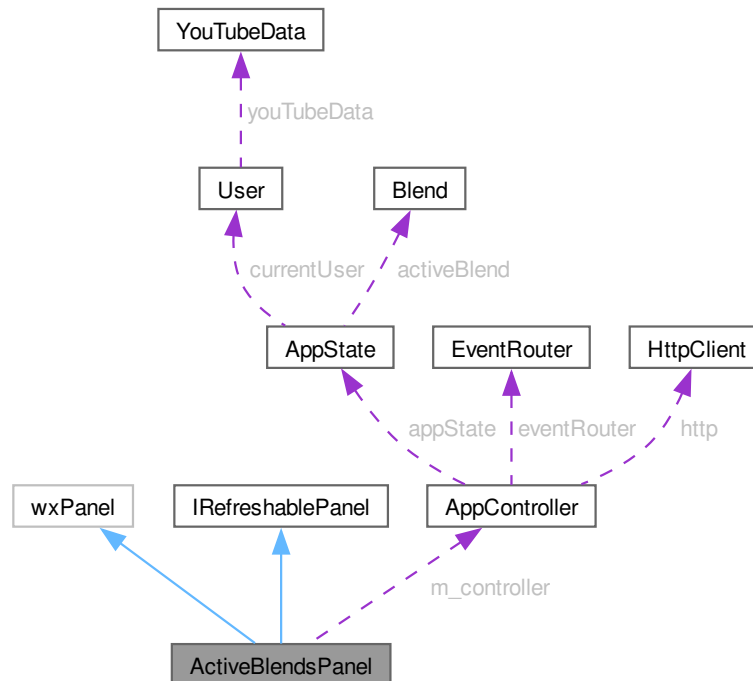
[ActiveBlendsPanel](#) class declaration.

```
#include <ActiveBlendsPanel.hpp>
```

Inheritance diagram for ActiveBlendsPanel:



Collaboration diagram for `ActiveBlendsPanel`:



Public Member Functions

- `ActiveBlendsPanel` (`wxWindow *parent`, `AppController &controller`, `NavigateFn nav`)
- void `Refresh` () override

Private Member Functions

- void `RebuildList` ()
- void `OnSelectBlend` (`const std::string &blendID`, `const std::string &title`)
- void `OnLeaveBlend` (`const std::string &blendID`, `const std::string &title`)

Private Attributes

- `AppController & m_controller`
- `NavigateFn m_nav`
- `wxScrolledWindow * m_listScroll`
- `wxPanel * m_listInner`
- `wxBoxSizer * m_listSizer`
- `wxStaticText * m_emptyLabel`

6.1.1 Detailed Description

[ActiveBlendsPanel](#) class declaration.

Definition at line 21 of file [ActiveBlendsPanel.hpp](#).

6.1.2 Constructor & Destructor Documentation

6.1.2.1 ActiveBlendsPanel()

```
ActiveBlendsPanel::ActiveBlendsPanel (  
    wxWindow * parent,  
    AppController & controller,  
    NavigateFn nav)
```

Definition at line 18 of file [ActiveBlendsPanel.cpp](#).

6.1.3 Member Function Documentation

6.1.3.1 OnLeaveBlend()

```
void ActiveBlendsPanel::OnLeaveBlend (  
    const std::string & blendID,  
    const std::string & title) [private]
```

Definition at line 179 of file [ActiveBlendsPanel.cpp](#).

6.1.3.2 OnSelectBlend()

```
void ActiveBlendsPanel::OnSelectBlend (  
    const std::string & blendID,  
    const std::string & title) [private]
```

Definition at line 158 of file [ActiveBlendsPanel.cpp](#).

6.1.3.3 RebuildList()

```
void ActiveBlendsPanel::RebuildList () [private]
```

Definition at line 76 of file [ActiveBlendsPanel.cpp](#).

6.1.3.4 Refresh()

```
void ActiveBlendsPanel::Refresh () [override], [virtual]
```

Refreshes the panel's display to reflect current application state. Called by [MainFrame](#) whenever the panel becomes active.

Implements [IRefreshablePanel](#).

Definition at line 71 of file [ActiveBlendsPanel.cpp](#).

6.1.4 Member Data Documentation

6.1.4.1 m_controller

[AppController](#)& ActiveBlendsPanel::m_controller [private]

Definition at line 27 of file [ActiveBlendsPanel.hpp](#).

6.1.4.2 m_emptyLabel

wxStaticText* ActiveBlendsPanel::m_emptyLabel [private]

Definition at line 33 of file [ActiveBlendsPanel.hpp](#).

6.1.4.3 m_listInner

wxPanel* ActiveBlendsPanel::m_listInner [private]

Definition at line 31 of file [ActiveBlendsPanel.hpp](#).

6.1.4.4 m_listScroll

wxScrolledWindow* ActiveBlendsPanel::m_listScroll [private]

Definition at line 30 of file [ActiveBlendsPanel.hpp](#).

6.1.4.5 m_listSizer

wxBoxSizer* ActiveBlendsPanel::m_listSizer [private]

Definition at line 32 of file [ActiveBlendsPanel.hpp](#).

6.1.4.6 m_nav

[NavigateFn](#) ActiveBlendsPanel::m_nav [private]

Definition at line 28 of file [ActiveBlendsPanel.hpp](#).

The documentation for this class was generated from the following files:

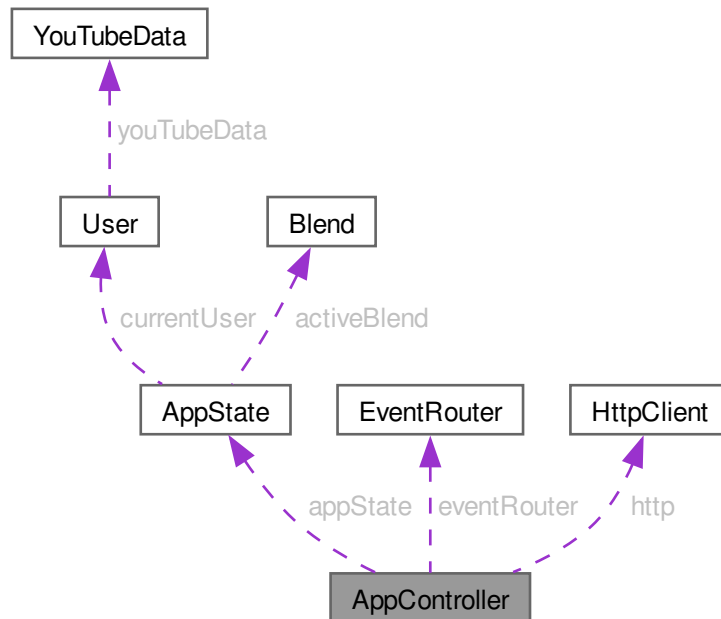
- [ActiveBlendsPanel.hpp](#)
- [ActiveBlendsPanel.cpp](#)

6.2 AppController Class Reference

Stub for future use.

```
#include <AppController.hpp>
```

Collaboration diagram for AppController:



Classes

- struct [BlendSummary](#)
[BlendSummary](#) class declaration.

Public Member Functions

- [AppController](#) ()
- [AppController](#) (const std::string &dbPath)
- [~AppController](#) ()
- [EventRouter](#) & [getEventRouter](#) ()
- void [setProgressReporter](#) (std::function< void(double, const std::string &)> reporter)
- void [clearProgressReporter](#) ()
- void [handleRegister](#) (const [EventPayload](#) &payload)
- void [handleLogin](#) (const [EventPayload](#) &payload)
- void [handleLogout](#) (const [EventPayload](#) &payload)
- void [handleDeleteAccount](#) (const [EventPayload](#) &payload)
- void [handleUploadData](#) (const [EventPayload](#) &payload)

- void `handleCreateBlend` (const `EventPayload` &payload)
- void `handlePlayVideo` (const `EventPayload` &payload)
- void `handleRefresh` (const `EventPayload` &payload)
- void `handleOpenChat` (const `EventPayload` &payload)
- bool `lookupUser` (const std::string &userID)
- void `handleSendMessage` (const `EventPayload` &payload)
- const `User` * `get_current_user` ()
- std::string `get_current_username` ()
- std::string `get_current_email` ()
- std::string `get_current_blend_id` ()
- std::string `get_current_blend_title` ()
- std::vector< `BlendSummary` > `fetchUserBlends` ()
- void `handleLeaveBlend` (const `EventPayload` &payload)
- void `handleSelectBlend` (const `EventPayload` &payload)

Static Public Member Functions

- static std::list< `Message` > `parseChatHistory` (const std::string &response)
- static std::list< `Video` > `enrichMissingMetadataSubsetMultithreaded` (const std::list< `Video` > &videos, std::size_t maxVideosToEnrich, std::size_t workerThreads, const std::function< std::list< `Video` >(const std::list< `Video` > &)> &enrichChunkFn)

Private Member Functions

- void `registerEvents` ()
- void `enrichIfMissingMetadata` (const std::string &userID, std::list< `Video` > &videos)
- void `enrichActiveBlendFeedIfMissingMetadata` ()
- std::list< `User` > `loadParticipantsWithWatchLater` (const std::vector< std::string > &participantIDs, std::vector< std::string > *missingData, bool enrichParticipantLists=false)
Loads participant users and enriches each with watch-later data.
- std::list< `User` > `loadParticipantsWithWatchLaterParallel` (const std::vector< std::string > &participantIDs, std::vector< std::string > *missingData, std::size_t workerThreads)
- void `reportProgress` (double progress, const std::string &message)
- bool `registerBlendOnServer` (const `Blend` &blend, const std::string &creatorID)

Static Private Member Functions

- static std::optional< `User` > `parseUserFromAuthResponse` (const std::string &response, const std::string &fallbackPassword)
Parses an authenticated user object from the login response payload.
- static std::optional< std::string > `parseBlendID` (const std::string &response)
Extracts a blend identifier from a blend-related JSON response.
- static std::vector< std::string > `parseParticipantIDs` (const std::string &response)
Parses participant user IDs from a blend participant response.
- static std::list< `Video` > `parseWatchLaterVideos` (const std::string &response)
Parses watch-later videos from a watch-later endpoint response.

Private Attributes

- [AppState](#) & [appState](#)
- [EventRouter](#) [eventRouter](#)
- const std::string [server_url](#)
- [HttpClient](#) [http](#)
- bool [isConnected](#)
- std::function< void(double, const std::string &)> [progressReporter](#)
- std::unique_ptr< [Blend](#) > [currentBlend](#)
- std::optional< [User](#) > [pendingRegistration_](#)
- bool [pendingAccountCreated_](#) = false

6.2.1 Detailed Description

Stub for future use.

Central application coordinator.

[AppController](#) class declaration.

Definition at line 39 of file [AppController.hpp](#).

6.2.2 Constructor & Destructor Documentation

6.2.2.1 AppController() [1/2]

```
AppController::AppController () [explicit]
```

Initialises [AppState](#), [EventRouter](#), and backend services.

Definition at line 327 of file [AppController.cpp](#).

6.2.2.2 AppController() [2/2]

```
AppController::AppController (  
    const std::string & dbPath) [explicit]
```

Compatibility constructor used by tests that inject a DB path. The current HTTP-backed controller ignores this value.

Definition at line 351 of file [AppController.cpp](#).

6.2.2.3 ~AppController()

```
AppController::~AppController ()
```

Persists any unsaved changes, deregisters all [EventRouter](#) listeners, and releases backend resources.

Definition at line 354 of file [AppController.cpp](#).

6.2.3 Member Function Documentation

6.2.3.1 clearProgressReporter()

```
void AppController::clearProgressReporter ()
```

Clears the active UI progress sink.

Definition at line 407 of file [AppController.cpp](#).

6.2.3.2 enrichActiveBlendFeedIfMissingMetadata()

```
void AppController::enrichActiveBlendFeedIfMissingMetadata () [private]
```

Re-enriches only the currently displayed blend feed videos when any metadata fields are missing. This keeps display quality high without re-processing full watch-history sized lists on the UI path.

Definition at line 1234 of file [AppController.cpp](#).

6.2.3.3 enrichIfMissingMetadata()

```
void AppController::enrichIfMissingMetadata (
    const std::string & userID,
    std::list< Video > & videos) [private]
```

Re-enriches `videos` via the YouTube API if any are missing a title. Saves the updated list back to the database when enrichment occurs. No-op if all videos already have titles.

Parameters

<i>userID</i>	Owner of the video list (used for the DB save).
<i>videos</i>	Video list to inspect and potentially update in-place.

Definition at line 1118 of file [AppController.cpp](#).

6.2.3.4 enrichMissingMetadataSubsetMultithreaded()

```
std::list< Video > AppController::enrichMissingMetadataSubsetMultithreaded (
    const std::list< Video > & videos,
    std::size_t maxVideosToEnrich,
    std::size_t workerThreads,
    const std::function< std::list< Video > (const std::list< Video > &) > & enrich←
ChunkFn) [static]
```

Applies bounded metadata enrichment to at most `maxVideosToEnrich` videos that have incomplete metadata, partitioned across worker threads.

This utility is shared by production blend-loading paths and unit tests to keep metadata workflow behavior consistent.

Parameters

<i>videos</i>	list of videos to go through
<i>maxVideosToEnrich</i>	upper bound on the number of videos the algorithm should process
<i>workerThreads</i>	number of threads to be working on video enrichment
<i>enrichChunkFn</i>	function each thread will use to enrich their assigned sub-set of videos

Definition at line 1152 of file [AppController.cpp](#).

6.2.3.5 fetchUserBlends()

```
std::vector< AppController::BlendSummary > AppController::fetchUserBlends ()
```

Fetches the list of blends the current user participates in.

Returns

Vector of [BlendSummary](#) with blend IDs and titles.

Definition at line 1548 of file [AppController.cpp](#).

6.2.3.6 get_current_blend_id()

```
std::string AppController::get_current_blend_id ()
```

Holds summary info about a blend for display in the active blends list. Processes the current blend selected or loaded by default by the user

Returns

`string` representing the current blend id

Definition at line 1530 of file [AppController.cpp](#).

6.2.3.7 get_current_blend_title()

```
std::string AppController::get_current_blend_title ()
```

Processes the current blends title

Returns

`string` representing the blends title

Definition at line 1539 of file [AppController.cpp](#).

6.2.3.8 get_current_email()

```
std::string AppController::get_current_email ()
```

Retrieves the current users email (if an email was used on signup)

Returns

string copy of users email. "" if [User](#) DNE or they provided no email on signup

Definition at line 1524 of file [AppController.cpp](#).

6.2.3.9 get_current_user()

```
const User * AppController::get_current_user ()
```

Retrieves the current user from the AppState

Returns

[User](#) shared pointer. nullptr if [User](#) DNE

Definition at line 1514 of file [AppController.cpp](#).

6.2.3.10 get_current_username()

```
std::string AppController::get_current_username ()
```

Retrieves the current users username

Returns

string copy of username. "" if [User](#) DNE

Definition at line 1518 of file [AppController.cpp](#).

6.2.3.11 getEventRouter()

```
EventRouter & AppController::getEventRouter ()
```

Returns

Reference to the [EventRouter](#) so panels can call dispatch().

Definition at line 399 of file [AppController.cpp](#).

6.2.3.12 handleCreateBlend()

```
void AppController::handleCreateBlend (  
    const EventPayload & payload)
```

Runs the blend algorithm over the listed participants, stores the result, and creates a [ChatRoom](#) for the new blend.

Parameters

<i>payload</i>	{ "userID_0": "...", "userID_1": "...", ... }
----------------	---

Definition at line 957 of file [AppController.cpp](#).

6.2.3.13 handleDeleteAccount()

```
void AppController::handleDeleteAccount (  
    const EventPayload & payload)
```

Deletes the current account after password re-authentication.

Parameters

<i>payload</i>	{ "userID": "...", "password": "..." }
----------------	--

Definition at line 671 of file [AppController.cpp](#).

6.2.3.14 handleLeaveBlend()

```
void AppController::handleLeaveBlend (  
    const EventPayload & payload)
```

Leaves a blend: notifies server to remove the user, posts a system message to the chat, and clears the active blend if it was the one left.

Parameters

<i>payload</i>	{ "blendID": "...", "blendTitle": "..." }
----------------	---

Definition at line 1592 of file [AppController.cpp](#).

6.2.3.15 handleLogin()

```
void AppController::handleLogin (  
    const EventPayload & payload)
```

Validates credentials and sets [AppState::currentUser](#) on success.

Parameters

<i>payload</i>	{ "userID": "...", "password": "..." }
----------------	--

Definition at line 492 of file [AppController.cpp](#).

6.2.3.16 handleLogout()

```
void AppController::handleLogout (  
    const EventPayload & payload)
```

Persists the current user, clears [AppState](#), and navigates to [LoginPanel](#).

Parameters

<i>payload</i>	{ }
----------------	-----

Definition at line 639 of file [AppController.cpp](#).

6.2.3.17 handleOpenChat()

```
void AppController::handleOpenChat (
    const EventPayload & payload)
```

Opens the ChatPanel for the active blend's [ChatRoom](#). No-ops if there is no active blend or no room for it.

Parameters

<i>payload</i>	{ }
----------------	-----

Definition at line 1400 of file [AppController.cpp](#).

6.2.3.18 handlePlayVideo()

```
void AppController::handlePlayVideo (
    const EventPayload & payload)
```

Opens the selected video (currently a stub).

Parameters

<i>payload</i>	{ "videoID": "..." }
----------------	----------------------

Definition at line 1303 of file [AppController.cpp](#).

6.2.3.19 handleRefresh()

```
void AppController::handleRefresh (
    const EventPayload & payload)
```

Generates a new blend from the existing participants, excluding videos already in the current blend. Re-enriches any videos missing metadata via the YouTube API before sampling.

Parameters

<i>payload</i>	{ }
----------------	-----

Definition at line 1309 of file [AppController.cpp](#).

6.2.3.20 handleRegister()

```
void AppController::handleRegister (
    const EventPayload & payload)
```

Creates a new account and logs the user in on success.

Parameters

<i>payload</i>	{ "userID": "...", "username": "...", "email": "...", "password": "..." }
----------------	---

Definition at line 423 of file [AppController.cpp](#).

6.2.3.21 handleSelectBlend()

```
void AppController::handleSelectBlend (
    const EventPayload & payload)
```

Switches the active blend to the specified blend ID, loading participants and regenerating the video feed.

Parameters

<i>payload</i>	{ "blendID": "..." }
----------------	----------------------

Definition at line 1643 of file [AppController.cpp](#).

6.2.3.22 handleSendMessage()

```
void AppController::handleSendMessage (
    const EventPayload & payload)
```

Validates the sender is a blend participant, then adds the message.

Parameters

<i>payload</i>	{ "userID": "...", "text": "..." }
----------------	------------------------------------

Definition at line 1497 of file [AppController.cpp](#).

6.2.3.23 handleUploadData()

```
void AppController::handleUploadData (
    const EventPayload & payload)
```

Imports the file at filePath and persists the result to the database. Metadata enrichment is conditional and may be skipped for large HTML uploads to avoid blocking the UI path.

Parameters

<i>payload</i>	{ "filePath": "...", "userID": "..." }
----------------	--

Definition at line 711 of file [AppController.cpp](#).

6.2.3.24 loadParticipantsWithWatchLater()

```
std::list< User > AppController::loadParticipantsWithWatchLater (
    const std::vector< std::string > & participantIDs,
    std::vector< std::string > * missingData,
    bool enrichParticipantLists = false) [private]
```

Loads participant users and enriches each with watch-later data.

For each user ID, this method fetches watch-later videos from the server and constructs an in-memory [User](#) object containing the parsed [YouTubeData](#).

Parameters

<i>participantIDs</i>	Ordered list of user IDs to resolve.
<i>missingData</i>	Optional output vector populated with user IDs that could not be loaded or had no available watch-later data.

Returns

List of participant users that have usable watch-later data.

Definition at line 186 of file [AppController.cpp](#).

6.2.3.25 loadParticipantsWithWatchLaterParallel()

```
std::list< User > AppController::loadParticipantsWithWatchLaterParallel (
    const std::vector< std::string > & participantIDs,
    std::vector< std::string > * missingData,
    std::size_t workerThreads) [private]
```

Parallel variant used by login restore flows when many participants exist. Uses separate [HttpClient](#) instances per worker to avoid shared-client contention.

Definition at line 241 of file [AppController.cpp](#).

6.2.3.26 lookupUser()

```
bool AppController::lookupUser (
    const std::string & userID)
```

Checks whether a user with the given ID exists in the backend.

This uses the auth user-profile endpoint so account existence checks are independent from feature data (for example users with no uploaded watch-later list yet).

Parameters

<i>userID</i>	The user ID to look up.
---------------	-------------------------

Returns

`true` if the user was found, `false` otherwise.

Definition at line 1062 of file [AppController.cpp](#).

6.2.3.27 parseBlendID()

```
std::optional< std::string > AppController::parseBlendID (
    const std::string & response) [static], [private]
```

Extracts a blend identifier from a blend-related JSON response.

Parameters

<i>response</i>	Raw JSON response body expected to include <code>blend_id</code> .
-----------------	--

Returns

`Blend` ID when present and non-empty; otherwise `std::nullopt`.

Definition at line 114 of file [AppController.cpp](#).

6.2.3.28 parseChatHistory()

```
std::list< Message > AppController::parseChatHistory (
    const std::string & response) [static]
```

Parses the JSON response from GET /chat-history into a list of Messages. Expected format:

```
{"messages":[{"sender_id":"...", "content":"...", "sent_at":"..."},...]}
```

Exposed as public static so it can be unit-tested without a live server.

Parameters

<i>response</i>	Raw JSON string from the server.
-----------------	----------------------------------

Returns

Ordered list of Messages with original server timestamps.

Definition at line 1457 of file [AppController.cpp](#).

6.2.3.29 parseParticipantIDs()

```
std::vector< std::string > AppController::parseParticipantIDs (
    const std::string & response) [static], [private]
```

Parses participant user IDs from a blend participant response.

Parameters

<i>response</i>	Raw JSON response body expected to include an array field named <code>participants</code> .
-----------------	---

Returns

Vector of parsed participant IDs. Returns an empty vector when payload is malformed or the field is absent.

Definition at line 128 of file [AppController.cpp](#).

6.2.3.30 parseUserFromAuthResponse()

```
std::optional< User > AppController::parseUserFromAuthResponse (
    const std::string & response,
    const std::string & fallbackPassword) [static], [private]
```

Parses an authenticated user object from the login response payload.

Expects a JSON object that includes at minimum a non-empty `user_id` field. Optional fields such as `username` and `email` are used when present.

Parameters

<i>response</i>	Raw JSON response body returned by the auth endpoint.
<i>fallbackPassword</i>	Password provided at login time, used to populate the in-memory User object when the response does not include one.

Returns

Parsed [User](#) on success, or `std::nullopt` when parsing fails or required fields are missing.

Definition at line 94 of file [AppController.cpp](#).

6.2.3.31 parseWatchLaterVideos()

```
std::list< Video > AppController::parseWatchLaterVideos (  
    const std::string & response) [static], [private]
```

Parses watch-later videos from a watch-later endpoint response.

Parameters

<i>response</i>	Raw JSON response body expected to include an array field named <code>videos</code> .
-----------------	---

Returns

List of parsed [Video](#) objects. Entries missing required `video_id` are skipped.

Definition at line 149 of file [AppController.cpp](#).

6.2.3.32 registerBlendOnServer()

```
bool AppController::registerBlendOnServer (  
    const Blend & blend,  
    const std::string & creatorID) [private]
```

POSTs the given blend to POST `/api/v1/blends` so the server provisions its chat room. No-op when offline.

Returns

true if the server accepted the blend (2xx), false otherwise.

Definition at line 1075 of file [AppController.cpp](#).

6.2.3.33 registerEvents()

```
void AppController::registerEvents () [private]
```

Registers all UI event handlers on the [EventRouter](#). Called once during construction.

Definition at line 380 of file [AppController.cpp](#).

6.2.3.34 reportProgress()

```
void AppController::reportProgress (
    double progress,
    const std::string & message) [private]
```

Sends a coarse-grained progress update to the active UI reporter.

Definition at line 411 of file [AppController.cpp](#).

6.2.3.35 setProgressReporter()

```
void AppController::setProgressReporter (
    std::function< void(double, const std::string &)> reporter)
```

Registers a UI progress sink used for long-running controller operations.

Definition at line 403 of file [AppController.cpp](#).

6.2.4 Member Data Documentation

6.2.4.1 appState

```
AppState& AppController::appState [private]
```

Definition at line 236 of file [AppController.hpp](#).

6.2.4.2 currentBlend

```
std::unique_ptr<Blend> AppController::currentBlend [private]
```

Owns the most recently generated [Blend](#) so [AppState](#) can hold a raw pointer.

Definition at line 244 of file [AppController.hpp](#).

6.2.4.3 eventRouter

```
EventRouter AppController::eventRouter [private]
```

Definition at line 237 of file [AppController.hpp](#).

6.2.4.4 http

```
HttpClient AppController::http [private]
```

Definition at line 239 of file [AppController.hpp](#).

6.2.4.5 isConnected

```
bool AppController::isConnected [private]
```

Definition at line 240 of file [AppController.hpp](#).

6.2.4.6 pendingAccountCreated_

```
bool AppController::pendingAccountCreated_ = false [private]
```

Definition at line 349 of file [AppController.hpp](#).

6.2.4.7 pendingRegistration_

```
std::optional<User> AppController::pendingRegistration_ [private]
```

Definition at line 348 of file [AppController.hpp](#).

6.2.4.8 progressReporter

```
std::function<void(double, const std::string&)> AppController::progressReporter [private]
```

Definition at line 241 of file [AppController.hpp](#).

6.2.4.9 server_url

```
const std::string AppController::server_url [private]
```

Definition at line 238 of file [AppController.hpp](#).

The documentation for this class was generated from the following files:

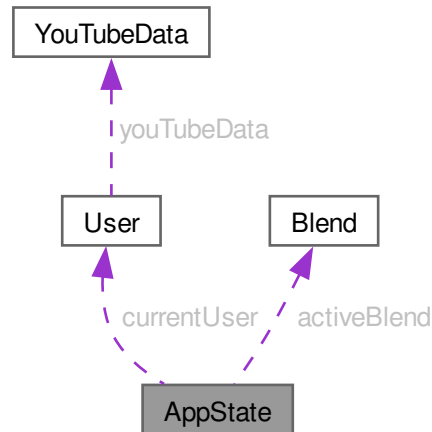
- [AppController.hpp](#)
- [AppController.cpp](#)

6.3 AppState Class Reference

[AppState](#) class declaration.

```
#include <AppState.hpp>
```

Collaboration diagram for AppState:



Public Member Functions

- `User * getCurrentUser () const`
- `void setCurrentUser (User *user)`
- `Blend * getActiveBlend () const`
- `void setActiveBlend (Blend *blend)`
- `bool isGeneratingBlend () const`
- `void setIsBlendGenerating (bool flag)`
- `ChatRoom * getActiveChatRoom ()`
- `void createChatRoom (const Blend &blend)`
- `ChatRoom * getChatRoom (const std::string &blendID)`
- `void addSessionUser (const User &user)`
- `std::map< std::string, User > getSessionUsers () const`
- `void clearSessionUsers ()`
- `void setUsersWithoutData (const std::vector< std::string > &users)`
- `std::vector< std::string > getUsersWithoutData () const`
- `void setPendingBlendMessage (const std::string &msg)`
- `std::string takePendingBlendMessage ()`
- `void setPendingUploadError (const std::string &msg)`
- `std::string takePendingUploadError ()`
- `void clearSession ()`

Static Public Member Functions

- `static AppState & getInstance ()`

Private Member Functions

- **AppState** (const [AppState](#) &)=delete
- **AppState** & **operator=** (const [AppState](#) &)=delete

Private Attributes

- [User](#) * **currentUser**
Logged-in user; null when no session is active.
- [Blend](#) * **activeBlend**
Most recently generated or loaded [Blend](#); null if none.
- bool **isBlendGenerating**
True while an [IBlendAlgorithm](#) is running.
- std::map< std::string, [ChatRoom](#) > **chatRooms**
- std::map< std::string, [User](#) > **sessionUsers**
- std::vector< std::string > **usersWithoutData**
- std::string **pendingBlendMessage**
- std::string **pendingUploadError**

6.3.1 Detailed Description

[AppState](#) class declaration.

Session-scoped singleton holding all runtime application state.

Definition at line 31 of file [AppState.hpp](#).

6.3.2 Constructor & Destructor Documentation

6.3.2.1 AppState()

```
AppState::AppState () [private]
```

Definition at line 11 of file [AppState.cpp](#).

6.3.3 Member Function Documentation

6.3.3.1 addSessionUser()

```
void AppState::addSessionUser (
    const User & user)
```

Adds or replaces the session entry for this user.

Parameters

<i>user</i>	User who has uploaded blend data.
-------------	---

Definition at line 90 of file [AppState.cpp](#).

6.3.3.2 clearSession()

```
void AppState::clearSession ()
```

Resets all session state: clears currentUser, activeBlend, isBlendGenerating, chat rooms, and session users. Called by [AppController](#) on logout.

Definition at line 134 of file [AppState.cpp](#).

6.3.3.3 clearSessionUsers()

```
void AppState::clearSessionUsers ()
```

Removes all session participants (call before starting a new blend).

Definition at line 98 of file [AppState.cpp](#).

6.3.3.4 createChatRoom()

```
void AppState::createChatRoom (  
    const Blend & blend)
```

Creates a [ChatRoom](#) for the given [Blend](#). Replaces any existing room with the same blendID.

Parameters

<i>blend</i>	The Blend whose participants populate the new room.
--------------	---

Definition at line 67 of file [AppState.cpp](#).

6.3.3.5 getActiveBlend()

```
Blend * AppState::getActiveBlend () const
```

Returns

Pointer to the active [Blend](#), or nullptr if none exists.

Definition at line 42 of file [AppState.cpp](#).

6.3.3.6 getActiveChatRoom()

```
ChatRoom * AppState::getActiveChatRoom ()
```

Returns a pointer to the [ChatRoom](#) for the active [Blend](#).

Returns

Pointer to the room, or nullptr if no active blend or no room exists.

Definition at line 62 of file [AppState.cpp](#).

6.3.3.7 getChatRoom()

```
ChatRoom * AppState::getChatRoom (  
    const std::string & blendID)
```

Returns a pointer to the [ChatRoom](#) for the given blendID.

Parameters

<i>blendID</i>	ID of the desired blend.
----------------	--------------------------

Returns

Pointer to the room, or nullptr if none exists.

Definition at line 82 of file [AppState.cpp](#).

6.3.3.8 getCurrentUser()

```
User * AppState::getCurrentUser () const
```

Returns

Pointer to the logged-in user, or nullptr if no session is active.

Definition at line 26 of file [AppState.cpp](#).

6.3.3.9 getInstance()

```
AppState & AppState::getInstance () [static]
```

Returns the single global [AppState](#) instance. Creates it on first call; returns the same instance on every subsequent call.

Definition at line 18 of file [AppState.cpp](#).

6.3.3.10 getSessionUsers()

```
std::map< std::string, User > AppState::getSessionUsers () const
```

Returns

All users that have uploaded data in this session.

Definition at line 94 of file [AppState.cpp](#).

6.3.3.11 getUsersWithoutData()

```
std::vector< std::string > AppState::getUsersWithoutData () const
```

Returns

userIDs of participants without data from the last blend creation.

Definition at line 108 of file [AppState.cpp](#).

6.3.3.12 isGeneratingBlend()

```
bool AppState::isGeneratingBlend () const
```

Returns

True while an [IBlendAlgorithm](#) is running.

Definition at line 52 of file [AppState.cpp](#).

6.3.3.13 setActiveBlend()

```
void AppState::setActiveBlend (  
    Blend * blend)
```

Parameters

<i>blend</i>	Raw pointer to the active Blend (AppController owns it).
--------------	---

Definition at line 46 of file [AppState.cpp](#).

6.3.3.14 setCurrentUser()

```
void AppState::setCurrentUser (  
    User * user)
```

Parameters

<i>user</i>	Pointer to the newly logged-in User (AppController owns it).
-------------	---

Definition at line 30 of file [AppState.cpp](#).

6.3.3.15 setIsBlendGenerating()

```
void AppState::setIsBlendGenerating (  
    bool flag)
```

Parameters

<i>flag</i>	True to mark blend generation as in-progress.
-------------	---

Definition at line 56 of file [AppState.cpp](#).

6.3.3.16 setPendingBlendMessage()

```
void AppState::setPendingBlendMessage (  
    const std::string & msg)
```

Sets a one-time message shown by [LoginPanel](#) when the user is a blend participant but has not yet uploaded data.

Parameters

<i>msg</i>	Message to display.
------------	---------------------

Definition at line 112 of file [AppState.cpp](#).

6.3.3.17 setPendingUploadError()

```
void AppState::setPendingUploadError (  
    const std::string & msg)
```

Sets a one-time message shown by [DataInstructionsPanel](#) when upload failed or produced no usable videos.

Parameters

<i>msg</i>	Message to display.
------------	---------------------

Definition at line 122 of file [AppState.cpp](#).

6.3.3.18 setUsersWithoutData()

```
void AppState::setUsersWithoutData (  
    const std::vector< std::string > & users)
```

Stores userIDs of participants who had no Watch Later data when the blend was last created.

Parameters

<i>users</i>	List of userIDs without data.
--------------	-------------------------------

Definition at line 104 of file [AppState.cpp](#).

6.3.3.19 takePendingBlendMessage()

```
std::string AppState::takePendingBlendMessage ()
```

Returns and clears the pending blend message.

Returns

The pending message string, then clears it.

Definition at line 116 of file [AppState.cpp](#).

6.3.3.20 takePendingUploadError()

```
std::string AppState::takePendingUploadError ()
```

Returns and clears the pending upload error message.

Returns

The pending upload error string, then clears it.

Definition at line 126 of file [AppState.cpp](#).

6.3.4 Member Data Documentation

6.3.4.1 activeBlend

```
Blend* AppState::activeBlend [private]
```

Most recently generated or loaded [Blend](#); null if none.

Definition at line 34 of file [AppState.hpp](#).

6.3.4.2 chatRooms

```
std::map<std::string, ChatRoom> AppState::chatRooms [private]
```

All chat rooms for this session, keyed by blendID.

Definition at line 38 of file [AppState.hpp](#).

6.3.4.3 currentUser

```
User* AppState::currentUser [private]
```

Logged-in user; null when no session is active.

Definition at line 33 of file [AppState.hpp](#).

6.3.4.4 isBlendGenerating

```
bool AppState::isBlendGenerating [private]
```

True while an [IBlendAlgorithm](#) is running.

Definition at line 35 of file [AppState.hpp](#).

6.3.4.5 pendingBlendMessage

```
std::string AppState::pendingBlendMessage [private]
```

One-time message set by handleLogin when the user belongs to a blend but has not yet uploaded data. Cleared after being read.

Definition at line 50 of file [AppState.hpp](#).

6.3.4.6 pendingUploadError

```
std::string AppState::pendingUploadError [private]
```

One-time upload failure message set by handleUploadData when parsing, enrichment, or persistence fails. Cleared after being read by UI.

Definition at line 56 of file [AppState.hpp](#).

6.3.4.7 sessionUsers

```
std::map<std::string, User> AppState::sessionUsers [private]
```

Participants that have uploaded Watch Later data, keyed by userID.

Definition at line 41 of file [AppState.hpp](#).

6.3.4.8 usersWithoutData

```
std::vector<std::string> AppState::usersWithoutData [private]
```

Participants who had no data when the last blend was created.

Definition at line 44 of file [AppState.hpp](#).

The documentation for this class was generated from the following files:

- [AppState.hpp](#)
- [AppState.cpp](#)

6.4 Blend Class Reference

[Blend](#) class declaration.

```
#include <Blend.hpp>
```


Public Member Functions

- [Blend](#) (const std::string &blendID, const std::string &title, const std::string &algorithmUsed, const std::list<[User](#) > &participants, const std::list< [Video](#) > &videoList)
- std::string [getBlendID](#) () const
- std::string [getTitle](#) () const
- std::string [getAlgorithmUsed](#) () const
- std::list< [User](#) > [getParticipants](#) () const
- std::list< [Video](#) > [getVideoList](#) () const
- void [setVideoList](#) (const std::list< [Video](#) > &videoList)
- [Video](#) [getVideo](#) (int index) const
- int [size](#) () const
- void [setBlendID](#) (const std::string &blendID)
- std::string [toString](#) () const

Private Attributes

- std::string [blendID](#)
- std::string [title](#)
- std::string [algorithmUsed](#)
- std::list< [User](#) > [participants](#)
- std::list< [Video](#) > [videoList](#)

6.4.1 Detailed Description

[Blend](#) class declaration.

Definition at line 24 of file [Blend.hpp](#).

6.4.2 Constructor & Destructor Documentation

6.4.2.1 Blend()

```
Blend::Blend (
    const std::string & blendID,
    const std::string & title,
    const std::string & algorithmUsed,
    const std::list< User > & participants,
    const std::list< Video > & videoList)
```

Constructs a [Blend](#) with all required fields.

Parameters

<i>blendID</i>	Unique identifier for this blend.
<i>title</i>	User-facing name for this blend.
<i>algorithmUsed</i>	Name of the IBlendAlgorithm that created it.
<i>participants</i>	Users whose Watch Later data was used as input.
<i>videoList</i>	Ordered list of videos selected by the algorithm.

Definition at line 14 of file [Blend.cpp](#).

6.4.3 Member Function Documentation

6.4.3.1 getAlgorithmUsed()

```
std::string Blend::getAlgorithmUsed () const
```

Returns

Name of the algorithm used to generate this blend.

Definition at line 35 of file [Blend.cpp](#).

6.4.3.2 getBlendID()

```
std::string Blend::getBlendID () const
```

Returns

Unique blend identifier.

Definition at line 27 of file [Blend.cpp](#).

6.4.3.3 getParticipants()

```
std::list< User > Blend::getParticipants () const
```

Returns

List of users who participated in this blend.

Definition at line 39 of file [Blend.cpp](#).

6.4.3.4 getTitle()

```
std::string Blend::getTitle () const
```

Returns

User-facing title of this blend.

Definition at line 31 of file [Blend.cpp](#).

6.4.3.5 getVideo()

```
Video Blend::getVideo (  
    int index) const
```

Returns the video at a given position, wrapping around the list.

Parameters

<i>index</i>	Zero-based index; behaviour is undefined for negative values.
--------------	---

Returns

[Video](#) at the requested position.

Definition at line 59 of file [Blend.cpp](#).

6.4.3.6 `getVideoList()`

```
std::list< Video > Blend::getVideoList () const
```

Returns

Ordered list of videos in this blend.

Definition at line 43 of file [Blend.cpp](#).

6.4.3.7 `setBlendID()`

```
void Blend::setBlendID (  
    const std::string & blendID)
```

Overrides the blend identifier (used when reconciling with server IDs).

Parameters

<i>blendID</i>	Server-authoritative blend identifier.
----------------	--

Definition at line 53 of file [Blend.cpp](#).

6.4.3.8 `setVideoList()`

```
void Blend::setVideoList (  
    const std::list< Video > & videoList)
```

Replaces the current video list (e.g. after API enrichment).

Parameters

<i>videoList</i>	New video list to store.
------------------	--------------------------

Definition at line 49 of file [Blend.cpp](#).

6.4.3.9 size()

```
int Blend::size () const
```

Returns

Number of videos in the blend.

Definition at line 69 of file [Blend.cpp](#).

6.4.3.10 toString()

```
std::string Blend::toString () const
```

Returns

JSON string representation of this blend.

Definition at line 73 of file [Blend.cpp](#).

6.4.4 Member Data Documentation

6.4.4.1 algorithmUsed

```
std::string Blend::algorithmUsed [private]
```

Definition at line 28 of file [Blend.hpp](#).

6.4.4.2 blendID

```
std::string Blend::blendID [private]
```

Definition at line 26 of file [Blend.hpp](#).

6.4.4.3 participants

```
std::list<User> Blend::participants [private]
```

Definition at line 29 of file [Blend.hpp](#).

6.4.4.4 title

```
std::string Blend::title [private]
```

Definition at line 27 of file [Blend.hpp](#).

6.4.4.5 videoList

```
std::list<Video> Blend::videoList [private]
```

Definition at line 30 of file [Blend.hpp](#).

The documentation for this class was generated from the following files:

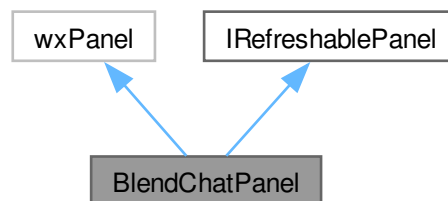
- [Blend.hpp](#)
- [Blend.cpp](#)

6.5 BlendChatPanel Class Reference

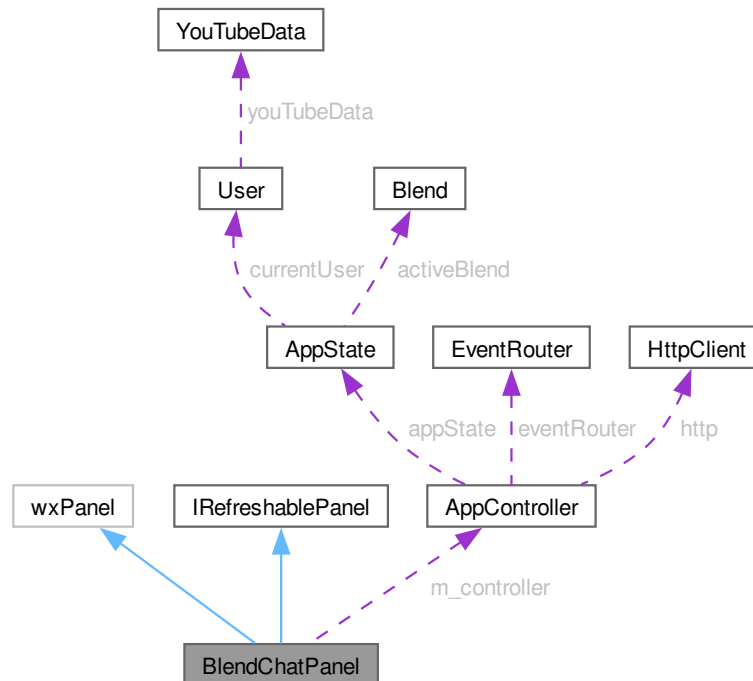
[BlendChatPanel](#) class declaration.

```
#include <BlendChatPanel.hpp>
```

Inheritance diagram for BlendChatPanel:



Collaboration diagram for BlendChatPanel:



Public Member Functions

- `BlendChatPanel` (`wxWindow *parent`, `AppController &controller`, `NavigateFn nav`)
- void `Refresh` () override
- void `Reload` ()

Private Member Functions

- void `ConnectWebSocket` (const std::string &blendID, const std::string &userID)
- void `DoSend` ()
- void `OnSend` (wxCommandEvent &evt)
- void `OnInputEnter` (wxCommandEvent &evt)
- void `OnPollTimer` (wxTimerEvent &evt)
- void `OnParticipantsToggle` (wxCommandEvent &evt)

Static Private Member Functions

- static wxString `FormatTimestamp` (std::time_t ts, std::time_t now)
Formats one chat message timestamp into a user-facing local-time label.

Private Attributes

- [AppController](#) & [m_controller](#)
- [NavigateFn](#) [m_nav](#)
- [wxScrolledWindow](#) * [m_msgScroll](#)
- [wxPanel](#) * [m_msgInner](#)
- [wxBoxSizer](#) * [m_msgSizer](#)
- [wxStaticText](#) * [m_noBlendLabel](#)
- [wxButton](#) * [m_participantsBtn](#)
- [wxPanel](#) * [m_participantsDropdown](#)
- [wxBoxSizer](#) * [m_participantsSizer](#)
- [wxTextCtrl](#) * [m_inputCtrl](#)
- [wxPanel](#) * [m_inputPanel](#)
- [wxTimer](#) * [m_pollTimer](#)
- [std::string](#) [m_wsBlendID](#)
- [std::mutex](#) [m_incomingMx](#)
- [std::vector< std::pair< std::string, std::string > >](#) [m_incoming](#)
- [std::unique_ptr< ChatWebSocket >](#) [m_ws](#)

6.5.1 Detailed Description

[BlendChatPanel](#) class declaration.

Definition at line 64 of file [BlendChatPanel.hpp](#).

6.5.2 Constructor & Destructor Documentation

6.5.2.1 BlendChatPanel()

```
BlendChatPanel::BlendChatPanel (
    wxWindow * parent,
    AppController & controller,
    NavigateFn nav)
```

Definition at line 82 of file [BlendChatPanel.cpp](#).

6.5.3 Member Function Documentation

6.5.3.1 ConnectWebSocket()

```
void BlendChatPanel::ConnectWebSocket (
    const std::string & blendID,
    const std::string & userID) [private]
```

Definition at line 406 of file [BlendChatPanel.cpp](#).

6.5.3.2 DoSend()

```
void BlendChatPanel::DoSend () [private]
```

Definition at line 355 of file [BlendChatPanel.cpp](#).

6.5.3.3 FormatTimestamp()

```
wxString BlendChatPanel::FormatTimestamp (
    std::time_t ts,
    std::time_t now) [static], [private]
```

Formats one chat message timestamp into a user-facing local-time label.

Formatting rules are relative to `now`:

- Same local day: "Today at HH:MM"
- Previous local day: "Yesterday at HH:MM"
- Older than yesterday but less than 12 months: "Mon DD at HH:MM"
- 12+ months old: "Mon DD, YYYY at HH:MM"

Parameters

<i>ts</i>	Message timestamp (Unix seconds).
<i>now</i>	Reference timestamp used for relative display (Unix seconds).

Returns

Formatted local timestamp string for display in the chat UI.

Definition at line 48 of file [BlendChatPanel.cpp](#).

6.5.3.4 OnInputEnter()

```
void BlendChatPanel::OnInputEnter (
    wxCommandEvent & evt) [private]
```

Definition at line 392 of file [BlendChatPanel.cpp](#).

6.5.3.5 OnParticipantsToggle()

```
void BlendChatPanel::OnParticipantsToggle (
    wxCommandEvent & evt) [private]
```

Definition at line 397 of file [BlendChatPanel.cpp](#).

6.5.3.6 OnPollTimer()

```
void BlendChatPanel::OnPollTimer (
    wxTimerEvent & evt) [private]
```

Definition at line 434 of file [BlendChatPanel.cpp](#).

6.5.3.7 OnSend()

```
void BlendChatPanel::OnSend (
    wxCommandEvent & evt) [private]
```

Definition at line 387 of file [BlendChatPanel.cpp](#).

6.5.3.8 Refresh()

```
void BlendChatPanel::Refresh () [override], [virtual]
```

Refreshes the panel's display to reflect current application state. Called by [MainFrame](#) whenever the panel becomes active.

Implements [IRefreshablePanel](#).

Definition at line 192 of file [BlendChatPanel.cpp](#).

6.5.3.9 Reload()

```
void BlendChatPanel::Reload ()
```

Definition at line 197 of file [BlendChatPanel.cpp](#).

6.5.4 Member Data Documentation

6.5.4.1 m_controller

```
AppController& BlendChatPanel::m_controller [private]
```

Definition at line 74 of file [BlendChatPanel.hpp](#).

6.5.4.2 m_incoming

```
std::vector<std::pair<std::string, std::string> > BlendChatPanel::m_incoming [private]
```

Definition at line 113 of file [BlendChatPanel.hpp](#).

6.5.4.3 m_incomingMx

```
std::mutex BlendChatPanel::m_incomingMx [private]
```

Definition at line 108 of file [BlendChatPanel.hpp](#).

6.5.4.4 m_inputCtrl

```
wxTextCtrl* BlendChatPanel::m_inputCtrl [private]
```

Definition at line 94 of file [BlendChatPanel.hpp](#).

6.5.4.5 m_inputPanel

```
wxPanel* BlendChatPanel::m_inputPanel [private]
```

Definition at line 97 of file [BlendChatPanel.hpp](#).

6.5.4.6 m_msgInner

```
wxPanel* BlendChatPanel::m_msgInner [private]
```

Definition at line 80 of file [BlendChatPanel.hpp](#).

6.5.4.7 m_msgScroll

```
wxScrolledWindow* BlendChatPanel::m_msgScroll [private]
```

Definition at line 78 of file [BlendChatPanel.hpp](#).

6.5.4.8 m_msgSizer

```
wxBoxSizer* BlendChatPanel::m_msgSizer [private]
```

Definition at line 82 of file [BlendChatPanel.hpp](#).

6.5.4.9 m_nav

```
NavigateFn BlendChatPanel::m_nav [private]
```

Definition at line 75 of file [BlendChatPanel.hpp](#).

6.5.4.10 m_noBlendLabel

```
wxStaticText* BlendChatPanel::m_noBlendLabel [private]
```

Definition at line 85 of file [BlendChatPanel.hpp](#).

6.5.4.11 m_participantsBtn

```
wxButton* BlendChatPanel::m_participantsBtn [private]
```

Definition at line 88 of file [BlendChatPanel.hpp](#).

6.5.4.12 m_participantsDropdown

```
wxPanel* BlendChatPanel::m_participantsDropdown [private]
```

Definition at line 90 of file [BlendChatPanel.hpp](#).

6.5.4.13 m_participantsSizer

```
wxBoxSizer* BlendChatPanel::m_participantsSizer [private]
```

Definition at line 91 of file [BlendChatPanel.hpp](#).

6.5.4.14 m_pollTimer

```
wxTimer* BlendChatPanel::m_pollTimer [private]
```

Definition at line 102 of file [BlendChatPanel.hpp](#).

6.5.4.15 m_ws

```
std::unique_ptr<ChatWebSocket> BlendChatPanel::m_ws [private]
```

Definition at line 116 of file [BlendChatPanel.hpp](#).

6.5.4.16 m_wsBlendID

```
std::string BlendChatPanel::m_wsBlendID [private]
```

Definition at line 105 of file [BlendChatPanel.hpp](#).

The documentation for this class was generated from the following files:

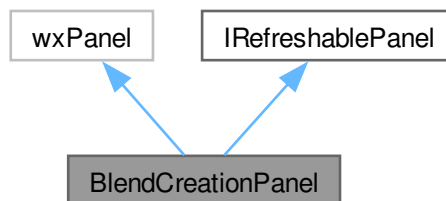
- [BlendChatPanel.hpp](#)
- [BlendChatPanel.cpp](#)

6.6 BlendCreationPanel Class Reference

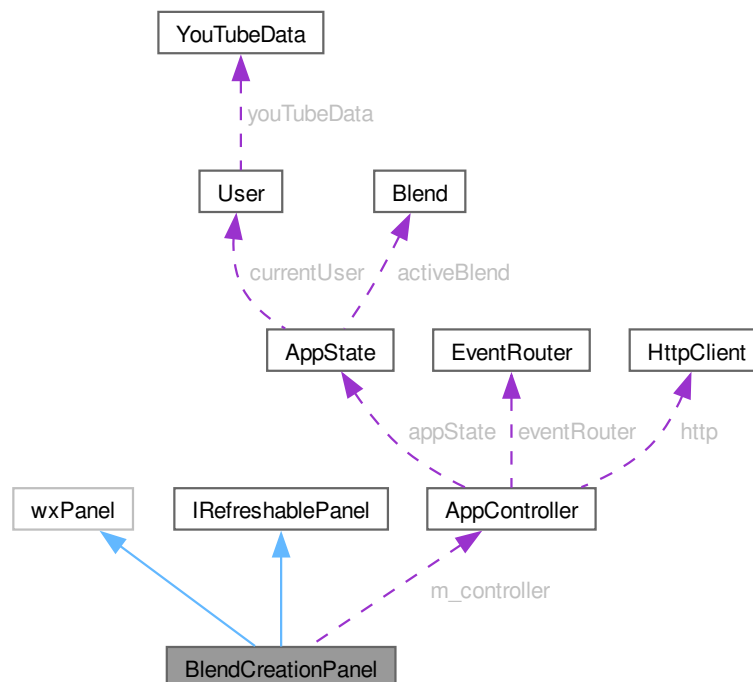
[BlendCreationPanel](#) class declaration.

```
#include <BlendCreationPanel.hpp>
```

Inheritance diagram for BlendCreationPanel:



Collaboration diagram for BlendCreationPanel:



Public Member Functions

- [BlendCreationPanel](#) (wxWindow *parent, [AppController](#) &controller, [NavigateFn](#) nav)
- void [Refresh](#) () override
- void [Reload](#) ()

Private Member Functions

- void [RebuildUserList](#) ()
- void [UpdateCountLabel](#) ()
- void [OnAdd](#) (wxCommandEvent &evt)
- void [OnCreate](#) (wxCommandEvent &evt)
- void [OnRemoveUser](#) (const std::string &username)

Private Attributes

- [AppController](#) & [m_controller](#)
- [NavigateFn](#) [m_nav](#)
- wxScrolledWindow * [m_userListScroll](#)
- wxPanel * [m_userListInner](#)
- wxBoxSizer * [m_userListSizer](#)
- wxStaticText * [m_emptyLabel](#)
- wxStaticText * [m_countLabel](#)
- wxTextCtrl * [m_blendNameCtrl](#)
- wxButton * [m_createBtn](#)
- std::vector< std::string > [m_addedUsers](#)

6.6.1 Detailed Description

[BlendCreationPanel](#) class declaration.

Definition at line 59 of file [BlendCreationPanel.hpp](#).

6.6.2 Constructor & Destructor Documentation

6.6.2.1 BlendCreationPanel()

```
BlendCreationPanel::BlendCreationPanel (  
    wxWindow * parent,  
    AppController & controller,  
    NavigateFn nav)
```

Definition at line 28 of file [BlendCreationPanel.cpp](#).

6.6.3 Member Function Documentation

6.6.3.1 OnAdd()

```
void BlendCreationPanel::OnAdd (  
    wxCommandEvent & evt) [private]
```

Definition at line 320 of file [BlendCreationPanel.cpp](#).

6.6.3.2 OnCreate()

```
void BlendCreationPanel::OnCreate (
    wxCommandEvent & evt) [private]
```

Definition at line 362 of file [BlendCreationPanel.cpp](#).

6.6.3.3 OnRemoveUser()

```
void BlendCreationPanel::OnRemoveUser (
    const std::string & username) [private]
```

Definition at line 418 of file [BlendCreationPanel.cpp](#).

6.6.3.4 RebuildUserList()

```
void BlendCreationPanel::RebuildUserList () [private]
```

Definition at line 227 of file [BlendCreationPanel.cpp](#).

6.6.3.5 Refresh()

```
void BlendCreationPanel::Refresh () [override], [virtual]
```

Refreshes the panel's display to reflect current application state. Called by [MainFrame](#) whenever the panel becomes active.

Implements [IRefreshablePanel](#).

Definition at line 200 of file [BlendCreationPanel.cpp](#).

6.6.3.6 Reload()

```
void BlendCreationPanel::Reload ()
```

Definition at line 205 of file [BlendCreationPanel.cpp](#).

6.6.3.7 UpdateCountLabel()

```
void BlendCreationPanel::UpdateCountLabel () [private]
```

Definition at line 311 of file [BlendCreationPanel.cpp](#).

6.6.4 Member Data Documentation

6.6.4.1 m_addedUsers

```
std::vector<std::string> BlendCreationPanel::m_addedUsers [private]
```

Definition at line 114 of file [BlendCreationPanel.hpp](#).

6.6.4.2 m_blendNameCtrl

```
wxTextCtrl* BlendCreationPanel::m_blendNameCtrl [private]
```

Definition at line 108 of file [BlendCreationPanel.hpp](#).

6.6.4.3 m_controller

```
AppController& BlendCreationPanel::m_controller [private]
```

Definition at line 91 of file [BlendCreationPanel.hpp](#).

6.6.4.4 m_countLabel

```
wxStaticText* BlendCreationPanel::m_countLabel [private]
```

Definition at line 105 of file [BlendCreationPanel.hpp](#).

6.6.4.5 m_createBtn

```
wxButton* BlendCreationPanel::m_createBtn [private]
```

Definition at line 111 of file [BlendCreationPanel.hpp](#).

6.6.4.6 m_emptyLabel

```
wxStaticText* BlendCreationPanel::m_emptyLabel [private]
```

Definition at line 102 of file [BlendCreationPanel.hpp](#).

6.6.4.7 m_nav

```
NavigateFn BlendCreationPanel::m_nav [private]
```

Definition at line 92 of file [BlendCreationPanel.hpp](#).

6.6.4.8 m_userListInner

```
wxPanel* BlendCreationPanel::m_userListInner [private]
```

Definition at line 97 of file [BlendCreationPanel.hpp](#).

6.6.4.9 m_userListScroll

```
wxScrolledWindow* BlendCreationPanel::m_userListScroll [private]
```

Definition at line 95 of file [BlendCreationPanel.hpp](#).

6.6.4.10 m_userListSizer

```
wxBoxSizer* BlendCreationPanel::m_userListSizer [private]
```

Definition at line 99 of file [BlendCreationPanel.hpp](#).

The documentation for this class was generated from the following files:

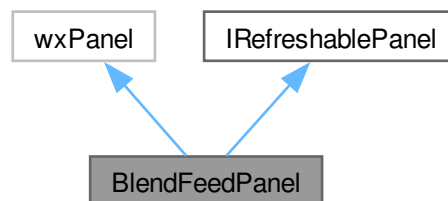
- [BlendCreationPanel.hpp](#)
- [BlendCreationPanel.cpp](#)

6.7 BlendFeedPanel Class Reference

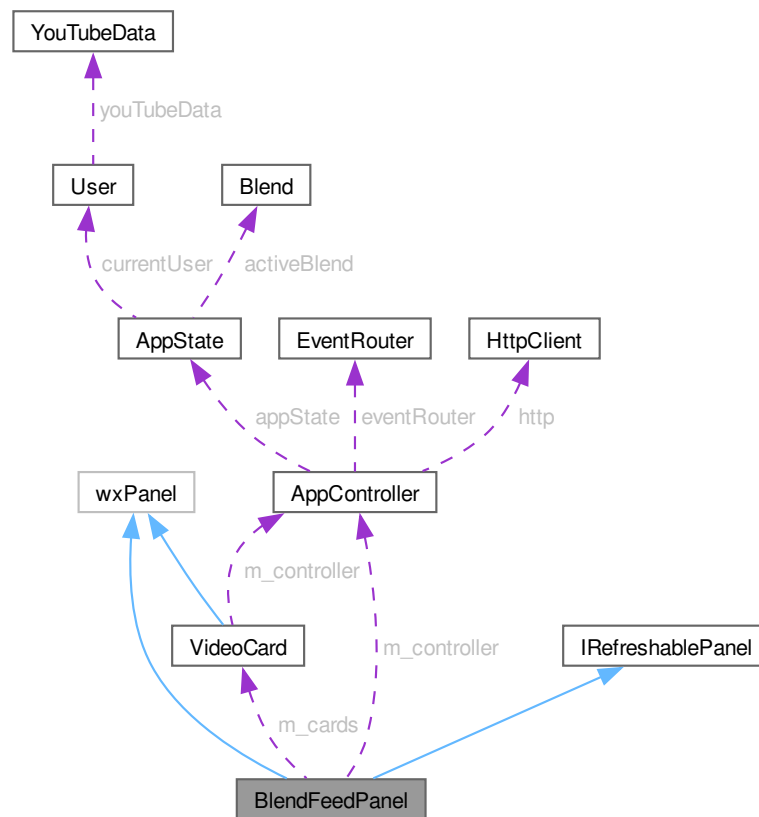
Panel that renders paged blend videos in a fixed 3x2 card layout.

```
#include <BlendFeedPanel.hpp>
```

Inheritance diagram for BlendFeedPanel:



Collaboration diagram for BlendFeedPanel:



Public Member Functions

- `BlendFeedPanel` (`wxWindow *parent`, `AppController &controller`)
Construct a `BlendFeedPanel`.
- `void Refresh ()` override
Refresh panel state from current application data.
- `void NextPage ()`
Advance to the next six videos, wrapping around the blend.
- `void LoadFromBlend ()`
Reload the current page from the active blend.

Private Attributes

- `AppController & m_controller`
- `VideoCard * m_cards [kPageSize]`
- `int m_offset`

Static Private Attributes

- `static constexpr int kPageSize = 6`

6.7.1 Detailed Description

Panel that renders paged blend videos in a fixed 3x2 card layout.

Definition at line 33 of file [BlendFeedPanel.hpp](#).

6.7.2 Constructor & Destructor Documentation

6.7.2.1 BlendFeedPanel()

```
BlendFeedPanel::BlendFeedPanel (
    wxWindow * parent,
    AppController & controller)
```

Construct a [BlendFeedPanel](#).

Parameters

<i>parent</i>	Parent wxWidgets window.
<i>controller</i>	Application controller used for refresh dispatches.

Definition at line 21 of file [BlendFeedPanel.cpp](#).

6.7.3 Member Function Documentation

6.7.3.1 LoadFromBlend()

```
void BlendFeedPanel::LoadFromBlend ()
```

Reload the current page from the active blend.

Clears all cards if there is no active blend.

Definition at line 76 of file [BlendFeedPanel.cpp](#).

6.7.3.2 NextPage()

```
void BlendFeedPanel::NextPage ()
```

Advance to the next six videos, wrapping around the blend.

Definition at line 53 of file [BlendFeedPanel.cpp](#).

6.7.3.3 Refresh()

```
void BlendFeedPanel::Refresh () [override], [virtual]
```

Refresh panel state from current application data.

Implements [IRefreshablePanel](#).

Definition at line 44 of file [BlendFeedPanel.cpp](#).

6.7.4 Member Data Documentation

6.7.4.1 kPageSize

```
int BlendFeedPanel::kPageSize = 6 [static], [constexpr], [private]
```

Definition at line 60 of file [BlendFeedPanel.hpp](#).

6.7.4.2 m_cards

```
VideoCard* BlendFeedPanel::m_cards[kPageSize] [private]
```

Definition at line 63 of file [BlendFeedPanel.hpp](#).

6.7.4.3 m_controller

```
AppController& BlendFeedPanel::m_controller [private]
```

Definition at line 62 of file [BlendFeedPanel.hpp](#).

6.7.4.4 m_offset

```
int BlendFeedPanel::m_offset [private]
```

Definition at line 64 of file [BlendFeedPanel.hpp](#).

The documentation for this class was generated from the following files:

- [BlendFeedPanel.hpp](#)
- [BlendFeedPanel.cpp](#)

6.8 AppController::BlendSummary Struct Reference

[BlendSummary](#) class declaration.

```
#include <AppController.hpp>
```

Public Attributes

- std::string [blendID](#)
- std::string [title](#)

6.8.1 Detailed Description

[BlendSummary](#) class declaration.

Definition at line 183 of file [AppController.hpp](#).

6.8.2 Member Data Documentation

6.8.2.1 blendID

```
std::string AppController::BlendSummary::blendID
```

Definition at line 184 of file [AppController.hpp](#).

6.8.2.2 title

```
std::string AppController::BlendSummary::title
```

Definition at line 185 of file [AppController.hpp](#).

The documentation for this struct was generated from the following file:

- [AppController.hpp](#)

6.9 BtnConfig Struct Reference

Width/height dimensions for a button preset.

```
#include <ButtonsConfig.hpp>
```

Public Attributes

- int [minWidth](#)
- int [minHeight](#)
- int [maxWidth](#)
- int [maxHeight](#)

6.9.1 Detailed Description

Width/height dimensions for a button preset.

A width of -1 means the control can expand horizontally in an expanding sizer while keeping the configured minimum height.

Definition at line 50 of file [ButtonsConfig.hpp](#).

6.9.2 Member Data Documentation

6.9.2.1 maxHeight

```
int BtnConfig::maxHeight
```

Definition at line 54 of file [ButtonsConfig.hpp](#).

6.9.2.2 maxWidth

```
int BtnConfig::maxWidth
```

Definition at line 53 of file [ButtonsConfig.hpp](#).

6.9.2.3 minHeight

```
int BtnConfig::minHeight
```

Definition at line 52 of file [ButtonsConfig.hpp](#).

6.9.2.4 minWidth

```
int BtnConfig::minWidth
```

Definition at line 51 of file [ButtonsConfig.hpp](#).

The documentation for this struct was generated from the following file:

- [ButtonsConfig.hpp](#)

6.10 Channel Class Reference

[Channel](#) class declaration.

```
#include <Channel.hpp>
```

Public Member Functions

- [Channel](#) (const std::string &channelID, const std::string &displayName, const std::list< std::string > &categories)
- std::string [getChannelID](#) () const
- std::string [getDisplayName](#) () const
- std::list< std::string > [getCategories](#) () const
- std::string [toString](#) () const

Private Attributes

- std::string [channelID](#)
- std::string [displayName](#)
- std::list< std::string > [categories](#)

6.10.1 Detailed Description

[Channel](#) class declaration.

Definition at line 17 of file [Channel.hpp](#).

6.10.2 Constructor & Destructor Documentation

6.10.2.1 Channel()

```
Channel::Channel (
    const std::string & channelID,
    const std::string & displayName,
    const std::list< std::string > & categories)
```

Constructs a [Channel](#).

Parameters

<i>channelID</i>	Unique YouTube channel identifier.
<i>displayName</i>	Human-readable channel name.
<i>categories</i>	List of content categories for this channel.

Definition at line 12 of file [Channel.cpp](#).

6.10.3 Member Function Documentation

6.10.3.1 `getCategories()`

```
std::list< std::string > Channel::getCategories () const
```

Returns

Content categories for this channel.

Definition at line 27 of file [Channel.cpp](#).

6.10.3.2 `getChannelID()`

```
std::string Channel::getChannelID () const
```

Returns

Unique YouTube channel identifier.

Definition at line 19 of file [Channel.cpp](#).

6.10.3.3 `getDisplayName()`

```
std::string Channel::getDisplayName () const
```

Returns

Human-readable channel name.

Definition at line 23 of file [Channel.cpp](#).

6.10.3.4 `toString()`

```
std::string Channel::toString () const
```

Definition at line 31 of file [Channel.cpp](#).

6.10.4 Member Data Documentation

6.10.4.1 categories

```
std::list<std::string> Channel::categories [private]
```

Definition at line 21 of file [Channel.hpp](#).

6.10.4.2 channelID

```
std::string Channel::channelID [private]
```

Definition at line 19 of file [Channel.hpp](#).

6.10.4.3 displayName

```
std::string Channel::displayName [private]
```

Definition at line 20 of file [Channel.hpp](#).

The documentation for this class was generated from the following files:

- [Channel.hpp](#)
- [Channel.cpp](#)

6.11 ChatRoom Class Reference

[ChatRoom](#) class declaration.

```
#include <ChatRoom.hpp>
```

Public Member Functions

- [ChatRoom](#) (const std::string &[blendID](#), const std::vector< std::string > &[participantIDs](#))
- std::string [getBlendID](#) () const
- std::vector< std::string > [getParticipantIDs](#) () const
- std::list< [Message](#) > [getMessages](#) () const
- void [addMessage](#) (const std::string &userID, const std::string &text)
- void [addMessage](#) (const [Message](#) &msg)
- bool [isParticipant](#) (const std::string &userID) const
- void [clearMessages](#) ()
- std::string [toString](#) () const

Private Attributes

- `std::string` `blendID`
ID of the owning [Blend](#).
- `std::vector< std::string >` `participantIDs`
userIDs pulled from the [Blend](#) on construction.
- `std::list< Message >` `messages`
In-memory history, oldest first.

6.11.1 Detailed Description

[ChatRoom](#) class declaration.

In-memory chat session associated with a [Blend](#).

Definition at line 28 of file [ChatRoom.hpp](#).

6.11.2 Constructor & Destructor Documentation

6.11.2.1 ChatRoom()

```
ChatRoom::ChatRoom (
    const std::string & blendID,
    const std::vector< std::string > & participantIDs)
```

Constructs a [ChatRoom](#) for a given blend.

Parameters

<i>blendID</i>	ID of the Blend this room belongs to.
<i>participantIDs</i>	userIDs of all blend participants.

Definition at line 13 of file [ChatRoom.cpp](#).

6.11.3 Member Function Documentation

6.11.3.1 addMessage() [1/2]

```
void ChatRoom::addMessage (
    const Message & msg)
```

Appends a pre-constructed [Message](#) (preserving its timestamp). Used when loading history from the server so original sent_at is kept.

Parameters

<i>msg</i>	The Message to append.
------------	--

Definition at line 37 of file [ChatRoom.cpp](#).

6.11.3.2 addMessage() [2/2]

```
void ChatRoom::addMessage (
    const std::string & userID,
    const std::string & text)
```

Appends a new message to the room's history.

Parameters

<i>userID</i>	ID of the sending user.
<i>text</i>	Message content.

Definition at line 33 of file [ChatRoom.cpp](#).

6.11.3.3 clearMessages()

```
void ChatRoom::clearMessages ()
```

Clears all messages from memory. Has no effect on persistence since persistence is not implemented yet.

Definition at line 46 of file [ChatRoom.cpp](#).

6.11.3.4 getBlendID()

```
std::string ChatRoom::getBlendID () const
```

Returns

ID of the [Blend](#) this room belongs to.

Definition at line 19 of file [ChatRoom.cpp](#).

6.11.3.5 getMessages()

```
std::list< Message > ChatRoom::getMessages () const
```

Returns

In-memory message history, oldest first.

Definition at line 27 of file [ChatRoom.cpp](#).

6.11.3.6 getParticipantIDs()

```
std::vector< std::string > ChatRoom::getParticipantIDs () const
```

Returns

List of participant userIDs.

Definition at line 23 of file [ChatRoom.cpp](#).

6.11.3.7 isParticipant()

```
bool ChatRoom::isParticipant (  
    const std::string & userID) const
```

Returns true if the given userID is a participant in this room. Used to validate that only blend participants can send messages.

Parameters

<i>userID</i>	ID to check.
---------------	--------------

Returns

true if the user is a participant.

Definition at line 41 of file [ChatRoom.cpp](#).

6.11.3.8 toString()

```
std::string ChatRoom::toString () const
```

Definition at line 50 of file [ChatRoom.cpp](#).

6.11.4 Member Data Documentation

6.11.4.1 blendID

```
std::string ChatRoom::blendID [private]
```

ID of the owning [Blend](#).

Definition at line 30 of file [ChatRoom.hpp](#).

6.11.4.2 messages

```
std::list<Message> ChatRoom::messages [private]
```

In-memory history, oldest first.

Definition at line 32 of file [ChatRoom.hpp](#).

6.11.4.3 participantIDs

```
std::vector<std::string> ChatRoom::participantIDs [private]
```

userIDs pulled from the [Blend](#) on construction.

Definition at line 31 of file [ChatRoom.hpp](#).

The documentation for this class was generated from the following files:

- [ChatRoom.hpp](#)
- [ChatRoom.cpp](#)

6.12 ChatWebSocket Class Reference

[ChatWebSocket](#) class declaration.

```
#include <ChatWebSocket.hpp>
```

Public Member Functions

- [ChatWebSocket](#) (const std::string url)
- [~ChatWebSocket](#) ()
- void [connect](#) ()
- void [setOnMessage](#) (std::function< void(const std::string &senderID, const std::string &content)> cb)
- void [set_blendID](#) (const std::string blendID)
- void [set_userID](#) (const std::string userID)
- std::string [get_fullUrl](#) ()
- void [send_message](#) (std::string msg)

Private Member Functions

- std::string [content_formatter](#) (std::string content)
- void [build_fullUrl](#) ()

Private Attributes

- ix::WebSocket [ws](#)
- std::string [url](#) = [kAppBackendWsPrefix](#)
- std::string [blendID](#)
- std::string [userID](#)
- std::string [fullUrl](#)
- bool [isStarted](#) = false
- std::function< void(const std::string &, const std::string &)> [m_onMessage](#)

6.12.1 Detailed Description

[ChatWebSocket](#) class declaration.

Definition at line 25 of file [ChatWebSocket.hpp](#).

6.12.2 Constructor & Destructor Documentation

6.12.2.1 ChatWebSocket()

```
ChatWebSocket::ChatWebSocket (  
    const std::string url)
```

Sets up WebSocket to perform specialized actions upon successful connection, message receiving, and errors. Given the WebSocket remains connected after starting up until its been told to stop listening for responses

Parameters

<i>url</i>	Is assigned to this->url
------------	--------------------------

Definition at line 38 of file [ChatWebSocket.cpp](#).

6.12.2.2 ~ChatWebSocket()

```
ChatWebSocket::~~ChatWebSocket ()
```

Stops the WebSocket from listening for responses.

Definition at line 68 of file [ChatWebSocket.cpp](#).

6.12.3 Member Function Documentation

6.12.3.1 build_fullUrl()

```
void ChatWebSocket::build_fullUrl () [private]
```

helper function that builds the full url path for the socket to send messages to

Definition at line 100 of file [ChatWebSocket.cpp](#).

6.12.3.2 connect()

```
void ChatWebSocket::connect ()
```

Opens the WebSocket connection without sending a message. Must be called after [set_blendID\(\)](#) and [set_userID\(\)](#) are set. No-op if already connected.

Definition at line 78 of file [ChatWebSocket.cpp](#).

6.12.3.3 content_formatter()

```
string ChatWebSocket::content_formatter (
    std::string content) [private]
```

Formats string into appropriate JSON structure as needed by the server.

Parameters

<i>content</i>	String to be formatted
----------------	------------------------

Returns

the structure JSON form of content ("{"content": <content>}")

Definition at line 96 of file [ChatWebSocket.cpp](#).

6.12.3.4 get_fullUrl()

```
string ChatWebSocket::get_fullUrl ()
```

Retrieves the full websocket url

Returns

string of currently configured websocket url

Definition at line 76 of file [ChatWebSocket.cpp](#).

6.12.3.5 send_message()

```
void ChatWebSocket::send_message (  
    std::string msg)
```

Sends a message through the WebSocket (given the url has been correctly configured with blend and user ID)

Parameters

<i>msg</i>	String message (leave as plain string, no formatting needed)
------------	--

Definition at line 91 of file [ChatWebSocket.cpp](#).

6.12.3.6 set_blendID()

```
void ChatWebSocket::set_blendID (  
    const std::string blendID)
```

Sets blendID

Parameters

<i>blendID</i>	the blend id to be set
----------------	------------------------

Definition at line 74 of file [ChatWebSocket.cpp](#).

6.12.3.7 set_userID()

```
void ChatWebSocket::set_userID (  
    const std::string userID)
```

Sets userID

Parameters

<i>userID</i>	the user id to be set
---------------	-----------------------

Definition at line 75 of file [ChatWebSocket.cpp](#).

6.12.3.8 setOnMessage()

```
void ChatWebSocket::setOnMessage (  
    std::function< void(const std::string &senderID, const std::string &content)> cb)
```

Registers a callback invoked (on the WebSocket background thread) whenever a chat message is received from the server.

Parameters

<i>cb</i>	Called with (senderID, messageContent).
-----------	---

Definition at line 86 of file [ChatWebSocket.cpp](#).

6.12.4 Member Data Documentation

6.12.4.1 blendID

```
std::string ChatWebSocket::blendID [private]
```

Definition at line 87 of file [ChatWebSocket.hpp](#).

6.12.4.2 fullUrl

```
std::string ChatWebSocket::fullUrl [private]
```

Definition at line 91 of file [ChatWebSocket.hpp](#).

6.12.4.3 isStarted

```
bool ChatWebSocket::isStarted = false [private]
```

Definition at line 93 of file [ChatWebSocket.hpp](#).

6.12.4.4 m_onMessage

```
std::function<void(const std::string&, const std::string&)> ChatWebSocket::m_onMessage [private]
```

Definition at line 95 of file [ChatWebSocket.hpp](#).

6.12.4.5 url

```
std::string ChatWebSocket::url = kAppBackendWsPrefix [private]
```

Definition at line 85 of file [ChatWebSocket.hpp](#).

6.12.4.6 userID

```
std::string ChatWebSocket::userID [private]
```

Definition at line 89 of file [ChatWebSocket.hpp](#).

6.12.4.7 ws

```
ix::WebSocket ChatWebSocket::ws [private]
```

Definition at line 83 of file [ChatWebSocket.hpp](#).

The documentation for this class was generated from the following files:

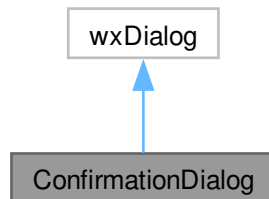
- [ChatWebSocket.hpp](#)
- [ChatWebSocket.cpp](#)

6.13 ConfirmationDialog Class Reference

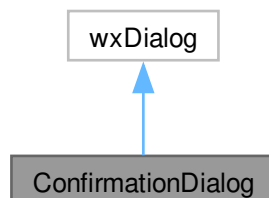
[ConfirmationDialog](#) class declaration.

```
#include <ConfirmationDialog.hpp>
```

Inheritance diagram for ConfirmationDialog:



Collaboration diagram for ConfirmationDialog:



Public Member Functions

- [ConfirmationDialog](#) (wxWindow *parent, const wxString &title, const wxString &message, const wxString &okLabel=wxT("OK"), const wxString &cancelLabel=wxT("Cancel"))

Static Public Member Functions

- static bool [Confirm](#) (wxWindow *parent, const wxString &title, const wxString &message)

6.13.1 Detailed Description

[ConfirmationDialog](#) class declaration.

Definition at line 43 of file [ConfirmationDialog.hpp](#).

6.13.2 Constructor & Destructor Documentation

6.13.2.1 ConfirmationDialog()

```
ConfirmationDialog::ConfirmationDialog (  
    wxWindow * parent,  
    const wxString & title,  
    const wxString & message,  
    const wxString & okLabel = wxT("OK"),  
    const wxString & cancelLabel = wxT("Cancel"))
```

Definition at line 18 of file [ConfirmationDialog.cpp](#).

6.13.3 Member Function Documentation

6.13.3.1 Confirm()

```
bool ConfirmationDialog::Confirm (  
    wxWindow * parent,  
    const wxString & title,  
    const wxString & message) [static]
```

Definition at line 107 of file [ConfirmationDialog.cpp](#).

The documentation for this class was generated from the following files:

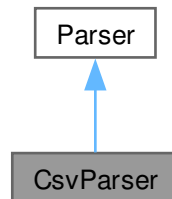
- [ConfirmationDialog.hpp](#)
- [ConfirmationDialog.cpp](#)

6.14 CsvParser Class Reference

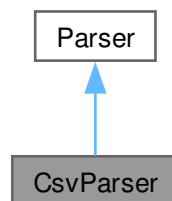
[CsvParser](#) class declaration.

```
#include <CsvParser.hpp>
```

Inheritance diagram for CsvParser:



Collaboration diagram for CsvParser:



Public Member Functions

- [CsvParser](#) (std::list< std::string > raw_data)
- std::list< std::unordered_map< std::string, std::string > > [parse](#) () override
- void [setData](#) (std::list< std::string > data) override
- int [getParseId](#) () override

Private Attributes

- std::list< std::string > [raw_data](#)
- int [parse_id](#)

6.14.1 Detailed Description

[CsvParser](#) class declaration.

CSV file parser implementing the [Parser](#) interface.

Definition at line 19 of file [CsvParser.hpp](#).

6.14.2 Constructor & Destructor Documentation

6.14.2.1 CsvParser() [1/2]

```
CsvParser::CsvParser ()
```

Definition at line 13 of file [CsvParser.cpp](#).

6.14.2.2 CsvParser() [2/2]

```
CsvParser::CsvParser (  
    std::list< std::string > raw_data)
```

Constructs a [CsvParser](#) pre-loaded with raw data.

Parameters

<i>raw_data</i>	Lines of raw CSV text to be parsed.
-----------------	-------------------------------------

6.14.2.3 ~CsvParser()

```
CsvParser::~~CsvParser ()
```

Definition at line 17 of file [CsvParser.cpp](#).

6.14.3 Member Function Documentation

6.14.3.1 getParsed()

```
int CsvParser::getParseId () [override], [virtual]
```

Returns

File_ID::CSV

Implements [Parser](#).

Definition at line 79 of file [CsvParser.cpp](#).

6.14.3.2 parse()

```
list< unordered_map< string, string > > CsvParser::parse () [override], [virtual]
```

Parses the loaded raw CSV data into a list of row dictionaries. The first row is used as column headers (keys).

Returns

List of maps from column name to cell value.

Implements [Parser](#).

Definition at line 19 of file [CsvParser.cpp](#).

6.14.3.3 setData()

```
void CsvParser::setData (
    std::list< std::string > data) [override], [virtual]
```

Replaces the raw data to be parsed on the next [parse\(\)](#) call.

Parameters

<i>data</i>	Lines of raw CSV text.
-------------	------------------------

Implements [Parser](#).

Definition at line 78 of file [CsvParser.cpp](#).

6.14.4 Member Data Documentation

6.14.4.1 parse_id

```
int CsvParser::parse_id [private]
```

Definition at line 47 of file [CsvParser.hpp](#).

6.14.4.2 raw_data

```
std::list<std::string> CsvParser::raw_data [private]
```

Definition at line 46 of file [CsvParser.hpp](#).

The documentation for this class was generated from the following files:

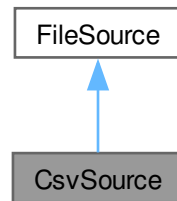
- [CsvParser.hpp](#)
- [CsvParser.cpp](#)

6.15 CsvSource Class Reference

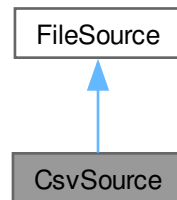
[CsvSource](#) class declaration.

```
#include <CsvSource.hpp>
```

Inheritance diagram for CsvSource:



Collaboration diagram for CsvSource:



Public Member Functions

- [CsvSource](#) (std::string src)
- std::list< std::string > [read](#) () override
- void [setSource](#) (std::string src) override
- int [getSourceId](#) () override

Private Attributes

- std::string [src](#)
- int [source_id](#)

6.15.1 Detailed Description

[CsvSource](#) class declaration.

Definition at line 18 of file [CsvSource.hpp](#).

6.15.2 Constructor & Destructor Documentation

6.15.2.1 CsvSource()

```
CsvSource::CsvSource (  
    std::string src)
```

Constructs a [CsvSource](#) configured for the given file path.

Parameters

<i>src</i>	Path to the CSV file to read.
------------	-------------------------------

Definition at line 11 of file [CsvSource.cpp](#).

6.15.3 Member Function Documentation

6.15.3.1 getSourceId()

```
int CsvSource::getSourceId () [override], [virtual]
```

Returns

File_ID::CSV

Implements [FileSource](#).

Definition at line 46 of file [CsvSource.cpp](#).

6.15.3.2 read()

```
list< string > CsvSource::read () [override], [virtual]
```

Reads the configured CSV file line-by-line.

Returns

List of raw text lines from the file.

Implements [FileSource](#).

Definition at line 15 of file [CsvSource.cpp](#).

6.15.3.3 setSource()

```
void CsvSource::setSource (  
    std::string src) [override], [virtual]
```

Changes the file path used by [read\(\)](#).

Parameters

<i>src</i>	New file path.
------------	----------------

Implements [FileSource](#).

Definition at line 45 of file [CsvSource.cpp](#).

6.15.4 Member Data Documentation

6.15.4.1 source_id

```
int CsvSource::source_id [private]
```

Definition at line 44 of file [CsvSource.hpp](#).

6.15.4.2 src

```
std::string CsvSource::src [private]
```

Definition at line 43 of file [CsvSource.hpp](#).

The documentation for this class was generated from the following files:

- [CsvSource.hpp](#)
- [CsvSource.cpp](#)

6.16 DataExtractor Class Reference

[DataExtractor](#) class declaration.

```
#include <DataExtractor.hpp>
```

Public Member Functions

- [DataExtractor](#) (std::unique_ptr< [FileSource](#) > src, std::unique_ptr< [Parser](#) > p)
- [~DataExtractor](#) ()
- std::list< std::unordered_map< std::string, std::string > > [extract](#) ()
- void [setSource](#) (std::unique_ptr< [FileSource](#) > src)
- void [setParser](#) (std::unique_ptr< [Parser](#) > p)

Private Attributes

- std::unique_ptr< [FileSource](#) > [src_format](#)
- std::unique_ptr< [Parser](#) > [par_format](#)

6.16.1 Detailed Description

[DataExtractor](#) class declaration.

Coordinates [FileSource](#) and [Parser](#) to extract and parse file data.

Definition at line 35 of file [DataExtractor.hpp](#).

6.16.2 Constructor & Destructor Documentation

6.16.2.1 DataExtractor()

```
DataExtractor::DataExtractor (
    std::unique_ptr< FileSource > src,
    std::unique_ptr< Parser > p)
```

Constructs a [DataExtractor](#) with [FileSource](#) and [Parser](#).

Parameters

<i>src</i>	Unique pointer to FileSource implementation (must be configured)
<i>p</i>	Unique pointer to Parser implementation (must be configured)

Note

[FileSource](#) and [Parser](#) must have matching format IDs

Definition at line 14 of file [DataExtractor.cpp](#).

6.16.2.2 ~DataExtractor()

```
DataExtractor::~DataExtractor ()
```

Destructor. Cleans up owned [FileSource](#) and [Parser](#) objects.

Definition at line 20 of file [DataExtractor.cpp](#).

6.16.3 Member Function Documentation

6.16.3.1 extract()

```
list< unordered_map< string, string > > DataExtractor::extract ()
```

Extracts and parses file data using configured [FileSource](#) and [Parser](#).

Reads raw data from [FileSource](#), validates format IDs match, feeds data to [Parser](#), and returns structured format.

Returns

List of dictionaries where each dict represents a data row

Exceptions

<i>Calls</i>	exit(1) if FileSource and Parser format IDs don't match
<i>Calls</i>	exit(1) if pointers are null or validation fails

Definition at line 22 of file [DataExtractor.cpp](#).

6.16.3.2 `setParser()`

```
void DataExtractor::setParser (
    std::unique_ptr< Parser > p)
```

Replaces the current [Parser](#) with a new one.

Parameters

<i>p</i>	Unique pointer to new Parser implementation
----------	---

Definition at line 60 of file [DataExtractor.cpp](#).

6.16.3.3 `setSource()`

```
void DataExtractor::setSource (
    std::unique_ptr< FileSource > src)
```

Replaces the current [FileSource](#) with a new one.

Parameters

<i>src</i>	Unique pointer to new FileSource implementation
------------	---

Definition at line 64 of file [DataExtractor.cpp](#).

6.16.4 Member Data Documentation

6.16.4.1 `par_format`

```
std::unique_ptr<Parser> DataExtractor::par_format [private]
```

Definition at line 70 of file [DataExtractor.hpp](#).

6.16.4.2 `src_format`

```
std::unique_ptr<FileSource> DataExtractor::src_format [private]
```

Definition at line 69 of file [DataExtractor.hpp](#).

The documentation for this class was generated from the following files:

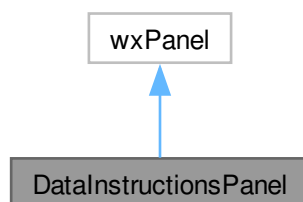
- [DataExtractor.hpp](#)
- [DataExtractor.cpp](#)

6.17 DataInstructionsPanel Class Reference

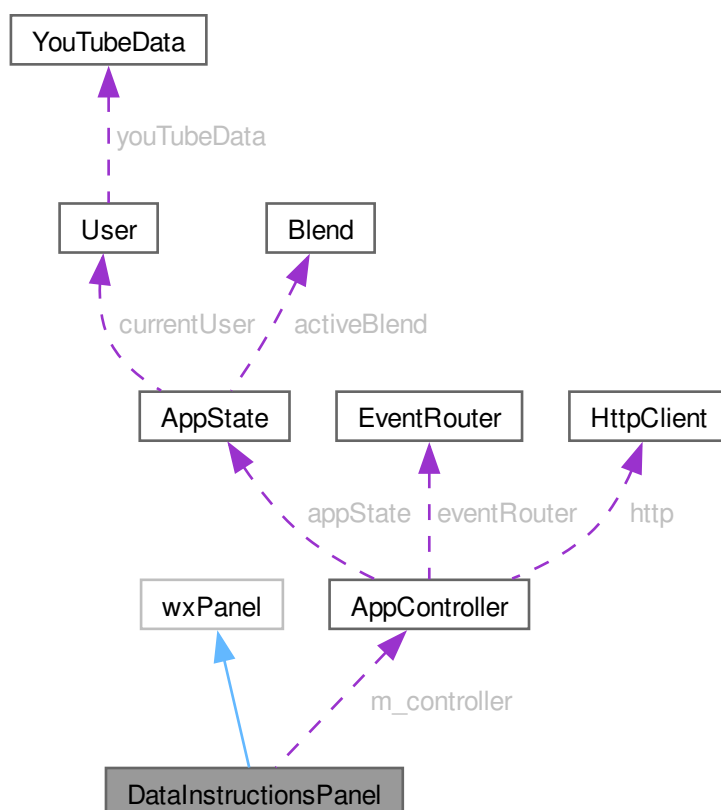
[DataInstructionsPanel](#) class declaration.

```
#include <DataInstructionsPanel.hpp>
```

Inheritance diagram for DataInstructionsPanel:



Collaboration diagram for DataInstructionsPanel:



Public Member Functions

- [DataInstructionsPanel](#) (wxWindow *parent, [AppController](#) &controller, [NavigateFn](#) nav)

Private Member Functions

- void [OnBrowse](#) (wxCommandEvent &)

Private Attributes

- [AppController](#) & m_controller
- [NavigateFn](#) m_nav

6.17.1 Detailed Description

[DataInstructionsPanel](#) class declaration.

Definition at line 50 of file [DataInstructionsPanel.hpp](#).

6.17.2 Constructor & Destructor Documentation

6.17.2.1 DataInstructionsPanel()

```
DataInstructionsPanel::DataInstructionsPanel (  
    wxWindow * parent,  
    AppController & controller,  
    NavigateFn nav)
```

Definition at line 24 of file [DataInstructionsPanel.cpp](#).

6.17.3 Member Function Documentation

6.17.3.1 OnBrowse()

```
void DataInstructionsPanel::OnBrowse (  
    wxCommandEvent & ) [private]
```

Definition at line 107 of file [DataInstructionsPanel.cpp](#).

6.17.4 Member Data Documentation

6.17.4.1 m_controller

```
AppController& DataInstructionsPanel::m_controller [private]
```

Definition at line 55 of file [DataInstructionsPanel.hpp](#).

6.17.4.2 m_nav

`NavigateFn` `DataInstructionsPanel::m_nav` [private]

Definition at line 56 of file [DataInstructionsPanel.hpp](#).

The documentation for this class was generated from the following files:

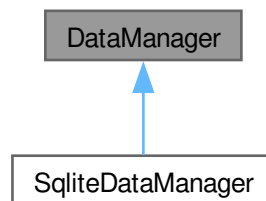
- [DataInstructionsPanel.hpp](#)
- [DataInstructionsPanel.cpp](#)

6.18 DataManager Class Reference

[DataManager](#) class declaration.

```
#include <DataManager.hpp>
```

Inheritance diagram for DataManager:



Public Member Functions

- virtual bool [createUser](#) (const [User](#) &user)=0
- virtual std::optional< [User](#) > [findUserByID](#) (const std::string &userID)=0
- virtual bool [validatePassword](#) (const std::string &userID, const std::string &password)=0
- virtual bool [saveWatchLater](#) (const std::string &userID, const std::list< [Video](#) > &videos)=0
- virtual std::list< [Video](#) > [loadWatchLater](#) (const std::string &userID)=0
- virtual bool [saveBlend](#) (const std::string &blendID, const std::string &creatorID, const std::string &algorithm, const std::vector< std::string > &participantIDs)=0
- virtual std::optional< std::string > [findBlendForUser](#) (const std::string &userID)=0
- virtual std::vector< std::string > [loadBlendParticipants](#) (const std::string &blendID)=0

6.18.1 Detailed Description

[DataManager](#) class declaration.

Definition at line 25 of file [DataManager.hpp](#).

6.18.2 Member Function Documentation

6.18.2.1 createUser()

```
virtual bool DataManager::createUser (
    const User & user) [pure virtual]
```

Persists a new user record.

Returns

true on success, false if the userID already exists or the insert otherwise fails.

Implemented in [SqliteDataManager](#).

6.18.2.2 findBlendForUser()

```
virtual std::optional< std::string > DataManager::findBlendForUser (
    const std::string & userID) [pure virtual]
```

Returns the blendID of the blend this user belongs to, if any.

Returns

The blendID string, or std::nullopt if the user has no blend.

Implemented in [SqliteDataManager](#).

6.18.2.3 findUserByID()

```
virtual std::optional< User > DataManager::findUserByID (
    const std::string & userID) [pure virtual]
```

Loads a user by their unique ID.

Returns

The [User](#) if found, std::nullopt otherwise.

Implemented in [SqliteDataManager](#).

6.18.2.4 loadBlendParticipants()

```
virtual std::vector< std::string > DataManager::loadBlendParticipants (
    const std::string & blendID) [pure virtual]
```

Returns all participant userIDs for a given blend.

Parameters

<i>blendID</i>	to be used for loading the respective participants
----------------	--

Returns

vector of strings representing userIDs

Implemented in [SqliteDataManager](#).

6.18.2.5 loadWatchLater()

```
virtual std::list< Video > DataManager::loadWatchLater (
    const std::string & userID) [pure virtual]
```

Loads the stored Watch Later video list for a user.

Returns

Empty list if no data has been saved for this user.

Implemented in [SqliteDataManager](#).

6.18.2.6 saveBlend()

```
virtual bool DataManager::saveBlend (
    const std::string & blendID,
    const std::string & creatorID,
    const std::string & algorithm,
    const std::vector< std::string > & participantIDs) [pure virtual]
```

Persists a blend and its participant list. Re-calling with the same blendID replaces the existing record.

Parameters

<i>creatorID</i>	userID of the user who initiated the blend.
<i>participantIDs</i>	All participant userIDs (including those without data).

Implemented in [SqliteDataManager](#).

6.18.2.7 saveWatchLater()

```
virtual bool DataManager::saveWatchLater (
    const std::string & userID,
    const std::list< Video > & videos) [pure virtual]
```

Persists a user's Watch Later video list, replacing any previously saved list.

Returns

true on success.

Implemented in [SqliteDataManager](#).

6.18.2.8 validatePassword()

```
virtual bool DataManager::validatePassword (
    const std::string & userID,
    const std::string & password) [pure virtual]
```

Checks whether the given plaintext password matches the stored record.

Returns

true if the userID exists and the password matches.

Implemented in [SqliteDataManager](#).

The documentation for this class was generated from the following file:

- [DataManager.hpp](#)

6.19 DataSaver Class Reference

[DataSaver](#) class declaration.

```
#include <DataSaver.hpp>
```

Public Member Functions

- [DataSaver](#) ([FileBin](#) b, [FileFormatter](#) ff)
- int [save](#) (std::list< std::unordered_map< std::string, std::string > > data)
- void [setFileBin](#) ([FileBin](#) b)
- void [setFileFormatter](#) ([FileFormatter](#) ff)

Private Attributes

- std::vector< std::unique_ptr< [FileBin](#) > > [bins](#)
Registered file output handlers.
- std::vector< std::unique_ptr< [FileFormatter](#) > > [formatters](#)
Registered data formatters.

6.19.1 Detailed Description

[DataSaver](#) class declaration.

Definition at line 22 of file [DataSaver.hpp](#).

6.19.2 Constructor & Destructor Documentation

6.19.2.1 DataSaver()

```
DataSaver::DataSaver (
    FileBin b,
    FileFormatter ff)
```

Constructs a [DataSaver](#) with an initial [FileBin](#) and [FileFormatter](#).

Parameters

<i>b</i>	The FileBin implementation for file output.
<i>ff</i>	The FileFormatter implementation for data formatting.

6.19.3 Member Function Documentation

6.19.3.1 save()

```
int DataSaver::save (
    std::list< std::unordered_map< std::string, std::string > > data)
```

Formats and writes data to file using the configured formatters and bins.

Parameters

<i>data</i>	List of row dictionaries to save.
-------------	-----------------------------------

Returns

0 on success, non-zero on failure.

6.19.3.2 setFileBin()

```
void DataSaver::setFileBin (
    FileBin b)
```

Adds or replaces the [FileBin](#) used for file output.

Parameters

<i>b</i>	New FileBin implementation.
----------	---

6.19.3.3 setFileFormatter()

```
void DataSaver::setFileFormatter (
    FileFormatter ff)
```

Adds or replaces the [FileFormatter](#) used for data formatting.

Parameters

<i>ff</i>	New FileFormatter implementation.
-----------	---

6.19.4 Member Data Documentation

6.19.4.1 bins

```
std::vector<std::unique_ptr<FileBin> > DataSaver::bins [private]
```

Registered file output handlers.

Definition at line 53 of file [DataSaver.hpp](#).

6.19.4.2 formatters

```
std::vector<std::unique_ptr<FileFormatter> > DataSaver::formatters [private]
```

Registered data formatters.

Definition at line 54 of file [DataSaver.hpp](#).

The documentation for this class was generated from the following file:

- [DataSaver.hpp](#)

6.20 EventRouter Class Reference

[EventRouter](#) class declaration.

```
#include <EventRouter.hpp>
```

Public Member Functions

- void [registerListener](#) (const std::string &eventName, [EventHandler](#) handler)
- void [dispatch](#) (const std::string &eventName, const [EventPayload](#) &payload={})
- void [deregisterAll](#) ()

Private Attributes

- std::map< std::string, [EventHandler](#) > [listeners](#)

6.20.1 Detailed Description

[EventRouter](#) class declaration.

Lightweight event-dispatch system connecting the UI layer to [AppController](#).

Definition at line 40 of file [EventRouter.hpp](#).

6.20.2 Constructor & Destructor Documentation

6.20.2.1 EventRouter()

```
EventRouter::EventRouter ()
```

Definition at line 10 of file [EventRouter.cpp](#).

6.20.3 Member Function Documentation

6.20.3.1 deregisterAll()

```
void EventRouter::deregisterAll ()
```

Removes all registered listeners. Called by [AppController](#)'s destructor during teardown.

Definition at line 25 of file [EventRouter.cpp](#).

6.20.3.2 dispatch()

```
void EventRouter::dispatch (  
    const std::string & eventName,  
    const EventPayload & payload = {})
```

Dispatches a named event to its registered handler. Logs a warning and does nothing if the event name is unrecognised.

Parameters

<i>eventName</i>	Name of the event to dispatch.
<i>payload</i>	Key-value data passed to the handler.

Definition at line 16 of file [EventRouter.cpp](#).

6.20.3.3 registerListener()

```
void EventRouter::registerListener (  
    const std::string & eventName,  
    EventHandler handler)
```

Registers a handler for the given event name. Replaces any existing handler registered under the same name.

Parameters

<i>eventName</i>	Unique name for this event.
<i>handler</i>	Callable invoked when the event is dispatched.

Definition at line 12 of file [EventRouter.cpp](#).

6.20.4 Member Data Documentation

6.20.4.1 listeners

```
std::map<std::string, EventHandler> EventRouter::listeners [private]
```

Definition at line 42 of file [EventRouter.hpp](#).

The documentation for this class was generated from the following files:

- [EventRouter.hpp](#)
- [EventRouter.cpp](#)

6.21 File_ID Class Reference

[File_ID](#) class declaration.

```
#include <File\_ID.hpp>
```

Static Public Attributes

- static constexpr int [CSV](#) = 1
- static constexpr int [HTML](#) = 2
- static constexpr int [JSON](#) = 3

6.21.1 Detailed Description

[File_ID](#) class declaration.

File format ID constants for validating source-parser compatibility.

Definition at line 16 of file [File_ID.hpp](#).

6.21.2 Member Data Documentation

6.21.2.1 CSV

```
int File\_ID::CSV = 1 [static], [constexpr]
```

Definition at line 18 of file [File_ID.hpp](#).

6.21.2.2 HTML

```
int File\_ID::HTML = 2 [static], [constexpr]
```

Definition at line 19 of file [File_ID.hpp](#).

6.21.2.3 JSON

```
int File_ID::JSON = 3 [static], [constexpr]
```

Definition at line 20 of file [File_ID.hpp](#).

The documentation for this class was generated from the following file:

- [File_ID.hpp](#)

6.22 FileBin Class Reference

[FileBin](#) class declaration.

```
#include <FileBin.hpp>
```

Public Member Functions

- virtual [~FileBin](#) ()=default
- virtual int [write](#) ()=0
- virtual void [setOutputFile](#) (std::string output)=0
- virtual void [setRawData](#) (std::string raw)=0
- virtual void [setFileName](#) (std::string fn)=0

6.22.1 Detailed Description

[FileBin](#) class declaration.

Abstract interface for writing data to files.

Definition at line 22 of file [FileBin.hpp](#).

6.22.2 Constructor & Destructor Documentation

6.22.2.1 ~FileBin()

```
virtual FileBin::~~FileBin () [virtual], [default]
```

Virtual destructor for proper cleanup in derived classes.

6.22.3 Member Function Documentation

6.22.3.1 setFileName()

```
virtual void FileBin::setFileName (  
    std::string fn) [pure virtual]
```

Sets the output file name (without directory path).

Parameters

<i>fn</i>	File name for output
-----------	----------------------

6.22.3.2 setOutputFile()

```
virtual void FileBin::setOutputFile (  
    std::string output) [pure virtual]
```

Sets the output file directory path for [write\(\)](#).

Parameters

<i>output</i>	Directory path where file will be written
---------------	---

6.22.3.3 setRawData()

```
virtual void FileBin::setRawData (  
    std::string raw) [pure virtual]
```

Sets the raw data string to be written to file.

Parameters

<i>raw</i>	Formatted data string to write
------------	--------------------------------

6.22.3.4 write()

```
virtual int FileBin::write () [pure virtual]
```

Writes configured data to the specified output file.

Creates or overwrites the file at the configured output path and writes the configured raw data to it.

Returns

Status code (0 on success, non-zero on failure)

The documentation for this class was generated from the following file:

- [FileBin.hpp](#)

6.23 FileFormatter Class Reference

[FileFormatter](#) class declaration.

```
#include <FileFormatter.hpp>
```

Public Member Functions

- virtual [~FileFormatter](#) ()=default
- virtual std::string [format](#) ()=0
- virtual void [setData](#) (std::list< std::unordered_map< std::string, std::string > > data)=0

6.23.1 Detailed Description

[FileFormatter](#) class declaration.

Definition at line 23 of file [FileFormatter.hpp](#).

6.23.2 Constructor & Destructor Documentation

6.23.2.1 ~FileFormatter()

```
virtual FileFormatter::~~FileFormatter () [virtual], [default]
```

Virtual destructor for proper cleanup in derived classes.

6.23.3 Member Function Documentation

6.23.3.1 format()

```
virtual std::string FileFormatter::format () [pure virtual]
```

Formats structured data into format-specific string representation.

Converts list<Dict<str,str>> configured via [setData\(\)](#) into a properly formatted string for writing to file (e.g., CSV with commas and newlines).

Returns

Formatted string ready for file output

6.23.3.2 setData()

```
virtual void FileFormatter::setData (
    std::list< std::unordered_map< std::string, std::string > > data) [pure virtual]
```

Sets the structured data to be formatted by [format\(\)](#).

Parameters

<i>data</i>	List of dictionaries representing data rows
-------------	---

The documentation for this class was generated from the following file:

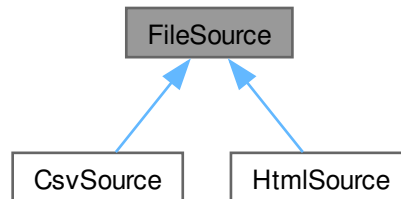
- [FileFormatter.hpp](#)

6.24 FileSource Class Reference

[FileSource](#) class declaration.

```
#include <FileSource.hpp>
```

Inheritance diagram for FileSource:



Public Member Functions

- virtual `std::list< std::string > read ()=0`
- virtual void `setSource (std::string src)=0`
- virtual int `getSourceId ()=0`

6.24.1 Detailed Description

[FileSource](#) class declaration.

Abstract interface for reading file source data.

Definition at line 22 of file [FileSource.hpp](#).

6.24.2 Member Function Documentation

6.24.2.1 getSourceId()

```
virtual int FileSource::getSourceId () [pure virtual]
```

Gets the file format ID associated with this source.

Returns

Integer ID representing the file format (e.g., `File_ID::CSV`)

Implemented in [CsvSource](#), and [HtmlSource](#).

6.24.2.2 read()

```
virtual std::list< std::string > FileSource::read () [pure virtual]
```

Reads the configured file source path and returns raw file data.

Returns

List of strings containing raw file data (line-by-line or complete content)

Implemented in [CsvSource](#), and [HtmlSource](#).

6.24.2.3 setSource()

```
virtual void FileSource::setSource (
    std::string src) [pure virtual]
```

Sets the file source path for [read\(\)](#) to use.

Parameters

<i>src</i>	File path to configure as the source for reading
------------	--

Implemented in [CsvSource](#).

The documentation for this class was generated from the following file:

- [FileSource.hpp](#)

6.25 Friend Class Reference

[Friend](#) class declaration.

```
#include <Friend.hpp>
```

Public Member Functions

- [Friend](#) (const std::string &userID, const std::string &displayName, const std::string &email)
- std::string [getUserID](#) () const
- std::string [getDisplayName](#) () const
- std::string [getEmail](#) () const
- void [setDisplayName](#) (const std::string &displayName)
- void [setEmail](#) (const std::string &email)
- std::string [toString](#) () const

Private Attributes

- `std::string` [userID](#)
- `std::string` [displayName](#)
- `std::string` [email](#)

6.25.1 Detailed Description

[Friend](#) class declaration.

A friend or contact entry stored on a [User](#)'s friends list.

Definition at line 17 of file [Friend.hpp](#).

6.25.2 Constructor & Destructor Documentation

6.25.2.1 Friend()

```
Friend::Friend (
    const std::string & userID,
    const std::string & displayName,
    const std::string & email)
```

Constructs a [Friend](#) entry.

Parameters

<i>userID</i>	Unique identifier of the friend's account.
<i>displayName</i>	Display name shown in the UI.
<i>email</i>	Email address of the friend.

Definition at line 11 of file [Friend.cpp](#).

6.25.3 Member Function Documentation

6.25.3.1 getDisplayName()

```
std::string Friend::getDisplayName () const
```

Returns

Display name of the friend.

Definition at line 22 of file [Friend.cpp](#).

6.25.3.2 getEmail()

```
std::string Friend::getEmail () const
```

Returns

Email address of the friend.

Definition at line 26 of file [Friend.cpp](#).

6.25.3.3 getUserID()

```
std::string Friend::getUserID () const
```

Returns

Unique identifier of the friend's account.

Definition at line 18 of file [Friend.cpp](#).

6.25.3.4 setDisplayName()

```
void Friend::setDisplayName (  
    const std::string & displayName)
```

Parameters

<i>displayName</i>	New display name.
--------------------	-------------------

Definition at line 32 of file [Friend.cpp](#).

6.25.3.5 setEmail()

```
void Friend::setEmail (  
    const std::string & email)
```

Parameters

<i>email</i>	New email address.
--------------	--------------------

Definition at line 36 of file [Friend.cpp](#).

6.25.3.6 toString()

```
std::string Friend::toString () const
```

String representation of [Friend](#)

Returns

string

Definition at line 40 of file [Friend.cpp](#).

6.25.4 Member Data Documentation

6.25.4.1 displayName

```
std::string Friend::displayName [private]
```

Definition at line 20 of file [Friend.hpp](#).

6.25.4.2 email

```
std::string Friend::email [private]
```

Definition at line 21 of file [Friend.hpp](#).

6.25.4.3 userID

```
std::string Friend::userID [private]
```

Definition at line 19 of file [Friend.hpp](#).

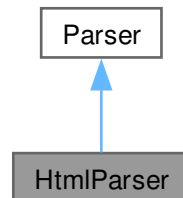
The documentation for this class was generated from the following files:

- [Friend.hpp](#)
- [Friend.cpp](#)

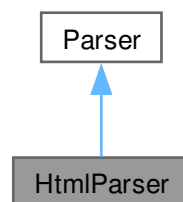
6.26 HtmlParser Class Reference

HTML file parser implementation using Gumbo.

Inheritance diagram for HtmlParser:



Collaboration diagram for HtmlParser:



Public Member Functions

- [HtmlParser](#) (list< string > raw_data)
- list< unordered_map< string, string > > [parse](#) ()
- void [setData](#) (list< string > data) override
- int [getParseId](#) () override

Public Member Functions inherited from [Parser](#)

- virtual void [setData](#) (std::list< std::string > data)=0

Private Member Functions

- void [extract_links](#) (GumboNode *node, list< unordered_map< string, string > > &out)
- string [get_text](#) (GumboNode *node)

Private Attributes

- list< string > [raw_data](#) = {}
- int const [parse_id](#) = File_ID::HTML

6.26.1 Detailed Description

HTML file parser implementation using Gumbo.

Definition at line 16 of file [HtmlParser.cpp](#).

6.26.2 Constructor & Destructor Documentation

6.26.2.1 HtmlParser() [1/2]

```
HtmlParser::HtmlParser (  
    list< string > raw_data) [inline]
```

Definition at line 18 of file [HtmlParser.cpp](#).

6.26.2.2 HtmlParser() [2/2]

```
HtmlParser::HtmlParser () [inline]
```

Definition at line 19 of file [HtmlParser.cpp](#).

6.26.3 Member Function Documentation

6.26.3.1 extract_links()

```
void HtmlParser::extract_links (  
    GumboNode * node,  
    list< unordered_map< string, string > > & out) [inline], [private]
```

Definition at line 56 of file [HtmlParser.cpp](#).

6.26.3.2 get_text()

```
string HtmlParser::get_text (  
    GumboNode * node) [inline], [private]
```

Definition at line 86 of file [HtmlParser.cpp](#).

6.26.3.3 getParseId()

```
int HtmlParser::getParseId () [inline], [override], [virtual]
```

Gets the file format ID associated with this parser.

Returns

Integer ID representing the file format (e.g., File_ID::CSV)

Implements [Parser](#).

Definition at line 48 of file [HtmlParser.cpp](#).

6.26.3.4 parse()

```
list< unordered_map< string, string > > HtmlParser::parse () [inline], [virtual]
```

Parses raw file data into a structured list of dictionaries.

Converts raw file data configured via setData() into a list of unordered string-to-string dictionaries. Each dictionary represents a row with column titles as keys and row contents as values.

Returns

List of unordered maps representing parsed file rows

Implements [Parser](#).

Definition at line 22 of file [HtmlParser.cpp](#).

6.26.3.5 setData()

```
void HtmlParser::setData (
    list< string > data) [inline], [override]
```

Definition at line 46 of file [HtmlParser.cpp](#).

6.26.4 Member Data Documentation

6.26.4.1 parse_id

```
int const HtmlParser::parse_id = File_ID::HTML [private]
```

Definition at line 53 of file [HtmlParser.cpp](#).

6.26.4.2 raw_data

```
list<string> HtmlParser::raw_data = {} [private]
```

Definition at line 52 of file [HtmlParser.cpp](#).

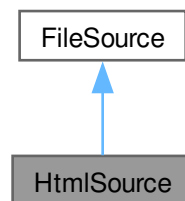
The documentation for this class was generated from the following file:

- [HtmlParser.cpp](#)

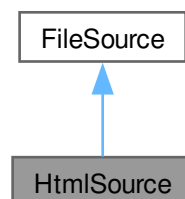
6.27 HtmlSource Class Reference

HTML file source/reader implementation.

Inheritance diagram for HtmlSource:



Collaboration diagram for HtmlSource:



Public Member Functions

- [HtmlSource](#) (std::string s)
- list< string > [read](#) () override
- int [getSourceId](#) ()
- void [setSource](#) (string new_source)

Public Member Functions inherited from [FileSource](#)

- virtual void [setSource](#) (std::string src)=0

Private Attributes

- string [src_path](#) = ""
- int const [source_id](#) = File_ID::HTML

6.27.1 Detailed Description

HTML file source/reader implementation.

Definition at line 16 of file [HtmlSource.cpp](#).

6.27.2 Constructor & Destructor Documentation

6.27.2.1 HtmlSource()

```
HtmlSource::HtmlSource (
    std::string s) [inline]
```

Definition at line 18 of file [HtmlSource.cpp](#).

6.27.3 Member Function Documentation

6.27.3.1 getSourceId()

```
int HtmlSource::getSourceId () [inline], [virtual]
```

Gets the file format ID associated with this source.

Returns

Integer ID representing the file format (e.g., File_ID::CSV)

Implements [FileSource](#).

Definition at line 42 of file [HtmlSource.cpp](#).

6.27.3.2 read()

```
list< string > HtmlSource::read () [inline], [override], [virtual]
```

Reads the configured file source path and returns raw file data.

Returns

List of strings containing raw file data (line-by-line or complete content)

Implements [FileSource](#).

Definition at line 21 of file [HtmlSource.cpp](#).

6.27.3.3 setSource()

```
void HtmlSource::setSource (
    string new_source) [inline]
```

Definition at line 43 of file [HtmlSource.cpp](#).

6.27.4 Member Data Documentation

6.27.4.1 source_id

```
int const HtmlSource::source_id = File_ID::HTML [private]
```

Definition at line 47 of file [HtmlSource.cpp](#).

6.27.4.2 src_path

```
string HtmlSource::src_path = " " [private]
```

Definition at line 46 of file [HtmlSource.cpp](#).

The documentation for this class was generated from the following file:

- [HtmlSource.cpp](#)

6.28 HttpClient Class Reference

[HttpClient](#) class declaration.

```
#include <HttpClient.hpp>
```

Public Member Functions

- [HttpClient](#) (std::string base)
- long [getLastStatusCode](#) () const
Returns the HTTP status code from the most recent successful libcurl transfer initiated by this client instance.
- bool [wasLastRequestSuccessful](#) (const std::string &method) const
Convenience helper that applies [isRequestSuccessful\(\)](#) to the latest response code captured by this client.
- std::string [get](#) (const std::string &path)
- std::string [post](#) (const std::string &path, const std::string &json)
- std::string [del](#) (const std::string &path, const std::string &json)
- std::string [build_watch_later_endpoint](#) (std::string userID)
Builds the watch-later endpoint for a specific user.
- std::string [build_latest_blend_endpoint](#) (std::string userID)
Builds the endpoint used to fetch a user's latest blend.
- std::string [build_blend_participant_endpoint](#) (std::string blendID)
Builds the endpoint used to fetch participant IDs for a blend.
- std::string [build_chatroom_detail_endpoint](#) (std::string blendID)
Builds the endpoint used to fetch chat room details for a blend.
- std::string [build_auth_user_lookup_endpoint](#) (std::string userID)
Builds the auth user lookup endpoint for a specific user.
- std::string [build_delete_user_endpoint](#) (std::string userID)
Builds the account-delete endpoint for a specific user.
- std::string [build_user_blends_endpoint](#) (std::string userID)
- std::string [build_leave_blend_endpoint](#) (std::string blendID)
- std::string [build_chat_history_endpoint](#) (std::string blendID, std::string userID)

Static Public Member Functions

- static bool [isRequestSuccessful](#) (const std::string &method, long statusCode)
Evaluates whether an HTTP status code should be treated as a successful result for a specific request method.

Public Attributes

- const std::string [P](#) = "POST"
- const std::string [G](#) = "GET"
- const std::string [D](#) = "DELETE"
- const std::string [REG_USER](#) = "/api/v1/auth/register"
- const std::string [LOGIN](#) = "/api/v1/auth/login"
- const std::string [AUTH_USER_PREFIX](#) = "/api/v1/auth/users/"
- const std::string [DELETE_USER_PREFIX](#) = "/api/v1/auth/users/"
- const std::string [BLEND](#) = "/api/v1/blends"
- const int [SUCC](#) = 200
- const int [ALT_SUCC](#) = 201
- const int [MAL_REQ_ERR](#) = 400
- const int [INV_USR_LOG_ERR](#) = 401
- const int [UNAUTH_RES_REQ_ERR](#) = 403
- const int [MISS_RM_CNTXT](#) = 404
- const int [DUPLICATE](#) = 409
- const int [SERVER_ERR](#) = 500

Private Member Functions

- std::string [request](#) (const std::string &method, const std::string &path, const std::string &body)

Static Private Member Functions

- static size_t [write_callback](#) (void *contents, size_t size, size_t nmem, void *out)

Private Attributes

- std::string [baseUrl](#)
- const std::string [hdr](#) = "Content-Type: application/json"
- long [lastStatusCode](#) = 0

6.28.1 Detailed Description

[HttpClient](#) class declaration.

Definition at line 22 of file [HttpClient.hpp](#).

6.28.2 Constructor & Destructor Documentation

6.28.2.1 HttpClient()

```
HttpClient::HttpClient (
    std::string base)
```

Basic constructor. Assigns base url to this instance

Parameters

<i>base</i>	string that will be saved to this instance
-------------	--

Definition at line 49 of file [HttpClient.cpp](#).

6.28.3 Member Function Documentation

6.28.3.1 build_auth_user_lookup_endpoint()

```
string HttpClient::build_auth_user_lookup_endpoint (  
    std::string userID)
```

Builds the auth user lookup endpoint for a specific user.

Result format: /api/v1/auth/users/{userID}

This endpoint is the canonical account-existence lookup in the frontend/backend contract and should be used instead of feature routes like watch-later when validating participant IDs.

Parameters

<i>userID</i>	User identifier used in the endpoint path.
---------------	--

Returns

Endpoint path string for user profile lookup.

Definition at line 103 of file [HttpClient.cpp](#).

6.28.3.2 build_blend_participant_endpoint()

```
string HttpClient::build_blend_participant_endpoint (  
    std::string blendID)
```

Builds the endpoint used to fetch participant IDs for a blend.

Result format: /api/v1/blends/{blendID}/participants

Parameters

<i>blendID</i>	Blend identifier used in the endpoint path.
----------------	---

Returns

Endpoint path string for participant lookup.

Definition at line 95 of file [HttpClient.cpp](#).

6.28.3.3 build_chat_history_endpoint()

```
string HttpClient::build_chat_history_endpoint (  
    std::string blendID,  
    std::string userID)
```

Builds the endpoint for fetching a blend's chat message history.

Parameters

<i>blendID</i>	ID of the blend whose chat history is requested.
<i>userID</i>	ID of the requesting user (passed as query param).

Returns

the structured string endpoint

Definition at line 192 of file [HttpClient.cpp](#).

6.28.3.4 build_chatroom_detail_endpoint()

```
string HttpClient::build_chatroom_detail_endpoint (  
    std::string blendID)
```

Builds the endpoint used to fetch chat room details for a blend.

Result format: `/api/v1/blends/{blendID}/chatroom`

Parameters

<i>blendID</i>	Blend identifier used as the chatroom context.
----------------	--

Returns

Endpoint path string for chatroom details.

Definition at line 99 of file [HttpClient.cpp](#).

6.28.3.5 build_delete_user_endpoint()

```
string HttpClient::build_delete_user_endpoint (  
    std::string userID)
```

Builds the account-delete endpoint for a specific user.

Result format: `/api/v1/auth/users/{userID}`

Parameters

<i>userID</i>	User identifier used in the endpoint path.
---------------	--

Returns

Endpoint path string for account deletion.

Definition at line 107 of file [HttpClient.cpp](#).

6.28.3.6 build_latest_blend_endpoint()

```
string HttpClient::build_latest_blend_endpoint (  
    std::string userID)
```

Builds the endpoint used to fetch a user's latest blend.

Result format: `/api/v1/users/{userID}/blend`

Parameters

<i>userID</i>	User identifier used in the endpoint path.
---------------	--

Returns

Endpoint path string for latest blend lookup.

Definition at line 91 of file [HttpClient.cpp](#).

6.28.3.7 build_leave_blend_endpoint()

```
string HttpClient::build_leave_blend_endpoint (  
    std::string blendID)
```

Builds the endpoint for leaving a blend. Result format: `/api/v1/blends/{blendID}/leave`

Parameters

<i>blendID</i>	Blend identifier.
----------------	-------------------

Returns

Endpoint path string for leaving a blend.

Definition at line 188 of file [HttpClient.cpp](#).

6.28.3.8 build_user_blends_endpoint()

```
string HttpClient::build_user_blends_endpoint (  
    std::string userID)
```

Builds the endpoint for fetching all blends a user participates in. Result format: `/api/v1/users/{userID}/blends`

Parameters

<i>userID</i>	User identifier.
---------------	------------------

Returns

Endpoint path string for listing user blends.

Definition at line 184 of file [HttpClient.cpp](#).

6.28.3.9 build_watch_later_endpoint()

```
string HttpClient::build_watch_later_endpoint (  
    std::string userID)
```

Builds the watch-later endpoint for a specific user.

Result format: `/api/v1/users/{userID}/watch-later`

Parameters

<i>userID</i>	User identifier used in the endpoint path.
---------------	--

Returns

Endpoint path string for persisting or retrieving watch-later data.

Definition at line 87 of file [HttpClient.cpp](#).

6.28.3.10 del()

```
string HttpClient::del (  
    const std::string & path,  
    const std::string & json)
```

Public facing delete method. Fires HTTP DELETE message to the server.

Parameters

<i>path</i>	endpoint string pointer
<i>json</i>	formatted json string pointer message to be sent

Returns

server response body

Definition at line 83 of file [HttpClient.cpp](#).

6.28.3.11 get()

```
string HttpClient::get (  
    const std::string & path)
```

Public facing get method. Acquires the formatted string based on the given path endpoint. Helps simplify pinging the server for data by providing the

Parameters

<i>path</i>	string endpoint path pointer the formatted string is for
-------------	--

Returns

formatted json string

Definition at line 75 of file [HttpClient.cpp](#).

6.28.3.12 getLastStatusCode()

```
long HttpClient::getLastStatusCode () const
```

Returns the HTTP status code from the most recent successful libcurl transfer initiated by this client instance.

This value is updated by `request()` after `curl_easy_perform()` and is available to callers who need post-call diagnostics.

Returns

Last response status code, or 0 before any request has run.

Definition at line 67 of file [HttpClient.cpp](#).

6.28.3.13 isRequestSuccessful()

```
bool HttpClient::isRequestSuccessful (
    const std::string & method,
    long statusCode) [static]
```

Evaluates whether an HTTP status code should be treated as a successful result for a specific request method.

This is the central error-code handler for [HttpClient](#). Use it whenever you need to decide if a completed request succeeded semantically, not just transport-wise.

Success rules are intentionally aligned with the frontend/backend contract in [Docs/Front_End_Requirements_for_Server_Communication.md](#):

- GET is successful only for 200 OK.
- POST is successful for 200 OK or 201 Created.
- All other status codes are considered failures.

Parameters

<i>method</i>	HTTP verb string ("GET" or "POST").
<i>statusCode</i>	Numeric HTTP status code returned by the server.

Returns

`true` when the code represents success for the given method; `false` otherwise.

Exceptions

<code>std::invalid_argument</code>	if the method is unsupported.
------------------------------------	-------------------------------

Definition at line 53 of file [HttpClient.cpp](#).

6.28.3.14 post()

```
string HttpClient::post (
    const std::string & path,
    const std::string & json)
```

Public facing post method. fires HTTP message to the server.

Parameters

<i>path</i>	the endpoint string pointer
<i>json</i>	formatted json string pointer message to be sent

Returns

Definition at line 79 of file [HttpClient.cpp](#).

6.28.3.15 request()

```
string HttpClient::request (  
    const std::string & method,  
    const std::string & path,  
    const std::string & body) [private]
```

Helper function. Does the grunt work of retrieving and posting data to the server

Parameters

<i>method</i>	Determines whether or not we POST or GET
<i>path</i>	determine the endpoint we need to use
<i>body</i>	the JSON body that needs to be POSTed. leave empty when GETting

Returns

the response from either operation

just have this here for completeness libcurl will by default request through GET. but with the above cmd, libcurl will switch to POST automatically

within the future can add extra blocks for other request methods

Definition at line 111 of file [HttpClient.cpp](#).

6.28.3.16 wasLastRequestSuccessful()

```
bool HttpClient::wasLastRequestSuccessful (  
    const std::string & method) const
```

Convenience helper that applies [isRequestSuccessful\(\)](#) to the latest response code captured by this client.

Parameters

<i>method</i>	HTTP verb to evaluate ("GET" or "POST").
---------------	--

Returns

true when the latest response code is successful for method.

Exceptions

<code>std::invalid_argument</code>	if the method is unsupported.
------------------------------------	-------------------------------

Definition at line 71 of file [HttpClient.cpp](#).

6.28.3.17 write_callback()

```
size_t HttpClient::write_callback (
    void * contents,
    size_t size,
    size_t nmemb,
    void * out) [static], [private]
```

Part of the libcurl design pattern. Used to determine how streamed data should be handled from the request response. (libcurl does not handle how to deal with streamed data)

Parameters

<i>contents</i>	Is the pointer to a chunk of raw bytes from the server
<i>size</i>	Describes how long the chunk is
<i>nmemb</i>	Aids in describing the size of the chunk (like typical malloc patterns)
<i>out</i>	A pointer to a string we would like to append the chunks to (in this case)

Returns

(size * nmemb) as part of the libcurl pattern to verify data handling from callback

Definition at line 196 of file [HttpClient.cpp](#).

6.28.4 Member Data Documentation

6.28.4.1 ALT_SUCC

```
const int HttpClient::ALT_SUCC = 201
```

Definition at line 47 of file [HttpClient.hpp](#).

6.28.4.2 AUTH_USER_PREFIX

```
const std::string HttpClient::AUTH_USER_PREFIX = "/api/v1/auth/users/"
```

Definition at line 38 of file [HttpClient.hpp](#).

6.28.4.3 baseUrl

```
std::string HttpClient::baseUrl [private]
```

Definition at line 220 of file [HttpClient.hpp](#).

6.28.4.4 BLEND

```
const std::string HttpClient::BLEND = "/api/v1/blends"
```

Definition at line 42 of file [HttpClient.hpp](#).

6.28.4.5 D

```
const std::string HttpClient::D = "DELETE"
```

Definition at line 29 of file [HttpClient.hpp](#).

6.28.4.6 DELETE_USER_PREFIX

```
const std::string HttpClient::DELETE_USER_PREFIX = "/api/v1/auth/users/"
```

Definition at line 40 of file [HttpClient.hpp](#).

6.28.4.7 DUPLICATE

```
const int HttpClient::DUPLICATE = 409
```

Definition at line 52 of file [HttpClient.hpp](#).

6.28.4.8 G

```
const std::string HttpClient::G = "GET"
```

Definition at line 27 of file [HttpClient.hpp](#).

6.28.4.9 hdr

```
const std::string HttpClient::hdr = "Content-Type: application/json" [private]
```

Definition at line 223 of file [HttpClient.hpp](#).

6.28.4.10 INV_USR_LOG_ERR

```
const int HttpClient::INV_USR_LOG_ERR = 401
```

Definition at line 49 of file [HttpClient.hpp](#).

6.28.4.11 lastStatusCode

```
long HttpClient::lastStatusCode = 0 [private]
```

Definition at line 226 of file [HttpClient.hpp](#).

6.28.4.12 LOGIN

```
const std::string HttpClient::LOGIN = "/api/v1/auth/login"
```

Definition at line 36 of file [HttpClient.hpp](#).

6.28.4.13 MAL_REQ_ERR

```
const int HttpClient::MAL_REQ_ERR = 400
```

Definition at line 48 of file [HttpClient.hpp](#).

6.28.4.14 MISS_RM_CNTXT

```
const int HttpClient::MISS_RM_CNTXT = 404
```

Definition at line 51 of file [HttpClient.hpp](#).

6.28.4.15 P

```
const std::string HttpClient::P = "POST"
```

Definition at line 25 of file [HttpClient.hpp](#).

6.28.4.16 REG_USER

```
const std::string HttpClient::REG_USER = "/api/v1/auth/register"
```

Definition at line 34 of file [HttpClient.hpp](#).

6.28.4.17 SERVER_ERR

```
const int HttpClient::SERVER_ERR = 500
```

Definition at line 53 of file [HttpClient.hpp](#).

6.28.4.18 SUCC

```
const int HttpClient::SUCC = 200
```

Definition at line 46 of file [HttpClient.hpp](#).

6.28.4.19 UNAUTH_RES_REQ_ERR

```
const int HttpClient::UNAUTH_RES_REQ_ERR = 403
```

Definition at line 50 of file [HttpClient.hpp](#).

The documentation for this class was generated from the following files:

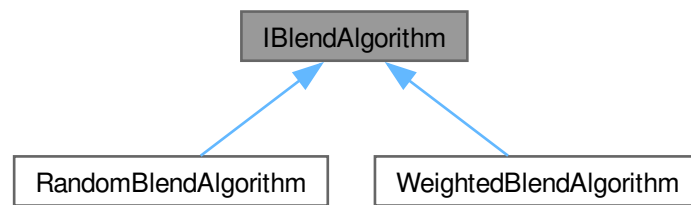
- [HttpClient.hpp](#)
- [HttpClient.cpp](#)

6.29 IBlendAlgorithm Class Reference

[IBlendAlgorithm](#) class declaration.

```
#include <IBlendAlgorithm.hpp>
```

Inheritance diagram for IBlendAlgorithm:



Public Member Functions

- virtual [Blend](#) [generateBlend](#) (const std::list< [User](#) > &participants, const std::string &title="")=0

6.29.1 Detailed Description

[IBlendAlgorithm](#) class declaration.

Abstract interface for blend-generation algorithms.

Definition at line 23 of file [IBlendAlgorithm.hpp](#).

6.29.2 Member Function Documentation

6.29.2.1 generateBlend()

```
virtual Blend IBlendAlgorithm::generateBlend (
    const std::list< User > & participants,
    const std::string & title = "") [pure virtual]
```

Generates a [Blend](#) from a collection of participants.

Parameters

<i>participants</i>	List of users whose Watch Later data is used as input.
<i>title</i>	User-facing name for the blend (default: empty).

Returns

A new [Blend](#) containing the algorithm's chosen video selection.

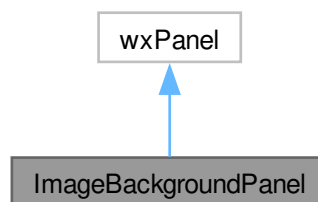
Implemented in [RandomBlendAlgorithm](#), and [WeightedBlendAlgorithm](#).

The documentation for this class was generated from the following file:

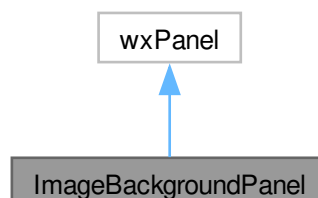
- [IBlendAlgorithm.hpp](#)

6.30 ImageBackgroundPanel Class Reference

Inheritance diagram for ImageBackgroundPanel:



Collaboration diagram for ImageBackgroundPanel:



Public Member Functions

- [ImageBackgroundPanel](#) (wxWindow *parent, const std::unordered_map< int, wxImage > &imageStore, int key)

Private Member Functions

- void [OnPaint](#) (wxPaintEvent &)

Private Attributes

- const std::unordered_map< int, wxImage > & [m_imageStore](#)
- int [m_key](#)

6.30.1 Detailed Description

Definition at line 63 of file [MainFrame.cpp](#).

6.30.2 Constructor & Destructor Documentation

6.30.2.1 ImageBackgroundPanel()

```
ImageBackgroundPanel::ImageBackgroundPanel (  
    wxWindow * parent,  
    const std::unordered_map< int, wxImage > & imageStore,  
    int key) [inline]
```

Definition at line 65 of file [MainFrame.cpp](#).

6.30.3 Member Function Documentation

6.30.3.1 OnPaint()

```
void ImageBackgroundPanel::OnPaint (  
    wxPaintEvent & ) [inline], [private]
```

Definition at line 80 of file [MainFrame.cpp](#).

6.30.4 Member Data Documentation

6.30.4.1 m_imageStore

```
const std::unordered_map<int, wxImage>& ImageBackgroundPanel::m_imageStore [private]
```

Definition at line 77 of file [MainFrame.cpp](#).

6.30.4.2 m_key

```
int ImageBackgroundPanel::m_key [private]
```

Definition at line 78 of file [MainFrame.cpp](#).

The documentation for this class was generated from the following file:

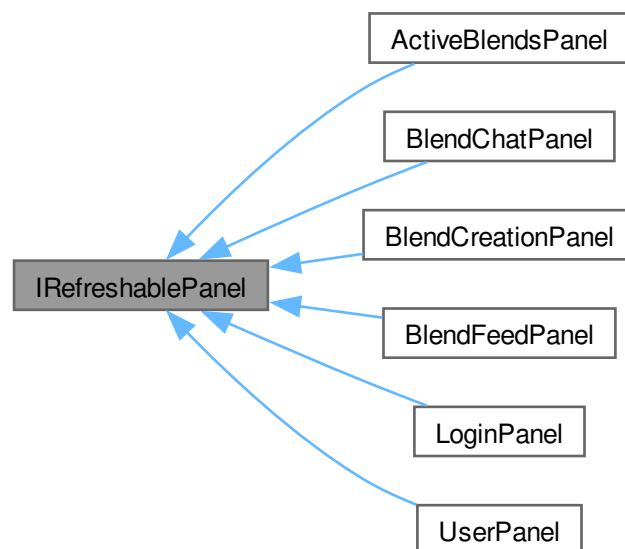
- [MainFrame.cpp](#)

6.31 IRefreshablePanel Class Reference

Interface for panels that support refresh operations.

```
#include <IRefreshablePanel.hpp>
```

Inheritance diagram for IRefreshablePanel:



Public Member Functions

- virtual void [Refresh](#) ()=0

6.31.1 Detailed Description

Interface for panels that support refresh operations.

[IRefreshablePanel](#) class declaration.

Definition at line 21 of file [IRefreshablePanel.hpp](#).

6.31.2 Member Function Documentation

6.31.2.1 Refresh()

```
virtual void IRefreshablePanel::Refresh () [pure virtual]
```

Refreshes the panel's display to reflect current application state. Called by [MainFrame](#) whenever the panel becomes active.

Implemented in [ActiveBlendsPanel](#), [BlendChatPanel](#), [BlendCreationPanel](#), [BlendFeedPanel](#), [LoginPanel](#), and [UserPanel](#).

The documentation for this class was generated from the following file:

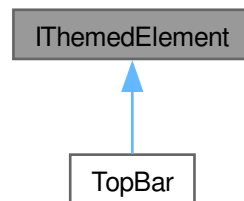
- [IRefreshablePanel.hpp](#)

6.32 IThemedElement Class Reference

Interface for any UI component that needs custom refresh logic (like icons).

```
#include <UIColors.hpp>
```

Inheritance diagram for IThemedElement:



Public Member Functions

- virtual void **OnThemeUpdate** ()=0

6.32.1 Detailed Description

Interface for any UI component that needs custom refresh logic (like icons).

-

Definition at line 50 of file [UIColors.hpp](#).

The documentation for this class was generated from the following file:

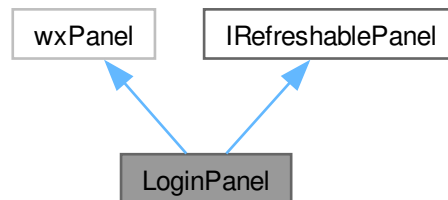
- [UIColors.hpp](#)

6.33 LoginPanel Class Reference

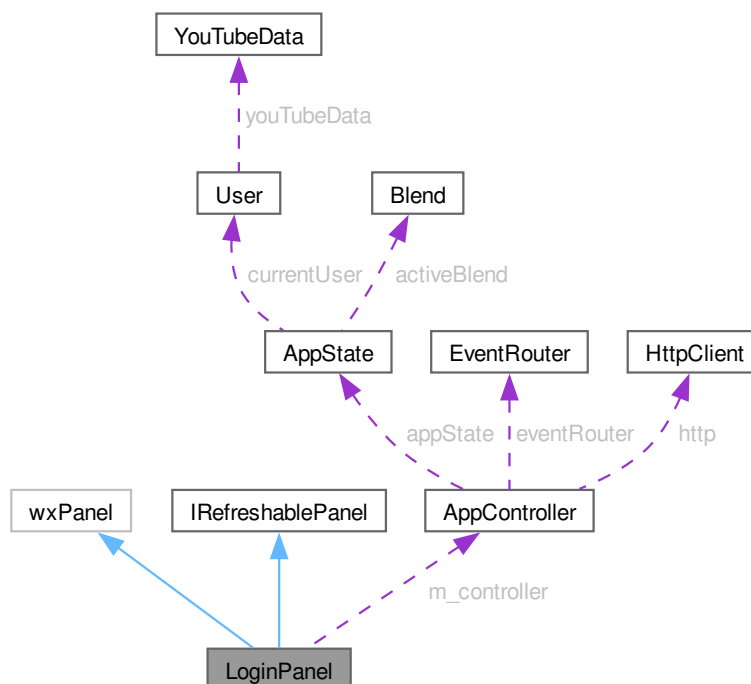
[LoginPanel](#) class declaration.

```
#include <LoginPanel.hpp>
```

Inheritance diagram for LoginPanel:



Collaboration diagram for LoginPanel:



Public Member Functions

- [LoginPanel](#) (wxWindow *parent, [AppController](#) &controller, [NavigateFn](#) nav, const wxImage &bglImage=wxNullImage)
- void [Refresh](#) () override
- void [Reset](#) ()

Private Member Functions

- void [BuildSignInForm](#) (wxWindow *parent, wxSizer *sizer)
- void [BuildRegisterForm](#) (wxWindow *parent, wxSizer *sizer)
- void [ShowTab](#) (int index)
- void [OnSignIn](#) (wxCommandEvent &)
- void [OnRegister](#) (wxCommandEvent &)
- void [OnPaint](#) (wxPaintEvent &)
- void [LoadThemedBackground](#) ()
Loads the background image matching the current theme.
- void [ProceedAfterLogin](#) ()

Private Attributes

- [AppController](#) & [m_controller](#)
- [NavigateFn](#) [m_nav](#)
- wxButton * [m_signinTab](#)
- wxButton * [m_registerTab](#)
- wxSimplebook * [m_formBook](#)
- wxTextCtrl * [m_siUsername](#)
- wxTextCtrl * [m_siPassword](#)
- wxStaticText * [m_siError](#)
- wxTextCtrl * [m_regUsername](#)
- wxTextCtrl * [m_regEmail](#)
- wxTextCtrl * [m_regPassword](#)
- wxTextCtrl * [m_regConfirm](#)
- wxStaticText * [m_regError](#)
- wxImage [m_bgImage](#)

6.33.1 Detailed Description

[LoginPanel](#) class declaration.

Definition at line 52 of file [LoginPanel.hpp](#).

6.33.2 Constructor & Destructor Documentation

6.33.2.1 LoginPanel()

```
LoginPanel::LoginPanel (  
    wxWindow * parent,  
    AppController & controller,  
    NavigateFn nav,  
    const wxImage & bgImage = wxNullImage)
```

Definition at line 54 of file [LoginPanel.cpp](#).

6.33.3 Member Function Documentation

6.33.3.1 BuildRegisterForm()

```
void LoginPanel::BuildRegisterForm (
    wxWindow * parent,
    wxSizer * sizer) [private]
```

Definition at line 186 of file [LoginPanel.cpp](#).

6.33.3.2 BuildSignInForm()

```
void LoginPanel::BuildSignInForm (
    wxWindow * parent,
    wxSizer * sizer) [private]
```

Definition at line 163 of file [LoginPanel.cpp](#).

6.33.3.3 LoadThemedBackground()

```
void LoginPanel::LoadThemedBackground () [private]
```

Loads the background image matching the current theme.

Definition at line 364 of file [LoginPanel.cpp](#).

6.33.3.4 OnPaint()

```
void LoginPanel::OnPaint (
    wxPaintEvent & ) [private]
```

Definition at line 393 of file [LoginPanel.cpp](#).

6.33.3.5 OnRegister()

```
void LoginPanel::OnRegister (
    wxCommandEvent & ) [private]
```

Definition at line 278 of file [LoginPanel.cpp](#).

6.33.3.6 OnSignIn()

```
void LoginPanel::OnSignIn (
    wxCommandEvent & ) [private]
```

Definition at line 236 of file [LoginPanel.cpp](#).

6.33.3.7 ProceedAfterLogin()

```
void LoginPanel::ProceedAfterLogin () [private]
```

Definition at line 316 of file [LoginPanel.cpp](#).

6.33.3.8 Refresh()

```
void LoginPanel::Refresh () [override], [virtual]
```

Refreshes the panel's display to reflect current application state. Called by [MainFrame](#) whenever the panel becomes active.

Implements [IRefreshablePanel](#).

Definition at line 344 of file [LoginPanel.cpp](#).

6.33.3.9 Reset()

```
void LoginPanel::Reset ()
```

Definition at line 349 of file [LoginPanel.cpp](#).

6.33.3.10 ShowTab()

```
void LoginPanel::ShowTab (  
    int index) [private]
```

Definition at line 212 of file [LoginPanel.cpp](#).

6.33.4 Member Data Documentation

6.33.4.1 m_bgImage

```
wxImage LoginPanel::m_bgImage [private]
```

Definition at line 85 of file [LoginPanel.hpp](#).

6.33.4.2 m_controller

```
AppController& LoginPanel::m_controller [private]
```

Definition at line 63 of file [LoginPanel.hpp](#).

6.33.4.3 m_formBook

```
wxSimplebook* LoginPanel::m_formBook [private]
```

Definition at line 71 of file [LoginPanel.hpp](#).

6.33.4.4 m_nav

```
NavigateFn LoginPanel::m_nav [private]
```

Definition at line 64 of file [LoginPanel.hpp](#).

6.33.4.5 m_regConfirm

```
wxTextCtrl* LoginPanel::m_regConfirm [private]
```

Definition at line 82 of file [LoginPanel.hpp](#).

6.33.4.6 m_regEmail

```
wxTextCtrl* LoginPanel::m_regEmail [private]
```

Definition at line 80 of file [LoginPanel.hpp](#).

6.33.4.7 m_regError

```
wxStaticText* LoginPanel::m_regError [private]
```

Definition at line 83 of file [LoginPanel.hpp](#).

6.33.4.8 m_registerTab

```
wxButton* LoginPanel::m_registerTab [private]
```

Definition at line 68 of file [LoginPanel.hpp](#).

6.33.4.9 m_regPassword

```
wxTextCtrl* LoginPanel::m_regPassword [private]
```

Definition at line 81 of file [LoginPanel.hpp](#).

6.33.4.10 m_regUsername

```
wxTextCtrl* LoginPanel::m_regUsername [private]
```

Definition at line 79 of file [LoginPanel.hpp](#).

6.33.4.11 m_siError

```
wxStaticText* LoginPanel::m_siError [private]
```

Definition at line 76 of file [LoginPanel.hpp](#).

6.33.4.12 m_signinTab

```
wxButton* LoginPanel::m_signinTab [private]
```

Definition at line 67 of file [LoginPanel.hpp](#).

6.33.4.13 m_siPassword

```
wxTextCtrl* LoginPanel::m_siPassword [private]
```

Definition at line 75 of file [LoginPanel.hpp](#).

6.33.4.14 m_siUsername

```
wxTextCtrl* LoginPanel::m_siUsername [private]
```

Definition at line 74 of file [LoginPanel.hpp](#).

The documentation for this class was generated from the following files:

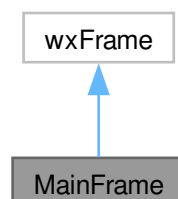
- [LoginPanel.hpp](#)
- [LoginPanel.cpp](#)

6.34 MainFrame Class Reference

Root application window containing the full-window page switcher.

```
#include <MainFrame.hpp>
```

Inheritance diagram for MainFrame:



- void **OnBlend** (wxCommandEvent &)
Navigates to `BLEND_CHAT` or `BLEND_CREATION`.
- void **OnRefresh** (wxCommandEvent &)
Advances the feed by one page.
- bool **LoadImage** (int key, const wxString &path)
- void **ShowSettingsDropdown** (wxWindow *anchor)
- void **ShowUserDropdown** (wxWindow *anchor)

Static Private Member Functions

- static wxBitmap **LoadThemedIcon** (const wxString &name, int size=24)
Load a themed icon bitmap from the resources directory.

Private Attributes

- [AppController](#) & controller
- wxSimplebook * [book](#)
- [LoginPanel](#) * loginPanel
- [DataInstructionsPanel](#) * dataInstrPanel
- [BlendCreationPanel](#) * creationPanel
- [BlendFeedPanel](#) * feedPanel
- [UserPanel](#) * userPanel
- [SettingsPanel](#) * settingsPanel
- [BlendChatPanel](#) * chatPanel
- [ActiveBlendsPanel](#) * activeBlendsPanel
- wxButton * [refreshBtn](#)
- wxButton * [settingsIconBtn](#) = nullptr
- wxButton * [userIconBtn](#) = nullptr
- wxButton * [blendsIconBtn](#) = nullptr
- wxButton * [chatIconBtn](#) = nullptr
- wxStaticText * [blendIndicatorLabel](#) = nullptr
- bool [refreshInProgress](#)
- std::unordered_map< int, wxImage > [images](#)

6.34.1 Detailed Description

Root application window containing the full-window page switcher.

[MainFrame](#) class declaration.

Definition at line 49 of file [MainFrame.hpp](#).

6.34.2 Member Enumeration Documentation

6.34.2.1 IconBase

```
enum class MainFrame::IconBase [strong], [private]
```

Definition at line 72 of file [MainFrame.hpp](#).

6.34.2.2 ImageKey

```
enum MainFrame::ImageKey [private]
```

Definition at line 77 of file [MainFrame.hpp](#).

6.34.3 Constructor & Destructor Documentation

6.34.3.1 MainFrame()

```
MainFrame::MainFrame (
    AppController & controller) [explicit]
```

Constructs and shows the [MainFrame](#). All sub-panels are created and added to the wxSimplebook here.

Parameters

<i>controller</i>	Application controller shared by all panels.
-------------------	--

Definition at line 99 of file [MainFrame.cpp](#).

6.34.4 Member Function Documentation

6.34.4.1 ApplyAppIcons()

```
void MainFrame::ApplyAppIcons () [private]
```

Load and apply available application icon assets to this frame.

Definition at line 34 of file [MainFrame.cpp](#).

6.34.4.2 BuildHomePage()

```
wxPanel * MainFrame::BuildHomePage (
    wxWindow * parent) [private]
```

Builds the home page composite (top bar + title + feed + refresh button).

Returns

Pointer to the constructed home page wxPanel.

Definition at line 395 of file [MainFrame.cpp](#).

6.34.4.3 LoadImage()

```
bool MainFrame::LoadImage (
    int key,
    const wxString & path) [private]
```

Definition at line 737 of file [MainFrame.cpp](#).

6.34.4.4 LoadThemedIcon()

```
wxBitmap MainFrame::LoadThemedIcon (
    const wxString & name,
    int size = 24) [static], [private]
```

Load a themed icon bitmap from the resources directory.

Parameters

<i>name</i>	Icon base name without extension.
<i>size</i>	Target icon size in pixels.

Returns

Loaded bitmap or wxNullBitmap if unavailable.

Definition at line 301 of file [MainFrame.cpp](#).

6.34.4.5 MakeIconButton()

```
wxButton * MainFrame::MakeIconButton (  
    wxWindow * parent,  
    const wxString & iconName,  
    IconBase base,  
    int iconSize = 24) [private]
```

Create a button that displays a themed icon and hover background.

Parameters

<i>parent</i>	Parent window that owns the button.
<i>iconName</i>	Icon base name without extension.
<i>base</i>	Determines which base background color to use.
<i>iconSize</i>	Icon size in pixels.

Returns

Newly created icon button.

Definition at line 314 of file [MainFrame.cpp](#).

6.34.4.6 NavigateTo()

```
void MainFrame::NavigateTo (  
    Page page)
```

Switches to the specified page and calls the target panel's Refresh() method where applicable so it always shows fresh data on arrival.

Parameters

<i>page</i>	Page to navigate to.
-------------	--------------------------------------

Definition at line 229 of file [MainFrame.cpp](#).

6.34.4.7 ReloadThemedIcons()

```
void MainFrame::ReloadThemedIcons () [private]
```

Reloads all themed icon bitmaps after a theme switch.

Definition at line 516 of file [MainFrame.cpp](#).

6.34.4.8 ShowSettingsDropdown()

```
void MainFrame::ShowSettingsDropdown (  
    wxWindow * anchor) [private]
```

Shows the settings dropdown (theme picker + logout) anchored below the given button.

Definition at line 562 of file [MainFrame.cpp](#).

6.34.4.9 ShowUserDropdown()

```
void MainFrame::ShowUserDropdown (  
    wxWindow * anchor) [private]
```

Shows the user dropdown (account actions) anchored below the given button.

Definition at line 644 of file [MainFrame.cpp](#).

6.34.4.10 TriggerFeedRefresh()

```
void MainFrame::TriggerFeedRefresh ()
```

Advances the blend feed by one page of videos. Called by the Refresh button on the home screen.

Definition at line 242 of file [MainFrame.cpp](#).

6.34.4.11 UpdateBlendIndicatorLabel()

```
void MainFrame::UpdateBlendIndicatorLabel () [private]
```

Updates the home-page active blend title label from controller state.

Definition at line 283 of file [MainFrame.cpp](#).

6.34.5 Member Data Documentation

6.34.5.1 activeBlendsPanel

```
ActiveBlendsPanel* MainFrame::activeBlendsPanel [private]
```

Definition at line 91 of file [MainFrame.hpp](#).

6.34.5.2 blendIndicatorLabel

```
wxStaticText* MainFrame::blendIndicatorLabel = nullptr [private]
```

Definition at line 98 of file [MainFrame.hpp](#).

6.34.5.3 blendsIconBtn

```
wxButton* MainFrame::blendsIconBtn = nullptr [private]
```

Definition at line 96 of file [MainFrame.hpp](#).

6.34.5.4 book

```
wxSimplebook* MainFrame::book [private]
```

Definition at line 83 of file [MainFrame.hpp](#).

6.34.5.5 chatIconBtn

```
wxButton* MainFrame::chatIconBtn = nullptr [private]
```

Definition at line 97 of file [MainFrame.hpp](#).

6.34.5.6 chatPanel

```
BlendChatPanel* MainFrame::chatPanel [private]
```

Definition at line 90 of file [MainFrame.hpp](#).

6.34.5.7 controller

```
AppController& MainFrame::controller [private]
```

Definition at line 81 of file [MainFrame.hpp](#).

6.34.5.8 creationPanel

```
BlendCreationPanel* MainFrame::creationPanel [private]
```

Definition at line 86 of file [MainFrame.hpp](#).

6.34.5.9 dataInstrPanel

```
DataInstructionsPanel* MainFrame::dataInstrPanel [private]
```

Definition at line 85 of file [MainFrame.hpp](#).

6.34.5.10 feedPanel

```
BlendFeedPanel* MainFrame::feedPanel [private]
```

Definition at line 87 of file [MainFrame.hpp](#).

6.34.5.11 images

```
std::unordered_map<int, wxImage> MainFrame::images [private]
```

Definition at line 131 of file [MainFrame.hpp](#).

6.34.5.12 loginPanel

```
LoginPanel* MainFrame::loginPanel [private]
```

Definition at line 84 of file [MainFrame.hpp](#).

6.34.5.13 refreshBtn

```
wxButton* MainFrame::refreshBtn [private]
```

Definition at line 93 of file [MainFrame.hpp](#).

6.34.5.14 refreshInProgress

```
bool MainFrame::refreshInProgress [private]
```

Definition at line 100 of file [MainFrame.hpp](#).

6.34.5.15 settingsIconBtn

```
wxButton* MainFrame::settingsIconBtn = nullptr [private]
```

Definition at line 94 of file [MainFrame.hpp](#).

6.34.5.16 settingsPanel

```
SettingsPanel* MainFrame::settingsPanel [private]
```

Definition at line 89 of file [MainFrame.hpp](#).

6.34.5.17 userIconBtn

```
wxButton* MainFrame::userIconBtn = nullptr [private]
```

Definition at line 95 of file [MainFrame.hpp](#).

6.34.5.18 userPanel

```
UserPanel* MainFrame::userPanel [private]
```

Definition at line 88 of file [MainFrame.hpp](#).

The documentation for this class was generated from the following files:

- [MainFrame.hpp](#)
- [MainFrame.cpp](#)

6.35 Message Struct Reference

A single chat message sent within a [ChatRoom](#).

```
#include <Message.hpp>
```

Public Member Functions

- [Message](#) (const std::string &[userID](#), const std::string &[text](#), std::time_t [timestamp](#)=std::time(nullptr))
- std::string [toString](#) () const

Public Attributes

- std::string [userID](#)
ID of the [User](#) who sent this message.
- std::string [text](#)
Content of the message.
- std::time_t [timestamp](#)
Unix timestamp of when the message was sent.

6.35.1 Detailed Description

A single chat message sent within a [ChatRoom](#).

[Message](#) class declaration.

Definition at line 22 of file [Message.hpp](#).

6.35.2 Constructor & Destructor Documentation

6.35.2.1 Message()

```
Message::Message (  
    const std::string & userID,  
    const std::string & text,  
    std::time_t timestamp = std::time(nullptr)) [inline]
```

Constructs a [Message](#) with an optional timestamp.

Parameters

<i>userID</i>	ID of the sending user.
<i>text</i>	Message content.
<i>timestamp</i>	Unix timestamp; defaults to the current time.

Definition at line 33 of file [Message.hpp](#).

6.35.3 Member Function Documentation

6.35.3.1 toString()

```
std::string Message::toString () const
```

String representation of [Message](#)

Returns

```
string
```

Definition at line 10 of file [Message.cpp](#).

6.35.4 Member Data Documentation

6.35.4.1 text

```
std::string Message::text
```

Content of the message.

Definition at line 24 of file [Message.hpp](#).

6.35.4.2 timestamp

```
std::time_t Message::timestamp
```

Unix timestamp of when the message was sent.

Definition at line 25 of file [Message.hpp](#).

6.35.4.3 userID

```
std::string Message::userID
```

ID of the [User](#) who sent this message.

Definition at line 23 of file [Message.hpp](#).

The documentation for this struct was generated from the following files:

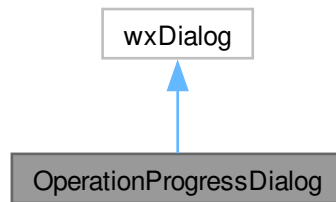
- [Message.hpp](#)
- [Message.cpp](#)

6.36 ProgressDialog Class Reference

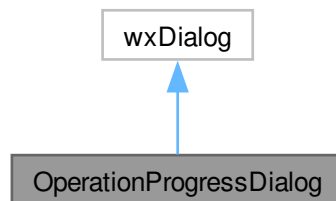
Non-modal progress dialog showing upload/processing progress.

```
#include <ProgressDialog.hpp>
```

Inheritance diagram for ProgressDialog:



Collaboration diagram for ProgressDialog:



Public Member Functions

- [ProgressDialog](#) (wxWindow *parent, const wxString &title)
Constructor with parent window and title.
- [~ProgressDialog](#) ()
Destructor.
- void [UpdateProgress](#) (double progress, const wxString &message=wxEmptyString)
Update progress bar and status message.
- virtual int [ShowModal](#) ()
Show dialog non-modally.

Private Member Functions

- void [UpdateProgressFill](#) ()

Private Attributes

- wxPanel * [progressTrack](#)
Progress track background.
- wxPanel * [progressFill](#)
Progress fill foreground.
- wxStaticText * [percentLabel](#)
Percentage display.
- wxStaticText * [messageLabel](#)
Status message.
- double [currentProgress](#) = 0.0

6.36.1 Detailed Description

Non-modal progress dialog showing upload/processing progress.

Definition at line 21 of file [OperationProgressDialog.hpp](#).

6.36.2 Constructor & Destructor Documentation

6.36.2.1 OperationProgressDialog()

```
OperationProgressDialog::OperationProgressDialog (  
    wxWindow * parent,  
    const wxString & title)
```

Constructor with parent window and title.

Definition at line 20 of file [OperationProgressDialog.cpp](#).

6.36.2.2 ~OperationProgressDialog()

```
OperationProgressDialog::~OperationProgressDialog ()
```

Destructor.

Definition at line 90 of file [OperationProgressDialog.cpp](#).

6.36.3 Member Function Documentation

6.36.3.1 ShowModal()

```
int OperationProgressDialog::ShowModal () [virtual]
```

Show dialog non-modally.

Definition at line 118 of file [OperationProgressDialog.cpp](#).

6.36.3.2 UpdateProgress()

```
void OperationProgressDialog::UpdateProgress (  
    double progress,  
    const wxString & message = wxEmptyString)
```

Update progress bar and status message.

Parameters

<i>progress</i>	Progress value 0.0 to 1.0.
<i>message</i>	Optional status message to display.

Definition at line 94 of file [OperationProgressDialog.cpp](#).

6.36.3.3 UpdateProgressFill()

```
void OperationProgressDialog::UpdateProgressFill () [private]
```

Definition at line 124 of file [OperationProgressDialog.cpp](#).

6.36.4 Member Data Documentation

6.36.4.1 currentProgress

```
double OperationProgressDialog::currentProgress = 0.0 [private]
```

Definition at line 52 of file [OperationProgressDialog.hpp](#).

6.36.4.2 messageLabel

```
wxStaticText* OperationProgressDialog::messageLabel [private]
```

Status message.

Definition at line 51 of file [OperationProgressDialog.hpp](#).

6.36.4.3 percentLabel

```
wxStaticText* OperationProgressDialog::percentLabel [private]
```

Percentage display.

Definition at line 50 of file [OperationProgressDialog.hpp](#).

6.36.4.4 progressFill

```
wxPanel* OperationProgressDialog::progressFill [private]
```

Progress fill foreground.

Definition at line 49 of file [OperationProgressDialog.hpp](#).

6.36.4.5 progressTrack

```
wxPanel* OperationProgressDialog::progressTrack [private]
```

Progress track background.

Track and fill are custom panels so Accent color is reliable across platforms.

Definition at line 48 of file [OperationProgressDialog.hpp](#).

The documentation for this class was generated from the following files:

- [OperationProgressDialog.hpp](#)
- [OperationProgressDialog.cpp](#)

6.37 Palette Struct Reference

[Palette](#) class declaration.

```
#include <UIColors.hpp>
```

Public Attributes

- wxString [Name](#)
- wxColour [Background](#)
- wxColour [Surface](#)
- wxColour [SurfaceRaised](#)
- wxColour [Accent](#)
- wxColour [AccentHover](#)
- wxColour [TextPrimary](#)
- wxColour [TextSecondary](#)
- wxColour [TextMuted](#)
- wxColour [Separator](#)
- wxColour [Danger](#)

6.37.1 Detailed Description

[Palette](#) class declaration.

Definition at line 60 of file [UIColors.hpp](#).

6.37.2 Member Data Documentation

6.37.2.1 Accent

```
wxColour Palette::Accent
```

Definition at line 67 of file [UIColors.hpp](#).

6.37.2.2 AccentHover

`wxColour Palette::AccentHover`

Definition at line 68 of file [UIColors.hpp](#).

6.37.2.3 Background

`wxColour Palette::Background`

Definition at line 64 of file [UIColors.hpp](#).

6.37.2.4 Danger

`wxColour Palette::Danger`

Definition at line 73 of file [UIColors.hpp](#).

6.37.2.5 Name

`wxString Palette::Name`

Definition at line 61 of file [UIColors.hpp](#).

6.37.2.6 Separator

`wxColour Palette::Separator`

Definition at line 72 of file [UIColors.hpp](#).

6.37.2.7 Surface

`wxColour Palette::Surface`

Definition at line 65 of file [UIColors.hpp](#).

6.37.2.8 SurfaceRaised

`wxColour Palette::SurfaceRaised`

Definition at line 66 of file [UIColors.hpp](#).

6.37.2.9 TextMuted

`wxColour Palette::TextMuted`

Definition at line 71 of file [UIColors.hpp](#).

6.37.2.10 TextPrimary

```
wxColour Palette::TextPrimary
```

Definition at line 69 of file [UIColors.hpp](#).

6.37.2.11 TextSecondary

```
wxColour Palette::TextSecondary
```

Definition at line 70 of file [UIColors.hpp](#).

The documentation for this struct was generated from the following file:

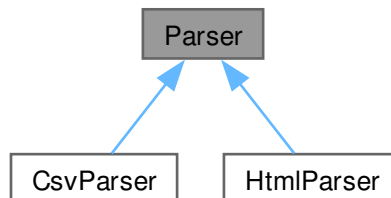
- [UIColors.hpp](#)

6.38 Parser Class Reference

[Parser](#) class declaration.

```
#include <Parser.hpp>
```

Inheritance diagram for Parser:



Public Member Functions

- virtual `std::list< std::unordered_map< std::string, std::string > >` [parse](#) ()=0
- virtual void [setData](#) (std::list< std::string > data)=0
- virtual int [getParseId](#) ()=0

6.38.1 Detailed Description

[Parser](#) class declaration.

Definition at line 28 of file [Parser.hpp](#).

6.38.2 Member Function Documentation

6.38.2.1 `getParseId()`

```
virtual int Parser::getParseId () [pure virtual]
```

Gets the file format ID associated with this parser.

Returns

Integer ID representing the file format (e.g., `File_ID::CSV`)

Implemented in [CsvParser](#), and [HtmlParser](#).

6.38.2.2 `parse()`

```
virtual std::list< std::unordered_map< std::string, std::string > > Parser::parse () [pure virtual]
```

Parses raw file data into a structured list of dictionaries.

Converts raw file data configured via [setData\(\)](#) into a list of unordered string-to-string dictionaries. Each dictionary represents a row with column titles as keys and row contents as values.

Returns

List of unordered maps representing parsed file rows

Implemented in [CsvParser](#), and [HtmlParser](#).

6.38.2.3 `setData()`

```
virtual void Parser::setData (
    std::list< std::string > data) [pure virtual]
```

Sets the raw file data to be parsed by [parse\(\)](#).

Parameters

<i>data</i>	List of strings containing raw file data
-------------	--

Implemented in [CsvParser](#).

The documentation for this class was generated from the following file:

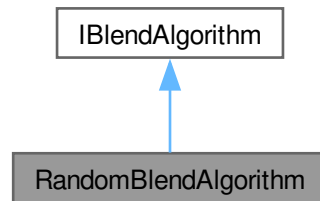
- [Parser.hpp](#)

6.39 RandomBlendAlgorithm Class Reference

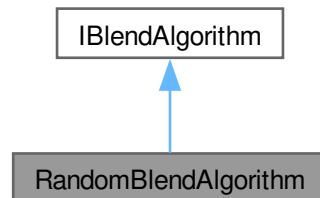
[RandomBlendAlgorithm](#) class declaration.

```
#include <RandomBlendAlgorithm.hpp>
```

Inheritance diagram for RandomBlendAlgorithm:



Collaboration diagram for RandomBlendAlgorithm:



Public Member Functions

- [RandomBlendAlgorithm](#) (int videosPerUser=5)
- [Blend generateBlend](#) (const std::list< [User](#) > &participants, const std::string &title="") override

Private Attributes

- int [videosPerUser](#)

6.39.1 Detailed Description

[RandomBlendAlgorithm](#) class declaration.

Definition at line 18 of file [RandomBlendAlgorithm.hpp](#).

6.39.2 Constructor & Destructor Documentation

6.39.2.1 RandomBlendAlgorithm()

```
RandomBlendAlgorithm::RandomBlendAlgorithm (  
    int videosPerUser = 5) [explicit]
```

Parameters

<i>videosPerUser</i>	How many videos to sample from each participant. If a user has fewer videos than this, all of their videos are included.
----------------------	--

Definition at line 14 of file [RandomBlendAlgorithm.cpp](#).

6.39.3 Member Function Documentation

6.39.3.1 generateBlend()

```
Blend RandomBlendAlgorithm::generateBlend (  
    const std::list< User > & participants,  
    const std::string & title = "") [override], [virtual]
```

Generates a [Blend](#) from a collection of participants.

Parameters

<i>participants</i>	List of users whose Watch Later data is used as input.
<i>title</i>	User-facing name for the blend (default: empty).

Returns

A new [Blend](#) containing the algorithm's chosen video selection.

Implements [IBlendAlgorithm](#).

Definition at line 17 of file [RandomBlendAlgorithm.cpp](#).

6.39.4 Member Data Documentation

6.39.4.1 videosPerUser

```
int RandomBlendAlgorithm::videosPerUser [private]
```

Definition at line 34 of file [RandomBlendAlgorithm.hpp](#).

The documentation for this class was generated from the following files:

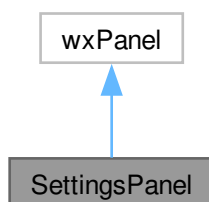
- [RandomBlendAlgorithm.hpp](#)
- [RandomBlendAlgorithm.cpp](#)

6.40 SettingsPanel Class Reference

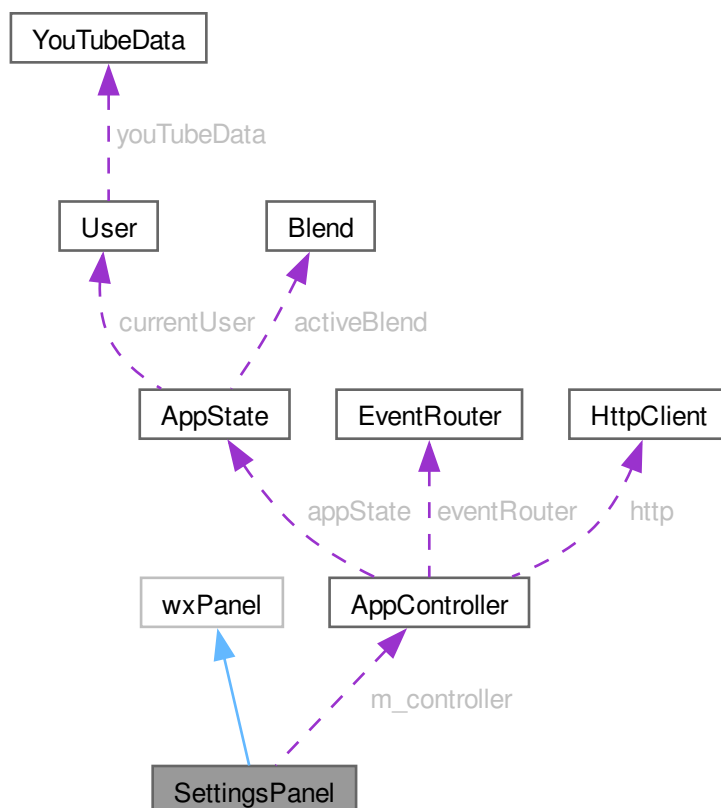
[SettingsPanel](#) class declaration.

```
#include <SettingsPanel.hpp>
```

Inheritance diagram for SettingsPanel:



Collaboration diagram for SettingsPanel:



Public Member Functions

- [SettingsPanel](#) (wxWindow *parent, [AppController](#) &controller, [NavigateFn](#) nav)

Private Member Functions

- void [RefreshUserInfo](#) ()
- void [OnShow](#) (wxShowEvent &evt)
- void [OnThemeChanged](#) (wxCommandEvent &evt)
- void [OnLogout](#) (wxCommandEvent &evt)

Private Attributes

- [AppController](#) & [m_controller](#)
- [NavigateFn](#) [m_nav](#)
- wxChoice * [m_themeChoice](#) = nullptr

6.40.1 Detailed Description

[SettingsPanel](#) class declaration.

Definition at line 20 of file [SettingsPanel.hpp](#).

6.40.2 Constructor & Destructor Documentation

6.40.2.1 SettingsPanel()

```
SettingsPanel::SettingsPanel (  
    wxWindow * parent,  
    AppController & controller,  
    NavigateFn nav)
```

Definition at line 26 of file [SettingsPanel.cpp](#).

6.40.3 Member Function Documentation

6.40.3.1 OnLogout()

```
void SettingsPanel::OnLogout (  
    wxCommandEvent & evt) [private]
```

Definition at line 112 of file [SettingsPanel.cpp](#).

6.40.3.2 OnShow()

```
void SettingsPanel::OnShow (  
    wxShowEvent & evt) [private]
```

Definition at line 100 of file [SettingsPanel.cpp](#).

6.40.3.3 OnThemeChanged()

```
void SettingsPanel::OnThemeChanged (
    wxCommandEvent & evt) [private]
```

Definition at line 123 of file [SettingsPanel.cpp](#).

6.40.3.4 RefreshUserInfo()

```
void SettingsPanel::RefreshUserInfo () [private]
```

Definition at line 93 of file [SettingsPanel.cpp](#).

6.40.4 Member Data Documentation

6.40.4.1 m_controller

```
AppController& SettingsPanel::m_controller [private]
```

Definition at line 25 of file [SettingsPanel.hpp](#).

6.40.4.2 m_nav

```
NavigateFn SettingsPanel::m_nav [private]
```

Definition at line 26 of file [SettingsPanel.hpp](#).

6.40.4.3 m_themeChoice

```
wxChoice* SettingsPanel::m_themeChoice = nullptr [private]
```

Definition at line 27 of file [SettingsPanel.hpp](#).

The documentation for this class was generated from the following files:

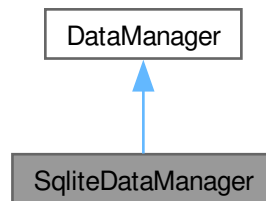
- [SettingsPanel.hpp](#)
- [SettingsPanel.cpp](#)

6.41 SqliteDataManager Class Reference

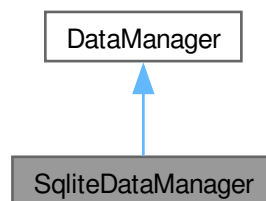
[SqliteDataManager](#) type declaration.

```
#include <SqliteDataManager.hpp>
```

Inheritance diagram for SqliteDataManager:



Collaboration diagram for SqliteDataManager:



Public Member Functions

- [SqliteDataManager](#) (const std::string &dbPath)
- bool [createUser](#) (const [User](#) &user) override
- std::optional< [User](#) > [findUserById](#) (const std::string &userID) override
- bool [validatePassword](#) (const std::string &userID, const std::string &password) override
- bool [saveWatchLater](#) (const std::string &userID, const std::list< [Video](#) > &videos) override
- std::list< [Video](#) > [loadWatchLater](#) (const std::string &userID) override
- bool [saveBlend](#) (const std::string &blendID, const std::string &creatorID, const std::string &algorithm, const std::vector< std::string > &participantIDs) override
- std::optional< std::string > [findBlendForUser](#) (const std::string &userID) override
- std::vector< std::string > [loadBlendParticipants](#) (const std::string &blendID) override

Private Member Functions

- void [initSchema](#) ()

Private Attributes

- `sqlite3 * db = nullptr`

6.41.1 Detailed Description

[SqliteDataManager](#) type declaration.

Definition at line 23 of file [SqliteDataManager.hpp](#).

6.41.2 Constructor & Destructor Documentation

6.41.2.1 SqliteDataManager()

```
SqliteDataManager::SqliteDataManager (  
    const std::string & dbPath) [explicit]
```

Definition at line 13 of file [SqliteDataManager.cpp](#).

6.41.2.2 ~SqliteDataManager()

```
SqliteDataManager::~SqliteDataManager () [override]
```

Definition at line 25 of file [SqliteDataManager.cpp](#).

6.41.3 Member Function Documentation

6.41.3.1 createUser()

```
bool SqliteDataManager::createUser (  
    const User & user) [override], [virtual]
```

Persists a new user record.

Returns

true on success, false if the userID already exists or the insert otherwise fails.

Implements [DataManager](#).

Definition at line 95 of file [SqliteDataManager.cpp](#).

6.41.3.2 findBlendForUser()

```
std::optional< std::string > SqliteDataManager::findBlendForUser (
    const std::string & userID) [override], [virtual]
```

Returns the blendID of the blend this user belongs to, if any.

Returns

The blendID string, or std::nullopt if the user has no blend.

Implements [DataManager](#).

Definition at line 304 of file [SqliteDataManager.cpp](#).

6.41.3.3 findUserByID()

```
std::optional< User > SqliteDataManager::findUserByID (
    const std::string & userID) [override], [virtual]
```

Loads a user by their unique ID.

Returns

The [User](#) if found, std::nullopt otherwise.

Implements [DataManager](#).

Definition at line 126 of file [SqliteDataManager.cpp](#).

6.41.3.4 initSchema()

```
void SqliteDataManager::initSchema () [private]
```

Definition at line 31 of file [SqliteDataManager.cpp](#).

6.41.3.5 loadBlendParticipants()

```
std::vector< std::string > SqliteDataManager::loadBlendParticipants (
    const std::string & blendID) [override], [virtual]
```

Returns all participant userIDs for a given blend.

Parameters

<i>blendID</i>	to be used for loading the respective participants
----------------	--

Returns

vector of strings representing userIDs

Implements [DataManager](#).

Definition at line 327 of file [SqliteDataManager.cpp](#).

6.41.3.6 loadWatchLater()

```
std::list< Video > SqliteDataManager::loadWatchLater (
    const std::string & userID) [override], [virtual]
```

Loads the stored Watch Later video list for a user.

Returns

Empty list if no data has been saved for this user.

Implements [DataManager](#).

Definition at line 211 of file [SqliteDataManager.cpp](#).

6.41.3.7 saveBlend()

```
bool SqliteDataManager::saveBlend (
    const std::string & blendID,
    const std::string & creatorID,
    const std::string & algorithm,
    const std::vector< std::string > & participantIDs) [override], [virtual]
```

Persists a blend and its participant list. Re-calling with the same blendID replaces the existing record.

Parameters

<i>creatorID</i>	userID of the user who initiated the blend.
<i>participantIDs</i>	All participant userIDs (including those without data).

Implements [DataManager](#).

Definition at line 250 of file [SqliteDataManager.cpp](#).

6.41.3.8 saveWatchLater()

```
bool SqliteDataManager::saveWatchLater (
    const std::string & userID,
    const std::list< Video > & videos) [override], [virtual]
```

Persists a user's Watch Later video list, replacing any previously saved list.

Returns

true on success.

Implements [DataManager](#).

Definition at line 164 of file [SqliteDataManager.cpp](#).

6.41.3.9 validatePassword()

```
bool SqliteDataManager::validatePassword (
    const std::string & userID,
    const std::string & password) [override], [virtual]
```

Checks whether the given plaintext password matches the stored record.

Returns

true if the userID exists and the password matches.

Implements [DataManager](#).

Definition at line 155 of file [SqliteDataManager.cpp](#).

6.41.4 Member Data Documentation

6.41.4.1 db

```
sqlite3* SqliteDataManager::db = nullptr [private]
```

Definition at line 47 of file [SqliteDataManager.hpp](#).

The documentation for this class was generated from the following files:

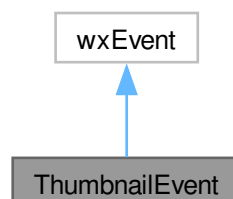
- [SqliteDataManager.hpp](#)
- [SqliteDataManager.cpp](#)

6.42 ThumbnailEvent Class Reference

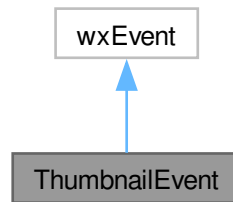
Event fired on the [VideoCard](#) when an async image fetch completes.

```
#include <VideoCard.hpp>
```

Inheritance diagram for ThumbnailEvent:



Collaboration diagram for ThumbnailEvent:



Public Member Functions

- [ThumbnailEvent](#) (wxEventType type, int id, wxImage img, bool ok, int gen)
- wxEvent * [Clone](#) () const override

Public Attributes

- wxImage [image](#)
Decoded image (valid only when success is true).
- bool [success](#)
True if the fetch and decode both succeeded.
- int [generation](#)
Must match [VideoCard::m_thumbGen](#) or [m_logoGen](#).

6.42.1 Detailed Description

Event fired on the [VideoCard](#) when an async image fetch completes.

Used for both the video thumbnail (EVT_THUMBNAIL_LOADED) and the channel logo (EVT_CHANNEL_LOGO_↔LOADED). The generation field must match the card's current counter or the event is treated as stale and discarded.

[ThumbnailEvent](#) class declaration.

Definition at line 37 of file [VideoCard.hpp](#).

6.42.2 Constructor & Destructor Documentation

6.42.2.1 ThumbnailEvent()

```
ThumbnailEvent::ThumbnailEvent (
    wxEventType type,
    int id,
    wxImage img,
    bool ok,
    int gen) [inline]
```

Definition at line 43 of file [VideoCard.hpp](#).

6.42.3 Member Function Documentation

6.42.3.1 Clone()

```
wxEvent * ThumbnailEvent::Clone () const [inline], [override]
```

Definition at line 46 of file [VideoCard.hpp](#).

6.42.4 Member Data Documentation

6.42.4.1 generation

```
int ThumbnailEvent::generation
```

Must match [VideoCard::m_thumbGen](#) or [m_logoGen](#).

Definition at line 41 of file [VideoCard.hpp](#).

6.42.4.2 image

```
wxImage ThumbnailEvent::image
```

Decoded image (valid only when success is true).

Definition at line 39 of file [VideoCard.hpp](#).

6.42.4.3 success

```
bool ThumbnailEvent::success
```

True if the fetch and decode both succeeded.

Definition at line 40 of file [VideoCard.hpp](#).

The documentation for this class was generated from the following file:

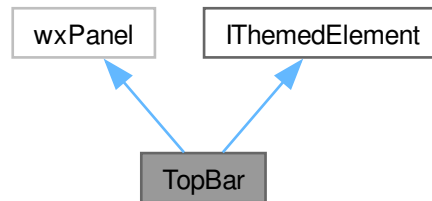
- [VideoCard.hpp](#)

6.43 TopBar Class Reference

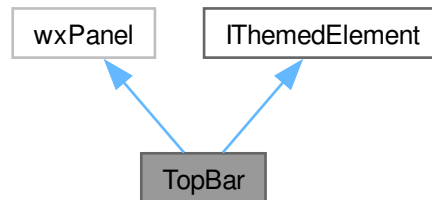
Reusable horizontal top bar for interior pages.

```
#include <TopBar.hpp>
```

Inheritance diagram for TopBar:



Collaboration diagram for TopBar:



Public Member Functions

- [TopBar](#) (wxWindow *parent, const wxString &title, [NavigateFn](#) nav, [Page](#) backDest)
Construct a [TopBar](#).
- void [OnThemeUpdate](#) () override

Private Member Functions

- void [OnBack](#) (wxCommandEvent &evt)
Handle clicks on the back button.

Private Attributes

- [NavigateFn m_nav](#)
- [Page m_backDest](#)
- `wxBtn * m\_backBtn = nullptr`

6.43.1 Detailed Description

Reusable horizontal top bar for interior pages.

Definition at line 21 of file [TopBar.hpp](#).

6.43.2 Constructor & Destructor Documentation

6.43.2.1 TopBar()

```
TopBar::TopBar (  
    wxWindow * parent,  
    const wxString & title,  
    NavigateFn nav,  
    Page backDest)
```

Construct a [TopBar](#).

Parameters

<i>parent</i>	Parent window that owns the bar.
<i>title</i>	Title text shown in the center.
<i>nav</i>	Navigation callback used for back navigation.
<i>backDest</i>	Destination page when the back button is clicked.

Definition at line 19 of file [TopBar.cpp](#).

6.43.3 Member Function Documentation

6.43.3.1 OnBack()

```
void TopBar::OnBack (  
    wxCommandEvent & evt) [private]
```

Handle clicks on the back button.

Parameters

<i>evt</i>	Button click event.
------------	---------------------

Definition at line 91 of file [TopBar.cpp](#).

6.43.3.2 OnThemeUpdate()

```
void TopBar::OnThemeUpdate () [override], [virtual]
```

Implements [IThemedElement](#).

Definition at line 74 of file [TopBar.cpp](#).

6.43.4 Member Data Documentation

6.43.4.1 m_backBtn

```
wxButton* TopBar::m_backBtn = nullptr [private]
```

Stored reference to the back button for icon reloading.

Definition at line 48 of file [TopBar.hpp](#).

6.43.4.2 m_backDest

```
Page TopBar::m_backDest [private]
```

Destination page used when the back button is activated.

Definition at line 46 of file [TopBar.hpp](#).

6.43.4.3 m_nav

```
NavigateFn TopBar::m_nav [private]
```

Navigation callback forwarded from the parent panel.

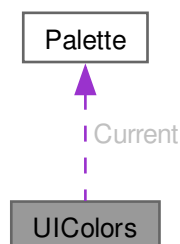
Definition at line 44 of file [TopBar.hpp](#).

The documentation for this class was generated from the following files:

- [TopBar.hpp](#)
- [TopBar.cpp](#)

6.44 UIColors Class Reference

Collaboration diagram for UIColors:



Static Public Member Functions

- static wxColour [Background](#) ()
- static wxColour [Surface](#) ()
- static wxColour [SurfaceRaised](#) ()
- static wxColour [Accent](#) ()
- static wxColour [AccentHover](#) ()
- static wxColour [TextPrimary](#) ()
- static wxColour [TextSecondary](#) ()
- static wxColour [TextMuted](#) ()
- static wxColour [Separator](#) ()
- static wxColour [Danger](#) ()
- static bool [LoadThemesFromFile](#) (const wxString &filepath)
Parses the themes.txt file and populates the ThemeMap.
- static void [SetTheme](#) (const wxString &themeName)
Switches the active theme by looking up the name in the map.
- static wxButton * [MakeIconButton](#) (wxWindow *parent, const wxString &iconName, ColorRole bgRole, int iconSize)
Centralized factory for icon buttons.
- static wxBitmapBundle [GetIcon](#) (const wxString &iconName, ColorRole role, int size)
Loads an SVG and replaces "CURRENT_COLOR" with a palette color.
- static void [UpdateUI](#) (wxWindow *win)
Updates the displaying UI with the active palette.

Static Public Attributes

- static std::unordered_map< wxString, [Palette](#) > [ThemeMap](#)
- static std::vector< wxString > [ThemeOrder](#)
- static [Palette](#) * [Current](#) = nullptr

Static Private Member Functions

- static ColorRole [GetRoleFromColour](#) (const wxColour &col)
Checks a color against all loaded themes to find its role.

6.44.1 Detailed Description

Definition at line 76 of file [UIColors.hpp](#).

6.44.2 Member Function Documentation

6.44.2.1 Accent()

```
wxColour UIColors::Accent () [inline], [static]
```

Definition at line 91 of file [UIColors.hpp](#).

6.44.2.2 AccentHover()

```
wxColour UIColors::AccentHover () [inline], [static]
```

Definition at line 92 of file [UIColors.hpp](#).

6.44.2.3 Background()

```
wxColour UIColors::Background () [inline], [static]
```

Definition at line 88 of file [UIColors.hpp](#).

6.44.2.4 Danger()

```
wxColour UIColors::Danger () [inline], [static]
```

Definition at line 97 of file [UIColors.hpp](#).

6.44.2.5 GetIcon()

```
wxBitmapBundle UIColors::GetIcon (
    const wxString & iconName,
    ColorRole role,
    int size) [static]
```

Loads an SVG and replaces "CURRENT_COLOR" with a palette color.

Parameters

<i>iconName</i>	The filename (e.g., "back.svg")
<i>role</i>	The color role to apply (e.g., ColorRole::TextPrimary)
<i>size</i>	Pixel size for the bundle

Definition at line 155 of file [UIColors.cpp](#).

6.44.2.6 GetRoleFromColour()

```
ColorRole UIColors::GetRoleFromColour (
    const wxColour & col) [static], [private]
```

Checks a color against all loaded themes to find its role.

Parameters

<i>col</i>	The colour value being passed to check its role.
------------	--

Returns

ColorRole The role matching the colour value.

Definition at line 188 of file [UIColors.cpp](#).

6.44.2.7 LoadThemesFromFile()

```
bool UIColors::LoadThemesFromFile (
    const wxString & filepath) [static]
```

Parses the themes.txt file and populates the ThemeMap.

Parameters

<i>filepath</i>	The path to the themes.txt file.
-----------------	----------------------------------

Returns

True if successful, false if the file couldn't be loaded.

Definition at line 26 of file [UIColors.cpp](#).

6.44.2.8 MakeIconButton()

```
wxButton * UIColors::MakeIconButton (  
    wxWindow * parent,  
    const wxString & iconName,  
    ColorRole bgRole,  
    int iconSize) [static]
```

Centralized factory for icon buttons.

Parameters

<i>parent</i>	The parent window.
<i>iconName</i>	The filename (e.g., "refresh").
<i>bgRole</i>	The color role for the button background.
<i>iconSize</i>	Pixel size for the icon.

Definition at line 127 of file [UIColors.cpp](#).

6.44.2.9 Separator()

```
wxColour UIColors::Separator () [inline], [static]
```

Definition at line 96 of file [UIColors.hpp](#).

6.44.2.10 SetTheme()

```
void UIColors::SetTheme (  
    const wxString & themeName) [static]
```

Switches the active theme by looking up the name in the map.

Parameters

<i>themeName</i>	The string name of the theme (default: "Dark Mode").
------------------	--

Definition at line 97 of file [UIColors.cpp](#).

6.44.2.11 Surface()

```
wxColour UIColors::Surface () [inline], [static]
```

Definition at line 89 of file [UIColors.hpp](#).

6.44.2.12 SurfaceRaised()

```
wxColour UIColors::SurfaceRaised () [inline], [static]
```

Definition at line 90 of file [UIColors.hpp](#).

6.44.2.13 TextMuted()

```
wxColour UIColors::TextMuted () [inline], [static]
```

Definition at line 95 of file [UIColors.hpp](#).

6.44.2.14 TextPrimary()

```
wxColour UIColors::TextPrimary () [inline], [static]
```

Definition at line 93 of file [UIColors.hpp](#).

6.44.2.15 TextSecondary()

```
wxColour UIColors::TextSecondary () [inline], [static]
```

Definition at line 94 of file [UIColors.hpp](#).

6.44.2.16 UpdateUI()

```
void UIColors::UpdateUI (  
    wxWindow * win) [static]
```

Updates the displaying UI with the active palette.

Parameters

<i>win</i>	The current window to be updated.
------------	-----------------------------------

Definition at line 205 of file [UIColors.cpp](#).

6.44.3 Member Data Documentation

6.44.3.1 Current

```
Palette * UIColors::Current = nullptr [static]
```

Definition at line 85 of file [UIColors.hpp](#).

6.44.3.2 ThemeMap

```
std::unordered_map< wxString, Palette > UIColors::ThemeMap [static]
```

Definition at line 79 of file [UIColors.hpp](#).

6.44.3.3 ThemeOrder

```
std::vector< wxString > UIColors::ThemeOrder [static]
```

Definition at line 82 of file [UIColors.hpp](#).

The documentation for this class was generated from the following files:

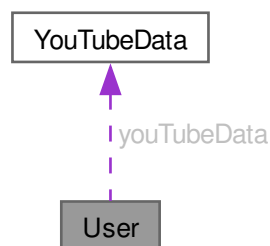
- [UIColors.hpp](#)
- [UIColors.cpp](#)

6.45 User Class Reference

[User](#) class declaration.

```
#include <User.hpp>
```

Collaboration diagram for User:



Public Member Functions

- [User](#) (const std::string &userID, const std::string &username, const std::string &email, const std::string &password)
- std::string [getUserID](#) () const
- std::string [getUsername](#) () const
- std::string [getEmail](#) () const
- std::string [getPassword](#) () const
- std::list< [Friend](#) > [getFriends](#) () const
- [YouTubeData](#) & [getYouTubeData](#) ()
- const [YouTubeData](#) & [getYouTubeData](#) () const
- void [setUsername](#) (const std::string &username)
- void [setEmail](#) (const std::string &email)
- void [setPassword](#) (const std::string &password)
- void [setYouTubeData](#) (const [YouTubeData](#) &youTubeData)
- void [addFriend](#) (const [User](#) &user)
- void [removeFriend](#) (const std::string &userID)
- std::string [toString](#) () const

Private Attributes

- std::string [userID](#)
- std::string [username](#)
- std::string [email](#)
- std::string [password](#)
Stored as plaintext - hashing is a future TODO.
- std::list< [Friend](#) > [friends](#)
- [YouTubeData](#) [youTubeData](#)

6.45.1 Detailed Description

[User](#) class declaration.

A registered application user.

Definition at line 24 of file [User.hpp](#).

6.45.2 Constructor & Destructor Documentation

6.45.2.1 User()

```
User::User (  
    const std::string & userID,  
    const std::string & username,  
    const std::string & email,  
    const std::string & password)
```

Constructs a [User](#) with identity fields.

Parameters

<i>userID</i>	Unique user identifier chosen at registration.
<i>username</i>	Display name shown in the UI.
<i>email</i>	User 's email address (may be empty).
<i>password</i>	Plaintext password.

Definition at line 13 of file [User.cpp](#).

6.45.3 Member Function Documentation

6.45.3.1 addFriend()

```
void User::addFriend (  
    const User & user)
```

Adds a friend entry for the given user.

Parameters

<i>user</i>	User to add as a friend.
-------------	--

Definition at line 43 of file [User.cpp](#).

6.45.3.2 getEmail()

```
std::string User::getEmail () const
```

Returns

Email address.

Definition at line 23 of file [User.cpp](#).

6.45.3.3 getFriends()

```
std::list< Friend > User::getFriends () const
```

Returns

List of this user's friends.

Definition at line 26 of file [User.cpp](#).

6.45.3.4 getPassword()

```
std::string User::getPassword () const
```

Returns

Plaintext password.

Definition at line 24 of file [User.cpp](#).

6.45.3.5 getUserID()

```
std::string User::getUserID () const
```

Returns

Unique user identifier.

Definition at line 21 of file [User.cpp](#).

6.45.3.6 getUsername()

```
std::string User::getUsername () const
```

Returns

Display name.

Definition at line 22 of file [User.cpp](#).

6.45.3.7 getYouTubeData() [1/2]

```
YouTubeData & User::getYouTubeData ()
```

Returns

Mutable reference to this user's YouTube data.

Definition at line 28 of file [User.cpp](#).

6.45.3.8 getYouTubeData() [2/2]

```
const YouTubeData & User::getYouTubeData () const
```

Returns

Const reference to this user's YouTube data.

Definition at line 29 of file [User.cpp](#).

6.45.3.9 removeFriend()

```
void User::removeFriend (  
    const std::string & userID)
```

Removes the friend entry with the given userID.

Parameters

<i>userID</i>	ID of the friend to remove.
---------------	-----------------------------

Definition at line 50 of file [User.cpp](#).

6.45.3.10 setEmail()

```
void User::setEmail (  
    const std::string & email)
```

Parameters

<i>email</i>	New email address.
--------------	--------------------

Definition at line 34 of file [User.cpp](#).

6.45.3.11 setPassword()

```
void User::setPassword (  
    const std::string & password)
```

Parameters

<i>password</i>	New plaintext password.
-----------------	-------------------------

Definition at line 35 of file [User.cpp](#).

6.45.3.12 setUsername()

```
void User::setUsername (  
    const std::string & username)
```

Parameters

<i>username</i>	New display name.
-----------------	-------------------

Definition at line 33 of file [User.cpp](#).

6.45.3.13 setYouTubeData()

```
void User::setYouTubeData (  
    const YouTubeData & youtubeData)
```

Parameters

<code>youTubeData</code>	Replaces the user's YouTube data in bulk.
--------------------------	---

Definition at line 37 of file [User.cpp](#).

6.45.3.14 toString()

```
std::string User::toString () const
```

String representation of [User](#)

Returns

```
string
```

Definition at line 56 of file [User.cpp](#).

6.45.4 Member Data Documentation

6.45.4.1 email

```
std::string User::email [private]
```

Definition at line 28 of file [User.hpp](#).

6.45.4.2 friends

```
std::list<Friend> User::friends [private]
```

Definition at line 30 of file [User.hpp](#).

6.45.4.3 password

```
std::string User::password [private]
```

Stored as plaintext - hashing is a future TODO.

Definition at line 29 of file [User.hpp](#).

6.45.4.4 userID

```
std::string User::userID [private]
```

Definition at line 26 of file [User.hpp](#).

6.45.4.5 username

```
std::string User::username [private]
```

Definition at line 27 of file [User.hpp](#).

6.45.4.6 youTubeData

```
YouTubeData User::youTubeData [private]
```

Definition at line 31 of file [User.hpp](#).

The documentation for this class was generated from the following files:

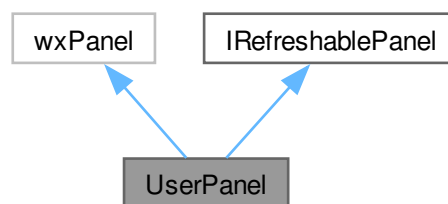
- [User.hpp](#)
- [User.cpp](#)

6.46 UserPanel Class Reference

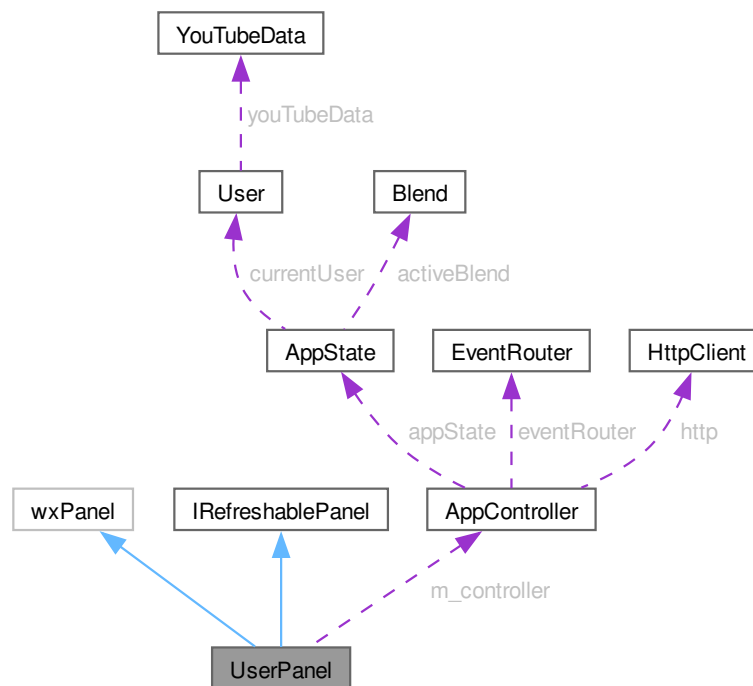
Panel for user account display and account actions.

```
#include <UserPanel.hpp>
```

Inheritance diagram for UserPanel:



Collaboration diagram for `UserPanel`:



Public Member Functions

- `UserPanel` (`wxWindow *parent`, `AppController` &controller, `NavigateFn` nav)
Construct a `UserPanel`.
- void `Refresh` () override
Refresh panel UI from current `AppState`.

Private Member Functions

- void `RefreshUserInfo` ()
- void `SetDeleteReauthVisible` (bool isVisible)
- void `OnDeleteRequest` (wxCommandEvent &evt)
- void `OnConfirmDelete` (wxCommandEvent &evt)
- void `OnReuploadCSV` (wxCommandEvent &evt)
- void `OnLogout` (wxCommandEvent &evt)
- void `OnShow` (wxShowEvent &evt)

Private Attributes

- `AppController` & `m_controller`
- `NavigateFn` `m_nav`
- `wxStaticText` * `m_usernameLabel`
- `wxStaticText` * `m_emailLabel`
- `wxTextCtrl` * `m_deletePasswordField`
- `wxStaticText` * `m_deleteErrorLabel`
- `wxButton` * `m_confirmDeleteBtn`

6.46.1 Detailed Description

Panel for user account display and account actions.

Definition at line 48 of file [UserPanel.hpp](#).

6.46.2 Constructor & Destructor Documentation

6.46.2.1 UserPanel()

```
UserPanel::UserPanel (  
    wxWindow * parent,  
    ApplicationController & controller,  
    NavigateFn nav)
```

Construct a [UserPanel](#).

Parameters

<i>parent</i>	Parent wxWidgets window.
<i>controller</i>	Application controller reference.
<i>nav</i>	Page navigation callback.

Definition at line 29 of file [UserPanel.cpp](#).

6.46.3 Member Function Documentation

6.46.3.1 OnConfirmDelete()

```
void UserPanel::OnConfirmDelete (  
    wxCommandEvent & evt) [private]
```

Handle delete-account confirmation click.

Definition at line 195 of file [UserPanel.cpp](#).

6.46.3.2 OnDeleteRequest()

```
void UserPanel::OnDeleteRequest (  
    wxCommandEvent & evt) [private]
```

Handle initial delete-account request button click.

Definition at line 179 of file [UserPanel.cpp](#).

6.46.3.3 OnLogout()

```
void UserPanel::OnLogout (
    wxCommandEvent & evt) [private]
```

Handle logout action.

Definition at line 266 of file [UserPanel.cpp](#).

6.46.3.4 OnReuploadCSV()

```
void UserPanel::OnReuploadCSV (
    wxCommandEvent & evt) [private]
```

Handle CSV re-upload action.

Definition at line 234 of file [UserPanel.cpp](#).

6.46.3.5 OnShow()

```
void UserPanel::OnShow (
    wxShowEvent & evt) [private]
```

Handle panel show event to trigger refresh.

Definition at line 172 of file [UserPanel.cpp](#).

6.46.3.6 Refresh()

```
void UserPanel::Refresh () [override], [virtual]
```

Refresh panel UI from current [AppState](#).

Implements [IRefreshablePanel](#).

Definition at line 128 of file [UserPanel.cpp](#).

6.46.3.7 RefreshUserInfo()

```
void UserPanel::RefreshUserInfo () [private]
```

Refresh user labels from [AppState](#).

Definition at line 133 of file [UserPanel.cpp](#).

6.46.3.8 SetDeleteReauthVisible()

```
void UserPanel::SetDeleteReauthVisible (  
    bool isVisible) [private]
```

Show or hide delete-account re-auth controls.

Definition at line 157 of file [UserPanel.cpp](#).

6.46.4 Member Data Documentation

6.46.4.1 m_confirmDeleteBtn

```
wxButton* UserPanel::m_confirmDeleteBtn [private]
```

Definition at line 72 of file [UserPanel.hpp](#).

6.46.4.2 m_controller

```
AppController& UserPanel::m_controller [private]
```

Definition at line 64 of file [UserPanel.hpp](#).

6.46.4.3 m_deleteErrorLabel

```
wxStaticText* UserPanel::m_deleteErrorLabel [private]
```

Definition at line 71 of file [UserPanel.hpp](#).

6.46.4.4 m_deletePasswordField

```
wxTextCtrl* UserPanel::m_deletePasswordField [private]
```

Definition at line 70 of file [UserPanel.hpp](#).

6.46.4.5 m_emailLabel

```
wxStaticText* UserPanel::m_emailLabel [private]
```

Definition at line 69 of file [UserPanel.hpp](#).

6.46.4.6 m_nav

```
NavigateFn UserPanel::m_nav [private]
```

Definition at line 65 of file [UserPanel.hpp](#).

6.46.4.7 m_usernameLabel

```
wxStaticText* UserPanel::m_usernameLabel [private]
```

Labels that display the logged-in user's info.

Definition at line 68 of file [UserPanel.hpp](#).

The documentation for this class was generated from the following files:

- [UserPanel.hpp](#)
- [UserPanel.cpp](#)

6.47 Video Class Reference

[Video](#) class declaration.

```
#include <Video.hpp>
```

Public Member Functions

- [Video](#) (const std::string &videoID, const std::string &title, const std::string &channelID, const std::string &thumbnailURL, int duration, const std::list< std::string > &tags, const std::string &channelName="", const std::string &channelLogoURL="")
- std::string [getVideoID](#) () const
- std::string [getTitle](#) () const
- std::string [getChannelID](#) () const
- std::string [getThumbnailURL](#) () const
- int [getDuration](#) () const
- std::list< std::string > [getTags](#) () const
- std::string [getChannelName](#) () const
- std::string [getChannelLogoURL](#) () const
- std::string [toString](#) () const

Private Attributes

- std::string [videoID](#)
- std::string [title](#)
- std::string [channelID](#)
- std::string [thumbnailURL](#)
- int [duration](#)
- std::list< std::string > [tags](#)
- std::string [channelName](#)
- std::string [channelLogoURL](#)

6.47.1 Detailed Description

[Video](#) class declaration.

Immutable data model for a single YouTube video.

Definition at line 21 of file [Video.hpp](#).

6.47.2 Constructor & Destructor Documentation

6.47.2.1 Video()

```
Video::Video (
    const std::string & videoID,
    const std::string & title,
    const std::string & channelID,
    const std::string & thumbnailURL,
    int duration,
    const std::list< std::string > & tags,
    const std::string & channelName = "",
    const std::string & channelLogoURL = "")
```

Constructs a [Video](#) with all metadata fields.

Parameters

<i>videoID</i>	YouTube video ID (e.g. "dQw4w9WgXcQ").
<i>title</i>	Video title returned by the Data API.
<i>channelID</i>	YouTube channel ID of the uploader.
<i>thumbnailURL</i>	URL of the best-available thumbnail image.
<i>duration</i>	Video duration in seconds (0 if unknown).
<i>tags</i>	List of tags associated with the video.
<i>channelName</i>	Display name of the uploading channel.
<i>channelLogoURL</i>	URL of the channel's profile picture.

Definition at line 12 of file [Video.cpp](#).

6.47.3 Member Function Documentation

6.47.3.1 getChannelID()

```
std::string Video::getChannelID () const
```

Returns

YouTube channel ID of the uploader.

Definition at line 37 of file [Video.cpp](#).

6.47.3.2 getChannelLogoURL()

```
std::string Video::getChannelLogoURL () const
```

Returns

URL of the channel's profile picture.

Definition at line 57 of file [Video.cpp](#).

6.47.3.3 `getChannelName()`

```
std::string Video::getChannelName () const
```

Returns

Display name of the uploading channel.

Definition at line 53 of file [Video.cpp](#).

6.47.3.4 `getDuration()`

```
int Video::getDuration () const
```

Returns

Duration in seconds (0 if unknown).

Definition at line 45 of file [Video.cpp](#).

6.47.3.5 `getTags()`

```
std::list< std::string > Video::getTags () const
```

Returns

List of tags associated with the video.

Definition at line 49 of file [Video.cpp](#).

6.47.3.6 `getThumbnailURL()`

```
std::string Video::getThumbnailURL () const
```

Returns

URL of the video thumbnail (empty if not yet enriched).

Definition at line 41 of file [Video.cpp](#).

6.47.3.7 `getTitle()`

```
std::string Video::getTitle () const
```

Returns

[Video](#) title (empty if not yet enriched by the API).

Definition at line 33 of file [Video.cpp](#).

6.47.3.8 `getVideoID()`

```
std::string Video::getVideoID () const
```

Returns

YouTube video ID string.

Definition at line 29 of file [Video.cpp](#).

6.47.3.9 `toString()`

```
std::string Video::toString () const
```

Returns

JSON string representation of this video.

Definition at line 61 of file [Video.cpp](#).

6.47.4 Member Data Documentation

6.47.4.1 `channelID`

```
std::string Video::channelID [private]
```

Definition at line 25 of file [Video.hpp](#).

6.47.4.2 `channelLogoURL`

```
std::string Video::channelLogoURL [private]
```

Definition at line 30 of file [Video.hpp](#).

6.47.4.3 `channelName`

```
std::string Video::channelName [private]
```

Definition at line 29 of file [Video.hpp](#).

6.47.4.4 `duration`

```
int Video::duration [private]
```

Definition at line 27 of file [Video.hpp](#).

6.47.4.5 tags

```
std::list<std::string> Video::tags [private]
```

Definition at line 28 of file [Video.hpp](#).

6.47.4.6 thumbnailURL

```
std::string Video::thumbnailURL [private]
```

Definition at line 26 of file [Video.hpp](#).

6.47.4.7 title

```
std::string Video::title [private]
```

Definition at line 24 of file [Video.hpp](#).

6.47.4.8 videoID

```
std::string Video::videoID [private]
```

Definition at line 23 of file [Video.hpp](#).

The documentation for this class was generated from the following files:

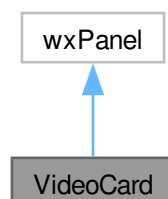
- [Video.hpp](#)
- [Video.cpp](#)

6.48 VideoCard Class Reference

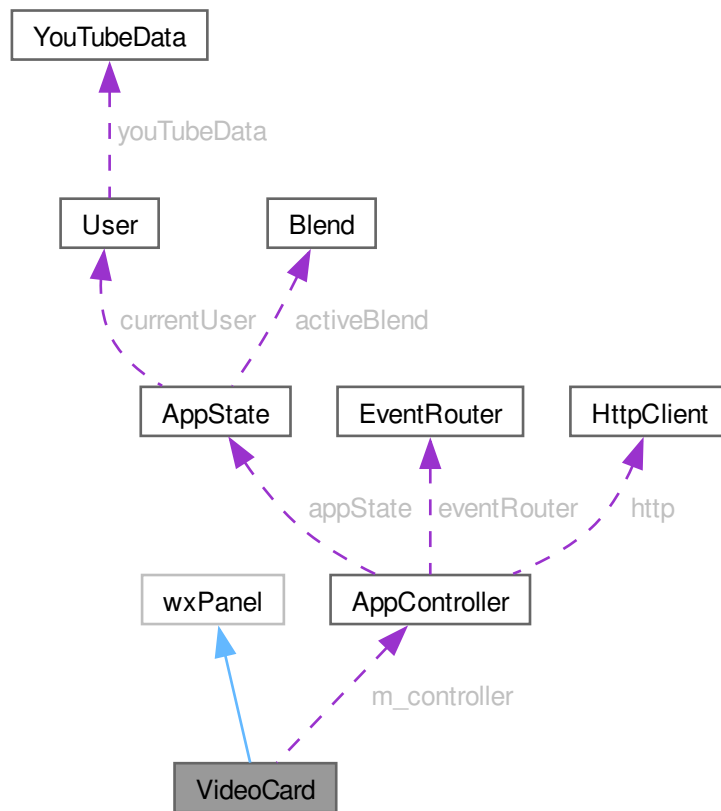
Custom wxPanel widget that displays a single YouTube video preview.

```
#include <VideoCard.hpp>
```

Inheritance diagram for VideoCard:



Collaboration diagram for VideoCard:



Public Member Functions

- **VideoCard** (wxWindow *parent, **AppController** &controller)
- void **SetVideo** (const **Video** &video)
- void **Clear** ()
- wxSize **DoGetBestSize** () const override

Private Member Functions

- void **FetchThumbnailAsync** ()
- void **FetchChannelLogoAsync** ()
- void **OnThumbnailLoaded** (**ThumbnailEvent** &evt)
- void **OnChannelLogoLoaded** (**ThumbnailEvent** &evt)
- void **OnPaint** (wxPaintEvent &evt)
- void **OnMouseEnter** (wxMouseEvent &evt)
- void **OnMouseLeave** (wxMouseEvent &evt)
- void **OnClick** (wxMouseEvent &evt)
- int **DrawWrappedText** (wxGraphicsContext *gc, const wxString &text, int maxLines, double x, double y, double maxWidth) const

Private Attributes

- [AppController](#) & [m_controller](#)
- wxString [m_videoID](#)
- wxString [m_title](#)
- wxString [m_channelID](#)
- wxString [m_channelName](#)
- wxString [m_thumbnailURL](#)
- wxString [m_channelLogoURL](#)
- wxBitmap [m_thumbnail](#)
- bool [m_thumbLoading](#) = false
- bool [m_thumbFailed](#) = false
- int [m_thumbGen](#) = 0

Incremented on each SetVideo/Clear to discard stale fetches.

- wxBitmap [m_channelLogo](#)
- bool [m_logoLoading](#) = false
- bool [m_logoFailed](#) = false
- int [m_logoGen](#) = 0

Incremented on each SetVideo/Clear to discard stale fetches.

- bool [m_hovered](#) = false

6.48.1 Detailed Description

Custom wxPanel widget that displays a single YouTube video preview.

[VideoCard](#) class declaration.

Definition at line 59 of file [VideoCard.hpp](#).

6.48.2 Constructor & Destructor Documentation

6.48.2.1 VideoCard()

```
VideoCard::VideoCard (
    wxWindow * parent,
    AppController & controller)
```

Constructs a [VideoCard](#).

Parameters

<i>parent</i>	Parent window.
<i>controller</i>	Application controller used to dispatch playVideo events.

Definition at line 72 of file [VideoCard.cpp](#).

6.48.3 Member Function Documentation

6.48.3.1 Clear()

```
void VideoCard::Clear ()
```

Resets the card to its empty placeholder state.

Definition at line 114 of file [VideoCard.cpp](#).

6.48.3.2 DoGetBestSize()

```
wxSize VideoCard::DoGetBestSize () const [inline], [override]
```

Definition at line 77 of file [VideoCard.hpp](#).

6.48.3.3 DrawWrappedText()

```
int VideoCard::DrawWrappedText (
    wxGraphicsContext * gc,
    const wxString & text,
    int maxLines,
    double x,
    double y,
    double maxWidth) const [private]
```

Draws word-wrapped text with ellipsis trimming on the last line.

Parameters

<i>gc</i>	Graphics context to draw into.
<i>text</i>	Text to render.
<i>maxLines</i>	Maximum number of lines before truncating.
<i>x</i>	Left edge of the text area.
<i>y</i>	Top edge of the first line.
<i>maxWidth</i>	Maximum pixel width of a line.

Returns

Y coordinate immediately below the last rendered line.

Definition at line 224 of file [VideoCard.cpp](#).

6.48.3.4 FetchChannelLogoAsync()

```
void VideoCard::FetchChannelLogoAsync () [private]
```

Spawns a background thread to fetch and decode the channel logo.

Definition at line 157 of file [VideoCard.cpp](#).

6.48.3.5 FetchThumbnailAsync()

```
void VideoCard::FetchThumbnailAsync () [private]
```

Spawns a background thread to fetch and decode the video thumbnail.

Definition at line 130 of file [VideoCard.cpp](#).

6.48.3.6 OnChannelLogoLoaded()

```
void VideoCard::OnChannelLogoLoaded (  
    ThumbnailEvent & evt) [private]
```

Definition at line 191 of file [VideoCard.cpp](#).

6.48.3.7 OnClick()

```
void VideoCard::OnClick (  
    wxMouseEvent & evt) [private]
```

Definition at line 206 of file [VideoCard.cpp](#).

6.48.3.8 OnMouseEnter()

```
void VideoCard::OnMouseEnter (  
    wxMouseEvent & evt) [private]
```

Definition at line 203 of file [VideoCard.cpp](#).

6.48.3.9 OnMouseLeave()

```
void VideoCard::OnMouseLeave (  
    wxMouseEvent & evt) [private]
```

Definition at line 204 of file [VideoCard.cpp](#).

6.48.3.10 OnPaint()

```
void VideoCard::OnPaint (  
    wxPaintEvent & evt) [private]
```

Definition at line 276 of file [VideoCard.cpp](#).

6.48.3.11 OnThumbnailLoaded()

```
void VideoCard::OnThumbnailLoaded (  
    ThumbnailEvent & evt) [private]
```

Definition at line 179 of file [VideoCard.cpp](#).

6.48.3.12 SetVideo()

```
void VideoCard::SetVideo (  
    const Video & video)
```

Loads a video into the card and starts async image fetches.

Parameters

<i>video</i>	Video whose metadata and images should be displayed.
--------------	--

Definition at line 95 of file [VideoCard.cpp](#).

6.48.4 Member Data Documentation

6.48.4.1 m_channelID

```
wxString VideoCard::m_channelID [private]
```

Definition at line 84 of file [VideoCard.hpp](#).

6.48.4.2 m_channelLogo

```
wxBitmap VideoCard::m_channelLogo [private]
```

Definition at line 94 of file [VideoCard.hpp](#).

6.48.4.3 m_channelLogoURL

```
wxString VideoCard::m_channelLogoURL [private]
```

Definition at line 87 of file [VideoCard.hpp](#).

6.48.4.4 m_channelName

```
wxString VideoCard::m_channelName [private]
```

Definition at line 85 of file [VideoCard.hpp](#).

6.48.4.5 m_controller

```
AppController& VideoCard::m_controller [private]
```

Definition at line 80 of file [VideoCard.hpp](#).

6.48.4.6 m_hovered

```
bool VideoCard::m_hovered = false [private]
```

Definition at line 99 of file [VideoCard.hpp](#).

6.48.4.7 m_logoFailed

```
bool VideoCard::m_logoFailed = false [private]
```

Definition at line 96 of file [VideoCard.hpp](#).

6.48.4.8 m_logoGen

```
int VideoCard::m_logoGen = 0 [private]
```

Incremented on each SetVideo/Clear to discard stale fetches.

Definition at line 97 of file [VideoCard.hpp](#).

6.48.4.9 m_logoLoading

```
bool VideoCard::m_logoLoading = false [private]
```

Definition at line 95 of file [VideoCard.hpp](#).

6.48.4.10 m_thumbFailed

```
bool VideoCard::m_thumbFailed = false [private]
```

Definition at line 91 of file [VideoCard.hpp](#).

6.48.4.11 m_thumbGen

```
int VideoCard::m_thumbGen = 0 [private]
```

Incremented on each SetVideo/Clear to discard stale fetches.

Definition at line 92 of file [VideoCard.hpp](#).

6.48.4.12 m_thumbLoading

```
bool VideoCard::m_thumbLoading = false [private]
```

Definition at line 90 of file [VideoCard.hpp](#).

6.48.4.13 m_thumbnail

```
wxBitmap VideoCard::m_thumbnail [private]
```

Definition at line 89 of file [VideoCard.hpp](#).

6.48.4.14 m_thumbnailURL

```
wxString VideoCard::m_thumbnailURL [private]
```

Definition at line 86 of file [VideoCard.hpp](#).

6.48.4.15 m_title

```
wxString VideoCard::m_title [private]
```

Definition at line 83 of file [VideoCard.hpp](#).

6.48.4.16 m_videoID

```
wxString VideoCard::m_videoID [private]
```

Definition at line 82 of file [VideoCard.hpp](#).

The documentation for this class was generated from the following files:

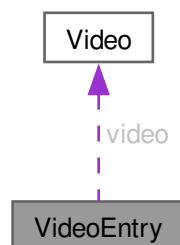
- [VideoCard.hpp](#)
- [VideoCard.cpp](#)

6.49 VideoEntry Class Reference

[VideoEntry](#) class declaration.

```
#include <VideoEntry.hpp>
```

Collaboration diagram for VideoEntry:



Public Member Functions

- [VideoEntry](#) (const [Video](#) &video, int watchCount, int lastWatched)
- [Video](#) [getVideo](#) () const
- int [getWatchCount](#) () const
- int [getLastWatched](#) () const
- void [setWatchCount](#) (int watchCount)
- void [setLastWatched](#) (int lastWatched)
- std::string [toString](#) () const

Private Attributes

- [Video](#) [video](#)
- int [watchCount](#)
- int [lastWatched](#)

6.49.1 Detailed Description

[VideoEntry](#) class declaration.

A [Video](#) decorated with watch-history metadata.

Definition at line 20 of file [VideoEntry.hpp](#).

6.49.2 Constructor & Destructor Documentation

6.49.2.1 VideoEntry()

```
VideoEntry::VideoEntry (  
    const Video & video,  
    int watchCount,  
    int lastWatched)
```

Constructs a [VideoEntry](#).

Parameters

<i>video</i>	The underlying Video object.
<i>watchCount</i>	Number of times the user has watched this video.
<i>lastWatched</i>	Unix timestamp of the most recent watch event.

Definition at line 9 of file [VideoEntry.cpp](#).

6.49.3 Member Function Documentation

6.49.3.1 getLastWatched()

```
int VideoEntry::getLastWatched () const
```

Returns

Unix timestamp of the most recent watch event.

Definition at line 22 of file [VideoEntry.cpp](#).

6.49.3.2 `getVideo()`

```
Video VideoEntry::getVideo () const
```

Returns

The underlying [Video](#).

Definition at line 14 of file [VideoEntry.cpp](#).

6.49.3.3 `getWatchCount()`

```
int VideoEntry::getWatchCount () const
```

Returns

Number of times the user has watched this video.

Definition at line 18 of file [VideoEntry.cpp](#).

6.49.3.4 `setLastWatched()`

```
void VideoEntry::setLastWatched (  
    int lastWatched)
```

Parameters

<i>lastWatched</i>	Updated last-watched Unix timestamp.
--------------------	--------------------------------------

Definition at line 31 of file [VideoEntry.cpp](#).

6.49.3.5 `setWatchCount()`

```
void VideoEntry::setWatchCount (  
    int watchCount)
```

Parameters

<i>watchCount</i>	Updated watch count.
-------------------	----------------------

Definition at line 27 of file [VideoEntry.cpp](#).

6.49.3.6 toString()

```
std::string VideoEntry::toString () const
```

String representation of [VideoEntry](#)

Returns

string

Definition at line 35 of file [VideoEntry.cpp](#).

6.49.4 Member Data Documentation

6.49.4.1 lastWatched

```
int VideoEntry::lastWatched [private]
```

Definition at line 24 of file [VideoEntry.hpp](#).

6.49.4.2 video

```
Video VideoEntry::video [private]
```

Definition at line 22 of file [VideoEntry.hpp](#).

6.49.4.3 watchCount

```
int VideoEntry::watchCount [private]
```

Definition at line 23 of file [VideoEntry.hpp](#).

The documentation for this class was generated from the following files:

- [VideoEntry.hpp](#)
- [VideoEntry.cpp](#)

6.50 YouTubeMetadataFetcher::VideoMeta Struct Reference

[VideoMeta](#) class declaration.

Public Attributes

- std::string [title](#)
- std::string [channelId](#)
- std::string [channelName](#)
- std::string [thumbnailUrl](#)

6.50.1 Detailed Description

[VideoMeta](#) class declaration.

Definition at line 70 of file [YouTubeMetadataFetcher.hpp](#).

6.50.2 Member Data Documentation

6.50.2.1 channelId

```
std::string YouTubeMetadataFetcher::VideoMeta::channelId
```

Definition at line 72 of file [YouTubeMetadataFetcher.hpp](#).

6.50.2.2 channelName

```
std::string YouTubeMetadataFetcher::VideoMeta::channelName
```

Definition at line 73 of file [YouTubeMetadataFetcher.hpp](#).

6.50.2.3 thumbnailUrl

```
std::string YouTubeMetadataFetcher::VideoMeta::thumbnailUrl
```

Definition at line 74 of file [YouTubeMetadataFetcher.hpp](#).

6.50.2.4 title

```
std::string YouTubeMetadataFetcher::VideoMeta::title
```

Definition at line 71 of file [YouTubeMetadataFetcher.hpp](#).

The documentation for this struct was generated from the following file:

- [YouTubeMetadataFetcher.hpp](#)

6.51 WatchHistoryParser Class Reference

[WatchHistoryParser](#) class declaration.

```
#include <WatchHistoryParser.hpp>
```

Public Member Functions

- [WatchHistoryParser](#) (const std::string &filePath)
- std::list< [Video](#) > [parse](#) ()
- std::list< [Video](#) > [parseMultiThreaded](#) (std::function< void(double)> progressCallback=nullptr)
- int [getParserId](#) () const

Static Private Member Functions

- static std::optional< std::string > [extractVideoIdFromHref](#) (const std::string &href)
- static std::list< [Video](#) > [parseRange](#) (const std::string &htmlContent, std::size_t startPos, std::size_t endPos)

Private Attributes

- std::string [filePath](#)
- int [parser_id](#)

6.51.1 Detailed Description

[WatchHistoryParser](#) class declaration.

Definition at line 24 of file [WatchHistoryParser.hpp](#).

6.51.2 Constructor & Destructor Documentation

6.51.2.1 WatchHistoryParser()

```
WatchHistoryParser::WatchHistoryParser (
    const std::string & filePath) [explicit]
```

Definition at line 18 of file [WatchHistoryParser.cpp](#).

6.51.3 Member Function Documentation

6.51.3.1 extractVideoIdFromHref()

```
std::optional< std::string > WatchHistoryParser::extractVideoIdFromHref (
    const std::string & href) [static], [private]
```

Pulls the YouTube video idea from href tags

Parameters

<i>href</i>	tag to extract the id from
-------------	----------------------------

Returns

`string`
of YouTube video id

Definition at line 22 of file [WatchHistoryParser.cpp](#).

6.51.3.2 getParserId()

```
int WatchHistoryParser::getParserId () const
```

Definition at line 109 of file [WatchHistoryParser.cpp](#).

6.51.3.3 parse()

```
std::list< Video > WatchHistoryParser::parse ()
```

Parses watch-history HTML and returns lightweight [Video](#) objects. Only videoID is populated.

Definition at line 65 of file [WatchHistoryParser.cpp](#).

6.51.3.4 parseMultiThreaded()

```
std::list< Video > WatchHistoryParser::parseMultiThreaded (  
    std::function< void(double)> progressCallback = nullptr)
```

Multi-threaded parse using 2-3 worker threads for faster processing. Each thread processes a section of the file to avoid duplicate work.

Parameters

<i>progressCallback</i>	Optional callback to report parsing progress (0.0 to 1.0)
-------------------------	---

Returns

List of unique [Video](#) objects extracted from all threads

Definition at line 113 of file [WatchHistoryParser.cpp](#).

6.51.3.5 parseRange()

```
std::list< Video > WatchHistoryParser::parseRange (  
    const std::string & htmlContent,  
    std::size_t startPos,  
    std::size_t endPos) [static], [private]
```

Parse a range of HTML content.

Parameters

<i>htmlContent</i>	Full HTML file content
<i>startPos</i>	Starting character position for this thread
<i>endPos</i>	Ending character position for this thread

Returns

List of [Video](#) objects found in this section

Definition at line 194 of file [WatchHistoryParser.cpp](#).

6.51.4 Member Data Documentation

6.51.4.1 filePath

```
std::string WatchHistoryParser::filePath [private]
```

Definition at line 45 of file [WatchHistoryParser.hpp](#).

6.51.4.2 parser_id

```
int WatchHistoryParser::parser_id [private]
```

Definition at line 46 of file [WatchHistoryParser.hpp](#).

The documentation for this class was generated from the following files:

- [WatchHistoryParser.hpp](#)
- [WatchHistoryParser.cpp](#)

6.52 WatchLaterParser Class Reference

[WatchLaterParser](#) class declaration.

```
#include <WatchLaterParser.hpp>
```

Public Member Functions

- [WatchLaterParser](#) (const std::string &filePath)
- std::list< [Video](#) > [parse](#) ()
- std::list< [Video](#) > [parseMultiThreaded](#) (std::function< void(double)> progressCallback=nullptr)
- int [getParserId](#) ()

Private Attributes

- std::string [filePath](#)
- int [parser_id](#)

6.52.1 Detailed Description

[WatchLaterParser](#) class declaration.

Specialized parser for YouTube Takeout Watch Later CSV files.

Definition at line 30 of file [WatchLaterParser.hpp](#).

6.52.2 Constructor & Destructor Documentation

6.52.2.1 WatchLaterParser()

```
WatchLaterParser::WatchLaterParser (  
    const std::string & filePath) [explicit]
```

Constructs [WatchLaterParser](#) for the given CSV file.

Parameters

<i>filePath</i>	Path to the YouTube Takeout Watch Later CSV file. Also ensures that the CSV parser id is assigned
-----------------	---

Definition at line 17 of file [WatchLaterParser.cpp](#).

6.52.3 Member Function Documentation

6.52.3.1 getParserId()

```
int WatchLaterParser::getParserId ()
```

Definition at line 47 of file [WatchLaterParser.cpp](#).

6.52.3.2 parse()

```
std::list< Video > WatchLaterParser::parse ()
```

Reads and parses the Watch Later CSV file into [Video](#) objects.

Each returned [Video](#) object has only the videoID field populated; all other fields (title, channelId, etc.) are empty/zero.

Returns

List of [Video](#) objects initialized from Watch Later CSV rows

Definition at line 21 of file [WatchLaterParser.cpp](#).

6.52.3.3 parseMultiThreaded()

```
std::list< Video > WatchLaterParser::parseMultiThreaded (  
    std::function< void(double)> progressCallback = nullptr)
```

Multi-threaded parse using 2-3 worker threads for faster CSV processing. Each thread processes a range of CSV rows to avoid duplicate work.

Parameters

<i>progressCallback</i>	Optional callback to report parsing progress (0.0 to 1.0)
-------------------------	---

Returns

List of [Video](#) objects extracted from all threads

Definition at line 49 of file [WatchLaterParser.cpp](#).

6.52.4 Member Data Documentation

6.52.4.1 filePath

```
std::string WatchLaterParser::filePath [private]
```

Definition at line 58 of file [WatchLaterParser.hpp](#).

6.52.4.2 parser_id

```
int WatchLaterParser::parser_id [private]
```

Definition at line 59 of file [WatchLaterParser.hpp](#).

The documentation for this class was generated from the following files:

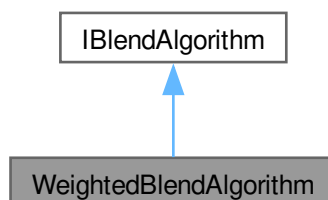
- [WatchLaterParser.hpp](#)
- [WatchLaterParser.cpp](#)

6.53 WeightedBlendAlgorithm Class Reference

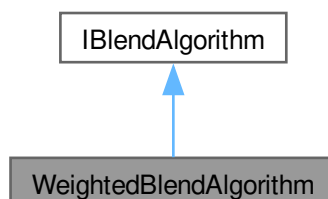
[WeightedBlendAlgorithm](#) class declaration.

```
#include <WeightedBlendAlgorithm.hpp>
```

Inheritance diagram for WeightedBlendAlgorithm:



Collaboration diagram for WeightedBlendAlgorithm:



Public Member Functions

- [WeightedBlendAlgorithm](#) (int [videosPerUser](#)=5)
- [WeightedBlendAlgorithm](#) (const std::vector< double > &weights, int [videosPerUser](#)=5)
- [Blend generateBlend](#) (const std::list< [User](#) > &participants, const std::string &title="") override

Private Member Functions

- void [normalizeWeights](#) (std::vector< double > &weights)
- std::list< [Video](#) > [selectWeightedVideos](#) (const [User](#) &user, double weight)

Private Attributes

- std::vector< double > [participantWeights](#)
- int [videosPerUser](#)

6.53.1 Detailed Description

[WeightedBlendAlgorithm](#) class declaration.

Definition at line 21 of file [WeightedBlendAlgorithm.hpp](#).

6.53.2 Constructor & Destructor Documentation

6.53.2.1 WeightedBlendAlgorithm() [1/2]

```
WeightedBlendAlgorithm::WeightedBlendAlgorithm (
    int videosPerUser = 5) [explicit]
```

Constructs a [WeightedBlendAlgorithm](#) with default weight distribution.

Parameters

<i>videosPerUser</i>	How many videos to select from each participant. If a user has fewer videos than this, all of their videos are included.
----------------------	--

6.53.2.2 WeightedBlendAlgorithm() [2/2]

```
WeightedBlendAlgorithm::WeightedBlendAlgorithm (
    const std::vector< double > & weights,
    int videosPerUser = 5)
```

Constructs a [WeightedBlendAlgorithm](#) with custom participant weights.

Parameters

<i>weights</i>	Vector of weights for each participant, in order. Weights should be normalized or will be normalized internally.
<i>videosPerUser</i>	How many videos to select from each participant.

6.53.3 Member Function Documentation

6.53.3.1 generateBlend()

```
Blend WeightedBlendAlgorithm::generateBlend (  
    const std::list< User > & participants,  
    const std::string & title = "") [override], [virtual]
```

Generates a weighted [Blend](#) from a collection of participants. Each participant contributes videos based on their assigned weight.

Parameters

<i>participants</i>	List of users whose Watch Later data is used as input.
<i>title</i>	User-facing name for the blend (default: empty).

Returns

A new [Blend](#) containing weighted video selections from participants.

Implements [IBlendAlgorithm](#).

6.53.3.2 normalizeWeights()

```
void WeightedBlendAlgorithm::normalizeWeights (  
    std::vector< double > & weights) [private]
```

Normalizes the weight vector so that weights sum to 1.0.

Parameters

<i>weights</i>	Vector of weights to normalize.
----------------	---------------------------------

6.53.3.3 selectWeightedVideos()

```
std::list< Video > WeightedBlendAlgorithm::selectWeightedVideos (  
    const User & user,  
    double weight) [private]
```

Selects videos from a participant based on their weight contribution.

Parameters

<i>user</i>	The participant to select videos from.
<i>weight</i>	The weight factor for this participant (0.0 to 1.0).

Returns

List of selected videos.

6.53.4 Member Data Documentation

6.53.4.1 participantWeights

```
std::vector<double> WeightedBlendAlgorithm::participantWeights [private]
```

Vector of weights assigned to participants.

Definition at line 66 of file [WeightedBlendAlgorithm.hpp](#).

6.53.4.2 videosPerUser

```
int WeightedBlendAlgorithm::videosPerUser [private]
```

Number of videos to attempt to select from each participant.

Definition at line 69 of file [WeightedBlendAlgorithm.hpp](#).

The documentation for this class was generated from the following file:

- [WeightedBlendAlgorithm.hpp](#)

6.54 YouTubeData Class Reference

[YouTubeData](#) class declaration.

```
#include <YouTubeData.hpp>
```

Public Member Functions

- `std::list< VideoEntry > getWatchHistory () const`
- `std::list< Video > getLikedVideos () const`
- `std::list< Channel > getSubscribedChannels () const`
- `std::list< Video > getWatchLaterVideos () const`
- `void setWatchHistory (const std::list< VideoEntry > &watchHistory)`
- `void setLikedVideos (const std::list< Video > &likedVideos)`
- `void setSubscribedChannels (const std::list< Channel > &subscribedChannels)`
- `void setWatchLaterVideos (const std::list< Video > &videos)`
- `std::list< Video > getMostWatchedVideos (int n) const`
- `std::list< Channel > getTopChannels (int n) const`
- `std::string toString () const`

Private Attributes

- `std::list< VideoEntry > watchHistory`
- `std::list< Video > likedVideos`
- `std::list< Channel > subscribedChannels`
- `std::list< Video > watchLaterVideos`

6.54.1 Detailed Description

[YouTubeData](#) class declaration.

Container for all YouTube data associated with a [User](#).

Definition at line 29 of file [YouTubeData.hpp](#).

6.54.2 Constructor & Destructor Documentation

6.54.2.1 YouTubeData()

```
YouTubeData::YouTubeData ()
```

Definition at line 13 of file [YouTubeData.cpp](#).

6.54.3 Member Function Documentation

6.54.3.1 getLikedVideos()

```
std::list< Video > YouTubeData::getLikedVideos () const
```

Returns

List of videos the user has liked.

Definition at line 20 of file [YouTubeData.cpp](#).

6.54.3.2 getMostWatchedVideos()

```
std::list< Video > YouTubeData::getMostWatchedVideos (  
    int n) const
```

Returns the n most-replayed videos from the watch history.

Parameters

<i>n</i>	Maximum number of results to return.
----------	--------------------------------------

Returns

List of up to n Videos sorted by descending watch count.

Definition at line 52 of file [YouTubeData.cpp](#).

6.54.3.3 getSubscribedChannels()

```
std::list< Channel > YouTubeData::getSubscribedChannels () const
```

Returns

List of channels the user is subscribed to.

Definition at line 24 of file [YouTubeData.cpp](#).

6.54.3.4 getTopChannels()

```
std::list< Channel > YouTubeData::getTopChannels (  
    int n) const
```

Returns the n most frequently watched channels from the watch history.

Parameters

<i>n</i>	Maximum number of results to return.
----------	--------------------------------------

Returns

List of up to n Channels sorted by descending view count.

Definition at line 72 of file [YouTubeData.cpp](#).

6.54.3.5 getWatchHistory()

```
std::list< VideoEntry > YouTubeData::getWatchHistory () const
```

Returns

Full watch-history list ([VideoEntry](#) objects with watch counts).

Definition at line 16 of file [YouTubeData.cpp](#).

6.54.3.6 getWatchLaterVideos()

```
std::list< Video > YouTubeData::getWatchLaterVideos () const
```

Returns

Watch Later playlist videos.

Definition at line 42 of file [YouTubeData.cpp](#).

6.54.3.7 setLikedVideos()

```
void YouTubeData::setLikedVideos (  
    const std::list< Video > & likedVideos)
```

Parameters

<i>likedVideos</i>	Replaces the liked-videos list.
--------------------	---------------------------------

Definition at line 34 of file [YouTubeData.cpp](#).

6.54.3.8 setSubscribedChannels()

```
void YouTubeData::setSubscribedChannels (
    const std::list< Channel > & subscribedChannels)
```

Parameters

<i>subscribedChannels</i>	Replaces the subscribed-channels list.
---------------------------	--

Definition at line 38 of file [YouTubeData.cpp](#).

6.54.3.9 setWatchHistory()

```
void YouTubeData::setWatchHistory (
    const std::list< VideoEntry > & watchHistory)
```

Parameters

<i>watchHistory</i>	Replaces the full watch-history list.
---------------------	---------------------------------------

Definition at line 30 of file [YouTubeData.cpp](#).

6.54.3.10 setWatchLaterVideos()

```
void YouTubeData::setWatchLaterVideos (
    const std::list< Video > & videos)
```

Parameters

<i>videos</i>	Replaces the Watch Later playlist.
---------------	------------------------------------

Definition at line 46 of file [YouTubeData.cpp](#).

6.54.3.11 toString()

```
std::string YouTubeData::toString () const
```

Returns

JSON string representation of this [YouTubeData](#) object.

Definition at line 99 of file [YouTubeData.cpp](#).

6.54.4 Member Data Documentation

6.54.4.1 likedVideos

```
std::list<Video> YouTubeData::likedVideos [private]
```

Definition at line 32 of file [YouTubeData.hpp](#).

6.54.4.2 subscribedChannels

```
std::list<Channel> YouTubeData::subscribedChannels [private]
```

Definition at line 33 of file [YouTubeData.hpp](#).

6.54.4.3 watchHistory

```
std::list<VideoEntry> YouTubeData::watchHistory [private]
```

Definition at line 31 of file [YouTubeData.hpp](#).

6.54.4.4 watchLaterVideos

```
std::list<Video> YouTubeData::watchLaterVideos [private]
```

Definition at line 34 of file [YouTubeData.hpp](#).

The documentation for this class was generated from the following files:

- [YouTubeData.hpp](#)
- [YouTubeData.cpp](#)

6.55 YouTubeDataImporter Class Reference

[YouTubeDataImporter](#) class declaration.

```
#include <YouTubeDataImporter.hpp>
```

Public Member Functions

- `std::list< Video > import` (const std::string &filePath) const
- `std::list< Video > importMultiThreaded` (const std::string &filePath, std::function< void(double)> progress↔ Callback=nullptr) const

Static Private Member Functions

- static std::string [lowerExtension](#) (const std::string &filePath)

6.55.1 Detailed Description

[YouTubeDataImporter](#) class declaration.

Definition at line 21 of file [YouTubeDataImporter.hpp](#).

6.55.2 Member Function Documentation

6.55.2.1 import()

```
std::list< Video > YouTubeDataImporter::import (
    const std::string & filePath) const
```

Imports YouTube data from a supported file path.

Supported extensions:

- .csv : Watch Later export
- .html/.htm : watch-history export

Parameters

<i>filePath</i>	path to the specified file
-----------------	----------------------------

Returns

```
list<video>
extracted and formatted from
filePath
```

Definition at line 28 of file [YouTubeDataImporter.cpp](#).

6.55.2.2 importMultiThreaded()

```
std::list< Video > YouTubeDataImporter::importMultiThreaded (
    const std::string & filePath,
    std::function< void(double)> progressCallback = nullptr) const
```

Multi-threaded import with progress feedback. Uses 3 worker threads to process the file faster.

Parameters

<i>filePath</i>	Path to YouTube export file
<i>progressCallback</i>	Optional callback receiving progress 0.0-1.0

Returns

List of [Video](#) objects extracted from file (ID-only)

Definition at line 44 of file [YouTubeDataImporter.cpp](#).

6.55.2.3 lowerExtension()

```
std::string YouTubeDataImporter::lowerExtension (
    const std::string & filePath) [static], [private]
```

Helper to ensure extensions are always lowered

Parameters

<i>filePath</i>	desired file
-----------------	--------------

Returns

`string`
of the file path with the lowered extension

Definition at line 16 of file [YouTubeDataImporter.cpp](#).

The documentation for this class was generated from the following files:

- [YouTubeDataImporter.hpp](#)
- [YouTubeDataImporter.cpp](#)

6.56 YouTubeMetadataFetcher Class Reference

[YouTubeMetadataFetcher](#) class declaration.

```
#include <YouTubeMetadataFetcher.hpp>
```

Classes

- struct [VideoMeta](#)
[VideoMeta](#) class declaration.

Public Member Functions

- [YouTubeMetadataFetcher](#) (std::string apiKey)
- std::list< [Video](#) > [enrich](#) (const std::list< [Video](#) > &videos)

Private Member Functions

- std::unordered_map< std::string, [VideoMeta](#) > [fetchVideoBatch](#) (const std::vector< std::string > &ids)
- std::unordered_map< std::string, std::string > [fetchChannelLogos](#) (const std::vector< std::string > &channelIds)
- std::string [httpGet](#) (const std::string &url)

Static Private Member Functions

- static size_t [writeCallback](#) (void *contents, size_t size, size_t nmemb, std::string *out)

Private Attributes

- std::string [apiKey](#)

Static Private Attributes

- static constexpr int `kBatchSize` = 50

6.56.1 Detailed Description

[YouTubeMetadataFetcher](#) class declaration.

Definition at line 50 of file [YouTubeMetadataFetcher.hpp](#).

6.56.2 Constructor & Destructor Documentation

6.56.2.1 YouTubeMetadataFetcher()

```
YouTubeMetadataFetcher::YouTubeMetadataFetcher (  
    std::string apiKey) [explicit]
```

Definition at line 23 of file [YouTubeMetadataFetcher.cpp](#).

6.56.3 Member Function Documentation

6.56.3.1 enrich()

```
std::list< Video > YouTubeMetadataFetcher::enrich (  
    const std::list< Video > & videos)
```

Enriches a list of Videos that only have videoID populated. Returns a new list where each [Video](#) has title, channelID, channelName, thumbnailURL, and channelLogoURL filled in from the API.

Definition at line 28 of file [YouTubeMetadataFetcher.cpp](#).

6.56.3.2 fetchChannelLogos()

```
std::unordered_map< std::string, std::string > YouTubeMetadataFetcher::fetchChannelLogos (  
    const std::vector< std::string > & channelIds) [private]
```

Definition at line 163 of file [YouTubeMetadataFetcher.cpp](#).

6.56.3.3 fetchVideoBatch()

```
std::unordered_map< std::string, YouTubeMetadataFetcher::VideoMeta > YouTubeMetadataFetcher↔  
::fetchVideoBatch (  
    const std::vector< std::string > & ids) [private]
```

Definition at line 112 of file [YouTubeMetadataFetcher.cpp](#).

6.56.3.4 httpGet()

```
std::string YouTubeMetadataFetcher::httpGet (  
    const std::string & url) [private]
```

Definition at line 207 of file [YouTubeMetadataFetcher.cpp](#).

6.56.3.5 writeCallback()

```
size_t YouTubeMetadataFetcher::writeCallback (  
    void * contents,  
    size_t size,  
    size_t nmemb,  
    std::string * out) [static], [private]
```

Definition at line 234 of file [YouTubeMetadataFetcher.cpp](#).

6.56.4 Member Data Documentation

6.56.4.1 apiKey

```
std::string YouTubeMetadataFetcher::apiKey [private]
```

Definition at line 62 of file [YouTubeMetadataFetcher.hpp](#).

6.56.4.2 kBatchSize

```
int YouTubeMetadataFetcher::kBatchSize = 50 [static], [constexpr], [private]
```

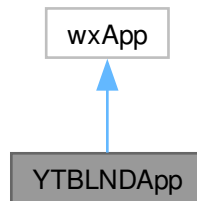
Definition at line 64 of file [YouTubeMetadataFetcher.hpp](#).

The documentation for this class was generated from the following files:

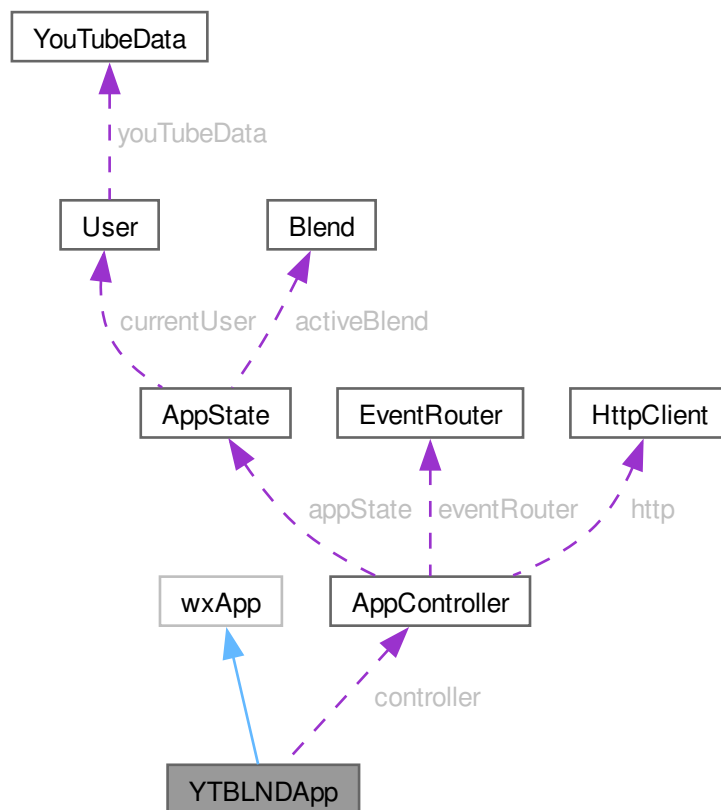
- [YouTubeMetadataFetcher.hpp](#)
- [YouTubeMetadataFetcher.cpp](#)

6.57 YTBLNDApp Class Reference

Inheritance diagram for YTBLNDApp:



Collaboration diagram for YTBLNDApp:



Public Member Functions

- bool `OnInit` () override

- int [OnExit](#) () override

Private Attributes

- [AppController](#) controller

6.57.1 Detailed Description

Definition at line 26 of file [main.cpp](#).

6.57.2 Member Function Documentation

6.57.2.1 OnExit()

```
int YTBLNDApp::OnExit () [override]
```

Definition at line 56 of file [main.cpp](#).

6.57.2.2 OnInit()

```
bool YTBLNDApp::OnInit () [override]
```

Definition at line 39 of file [main.cpp](#).

6.57.3 Member Data Documentation

6.57.3.1 controller

```
AppController YTBLNDApp::controller [private]
```

Definition at line 34 of file [main.cpp](#).

The documentation for this class was generated from the following file:

- [main.cpp](#)

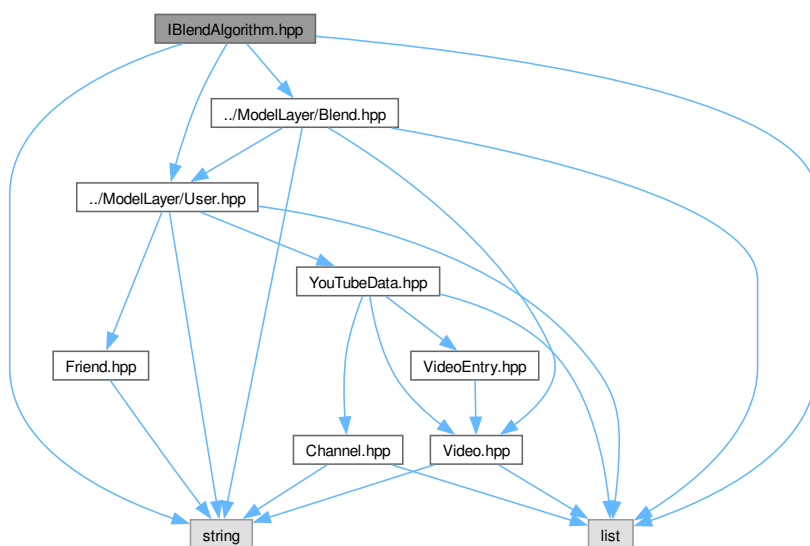
Chapter 7

File Documentation

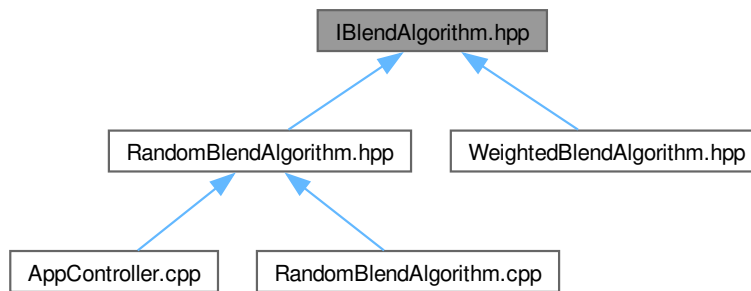
7.1 IBlendAlgorithm.hpp File Reference

Abstract interface for blend-generation algorithms.

```
#include <list>
#include <string>
#include "../ModelLayer/User.hpp"
#include "../ModelLayer/Blend.hpp"
Include dependency graph for IBlendAlgorithm.hpp:
```



This graph shows which files directly or indirectly include this file:



Classes

- class [IBlendAlgorithm](#)
[IBlendAlgorithm](#) class declaration.

7.1.1 Detailed Description

Abstract interface for blend-generation algorithms.

Author

Jasmine Kumar

All blend algorithms must implement this interface so [AppController](#) can swap strategies without changing any other code.

Definition in file [IBlendAlgorithm.hpp](#).

7.2 IBlendAlgorithm.hpp

[Go to the documentation of this file.](#)

```

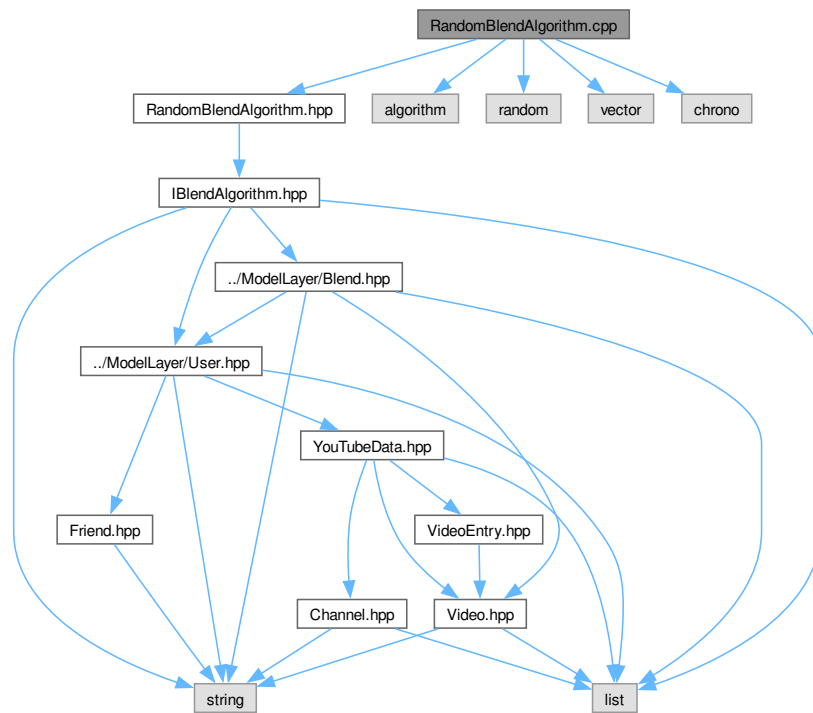
00001 #ifndef IBLEND_ALGORITHM_H
00002 #define IBLEND_ALGORITHM_H
00003
00012
00013 #include <list>
00014 #include <string>
00015 #include "../ModelLayer/User.hpp"
00016 #include "../ModelLayer/Blend.hpp"
00017
00023 class IBlendAlgorithm {
00024 public:
00025     virtual ~IBlendAlgorithm() = default;
00026
00033     virtual Blend generateBlend(const std::list<User>& participants,
00034                                const std::string& title = "") = 0;
00035 };
00036
00037 #endif
  
```


7.3 RandomBlendAlgorithm.cpp File Reference

Implementation for [RandomBlendAlgorithm](#).

```
#include "RandomBlendAlgorithm.hpp"
#include <algorithm>
#include <random>
#include <vector>
#include <chrono>
```

Include dependency graph for RandomBlendAlgorithm.cpp:



7.3.1 Detailed Description

Implementation for [RandomBlendAlgorithm](#).

Author

Jasmine Kumar

Definition in file [RandomBlendAlgorithm.cpp](#).

7.4 RandomBlendAlgorithm.cpp

[Go to the documentation of this file.](#)

```

00001
00006
00007 #include "RandomBlendAlgorithm.hpp"
00008
00009 #include <algorithm>
00010 #include <random>
00011 #include <vector>
00012 #include <chrono>
00013
00014 RandomBlendAlgorithm::RandomBlendAlgorithm(int videosPerUser)
00015     : videosPerUser(videosPerUser) {}
00016
00017 Blend RandomBlendAlgorithm::generateBlend(const std::list<User>& participants,
00018                                           const std::string& title) {
00019     std::list<Video> combined;
00020
00021     // Seed RNG from the system clock so each blend is different
00022     unsigned seed = static_cast<unsigned>(
00023         std::chrono::steady_clock::now().time_since_epoch().count());
00024     std::mt19937 rng(seed);
00025
00026     // collect a global pool of available videos (for filling empty slots)
00027     std::vector<Video> globalPool;
00028
00029     for (const User& user : participants) {
00030         std::list<Video> watchLater = user.getYouTubeData().getWatchLaterVideos();
00031
00032         // Copy to vector for random_shuffle / sample
00033         std::vector<Video> pool(watchLater.begin(), watchLater.end());
00034         std::shuffle(pool.begin(), pool.end(), rng);
00035
00036         int poolSize = static_cast<int>(pool.size());
00037         // add pool contents to global pool for later filling
00038         if (poolSize > 0) {
00039             globalPool.insert(globalPool.end(), pool.begin(), pool.end());
00040             // Fill videosPerUser slots for this user (allow duplicates by wrapping)
00041             for (int i = 0; i < videosPerUser; ++i) {
00042                 combined.push_back(pool[i % poolSize]);
00043             }
00044         }
00045     }
00046
00047     // If some users had no videos, fill remaining slots from the global pool
00048     int desiredTotal = static_cast<int>(participants.size()) * videosPerUser;
00049     if (static_cast<int>(combined.size()) < desiredTotal && !globalPool.empty()) {
00050         std::uniform_int_distribution<size_t> dist(0, globalPool.size() - 1);
00051         while (static_cast<int>(combined.size()) < desiredTotal) {
00052             combined.push_back(globalPool[dist(rng)]);
00053         }
00054     }
00055
00056     // Shuffle final selection so videos are mixed across users
00057     std::vector<Video> finalVec(combined.begin(), combined.end());
00058     std::shuffle(finalVec.begin(), finalVec.end(), rng);
00059     combined.assign(finalVec.begin(), finalVec.end());
00060
00061     // Build a simple blendID from participant IDs
00062     std::string blendID;
00063     for (const User& user : participants) {
00064         if (!blendID.empty()) blendID += "_";
00065         blendID += user.getUserID();
00066     }
00067     blendID += "_random";
00068
00069     return Blend(blendID, title, "random", participants, combined);
00070 }

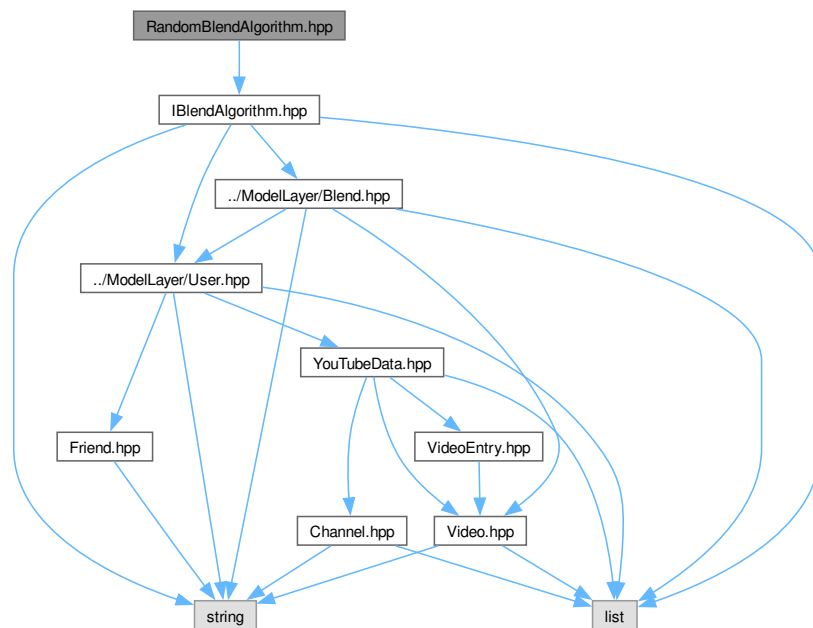
```

7.5 RandomBlendAlgorithm.hpp File Reference

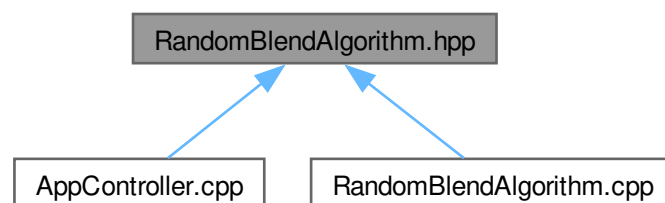
Random [Blend](#) implementation of [IBlendAlgorithm](#).

```
#include "IBlendAlgorithm.hpp"
```

Include dependency graph for RandomBlendAlgorithm.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [RandomBlendAlgorithm](#)
RandomBlendAlgorithm class declaration.

7.5.1 Detailed Description

Random [Blend](#) implementation of [IBlendAlgorithm](#).

Author

Jasmine Kumar

Blend algorithm that picks a random subset of videos from each participant's Watch Later list and combines them into one **Blend**.

Definition in file [RandomBlendAlgorithm.hpp](#).

7.6 RandomBlendAlgorithm.hpp

[Go to the documentation of this file.](#)

```

00001 #ifndef RANDOM_BLEND_ALGORITHM_H
00002 #define RANDOM_BLEND_ALGORITHM_H
00003
00004 #include "IBlendAlgorithm.hpp"
00005
00018 class RandomBlendAlgorithm : public IBlendAlgorithm {
00019 public:
00025     explicit RandomBlendAlgorithm(int videosPerUser = 5);
00026
00030     Blend generateBlend(const std::list<User>& participants,
00031                       const std::string& title = "") override;
00032
00033 private:
00034     int videosPerUser;
00035 };
00036
00037 #endif

```

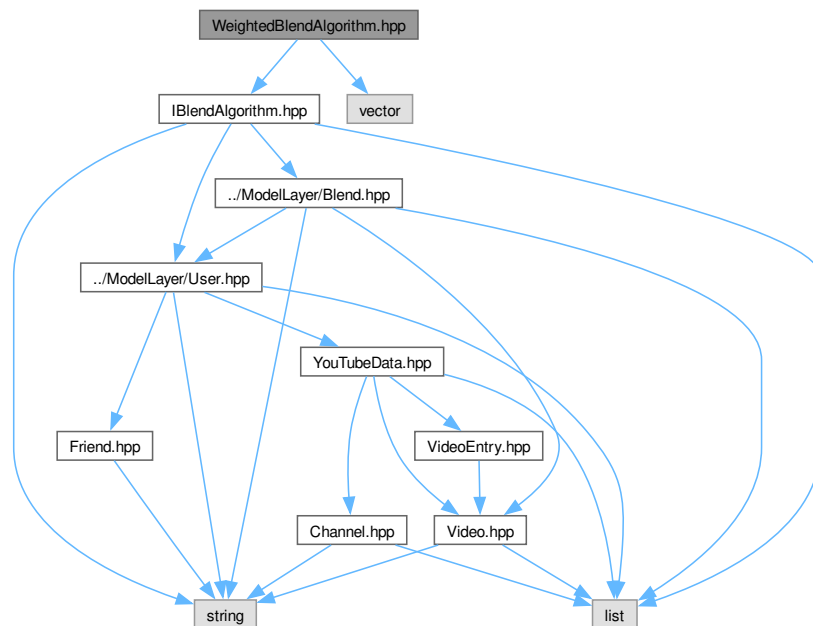
7.7 WeightedBlendAlgorithm.hpp File Reference

Weighted **Blend** implementation of [IBlendAlgorithm](#).

```
#include "IBlendAlgorithm.hpp"
```

```
#include <vector>
```

Include dependency graph for [WeightedBlendAlgorithm.hpp](#):



Classes

- class [WeightedBlendAlgorithm](#)
[WeightedBlendAlgorithm](#) class declaration.

7.7.1 Detailed Description

Weighted [Blend](#) implementation of [IBlendAlgorithm](#).

Author

Shamar Pennant

Jasmine Kumar

[Blend](#) algorithm that generates a blend by weighing contributions from each participant. Participants with higher weights will have more of their videos selected for the final blend.

Definition in file [WeightedBlendAlgorithm.hpp](#).

7.8 WeightedBlendAlgorithm.hpp

[Go to the documentation of this file.](#)

```

00001 #ifndef WEIGHTED_BLEND_ALGORITHM_H
00002 #define WEIGHTED_BLEND_ALGORITHM_H
00003
00004 #include "IBlendAlgorithm.hpp"
00005 #include <vector>
00006
00021 class WeightedBlendAlgorithm : public IBlendAlgorithm {
00022 public:
00029     explicit WeightedBlendAlgorithm(int videosPerUser = 5);
00030
00037     WeightedBlendAlgorithm(const std::vector<double>& weights, int videosPerUser = 5);
00038
00047     Blend generateBlend(const std::list<User>& participants,
00048                        const std::string& title = "") override;
00049
00050 private:
00055     void normalizeWeights(std::vector<double>& weights);
00056
00063     std::list<Video> selectWeightedVideos(const User& user, double weight);
00064
00066     std::vector<double> participantWeights;
00067
00069     int videosPerUser;
00070 };
00071
00072 #endif

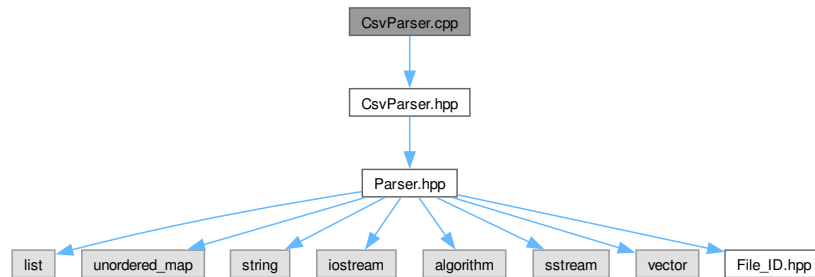
```

7.9 CsvParser.cpp File Reference

Implementation for [CsvParser](#).

```
#include "CsvParser.hpp"
```

Include dependency graph for CsvParser.cpp:



7.9.1 Detailed Description

Implementation for [CsvParser](#).

Author

Shamar Pennant

Definition in file [CsvParser.cpp](#).

7.10 CsvParser.cpp

[Go to the documentation of this file.](#)

```

00001
00006
00007
00008
00009 #include "CsvParser.hpp"
00010
00011 using namespace std;
00012
00013 CsvParser::CsvParser() {parse_id = File_ID::CSV;}
00014 CsvParser::CsvParser(list<string> raw_data) : raw_data(raw_data) {
00015     parse_id = File_ID::CSV;
00016 }
00017 CsvParser::~CsvParser() {}
00018
00019 list<unordered_map<string, string> > CsvParser::parse() {
00020     // check if raw_data is empty
00021     if (raw_data.empty()) {
00022         cerr << "error: raw data string is empty in CsvParser";
00023         exit(1);
00024     }
00025
00026     // returned value at end of function
00027     list<unordered_map<string, string> > ret;
00028
00029     // header contents
00030     vector<string> col_titles;
00031
00032     int line_num = 0;
00033     // loop through the raw_data list until it's empty
00034     while (!raw_data.empty()) {
00035         // grab the next line and advance
00036         string line = raw_data.front();
00037         raw_data.pop_front();
00038

```

```

00039         if (line_num == 0) {
00040             /*
00041              * When on the first line (the col titles)
00042              * save their names within col_titles to be
00043              * used later. formatting each dictionary
00044              * contents as key = col_title and value =
00045              * row_entry
00046              */
00047             stringstream temp_ss(line);
00048             string t = "";
00049             while (getline(temp_ss, t, ',')) {
00050                 // trim any white space/newline char
00051                 if (!t.empty() && t.back() == '\\n') {t.pop_back();}
00052                 // save col
00053                 col_titles.push_back(t);
00054             }
00055         } else { // for any other line, they're normal rows
00056             int num_col = 0;
00057             unordered_map<string, string> row_contents;
00058
00059             /* delimit the line/row and place key value pay
00060              * <"col_name", "row_value"> inside the dictionary*/
00061             stringstream temp_ss(line);
00062             string t = "";
00063             while (getline(temp_ss, t, ',')) {
00064                 if (!t.empty() && t.back() == '\\n') {t.pop_back();}
00065                 row_contents[col_titles.at(num_col++)] = t;
00066             }
00067
00068             // add the dictionary as a line to ret
00069             ret.push_back(row_contents);
00070         }
00071     }
00072     line_num++;
00073 }
00074
00075 return ret;
00076 }
00077
00078 void CsvParser::setData(list<string> data) {this->raw_data = data;}
00079 int CsvParser::getParseId() {return parse_id;}

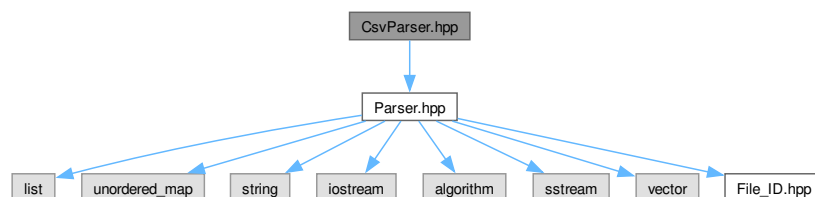
```

7.11 CsvParser.hpp File Reference

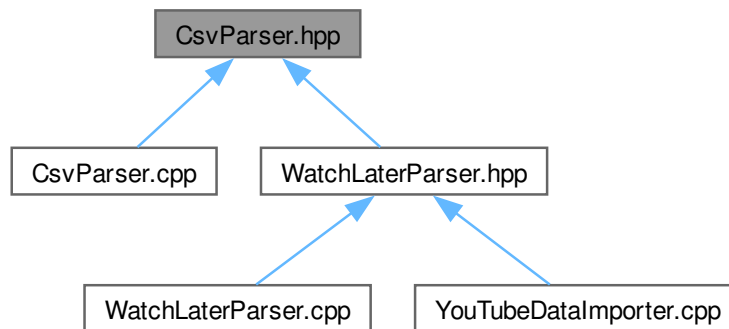
CSV file parser implementing the [Parser](#) interface.

```
#include "Parser.hpp"
```

Include dependency graph for CsvParser.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [CsvParser](#)
CsvParser class declaration.

7.11.1 Detailed Description

CSV file parser implementing the [Parser](#) interface.

Author

Shamar Pennant

Parses comma-separated values into a `list<Dict<string,string>>` format where the first row is treated as column headers.

Definition in file [CsvParser.hpp](#).

7.12 CsvParser.hpp

[Go to the documentation of this file.](#)

```

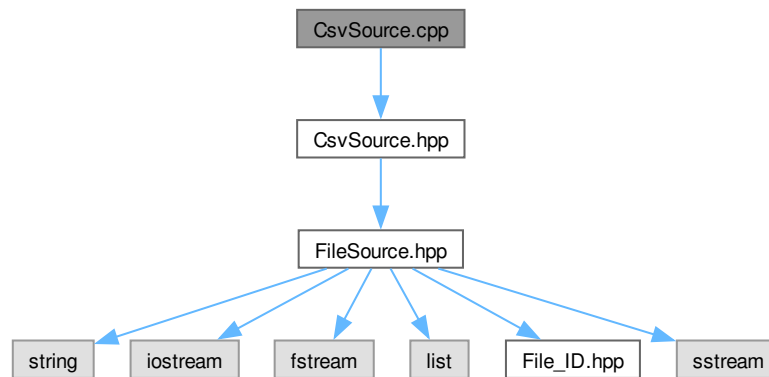
00001
00009
00010 #pragma once
00011
00012 #include "Parser.hpp"
00013
00019 class CsvParser : public Parser {
00020     public:
00021         CsvParser();
00026         CsvParser(std::list<std::string> raw_data);
00027         ~CsvParser();
00028
00034         std::list<std::unordered_map<std::string, std::string>> parse() override;
00035
00040         void setData(std::list<std::string> data) override;
00041
00043         int getParseId() override;
00044
00045     private:
00046         std::list<std::string> raw_data;
00047         int parse_id;
00048 };
  
```


7.13 CsvSource.cpp File Reference

Implementation for [CsvSource](#).

```
#include "CsvSource.hpp"
```

Include dependency graph for CsvSource.cpp:



7.13.1 Detailed Description

Implementation for [CsvSource](#).

Author

Shamar Pennant

Definition in file [CsvSource.cpp](#).

7.14 CsvSource.cpp

[Go to the documentation of this file.](#)

```

00001
00006
00007 #include "CsvSource.hpp"
00008
00009 using namespace std;
00010
00011 CsvSource::CsvSource(string src) : src(src) {
00012     source_id = File_ID::CSV;
00013 }
00014
00015 list<string> CsvSource::read() {
00016     if (src.empty()) { // in the event that src is empty
00017         cerr << "error: src string is empty in CsvSource";
00018         exit(1);
00019     }
00020
00021     // returned string container
00022     list<string> ret;
00023
00024     // creating (in) file stream with given file source path
  
```

```

00025     fstream read_file_in(src, ios::in);
00026     if (!read_file_in.is_open()) {
00027         cerr << "unable to open " << src << " within CsvSource" << endl;
00028         exit(1);
00029     }
00030
00031     if (!read_file_in.is_open()) {
00032         cerr << "error: could not open file: " << src;
00033         exit(1);
00034     }
00035
00036     // holds file line
00037     string line = "";
00038
00039     // reading the opened file stream into a temporary string
00040     while (getline(read_file_in, line)) {ret.push_back(line);}
00041
00042     return ret;
00043 }
00044
00045 void CsvSource::setSource(string src) {this->src = src;}
00046 int CsvSource::getSourceId() {return source_id;}

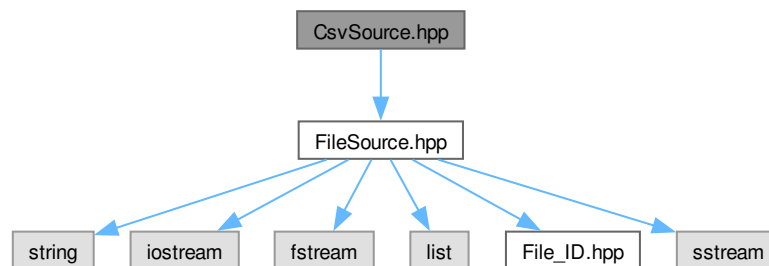
```

7.15 CsvSource.hpp File Reference

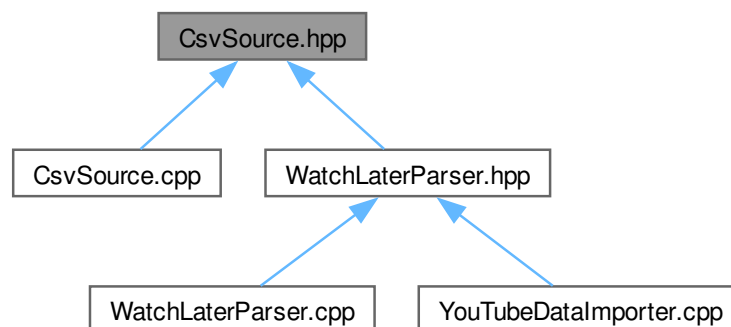
CSV file reader implementing the [FileSource](#) interface.

```
#include "FileSource.hpp"
```

Include dependency graph for CsvSource.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [CsvSource](#)
CsvSource class declaration.

7.15.1 Detailed Description

CSV file reader implementing the [FileSource](#) interface.

Author

Shamar Pennant

Reads a CSV file line-by-line and returns each line as a string in a list, ready to be handed to a [CsvParser](#).

Definition in file [CsvSource.hpp](#).

7.16 CsvSource.hpp

[Go to the documentation of this file.](#)

```

00001
00009
00010 #pragma once
00011
00012 #include "FileSource.hpp"
00013
00018 class CsvSource : public FileSource {
00019     public:
00024         CsvSource(std::string src);
00025         ~CsvSource() override = default;
00026
00031         std::list<std::string> read() override;
00032
00037         void setSource(std::string src) override;
00038
00040         int getSourceId() override;
00041
00042     private:
00043         std::string src;
00044         int source_id;
00045 };

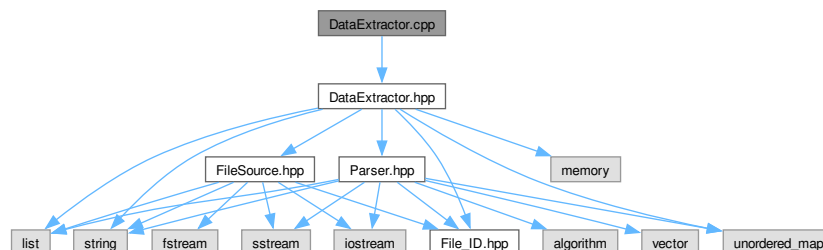
```

7.17 DataExtractor.cpp File Reference

[DataExtractor](#) class implementation.

```
#include "DataExtractor.hpp"
```

Include dependency graph for DataExtractor.cpp:



7.17.1 Detailed Description

[DataExtractor](#) class implementation.

Author

Shamar Pennant

Implements the [DataExtractor](#) class which coordinates [FileSource](#) and [Parser](#) objects to read and parse file data into structured format.

Definition in file [DataExtractor.cpp](#).

7.18 DataExtractor.cpp

[Go to the documentation of this file.](#)

```

00001
00009
00010 #include "DataExtractor.hpp"
00011
00012 using namespace std;
00013
00014 DataExtractor::DataExtractor(unique_ptr<FileSource> src, unique_ptr<Parser> p) {
00015     // change ownership of the param to this objects instance
00016     src_format = move(src);
00017     par_format = move(p);
00018 }
00019
00020 DataExtractor::~DataExtractor() {}
00021
00022 list<unordered_map<string, string>> DataExtractor::extract() {
00023     // grab the respective file format ids
00024     int source_id_in_use = src_format->getSourceId();
00025     int parse_id_in_use = par_format->getParseId();
00026
00027     // check if either id's are out of bounds (they shouldn't be)
00028     if (source_id_in_use <= -1 || parse_id_in_use <= -1) {
00029         cerr << "One of the provided ID's is out of bounds:\nsrc_id = " << source_id_in_use <<
00030             "\nparse_id = " << parse_id_in_use;
00031         exit(1);
00032     }
00033
00034     // check if id's match (how we know they match formats)
00035     if (source_id_in_use != parse_id_in_use) {
00036         cerr << "selected FileSource and Parser id's don't match: \nsrc_id = " << source_id_in_use <<
00037             "\nparse_id = " << parse_id_in_use;
00038         exit(1);
00039     }
00040
00041     // ensure neither pointers are null
00042     if (src_format == nullptr || par_format == nullptr) {
00043         cerr << "Unable to find FileSource or Parser in extract():\nsrc_id = "
00044             << source_id_in_use << ", " << (src_format == nullptr ? "contains nothing (nullptr)" : "has
obj")
00045             << "\nparse_id = "
00046             << parse_id_in_use << ", " << (par_format == nullptr ? "contains nothing (nullptr)" : "has
obj");
00047         exit(1);
00048     }
00049
00050     // --- assuming the both source and parser are already configured ---
00051
00052     // grab the raw data from the file
00053     auto raw_data = src_format->read();
00054     // feed the raw data into the given parser
00055     par_format->setData(raw_data);
00056     // format the data with the repsective parser
00057     list<unordered_map<string, string>> formatted_data = par_format->parse();
00058
00059     return formatted_data;
00060 }
00061
00062 void DataExtractor::setParser(unique_ptr<Parser> parser) {
00063     par_format = move(parser);
00064 }
00065
00066 void DataExtractor::setSource(unique_ptr<FileSource> src) {
00067     src_format = move(src);
00068 }

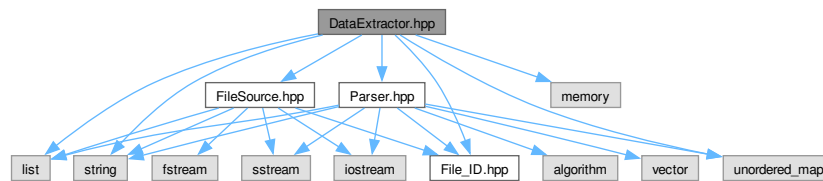
```

7.19 DataExtractor.hpp File Reference

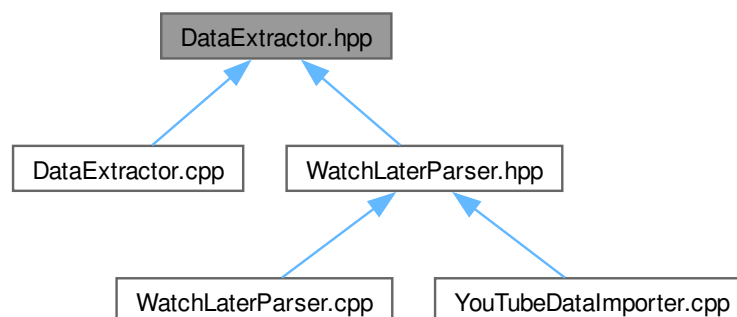
Extracts raw file data and parses it into structured format.

```
#include <list>
#include <memory>
#include <string>
#include <unordered_map>
#include "FileSource.hpp"
#include "Parser.hpp"
#include "File_ID.hpp"
```

Include dependency graph for DataExtractor.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [DataExtractor](#)
DataExtractor class declaration.

7.19.1 Detailed Description

Extracts raw file data and parses it into structured format.

Author

Shamar Pennant

Coordinates file reading (via [FileSource](#)) and data parsing (via [Parser](#)) to produce a consistent list<Dict<str,str>> format. Validates that [FileSource](#) and [Parser](#) objects have matching file format IDs before processing.

Example usage:

```
auto source = std::make_unique<CsvSource>("data.csv");
auto parser = std::make_unique<CsvParser>();
DataExtractor extractor(std::move(source), std::move(parser));
auto data = extractor.extract();
```

Definition in file [DataExtractor.hpp](#).

7.20 DataExtractor.hpp

[Go to the documentation of this file.](#)

```
00001 #pragma once
00002
00003 #include <list>
00004 #include <memory>
00005 #include <string>
00006 #include <unordered_map>
00007
00008 #include "FileSource.hpp"
00009 #include "Parser.hpp"
00010 #include "File_ID.hpp"
00011 #include <memory>
00012
00035 class DataExtractor {
00036     public:
00042         DataExtractor(std::unique_ptr<FileSource> src, std::unique_ptr<Parser> p);
00043
00044         ~DataExtractor();
00045
00046         std::list<std::unordered_map<std::string, std::string>> extract();
00056
00057         void setSource(std::unique_ptr<FileSource> src);
00061
00062         void setParser(std::unique_ptr<Parser> p);
00066
00067     private:
00068         std::unique_ptr<FileSource> src_format;
00069         std::unique_ptr<Parser> par_format;
00070
00071 };
```

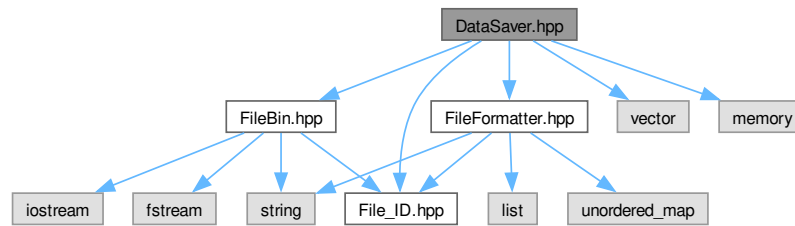
7.21 DataSaver.hpp File Reference

Coordinates [FileFormatter](#) and [FileBin](#) to persist structured data to files.

```
#include "FileBin.hpp"
#include "FileFormatter.hpp"
#include <vector>
#include <memory>
```

```
#include "File_ID.hpp"
```

Include dependency graph for DataSaver.hpp:



Classes

- class [DataSaver](#)
DataSaver class declaration.

7.21.1 Detailed Description

Coordinates [FileFormatter](#) and [FileBin](#) to persist structured data to files.

Author

Shamar Pennant

Maintains collections of [FileBin](#) and [FileFormatter](#) objects to support saving `list<Dict<string,string>>` data in multiple file formats simultaneously.

Definition in file [DataSaver.hpp](#).

7.22 DataSaver.hpp

[Go to the documentation of this file.](#)

```

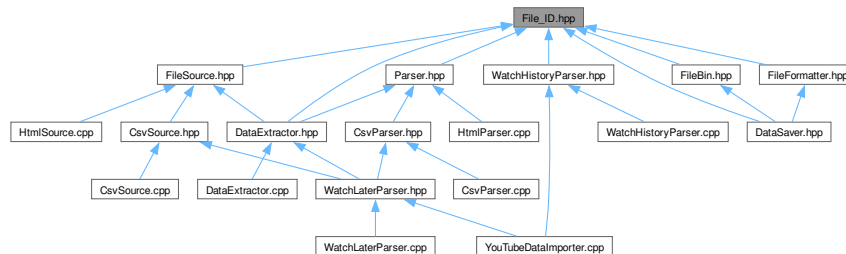
00001 #pragma once
00002
00011
00012 #include "FileBin.hpp"
00013 #include "FileFormatter.hpp"
00014 #include <vector>
00015 #include <memory>
00016 #include "File_ID.hpp"
00017
00022 class DataSaver {
00023     public:
00029         DataSaver(FileBin b, FileFormatter ff);
00030
00031         ~DataSaver();
00032
00038         int save(std::list<std::unordered_map<std::string, std::string>> data);
00039
00044         void setFileBin(FileBin b);
00045
00050         void setFileFormatter(FileFormatter ff);
00051
00052     private:
00053         std::vector<std::unique_ptr<FileBin>> bins;
00054         std::vector<std::unique_ptr<FileFormatter>> formatters;
00055 };

```

7.23 File_ID.hpp File Reference

File format ID constants.

This graph shows which files directly or indirectly include this file:



Classes

- class [File_ID](#)
[File_ID](#) class declaration.

7.23.1 Detailed Description

File format ID constants.

Author

Shamar Pennant

Defines static integer constants mapping file types to unique identifiers. Used to validate that [FileSource](#) and [Parser](#) objects are compatible.

Definition in file [File_ID.hpp](#).

7.24 File_ID.hpp

[Go to the documentation of this file.](#)

```

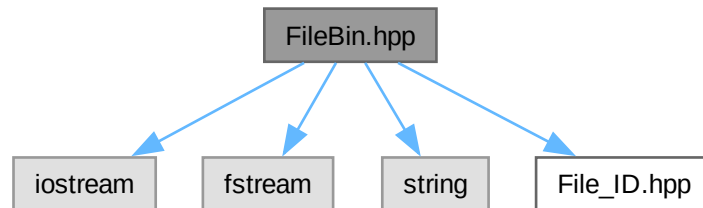
00001
00009 #pragma once
00010
00016 class File_ID {
00017     public:
00018         static constexpr int CSV = 1;
00019         static constexpr int HTML = 2;
00020         static constexpr int JSON = 3;
00021 };
  
```


7.25 FileBin.hpp File Reference

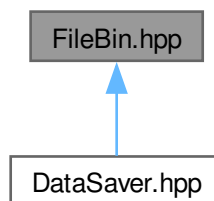
Abstract base class for writing data to files.

```
#include <iostream>
#include <fstream>
#include <string>
#include "File_ID.hpp"
```

Include dependency graph for FileBin.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [FileBin](#)
FileBin class declaration.

7.25.1 Detailed Description

Abstract base class for writing data to files.

Author

Shamar Pennant

Defines the interface for persisting formatted string data to files. Subclasses implement format-specific file writing operations (e.g., CSV files, HTML files, etc.).

Definition in file [FileBin.hpp](#).

7.26 FileBin.hpp

[Go to the documentation of this file.](#)

```

00001 #pragma once
00002
00003 #include <iostream>
00004 #include <fstream>
00005 #include <string>
00006 #include "File_ID.hpp"
00007
00022 class FileBin {
00023     public:
00025         virtual ~FileBin() = default;
00026
00034         virtual int write() = 0;
00035
00039         virtual void setOutputFile(std::string output) = 0;
00040
00044         virtual void setRawData(std::string raw) = 0;
00045
00049         virtual void setFileName(std::string fn) = 0;
00050
00051     protected:
00052         FileBin() = default;
00053 };

```

7.27 FileFormatter.hpp File Reference

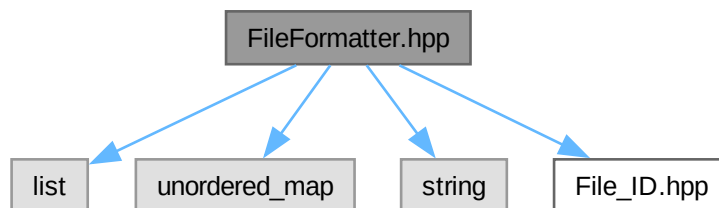
Abstract base class for formatting structured data to file format.

```

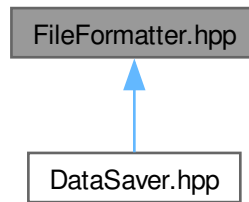
#include <list>
#include <unordered_map>
#include <string>
#include "File_ID.hpp"

```

Include dependency graph for FileFormatter.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [FileFormatter](#)
[FileFormatter](#) class declaration.

7.27.1 Detailed Description

Abstract base class for formatting structured data to file format.

Author

Shamar Pennant

Converts `list<Dict<str,str>>` structured data into format-specific string representation (e.g., CSV, HTML, JSON). Subclasses implement format-specific formatting logic for different file types. Abstract interface for formatting structured data to file format.

Definition in file [FileFormatter.hpp](#).

7.28 FileFormatter.hpp

[Go to the documentation of this file.](#)

```

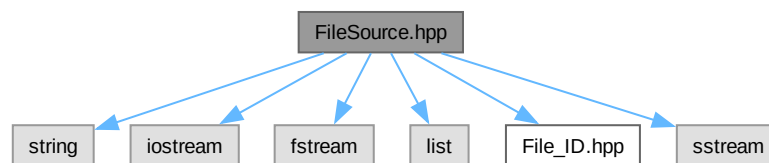
00001 #pragma once
00002
00003 #include <list>
00004 #include <unordered_map>
00005 #include <string>
00006 #include "File_ID.hpp"
00007
00018
00023 class FileFormatter {
00024     public:
00026         virtual ~FileFormatter() = default;
00027
00035         virtual std::string format() = 0;
00036
00040         virtual void setData(std::list<std::unordered_map<std::string, std::string> data) = 0;
00041
00042     protected:
00043         FileFormatter() = default;
00044 };
  
```

7.29 FileSource.hpp File Reference

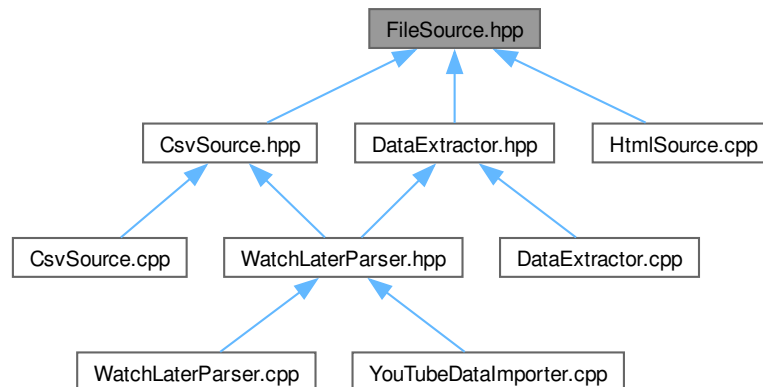
Abstract base class for reading file source data.

```
#include <string>
#include <iostream>
#include <fstream>
#include <list>
#include "File_ID.hpp"
#include <sstream>
```

Include dependency graph for FileSource.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [FileSource](#)
FileSource class declaration.

7.29.1 Detailed Description

Abstract base class for reading file source data.

Author

Shamar Pennant

Defines the interface for file reading operations across different file formats. Subclasses implement format-specific source reading (CSV, HTML, etc.).

Definition in file [FileSource.hpp](#).

7.30 FileSource.hpp

[Go to the documentation of this file.](#)

```

00001 #pragma once
00002 #include <string>
00003 #include <iostream>
00004 #include <fstream>
00005 #include <list>
00006 #include "File_ID.hpp"
00007 #include <sstream>
00008
00022 class FileSource {
00023     public:
00024         virtual ~FileSource() = default;
00025
00029         virtual std::list<std::string> read() = 0;
00030
00034         virtual void setSource(std::string src) = 0;
00035
00039         virtual int getSourceId() = 0;
00040
00041     protected:
00042         FileSource() = default;
00043 };

```

7.31 HtmlParser.cpp File Reference

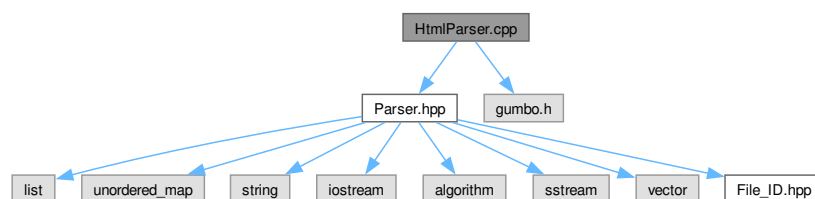
HTML file parser implementation using Gumbo.

```

#include "Parser.hpp"
#include <gumbo.h>

```

Include dependency graph for HtmlParser.cpp:



Classes

- class [HtmlParser](#)

HTML file parser implementation using Gumbo.

7.31.1 Detailed Description

HTML file parser implementation using Gumbo.

Author

Shamar Pennant

Implements the [Parser](#) interface for HTML files using the Gumbo HTML5 parser. Extracts HTML links and their text content into list<Dict<str,str>> format.

Definition in file [HtmlParser.cpp](#).

7.32 HtmlParser.cpp

[Go to the documentation of this file.](#)

```

00001
00009
00010 #include "Parser.hpp"
00011 #include <gumbo.h>
00012
00013 using namespace std;
00014
00016 class HtmlParser : public Parser {
00017     public:
00018         HtmlParser(list<string> raw_data) {this->raw_data = raw_data;}
00019         HtmlParser(){}
00020         ~HtmlParser() = default;
00021
00022         list<unordered_map<string,string>> parse() {
00023             if (raw_data.empty()) {
00024                 cerr << "HtmlParser: raw_data is empty" << endl;
00025                 exit(1);
00026             }
00027
00028             // given working with gumbo, everything is within a single item in the provided list
00029             string html = raw_data.front();
00030
00031             //grab the output through thr gumbo parser
00032             GumboOutput* output = gumbo_parse(html.c_str());
00033
00034             // container for returned results
00035             list<unordered_map<string,string>> results;
00036
00037             // place extracted links from the output into results
00038             extract_links(output->root, results);
00039
00040             // destroy ptr
00041             gumbo_destroy_output(&kGumboDefaultOptions, output);
00042
00043             return results;
00044         }
00045
00046         void setData(list<string> data) override {raw_data = data;}
00047
00048         int getParseId() override {return parse_id;}
00049
00050     private:
00051         list<string> raw_data = {};
00052         int const parse_id = File_ID::HTML;
00053
00054         // helper funtion to extracts links and tags using gumbo
00055

```

```

00056     void extract_links(GumboNode* node, list<unordered_map<string,string>& out) {
00057         if (node->type != GUMBO_NODE_ELEMENT)
00058             return;
00059
00060         GumboElement& el = node->v.element;
00061
00062         if (el.tag == GUMBO_TAG_A) {
00063             unordered_map<string,string> row;
00064
00065             // Extract href attribute
00066             GumboAttribute* href = gumbo_get_attribute(&el.attributes, "href");
00067             if (href) row["href"] = href->value;
00068
00069             // Extract visible text
00070             row["text"] = get_text(node);
00071
00072             // Tag name for debugging/consistency
00073             row["tag"] = "a";
00074
00075             out.push_back(row);
00076         }
00077
00078         // Recurse into children
00079         GumboVector* children = &el.children;
00080         for (unsigned i = 0; i < children->length; i++) {
00081             extract_links(static_cast<GumboNode*>(children->data[i]), out);
00082         }
00083     }
00084
00085     // Extract all text under a node
00086     string get_text(GumboNode* node) {
00087         if (node->type == GUMBO_NODE_TEXT)
00088             return node->v.text.text;
00089
00090         if (node->type != GUMBO_NODE_ELEMENT)
00091             return "";
00092
00093         string result;
00094         GumboVector* children = &node->v.element.children;
00095         for (unsigned i = 0; i < children->length; i++) {
00096             result += get_text(static_cast<GumboNode*>(children->data[i]));
00097         }
00098         return result;
00099     }
00100 };

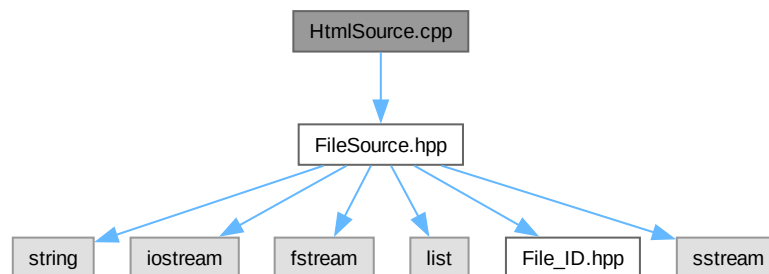
```

7.33 HtmlSource.cpp File Reference

HTML file source/reader implementation.

```
#include "FileSource.hpp"
```

Include dependency graph for HtmlSource.cpp:



Classes

- class [HtmlSource](#)

HTML file source/reader implementation.

7.33.1 Detailed Description

HTML file source/reader implementation.

Author

Shamar Pennant

Implements the [FileSource](#) interface for reading HTML files. Reads entire HTML file contents as a single string in `list<string>`. Note: Returns list with single item due to Gumbo parser requirements.

Definition in file [HtmlSource.cpp](#).

7.34 HtmlSource.cpp

[Go to the documentation of this file.](#)

```

00001
00010
00011 #include "FileSource.hpp"
00012
00013 using namespace std;
00014
00016 class HtmlSource : public FileSource {
00017     public:
00018         HtmlSource(std::string s) {src_path = s;}
00019         ~HtmlSource() = default;
00020
00021         list<string> read() override {
00022             // check if src is empty
00023             if (src_path.empty()) {
00024                 cerr << "html source path is empty" << endl;
00025                 exit(1);
00026             }
00027
00028             // open a file stream
00029             ifstream read_file_in(src_path, ios::in);
00030             if (!read_file_in.is_open()) {
00031                 cerr << "unable to open " << src_path << " within HtmlSource" << endl;
00032                 exit(1);
00033             }
00034
00035             // retrieve file contents
00036             stringstream buff;
00037             buff << read_file_in.rdbuf();
00038
00039             return {buff.str()};
00040         }
00041
00042         int getSourceId() {return source_id;}
00043         void setSource(string new_source) {src_path = new_source;}
00044
00045     private:
00046         string src_path = " ";
00047         int const source_id = File_ID::HTML;
00048 };

```

7.35 Parser.hpp File Reference

Abstract base class for parsing file data into structured format.

```

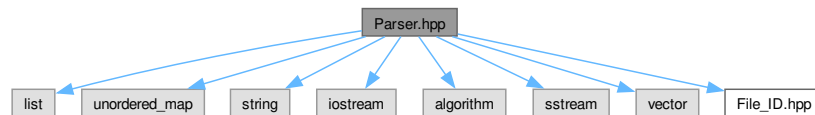
#include <list>
#include <unordered_map>
#include <string>
#include <iostream>

```

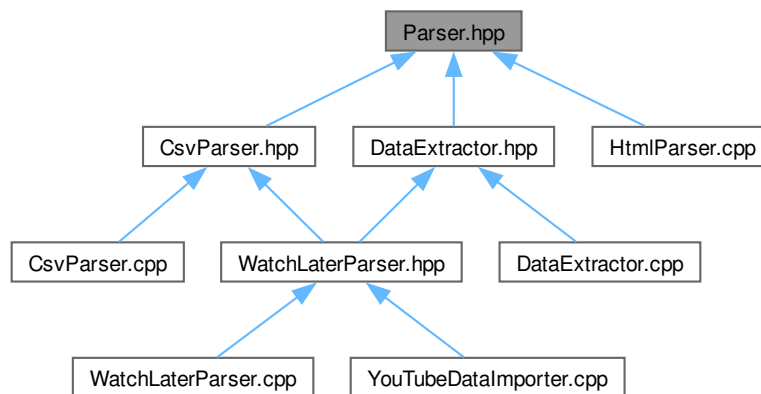


```
#include <algorithm>
#include <sstream>
#include <vector>
#include "File_ID.hpp"
```

Include dependency graph for Parser.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [Parser](#)
[Parser](#) class declaration.

7.35.1 Detailed Description

Abstract base class for parsing file data into structured format.

Author

Shamar Pennant

Defines the interface for converting raw file data into a consistent list of dictionaries format (`list<↔ Dict<string,string>>`). Each dictionary represents a row/entry with column headers as keys.

Definition in file [Parser.hpp](#).

7.36 Parser.hpp

[Go to the documentation of this file.](#)

```

00001 #pragma once
00002
00003 #include <list>
00004 #include <unordered_map>
00005 #include <string>
00006
00007 #include <iostream>
00008 #include <algorithm>
00009 #include <sstream>
00010
00011 #include <vector>
00012 #include "File_ID.hpp"
00013
00023
00028 class Parser {
00029     public:
00030         virtual ~Parser() = default;
00031
00041         virtual std::list<std::unordered_map <std::string, std::string>> parse() = 0;
00042
00046         virtual void setData(std::list<std::string> data) = 0;
00047
00051         virtual int getParseId() = 0;
00052
00053     protected:
00054         Parser() = default;
00055 };

```

7.37 WatchHistoryParser.cpp File Reference

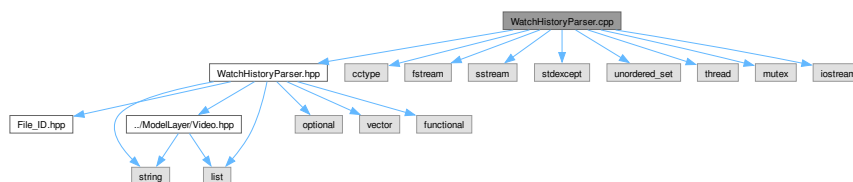
Implementation for [WatchHistoryParser](#).

```

#include "WatchHistoryParser.hpp"
#include <cctype>
#include <fstream>
#include <sstream>
#include <stdexcept>
#include <unordered_set>
#include <thread>
#include <mutex>
#include <iostream>

```

Include dependency graph for WatchHistoryParser.cpp:



7.37.1 Detailed Description

Implementation for [WatchHistoryParser](#).

Author

Shamar Pennant

Definition in file [WatchHistoryParser.cpp](#).

7.38 WatchHistoryParser.cpp

[Go to the documentation of this file.](#)

```

00001
00006
00007 #include "WatchHistoryParser.hpp"
00008
00009 #include <cctype>
00010 #include <fstream>
00011 #include <sstream>
00012 #include <stdexcept>
00013 #include <unordered_set>
00014 #include <thread>
00015 #include <mutex>
00016 #include <iostream>
00017
00018 WatchHistoryParser::WatchHistoryParser(const std::string& filePath) : filePath(filePath) {
00019     parser_id = File_ID::HTML;
00020 }
00021
00022 std::optional<std::string> WatchHistoryParser::extractVideoIdFromHref(const std::string& href) {
00023     std::string normalized = href;
00024     for (char& c : normalized) {
00025         c = static_cast<char>(std::tolower(static_cast<unsigned char>(c)));
00026     }
00027
00028     std::size_t watchPos = normalized.find("youtube.com/watch");
00029     if (watchPos != std::string::npos) {
00030         std::size_t queryPos = href.find('?', watchPos);
00031         if (queryPos == std::string::npos) {
00032             return std::nullopt;
00033         }
00034
00035         std::string query = href.substr(queryPos + 1);
00036         std::size_t start = 0;
00037         while (start < query.size()) {
00038             std::size_t amp = query.find('&', start);
00039             std::string token = query.substr(start, amp == std::string::npos ? std::string::npos : amp
- start);
00040
00041             if (token.rfind("v=", 0) == 0 && token.size() > 2) {
00042                 return token.substr(2);
00043             }
00044
00045             if (amp == std::string::npos) {
00046                 break;
00047             }
00048             start = amp + 1;
00049         }
00050     }
00051
00052     std::size_t shortPos = normalized.find("youtu.be/");
00053     if (shortPos != std::string::npos) {
00054         std::size_t idStart = shortPos + std::string("youtu.be/").size();
00055         std::size_t idEnd = href.find_first_of("?&#\"", idStart);
00056         std::string id = href.substr(idStart, idEnd == std::string::npos ? std::string::npos : idEnd -
idStart);
00057         if (!id.empty()) {
00058             return id;
00059         }
00060     }
00061
00062     return std::nullopt;
00063 }
00064
00065 std::list<Video> WatchHistoryParser::parse() {
00066     if (filePath.empty()) {
00067         throw std::invalid_argument("WatchHistoryParser: file path is empty");
00068     }
00069
00070     std::ifstream input(filePath, std::ios::in);
00071     if (!input.is_open()) {
00072         throw std::runtime_error("WatchHistoryParser: unable to open file " + filePath);
00073     }
00074
00075     std::stringstream buffer;
00076     buffer << input.rdbuf();
00077     std::string html = buffer.str();
00078
00079     std::list<Video> videos;
00080     std::unordered_set<std::string> seenVideoIds;
00081
00082     // Lightweight anchor scan to avoid hard dependency on external HTML parsers.
00083     std::size_t searchPos = 0;
00084     while (true) {

```

```

00085         std::size_t hrefPos = html.find("href=\"", searchPos);
00086         if (hrefPos == std::string::npos) {
00087             break;
00088         }
00089
00090         std::size_t valueStart = hrefPos + 6;
00091         std::size_t valueEnd = html.find("'", valueStart);
00092         if (valueEnd == std::string::npos) {
00093             break;
00094         }
00095
00096         std::string href = html.substr(valueStart, valueEnd - valueStart);
00097         auto videoId = extractVideoIdFromHref(href);
00098
00099         if (videoId.has_value() && seenVideoIds.insert(*videoId).second) {
00100             videos.emplace_back(*videoId, "", "", "", 0, std::list<std::string>{});
00101         }
00102
00103         searchPos = valueEnd + 1;
00104     }
00105
00106     return videos;
00107 }
00108
00109 int WatchHistoryParser::getParserId() const {
00110     return parser_id;
00111 }
00112
00113 std::list<Video> WatchHistoryParser::parseMultiThreaded(std::function<void(double)> progressCallback)
00114 {
00115     // Multi-threaded parsing: spawn 3 worker threads to process the file in parallel.
00116     // Each thread processes a distinct section to avoid duplicate work and improve
00117     // throughput on large HTML exports (e.g., 30K+ video IDs).
00118
00119     if (filePath.empty()) {
00120         throw std::invalid_argument("WatchHistoryParser: file path is empty");
00121     }
00122
00123     std::ifstream input(filePath, std::ios::in | std::ios::binary | std::ios::ate);
00124     if (!input.is_open()) {
00125         throw std::runtime_error("WatchHistoryParser: unable to open file " + filePath);
00126     }
00127
00128     // Read entire file at once for consistent parsing across threads.
00129     std::size_t fileSize = input.tellg();
00130     input.seekg(0);
00131
00132     std::string htmlContent(fileSize, '\0');
00133     input.read(&htmlContent[0], fileSize);
00134     input.close();
00135
00136     if (progressCallback) progressCallback(0.1);
00137
00138     // Divide file into 3 sections for parallel processing.
00139     const std::size_t section1End = fileSize / 3;
00140     const std::size_t section2End = (2 * fileSize) / 3;
00141     const std::size_t section3End = fileSize;
00142
00143     std::list<Video> videos1, videos2, videos3;
00144     std::mutex videoMutex;
00145     std::vector<std::thread> threads;
00146
00147     auto parseThreadFunc = [&](std::size_t startPos, std::size_t endPos, std::list<Video>&
00148     resultVideos) {
00149         // Each thread scans its section of the HTML file independently.
00150         // Results are collected into a separate list to avoid synchronization overhead.
00151         resultVideos = parseRange(htmlContent, startPos, endPos);
00152     };
00153
00154     // Spawn 3 threads to process file sections simultaneously.
00155     threads.emplace_back(parseThreadFunc, 0, section1End, std::ref(videos1));
00156     threads.emplace_back(parseThreadFunc, section1End, section2End, std::ref(videos2));
00157     threads.emplace_back(parseThreadFunc, section2End, section3End, std::ref(videos3));
00158
00159     if (progressCallback) progressCallback(0.3);
00160
00161     // Wait for all threads to complete.
00162     for (auto& thread : threads) {
00163         thread.join();
00164     }
00165
00166     if (progressCallback) progressCallback(0.7);
00167
00168     // Merge results from all threads, deduplicating by video ID.
00169     std::unordered_set<std::string> seenVideoIds;
00170     std::list<Video> mergedVideos;

```

```

00170     for (const auto& video : videos1) {
00171         if (seenVideoIds.insert(video.getVideoID()).second) {
00172             mergedVideos.push_back(video);
00173         }
00174     }
00175     for (const auto& video : videos2) {
00176         if (seenVideoIds.insert(video.getVideoID()).second) {
00177             mergedVideos.push_back(video);
00178         }
00179     }
00180     for (const auto& video : videos3) {
00181         if (seenVideoIds.insert(video.getVideoID()).second) {
00182             mergedVideos.push_back(video);
00183         }
00184     }
00185
00186     if (progressCallback) progressCallback(1.0);
00187
00188     std::cout << "[WatchHistoryParser] Multi-threaded parse complete: \""
00189               << filePath << "\" extracted " << mergedVideos.size() << " unique videos\\n";
00190
00191     return mergedVideos;
00192 }
00193
00194 std::list<Video> WatchHistoryParser::parseRange(const std::string& htmlContent, std::size_t startPos,
std::size_t endPos) {
00195     // Parse a range of HTML content, starting from startPos up to (but not including) endPos.
00196     // To avoid missing hrefs that span section boundaries, we search backwards from startPos
00197     // to find the previous href boundary and forwards from endPos to find the next boundary.
00198
00199     std::list<Video> videos;
00200     std::unordered_set<std::string> seenVideoIds;
00201
00202     // Adjust boundaries to avoid splitting href attributes.
00203     if (startPos > 0) {
00204         // Back up to previous href=" boundary.
00205         while (startPos > 0 && htmlContent.substr(startPos, 6) != "href=\"") {
00206             startPos--;
00207         }
00208     }
00209
00210     if (endPos < htmlContent.size()) {
00211         // Move forward to the next href=" boundary or end of file.
00212         while (endPos < htmlContent.size() && htmlContent.substr(endPos, 6) != "href=\"") {
00213             endPos++;
00214         }
00215     }
00216
00217     // Scan for href attributes within this adjusted range.
00218     std::size_t searchPos = startPos;
00219     while (searchPos < endPos) {
00220         std::size_t hrefPos = htmlContent.find("href=\"", searchPos);
00221         if (hrefPos == std::string::npos || hrefPos >= endPos) {
00222             break;
00223         }
00224
00225         std::size_t valueStart = hrefPos + 6;
00226         std::size_t valueEnd = htmlContent.find("'", valueStart);
00227         if (valueEnd == std::string::npos || valueEnd > endPos) {
00228             break;
00229         }
00230
00231         std::string href = htmlContent.substr(valueStart, valueEnd - valueStart);
00232         auto videoId = extractVideoIdFromHref(href);
00233
00234         if (videoId.has_value() && seenVideoIds.insert(*videoId).second) {
00235             videos.emplace_back(*videoId, "", "", "", 0, std::list<std::string>{});
00236         }
00237
00238         searchPos = valueEnd + 1;
00239     }
00240
00241     return videos;
00242 }

```

7.39 WatchHistoryParser.hpp File Reference

Specialized parser for YouTube watch-history HTML exports.

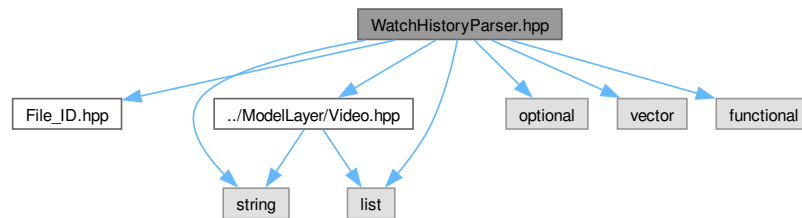
```

#include "File_ID.hpp"
#include "../ModelLayer/Video.hpp"

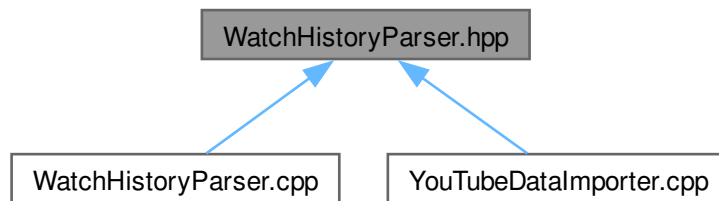
```

```
#include <list>
#include <optional>
#include <string>
#include <vector>
#include <functional>
```

Include dependency graph for WatchHistoryParser.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [WatchHistoryParser](#)
[WatchHistoryParser](#) class declaration.

7.39.1 Detailed Description

Specialized parser for YouTube watch-history HTML exports.

Author

Shamar Pennant

Reads a Google Takeout watch-history HTML file and extracts YouTube video IDs from watched-video links. Supports both single-threaded and multi-threaded parsing.

Definition in file [WatchHistoryParser.hpp](#).

7.40 WatchHistoryParser.hpp

[Go to the documentation of this file.](#)

```

00001 #pragma once
00002
00003 #include "File_ID.hpp"
00004 #include "../ModelLayer/Video.hpp"
00005
00006 #include <list>
00007 #include <optional>
00008 #include <string>
00009 #include <vector>
00010 #include <functional>
00011
00024 class WatchHistoryParser {
00025 public:
00026     explicit WatchHistoryParser(const std::string& filePath);
00027
00032     std::list<Video> parse();
00033
00040     std::list<Video> parseMultiThreaded(std::function<void(double)> progressCallback = nullptr);
00041
00042     int getParserId() const;
00043
00044 private:
00045     std::string filePath;
00046     int parser_id;
00047
00053     static std::optional<std::string> extractVideoIdFromHref(const std::string& href);
00054
00062     static std::list<Video> parseRange(const std::string& htmlContent, std::size_t startPos,
std::size_t endPos);
00063 };

```

7.41 WatchLaterParser.cpp File Reference

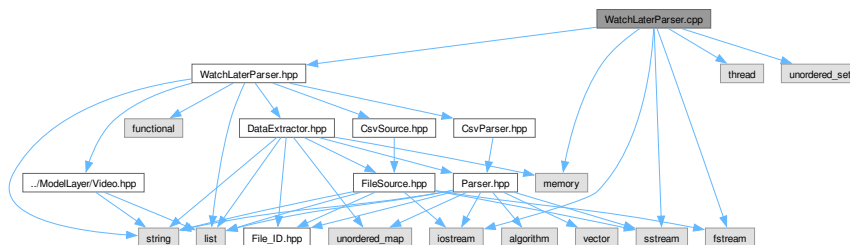
Implementation for [WatchLaterParser](#).

```

#include "WatchLaterParser.hpp"
#include <memory>
#include <fstream>
#include <sstream>
#include <thread>
#include <unordered_set>
#include <iostream>

```

Include dependency graph for WatchLaterParser.cpp:



7.41.1 Detailed Description

Implementation for [WatchLaterParser](#).

Author

Shamar Pennant

Jasmine Kumar

Definition in file [WatchLaterParser.cpp](#).

7.42 WatchLaterParser.cpp

[Go to the documentation of this file.](#)

```

00001
00002
00003 #include "WatchLaterParser.hpp"
00004
00005 #include <memory>
00006 #include <fstream>
00007 #include <sstream>
00008 #include <thread>
00009 #include <unordered_set>
00010 #include <iostream>
00011
00012 WatchLaterParser::WatchLaterParser(const std::string& filePath) : filePath(filePath) {
00013     parser_id = File_ID::CSV;
00014 }
00015
00016 std::list<Video> WatchLaterParser::parse() {
00017     DataExtractor extractor(
00018         std::make_unique<CsvSource>(filePath),
00019         std::make_unique<CsvParser>()
00020     );
00021
00022     auto rows = extractor.extract();
00023     std::list<Video> videos;
00024
00025     for (const auto& row : rows) {
00026         auto it = row.find("Video ID");
00027         if (it != row.end() && !it->second.empty()) {
00028             videos.emplace_back(
00029                 it->second,
00030                 "",
00031                 "",
00032                 "",
00033                 0,
00034                 std::list<std::string>{}
00035             );
00036         }
00037     }
00038     return videos;
00039 }
00040
00041 int WatchLaterParser::getParserId() {return parser_id;}
00042
00043 std::list<Video> WatchLaterParser::parseMultiThreaded(std::function<void(double)> progressCallback) {
00044     // Multi-threaded CSV parsing: spawn 3 worker threads to process rows in parallel.
00045     // Each thread processes a range of rows to maximize throughput.
00046
00047     std::ifstream input(filePath);
00048     if (!input.is_open()) {
00049         throw std::runtime_error("WatchLaterParser: unable to open file " + filePath);
00050     }
00051
00052     if (progressCallback) progressCallback(0.1);
00053
00054     // Read all lines into memory for parallel processing.
00055     std::vector<std::string> lines;
00056     std::string line;
00057     while (std::getline(input, line)) {
00058         lines.push_back(line);
00059     }
00060     input.close();
00061
00062     if (lines.empty()) {
00063         if (progressCallback) progressCallback(1.0);
00064         return std::list<Video>();
00065     }
00066
00067     if (progressCallback) progressCallback(0.2);

```



```

00074
00075 // First line is typically the header; process starting from line 1.
00076 std::size_t dataStartIdx = (lines.size() > 0 && lines[0].find("Video ID") != std::string::npos) ?
1 : 0;
00077 std::size_t totalRows = lines.size() - dataStartIdx;
00078
00079 if (totalRows == 0) {
00080     if (progressCallback) progressCallback(1.0);
00081     return std::list<Video>();
00082 }
00083
00084 // Divide rows into 3 sections for parallel processing.
00085 const std::size_t section1Start = dataStartIdx;
00086 const std::size_t section1End = dataStartIdx + (totalRows / 3);
00087 const std::size_t section2Start = section1End;
00088 const std::size_t section2End = dataStartIdx + (2 * totalRows / 3);
00089 const std::size_t section3Start = section2End;
00090 const std::size_t section3End = lines.size();
00091
00092 std::list<Video> videos1, videos2, videos3;
00093 std::vector<std::thread> threads;
00094
00095 auto parseRowsFunc = [&lines](std::size_t startIdx, std::size_t endIdx) -> std::list<Video> {
00096     // Each thread parses its range of CSV rows independently.
00097     // Extracts the Video ID (first column, comma-separated).
00098     std::list<Video> result;
00099     if (lines.empty() || startIdx >= lines.size()) {
00100         return result;
00101     }
00102
00103     for (std::size_t i = startIdx; i < endIdx && i < lines.size(); ++i) {
00104         const std::string& row = lines[i];
00105         if (row.empty()) {
00106             continue;
00107         }
00108
00109         // CSV format: "Video ID, Added on ..."
00110         std::size_t commaPos = row.find(',');
00111         std::string videoID = (commaPos != std::string::npos) ? row.substr(0, commaPos) : row;
00112
00113         // Trim quotes if present.
00114         if (!videoID.empty() && videoID.front() == '"' && videoID.back() == '"') {
00115             videoID = videoID.substr(1, videoID.size() - 2);
00116         }
00117
00118         if (!videoID.empty()) {
00119             result.emplace_back(videoID, "", "", "", 0, std::list<std::string>{});
00120         }
00121     }
00122     return result;
00123 };
00124
00125 // Spawn 3 threads.
00126 threads.emplace_back([&]() { videos1 = parseRowsFunc(section1Start, section1End); });
00127 threads.emplace_back([&]() { videos2 = parseRowsFunc(section2Start, section2End); });
00128 threads.emplace_back([&]() { videos3 = parseRowsFunc(section3Start, section3End); });
00129
00130 if (progressCallback) progressCallback(0.5);
00131
00132 // Wait for all threads.
00133 for (auto& thread : threads) {
00134     thread.join();
00135 }
00136
00137 if (progressCallback) progressCallback(0.8);
00138
00139 // Merge results, deduplicating by video ID.
00140 std::unordered_set<std::string> seenVideoIDs;
00141 std::list<Video> mergedVideos;
00142
00143 for (const auto& video : videos1) {
00144     if (seenVideoIDs.insert(video.getVideoID()).second) {
00145         mergedVideos.push_back(video);
00146     }
00147 }
00148 for (const auto& video : videos2) {
00149     if (seenVideoIDs.insert(video.getVideoID()).second) {
00150         mergedVideos.push_back(video);
00151     }
00152 }
00153 for (const auto& video : videos3) {
00154     if (seenVideoIDs.insert(video.getVideoID()).second) {
00155         mergedVideos.push_back(video);
00156     }
00157 }
00158
00159 if (progressCallback) progressCallback(1.0);

```

```

00160
00161     std::cout << "[WatchLaterParser] Multi-threaded parse complete: \"\"
00162               << filePath << "\" extracted \" \" mergedVideos.size() << \" unique videos\n\";
00163
00164     return mergedVideos;
00165 }

```

7.43 WatchLaterParser.hpp File Reference

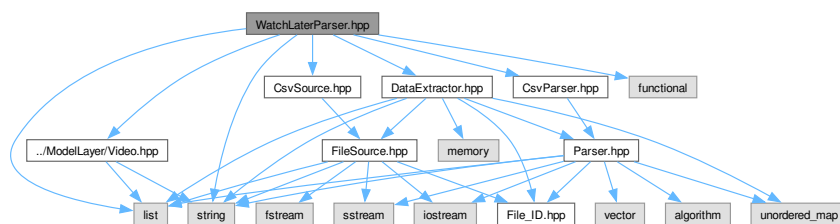
Specialized parser for YouTube Takeout Watch Later CSV files.

```

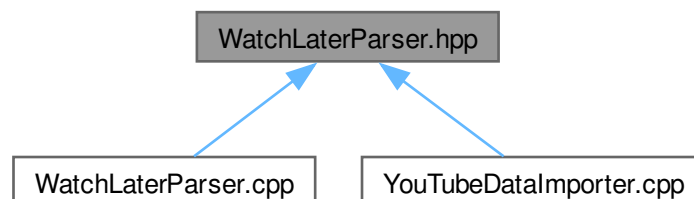
#include "DataExtractor.hpp"
#include "CsvSource.hpp"
#include "CsvParser.hpp"
#include "../ModelLayer/Video.hpp"
#include <list>
#include <string>
#include <functional>

```

Include dependency graph for WatchLaterParser.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [WatchLaterParser](#)
WatchLaterParser class declaration.

7.43.1 Detailed Description

Specialized parser for YouTube Takeout Watch Later CSV files.

Author

Shamar Pennant

Jasmine Kumar

Parses YouTube Takeout 'Watch later-videos.csv' files into [Video](#) objects with only the videoID field populated. Internally uses [CsvSource](#) for reading and [CsvParser](#) for parsing. The resulting Videos have empty strings/zero for all other field since Watch Later CSV only provides ID and timestamp. Supports both single-threaded and multi-threaded parsing.

Definition in file [WatchLaterParser.hpp](#).

7.44 WatchLaterParser.hpp

[Go to the documentation of this file.](#)

```
00001 #pragma once
00002
00003 #include "DataExtractor.hpp"
00004 #include "CsvSource.hpp"
00005 #include "CsvParser.hpp"
00006 #include "../ModelLayer/Video.hpp"
00007
00008 #include <list>
00009 #include <string>
00010 #include <functional>
00011
00024
00030 class WatchLaterParser {
00031 public:
00036     explicit WatchLaterParser(const std::string& filePath);
00037
00045     std::list<Video> parse();
00046
00053     std::list<Video> parseMultiThreaded(std::function<void(double)> progressCallback = nullptr);
00054
00055     int getParserId();
00056
00057 private:
00058     std::string filePath;
00059     int parser_id;
00060 };
```

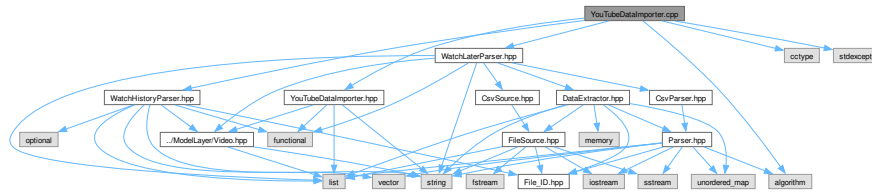
7.45 YouTubeDataImporter.cpp File Reference

Implementation for [YouTubeDataImporter](#).

```
#include "YouTubeDataImporter.hpp"
#include "WatchHistoryParser.hpp"
#include "WatchLaterParser.hpp"
#include <algorithm>
#include <cctype>
```

```
#include <stdexcept>
```

Include dependency graph for YouTubeDataImporter.cpp:



7.45.1 Detailed Description

Implementation for [YouTubeDataImporter](#).

Author

Shamar Pennant

Definition in file [YouTubeDataImporter.cpp](#).

7.46 YouTubeDataImporter.cpp

[Go to the documentation of this file.](#)

```
00001
00006
00007 #include "YouTubeDataImporter.hpp"
00008
00009 #include "WatchHistoryParser.hpp"
00010 #include "WatchLaterParser.hpp"
00011
00012 #include <algorithm>
00013 #include <cctype>
00014 #include <stdexcept>
00015
00016 std::string YouTubeDataImporter::lowerExtension(const std::string& filePath) {
00017     std::size_t dotPos = filePath.find_last_of('.');
00018     if (dotPos == std::string::npos) {
00019         return "";
00020     }
00021
00022     std::string ext = filePath.substr(dotPos);
00023     std::transform(ext.begin(), ext.end(), ext.begin(),
00024         [](unsigned char c) { return static_cast<char>(std::tolower(c)); });
00025     return ext;
00026 }
00027
00028 std::list<Video> YouTubeDataImporter::import(const std::string& filePath) const {
00029     const std::string ext = lowerExtension(filePath);
00030
00031     if (ext == ".csv") {
00032         return WatchLaterParser(filePath).parse();
00033     }
00034
00035     if (ext == ".html" || ext == ".htm") {
00036         return WatchHistoryParser(filePath).parse();
00037     }
00038
00039     throw std::invalid_argument(
00040         "YouTubeDataImporter: unsupported file extension for path: " + filePath
00041     );
00042 }
00043
00044 std::list<Video> YouTubeDataImporter::importMultiThreaded(
00045     const std::string& filePath,
```

```

00046     std::function<void(double)> progressCallback) const {
00047         // Multi-threaded import that leverages 3 worker threads for faster file processing.
00048         // Routes to the appropriate parser based on file extension, then invokes
00049         // that parser's multi-threaded parse method with progress feedback.
00050
00051         const std::string ext = lowerExtension(filePath);
00052
00053         if (ext == ".csv") {
00054             return WatchLaterParser(filePath).parseMultiThreaded(progressCallback);
00055         }
00056
00057         if (ext == ".html" || ext == ".htm") {
00058             return WatchHistoryParser(filePath).parseMultiThreaded(progressCallback);
00059         }
00060
00061         throw std::invalid_argument(
00062             "YouTubeDataImporter: unsupported file extension for path: " + filePath
00063         );
00064     }

```

7.47 YouTubeDataImporter.hpp File Reference

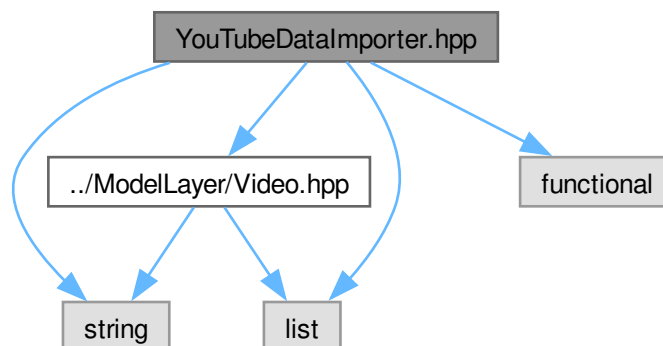
Entry-point importer that routes YouTube exports to the right parser.

```

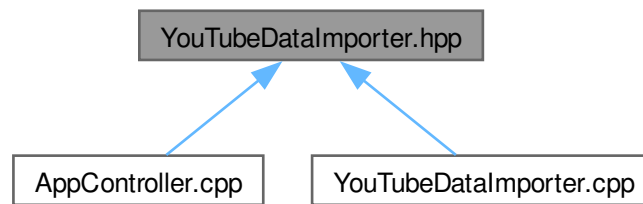
#include "../ModelLayer/Video.hpp"
#include <list>
#include <string>
#include <functional>

```

Include dependency graph for YouTubeDataImporter.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [YouTubeDataImporter](#)
[YouTubeDataImporter](#) class declaration.

7.47.1 Detailed Description

Entry-point importer that routes YouTube exports to the right parser.

Author

Shamar Pennant

Keeps [AppController](#) clean by hiding CSV-vs-HTML parser selection. Supports both single-threaded and multi-threaded import with progress tracking.

Definition in file [YouTubeDataImporter.hpp](#).

7.48 YouTubeDataImporter.hpp

[Go to the documentation of this file.](#)

```

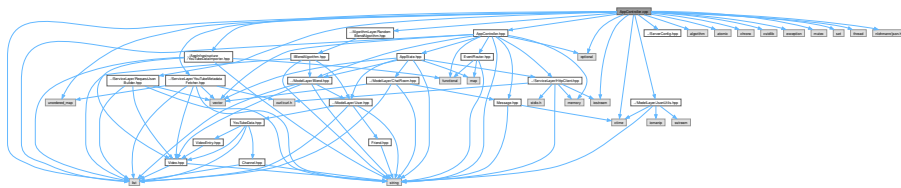
00001 #pragma once
00002
00003 #include "../ModelLayer/Video.hpp"
00004
00005 #include <list>
00006 #include <string>
00007 #include <functional>
00008
00021 class YouTubeDataImporter {
00022 public:
00033     std::list<Video> import(const std::string& filePath) const;
00034
00042     std::list<Video> importMultiThreaded(const std::string& filePath,
00043                                         std::function<void(double)> progressCallback = nullptr)
00044     const;
00045 private:
00051     static std::string lowerExtension(const std::string& filePath);
00052 };
  
```

7.49 AppController.cpp File Reference

Implementation for [AppController](#).

```
#include "AppController.hpp"
#include "../ServerConfig.hpp"
#include "../AppInfrastructure/YouTubeDataImporter.hpp"
#include "../AlgorithmLayer/RandomBlendAlgorithm.hpp"
#include "../ModelLayer/JsonUtils.hpp"
#include "../ServiceLayer/YouTubeMetadataFetcher.hpp"
#include "../ServiceLayer/RequestJsonBuilder.hpp"
#include <algorithm>
#include <atomic>
#include <chrono>
#include <cstdlib>
#include <ctime>
#include <exception>
#include <iostream>
#include <list>
#include <mutex>
#include <set>
#include <thread>
#include <unordered_map>
#include <nlohmann/json.hpp>
#include <optional>
```

Include dependency graph for AppController.cpp:



Typedefs

- using [Json](#) = nlohmann::json

Variables

- static const std::string [kYouTubeAPIKey](#) = "AlzaSyBDzH4_T9NSaAh_s59sFYrHtNnKvHmyARM"

7.49.1 Detailed Description

Implementation for [AppController](#).

Author

Jasmine Kumar
Shamar Pennant

Definition in file [AppController.cpp](#).

7.49.2 Typedef Documentation

7.49.2.1 Json

```
using Json = nlohmann::json
```

Definition at line 30 of file [AppController.cpp](#).

7.49.3 Variable Documentation

7.49.3.1 kYouTubeAPIKey

```
const std::string kYouTubeAPIKey = "AIzaSyBDzH4_T9NSaAh_s59sFYrHtNnKvHmyARM" [static]
```

Definition at line 323 of file [AppController.cpp](#).

7.50 AppController.cpp

[Go to the documentation of this file.](#)

```
00001
00007
00008 #include "AppController.hpp"
00009 #include "../ServerConfig.hpp"
00010 #include "../AppInfrastructure/YouTubeDataImporter.hpp"
00011 #include "../AlgorithmLayer/RandomBlendAlgorithm.hpp"
00012 #include "../ModelLayer/JsonUtils.hpp"
00013 #include "../ServiceLayer/YouTubeMetadataFetcher.hpp"
00014 #include "../ServiceLayer/RequestJsonBuilder.hpp"
00015 #include <algorithm>
00016 #include <atomic>
00017 #include <chrono>
00018 #include <cstdlib>
00019 #include <ctime>
00020 #include <exception>
00021 #include <iostream>
00022 #include <list>
00023 #include <mutex>
00024 #include <set>
00025 #include <thread>
00026 #include <unordered_map>
00027 #include <nlohmann/json.hpp>
00028 #include <optional>
00029
00030 using Json = nlohmann::json;
00031 using namespace std;
00032
00033 namespace {
00034
00035 std::string resolveBackendBaseUrl() {
00036     return kAppBackendBaseUrl;
00037 }
00038
00039 constexpr std::size_t kHtmlUploadMetadataThreshold = 500;
00040
00041 constexpr std::size_t kMaxUploadEnrichmentCount = 50;
00042
00043 constexpr std::size_t kMaxBlendFeedEnrichmentCount = 50;
00044
00045 constexpr std::size_t kBlendMetadataWorkerThreads = 3;
00046
00047 constexpr std::size_t kLoginParticipantLoadWorkerThreads = 3;
00048
00049 constexpr std::size_t kMaxLoginMetadataEnrichmentCount = 50;
00050
00051 bool isMetadataIncomplete(const Video& v) {
00052     return v.getTitle().empty() ||
00053         v.getChannelID().empty() ||
00054         v.getChannelName().empty() ||
```



```

00068         v.getThumbnailURL().empty() ||
00069         v.getChannelLogoURL().empty();
00070     }
00071
00072     bool hasCompleteMetadata(const Video& v) {
00073         return !v.getTitle().empty() &&
00074             !v.getChannelID().empty() &&
00075             !v.getChannelName().empty() &&
00076             !v.getThumbnailURL().empty() &&
00077             !v.getChannelLogoURL().empty();
00078     }
00079
00080     std::string lowerExtension(const std::string& filePath) {
00081         const std::size_t dotPos = filePath.find_last_of('.');
00082         if (dotPos == std::string::npos) {
00083             return "";
00084         }
00085
00086         std::string ext = filePath.substr(dotPos);
00087         std::transform(ext.begin(), ext.end(), ext.begin(),
00088             [](unsigned char c) { return static_cast<char>(std::tolower(c)); });
00089         return ext;
00090     }
00091
00092 }
00093
00094 std::optional<User> AppController::parseUserFromAuthResponse(const std::string& response,
00095                                                             const std::string& fallbackPassword) {
00096     Json parsed = Json::parse(response, nullptr, false);
00097     if (parsed.is_discarded() || !parsed.is_object() || !parsed.contains("user_id")) {
00098         return std::nullopt;
00099     }
00100
00101     const std::string userID = parsed.value("user_id", "");
00102     if (userID.empty()) {
00103         return std::nullopt;
00104     }
00105
00106     return User(
00107         userID,
00108         parsed.value("username", userID),
00109         parsed.value("email", ""),
00110         fallbackPassword
00111     );
00112 }
00113
00114 std::optional<std::string> AppController::parseBlendID(const std::string& response) {
00115     Json parsed = Json::parse(response, nullptr, false);
00116     if (parsed.is_discarded() || !parsed.is_object()) {
00117         return std::nullopt;
00118     }
00119
00120     const std::string blendID = parsed.value("blend_id", "");
00121     if (blendID.empty()) {
00122         return std::nullopt;
00123     }
00124
00125     return blendID;
00126 }
00127
00128 std::vector<std::string> AppController::parseParticipantIDs(const std::string& response) {
00129     std::vector<std::string> participantIDs;
00130
00131     Json parsed = Json::parse(response, nullptr, false);
00132     if (parsed.is_discarded() || !parsed.is_object() || !parsed.contains("participants")) {
00133         return participantIDs;
00134     }
00135
00136     if (!parsed["participants"].is_array()) {
00137         return participantIDs;
00138     }
00139
00140     for (const auto& participant : parsed["participants"]) {
00141         if (participant.is_string()) {
00142             participantIDs.push_back(participant.get<std::string>());
00143         }
00144     }
00145
00146     return participantIDs;
00147 }
00148
00149 std::list<Video> AppController::parseWatchLaterVideos(const std::string& response) {
00150     std::list<Video> videos;
00151
00152     Json parsed = Json::parse(response, nullptr, false);
00153     if (parsed.is_discarded() || !parsed.is_object() || !parsed.contains("videos")) {
00154         return videos;

```

```

00155     }
00156
00157     if (!parsed["videos"].is_array()) {
00158         return videos;
00159     }
00160
00161     for (const auto& item : parsed["videos"]) {
00162         if (!item.is_object()) {
00163             continue;
00164         }
00165
00166         const std::string videoID = item.value("video_id", "");
00167         if (videoID.empty()) {
00168             continue;
00169         }
00170
00171         videos.emplace_back(
00172             videoID,
00173             item.value("title", ""),
00174             item.value("channel_id", ""),
00175             item.value("thumbnail_url", ""),
00176             item.value("duration", 0),
00177             std::list<std::string>{},
00178             item.value("channel_name", ""),
00179             item.value("channel_logo_url", "")
00180         );
00181     }
00182
00183     return videos;
00184 }
00185
00186 std::list<User> AppController::loadParticipantsWithWatchLater(const std::vector<std::string>&
participantIDs,
00187                                                             std::vector<std::string>* missingData,
00188                                                             bool enrichParticipantLists) {
00189     std::list<User> participants;
00190
00191     for (const auto& uid : participantIDs) {
00192         std::string watchLaterResponse;
00193         try {
00194             watchLaterResponse = http.get(http.build_watch_later_endpoint(uid));
00195         } catch (const std::exception& ex) {
00196             std::cerr << "[AppController] loadParticipantsWithWatchLater: failed to fetch watch-later
for '"
00197                 << uid << "' - " << ex.what() << "\n";
00198             if (missingData != nullptr) {
00199                 missingData->push_back(uid);
00200             }
00201             continue;
00202         }
00203
00204         if (!http.wasLastRequestSuccessful(http.G)) {
00205             if (missingData != nullptr) {
00206                 missingData->push_back(uid);
00207             }
00208             continue;
00209         }
00210
00211         std::list<Video> watchLater = parseWatchLaterVideos(watchLaterResponse);
00212         if (watchLater.empty()) {
00213             Json debug = Json::parse(watchLaterResponse, nullptr, false);
00214             if (debug.is_discarded() || !debug.is_object() || !debug.contains("videos")) {
00215                 std::cerr << "[AppController] loadParticipantsWithWatchLater: malformed watch-later
payload for '"
00216                     << uid << "'\n";
00217             }
00218             if (missingData != nullptr) {
00219                 missingData->push_back(uid);
00220             }
00221             continue;
00222         }
00223
00224         // Optional: enrich participant pools. For blend-create/select paths this
00225         // is intentionally disabled to avoid expensive full-list metadata pulls.
00226         if (enrichParticipantLists) {
00227             enrichIfMissingMetadata(uid, watchLater);
00228         }
00229
00230         // create a local representation of the user
00231         User user(uid, uid, "", "");
00232         YouTubeData yt;
00233         yt.setWatchLaterVideos(watchLater);
00234         user.setYouTubeData(yt);
00235         participants.push_back(user);
00236     }
00237
00238     return participants;

```

```

00239 }
00240
00241 std::list<User> AppController::loadParticipantsWithWatchLaterParallel(
00242     const std::vector<std::string>& participantIDs,
00243     std::vector<std::string>* missingData,
00244     std::size_t workerThreads) {
00245     std::list<User> participants;
00246     if (participantIDs.empty() || workerThreads == 0) {
00247         return participants;
00248     }
00249
00250     const std::size_t workerCount = std::min<std::size_t>(workerThreads, participantIDs.size());
00251     std::vector<std::optional<User>> usersByIndex(participantIDs.size());
00252     std::mutex missingMutex;
00253     std::atomic<std::size_t> completed{0};
00254     std::vector<std::thread> workers;
00255     workers.reserve(workerCount);
00256
00257     for (std::size_t workerIndex = 0; workerIndex < workerCount; ++workerIndex) {
00258         workers.emplace_back([&, workerIndex]() {
00259             HttpClient threadHttp(server_url);
00260
00261             for (std::size_t i = workerIndex; i < participantIDs.size(); i += workerCount) {
00262                 const std::string& uid = participantIDs[i];
00263                 try {
00264                     const std::string response =
threadHttp.get(threadHttp.build_watch_later_endpoint(uid));
00265                     if (!threadHttp.wasLastRequestSuccessful(threadHttp.G)) {
00266                         if (missingData != nullptr) {
00267                             std::lock_guard<std::mutex> lock(missingMutex);
00268                             missingData->push_back(uid);
00269                         }
00270                         completed.fetch_add(1, std::memory_order_relaxed);
00271                         continue;
00272                     }
00273
00274                     std::list<Video> watchLater = parseWatchLaterVideos(response);
00275                     if (watchLater.empty()) {
00276                         if (missingData != nullptr) {
00277                             std::lock_guard<std::mutex> lock(missingMutex);
00278                             missingData->push_back(uid);
00279                         }
00280                         completed.fetch_add(1, std::memory_order_relaxed);
00281                         continue;
00282                     }
00283
00284                     User user(uid, uid, "", "");
00285                     YouTubeData yt;
00286                     yt.setWatchLaterVideos(watchLater);
00287                     user.setYouTubeData(yt);
00288                     usersByIndex[i] = user;
00289                 } catch (const std::exception&) {
00290                     if (missingData != nullptr) {
00291                         std::lock_guard<std::mutex> lock(missingMutex);
00292                         missingData->push_back(uid);
00293                     }
00294                 }
00295
00296                 completed.fetch_add(1, std::memory_order_relaxed);
00297             }
00298         });
00299     }
00300
00301     while (completed.load(std::memory_order_relaxed) < participantIDs.size()) {
00302         const double ratio = static_cast<double>(completed.load(std::memory_order_relaxed)) /
00303             static_cast<double>(participantIDs.size());
00304         reportProgress(0.6 + (ratio * 0.22), "Loading blend participant data...");
00305         std::this_thread::sleep_for(std::chrono::milliseconds(20));
00306     }
00307
00308     for (auto& worker : workers) {
00309         if (worker.joinable()) {
00310             worker.join();
00311         }
00312     }
00313
00314     for (const auto& maybeUser : usersByIndex) {
00315         if (maybeUser.has_value()) {
00316             participants.push_back(*maybeUser);
00317         }
00318     }
00319
00320     return participants;
00321 }
00322
00323 static const std::string kYouTubeAPIKey = "AIzaSyBDzH4_T9NSaAh_s59sFYrHtNnKvHmyARM";
00324

```

```

00325 // -- Constructor / Destructor -----
00326
00327 AppController::AppController():
00328     appState(AppState::getInstance()),
00329     server_url(resolveBackendBaseUrl()),
00330     http(server_url),
00331     isConnected(false) {
00332     registerEvents();
00333
00334     std::string resp = http.get("/ping");
00335     try {
00336         if (resp.find("pong") != std::string::npos) {
00337             std::cout << "[AppState] Successful connection to the server" << std::endl;
00338             isConnected = true;
00339         } else {
00340             std::cout << "[AppState] Server reachable but unexpected response..."
00341                 << resp
00342                 << " "
00343                 << endl;
00344         }
00345     } catch (const std::exception& e) {
00346         std::cout << "[AppState] Server unavailable, running offline: " << e.what() << std::endl;
00347         cout << "[AppState] Server response: " << resp << endl;
00348     }
00349 }
00350
00351 AppController::AppController(const std::string& /*dbPath*/)
00352     : AppController() {}
00353
00354 AppController::~AppController() {
00355     // If application shutting down and there's a server-created pending account,
00356     // attempt to delete it so username isn't left reserved.
00357     if (pendingAccountCreated_ && pendingRegistration_.has_value() && isConnected) {
00358         try {
00359             const std::string delResp = http.del(
00360                 http.build_delete_user_endpoint(pendingRegistration_>getUserID()),
00361                 JsonRequestBuilder::buildDeleteAccountJson(pendingRegistration_>getPassword())
00362             );
00363             if (http.wasLastRequestSuccessful(http.D)) {
00364                 std::cout << "[AppController] Deleted pending account '" <<
00365                     pendingRegistration_>getUserID()
00366                     << "' on shutdown\n";
00367             } else {
00368                 std::cerr << "[AppController] Failed to delete pending account on shutdown, HTTP "
00369                     << http.getLastStatusCode() << " response=" << delResp << "\n";
00370             }
00371         } catch (...) {
00372             // best-effort; swallow errors
00373         }
00374     }
00375     eventRouter.deregisterAll();
00376 }
00377
00378 // -- Event Registration -----
00379
00380 void AppController::registerEvents() {
00381     // Binds each named event to the corresponding handler on this controller.
00382     // Panels dispatch events by name; EventRouter calls the handler below.
00383     eventRouter.registerListener("register", [this](const EventPayload& p){ handleRegister(p); });
00384     eventRouter.registerListener("login", [this](const EventPayload& p){ handleLogin(p); });
00385     eventRouter.registerListener("logout", [this](const EventPayload& p){ handleLogout(p); });
00386     eventRouter.registerListener("deleteAccount", [this](const EventPayload& p){
00387         handleDeleteAccount(p); });
00387     eventRouter.registerListener("uploadData", [this](const EventPayload& p){ handleUploadData(p);
00388 });
00388     eventRouter.registerListener("createBlend", [this](const EventPayload& p){ handleCreateBlend(p);
00389 });
00389     eventRouter.registerListener("playVideo", [this](const EventPayload& p){ handlePlayVideo(p); });
00390     eventRouter.registerListener("refresh", [this](const EventPayload& p){ handleRefresh(p); });
00391     eventRouter.registerListener("openChat", [this](const EventPayload& p){ handleOpenChat(p); });
00392     eventRouter.registerListener("sendMessage", [this](const EventPayload& p){ handleSendMessage(p);
00393 });
00393     eventRouter.registerListener("leaveBlend", [this](const EventPayload& p){ handleLeaveBlend(p);
00394 });
00394     eventRouter.registerListener("selectBlend", [this](const EventPayload& p){ handleSelectBlend(p);
00395 });
00396
00397 // -- Getter -----
00398
00399 EventRouter& AppController::getEventRouter() {
00400     return eventRouter;
00401 }
00402
00403 void AppController::setProgressReporter(std::function<void(double, const std::string&)> reporter) {
00404     progressReporter = std::move(reporter);

```

```

00405 }
00406
00407 void AppController::clearProgressReporter() {
00408     progressReporter = nullptr;
00409 }
00410
00411 void AppController::reportProgress(double progress, const std::string& message) {
00412     if (!progressReporter) {
00413         return;
00414     }
00415
00416     if (progress < 0.0) progress = 0.0;
00417     if (progress > 1.0) progress = 1.0;
00418     progressReporter(progress, message);
00419 }
00420
00421 // -- Event Handlers -----
00422
00423 void AppController::handleRegister(const EventPayload& payload) {
00424     // verifying user data structure
00425     auto idIt = payload.find("userID");
00426     auto unIt = payload.find("username");
00427     auto emIt = payload.find("email");
00428     auto pwIt = payload.find("password");
00429
00430     if (idIt == payload.end() || unIt == payload.end() ||
00431         pwIt == payload.end()) { // if user data is malformed, notify
00432         std::cerr << "[AppController] handleRegister: missing required fields\n";
00433         return;
00434     }
00435
00436     // Build a local User object
00437     User newUser(idIt->second,
00438                 unIt->second,
00439                 emIt != payload.end() ? emIt->second : "",
00440                 pwIt->second);
00441
00442     // Attempt to create the account on the server immediately so we can delete it if
00443     // the user cancels or exits before uploading their data (backend requires it).
00444     if (isConnected) {
00445         try {
00446             const std::string registerResponse = http.post(
00447                 http.REG_USER,
00448                 RequestJsonBuilder::buildRegisterJson(
00449                     newUser.getUserID(),
00450                     newUser.getUsername(),
00451                     newUser.getEmail(),
00452                     newUser.getPassword()
00453                 )
00454             );
00455
00456             if (!http.wasLastRequestSuccessful(http.P)) {
00457                 if (http.getLastStatusCode() == http.DUPLICATE) {
00458                     // Account already exists -- try logging in instead
00459                     std::cout << "[AppController] handleRegister: account already exists for '"
00460                         << newUser.getUserID() << "', attempting login\n";
00461                     handleLogin({{"userID", idIt->second}, {"password", pwIt->second}});
00462                     return;
00463                 }
00464                 std::cerr << "[AppController] handleRegister: registration failed, HTTP "
00465                     << http.getLastStatusCode() << " response=" << registerResponse << "\n";
00466                 return;
00467             }
00468
00469             // success - keep pendingRegistration_ so we can delete it if upload never happens
00470             pendingRegistration_ = newUser;
00471             pendingAccountCreated_ = true;
00472             std::cout << "[AppController] Account created on server for '" << newUser.getUserID()
00473                 << "' (pending upload)\n";
00474         } catch (const std::exception& ex) {
00475             std::cerr << "[AppController] handleRegister: HTTP error creating account: "
00476                 << ex.what() << "\n";
00477             return;
00478         }
00479     } else {
00480         // Offline: still store pendingRegistration_ locally (account not created on server).
00481         pendingRegistration_ = newUser;
00482         pendingAccountCreated_ = false;
00483         std::cout << "[AppController] Pending local registration stored for '" << newUser.getUserID()
00484             << "' (offline)\n";
00485     }
00486
00487     // Set the current user locally so the UI can continue to the DataInstructionsPanel.
00488     appState.setCurrentUser(new User(newUser));
00489     return;
00490 }
00491

```

```

00492 void AppController::handleLogin(const EventPayload& payload) {
00493     // verifying incorrect userID and password
00494     auto idIt = payload.find("userID");
00495     auto pwIt = payload.find("password");
00496
00497     if (idIt == payload.end() || pwIt == payload.end()) {
00498         std::cerr << "[AppController] handleLogin: missing 'userID' or 'password'\n";
00499         return;
00500     }
00501
00502     reportProgress(0.05, "Authenticating account...");
00503
00504     const auto loginResponse = http.post(
00505         http.LOGIN,
00506         JsonRequestBuilder::buildLoginJson(idIt->second, pwIt->second)
00507     );
00508
00509     // retrieve the status of the request
00510     const bool req_status = http.isRequestSuccessful(http.P, http.getLastStatusCode());
00511
00512     if (!req_status) {
00513         cout << "[AppController] handleLogin: invalid credentials or '"
00514             << idIt->second
00515             << "' does not exist..."
00516             << endl;
00517         return;
00518     }
00519
00520     reportProgress(0.2, "Loading your saved data...");
00521
00522     std::optional<User> user = parseUserFromAuthResponse(loginResponse, pwIt->second);
00523     if (!user.has_value()) {
00524         std::cerr << "[AppController] handleLogin: malformed login response\n";
00525         return;
00526     }
00527
00528     string watchLaterResponse;
00529     // Restore persisted watch-later data from the backend
00530     list<Video> watchLater;
00531     try {
00532         watchLaterResponse = http.get(http.build_watch_later_endpoint(idIt->second));
00533     } catch (exception& e) {
00534         std::cerr << "[AppController] handleRegister: HTTP error, unable to move forward: "
00535             << e.what() << "\n";
00536         cout << "Server response: "
00537             << watchLaterResponse
00538             << endl;
00539
00540         return;
00541     }
00542
00543     if (http.wasLastRequestSuccessful(http.G)) {
00544         watchLater = parseWatchLaterVideos(watchLaterResponse);
00545         reportProgress(0.3, "Refreshing metadata subset...");
00546
00547         // Keep login responsive by enriching only a bounded subset and skipping
00548         // DB write-back on this path. Full enrichment can happen later during
00549         // upload/blend-display workflows.
00550         watchLater = enrichMissingMetadataSubsetMultithreaded(
00551             watchLater,
00552             kMaxLoginMetadataEnrichmentCount,
00553             kBlendMetadataWorkerThreads,
00554             [](const std::list<Video>& chunk) {
00555                 return YouTubeMetadataFetcher(kYouTubeAPIKey).enrich(chunk);
00556             }
00557         );
00558         reportProgress(0.42, "Metadata step complete");
00559     }
00560
00561     if (!watchLater.empty()) {
00562         YouTubeData yd;
00563         yd.setWatchLaterVideos(watchLater);
00564         user->setYouTubeData(yd);
00565         appState.addSessionUser(*user);
00566     }
00567
00568     appState.setCurrentUser(new User(*user));
00569     std::cout << "[AppController] Logged in as '" << user->getUsername() << "'\n";
00570     reportProgress(0.45, "Checking blend memberships...");
00571
00572     // Check if this user belongs to a previously saved blend
00573     const std::string latestBlendResponse = http.get(http.build_latest_blend_endpoint(idIt->second));
00574     if (!http.wasLastRequestSuccessful(http.G)) {
00575         return;
00576     }
00577     std::optional<std::string> blendID = parseBlendID(latestBlendResponse);
00578     if (!blendID.has_value()) {

```

```

00579         return;
00580     }
00581
00582     const std::string participantResponse = http.get(http.build_blend_participant_endpoint(*blendID));
00583     if (!http.wasLastRequestSuccessful(http.G)) {
00584         return;
00585     }
00586
00587     const std::vector<std::string> participantIDs = parseParticipantIDs(participantResponse);
00588     reportProgress(0.6, "Loading blend participant data...");
00589     std::list<User> participants = loadParticipantsWithWatchLaterParallel(
00590         participantIDs,
00591         nullptr,
00592         kLoginParticipantLoadWorkerThreads
00593     );
00594
00595     if (participants.empty()) return; // no one has data yet - nothing to show
00596
00597     // Try to recover the blend title from the server
00598     std::string restoredTitle;
00599     try {
00600         const std::string blendsResp = http.get(http.build_user_blends_endpoint(idIt->second));
00601         if (http.wasLastRequestSuccessful(http.G)) {
00602             Json blendsParsed = Json::parse(blendsResp, nullptr, false);
00603             if (!blendsParsed.is_discarded() && blendsParsed.contains("blends") &&
00604                 blendsParsed["blends"].is_array()) {
00605                 for (const auto& item : blendsParsed["blends"]) {
00606                     if (item.value("blend_id", "") == *blendID) {
00607                         restoredTitle = item.value("title", "");
00608                         break;
00609                     }
00610                 }
00611             }
00612         } catch (...) {}
00613
00614         // Re-run the algorithm to produce a fresh blend from current persisted data
00615         reportProgress(0.85, "Generating blend feed...");
00616         currentBlend = std::make_unique<Blend>(
00617             RandomBlendAlgorithm(5).generateBlend(participants, restoredTitle)
00618         );
00619         currentBlend->setBlendID(*blendID);
00620         // Attempt to fill in missing metadata for the restored blend's feed videos.
00621         // If this blend contains videos from a large HTML import, they may lack title/channel
00622         // data until this point. Metadata enrichment here is deferred and happens on-demand.
00623         enrichActiveBlendFeedIfMissingMetadata();
00624         appState.setActiveBlend(currentBlend.get());
00625         appState.createChatRoom(*currentBlend);
00626
00627         // If this user has no data, leave a message for LoginPanel to display
00628         if (watchLater.empty()) {
00629             appState.setPendingBlendMessage(
00630                 "You have been added to a blend. Upload your YouTube playlist "
00631                 "data to contribute your videos to it.");
00632         }
00633
00634         std::cout << "[AppController] Restored blend '" << *blendID
00635             << "' with " << currentBlend->size() << " videos\n";
00636         reportProgress(1.0, "Login complete");
00637     }
00638
00639 void AppController::handleLogout(const EventPayload& payload) {
00640     // If there is a server-created pending account and the user is logging out
00641     // before uploading, attempt to delete it.
00642     if (pendingAccountCreated_ && pendingRegistration_.has_value() && isConnected) {
00643         try {
00644             const std::string delResp = http.del(
00645                 http.build_delete_user_endpoint(pendingRegistration_->getUserID()),
00646                 JsonRequestBuilder::buildDeleteAccountJson(pendingRegistration_->getPassword())
00647             );
00648             if (http.wasLastRequestSuccessful(http.D)) {
00649                 std::cout << "[AppController] Deleted pending account '"
00650                     << pendingRegistration_->getUserID() << "' on logout\n";
00651             } else {
00652                 std::cerr << "[AppController] Failed to delete pending account on logout, HTTP "
00653                     << http.getLastStatusCode() << " response=" << delResp << "\n";
00654             }
00655         } catch (const std::exception& ex) {
00656             std::cerr << "[AppController] handleLogout: failed to delete pending account: " << ex.what()
00657                 << "\n";
00658         }
00659         pendingRegistration_.reset();
00660         pendingAccountCreated_ = false;
00661     }
00662     User* u = appState.getCurrentUser();
00663     if (u) {

```

```

00664         std::cout << "[AppController] Logging out '" << u->getUsername() << "'\n";
00665         delete u;
00666     }
00667     appState.setCurrentUser(nullptr);
00668     appState.clearSession();
00669 }
00670
00671 void AppController::handleDeleteAccount(const EventPayload& payload) {
00672     auto userIt = payload.find("userID");
00673     auto pwIt = payload.find("password");
00674
00675     if (userIt == payload.end() || pwIt == payload.end()) {
00676         std::cerr << "[AppController] handleDeleteAccount: missing 'userID' or 'password'\n";
00677         return;
00678     }
00679
00680     if (!isConnected) {
00681         std::cerr << "[AppController] handleDeleteAccount: server unavailable in offline mode\n";
00682         return;
00683     }
00684
00685     try {
00686         const std::string response = http.del(
00687             http.build_delete_user_endpoint(userIt->second),
00688             JsonRequestBuilder::buildDeleteAccountJson(pwIt->second)
00689         );
00690
00691         if (!http.wasLastRequestSuccessful(http.D)) {
00692             std::cerr << "[AppController] handleDeleteAccount: delete failed, HTTP "
00693                 << http.getLastStatusCode() << " response=" << response << "\n";
00694             return;
00695         }
00696
00697         User* currentUser = appState.getCurrentUser();
00698         if (currentUser != nullptr && currentUser->getUserID() == userIt->second) {
00699             handleLogout({});
00700         }
00701
00702         std::cout << "[AppController] handleDeleteAccount: deleted user '"
00703             << userIt->second << "'\n";
00704     } catch (const std::exception& e) {
00705         std::cerr << "[AppController] handleDeleteAccount: HTTP failed - "
00706             << e.what() << "\n";
00707     }
00708 }
00709 }
00710
00711 void AppController::handleUploadData(const EventPayload& payload) {
00712     auto fileIt = payload.find("filePath");
00713     auto userIt = payload.find("userID");
00714     if (fileIt == payload.end() || userIt == payload.end()) {
00715         std::cerr << "[AppController] handleUploadData: missing 'filePath' or 'userID' in payload\n";
00716         appState.setPendingUploadError("Upload failed: missing file path or user ID.");
00717         return;
00718     }
00719
00720     const std::string& filePath = fileIt->second;
00721     const std::string& userID = userIt->second;
00722
00723     // If there's a pending registration for this user, create the server account first
00724     // (only if it wasn't already created during handleRegister).
00725     if (pendingRegistration_.has_value() && pendingRegistration_->getUserID() == userID) {
00726         if (pendingAccountCreated_) {
00727             // Account was already created on the server during handleRegister -- skip re-registration
00728             std::cout << "[AppController] handleUploadData: account for '" << userID
00729                 << "' already created during registration, skipping\n";
00730             pendingRegistration_.reset();
00731             pendingAccountCreated_ = false;
00732         } else if (!isConnected) {
00733             appState.setPendingUploadError("Cannot create account while offline. Please connect and
00734 try again.");
00735             return;
00736         } else {
00737             std::string registerResponse;
00738             try {
00739                 registerResponse = http.post(
00740                     http.REG_USER,
00741                     JsonRequestBuilder::buildRegisterJson(
00742                         pendingRegistration_->getUserID(),
00743                         pendingRegistration_->getUsername(),
00744                         pendingRegistration_->getEmail(),
00745                         pendingRegistration_->getPassword()
00746                     )
00747                 );
00748                 if (!http.wasLastRequestSuccessful(http.P)) {
00749                     if (http.getLastStatusCode() == http.DUPLICATE) {
00750                         // Account exists -- not an error, just continue with the upload

```



```

00750         std::cout << "[AppController] handleUploadData: account already exists for '"
00751                     << userID << "'", continuing with upload\n";
00752     } else {
00753         appState.setPendingUploadError("Registration failed: server returned an error.
Please try again.");
00754         pendingRegistration_.reset();
00755         return;
00756     }
00757 }
00758 pendingRegistration_.reset();
00759 pendingAccountCreated_ = false;
00760 } catch (const std::exception& ex) {
00761     appState.setPendingUploadError("Registration failed: network error. Please try
again.");
00762     std::cerr << "[AppController] handleUploadData: failed to create account for '"
00763                 << userID << "' - " << ex.what() << "\n";
00764     return;
00765 }
00766 }
00767 }
00768
00769 std::cout << "[AppController] Starting file upload process for '" << filePath << "'\n";
00770
00771 std::list<Video> watchLater;
00772 try {
00773     // Multi-threaded import with progress feedback via logging.
00774     // Divides file processing across 3 worker threads for ~3x faster parsing of large files.
00775     // Progress callback logs to stdout so users can monitor progress.
00776     auto progressCallback = [](double progress) {
00777         int percent = static_cast<int>(progress * 100);
00778         std::cout << "[AppController] File parsing progress: " << percent << "%\n";
00779     };
00780
00781     std::cout << "[AppController] Starting multi-threaded file import...\n";
00782     watchLater = YouTubeDataImporter().importMultiThreaded(filePath, progressCallback);
00783     std::cout << "[AppController] File import complete: " << watchLater.size()
00784                 << " videos extracted\n";
00785 } catch (const std::exception& ex) {
00786     std::cerr << "[AppController] handleUploadData: import failed for '"
00787                 << filePath << "' - " << ex.what() << "\n";
00788     appState.setPendingUploadError(
00789         "Upload failed: could not parse this file. Please select a supported Google Takeout export
(.csv, .html, or .htm).");
00790     };
00791     return;
00792 }
00793
00794 if (watchLater.empty()) {
00795     std::cerr << "[AppController] handleUploadData: no videos found in '"
00796                 << filePath << "'\n";
00797     appState.setPendingUploadError(
00798         "No videos were found in the selected file. Please verify you selected a valid YouTube
playlist CSV or watch-history HTML export from Google Takeout.");
00799     };
00800     return;
00801 }
00802
00803 const std::string fileExt = lowerExtension(filePath);
00804 const bool isCsvUpload = (fileExt == ".csv");
00805 const bool isHtmlUpload = (fileExt == ".html" || fileExt == ".htm");
00806
00807 // Smart metadata enrichment strategy:
00808 // - On upload, enrich ONLY the first kMaxUploadEnrichmentCount videos to keep upload fast.
00809 // - CSV: always enrich first 50
00810 // - HTML <=500 videos: enrich all
00811 // - HTML >500 videos: enrich first 50, rest at display time
00812 //
00813 // This avoids blocking the UI while still providing good metadata for the most important videos.
00814
00815 std::list<Video> videosToEnrich;
00816 std::list<Video> videosNotEnriched;
00817
00818 if (isHtmlUpload && watchLater.size() > kHtmlUploadMetadataThreshold) {
00819     // Large HTML: enrich only first 50
00820     auto it = watchLater.begin();
00821     std::advance(it, std::min(watchLater.size(), kMaxUploadEnrichmentCount));
00822     videosToEnrich.splice(videosToEnrich.end(), watchLater, watchLater.begin(), it);
00823     videosNotEnriched.splice(videosNotEnriched.end(), watchLater);
00824     std::cout << "[AppController] Large HTML upload (" << (videosToEnrich.size() +
videosNotEnriched.size())
00825                 << " videos): enriching first " << videosToEnrich.size() << ", deferring "
00826                 << videosNotEnriched.size() << "\n";
00827 } else if (isCsvUpload && watchLater.size() > kMaxUploadEnrichmentCount) {
00828     // CSV with many videos: enrich first 50
00829     auto it = watchLater.begin();
00830     std::advance(it, std::min(watchLater.size(), kMaxUploadEnrichmentCount));
00831     videosToEnrich.splice(videosToEnrich.end(), watchLater, watchLater.begin(), it);

```

```

00832         videosNotEnriched.splice(videosNotEnriched.end(), watchLater);
00833         std::cout << "[AppController] CSV with " << (videosToEnrich.size() + videosNotEnriched.size())
00834             << " videos: enriching first " << videosToEnrich.size() << "\n";
00835     } else {
00836         // Small HTML or CSV: enrich all
00837         videosToEnrich = watchLater;
00838         std::cout << "[AppController] Small file (" << videosToEnrich.size()
00839             << " videos): enriching all\n";
00840     }
00841
00842     // Execute metadata enrichment only on the manageable subset.
00843     bool enriched = false;
00844     if (!videosToEnrich.empty()) {
00845         std::cout << "[AppController] Starting metadata enrichment for "
00846             << videosToEnrich.size() << " videos...\n";
00847         try {
00848             videosToEnrich = YouTubeMetadataFetcher(kYouTubeAPIKey).enrich(videosToEnrich);
00849             enriched = !kYouTubeAPIKey.empty();
00850             std::cout << "[AppController] Metadata enrichment complete\n";
00851         } catch (const std::exception& ex) {
00852             std::cerr << "[AppController] handleUploadData: metadata fetch failed: "
00853                 << ex.what() << "\n";
00854             // Non-fatal - continue with whatever data we have
00855         }
00856     }
00857
00858     // Recombine enriched and non-enriched videos.
00859     videosToEnrich.splice(videosToEnrich.end(), videosNotEnriched);
00860     watchLater = videosToEnrich;
00861
00862     // Drop videos the API couldn't find - they are deleted, private, or unavailable.
00863     if (enriched) {
00864         watchLater.remove_if([&](const Video& v) { return v.getTitle().empty(); });
00865         std::cout << "[AppController] " << watchLater.size()
00866             << " videos available after filtering unavailable entries\n";
00867     }
00868
00869     std::cout << "[AppController] Uploading " << watchLater.size()
00870         << " videos to server...\n";
00871
00872     // Attempt to post data to the server
00873     std::string saveWatchLaterResponse;
00874     try {
00875         saveWatchLaterResponse =
00876             http.post(
00877                 http.build_watch_later_endpoint(userID),
00878                 RequestJsonBuilder::buildWatchLaterJson(watchLater)
00879             );
00880         if (!http.wasLastRequestSuccessful(http.P)) {
00881             std::cerr << "[AppController] handleUploadData: failed to persist watch-later list, HTTP "
00882                 << http.getLastStatusCode() << " response=" << saveWatchLaterResponse << "\n";
00883             appState.setPendingUploadError("Upload failed: server could not save your data. Please try
again.");
00884             return;
00885         }
00886
00887         cout << "[AppController] handleUploadData: Successful upload" << endl;
00888     } catch (exception& e) {
00889         std::cerr << "[AppController] handleUploadData: upload failed." << endl;
00890         cerr << "[AppController] handleUploadData - server response: " << saveWatchLaterResponse << endl;
00891         appState.setPendingUploadError("Upload failed: unable to reach the server. Please try
again.");
00892         return;
00893     }
00894
00895     // Clear pending registration state (account created + upload complete)
00896     if (pendingRegistration_.has_value() && pendingRegistration_>getUserID() == userID) {
00897         pendingRegistration_.reset();
00898         pendingAccountCreated_ = false;
00899     }
00900
00901     // Update the current user's in-memory YouTubeData
00902     User* currentUser = appState.getCurrentUser();
00903     if (currentUser && currentUser->getUserID() == userID) {
00904         YouTubeData yd;
00905         yd.setWatchLaterVideos(watchLater);
00906         currentUser->setYouTubeData(yd);
00907         appState.addSessionUser(*currentUser);
00908     }
00909
00910     std::cout << "[AppController] Saved " << watchLater.size()
00911         << " watch-later videos for user '" << userID << "'\n";
00912     appState.setPendingUploadError("");
00913
00914     // If this user is already a blend participant, re-run the algorithm
00915     // so their data is included and the blend stays up to date.
00916     const std::string latestBlendResponse = http.get(http.build_latest_blend_endpoint(userID));

```

```

00917     if (!http.wasLastRequestSuccessful(http.G)) {
00918         return;
00919     }
00920
00921     std::optional<std::string> blendID = parseBlendID(latestBlendResponse);
00922     if (!blendID.has_value()) {
00923         return;
00924     }
00925
00926     const std::string participantResponse = http.get(http.build_blend_participant_endpoint(*blendID));
00927     if (!http.wasLastRequestSuccessful(http.G)) {
00928         return;
00929     }
00930
00931     const std::vector<std::string> participantIDs = parseParticipantIDs(participantResponse);
00932     std::list<User> participants = loadParticipantsWithWatchLater(participantIDs, nullptr, false);
00933
00934     if (participants.empty()) {
00935         return;
00936     }
00937
00938     // Preserve existing blend title if available
00939     std::string existingTitle;
00940     Blend* active = appState.getActiveBlend();
00941     if (active) existingTitle = active->getTitle();
00942
00943     currentBlend = std::make_unique<Blend>(
00944         RandomBlendAlgorithm(5).generateBlend(participants, existingTitle)
00945     );
00946     currentBlend->setBlendID(*blendID);
00947     // After regenerating the blend with new participant data, try to enrich
00948     // any videos that may be missing metadata (e.g., from large HTML uploads).
00949     enrichActiveBlendFeedIfMissingMetadata();
00950     appState.setActiveBlend(currentBlend.get());
00951     appState.createChatRoom(*currentBlend);
00952
00953     std::cout << "[AppController] Re-generated blend '" << *blendID
00954               << "' after data upload - now " << currentBlend->size() << " videos\n";
00955 }
00956
00957 void AppController::handleCreateBlend(const EventPayload& payload) {
00958     // Extract blend title
00959     auto titleIt = payload.find("blendTitle");
00960     std::string blendTitle = (titleIt != payload.end()) ? titleIt->second : "";
00961
00962     // Collect participant userIDs from the payload (userID_0, userID_1, ...)
00963     std::vector<std::string> allParticipantIDs;
00964     for (int i = 0; ; ++i) {
00965         auto it = payload.find("userID_" + std::to_string(i));
00966         if (it == payload.end()) break;
00967         allParticipantIDs.push_back(it->second);
00968     }
00969
00970     if (allParticipantIDs.empty()) {
00971         std::cerr << "[AppController] handleCreateBlend: no participant IDs in payload\n";
00972         return;
00973     }
00974
00975     reportProgress(0.1, "Loading participant data...");
00976     std::vector<std::string> missingData;
00977     std::list<User> participants =
00978         loadParticipantsWithWatchLater(allParticipantIDs, &missingData, false);
00979
00980     // Report users without uploaded data back to the UI via AppState
00981     // (non-existent users are excluded - they should have been rejected at the UI level)
00982     appState.setUsersWithoutData(missingData);
00983
00984     // Enforce upload: if any participant is missing Watch Later data, refuse to create the blend.
00985     if (!missingData.empty()) {
00986         appState.setIsBlendGenerating(false);
00987         std::cerr << "[AppController] handleCreateBlend: cannot create blend - the following
participants have not uploaded Watch Later data:\n";
00988         for (const auto& uid : missingData) std::cerr << " " << uid << "\n";
00989         // UI should show users without data via appState; do not proceed.
00990         return;
00991     }
00992
00993     if (participants.empty()) {
00994         std::cerr << "[AppController] handleCreateBlend: no participants have data\n";
00995         return;
00996     }
00997
00998     appState.setIsBlendGenerating(true);
00999     reportProgress(0.3, "Generating blend...");
01000
01001     currentBlend = std::make_unique<Blend>(
01002         RandomBlendAlgorithm(5).generateBlend(participants, blendTitle)

```

```

01003     );
01004
01005     // Persist the blend and all valid participants (existing users, even those
01006     // without data yet, so they are found on their next login).
01007     // Non-existent users (notFound) are excluded - they have no DB row.
01008     std::vector<std::string> validParticipantIDs;
01009     for (const auto& uid : allParticipantIDs) {
01010         if (lookupUser(uid)) {
01011             validParticipantIDs.push_back(uid);
01012         }
01013     }
01014
01015     if (validParticipantIDs.empty()) {
01016         appState.setIsBlendGenerating(false);
01017         std::cerr << "[AppController] handleCreateBlend: no valid participants found\n";
01018         return;
01019     }
01020
01021     reportProgress(0.55, "Saving blend to server...");
01022     User* creator = appState.getCurrentUser();
01023     std::string creatorID = creator ? creator->getUserID() : allParticipantIDs[0];
01024     const std::string createBlendResponse =
01025         http.post(http.BLEND,
01026                 RequestJsonBuilder::buildBlendJson(currentBlend->getBlendID(),
01027                                                     currentBlend->getTitle(),
01028                                                     creatorID,
01029                                                     currentBlend->getAlgorithmUsed(),
01030                                                     validParticipantIDs));
01031
01032     if (!http.wasLastRequestSuccessful(http.P)) {
01033         appState.setIsBlendGenerating(false);
01034         std::cerr << "[AppController] handleCreateBlend: failed to persist blend, HTTP "
01035                 << http.getLastStatusCode() << " response=" << createBlendResponse << "\n";
01036         return;
01037     }
01038
01039     // Set active blend first so enrichment can access it if needed
01040     appState.setActiveBlend(currentBlend.get());
01041
01042     // Attempt to fill in missing metadata for the newly created blend's videos.
01043     reportProgress(0.8, "Enriching video metadata...");
01044     try {
01045         enrichActiveBlendFeedIfMissingMetadata();
01046     } catch (const std::exception& e) {
01047         std::cerr << "[AppController] handleCreateBlend: metadata enrichment failed (non-fatal): "
01048                 << e.what() << "\n";
01049     } catch (...) {
01050         std::cerr << "[AppController] handleCreateBlend: metadata enrichment failed (non-fatal)\n";
01051     }
01052     appState.createChatRoom(*currentBlend);
01053     appState.setIsBlendGenerating(false);
01054     reportProgress(1.0, "Blend created!");
01055
01056     // Register with server so the chat room is provisioned there too.
01057     registerBlendOnServer(*currentBlend, creatorID);
01058
01059     std::cout << "[AppController] Blend created with "
01060             << currentBlend->size() << " videos\n";
01061 }
01062
01063 bool AppController::lookupUser(const std::string& userID) {
01064     try {
01065         // User existence is validated via auth profile lookup. This avoids
01066         // false negatives for valid users who have not uploaded watch-later yet.
01067         http.get(http.build_auth_user_lookup_endpoint(userID));
01068         return http.wasLastRequestSuccessful(http.G);
01069     } catch (const std::exception&) {
01070         return false;
01071     }
01072 }
01073
01074 // -- Private helpers -----
01075
01076 bool AppController::registerBlendOnServer(const Blend& blend,
01077                                         const std::string& creatorID) {
01078     if (!isConnected) return false;
01079
01080     // Build participant array: ["id1","id2",...]
01081     std::string participantsJson = "[";
01082     bool first = true;
01083     for (const User& p : blend.getParticipants()) {
01084         if (!first) participantsJson += ",";
01085         participantsJson += ModelJson::quote(p.getUserID());
01086         first = false;
01087     }
01088     participantsJson += "]";
01089
01090     const std::string body =

```

```

01090         {"blend_id\":" + ModelJson::quote(blend.getBlendID()) + ","
01091         "\"title\":" + ModelJson::quote(blend.getTitle()) + ","
01092         "\"creator_id\":" + ModelJson::quote(creatorID) + ","
01093         "\"algorithm\":" + ModelJson::quote(blend.getAlgorithmUsed()) + ","
01094         "\"participants\":" + participantsJson + "});
01095
01096     try {
01097         http.post(http.BLEND, body);
01098         if (http.wasLastRequestSuccessful(http.P)) {
01099             std::cout << "[AppController] registerBlendOnServer: blend '"
01100                 << blend.getBlendID() << "' registered\n";
01101             return true;
01102         }
01103         // 409 Conflict means it already exists - treat as success.
01104         if (http.getLastStatusCode() == http.DUPLICATE) {
01105             std::cout << "[AppController] registerBlendOnServer: blend '"
01106                 << blend.getBlendID() << "' already on server\n";
01107             return true;
01108         }
01109         std::cerr << "[AppController] registerBlendOnServer: server returned "
01110             << http.getLastStatusCode() << "\n";
01111     } catch (const std::exception& e) {
01112         std::cerr << "[AppController] registerBlendOnServer: HTTP failed - "
01113             << e.what() << "\n";
01114     }
01115     return false;
01116 }
01117
01118 void AppController::enrichIfMissingMetadata(const std::string& userID,
01119     std::list<Video>& videos) {
01120     bool needsEnrichment = std::any_of(videos.begin(), videos.end(),
01121         [](const Video& v) {
01122             return v.getTitle().empty() ||
01123                 v.getChannelID().empty() ||
01124                 v.getChannelName().empty() ||
01125                 v.getThumbnailURL().empty() ||
01126                 v.getChannelLogoURL().empty();
01127         });
01128     if (!needsEnrichment) return;
01129
01130     std::cout << "[AppController] enrichIfMissingMetadata: re-enriching "
01131         << videos.size() << " videos for '" << userID << "'\n";
01132     try {
01133         videos = YouTubeMetadataFetcher(kYouTubeAPIKey).enrich(videos);
01134         // Drop any video still missing a title - confirmed unavailable on YouTube
01135         if (!kYouTubeAPIKey.empty())
01136             videos.remove_if([](const Video& v) { return v.getTitle().empty(); });
01137
01138         const std::string saveResponse =
01139             http.post(http.build_watch_later_endpoint(userID),
01140                 RequestJsonBuilder::buildWatchLaterJson(videos));
01141         if (!http.wasLastRequestSuccessful(http.P)) {
01142             std::cerr << "[AppController] enrichIfMissingMetadata: failed to persist refreshed metadata
01143 for '"
01144                 << userID << "', HTTP " << http.getLastStatusCode()
01145                 << " response=" << saveResponse << "\n";
01146         } catch (const std::exception& ex) {
01147             std::cerr << "[AppController] enrichIfMissingMetadata failed for '"
01148                 << userID << "': " << ex.what() << "\n";
01149         }
01150     }
01151
01152     std::list<Video> AppController::enrichMissingMetadataSubsetMultithreaded(
01153         const std::list<Video>& videos,
01154         std::size_t maxVideosToEnrich,
01155         std::size_t workerThreads,
01156         const std::function<std::list<Video>(const std::list<Video>&)>& enrichChunkFn) {
01157         if (videos.empty() || maxVideosToEnrich == 0 || workerThreads == 0) {
01158             return videos;
01159         }
01160
01161         std::list<Video> updatedVideos = videos;
01162
01163         std::vector<Video> pending;
01164         pending.reserve(std::min(maxVideosToEnrich, videos.size()));
01165         for (const auto& video : videos) {
01166             if (!isMetadataIncomplete(video)) {
01167                 continue;
01168             }
01169             pending.push_back(video);
01170             if (pending.size() >= maxVideosToEnrich) {
01171                 break;
01172             }
01173         }
01174
01175         if (pending.empty()) {

```

```

01176         return updatedVideos;
01177     }
01178
01179     const std::size_t workerCount = std::max<std::size_t>(
01180         1,
01181         std::min<std::size_t>(workerThreads, pending.size())
01182     );
01183
01184     std::unordered_map<std::string, Video> enrichedByID;
01185     std::mutex resultMutex;
01186     std::vector<std::thread> workers;
01187     workers.reserve(workerCount);
01188
01189     for (std::size_t workerIndex = 0; workerIndex < workerCount; ++workerIndex) {
01190         const std::size_t start = workerIndex * pending.size() / workerCount;
01191         const std::size_t end = (workerIndex + 1) * pending.size() / workerCount;
01192         if (start >= end) {
01193             continue;
01194         }
01195
01196         workers.emplace_back([&, start, end]() {
01197             std::list<Video> chunk;
01198             for (std::size_t i = start; i < end; ++i) {
01199                 chunk.push_back(pending[i]);
01200             }
01201
01202             std::list<Video> chunkEnriched;
01203             try {
01204                 chunkEnriched = enrichChunkFn(chunk);
01205             } catch (...) {
01206                 return;
01207             }
01208
01209             std::lock_guard<std::mutex> lock(resultMutex);
01210             for (const auto& enriched : chunkEnriched) {
01211                 if (hasCompleteMetadata(enriched)) {
01212                     enrichedByID.insert_or_assign(enriched.getVideoID(), enriched);
01213                 }
01214             }
01215         });
01216     }
01217
01218     for (auto& worker : workers) {
01219         if (worker.joinable()) {
01220             worker.join();
01221         }
01222     }
01223
01224     for (auto& video : updatedVideos) {
01225         auto enrichedIt = enrichedByID.find(video.getVideoID());
01226         if (enrichedIt != enrichedByID.end()) {
01227             video = enrichedIt->second;
01228         }
01229     }
01230
01231     return updatedVideos;
01232 }
01233
01234 void AppController::enrichActiveBlendFeedIfMissingMetadata() {
01235     // Re-enriches only the currently displayed blend's video feed when any metadata
01236     // fields are incomplete. This keeps display quality high without re-processing
01237     // full watch-history sized lists on the UI path.
01238     //
01239     // This complements handleUploadData which may skip enrichment for large HTML
01240     // uploads. By deferring metadata fetch to blend-display time, we:
01241     // - Keep upload fast (avoid blocking the UI during file ingestion).
01242     // - Enrich only a small subset of videos (the active blend feed).
01243     // - Still provide rich metadata when users view their blend.
01244
01245     if (!currentBlend) {
01246         return;
01247     }
01248
01249     std::list<Video> feedVideos = currentBlend->getVideoList();
01250     const bool needsEnrichment =
01251         std::any_of(feedVideos.begin(), feedVideos.end(), isMetadataIncomplete);
01252
01253     if (!needsEnrichment) {
01254         return;
01255     }
01256
01257     std::size_t incompleteCount = 0;
01258     for (const auto& video : feedVideos) {
01259         if (isMetadataIncomplete(video)) {
01260             ++incompleteCount;
01261         }
01262     }

```

```

01263     const std::size_t deferredCount = incompleteCount > kMaxBlendFeedEnrichmentCount
01264         ? (incompleteCount - kMaxBlendFeedEnrichmentCount)
01265         : 0;
01266
01267     std::cout << "[AppController] enrichActiveBlendFeedIfMissingMetadata: enriching up to "
01268         << kMaxBlendFeedEnrichmentCount << " blend videos ("
01269         << deferredCount << " deferred)\n";
01270
01271     try {
01272         const std::list<Video> enrichedVideos = enrichMissingMetadataSubsetMultithreaded(
01273             feedVideos,
01274             kMaxBlendFeedEnrichmentCount,
01275             kBlendMetadataWorkerThreads,
01276             [](const std::list<Video>& chunk) {
01277                 return YouTubeMetadataFetcher(kYouTubeAPIKey).enrich(chunk);
01278             }
01279         );
01280
01281         std::size_t appliedCount = 0;
01282         auto oldIt = feedVideos.begin();
01283         auto newIt = enrichedVideos.begin();
01284         for (; oldIt != feedVideos.end() && newIt != enrichedVideos.end(); ++oldIt, ++newIt) {
01285             if (isMetadataIncomplete(*oldIt) && hasCompleteMetadata(*newIt)) {
01286                 ++appliedCount;
01287             }
01288         }
01289
01290         if (appliedCount > 0) {
01291             currentBlend->setVideoList(enrichedVideos);
01292         }
01293
01294         std::cout << "[AppController] enrichActiveBlendFeedIfMissingMetadata: completed "
01295             << appliedCount << " updates using "
01296             << kBlendMetadataWorkerThreads << " workers\n";
01297     } catch (const std::exception& ex) {
01298         std::cerr << "[AppController] enrichActiveBlendFeedIfMissingMetadata failed: "
01299             << ex.what() << "\n";
01300     }
01301 }
01302
01303 void AppController::handlePlayVideo(const EventPayload& payload) {
01304     // TODO: read "videoID" from payload
01305     // TODO: open the video - exact mechanism (browser, in-app) TBD
01306     std::cout << "[AppController] handlePlayVideo called (stub)\n";
01307 }
01308
01309 void AppController::handleRefresh(const EventPayload& /*payload*/) {
01310     reportProgress(0.05, "Preparing blend refresh...");
01311
01312     Blend* existing = appState.getActiveBlend();
01313     if (!existing) {
01314         std::cerr << "[AppController] handleRefresh: no active blend\n";
01315         return;
01316     }
01317
01318     // Build the set of video IDs already shown in the current blend
01319     std::set<std::string> shownIDs;
01320     for (const auto& v : existing->getVideoList())
01321         shownIDs.insert(v.getVideoID());
01322
01323     // Reload each participant's full video pool from the backend, then exclude shown videos
01324     std::vector<std::string> participantIDs;
01325     for (const User& participant : existing->getParticipants()) {
01326         participantIDs.push_back(participant.getUserID());
01327     }
01328     reportProgress(0.2, "Loading participant video pools...");
01329
01330     // Lightweight load for refresh: fetch each participant's watch-later list
01331     // but DO NOT call enrichIfMissingMetadata (that can trigger slow network calls).
01332     std::list<User> participants;
01333     for (std::size_t i = 0; i < participantIDs.size(); ++i) {
01334         const auto& uid = participantIDs[i];
01335         const std::string watchLaterResponse = http.get(http.build_watch_later_endpoint(uid));
01336         if (!http.wasLastRequestSuccessful(http.G)) continue;
01337
01338         std::list<Video> allVideos = parseWatchLaterVideos(watchLaterResponse);
01339         if (allVideos.empty()) continue;
01340
01341         // Build a fresh pool of videos not yet shown
01342         std::list<Video> freshPool;
01343         for (const auto& v : allVideos) {
01344             if (shownIDs.find(v.getVideoID()) == shownIDs.end()) {
01345                 freshPool.push_back(v);
01346             }
01347         }
01348         // If all of this user's videos were already shown, reset to the full pool
01349         if (freshPool.empty()) {

```

```

01350         freshPool = allVideos;
01351     }
01352
01353     YouTubeData pyd;
01354     pyd.setWatchLaterVideos(freshPool);
01355     User p(uid, uid, "", "");
01356     p.setYouTubeData(pyd);
01357     participants.push_back(p);
01358
01359     if (!participantIDs.empty()) {
01360         const double ratio = static_cast<double>(i + 1) /
static_cast<double>(participantIDs.size());
01361         reportProgress(0.2 + (ratio * 0.35), "Loading participant video pools...");
01362     }
01363 }
01364
01365 if (participants.empty()) {
01366     std::cerr << "[AppController] handleRefresh: no participants have data\n";
01367     return;
01368 }
01369
01370 std::string refreshTitle = existing->getTitle();
01371 std::string refreshID = existing->getBlendID();
01372 reportProgress(0.62, "Generating refreshed blend...");
01373 currentBlend = std::make_unique<Blend>(
01374     RandomBlendAlgorithm(5).generateBlend(participants, refreshTitle)
01375 );
01376 currentBlend->setBlendID(refreshID);
01377
01378 // Prevent duplicate videos within the same refreshed blend payload.
01379 std::set<std::string> seenVideoIDs;
01380 std::list<Video> uniqueVideos;
01381 for (const auto& video : currentBlend->getVideoList()) {
01382     if (seenVideoIDs.insert(video.getVideoID()).second) {
01383         uniqueVideos.push_back(video);
01384     }
01385 }
01386 currentBlend->setVideoList(uniqueVideos);
01387 reportProgress(0.8, "Enriching missing metadata...");
01388 // Attempt to enrich any missing metadata in the refreshed blend feed.
01389 // After filtering and re-sampling, some videos may lack enriched fields, especially
01390 // if the participant data originated from large HTML imports.
01391 enrichActiveBlendFeedIfMissingMetadata();
01392
01393 appState.setActiveBlend(currentBlend.get());
01394 reportProgress(1.0, "Refresh complete");
01395
01396 std::cout << "[AppController] Blend refreshed with "
01397     << currentBlend->size() << " new videos\n";
01398 }
01399
01400 void AppController::handleOpenChat(const EventPayload& payload) {
01401     auto blendIt = payload.find("blendID");
01402     auto userIt = payload.find("userID");
01403     if (blendIt == payload.end() || userIt == payload.end()) {
01404         std::cerr << "[AppController] handleOpenChat: missing 'blendID' or 'userID'\n";
01405         return;
01406     }
01407
01408     ChatRoom* room = appState.getActiveChatRoom();
01409     if (!room) {
01410         std::cerr << "[AppController] handleOpenChat: no active ChatRoom\n";
01411         return;
01412     }
01413
01414     if (!isConnected) {
01415         std::cout << "[AppController] handleOpenChat: offline - skipping history fetch\n";
01416         return;
01417     }
01418
01419     try {
01420         const std::string endpoint =
01421             http.build_chat_history_endpoint(blendIt->second, userIt->second);
01422         std::string response = http.get(endpoint);
01423
01424         if (http.getLastStatusCode() == http.MISS_RM_CNXT) {
01425             // Blend not registered on server yet - provision it now and retry.
01426             Blend* blend = appState.getActiveBlend();
01427             if (blend) {
01428                 User* creator = appState.getCurrentUser();
01429                 const std::string creatorID = creator ? creator->getUserID() : userIt->second;
01430                 if (registerBlendOnServer(*blend, creatorID)) {
01431                     response = http.get(endpoint);
01432                 }
01433             }
01434         }
01435     }

```



```

01436         if (!http.wasLastRequestSuccessful(http.G)) {
01437             std::cerr << "[AppController] handleOpenChat: server returned "
01438                 << http.getLastStatusCode() << " - leaving ChatRoom empty\n";
01439             return;
01440         }
01441
01442         std::list<Message> history = parseChatHistory(response);
01443         room->clearMessages();
01444         for (const Message& msg : history) {
01445             room->addMessage(msg);
01446         }
01447         std::cout << "[AppController] handleOpenChat: loaded "
01448             << history.size() << " messages for blend '"
01449             << blendIt->second << "'\n";
01450
01451     } catch (const std::exception& e) {
01452         std::cerr << "[AppController] handleOpenChat: HTTP failed - "
01453             << e.what() << "\n";
01454     }
01455 }
01456
01457 std::list<Message> AppController::parseChatHistory(const std::string& response) {
01458     std::list<Message> result;
01459
01460     Json parsed = Json::parse(response, nullptr, false);
01461     if (parsed.is_discarded() || !parsed.is_object() || !parsed.contains("messages")) {
01462         return result;
01463     }
01464
01465     const Json& messages = parsed["messages"];
01466     if (!messages.is_array()) {
01467         return result;
01468     }
01469
01470     for (const auto& item : messages) {
01471         if (!item.is_object()) {
01472             continue;
01473         }
01474
01475         const std::string sender = item.value("sender_id", "");
01476         const std::string content = item.value("content", "");
01477         const std::string sentAt = item.value("sent_at", "");
01478
01479         if (sender.empty() || content.empty()) {
01480             continue;
01481         }
01482
01483         std::time_t ts = std::time(nullptr);
01484         if (!sentAt.empty()) {
01485             std::tm tm{};
01486             if (strptime(sentAt.c_str(), "%Y-%m-%dT%H:%M:%SZ", &tm) != nullptr) {
01487                 ts = timegm(&tm);
01488             }
01489         }
01490
01491         result.emplace_back(sender, content, ts);
01492     }
01493
01494     return result;
01495 }
01496
01497 void AppController::handleSendMessage(const EventPayload& payload) {
01498     auto idIt = payload.find("userID");
01499     auto textIt = payload.find("text");
01500     if (idIt == payload.end() || textIt == payload.end()) return;
01501
01502     ChatRoom* room = appState.getActiveChatRoom();
01503     if (!room) return;
01504
01505     if (!room->isParticipant(idIt->second)) {
01506         std::cerr << "[AppController] handleSendMessage: '"
01507             << idIt->second << "' is not a participant\n";
01508         return;
01509     }
01510
01511     room->addMessage(idIt->second, textIt->second);
01512 }
01513
01514 const User* AppController::get_current_user() {
01515     return appState.getCurrentUser();
01516 }
01517
01518 std::string AppController::get_current_username() {
01519     if (get_current_user() == nullptr) return "";
01520
01521     return appState.getCurrentUser()->getUsername();
01522 }

```

```

01523
01524 std::string AppController::get_current_email() {
01525     if (get_current_user() == nullptr) return "";
01526
01527     return appState.getCurrentUser()->getEmail();
01528 }
01529
01530 std::string AppController::get_current_blend_id() {
01531     if (currentBlend == nullptr) {
01532         std::cout << "[AppController] current blend unique pointer has not been initialized with a
Blend obj" << std::endl;
01533         return "";
01534     } else {
01535         return currentBlend->getBlendID();
01536     }
01537 }
01538
01539 std::string AppController::get_current_blend_title() {
01540     if (currentBlend == nullptr) {
01541         std::cout << "[AppController] current blend unique pointer has not been initialized with a
Blend obj" << std::endl;
01542         return "";
01543     } else {
01544         return currentBlend->getTitle();
01545     }
01546 }
01547
01548 std::vector<AppController::BlendSummary> AppController::fetchUserBlends() {
01549     std::vector<BlendSummary> blends;
01550     User* user = appState.getCurrentUser();
01551     if (!user || !isConnected) return blends;
01552
01553     try {
01554         // Try the new /users/:userID/blends endpoint first (returns all blends).
01555         const std::string response = http.get(http.build_user_blends_endpoint(user->getUserID()));
01556         if (http.wasLastRequestSuccessful(http.G)) {
01557             Json parsed = Json::parse(response, nullptr, false);
01558             if (!parsed.is_discarded() && parsed.contains("blends") && parsed["blends"].is_array()) {
01559                 for (const auto& item : parsed["blends"]) {
01560                     if (!item.is_object()) continue;
01561                     BlendSummary s;
01562                     s.blendID = item.value("blend_id", "");
01563                     s.title = item.value("title", s.blendID);
01564                     if (!s.blendID.empty())
01565                         blends.push_back(std::move(s));
01566                 }
01567                 return blends;
01568             }
01569         }
01570
01571         // Fallback: old server only has GET /users/:userID/blend (single latest blend).
01572         const std::string fallback = http.get(http.build_latest_blend_endpoint(user->getUserID()));
01573         if (http.wasLastRequestSuccessful(http.G)) {
01574             std::optional<std::string> blendID = parseBlendID(fallback);
01575             if (blendID.has_value() && !blendID->empty()) {
01576                 BlendSummary s;
01577                 s.blendID = *blendID;
01578                 // Try to use the active blend's title if it matches
01579                 Blend* active = appState.getActiveBlend();
01580                 s.title = (active && active->getBlendID() == s.blendID)
01581                     ? active->getTitle() : s.blendID;
01582                 if (s.title.empty()) s.title = s.blendID;
01583                 blends.push_back(std::move(s));
01584             }
01585         }
01586     } catch (const std::exception& e) {
01587         std::cerr << "[AppController] fetchUserBlends: " << e.what() << "\n";
01588     }
01589     return blends;
01590 }
01591
01592 void AppController::handleLeaveBlend(const EventPayload& payload) {
01593     auto blendIt = payload.find("blendID");
01594     auto titleIt = payload.find("blendTitle");
01595     if (blendIt == payload.end()) return;
01596
01597     User* user = appState.getCurrentUser();
01598     if (!user) return;
01599
01600     const std::string& blendID = blendIt->second;
01601     const std::string blendTitle = (titleIt != payload.end()) ? titleIt->second : blendID;
01602
01603     // Send system message to chat before leaving
01604     if (isConnected) {
01605         try {
01606             const std::string systemMsg = user->getUsername() + " has left " + blendTitle + ".";
01607             // Post via the WebSocket-compatible chat message endpoint

```

```

01608         ChatRoom* room = appState.getChatRoom(blendID);
01609         if (room) {
01610             room->addMessage("system", systemMsg);
01611         }
01612     } catch (...) {}
01613 }
01614
01615 // Notify server
01616 if (isConnected) {
01617     try {
01618         const std::string body =
01619             "{ \"user_id\": \"" + ModelJson::quote(user->getUserID()) + "\" }";
01620         http.post(http.build_leave_blend_endpoint(blendID), body);
01621     } catch (const std::exception& e) {
01622         std::cerr << "[AppController] handleLeaveBlend: " << e.what() << "\n";
01623     }
01624 }
01625
01626 // Clear local state if this was the active blend, then try to load another
01627 Blend* active = appState.getActiveBlend();
01628 if (active && active->getBlendID() == blendID) {
01629     currentBlend.reset();
01630     appState.setActiveBlend(nullptr);
01631
01632     // Try to fall back to the next most recent blend
01633     auto remaining = fetchUserBlends();
01634     if (!remaining.empty()) {
01635         handleSelectBlend({"blendID", remaining[0].blendID,
01636             "blendTitle", remaining[0].title});
01637     }
01638 }
01639
01640 std::cout << "[AppController] Left blend '" << blendTitle << "'\n";
01641 }
01642
01643 void AppController::handleSelectBlend(const EventPayload& payload) {
01644     auto blendIt = payload.find("blendID");
01645     if (blendIt == payload.end()) return;
01646
01647     const std::string& blendID = blendIt->second;
01648
01649     // Fetch participants for this blend
01650     reportProgress(0.1, "Fetching participants...");
01651     const std::string participantResponse = http.get(http.build_blend_participant_endpoint(blendID));
01652     if (!http.wasLastRequestSuccessful(http.G)) {
01653         std::cerr << "[AppController] handleSelectBlend: failed to fetch participants\n";
01654         return;
01655     }
01656
01657     reportProgress(0.3, "Loading participant data...");
01658     const std::vector<std::string> participantIDs = parseParticipantIDs(participantResponse);
01659     std::list<User> participants = loadParticipantsWithWatchLater(participantIDs, nullptr, false);
01660
01661     if (participants.empty()) {
01662         std::cerr << "[AppController] handleSelectBlend: no participants have data\n";
01663         return;
01664     }
01665
01666     // Extract title from payload if available
01667     auto titleIt = payload.find("blendTitle");
01668     std::string title = (titleIt != payload.end()) ? titleIt->second : "";
01669
01670     reportProgress(0.6, "Generating blend feed...");
01671     currentBlend = std::make_unique<Blend>(
01672         RandomBlendAlgorithm(5).generateBlend(participants, title)
01673     );
01674     currentBlend->setBlendID(blendID);
01675     // Attempt to fill in missing metadata for the selected blend's feed.
01676     // If the blend contains videos from an HTML import that was saved without enrichment,
01677     // metadata will be fetched here to provide a complete display experience.
01678     reportProgress(0.8, "Enriching video metadata...");
01679     enrichActiveBlendFeedIfMissingMetadata();
01680     appState.setActiveBlend(currentBlend.get());
01681     appState.createChatRoom(*currentBlend);
01682     reportProgress(1.0, "Blend loaded!");
01683
01684     std::cout << "[AppController] Switched to blend '" << blendID
01685         << "' with " << currentBlend->size() << " videos\n";
01686 }

```

7.51 AppController.hpp File Reference

Central application coordinator.

7.52 AppController.hpp

[Go to the documentation of this file.](#)

```

00001 #ifndef APPCONTROLLER_HPP
00002 #define APPCONTROLLER_HPP
00003
00018
00019 #include <memory>
00020 #include <list>
00021 #include <functional>
00022 #include <optional>
00023 #include <string>
00024 #include <vector>
00025
00026 #include "AppState.hpp"
00027 #include "EventRouter.hpp"
00028 #include "../ModelLayer/Blend.hpp"
00029 #include "../ModelLayer/Message.hpp"
00030 #include "../ServiceLayer/HttpClient.hpp"
00031
00032 class YouTubeDataParser;
00033
00039 class AppController {
00040 public:
00044     explicit AppController();
00045
00050     explicit AppController(const std::string& dbPath);
00051
00056     ~AppController();
00057
00059     EventRouter& getEventRouter();
00060
00062     void setProgressReporter(std::function<void(double, const std::string&)> reporter);
00063
00065     void clearProgressReporter();
00066
00071     void handleRegister(const EventPayload& payload);
00072
00077     void handleLogin(const EventPayload& payload);
00078
00083     void handleLogout(const EventPayload& payload);
00084
00089     void handleDeleteAccount(const EventPayload& payload);
00090
00097     void handleUploadData(const EventPayload& payload);
00098
00104     void handleCreateBlend(const EventPayload& payload);
00105
00110     void handlePlayVideo(const EventPayload& payload);
00111
00118     void handleRefresh(const EventPayload& payload);
00119
00125     void handleOpenChat(const EventPayload& payload);
00126
00136     bool lookupUser(const std::string& userID);
00137
00142     void handleSendMessage(const EventPayload& payload);
00143
00144     // --- helpers for GUI to easily access user information such as their
00145
00150     const User* get_current_user();
00151
00156     std::string get_current_username();
00157
00162     std::string get_current_email();
00163
00167
00172     std::string get_current_blend_id();
00173
00178     std::string get_current_blend_title();
00183     struct BlendSummary {
00184         std::string blendID;
00185         std::string title;
00186     };
00187
00192     std::vector<BlendSummary> fetchUserBlends();
00193
00199     void handleLeaveBlend(const EventPayload& payload);
00200
00206     void handleSelectBlend(const EventPayload& payload);
00207
00215     static std::list<Message> parseChatHistory(const std::string& response);
00216
00229     static std::list<Video> enrichMissingMetadataSubsetMultithreaded(
00230         const std::list<Video>& videos,

```

```

00231         std::size_t maxVideosToEnrich,
00232         std::size_t workerThreads,
00233         const std::function<std::list<Video>(const std::list<Video>&)>& enrichChunkFn
00234     );
00235 private:
00236     AppState&    appState;    // Singleton session state
00237     EventRouter eventRouter; // Messenger between UI and controller
00238     const std::string server_url; // url and port to the server
00239     HttpClient http; // http client wrapper for server communications
00240     bool isConnected; // tracks whether AppState has a connection to the server
00241     std::function<void(double, const std::string)> progressReporter; // optional UI progress sink
00242
00244     std::unique_ptr<Blend> currentBlend;
00245
00250     void registerEvents();
00251
00259     void enrichIfMissingMetadata(const std::string& userID,
00260                                 std::list<Video>& videos);
00261
00267     void enrichActiveBlendFeedIfMissingMetadata();
00268
00281     static std::optional<User> parseUserFromAuthResponse(const std::string& response,
00282                                                           const std::string& fallbackPassword);
00283
00290     static std::optional<std::string> parseBlendID(const std::string& response);
00291
00300     static std::vector<std::string> parseParticipantIDs(const std::string& response);
00301
00310     static std::list<Video> parseWatchLaterVideos(const std::string& response);
00311
00323     std::list<User> loadParticipantsWithWatchLater(const std::vector<std::string>& participantIDs,
00324                                                    std::vector<std::string>* missingData,
00325                                                    bool enrichParticipantLists = false);
00326
00331     std::list<User> loadParticipantsWithWatchLaterParallel(
00332         const std::vector<std::string>& participantIDs,
00333         std::vector<std::string>* missingData,
00334         std::size_t workerThreads
00335     );
00336
00338     void reportProgress(double progress, const std::string& message);
00339
00345     bool registerBlendOnServer(const Blend& blend, const std::string& creatorID);
00346
00347     // Holds a pending registration until upload completes (or is cancelled).
00348     std::optional<User> pendingRegistration_;
00349     bool pendingAccountCreated_ = false;
00350
00351
00352 };
00353
00354 #endif // APPCONTROLLER_HPP

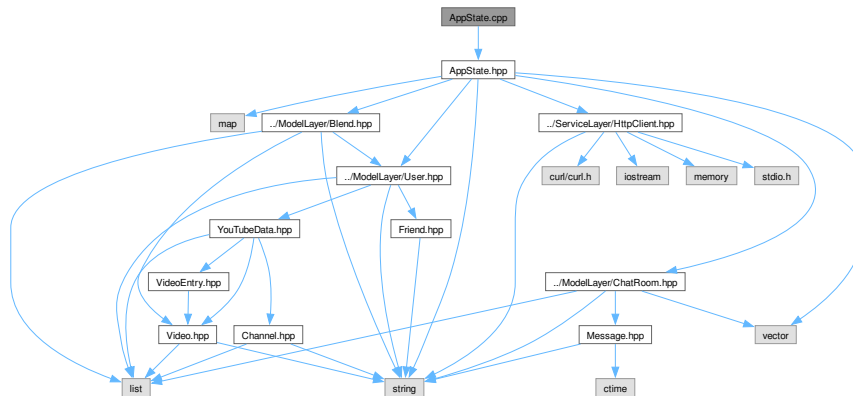
```

7.53 AppState.cpp File Reference

Implementation for [AppState](#).

```
#include "AppState.hpp"
```

Include dependency graph for AppState.cpp:



7.53.1 Detailed Description

Implementation for [AppState](#).

Author

Jasmine Kumar

Definition in file [AppState.cpp](#).

7.54 AppState.cpp

[Go to the documentation of this file.](#)

```

00001
00006
00007 #include "AppState.hpp"
00008
00009 using namespace std;
00010
00011 AppState::AppState():
00012     currentUser(nullptr),
00013     activeBlend(nullptr),
00014     isBlendGenerating(false) {}
00015
00016 // -- Singleton -----
00017
00018 AppState& AppState::getInstance() {
00019     // Constructed once on first call; guaranteed by C++11 to be thread-safe.
00020     static AppState instance;
00021     return instance;
00022 }
00023
00024 // -- Current User -----
00025
00026 User* AppState::getCurrentUser() const {
00027     return currentUser;
00028 }
00029
00030 void AppState::setCurrentUser(User* user) {
00031     // If a different user is logging in, the active blend is no longer valid.
00032     if (currentUser != nullptr && user != nullptr &&
00033         currentUser->getUserID() != user->getUserID()) {
00034         activeBlend = nullptr;
00035     }
00036 }

```

```

00035         chatRooms.clear();
00036     }
00037     currentUser = user;
00038 }
00039
00040 // -- Active Blend -----
00041
00042 Blend* AppState::getActiveBlend() const {
00043     return activeBlend;
00044 }
00045
00046 void AppState::setActiveBlend(Blend* blend) {
00047     activeBlend = blend;
00048 }
00049
00050 // -- Blend Generation Flag -----
00051
00052 bool AppState::isGeneratingBlend() const {
00053     return isBlendGenerating;
00054 }
00055
00056 void AppState::setIsBlendGenerating(bool flag) {
00057     isBlendGenerating = flag;
00058 }
00059
00060 // -- Chat Rooms -----
00061
00062 ChatRoom* AppState::getActiveChatRoom() {
00063     if (activeBlend == nullptr) return nullptr;
00064     return getChatRoom(activeBlend->getBlendID());
00065 }
00066
00067 void AppState::createChatRoom(const Blend& blend) {
00068     // Build participant ID list from the Blend's User list
00069     std::vector<std::string> participantIDs;
00070     for (const auto& user : blend.getParticipants()) {
00071         participantIDs.push_back(user.getUserID());
00072     }
00073
00074     // Insert or replace the room for this blendID
00075     chatRooms.emplace(
00076         std::piecewise_construct,
00077         std::forward_as_tuple(blend.getBlendID()),
00078         std::forward_as_tuple(blend.getBlendID(), participantIDs)
00079     );
00080 }
00081
00082 ChatRoom* AppState::getChatRoom(const std::string& blendID) {
00083     auto it = chatRooms.find(blendID);
00084     if (it == chatRooms.end()) return nullptr;
00085     return &it->second;
00086 }
00087
00088 // -- Session Users -----
00089
00090 void AppState::addSessionUser(const User& user) {
00091     sessionUsers.insert_or_assign(user.getUserID(), user);
00092 }
00093
00094 std::map<std::string, User> AppState::getSessionUsers() const {
00095     return sessionUsers;
00096 }
00097
00098 void AppState::clearSessionUsers() {
00099     sessionUsers.clear();
00100 }
00101
00102 // -- Blend creation feedback -----
00103
00104 void AppState::setUsersWithoutData(const std::vector<std::string>& users) {
00105     usersWithoutData = users;
00106 }
00107
00108 std::vector<std::string> AppState::getUsersWithoutData() const {
00109     return usersWithoutData;
00110 }
00111
00112 void AppState::setPendingBlendMessage(const std::string& msg) {
00113     pendingBlendMessage = msg;
00114 }
00115
00116 std::string AppState::takePendingBlendMessage() {
00117     std::string msg = pendingBlendMessage;
00118     pendingBlendMessage.clear();
00119     return msg;
00120 }
00121

```



```

00122 void AppState::setPendingUploadError(const std::string& msg) {
00123     pendingUploadError = msg;
00124 }
00125
00126 std::string AppState::takePendingUploadError() {
00127     std::string msg = pendingUploadError;
00128     pendingUploadError.clear();
00129     return msg;
00130 }
00131
00132 // -- Session -----
00133
00134 void AppState::clearSession() {
00135     currentUser      = nullptr;
00136     activeBlend       = nullptr;
00137     isBlendGenerating = false;
00138     chatRooms.clear();
00139     sessionUsers.clear();
00140     usersWithoutData.clear();
00141     pendingBlendMessage.clear();
00142     pendingUploadError.clear();
00143 }

```

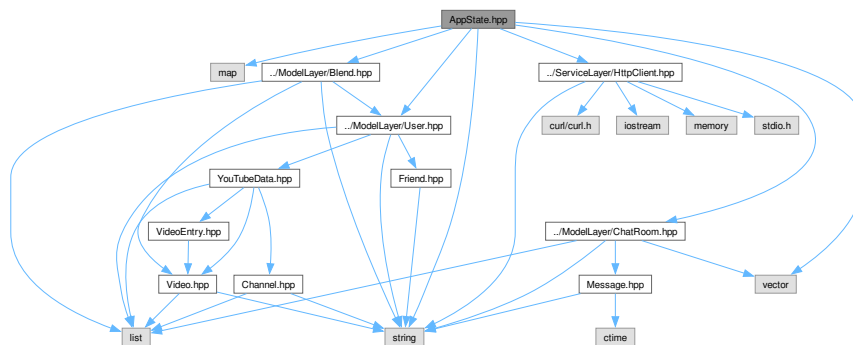
7.55 AppState.hpp File Reference

Session-scoped singleton holding all runtime application state.

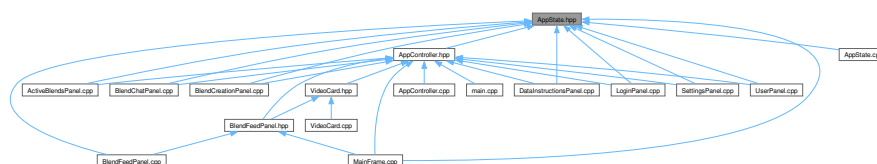
```

#include <map>
#include <string>
#include <vector>
#include "../ModelLayer/User.hpp"
#include "../ModelLayer/Blend.hpp"
#include "../ModelLayer/ChatRoom.hpp"
#include "../ServiceLayer/HttpClient.hpp"
Include dependency graph for AppState.hpp:

```



This graph shows which files directly or indirectly include this file:



Classes

- class [AppState](#)
[AppState](#) class declaration.

7.55.1 Detailed Description

Session-scoped singleton holding all runtime application state.

Author

Jasmine Kumar

[AppState](#) is the application's in-memory session store. It records who is logged in, what blend is active, and what chat rooms exist.

This is a Singleton - use [AppState::getInstance\(\)](#) everywhere. The constructor is private to enforce this. [AppState](#) holds no business logic and performs no I/O; it is updated exclusively by [AppController](#).

Definition in file [AppState.hpp](#).

7.56 AppState.hpp

[Go to the documentation of this file.](#)

```

00001 #ifndef APPSTATE_H
00002 #define APPSTATE_H
00003
00016
00017 #include <map>
00018 #include <string>
00019 #include <vector>
00020
00021 #include "../ModelLayer/User.hpp"
00022 #include "../ModelLayer/Blend.hpp"
00023 #include "../ModelLayer/ChatRoom.hpp"
00024 #include "../ServiceLayer/HttpClient.hpp"
00025
00031 class AppState {
00032     private:
00033         User*   currentUser;
00034         Blend*  activeBlend;
00035         bool    isBlendGenerating;
00036
00038         std::map<std::string, ChatRoom> chatRooms;
00039
00041         std::map<std::string, User> sessionUsers;
00042
00044         std::vector<std::string> usersWithoutData;
00045
00050         std::string pendingBlendMessage;
00051
00056         std::string pendingUploadError;
00057
00058         AppState();
00059         AppState(const AppState&) = delete;
00060         AppState& operator=(const AppState&) = delete;
00061
00062     public:
00067         static AppState& getInstance();
00068
00070         User*   getCurrentUser() const;
00072         void    setCurrentUser(User* user);
00073
00075         Blend*  getActiveBlend() const;
00077         void    setActiveBlend(Blend* blend);
00078
00080         bool    isGeneratingBlend() const;

```

```

00082     void setIsBlendGenerating(bool flag);
00083
00088     ChatRoom* getActiveChatRoom();
00089
00095     void createChatRoom(const Blend& blend);
00096
00102     ChatRoom* getChatRoom(const std::string& blendID);
00103
00108     void addSessionUser(const User& user);
00109
00111     std::map<std::string, User> getSessionUsers() const;
00112
00114     void clearSessionUsers();
00115
00121     void setUsersWithoutData(const std::vector<std::string>& users);
00122
00124     std::vector<std::string> getUsersWithoutData() const;
00125
00131     void setPendingBlendMessage(const std::string& msg);
00132
00137     std::string takePendingBlendMessage();
00138
00144     void setPendingUploadError(const std::string& msg);
00145
00150     std::string takePendingUploadError();
00151
00157     void clearSession();
00158 };
00159
00160 #endif

```

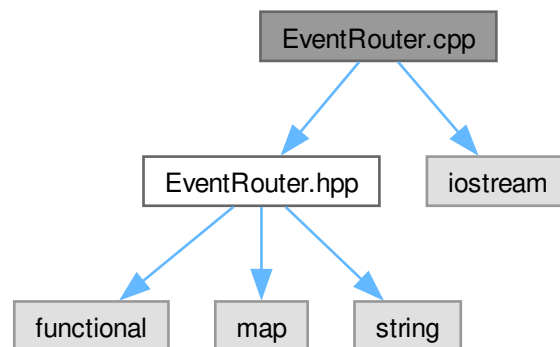
7.57 EventRouter.cpp File Reference

Implementation for [EventRouter](#).

```
#include "EventRouter.hpp"
```

```
#include <iostream>
```

Include dependency graph for EventRouter.cpp:



7.57.1 Detailed Description

Implementation for [EventRouter](#).

Author

Jasmine Kumar

Definition in file [EventRouter.cpp](#).

7.58 EventRouter.cpp

[Go to the documentation of this file.](#)

```

00001
00006
00007 #include "EventRouter.hpp"
00008 #include <iostream>
00009
00010 EventRouter::EventRouter() {}
00011
00012 void EventRouter::registerListener(const std::string& eventName, EventHandler handler) {
00013     listeners[eventName] = std::move(handler);
00014 }
00015
00016 void EventRouter::dispatch(const std::string& eventName, const EventPayload& payload) {
00017     auto it = listeners.find(eventName);
00018     if (it == listeners.end()) {
00019         std::cerr << "[EventRouter] Unknown event: " << eventName << "\n";
00020         return;
00021     }
00022     it->second(payload);
00023 }
00024
00025 void EventRouter::deregisterAll() {
00026     listeners.clear();
00027 }

```

7.59 EventRouter.hpp File Reference

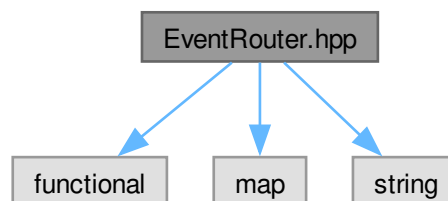
Lightweight event-dispatch system connecting the UI layer to [AppController](#).

```

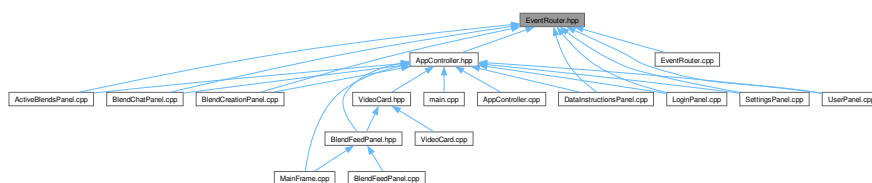
#include <functional>
#include <map>
#include <string>

```

Include dependency graph for EventRouter.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [EventRouter](#)
[EventRouter](#) class declaration.

Typedefs

- using [EventPayload](#) = std::map<std::string, std::string>
- using [EventHandler](#) = std::function<void(const [EventPayload](#)&)>

7.59.1 Detailed Description

Lightweight event-dispatch system connecting the UI layer to [AppController](#).

Author

Jasmine Kumar

UI panels dispatch named events with a string-to-string payload map. [AppController](#) registers a handler for each event name during construction. Panels never call [AppController](#) directly; [AppController](#) never polls the UI.

Payload conventions by event name:

- "login" ->
 { "userID": "...", "password": "..." }
- "logout" -> {}
- "deleteAccount" ->
 { "userID": "...", "password": "..." }
- "uploadData" ->
 { "filePath": "...", "userID": "..." }
- "createBlend" ->
 { "userID_0": "...", "userID_1": "..." }
- "playVideo" ->
 { "videoID": "..." }
- "refresh" -> {}
- "openChat" -> {}
- "sendMessage" ->
 { "userID": "...", "text": "..." }

Definition in file [EventRouter.hpp](#).

7.59.2 Typedef Documentation

7.59.2.1 EventHandler

```
using EventHandler = std::function<void(const EventPayload&)>
```

Any callable that accepts an [EventPayload](#).

Definition at line 33 of file [EventRouter.hpp](#).

Classes

- class [YTBLNDApp](#)

Functions

- `wxIMPLEMENT_APP` ([YTBLNDApp](#))

7.61.1 Detailed Description

Implementation for main.

Author

Jasmine Kumar

Definition in file [main.cpp](#).

7.62 main.cpp

[Go to the documentation of this file.](#)

```

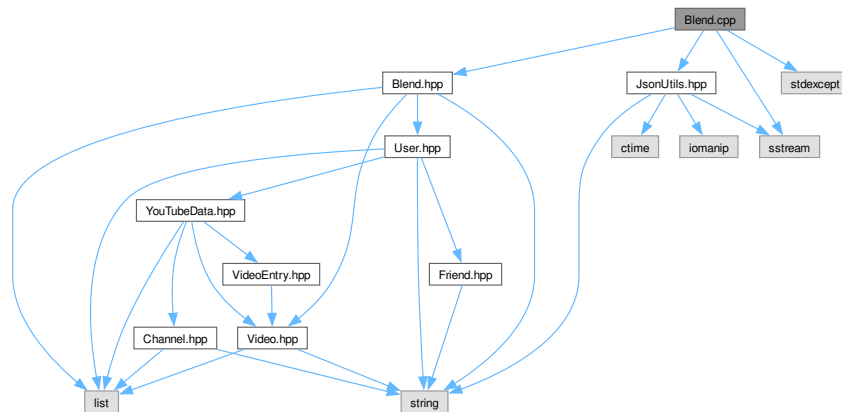
00001
00006
00007 // =====
00008 // main.cpp - Application entry point
00009 //
00010 // Creates the single AppController (which owns AppState and EventRouter),
00011 // initialises wxWidgets image handlers for thumbnail
00012 // loading, then shows the MainFrame starting at the LOGIN page.
00013 //
00014 // Design note: AppController is a member of YTBLNDApp so its lifetime covers
00015 // the entire run of the app. MainFrame receives a reference and forwards it
00016 // to every sub-panel via its constructor.
00017 // =====
00018
00019 #include <wx/wx.h>
00020 #include <wx/image.h>
00021 #include <wx/socket.h>
00022
00023 #include "UILayer/MainFrame.hpp"
00024 #include "AppLayer/AppController.hpp"
00025
00026 class YTBLNDApp : public wxApp {
00027 public:
00028     bool OnInit() override;
00029     int OnExit() override;
00030
00031 private:
00032     // AppController owns AppState (singleton) and EventRouter.
00033     // Declared here so its lifetime matches the application.
00034     AppController controller;
00035 };
00036
00037 wxIMPLEMENT_APP(YTBLNDApp);
00038
00039 bool YTBLNDApp::OnInit() {
00040     // Enable JPEG and PNG handlers for VideoCard thumbnail loading.
00041     // Must be called before any wxImage::LoadFile() / wxURL usage.
00042     wxInitAllImageHandlers();
00043
00044     // Initialise wxSocket on the main thread before any background thread
00045     // calls wxURL::GetInputStream(). wxWidgets requires socket init to
00046     // happen on the main thread; failing to do this causes the
00047     // "wxWidgets sockets must be initialized in the main thread" error.
00048     wxSocketBase::Initialize();
00049
00050     // MainFrame starts maximised and shows the LOGIN page.
00051     auto* frame = new MainFrame(controller);
00052     frame->Show(true);
00053     return true;
00054 }
00055
00056 int YTBLNDApp::OnExit() {
00057     wxSocketBase::Shutdown();
00058     return wxApp::OnExit();
00059 }

```

7.63 Blend.cpp File Reference

Implementation for [Blend](#).

```
#include "Blend.hpp"
#include "JsonUtils.hpp"
#include <stdexcept>
#include <sstream>
Include dependency graph for Blend.cpp:
```



7.63.1 Detailed Description

Implementation for [Blend](#).

Author

Jasmine Kumar

Definition in file [Blend.cpp](#).

7.64 Blend.cpp

[Go to the documentation of this file.](#)

```
00001
00006
00007 #include "Blend.hpp"
00008 #include "JsonUtils.hpp"
00009
00010 #include <stdexcept>
00011
00012 #include <sstream>
00013
00014 Blend::Blend(const std::string& blendID,
00015             const std::string& title,
00016             const std::string& algorithmUsed,
00017             const std::list<User>& participants,
00018             const std::list<Video>& videoList)
00019 : blendID(blendID),
00020   title(title),
00021   algorithmUsed(algorithmUsed),
```



```

00022     participants(participants),
00023     videoList(videoList) {}
00024
00025 // Getters
00026
00027 std::string Blend::getBlendID() const {
00028     return blendID;
00029 }
00030
00031 std::string Blend::getTitle() const {
00032     return title;
00033 }
00034
00035 std::string Blend::getAlgorithmUsed() const {
00036     return algorithmUsed;
00037 }
00038
00039 std::list<User> Blend::getParticipants() const {
00040     return participants;
00041 }
00042
00043 std::list<Video> Blend::getVideoList() const {
00044     return videoList;
00045 }
00046
00047 // Setters
00048
00049 void Blend::setVideoList(const std::list<Video>& videoList) {
00050     this->videoList = videoList;
00051 }
00052
00053 void Blend::setBlendID(const std::string& blendID) {
00054     this->blendID = blendID;
00055 }
00056
00057 // Indexed Access
00058
00059 Video Blend::getVideo(int index) const {
00060     if (index < 0 || index >= static_cast<int>(videoList.size())) {
00061         throw std::out_of_range("Blend::getVideo - index " + std::to_string(index) +
00062             " is out of range (size = " + std::to_string(videoList.size()) + ")");
00063     }
00064     auto it = videoList.begin();
00065     std::advance(it, index);
00066     return *it;
00067 }
00068
00069 int Blend::size() const {
00070     return static_cast<int>(videoList.size());
00071 }
00072
00073 std::string Blend::toString() const {
00074     std::ostringstream participantsJson;
00075     participantsJson << "[";
00076     bool firstParticipant = true;
00077     for (const auto& participant : participants) {
00078         if (!firstParticipant) {
00079             participantsJson << ",";
00080         }
00081         participantsJson << ModelJson::quote(participant.getUserID());
00082         firstParticipant = false;
00083     }
00084     participantsJson << "]";
00085
00086     std::ostringstream videosJson;
00087     videosJson << "[";
00088     bool firstVideo = true;
00089     for (const auto& video : videoList) {
00090         if (!firstVideo) {
00091             videosJson << ",";
00092         }
00093         videosJson << video.toString();
00094         firstVideo = false;
00095     }
00096     videosJson << "]";
00097
00098     return "{"
00099         "\"blend_id\":\"" + ModelJson::quote(blendID) + ","
00100         "\"title\":\"" + ModelJson::quote(title) + ","
00101         "\"chat_room_id\":\"" + ModelJson::quote(blendID) + ","
00102         "\"algorithm\":\"" + ModelJson::quote(algorithmUsed) + ","
00103         "\"participants\":\"" + participantsJson.str() + ","
00104         "\"videos\":\"" + videosJson.str()
00105         + "}";
00106 }

```


7.66 Blend.hpp

[Go to the documentation of this file.](#)

```

00001 #ifndef BLEND_H
00002 #define BLEND_H
00003
00013
00014 #include <string>
00015 #include <list>
00016
00017 #include "User.hpp"
00018 #include "Video.hpp"
00019
00024 class Blend {
00025     private:
00026         std::string blendID;           // indentifying blend method
00027         std::string title;             // title of the blend
00028         std::string algorithmUsed;     // algorithm to be used for parsing through the blend
00029         std::list<User> participants;  // users included in the blend
00030         std::list<Video> videoList;    // the video list included within the blend
00031
00032     public:
00041         Blend(const std::string& blendID,
00042               const std::string& title,
00043               const std::string& algorithmUsed,
00044               const std::list<User>& participants,
00045               const std::list<Video>& videoList);
00046
00048         std::string getBlendID() const;
00050         std::string getTitle() const;
00052         std::string getAlgorithmUsed() const;
00054         std::list<User> getParticipants() const;
00056         std::list<Video> getVideoList() const;
00057
00062         void setVideoList(const std::list<Video>& videoList);
00063
00069         Video getVideo(int index) const;
00070
00072         int size() const;
00073
00078         void setBlendID(const std::string& blendID);
00079
00081         std::string toString() const;
00082 };
00083
00084 #endif

```

7.67 Channel.cpp File Reference

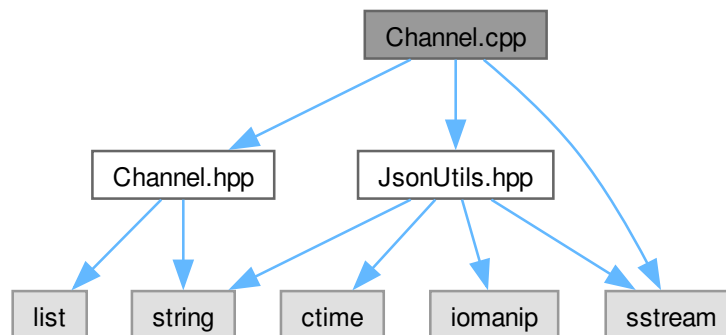
Implementation for [Channel](#).

```

#include "Channel.hpp"
#include "JsonUtils.hpp"
#include <sstream>

```

Include dependency graph for Channel.cpp:



7.67.1 Detailed Description

Implementation for [Channel](#).

Author

Jasmine Kumar

Definition in file [Channel.cpp](#).

7.68 Channel.cpp

[Go to the documentation of this file.](#)

```

00001
00006
00007 #include "Channel.hpp"
00008 #include "JsonUtils.hpp"
00009
00010 #include <sstream>
00011
00012 Channel::Channel(const std::string& channelId,
00013                 const std::string& displayName,
00014                 const std::list<std::string>& categories)
00015     : channelId(channelId), displayName(displayName), categories(categories) {}
00016
00017 // Getters
00018
00019 std::string Channel::getChannelID() const {
00020     return channelId;
00021 }
00022
00023 std::string Channel::getDisplayName() const {
00024     return displayName;
00025 }
00026
00027 std::list<std::string> Channel::getCategories() const {
00028     return categories;
00029 }
00030
00031 std::string Channel::toString() const {
00032     std::ostringstream categoriesJson;

```

```
00033 categoriesJson << "[";
00034 bool first = true;
00035 for (const auto& category : categories) {
00036     if (!first) {
00037         categoriesJson << ",";
00038     }
00039     categoriesJson << ModelJson::quote(category);
00040     first = false;
00041 }
00042 categoriesJson << "]"";
00043
00044 return "{"
00045     "\"channel_id\": " + ModelJson::quote(channelID) + ","
00046     "\"display_name\": " + ModelJson::quote(displayName) + ","
00047     "\"categories\": " + categoriesJson.str()
00048     + "}";
00049 }
```

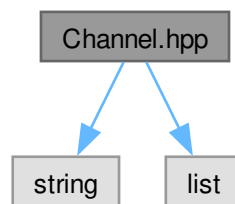
7.69 Channel.hpp File Reference

Model representing a YouTube channel.

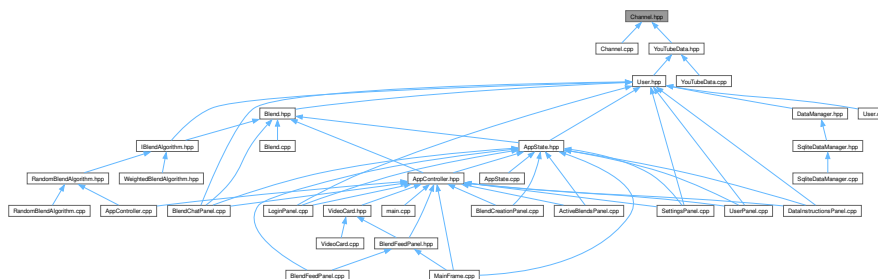
```
#include <string>
```

```
#include <list>
```

Include dependency graph for Channel.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class `Channel`
Channel class declaration.

7.69.1 Detailed Description

Model representing a YouTube channel.

Author

Jasmine Kumar

Definition in file [Channel.hpp](#).

7.70 Channel.hpp

[Go to the documentation of this file.](#)

```

00001 #ifndef CHANNEL_H
00002 #define CHANNEL_H
00003
00009
00010 #include <string>
00011 #include <list>
00012
00017 class Channel {
00018     private:
00019         std::string channelID;           // identifying channel method
00020         std::string displayName;         // the channel front facing name (as seen on YouTube)
00021         std::list<std::string> categories; // categories/themes of the channel
00022
00023     public:
00030         Channel(const std::string& channelID,
00031                const std::string& displayName,
00032                const std::list<std::string>& categories);
00033
00035         std::string getChannelID() const;
00037         std::string getDisplayName() const;
00039         std::list<std::string> getCategories() const;
00040
00041         std::string toString() const;
00042 };
00043
00044 #endif

```

7.71 ChatRoom.cpp File Reference

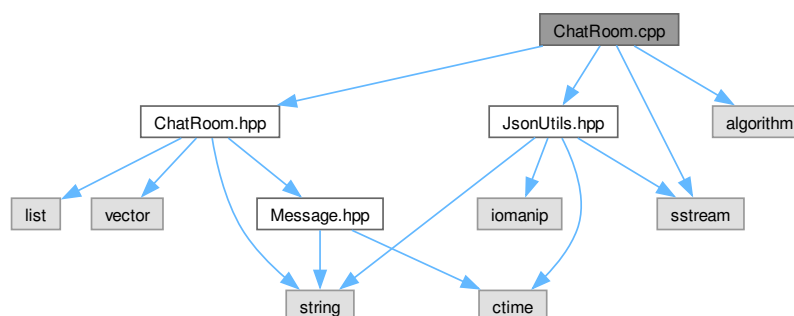
Implementation for [ChatRoom](#).

```

#include "ChatRoom.hpp"
#include "JsonUtils.hpp"
#include <algorithm>
#include <sstream>

```

Include dependency graph for ChatRoom.cpp:



7.71.1 Detailed Description

Implementation for [ChatRoom](#).

Author

Jasmine Kumar

Definition in file [ChatRoom.cpp](#).

7.72 ChatRoom.cpp

[Go to the documentation of this file.](#)

```

00001
00006
00007 #include "ChatRoom.hpp"
00008 #include "JsonUtils.hpp"
00009
00010 #include <algorithm>
00011 #include <sstream>
00012
00013 ChatRoom::ChatRoom(const std::string& blendID,
00014                   const std::vector<std::string>& participantIDs)
00015     : blendID(blendID), participantIDs(participantIDs) {}
00016
00017 // -- Getters -----
00018
00019 std::string ChatRoom::getBlendID() const {
00020     return blendID;
00021 }
00022
00023 std::vector<std::string> ChatRoom::getParticipantIDs() const {
00024     return participantIDs;
00025 }
00026
00027 std::list<Message> ChatRoom::getMessages() const {
00028     return messages;
00029 }
00030
00031 // -- Messaging -----
00032
00033 void ChatRoom::addMessage(const std::string& userID, const std::string& text) {
00034     messages.emplace_back(userID, text);
00035 }
00036
00037 void ChatRoom::addMessage(const Message& msg) {
00038     messages.push_back(msg);
00039 }
00040
00041 bool ChatRoom::isParticipant(const std::string& userID) const {
00042     return std::find(participantIDs.begin(), participantIDs.end(), userID)
00043         != participantIDs.end();
00044 }
00045
00046 void ChatRoom::clearMessages() {
00047     messages.clear();
00048 }
00049
00050 std::string ChatRoom::toString() const {
00051     std::ostringstream membersJson;
00052     membersJson << "[";
00053     bool firstMember = true;
00054     for (const auto& participantID : participantIDs) {
00055         if (!firstMember) {
00056             membersJson << ",";
00057         }
00058         membersJson << ModelJson::quote(participantID);
00059         firstMember = false;
00060     }
00061     membersJson << "]";
00062
00063     std::ostringstream messagesJson;
00064     messagesJson << "[";
00065     bool firstMessage = true;
00066     for (const auto& message : messages) {

```

```

00067         if (!firstMessage) {
00068             messagesJson < " , ";
00069         }
00070         messagesJson < " { "
00071             < " \"type\": \"chat_message\", "
00072             < " \"room_id\": \" < ModelJson::quote(blendID) < " , "
00073             < " \"sender_id\": \" < ModelJson::quote(message.userID) < " , "
00074             < " \"content\": \" < ModelJson::quote(message.text) < " , "
00075             < " \"sent_at\": \" < ModelJson::quote(ModelJson::toIso8601(message.timestamp))
00076             < " } ";
00077         firstMessage = false;
00078     }
00079     messagesJson < " ] ";
00080
00081     return " { "
00082         " \"blend_id\": \" < ModelJson::quote(blendID) < " , "
00083         " \"chat_room_id\": \" < ModelJson::quote(blendID) < " , "
00084         " \"member_users\": \" < membersJson.str() < " , "
00085         " \"messages\": \" < messagesJson.str()
00086         + " } ";
00087 }

```

7.73 ChatRoom.hpp File Reference

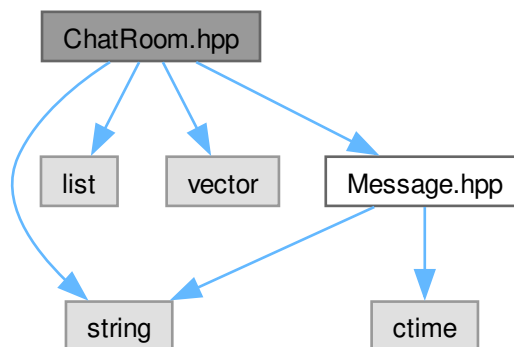
In-memory chat session tied to a specific [Blend](#).

```

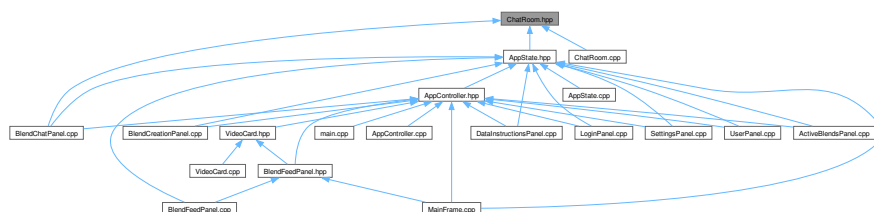
#include <string>
#include <list>
#include <vector>
#include "Message.hpp"

```

Include dependency graph for ChatRoom.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [ChatRoom](#)
[ChatRoom](#) class declaration.

7.73.1 Detailed Description

In-memory chat session tied to a specific [Blend](#).

Author

Jasmine Kumar

Identified by the blendID of the [Blend](#) it belongs to. Messages are not persisted in this version - all chat history is lost when the session ends.

Currently only one [ChatRoom](#) is active at a time (the one matching the active [Blend](#) in [AppState](#)), but [AppState](#) holds a map<string, ChatRoom> keyed by blendID, so supporting multiple simultaneous rooms requires no changes here.

Definition in file [ChatRoom.hpp](#).

7.74 ChatRoom.hpp

[Go to the documentation of this file.](#)

```

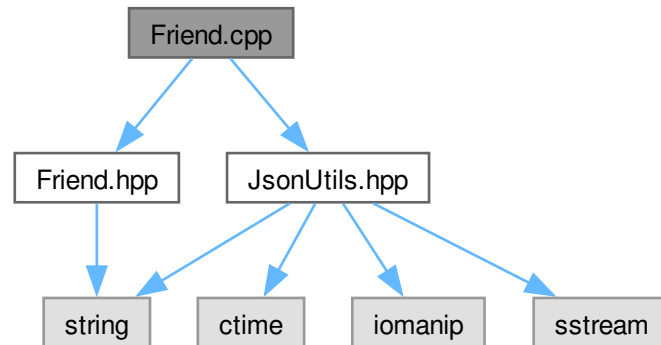
00001 #ifndef CHATROOM_H
00002 #define CHATROOM_H
00003
00016
00017 #include <string>
00018 #include <list>
00019 #include <vector>
00020
00021 #include "Message.hpp"
00022
00028 class ChatRoom {
00029     private:
00030         std::string blendID;
00031         std::vector<std::string> participantIDs;
00032         std::list<Message> messages;
00033
00034     public:
00040         ChatRoom(const std::string& blendID,
00041                 const std::vector<std::string>& participantIDs);
00042
00044         std::string getBlendID() const;
00046         std::vector<std::string> getParticipantIDs() const;
00048         std::list<Message> getMessages() const;
00049
00055         void addMessage(const std::string& userID, const std::string& text);
00056
00062         void addMessage(const Message& msg);
00063
00070         bool isParticipant(const std::string& userID) const;
00071
00076         void clearMessages();
00077
00078         std::string toString() const;
00079 };
00080
00081 #endif

```

7.75 Friend.cpp File Reference

Implementation for [Friend](#).

```
#include "Friend.hpp"
#include "JsonUtils.hpp"
Include dependency graph for Friend.cpp:
```



7.75.1 Detailed Description

Implementation for [Friend](#).

Author

Jasmine Kumar

Definition in file [Friend.cpp](#).

7.76 Friend.cpp

[Go to the documentation of this file.](#)

```

00001
00006
00007 #include "Friend.hpp"
00008 #include "JsonUtils.hpp"
00009
00010 //
00011 Friend::Friend(const std::string& userID,
00012               const std::string& displayName,
00013               const std::string& email)
00014     : userID(userID), displayName(displayName), email(email) {}
00015
00016 // Getters
00017
00018 std::string Friend::getUserID() const {
00019     return userID;
00020 }
00021
```

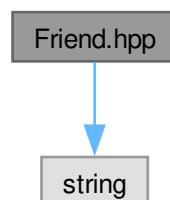
```
00022 std::string Friend::getDisplayName() const {
00023     return displayName;
00024 }
00025
00026 std::string Friend::getEmail() const {
00027     return email;
00028 }
00029
00030 //Setters (userID is intentionally omitted - it is immutable after construction)
00031
00032 void Friend::setDisplayName(const std::string& displayName) {
00033     this->displayName = displayName;
00034 }
00035
00036 void Friend::setEmail(const std::string& email) {
00037     this->email = email;
00038 }
00039
00040 std::string Friend::toString() const {
00041     return "{"
00042         "\"user_id\": " + ModelJson::quote(userID) + ", "
00043         "\"username\": " + ModelJson::quote(displayName) + ", "
00044         "\"email\": " + ModelJson::quote(email)
00045         + "}";
00046 }
```

7.77 Friend.hpp File Reference

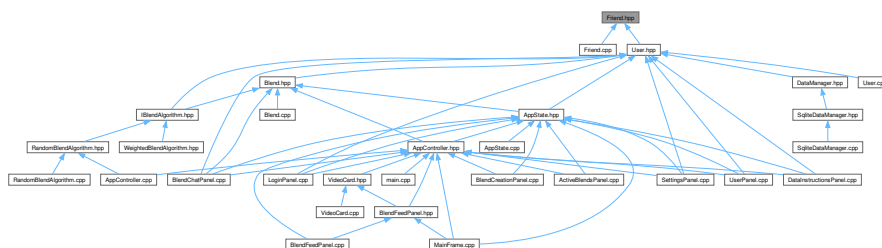
Model representing a friend/contact entry on a [User's](#) list.

```
#include <string>
```

Include dependency graph for Friend.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [Friend](#)

[Friend](#) class declaration.

7.77.1 Detailed Description

Model representing a friend/contact entry on a [User](#)'s list.

Author

Jasmine Kumar

Definition in file [Friend.hpp](#).

7.78 Friend.hpp

[Go to the documentation of this file.](#)

```
00001 #ifndef FRIEND_H
00002 #define FRIEND_H
00003
00009
00010 #include <string>
00011
00017 class Friend {
00018     private:
00019         std::string userID;
00020         std::string displayName;
00021         std::string email;
00022
00023     public:
00030         Friend(const std::string& userID,
00031               const std::string& displayName,
00032               const std::string& email);
00033
00035         std::string getUserID() const;
00037         std::string getDisplayName() const;
00039         std::string getEmail() const;
00040
00042         void setDisplayName(const std::string& displayName);
00044         void setEmail(const std::string& email);
00045
00050         std::string toString() const;
00051 };
00052
00053 #endif
```

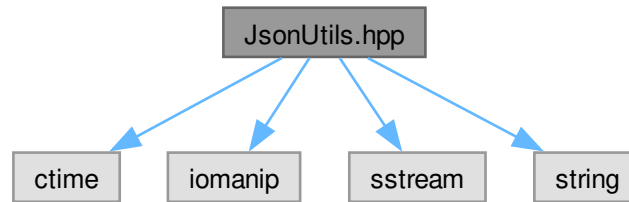
7.79 JsonUtils.hpp File Reference

Utility functions for JSON string formatting and serialization.

```
#include <ctime>
#include <iomanip>
#include <sstream>
```

```
#include <string>
```

Include dependency graph for JsonUtils.hpp:



This graph shows which files directly or indirectly include this file:



Namespaces

- namespace [ModelJson](#)
JSON formatting utilities for model serialization.

Functions

- `std::string` [ModelJson::escape](#) (const std::string &input)
- `std::string` [ModelJson::quote](#) (const std::string &input)
- `std::string` [ModelJson::toISOString8601](#) (std::time_t timestamp)

7.79.1 Detailed Description

Utility functions for JSON string formatting and serialization.

Author

Shamar Pennant

Provides helper functions for escaping special characters in JSON strings, quoting values, and converting timestamps to ISO8601 format.

Definition in file [JsonUtils.hpp](#).

7.80 JsonUtils.hpp

[Go to the documentation of this file.](#)

```

00001
00009
00010 #ifndef MODEL_JSON_UTILS_H
00011 #define MODEL_JSON_UTILS_H
00012
00013 #include <ctime>
00014 #include <iomanip>
00015 #include <sstream>
00016 #include <string>
00017
00022 namespace ModelJson {
00023
00024
00035 inline std::string escape(const std::string& input) {
00036     std::ostringstream out;
00037     for (unsigned char c : input) {
00038         switch (c) {
00039             case '"': out << "\\\""; break;
00040             case '\\': out << "\\\""; break;
00041             case 'b': out << "\\b"; break;
00042             case 'f': out << "\\f"; break;
00043             case 'n': out << "\\n"; break;
00044             case 'r': out << "\\r"; break;
00045             case 't': out << "\\t"; break;
00046             default:
00047                 if (c < 0x20) {
00048                     out << "\\u"
00049                         << std::hex << std::uppercase << std::setw(4)
00050                         << std::setfill('0') << static_cast<int>(c)
00051                         << std::dec;
00052                 } else {
00053                     out << static_cast<char>(c);
00054                 }
00055             }
00056         }
00057     }
00058     return out.str();
00059 }
00069 inline std::string quote(const std::string& input) {
00070     return std::string("\"") + escape(input) + "\"";
00071 }
00072
00082 inline std::string toIso8601(std::time_t timestamp) {
00083     std::tm utc{};
00084     #if defined(_WIN32)
00085         gmtime_s(&utc, &timestamp);
00086     #else
00087         gmtime_r(&timestamp, &utc);
00088     #endif
00089
00090     std::ostringstream out;
00091     out << std::put_time(&utc, "%Y-%m-%dT%H:%M:%SZ");
00092     return out.str();
00093 }
00094
00095 } // namespace ModelJson
00096
00097 #endif

```

7.81 Message.cpp File Reference

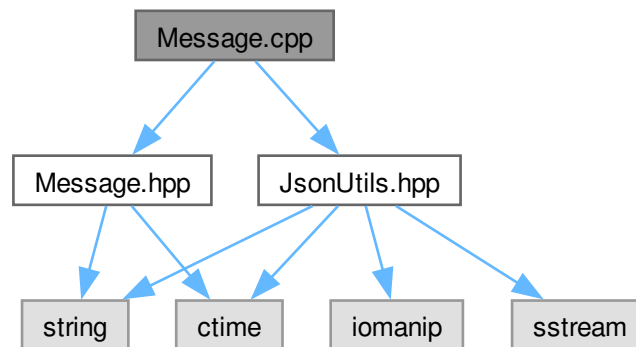
Implementation for [Message](#).

```

#include "Message.hpp"
#include "JsonUtils.hpp"

```

Include dependency graph for Message.cpp:



7.81.1 Detailed Description

Implementation for [Message](#).

Author

Jasmine Kumar

Definition in file [Message.cpp](#).

7.82 Message.cpp

[Go to the documentation of this file.](#)

```

00001
00006
00007 #include "Message.hpp"
00008 #include "JsonUtils.hpp"
00009
00010 std::string Message::toString() const {
00011     return "{"
00012         "\"sender_id\": " + ModelJson::quote(userID) + ", "
00013         "\"content\": " + ModelJson::quote(text) + ", "
00014         "\"sent_at\": " + ModelJson::quote(ModelJson::toIso8601(timestamp))
00015         + "}";
00016 }

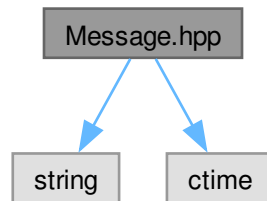
```

7.83 Message.hpp File Reference

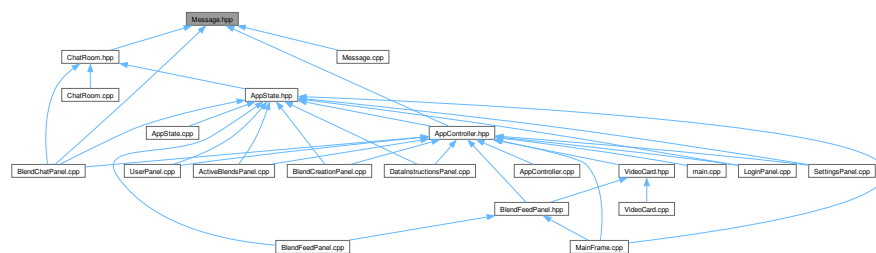
A single chat message sent within a [ChatRoom](#).

```
#include <string>
#include <ctime>
```

Include dependency graph for Message.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- struct [Message](#)
A single chat message sent within a [ChatRoom](#).

7.83.1 Detailed Description

A single chat message sent within a [ChatRoom](#).

Author

Jasmine Kumar

Kept as a plain struct since it is pure data with no behaviour.

Definition in file [Message.hpp](#).

7.84 Message.hpp

[Go to the documentation of this file.](#)

```

00001 #ifndef MESSAGE_H
00002 #define MESSAGE_H
00003
00011
00012 #include <string>
00013 #include <ctime>
00014
00022 struct Message {
00023     std::string userID;
00024     std::string text;
00025     std::time_t timestamp;
00026
00033     Message(const std::string& userID,
00034             const std::string& text,
00035             std::time_t timestamp = std::time(nullptr))
00036         : userID(userID), text(text), timestamp(timestamp) {}
00037
00042     std::string toString() const;
00043 };
00044
00045 #endif

```

7.85 User.cpp File Reference

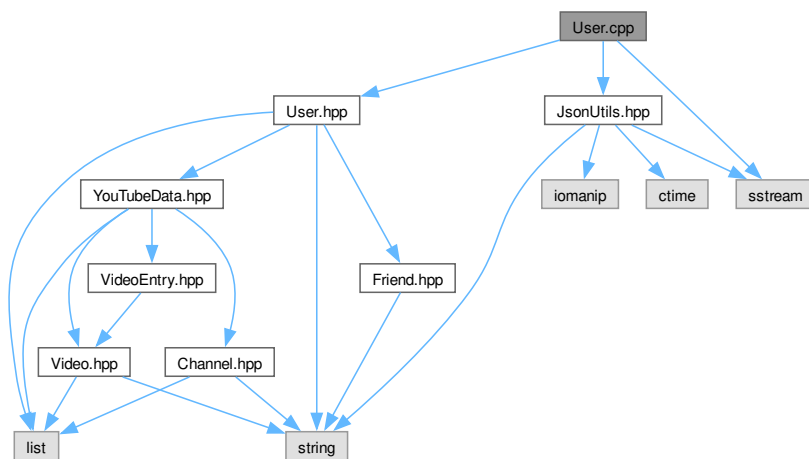
Implementation for [User](#).

```

#include "User.hpp"
#include "JsonUtils.hpp"
#include <sstream>

```

Include dependency graph for User.cpp:



7.85.1 Detailed Description

Implementation for [User](#).

Author

Jasmine Kumar

Definition in file [User.cpp](#).

7.86 User.cpp

[Go to the documentation of this file.](#)

```

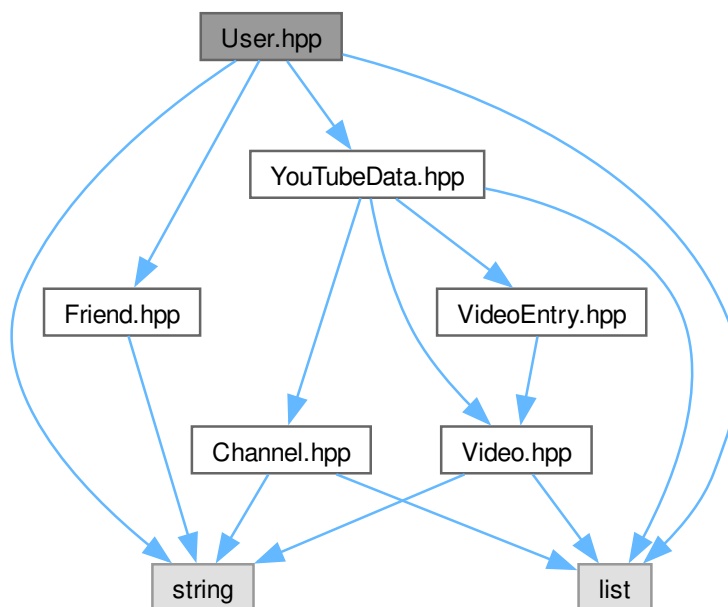
00001
00006
00007 #include "User.hpp"
00008
00009 #include "JsonUtils.hpp"
00010
00011 #include <sstream>
00012
00013 User::User(const std::string& userID,
00014           const std::string& username,
00015           const std::string& email,
00016           const std::string& password)
00017     : userID(userID), username(username), email(email), password(password) {}
00018
00019 // Getters
00020
00021 std::string User::getUserID() const { return userID; }
00022 std::string User::getUsername() const { return username; }
00023 std::string User::getEmail() const { return email; }
00024 std::string User::getPassword() const { return password; }
00025
00026 std::list<Friend> User::getFriends() const { return friends; }
00027
00028 YouTubeData& User::getYouTubeData() { return youtubeData; }
00029 const YouTubeData& User::getYouTubeData() const { return youtubeData; }
00030
00031 // Setters
00032
00033 void User::setUsername(const std::string& username) { this->username = username; }
00034 void User::setEmail(const std::string& email) { this->email = email; }
00035 void User::setPassword(const std::string& password) { this->password = password; }
00036
00037 void User::setYouTubeData(const YouTubeData& youtubeData) {
00038     this->youtubeData = youtubeData;
00039 }
00040
00041 // Friend management
00042
00043 void User::addFriend(const User& user) {
00044     for (const auto& f : friends) {
00045         if (f.getUserID() == user.getUserID()) return; // no duplicates
00046     }
00047     friends.emplace_back(user.getUserID(), user.getUsername(), user.getEmail());
00048 }
00049
00050 void User::removeFriend(const std::string& userID) {
00051     friends.remove_if([&userID](const Friend& f) {
00052         return f.getUserID() == userID;
00053     });
00054 }
00055
00056 std::string User::toString() const {
00057     std::ostringstream friendsJson;
00058     friendsJson << "[";
00059     bool firstFriend = true;
00060     for (const auto& friendUser : friends) {
00061         if (!firstFriend) {
00062             friendsJson << ",";
00063         }
00064         friendsJson << friendUser.toString();
00065         firstFriend = false;
00066     }
00067     friendsJson << "];";
00068
00069     return "{"
00070         "\"user_id\":\"" + ModelJson::quote(userID) + "\", "
00071         "\"username\":\"" + ModelJson::quote(username) + "\", "
00072         "\"email\":\"" + ModelJson::quote(email) + "\", "
00073         "\"friends\":\"" + friendsJson.str() + "\", "
00074         "\"youtube_data\":\"" + youtubeData.toString()
00075         + "\"}";
00076 }

```

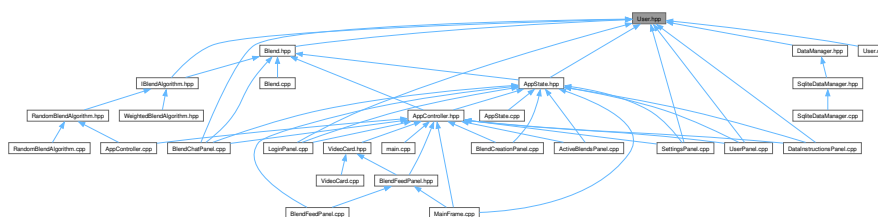
7.87 User.hpp File Reference

Model representing a registered application user.

```
#include <string>
#include <list>
#include "Friend.hpp"
#include "YouTubeData.hpp"
Include dependency graph for User.hpp:
```



This graph shows which files directly or indirectly include this file:



Classes

- class `User`
`User` class declaration.

7.87.1 Detailed Description

Model representing a registered application user.

Author

Jasmine Kumar

Holds identity fields and the user's associated YouTube data. Passwords are stored as plaintext at this stage of development.

Definition in file [User.hpp](#).

7.88 User.hpp

[Go to the documentation of this file.](#)

```

00001 #ifndef USER_H
00002 #define USER_H
00003
00012
00013 #include <string>
00014 #include <list>
00015
00016 #include "Friend.hpp"
00017 #include "YouTubeData.hpp"
00018
00024 class User {
00025     private:
00026         std::string userID; // generated id
00027         std::string username; // provided username
00028         std::string email; // provided user email
00029         std::string password;
00030         std::list<Friend> friends; // list of friends -- to be fully implemented as a
        backend/client/UI feature in the future
        YouTubeData youTubeData; // provided YouTube Data on file upload
00031
00032     public:
00033         User(const std::string& userID,
00041             const std::string& username,
00042             const std::string& email,
00043             const std::string& password);
00044
00045         std::string      getUserID() const;
00047         std::string      getUsername() const;
00049         std::string      getEmail() const;
00051         std::string      getPassword() const;
00053         std::list<Friend> getFriends() const;
00055         YouTubeData&      getYouTubeData();
00057         const YouTubeData& getYouTubeData() const;
00059
00060         void setUsername(const std::string& username);
00062         void setEmail(const std::string& email);
00064         void setPassword(const std::string& password);
00066         void setYouTubeData(const YouTubeData& youTubeData);
00068
00069         void addFriend(const User& user);
00074
00075         void removeFriend(const std::string& userID);
00080
00081         std::string toString() const;
00086
00087 };
00088
00089 #endif

```

7.89 Video.cpp File Reference

Implementation for [Video](#).

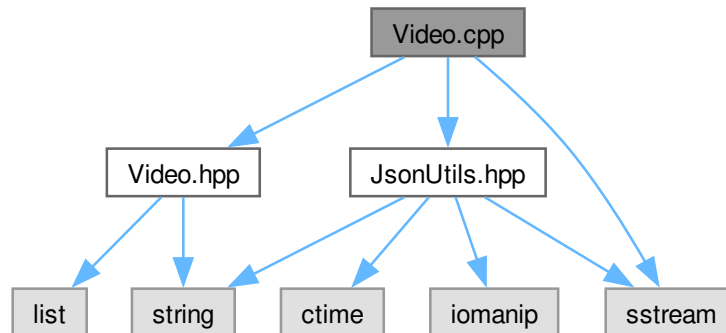
```

#include "Video.hpp"
#include "JsonUtils.hpp"

```

```
#include <sstream>
```

Include dependency graph for Video.cpp:



7.89.1 Detailed Description

Implementation for [Video](#).

Author

Jasmine Kumar

Definition in file [Video.cpp](#).

7.90 Video.cpp

[Go to the documentation of this file.](#)

```

00001
00006
00007 #include "Video.hpp"
00008 #include "JsonUtils.hpp"
00009
00010 #include <sstream>
00011
00012 Video::Video(const std::string& videoID,
00013             const std::string& title,
00014             const std::string& channelID,
00015             const std::string& thumbnailURL,
00016             int duration,
00017             const std::list<std::string>& tags,
00018             const std::string& channelName,
00019             const std::string& channelLogoURL)
00020 : videoID(videoID),
00021   title(title),
00022   channelID(channelID),
00023   thumbnailURL(thumbnailURL),
00024   duration(duration),
00025   tags(tags),
00026   channelName(channelName),
00027   channelLogoURL(channelLogoURL) {}
00028
00029 std::string Video::getVideoID() const {
00030     return videoID;
00031 }
  
```

```

00032
00033 std::string Video::getTitle() const {
00034     return title;
00035 }
00036
00037 std::string Video::getChannelID() const {
00038     return channelID;
00039 }
00040
00041 std::string Video::getThumbnailURL() const {
00042     return thumbnailURL;
00043 }
00044
00045 int Video::getDuration() const {
00046     return duration;
00047 }
00048
00049 std::list<std::string> Video::getTags() const {
00050     return tags;
00051 }
00052
00053 std::string Video::getChannelName() const {
00054     return channelName;
00055 }
00056
00057 std::string Video::getChannelLogoURL() const {
00058     return channelLogoURL;
00059 }
00060
00061 std::string Video::toString() const {
00062     std::ostringstream tagsJson;
00063     tagsJson << "[";
00064     bool first = true;
00065     for (const auto& tag : tags) {
00066         if (!first) {
00067             tagsJson << ",";
00068         }
00069         tagsJson << ModelJson::quote(tag);
00070         first = false;
00071     }
00072     tagsJson << "]";
00073
00074     return "{"
00075         "\"video_id\": " + ModelJson::quote(videoID) + ","
00076         "\"title\": " + ModelJson::quote(title) + ","
00077         "\"channel_id\": " + ModelJson::quote(channelID) + ","
00078         "\"thumbnail_url\": " + ModelJson::quote(thumbnailURL) + ","
00079         "\"duration\": " + std::to_string(duration) + ","
00080         "\"tags\": " + tagsJson.str()
00081         + "}";
00082 }

```

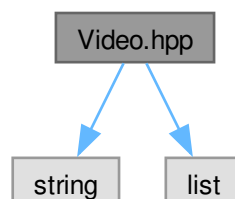
7.91 Video.hpp File Reference

Immutable data model for a single YouTube video.

```
#include <string>
```

```
#include <list>
```

Include dependency graph for Video.hpp:




```

00051         const std::string& channelLogoURL = "");
00052
00053     std::string getVideoID() const;
00054     std::string getTitle() const;
00055     std::string getChannelID() const;
00056     std::string getThumbnailURL() const;
00057     int getDuration() const;
00058     std::list<std::string> getTags() const;
00059     std::string getChannelName() const;
00060     std::string getChannelLogoURL() const;
00061
00062     std::string toString() const;
00063 };
00064 #endif

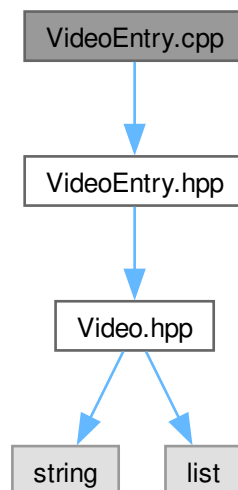
```

7.93 VideoEntry.cpp File Reference

Implementation for [VideoEntry](#).

```
#include "VideoEntry.hpp"
```

Include dependency graph for VideoEntry.cpp:



7.93.1 Detailed Description

Implementation for [VideoEntry](#).

Author

Jasmine Kumar

Definition in file [VideoEntry.cpp](#).

7.94 VideoEntry.cpp

[Go to the documentation of this file.](#)

```

00001
00006
00007 #include "VideoEntry.hpp"
00008
00009 VideoEntry::VideoEntry(const Video& video, int watchCount, int lastWatched)
00010     : video(video), watchCount(watchCount), lastWatched(lastWatched) {}
00011
00012 // Getters
00013
00014 Video VideoEntry::getVideo() const {
00015     return video;
00016 }
00017
00018 int VideoEntry::getWatchCount() const {
00019     return watchCount;
00020 }
00021
00022 int VideoEntry::getLastWatched() const {
00023     return lastWatched;
00024 }
00025
00026 // Setters
00027 void VideoEntry::setWatchCount(int watchCount) {
00028     this->watchCount = watchCount;
00029 }
00030
00031 void VideoEntry::setLastWatched(int lastWatched) {
00032     this->lastWatched = lastWatched;
00033 }
00034
00035 std::string VideoEntry::toString() const {
00036     return "{" +
00037         "\"video\": " + video.toString() + ", " +
00038         "\"watch_count\": " + std::to_string(watchCount) + ", " +
00039         "\"last_watched\": " + std::to_string(lastWatched) +
00040         "}";
00041 }

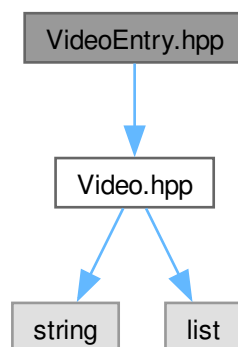
```

7.95 VideoEntry.hpp File Reference

A [Video](#) decorated with watch-history metadata.

```
#include "Video.hpp"
```

Include dependency graph for VideoEntry.hpp:

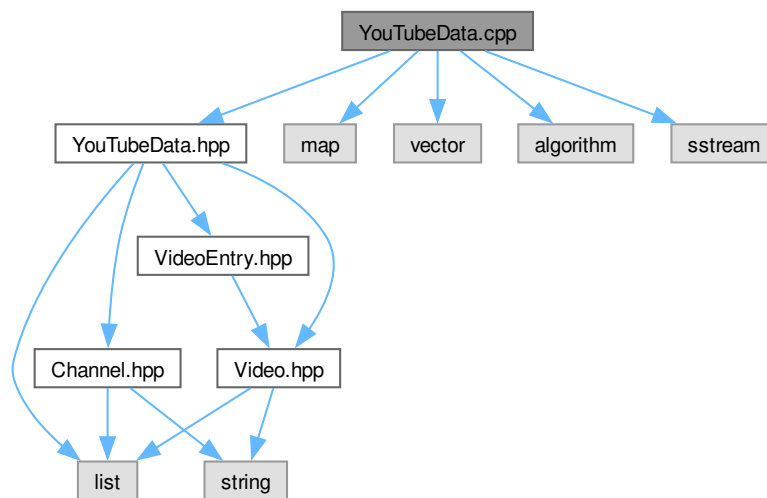


7.97 YouTubeData.cpp File Reference

Implementation for [YouTubeData](#).

```
#include "YouTubeData.hpp"
#include <map>
#include <vector>
#include <algorithm>
#include <sstream>
```

Include dependency graph for YouTubeData.cpp:



7.97.1 Detailed Description

Implementation for [YouTubeData](#).

Author

Jasmine Kumar

Definition in file [YouTubeData.cpp](#).

7.98 YouTubeData.cpp

[Go to the documentation of this file.](#)

```
00001
00006
00007 #include "YouTubeData.hpp"
00008 #include <map>
00009 #include <vector>
00010 #include <algorithm>
00011 #include <sstream>
00012
```

```

00013 YouTubeData::YouTubeData() {}
00014
00015 // Getters
00016 std::list<VideoEntry> YouTubeData::getWatchHistory() const {
00017     return watchHistory;
00018 }
00019
00020 std::list<Video> YouTubeData::getLikedVideos() const {
00021     return likedVideos;
00022 }
00023
00024 std::list<Channel> YouTubeData::getSubscribedChannels() const {
00025     return subscribedChannels;
00026 }
00027
00028 //Setters
00029
00030 void YouTubeData::setWatchHistory(const std::list<VideoEntry>& watchHistory) {
00031     this->watchHistory = watchHistory;
00032 }
00033
00034 void YouTubeData::setLikedVideos(const std::list<Video>& likedVideos) {
00035     this->likedVideos = likedVideos;
00036 }
00037
00038 void YouTubeData::setSubscribedChannels(const std::list<Channel>& subscribedChannels) {
00039     this->subscribedChannels = subscribedChannels;
00040 }
00041
00042 std::list<Video> YouTubeData::getWatchLaterVideos() const {
00043     return watchLaterVideos;
00044 }
00045
00046 void YouTubeData::setWatchLaterVideos(const std::list<Video>& videos) {
00047     this->watchLaterVideos = videos;
00048 }
00049
00050 // Computed Queries
00051
00052 std::list<Video> YouTubeData::getMostWatchedVideos(int n) const {
00053     // Copy watch history into a vector for sorting
00054     std::vector<VideoEntry> entries(watchHistory.begin(), watchHistory.end());
00055
00056     // Sort copied vector
00057     std::sort(entries.begin(), entries.end(),
00058         [](const VideoEntry& a, const VideoEntry& b) {
00059             return a.getWatchCount() > b.getWatchCount();
00060         });
00061     // Add sorted results to a list
00062     std::list<Video> result;
00063     int count = 0;
00064     for (const auto& entry : entries) {
00065         if (count >= n) break;
00066         result.push_back(entry.getVideo());
00067         count++;
00068     }
00069     return result;
00070 }
00071
00072 std::list<Channel> YouTubeData::getTopChannels(int n) const {
00073     // Count how many watched videos belong to each channelId
00074     std::map<std::string, int> channelWatchCounts;
00075     for (const auto& entry : watchHistory) {
00076         const std::string& cid = entry.getVideo().getChannelID();
00077         channelWatchCounts[cid] += entry.getWatchCount();
00078     }
00079
00080     // Sort subscribedChannels by their accumulated watch count
00081     std::vector<Channel> channels(subscribedChannels.begin(), subscribedChannels.end());
00082     std::sort(channels.begin(), channels.end(),
00083         [&channelWatchCounts](const Channel& a, const Channel& b) {
00084             int countA = channelWatchCounts.count(a.getChannelID()) ?
channelWatchCounts.at(a.getChannelID()) : 0;
00085             int countB = channelWatchCounts.count(b.getChannelID()) ?
channelWatchCounts.at(b.getChannelID()) : 0;
00086             return countA > countB;
00087         });
00088
00089     std::list<Channel> result;
00090     int count = 0;
00091     for (const auto& channel : channels) {
00092         if (count >= n) break;
00093         result.push_back(channel);
00094         count++;
00095     }
00096     return result;
00097 }

```

```

00098
00099 std::string YouTubeData::toString() const {
00100     std::ostringstream watchHistoryJson;
00101     watchHistoryJson << "[";
00102     bool firstWatch = true;
00103     for (const auto& entry : watchHistory) {
00104         if (!firstWatch) {
00105             watchHistoryJson << ",";
00106         }
00107         watchHistoryJson << entry.toString();
00108         firstWatch = false;
00109     }
00110     watchHistoryJson << "]";
00111
00112     std::ostringstream likedJson;
00113     likedJson << "[";
00114     bool firstLiked = true;
00115     for (const auto& video : likedVideos) {
00116         if (!firstLiked) {
00117             likedJson << ",";
00118         }
00119         likedJson << video.toString();
00120         firstLiked = false;
00121     }
00122     likedJson << "]";
00123
00124     std::ostringstream channelsJson;
00125     channelsJson << "[";
00126     bool firstChannel = true;
00127     for (const auto& channel : subscribedChannels) {
00128         if (!firstChannel) {
00129             channelsJson << ",";
00130         }
00131         channelsJson << channel.toString();
00132         firstChannel = false;
00133     }
00134     channelsJson << "]";
00135
00136     std::ostringstream watchLaterJson;
00137     watchLaterJson << "[";
00138     bool firstLater = true;
00139     for (const auto& video : watchLaterVideos) {
00140         if (!firstLater) {
00141             watchLaterJson << ",";
00142         }
00143         watchLaterJson << video.toString();
00144         firstLater = false;
00145     }
00146     watchLaterJson << "]";
00147
00148     return "{"
00149         "\"watch_history\": " + watchHistoryJson.str() + ","
00150         "\"liked_videos\": " + likedJson.str() + ","
00151         "\"subscribed_channels\": " + channelsJson.str() + ","
00152         "\"watch_later\":{" + "\"videos\": " + watchLaterJson.str() + "}"
00153         + "}";
00154 }

```

7.99 YouTubeData.hpp File Reference

Container for all YouTube data associated with a [User](#).

```

#include <list>
#include "VideoEntry.hpp"
#include "Channel.hpp"
#include "Video.hpp"

```


7.100 YouTubeData.hpp

[Go to the documentation of this file.](#)

```

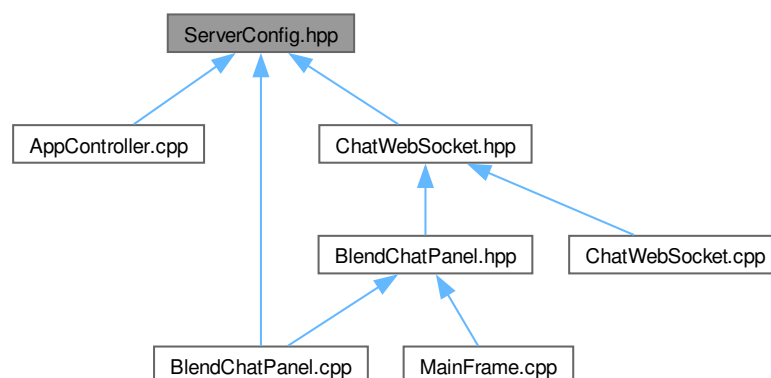
00001 #ifndef YOUTUBEDATA_H
00002 #define YOUTUBEDATA_H
00003
00017
00018 #include <list>
00019
00020 #include "VideoEntry.hpp"
00021 #include "Channel.hpp"
00022 #include "Video.hpp"
00023
00029 class YouTubeData {
00030     private:
00031         std::list<VideoEntry> watchHistory; // further enriched list of videos from users watch
00032     history
00033         std::list<Video> likedVideos; // list of videos from users liked playlist
00034         std::list<Channel> subscribedChannels; // list of users subscribed channels
00035         std::list<Video> watchLaterVideos; // list of videos from users watch later (any
00036     playlist)
00037     public:
00038         YouTubeData();
00039
00040         std::list<VideoEntry> getWatchHistory() const;
00041         std::list<Video> getLikedVideos() const;
00042         std::list<Channel> getSubscribedChannels() const;
00043         std::list<Video> getWatchLaterVideos() const;
00044
00045         void setWatchHistory(const std::list<VideoEntry>& watchHistory);
00046         void setLikedVideos(const std::list<Video>& likedVideos);
00047         void setSubscribedChannels(const std::list<Channel>& subscribedChannels);
00048         void setWatchLaterVideos(const std::list<Video>& videos);
00049
00050         std::list<Video> getMostWatchedVideos(int n) const;
00051
00052         std::list<Channel> getTopChannels(int n) const;
00053
00054         std::string toString() const;
00055 };
00056
00057 #endif

```

7.101 ServerConfig.hpp File Reference

Centralized backend server addresses.

This graph shows which files directly or indirectly include this file:



Variables

- `constexpr const char * kAppBackendBaseUrl = "http://137.220.58.22:8080"`
HTTP base URL used by [AppController](#) and other app-side HTTP calls.
- `constexpr const char * kAppBackendWsPrefix = "ws://137.220.58.22:8080/api/v1/ws/chats/"`
WebSocket endpoint prefix used by [BlendChatPanel](#).
- `constexpr const char * kTestBackendBaseUrl = "http://137.220.58.22:8080"`
HTTP base URL used by the test suite.
- `constexpr const char * kTestBackendWsPrefix = "ws://137.220.58.22:8080/api/v1/ws/chats/"`
WebSocket endpoint prefix used by tests that exercise WS connectivity.

7.101.1 Detailed Description

Centralized backend server addresses.

Author

Shamar Pennant

Edit the constants below to retarget the application and/or the test suite to a different server. No other file needs to change.

- `kApp*` constants are used by the running application ([AppController](#), [UILayer](#)).
- `kTest*` constants are used by the test suite and can point at a separate staging instance if needed.

Definition in file [ServerConfig.hpp](#).

7.101.2 Variable Documentation

7.101.2.1 kAppBackendBaseUrl

```
const char* kAppBackendBaseUrl = "http://137.220.58.22:8080" [inline], [constexpr]
```

HTTP base URL used by [AppController](#) and other app-side HTTP calls.

Definition at line 17 of file [ServerConfig.hpp](#).

7.101.2.2 kAppBackendWsPrefix

```
const char* kAppBackendWsPrefix = "ws://137.220.58.22:8080/api/v1/ws/chats/" [inline], [constexpr]
```

WebSocket endpoint prefix used by [BlendChatPanel](#).

Definition at line 20 of file [ServerConfig.hpp](#).

7.101.2.3 kTestBackendBaseUrl

```
const char* kTestBackendBaseUrl = "http://137.220.58.22:8080" [inline], [constexpr]
```

HTTP base URL used by the test suite.

Definition at line 23 of file [ServerConfig.hpp](#).

7.101.2.4 kTestBackendWsPrefix

```
const char* kTestBackendWsPrefix = "ws://137.220.58.22:8080/api/v1/ws/chats/" [inline], [constexpr]
```

WebSocket endpoint prefix used by tests that exercise WS connectivity.

Definition at line 26 of file [ServerConfig.hpp](#).

7.102 ServerConfig.hpp

[Go to the documentation of this file.](#)

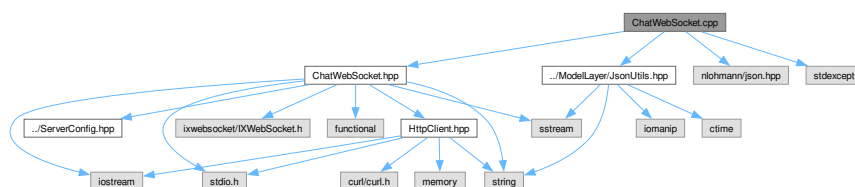
```
00001 #pragma once
00002
00015
00017 inline constexpr const char* kAppBackendBaseUrl = "http://137.220.58.22:8080";
00018
00020 inline constexpr const char* kAppBackendWsPrefix = "ws://137.220.58.22:8080/api/v1/ws/chats/";
00021
00023 inline constexpr const char* kTestBackendBaseUrl = "http://137.220.58.22:8080";
00024
00026 inline constexpr const char* kTestBackendWsPrefix = "ws://137.220.58.22:8080/api/v1/ws/chats/";
```

7.103 ChatWebSocket.cpp File Reference

Implementation for [ChatWebSocket](#).

```
#include "ChatWebSocket.hpp"
#include "../ModelLayer/JsonUtils.hpp"
#include <nlohmann/json.hpp>
#include <stdexcept>
```

Include dependency graph for ChatWebSocket.cpp:



Functions

- static std::string [urlEncode](#) (const std::string &value)

7.103.1 Detailed Description

Implementation for [ChatWebSocket](#).

Author

Shamar Pennant

Definition in file [ChatWebSocket.cpp](#).

7.103.2 Function Documentation

7.103.2.1 urlEncode()

```
std::string urlEncode (
    const std::string & value) [static]
```

Definition at line 16 of file [ChatWebSocket.cpp](#).

7.104 ChatWebSocket.cpp

[Go to the documentation of this file.](#)

```
00001
00006
00007 #include "ChatWebSocket.hpp"
00008 #include "../ModelLayer/JsonUtils.hpp"
00009
00010 #include <nlohmann/json.hpp>
00011 #include <stdexcept>
00012
00013 using namespace std;
00014 using Json = nlohmann::json;
00015
00016 static std::string urlEncode(const std::string& value) {
00017     if (value.empty()) {
00018         return value;
00019     }
00020
00021     CURL* encoder = curl_easy_init();
00022     if (encoder == nullptr) {
00023         return value;
00024     }
00025
00026     char* escaped = curl_easy_escape(encoder, value.c_str(), static_cast<int>(value.size()));
00027     if (escaped == nullptr) {
00028         curl_easy_cleanup(encoder);
00029         return value;
00030     }
00031
00032     std::string encoded(escaped);
00033     curl_free(escaped);
00034     curl_easy_cleanup(encoder);
00035     return encoded;
00036 }
00037
00038 ChatWebSocket::ChatWebSocket(const string url) {
00039     this->url = url;
00040
00041     // setting up specialized broadcasting and functions
00042     ws.setOnMessageCallback([this](const ix::WebSocketMessagePtr& msg) {
00043         if (msg->type == ix::WebSocketMessageType::Message) {
00044             std::cout << "WS Received: " << msg->str << "\n";
00045             // Parse sender/content from contract-shaped outbound server messages.
00046             if (m_onMessage) {
00047                 Json parsed = Json::parse(msg->str, nullptr, false);
00048                 if (parsed.is_discarded() || !parsed.is_object()) {
00049                     return;
```

```

00050         }
00051
00052         std::string sender = parsed.value("sender_id", "");
00053         std::string content = parsed.value("content", "");
00054         if (!sender.empty() && !content.empty()) {
00055             m_onMessage(sender, content);
00056         }
00057     }
00058 }
00059 else if (msg->type == ix::WebSocketMessageType::Open) {
00060     std::cout << "WS Connected\n";
00061 }
00062 else if (msg->type == ix::WebSocketMessageType::Error) {
00063     std::cerr << "WS Error: " << msg->errorInfo.reason << "\n";
00064 }
00065 });
00066 }
00067
00068 ChatWebSocket::~ChatWebSocket() {
00069     if (isStarted) {
00070         ws.stop();
00071     }
00072 }
00073
00074 void ChatWebSocket::set_blendID(const string blendID) { this->blendID = blendID; }
00075 void ChatWebSocket::set_userID(const string userID) { this->userID = userID; }
00076 string ChatWebSocket::get_fullUrl() { return fullUrl; }
00077
00078 void ChatWebSocket::connect() {
00079     if (isStarted) return;
00080     build_fullUrl();
00081     ws.setUrl(fullUrl);
00082     ws.start();
00083     isStarted = true;
00084 }
00085
00086 void ChatWebSocket::setOnMessage(
00087     std::function<void(const std::string&, const std::string&)> cb) {
00088     m_onMessage = std::move(cb);
00089 }
00090
00091 void ChatWebSocket::send_message(string msg) {
00092     connect(); // no-op if already started
00093     ws.send(content_formatter(msg));
00094 }
00095
00096 string ChatWebSocket::content_formatter(string content) {
00097     return "{\"content\":\"" + ModelJson::quote(content) + "\"}";
00098 }
00099
00100 void ChatWebSocket::build_fullUrl() {
00101     // checks if blendID and userID have been correctly configured
00102     if (blendID.empty() || userID.empty()) {
00103         throw invalid_argument("ChatWebSocket requires blendID and userID before sending messages");
00104     }
00105
00106     stringstream fu;
00107     fu << url << urlEncode(blendID) << "?user_id=" << urlEncode(userID);
00108     fullUrl = fu.str();
00109 }

```

7.105 ChatWebSocket.hpp File Reference

Simple WebSocket wrapper.

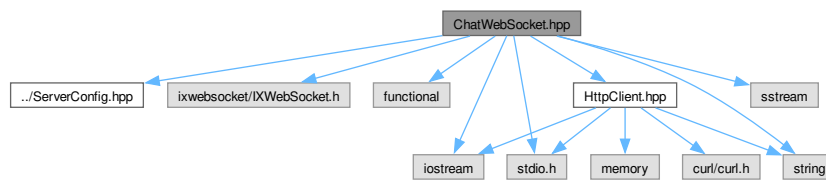
```

#include "../ServerConfig.hpp"
#include <ixwebsocket/IXWebSocket.h>
#include <functional>
#include <iostream>
#include <stdio.h>
#include <sstream>
#include <string>

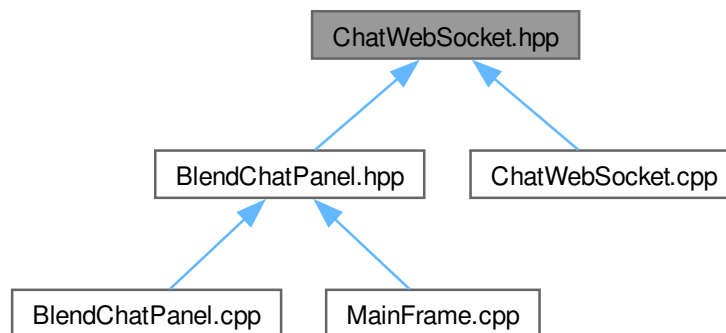
```

```
#include "HttpClient.hpp"
```

Include dependency graph for ChatWebSocket.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [ChatWebSocket](#)
[ChatWebSocket](#) class declaration.

7.105.1 Detailed Description

Simple WebSocket wrapper.

Author

Shamar Pennant

Simple websocket wrapper to ease communications with the server in regards to chatrooms

Definition in file [ChatWebSocket.hpp](#).

7.106 ChatWebSocket.hpp

[Go to the documentation of this file.](#)

```

00001 #pragma once
00002
00003 #include "../ServerConfig.hpp"
00004 #include <ixwebsocket/IXWebSocket.h>
00005 #include <functional>
00006 #include <iostream>
00007 #include <stdio.h>
00008 #include <sstream>
00009 #include <string>
00010 #include "HttpClient.hpp"
00011
00020
00025 class ChatWebSocket {
00026     public:
00034         ChatWebSocket(const std::string url);
00035
00039         ~ChatWebSocket();
00040
00046         void connect();
00047
00053         void setOnMessage(std::function<void(const std::string& senderID,
00054                                             const std::string& content)> cb);
00055
00060         void set_blendID(const std::string blendID);
00061
00066         void set_userID(const std::string userID);
00067
00072         std::string get_fullUrl();
00073
00079         void send_message(std::string msg);
00080
00081     private:
00082         // WebSocket object
00083         ix::WebSocket ws;
00084         // copy of the last given url
00085         std::string url = kAppBackendWsPrefix;
00086         // copy of the last given blendID for the socket path
00087         std::string blendID;
00088         // copy of the last given userID for the socket path
00089         std::string userID;
00090         // full websocket url
00091         std::string fullUrl;
00092         // Tracks whether the socket loop has been started.
00093         bool isStarted = false;
00094         // Optional callback invoked on incoming chat messages.
00095         std::function<void(const std::string&, const std::string&)> m_onMessage;
00096
00103         std::string content_formatter(std::string content);
00104
00109         void build_fullUrl();
00110 };

```

7.107 DataManager.hpp File Reference

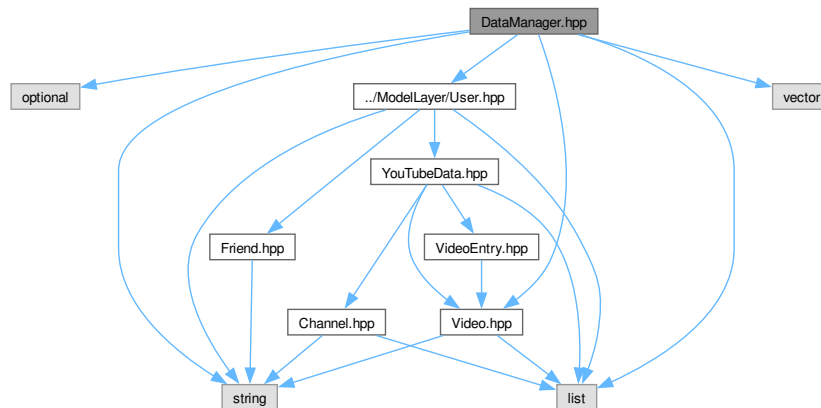
Interface for all user-persistence operations.

```

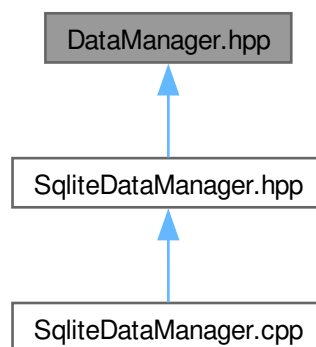
#include <optional>
#include <string>
#include <vector>
#include <list>
#include "../ModelLayer/User.hpp"
#include "../ModelLayer/Video.hpp"

```

Include dependency graph for DataManager.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [DataManager](#)
[DataManager](#) class declaration.

7.107.1 Detailed Description

Interface for all user-persistence operations.

Author

Shamar Pennant
Jasmine Kumar

[AppController](#) depends only on this interface, keeping it decoupled from the concrete storage back-end (SQLite today, something else tomorrow). Tests can inject a mock implementation without touching the real DB.

Definition in file [DataManager.hpp](#).

7.108 DataManager.hpp

[Go to the documentation of this file.](#)

```

00001 #ifndef DATA_MANAGER_H
00002 #define DATA_MANAGER_H
00003
00004 #include <optional>
00005 #include <string>
00006 #include <vector>
00007 #include <list>
00008 #include "../ModelLayer/User.hpp"
00009 #include "../ModelLayer/Video.hpp"
00010
00025 class DataManager {
00026 public:
00027     virtual ~DataManager() = default;
00028
00034     virtual bool createUser(const User& user) = 0;
00035
00040     virtual std::optional<User> findUserByID(const std::string& userID) = 0;
00041
00046     virtual bool validatePassword(const std::string& userID,
00047                                   const std::string& password) = 0;
00048
00053     virtual bool saveWatchLater(const std::string& userID,
00054                                  const std::list<Video>& videos) = 0;
00055
00060     virtual std::list<Video> loadWatchLater(const std::string& userID) = 0;
00061
00068     virtual bool saveBlend(const std::string& blendID,
00069                            const std::string& creatorID,
00070                            const std::string& algorithm,
00071                            const std::vector<std::string>& participantIDs) = 0;
00072
00077     virtual std::optional<std::string> findBlendForUser(const std::string& userID) = 0;
00078
00084     virtual std::vector<std::string> loadBlendParticipants(const std::string& blendID) = 0;
00085 };
00086
00087 #endif

```

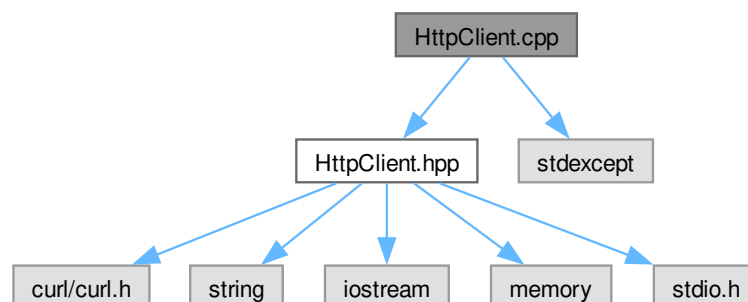
7.109 HttpClient.cpp File Reference

Implementation for [HttpClient](#).

```
#include "HttpClient.hpp"
```

```
#include <stdexcept>
```

Include dependency graph for HttpClient.cpp:



7.109.1 Detailed Description

Implementation for [HttpClient](#).

Author

Shamar Pennant

Definition in file [HttpClient.cpp](#).

7.110 HttpClient.cpp

[Go to the documentation of this file.](#)

```

00001
00006
00012
00013 #include "HttpClient.hpp"
00014
00015 #include <stdexcept>
00016
00017 using namespace std;
00018
00019 namespace {
00020
00021 string urlEncodeComponent(const string& value) {
00022     if (value.empty()) {
00023         return value;
00024     }
00025
00026     CURL* encoder = curl_easy_init();
00027     if (encoder == nullptr) {
00028         return value;
00029     }
00030
00031     char* escaped = curl_easy_escape(encoder, value.c_str(), static_cast<int>(value.size()));
00032     if (escaped == nullptr) {
00033         curl_easy_cleanup(encoder);
00034         return value;
00035     }
00036
00037     string encoded(escaped);
00038     curl_free(escaped);
00039     curl_easy_cleanup(encoder);
00040     return encoded;
00041 }
00042
00043 string sanitizePathSegment(const string& value) {
00044     return urlEncodeComponent(value);
00045 }
00046
00047 }
00048
00049 HttpClient::HttpClient(string base) {
00050     baseUrl = move(base);
00051 }
00052
00053 bool HttpClient::isRequestSuccessful(const string& method, long statusCode) {
00054     if (method == "GET") {
00055         return statusCode == 200;
00056     }
00057     if (method == "POST") {
00058         return statusCode == 200 || statusCode == 201;
00059     }
00060     if (method == "DELETE") {
00061         return statusCode == 200 || statusCode == 204;
00062     }
00063
00064     throw invalid_argument("HttpClient::isRequestSuccessful received unsupported method: " + method);
00065 }
00066
00067 long HttpClient::getLastStatusCode() const {
00068     return lastStatusCode;
00069 }
00070
00071 bool HttpClient::wasLastRequestSuccessful(const string& method) const {

```



```

00072     return isRequestSuccessful(method, lastStatusCode);
00073 }
00074
00075 string HttpClient::get(const string& path) {
00076     return request(G, path, "");
00077 }
00078
00079 string HttpClient::post(const string& path, const string& json) {
00080     return request(P, path, json);
00081 }
00082
00083 string HttpClient::del(const string& path, const string& json) {
00084     return request(D, path, json);
00085 }
00086
00087 string HttpClient::build_watch_later_endpoint(string userID) {
00088     return "/api/v1/users/" + sanitizePathSegment(userID) + "/watch-later";
00089 }
00090
00091 string HttpClient::build_latest_blend_endpoint(string userID) {
00092     return "/api/v1/users/" + sanitizePathSegment(userID) + "/blend";
00093 }
00094
00095 string HttpClient::build_blend_participant_endpoint(string blendID) {
00096     return "/api/v1/blends/" + sanitizePathSegment(blendID) + "/participants";
00097 }
00098
00099 string HttpClient::build_chatroom_detail_endpoint(string blendID) {
00100     return "/api/v1/blends/" + sanitizePathSegment(blendID) + "/chatroom";
00101 }
00102
00103 string HttpClient::build_auth_user_lookup_endpoint(string userID) {
00104     return AUTH_USER_PREFIX + sanitizePathSegment(userID);
00105 }
00106
00107 string HttpClient::build_delete_user_endpoint(string userID) {
00108     return DELETE_USER_PREFIX + sanitizePathSegment(userID);
00109 }
00110
00111 string HttpClient::request(const string& method, const string& path, const string&body) {
00112     // grabbing a new curl object
00113     CURL* curl = curl_easy_init();
00114     if (!curl) {
00115         throw runtime_error("HttpClient failed to initialize CURL handle");
00116     }
00117
00118     // string containing the response from the HTTP method
00119     string response;
00120     // full path to endpoint
00121     const string fullUrl = baseUrl + path;
00122
00123     // --- setting up curl for requests ---
00124
00125     // instruct curl where to send the the request
00126     curl_easy_setopt(curl, CURLOPT_URL, fullUrl.c_str());
00127     // curl doesnt return the call directly, so we need write_callback()
00128     // before being able to grab the response
00129     curl_easy_setopt(curl, CURLOPT_WRITEFUNCTION, write_callback);
00130     curl_easy_setopt(curl, CURLOPT_WRITEDATA, &response);
00131
00132     // create the header for our request (following curl design patterns)
00133     struct curl_slist* headers = nullptr;
00134     headers = curl_slist_append(headers, hdr.c_str());
00135     curl_easy_setopt(curl, CURLOPT_HTTPHEADER, headers);
00136
00137     // --- handling given method ---
00138     if (method == P) { // POST
00139         // send the given body
00140         curl_easy_setopt(curl, CURLOPT_POST, 1L);
00141         curl_easy_setopt(curl, CURLOPT_POSTFIELDS, body.c_str());
00142         curl_easy_setopt(curl, CURLOPT_POSTFIELDSIZE, static_cast<long>(body.size()));
00143     } else if (method == G) { // GET
00144
00145         curl_easy_setopt(curl, CURLOPT_HTTPGET, 1L);
00146     } else if (method == D) { // DELETE
00147         curl_easy_setopt(curl, CURLOPT_CUSTOMREQUEST, D.c_str());
00148         curl_easy_setopt(curl, CURLOPT_POSTFIELDS, body.c_str());
00149         curl_easy_setopt(curl, CURLOPT_POSTFIELDSIZE, static_cast<long>(body.size()));
00150     } else {
00151         curl_easy_cleanup(curl);
00152         curl_slist_free_all(headers);
00153         throw invalid_argument("HttpClient::request received unsupported method: " + method);
00154     }
00155
00156     // firing off the request and clean up all pointers

```

```

00168     CURLcode result = curl_easy_perform(curl);
00169
00170     long statusCode = 0;
00171     curl_easy_getinfo(curl, CURLINFO_RESPONSE_CODE, &statusCode);
00172     lastStatusCode = statusCode;
00173
00174     curl_easy_cleanup(curl);
00175     curl_slist_free_all(headers);
00176
00177     if (result != CURLE_OK) {
00178         throw runtime_error(string("HttpClient request failed: ") + curl_easy_strerror(result));
00179     }
00180
00181     return response;
00182 }
00183
00184 string HttpClient::build_user_blends_endpoint(string userID) {
00185     return "/api/v1/users/" + sanitizePathSegment(userID) + "/blends";
00186 }
00187
00188 string HttpClient::build_leave_blend_endpoint(string blendID) {
00189     return "/api/v1/blends/" + sanitizePathSegment(blendID) + "/leave";
00190 }
00191
00192 string HttpClient::build_chat_history_endpoint(string blendID, string userID) {
00193     return "/api/v1/blends/" + sanitizePathSegment(blendID) + "/chat-history?user_id=" +
        sanitizePathSegment(userID);
00194 }
00195
00196 size_t HttpClient::write_callback(void* contents, size_t size, size_t nmemb, void* out) {
00197     auto* response = static_cast<string*>(out);
00198     response->append(static_cast<char*>(contents), size * nmemb);
00199
00200     return size * nmemb;
00201 }

```

7.111 HttpClient.hpp File Reference

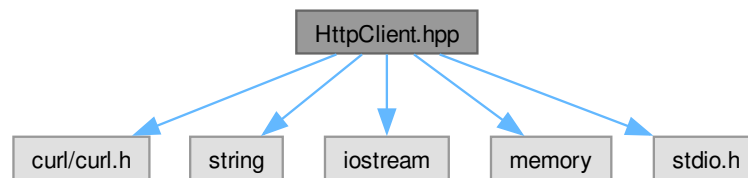
Simple HTTP client wrapper.

```

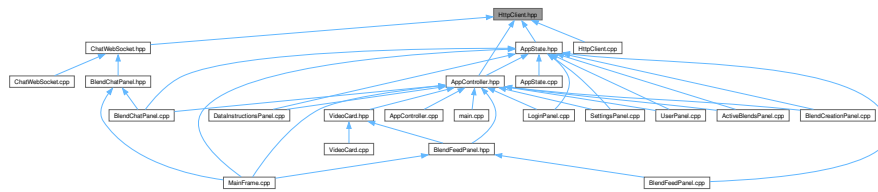
#include <curl/curl.h>
#include <string>
#include <iostream>
#include <memory>
#include <stdio.h>

```

Include dependency graph for HttpClient.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class `HttpClient`
HttpClient class declaration.

7.111.1 Detailed Description

Simple HTTP client wrapper.

Author

Shamar Pennant

Handles sending and receiving HTTP messages to our custom server. Using custom formatting and structure as defined in the `route` layer/

Definition in file [HttpClient.hpp](#).

7.112 HttpClient.hpp

[Go to the documentation of this file.](#)

```
00001 #pragma once
00002
00003 #include <curl/curl.h>
00004 #include <string>
00005 #include <iostream>
00006 #include <memory>
00007 #include <stdio.h>
00008
00009
00010
00011
00012 class HttpClient {
00013 public:
00014     // Constant for posting to eliminate typing errors
00015     const std::string P = "POST";
00016     // Constant for getting to eliminate typing errors
00017     const std::string G = "GET";
00018     // Constant for deleting to eliminate typing errors
00019     const std::string D = "DELETE";
00020
00021     // --- endpoint constants ---
00022
00023     // user registration endpoint
00024     const std::string REG_USER = "/api/v1/auth/register";
00025     // user login endpoint
00026     const std::string LOGIN = "/api/v1/auth/login";
00027     // user profile lookup endpoint prefix
00028     const std::string AUTH_USER_PREFIX = "/api/v1/auth/users/";
00029     // user delete endpoint prefix
00030     const std::string DELETE_USER_PREFIX = "/api/v1/auth/users/";
00031     // endpoint pertaining to creating a blend
```

```

00042         const std::string BLEND = "/api/v1/blends";
00043
00044         // --- status codes ---
00045
00046         const int SUCC = 200; // success
00047         const int ALT_SUCC = 201; // alt success
00048         const int MAL_REQ_ERR = 400; // malformed request error
00049         const int INV_USR_LOG_ERR = 401; // invalid login credentials
00050         const int UNAUTH_RES_REQ_ERR = 403; // user not authorized for request resource
00051         const int MISS_RM_CNTXT = 404; // missing user or blend room context
00052         const int DUPLICATE = 409; // duplicate entry
00053         const int SERVER_ERR = 500; // server error
00054         //
00055
00060         HttpClient(std::string base);
00061         ~HttpClient() = default;
00062
00063         static bool isRequestSuccessful(const std::string& method, long statusCode);
00064
00065         long getLastStatusCode() const;
00066
00067         bool wasLastRequestSuccessful(const std::string& method) const;
00068
00069         std::string get(const std::string& path);
00070         std::string post(const std::string& path, const std::string& json);
00071
00072         std::string del(const std::string& path, const std::string& json);
00073
00074         std::string build_watch_later_endpoint(std::string userID);
00075
00076         std::string build_latest_blend_endpoint(std::string userID);
00077
00078         std::string build_blend_participant_endpoint(std::string blendID);
00079
00080         std::string build_chatroom_detail_endpoint(std::string blendID);
00081
00082         std::string build_auth_user_lookup_endpoint(std::string userID);
00083
00084         std::string build_delete_user_endpoint(std::string userID);
00085
00086         std::string build_user_blends_endpoint(std::string userID);
00087
00088         std::string build_leave_blend_endpoint(std::string blendID);
00089
00090         std::string build_chat_history_endpoint(std::string blendID, std::string userID);
00091
00092     private:
00093         // Holds the base url for server communication
00094         std::string baseUrl;
00095
00096         // used as a header internally (specifically for our server)
00097         const std::string hdr = "Content-Type: application/json";
00098
00099         // HTTP status code from the latest response (0 means not set yet).
00100         long lastStatusCode = 0;
00101
00102         std::string request(const std::string& method, const std::string& path, const std::string&
00103             body);
00104
00105         static size_t write_callback(void* contents, size_t size, size_t nmemb, void* out);
00106     };

```

7.113 RequestJsonBuilder.cpp File Reference

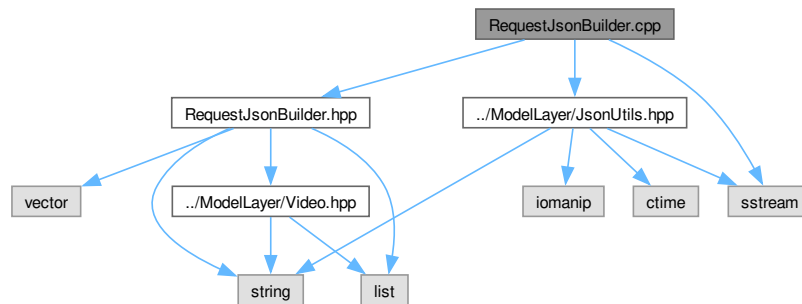
Implementation for RequestJsonBuilder.

```

#include "RequestJsonBuilder.hpp"
#include "../ModelLayer/JsonUtils.hpp"
#include <sstream>

```

Include dependency graph for RequestJsonBuilder.cpp:



Functions

- `std::string RequestJsonBuilder::buildRegisterJson` (const `std::string` &userID, const `std::string` &username, const `std::string` &email, const `std::string` &password)
Builds the registration request body for the auth/register endpoint.
- `std::string RequestJsonBuilder::buildLoginJson` (const `std::string` &userID, const `std::string` &password)
Builds the login request body for the auth/login endpoint.
- `std::string RequestJsonBuilder::buildDeleteAccountJson` (const `std::string` &password)
Builds the delete-account request body.
- `std::string RequestJsonBuilder::buildBlendJson` (const `std::string` &blendID, const `std::string` &title, const `std::string` &creatorID, const `std::string` &algorithm, const `std::vector`< `std::string` > &participants)
Builds the blend creation request body.
- `std::string RequestJsonBuilder::buildWatchLaterJson` (const `std::list`< `Video` > &videos)
Builds the watch-later persistence request body.

7.113.1 Detailed Description

Implementation for RequestJsonBuilder.

Author

Shamar Pennant

Definition in file [RequestJsonBuilder.cpp](#).

7.113.2 Function Documentation

7.113.2.1 buildBlendJson()

```

std::string RequestJsonBuilder::buildBlendJson (
    const std::string & blendID,
    const std::string & title,
    const std::string & creatorID,

```

```
const std::string & algorithm,  
const std::vector< std::string > & participants)
```

Builds the blend creation request body.

Output format:

```
{  
  "blend_id": "...",  
  "creator_id": "...",  
  "algorithm": "...",  
  "participants": ["...", "..."]  
}
```

Parameters

<i>blendID</i>	Unique blend identifier generated client-side.
<i>creatorID</i>	User ID of the blend creator.
<i>algorithm</i>	Name of the algorithm used to generate the blend.
<i>participants</i>	Ordered participant user IDs for this blend.

Returns

JSON string safe for HTTP POST to the blend endpoint.

Definition at line 40 of file [RequestJsonBuilder.cpp](#).

7.113.2.2 buildDeleteAccountJson()

```
std::string RequestJsonBuilder::buildDeleteAccountJson (  
    const std::string & password)
```

Builds the delete-account request body.

Output format:

```
{ "password": "..." }
```

Parameters

<i>password</i>	Plain-text password entered by the user for re-authentication.
-----------------	--

Returns

JSON string safe for HTTP DELETE to the user-delete endpoint.

Definition at line 34 of file [RequestJsonBuilder.cpp](#).

7.113.2.3 buildLoginJson()

```
std::string RequestJsonBuilder::buildLoginJson (  
    const std::string & userID,  
    const std::string & password)
```

Builds the login request body for the auth/login endpoint.

Output format:

```
{ "user_id": "...", "password": "..." }
```

Parameters

<i>userID</i>	User identifier provided by the client.
<i>password</i>	Plain-text password entered by the user.

Returns

JSON string safe for HTTP POST to the login endpoint.

Definition at line 27 of file [RequestJsonBuilder.cpp](#).

7.113.2.4 buildRegisterJson()

```
std::string RequestJsonBuilder::buildRegisterJson (
    const std::string & userID,
    const std::string & username,
    const std::string & email,
    const std::string & password)
```

Builds the registration request body for the auth/register endpoint.

Output format:

```
{
  "user_id": "...",
  "username": "...",
  "email": "...",
  "password": "..."
}
```

Parameters

<i>userID</i>	User identifier chosen by the registering user.
<i>username</i>	Display name to store with the account.
<i>email</i>	Optional email value (can be empty).
<i>password</i>	Plain-text password entered during registration.

Returns

JSON string safe for HTTP POST to the register endpoint.

Definition at line 15 of file [RequestJsonBuilder.cpp](#).

7.113.2.5 buildWatchLaterJson()

```
std::string RequestJsonBuilder::buildWatchLaterJson (
    const std::list< Video > & videos)
```

Builds the watch-later persistence request body.

Output format:

```
{
  "videos": [
    { "video_id": "...", "title": "..." }
  ]
}
```

Parameters

<i>videos</i>	Watch-later videos to serialize.
---------------	----------------------------------

Returns

JSON string safe for HTTP POST to the watch-later endpoint.

Definition at line 64 of file [RequestJsonBuilder.cpp](#).

7.114 RequestJsonBuilder.cpp

[Go to the documentation of this file.](#)

```

00001
00006
00007 #include "RequestJsonBuilder.hpp"
00008
00009 #include "../ModelLayer/JsonUtils.hpp"
00010
00011 #include <sstream>
00012
00013 namespace RequestJsonBuilder {
00014
00015 std::string buildRegisterJson(const std::string& userID,
00016                             const std::string& username,
00017                             const std::string& email,
00018                             const std::string& password) {
00019     return "{"
00020         "\"user_id\": " + ModelJson::quote(userID) + ", "
00021         "\"username\": " + ModelJson::quote(username) + ", "
00022         "\"email\": " + ModelJson::quote(email) + ", "
00023         "\"password\": " + ModelJson::quote(password)
00024         + "}";
00025 }
00026
00027 std::string buildLoginJson(const std::string& userID, const std::string& password) {
00028     return "{"
00029         "\"user_id\": " + ModelJson::quote(userID) + ", "
00030         "\"password\": " + ModelJson::quote(password)
00031         + "}";
00032 }
00033
00034 std::string buildDeleteAccountJson(const std::string& password) {
00035     return "{"
00036         "\"password\": " + ModelJson::quote(password)
00037         + "}";
00038 }
00039
00040 std::string buildBlendJson(const std::string& blendID,
00041                           const std::string& title,
00042                           const std::string& creatorID,
00043                           const std::string& algorithm,
00044                           const std::vector<std::string>& participants) {
00045     std::ostringstream participantsJson;
00046     participantsJson << "[";
00047     for (size_t i = 0; i < participants.size(); ++i) {
00048         if (i > 0) {
00049             participantsJson << ", ";
00050         }
00051         participantsJson << ModelJson::quote(participants[i]);
00052     }
00053     participantsJson << "]";
00054
00055     return "{"
00056         "\"blend_id\": " + ModelJson::quote(blendID) + ", "
00057         "\"title\": " + ModelJson::quote(title) + ", "
00058         "\"creator_id\": " + ModelJson::quote(creatorID) + ", "
00059         "\"algorithm\": " + ModelJson::quote(algorithm) + ", "
00060         "\"participants\": " + participantsJson.str()
00061         + "}";
00062 }
00063
00064 std::string buildWatchLaterJson(const std::list<Video>& videos) {
00065     std::ostringstream videosJson;

```



```

00066     videosJson « "[";
00067
00068     bool first = true;
00069     for (const auto& video : videos) {
00070         if (!first) {
00071             videosJson « ",";
00072         }
00073         first = false;
00074
00075         videosJson « "{"
00076             « "\"video_id\": \" « ModelJson::quote(video.getVideoID()) « ","
00077             « "\"title\": \" « ModelJson::quote(video.getTitle()) « ","
00078             « "\"channel_id\": \" « ModelJson::quote(video.getChannelID()) « ","
00079             « "\"channel_name\": \" « ModelJson::quote(video.getChannelName()) « ","
00080             « "\"thumbnail_url\": \" « ModelJson::quote(video.getThumbnailURL()) « ","
00081             « "\"channel_logo_url\": \" « ModelJson::quote(video.getChannelLogoURL())
00082             « "}"
00083     }
00084
00085     videosJson « "]";
00086     return "{\"videos\": " + videosJson.str() + "}";
00087 }
00088
00089 } // namespace RequestJsonBuilder

```

7.115 RequestJsonBuilder.hpp File Reference

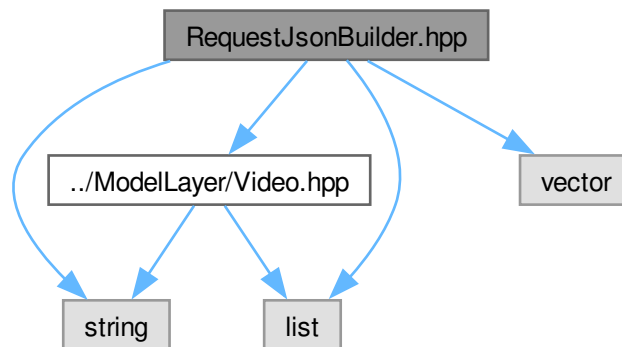
JSON request body builders used by ServiceLayer HTTP calls.

```

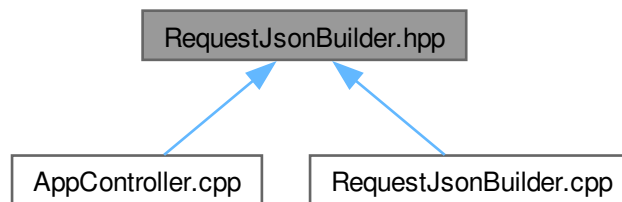
#include "../ModelLayer/Video.hpp"
#include <list>
#include <string>
#include <vector>

```

Include dependency graph for RequestJsonBuilder.hpp:



This graph shows which files directly or indirectly include this file:



Functions

- `std::string RequestJsonBuilder::buildRegisterJson` (`const std::string &userID`, `const std::string &username`, `const std::string &email`, `const std::string &password`)
Builds the registration request body for the auth/register endpoint.
- `std::string RequestJsonBuilder::buildLoginJson` (`const std::string &userID`, `const std::string &password`)
Builds the login request body for the auth/login endpoint.
- `std::string RequestJsonBuilder::buildDeleteAccountJson` (`const std::string &password`)
Builds the delete-account request body.
- `std::string RequestJsonBuilder::buildBlendJson` (`const std::string &blendID`, `const std::string &title`, `const std::string &creatorID`, `const std::string &algorithm`, `const std::vector< std::string > &participants`)
Builds the blend creation request body.
- `std::string RequestJsonBuilder::buildWatchLaterJson` (`const std::list< Video > &videos`)
Builds the watch-later persistence request body.

7.115.1 Detailed Description

JSON request body builders used by ServiceLayer HTTP calls.

Author

Shamar Pennant

Definition in file [RequestJsonBuilder.hpp](#).

7.115.2 Function Documentation

7.115.2.1 buildBlendJson()

```

std::string RequestJsonBuilder::buildBlendJson (
    const std::string & blendID,
    const std::string & title,
    const std::string & creatorID,

```

```
const std::string & algorithm,  
const std::vector< std::string > & participants)
```

Builds the blend creation request body.

Output format:

```
{  
  "blend_id": "...",  
  "creator_id": "...",  
  "algorithm": "...",  
  "participants": ["...", "..."]  
}
```

Parameters

<i>blendID</i>	Unique blend identifier generated client-side.
<i>creatorID</i>	User ID of the blend creator.
<i>algorithm</i>	Name of the algorithm used to generate the blend.
<i>participants</i>	Ordered participant user IDs for this blend.

Returns

JSON string safe for HTTP POST to the blend endpoint.

Definition at line 40 of file [RequestJsonBuilder.cpp](#).

7.115.2.2 buildDeleteAccountJson()

```
std::string JsonRequestBuilder::buildDeleteAccountJson (  
    const std::string & password)
```

Builds the delete-account request body.

Output format:

```
{ "password": "..." }
```

Parameters

<i>password</i>	Plain-text password entered by the user for re-authentication.
-----------------	--

Returns

JSON string safe for HTTP DELETE to the user-delete endpoint.

Definition at line 34 of file [RequestJsonBuilder.cpp](#).

7.115.2.3 buildLoginJson()

```
std::string JsonRequestBuilder::buildLoginJson (  
    const std::string & userID,  
    const std::string & password)
```

Builds the login request body for the auth/login endpoint.

Output format:

```
{ "user_id": "...", "password": "..." }
```

Parameters

<i>userID</i>	User identifier provided by the client.
<i>password</i>	Plain-text password entered by the user.

Returns

JSON string safe for HTTP POST to the login endpoint.

Definition at line 27 of file [RequestJsonBuilder.cpp](#).

7.115.2.4 buildRegisterJson()

```
std::string RequestJsonBuilder::buildRegisterJson (
    const std::string & userID,
    const std::string & username,
    const std::string & email,
    const std::string & password)
```

Builds the registration request body for the auth/register endpoint.

Output format:

```
{
  "user_id": "...",
  "username": "...",
  "email": "...",
  "password": "..."
}
```

Parameters

<i>userID</i>	User identifier chosen by the registering user.
<i>username</i>	Display name to store with the account.
<i>email</i>	Optional email value (can be empty).
<i>password</i>	Plain-text password entered during registration.

Returns

JSON string safe for HTTP POST to the register endpoint.

Definition at line 15 of file [RequestJsonBuilder.cpp](#).

7.115.2.5 buildWatchLaterJson()

```
std::string RequestJsonBuilder::buildWatchLaterJson (
    const std::list< Video > & videos)
```

Builds the watch-later persistence request body.

Output format:

```
{
  "videos": [
    { "video_id": "...", "title": "..." }
  ]
}
```

Parameters

<i>videos</i>	Watch-later videos to serialize.
---------------	----------------------------------

Returns

JSON string safe for HTTP POST to the watch-later endpoint.

Definition at line 64 of file [RequestJsonBuilder.cpp](#).

7.116 JsonRequestBuilder.hpp

[Go to the documentation of this file.](#)

```

00001
00006
00007 #pragma once
00008
00009 #include "../ModelLayer/Video.hpp"
00010
00011 #include <list>
00012 #include <string>
00013 #include <vector>
00014
00015 namespace JsonRequestBuilder {
00016
00036 std::string buildRegisterJson(const std::string& userID,
00037                               const std::string& username,
00038                               const std::string& email,
00039                               const std::string& password);
00040
00053 std::string buildLoginJson(const std::string& userID, const std::string& password);
00054
00066 std::string buildDeleteAccountJson(const std::string& password);
00067
00087 std::string buildBlendJson(const std::string& blendID,
00088                             const std::string& title,
00089                             const std::string& creatorID,
00090                             const std::string& algorithm,
00091                             const std::vector<std::string>& participants);
00092
00108 std::string buildWatchLaterJson(const std::list<Video>& videos);
00109
00110 } // namespace JsonRequestBuilder

```

7.117 SqliteDataManager.cpp File Reference

Implementation for [SqliteDataManager](#).

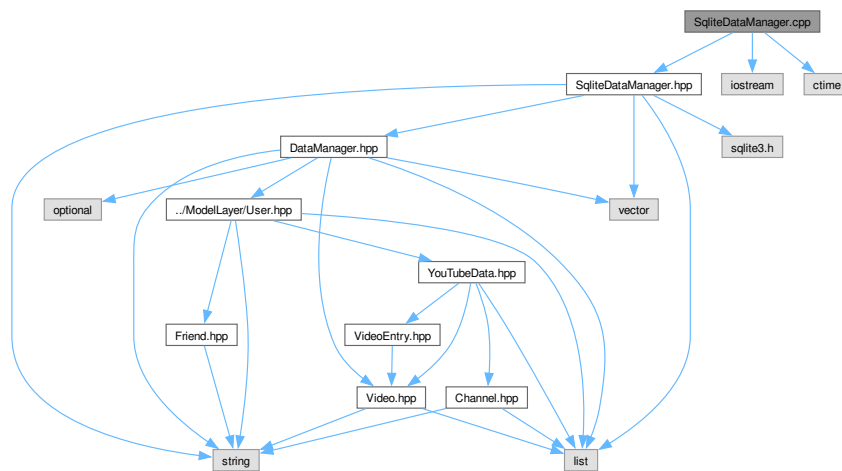
```

#include "SqliteDataManager.hpp"
#include <iostream>

```

```
#include <ctime>
```

Include dependency graph for SqliteDataManager.cpp:



7.117.1 Detailed Description

Implementation for [SqliteDataManager](#).

Author

Jasmine Kumar

Definition in file [SqliteDataManager.cpp](#).

7.118 SqliteDataManager.cpp

[Go to the documentation of this file.](#)

```

00001
00006
00007 #include "SqliteDataManager.hpp"
00008 #include <iostream>
00009 #include <ctime>
00010
00011 // -- Constructor / Destructor -----
00012
00013 SqliteDataManager::SqliteDataManager(const std::string& dbPath) {
00014     int rc = sqlite3_open(dbPath.c_str(), &db);
00015     if (rc != SQLITE_OK) {
00016         std::cerr << "[SqliteDataManager] Failed to open DB: "
00017                 << sqlite3_errmsg(db) << "\n";
00018         sqlite3_close(db);
00019         db = nullptr;
00020         return;
00021     }
00022     initSchema();
00023 }
00024
00025 SqliteDataManager::~SqliteDataManager() {
00026     if (db) sqlite3_close(db);
00027 }
00028
00029 // -- Private helpers -----
00030

```

```

00031 void SqliteDataManager::initSchema() {
00032     const char* sql =
00033         // -- Users -----
00034         "CREATE TABLE IF NOT EXISTS users ("
00035         "    user_id TEXT PRIMARY KEY,"
00036         "    username TEXT NOT NULL,"
00037         "    email TEXT,"
00038         "    password TEXT NOT NULL"
00039         ");"
00040
00041         // -- Watch Later videos per user -----
00042         // Stores each parsed + API-enriched video as a row; replaced in bulk
00043         // on re-upload. position preserves original playlist order for display.
00044         "CREATE TABLE IF NOT EXISTS user_watch_later ("
00045         "    user_id TEXT NOT NULL,"
00046         "    video_id TEXT NOT NULL,"
00047         "    title TEXT,"
00048         "    channel_id TEXT,"
00049         "    channel_name TEXT,"
00050         "    channel_logo_url TEXT,"
00051         "    thumbnail_url TEXT,"
00052         "    position INTEGER NOT NULL DEFAULT 0,"
00053         "    PRIMARY KEY (user_id, video_id)"
00054         ");"
00055
00056         // -- Blend metadata -----
00057         // creator_id kept separate for future delete/manage privileges.
00058         "CREATE TABLE IF NOT EXISTS blends ("
00059         "    blend_id TEXT PRIMARY KEY,"
00060         "    creator_id TEXT NOT NULL,"
00061         "    algorithm TEXT NOT NULL,"
00062         "    created_at TEXT NOT NULL"
00063         ");"
00064
00065         // -- Blend participants -----
00066         // One row per (blend, user) pair - enables fast lookup in both directions.
00067         "CREATE TABLE IF NOT EXISTS blend_participants ("
00068         "    blend_id TEXT NOT NULL,"
00069         "    user_id TEXT NOT NULL,"
00070         "    PRIMARY KEY (blend_id, user_id)"
00071         ");";
00072
00073     char* errMsg = nullptr;
00074     int rc = sqlite3_exec(db, sql, nullptr, nullptr, &errMsg);
00075     if (rc != SQLITE_OK) {
00076         std::cerr << "[SqliteDataManager] Schema init failed: " << errMsg << "\n";
00077         sqlite3_free(errMsg);
00078     }
00079
00080     // Migration: add new columns to pre-existing databases that have the old
00081     // user_watch_later schema (SQLite ignores "duplicate column" errors).
00082     const char* migrations[] = {
00083         "ALTER TABLE user_watch_later ADD COLUMN channel_id TEXT;",
00084         "ALTER TABLE user_watch_later ADD COLUMN channel_name TEXT;",
00085         "ALTER TABLE user_watch_later ADD COLUMN channel_logo_url TEXT;",
00086         "ALTER TABLE user_watch_later ADD COLUMN thumbnail_url TEXT;",
00087     };
00088     for (const char* m : migrations) {
00089         sqlite3_exec(db, m, nullptr, nullptr, nullptr); // ignore error if column exists
00090     }
00091 }
00092
00093 // -- DataManager interface -----
00094
00095 bool SqliteDataManager::createUser(const User& user) {
00096     if (!db) return false;
00097
00098     const char* sql =
00099         "INSERT INTO users (user_id, username, email, password) "
00100         "VALUES (?, ?, ?, ?);";
00101
00102     sqlite3_stmt* stmt = nullptr;
00103     if (sqlite3_prepare_v2(db, sql, -1, &stmt, nullptr) != SQLITE_OK) {
00104         std::cerr << "[SqliteDataManager] createUser prepare failed: "
00105             << sqlite3_errmsg(db) << "\n";
00106         return false;
00107     }
00108
00109     sqlite3_bind_text(stmt, 1, user.getUserID().c_str(), -1, SQLITE_TRANSIENT);
00110     sqlite3_bind_text(stmt, 2, user.getUsername().c_str(), -1, SQLITE_TRANSIENT);
00111     sqlite3_bind_text(stmt, 3, user.getEmail().c_str(), -1, SQLITE_TRANSIENT);
00112     sqlite3_bind_text(stmt, 4, user.getPassword().c_str(), -1, SQLITE_TRANSIENT);
00113
00114     int rc = sqlite3_step(stmt);
00115     sqlite3_finalize(stmt);
00116
00117     if (rc != SQLITE_DONE) {

```

```

00118         // SQLITE_CONSTRAINT means the PRIMARY KEY (user_id) already exists
00119         std::cerr << "[SqliteDataManager] createUser failed: "
00120             << sqlite3_errmsg(db) << "\n";
00121         return false;
00122     }
00123     return true;
00124 }
00125
00126 std::optional<User> SqliteDataManager::findUserByID(const std::string& userID) {
00127     if (!db) return std::nullopt;
00128
00129     const char* sql =
00130         "SELECT user_id, username, email, password "
00131         "FROM users WHERE user_id = ?;";
00132
00133     sqlite3_stmt* stmt = nullptr;
00134     if (sqlite3_prepare_v2(db, sql, -1, &stmt, nullptr) != SQLITE_OK) {
00135         std::cerr << "[SqliteDataManager] findUserByID prepare failed: "
00136             << sqlite3_errmsg(db) << "\n";
00137         return std::nullopt;
00138     }
00139
00140     sqlite3_bind_text(stmt, 1, userID.c_str(), -1, SQLITE_TRANSIENT);
00141
00142     std::optional<User> result = std::nullopt;
00143     if (sqlite3_step(stmt) == SQLITE_ROW) {
00144         std::string id = reinterpret_cast<const char*>(sqlite3_column_text(stmt, 0));
00145         std::string uname = reinterpret_cast<const char*>(sqlite3_column_text(stmt, 1));
00146         std::string email = reinterpret_cast<const char*>(sqlite3_column_text(stmt, 2));
00147         std::string password = reinterpret_cast<const char*>(sqlite3_column_text(stmt, 3));
00148         result = User(id, uname, email, password);
00149     }
00150
00151     sqlite3_finalize(stmt);
00152     return result;
00153 }
00154
00155 bool SqliteDataManager::validatePassword(const std::string& userID,
00156     const std::string& password) {
00157     std::optional<User> user = findUserByID(userID);
00158     if (!user.has_value()) return false;
00159     return user->getPassword() == password;
00160 }
00161
00162 // -- Watch Later -----
00163
00164 bool SqliteDataManager::saveWatchLater(const std::string& userID,
00165     const std::list<Video>& videos) {
00166     if (!db) return false;
00167
00168     // Delete existing rows for this user then re-insert - handles re-uploads cleanly.
00169     const char* delSql = "DELETE FROM user_watch_later WHERE user_id = ?;";
00170     sqlite3_stmt* del = nullptr;
00171     if (sqlite3_prepare_v2(db, delSql, -1, &del, nullptr) != SQLITE_OK) {
00172         std::cerr << "[SqliteDataManager] saveWatchLater delete prepare failed: "
00173             << sqlite3_errmsg(db) << "\n";
00174         return false;
00175     }
00176     sqlite3_bind_text(del, 1, userID.c_str(), -1, SQLITE_TRANSIENT);
00177     sqlite3_step(del);
00178     sqlite3_finalize(del);
00179
00180     const char* insSql =
00181         "INSERT OR REPLACE INTO user_watch_later "
00182         "(user_id, video_id, title, channel_id, channel_name, channel_logo_url, thumbnail_url,
00183 position) "
00184         "VALUES (?, ?, ?, ?, ?, ?, ?, ?);";
00185     sqlite3_stmt* ins = nullptr;
00186     if (sqlite3_prepare_v2(db, insSql, -1, &ins, nullptr) != SQLITE_OK) {
00187         std::cerr << "[SqliteDataManager] saveWatchLater insert prepare failed: "
00188             << sqlite3_errmsg(db) << "\n";
00189         return false;
00190     }
00191
00192     int pos = 0;
00193     for (const Video& v : videos) {
00194         sqlite3_bind_text(ins, 1, userID.c_str(), -1, SQLITE_TRANSIENT);
00195         sqlite3_bind_text(ins, 2, v.getVideoID().c_str(), -1, SQLITE_TRANSIENT);
00196         sqlite3_bind_text(ins, 3, v.getTitle().c_str(), -1, SQLITE_TRANSIENT);
00197         sqlite3_bind_text(ins, 4, v.getChannelID().c_str(), -1, SQLITE_TRANSIENT);
00198         sqlite3_bind_text(ins, 5, v.getChannelName().c_str(), -1, SQLITE_TRANSIENT);
00199         sqlite3_bind_text(ins, 6, v.getChannelLogoURL().c_str(), -1, SQLITE_TRANSIENT);
00200         sqlite3_bind_text(ins, 7, v.getThumbnailURL().c_str(), -1, SQLITE_TRANSIENT);
00201         sqlite3_bind_int(ins, 8, pos++);
00202         if (sqlite3_step(ins) != SQLITE_DONE) {
00203             std::cerr << "[SqliteDataManager] saveWatchLater insert failed: "
00204                 << sqlite3_errmsg(db) << "\n";

```



```

00204     }
00205     sqlite3_reset(ins);
00206 }
00207 sqlite3_finalize(ins);
00208 return true;
00209 }
00210
00211 std::list<Video> SqliteDataManager::loadWatchLater(const std::string& userID) {
00212     std::list<Video> result;
00213     if (!db) return result;
00214
00215     const char* sql =
00216         "SELECT video_id, title, channel_id, channel_name, channel_logo_url, thumbnail_url "
00217         "FROM user_watch_later "
00218         "WHERE user_id = ? ORDER BY position ASC;";
00219     sqlite3_stmt* stmt = nullptr;
00220     if (sqlite3_prepare_v2(db, sql, -1, &stmt, nullptr) != SQLITE_OK) {
00221         std::cerr << "[SqliteDataManager] loadWatchLater prepare failed: "
00222             << sqlite3_errmsg(db) << "\n";
00223         return result;
00224     }
00225     sqlite3_bind_text(stmt, 1, userID.c_str(), -1, SQLITE_TRANSIENT);
00226
00227     auto colText = [&](int col) -> std::string {
00228         const unsigned char* p = sqlite3_column_text(stmt, col);
00229         return p ? reinterpret_cast<const char*>(p) : "";
00230     };
00231
00232     while (sqlite3_step(stmt) == SQLITE_ROW) {
00233         result.emplace_back(
00234             colText(0), // videoID
00235             colText(1), // title
00236             colText(2), // channelID
00237             colText(5), // thumbnailURL
00238             0,
00239             std::list<std::string>{},
00240             colText(3), // channelName
00241             colText(4) // channelLogoURL
00242         );
00243     }
00244     sqlite3_finalize(stmt);
00245     return result;
00246 }
00247
00248 // -- Blend persistence -----
00249
00250 bool SqliteDataManager::saveBlend(const std::string& blendID,
00251     const std::string& creatorID,
00252     const std::string& algorithm,
00253     const std::vector<std::string>& participantIDs) {
00254     if (!db) return false;
00255
00256     // ISO timestamp for created_at
00257     std::time_t now = std::time(nullptr);
00258     char timeBuf[32];
00259     std::strftime(timeBuf, sizeof(timeBuf), "%Y-%m-%dT%H:%M:%S", std::gmtime(&now));
00260
00261     // Upsert blend row
00262     const char* blendSql =
00263         "INSERT OR REPLACE INTO blends (blend_id, creator_id, algorithm, created_at) "
00264         "VALUES (?, ?, ?, ?);";
00265     sqlite3_stmt* bs = nullptr;
00266     if (sqlite3_prepare_v2(db, blendSql, -1, &bs, nullptr) != SQLITE_OK) {
00267         std::cerr << "[SqliteDataManager] saveBlend blend prepare failed: "
00268             << sqlite3_errmsg(db) << "\n";
00269         return false;
00270     }
00271     sqlite3_bind_text(bs, 1, blendID.c_str(), -1, SQLITE_TRANSIENT);
00272     sqlite3_bind_text(bs, 2, creatorID.c_str(), -1, SQLITE_TRANSIENT);
00273     sqlite3_bind_text(bs, 3, algorithm.c_str(), -1, SQLITE_TRANSIENT);
00274     sqlite3_bind_text(bs, 4, timeBuf, -1, SQLITE_TRANSIENT);
00275     sqlite3_step(bs);
00276     sqlite3_finalize(bs);
00277
00278     // Replace participant rows - delete first so removed users are cleaned up
00279     const char* delSql = "DELETE FROM blend_participants WHERE blend_id = ?;";
00280     sqlite3_stmt* del = nullptr;
00281     sqlite3_prepare_v2(db, delSql, -1, &del, nullptr);
00282     sqlite3_bind_text(del, 1, blendID.c_str(), -1, SQLITE_TRANSIENT);
00283     sqlite3_step(del);
00284     sqlite3_finalize(del);
00285
00286     const char* partSql =
00287         "INSERT OR IGNORE INTO blend_participants (blend_id, user_id) VALUES (?, ?);";
00288     sqlite3_stmt* ps = nullptr;
00289     if (sqlite3_prepare_v2(db, partSql, -1, &ps, nullptr) != SQLITE_OK) {
00290         std::cerr << "[SqliteDataManager] saveBlend participants prepare failed: "

```

```

00291         « sqlite3_errmsg(db) « "\n";
00292     return false;
00293 }
00294 for (const auto& uid : participantIDs) {
00295     sqlite3_bind_text(ps, 1, blendID.c_str(), -1, SQLITE_TRANSIENT);
00296     sqlite3_bind_text(ps, 2, uid.c_str(), -1, SQLITE_TRANSIENT);
00297     sqlite3_step(ps);
00298     sqlite3_reset(ps);
00299 }
00300 sqlite3_finalize(ps);
00301 return true;
00302 }
00303
00304 std::optional<std::string> SqliteDataManager::findBlendForUser(const std::string& userID) {
00305     if (!db) return std::nullopt;
00306
00307     const char* sql =
00308         "SELECT bp.blend_id FROM blend_participants bp "
00309         "INNER JOIN blends b ON bp.blend_id = b.blend_id "
00310         "WHERE bp.user_id = ? "
00311         "ORDER BY b.created_at DESC LIMIT 1;";
00312     sqlite3_stmt* stmt = nullptr;
00313     if (sqlite3_prepare_v2(db, sql, -1, &stmt, nullptr) != SQLITE_OK) {
00314         std::cerr « "[SqliteDataManager] findBlendForUser prepare failed: "
00315                 « sqlite3_errmsg(db) « "\n";
00316         return std::nullopt;
00317     }
00318     sqlite3_bind_text(stmt, 1, userID.c_str(), -1, SQLITE_TRANSIENT);
00319
00320     std::optional<std::string> result = std::nullopt;
00321     if (sqlite3_step(stmt) == SQLITE_ROW)
00322         result = reinterpret_cast<const char*>(sqlite3_column_text(stmt, 0));
00323     sqlite3_finalize(stmt);
00324     return result;
00325 }
00326
00327 std::vector<std::string> SqliteDataManager::loadBlendParticipants(const std::string& blendID) {
00328     std::vector<std::string> result;
00329     if (!db) return result;
00330
00331     const char* sql =
00332         "SELECT user_id FROM blend_participants WHERE blend_id = ?;";
00333     sqlite3_stmt* stmt = nullptr;
00334     if (sqlite3_prepare_v2(db, sql, -1, &stmt, nullptr) != SQLITE_OK) {
00335         std::cerr « "[SqliteDataManager] loadBlendParticipants prepare failed: "
00336                 « sqlite3_errmsg(db) « "\n";
00337         return result;
00338     }
00339     sqlite3_bind_text(stmt, 1, blendID.c_str(), -1, SQLITE_TRANSIENT);
00340
00341     while (sqlite3_step(stmt) == SQLITE_ROW)
00342         result.emplace_back(reinterpret_cast<const char*>(sqlite3_column_text(stmt, 0)));
00343     sqlite3_finalize(stmt);
00344     return result;
00345 }

```

7.119 SqliteDataManager.hpp File Reference

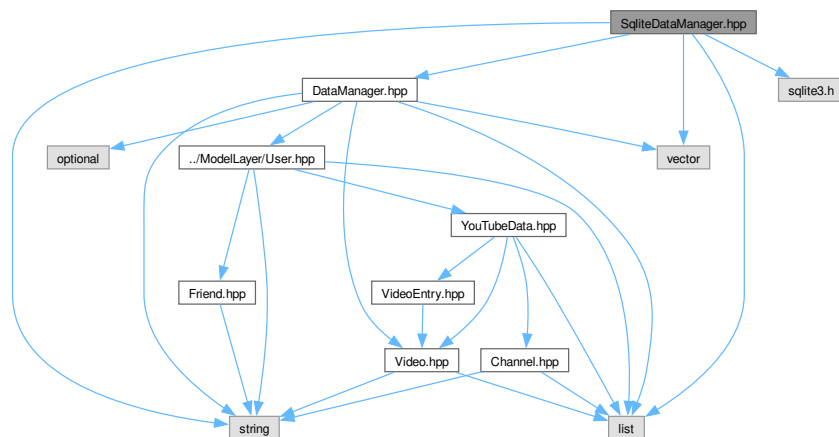
SQLite-backed implementation of [DataManager](#).

```

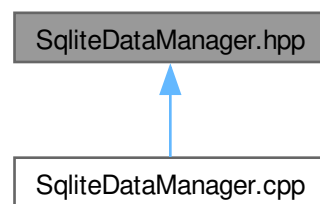
#include "DataManager.hpp"
#include <sqlite3.h>
#include <string>
#include <vector>
#include <list>

```

Include dependency graph for SqliteDataManager.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [SqliteDataManager](#)
SqliteDataManager type declaration.

7.119.1 Detailed Description

SQLite-backed implementation of [DataManager](#).

Author

Jasmine Kumar

Opens (or creates) a .db file on construction and runs the schema migration automatically. Pass ":memory:" as the path to get an in-memory database - useful for unit tests.

Definition in file [SqliteDataManager.hpp](#).

7.120 SqliteDataManager.hpp

[Go to the documentation of this file.](#)

```

00001 #ifndef SQLITE_DATA_MANAGER_H
00002 #define SQLITE_DATA_MANAGER_H
00003
00004 #include "DataManager.hpp"
00005 #include <sqlite3.h>
00006 #include <string>
00007 #include <vector>
00008 #include <list>
00009
00023 class SqliteDataManager : public DataManager {
00024 public:
00025     explicit SqliteDataManager(const std::string& dbPath);
00026     ~SqliteDataManager() override;
00027
00028     bool createUser(const User& user) override;
00029     std::optional<User> findUserByID(const std::string& userID) override;
00030     bool validatePassword(const std::string& userID,
00031                          const std::string& password) override;
00032
00033     bool saveWatchLater(const std::string& userID,
00034                        const std::list<Video>& videos) override;
00035     std::list<Video> loadWatchLater(const std::string& userID) override;
00036
00037     bool saveBlend(const std::string& blendID,
00038                  const std::string& creatorID,
00039                  const std::string& algorithm,
00040                  const std::vector<std::string>& participantIDs) override;
00041     std::optional<std::string> findBlendForUser(
00042         const std::string& userID) override;
00043     std::vector<std::string> loadBlendParticipants(
00044         const std::string& blendID) override;
00045
00046 private:
00047     sqlite3* db = nullptr;
00048
00049     // Runs CREATE TABLE IF NOT EXISTS for all tables on startup.
00050     void initSchema();
00051 };
00052
00053 #endif

```

7.121 YouTubeMetadataFetcher.cpp File Reference

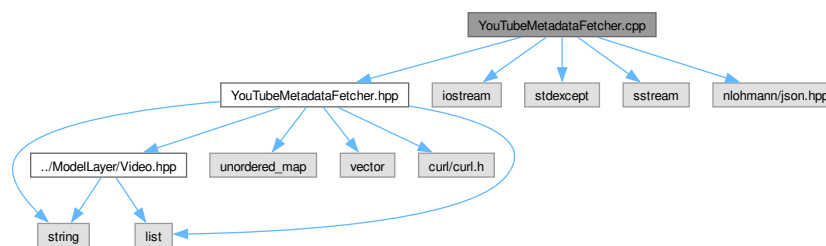
Implementation for [YouTubeMetadataFetcher](#).

```

#include "YouTubeMetadataFetcher.hpp"
#include <iostream>
#include <stdexcept>
#include <sstream>
#include <nlohmann/json.hpp>

```

Include dependency graph for YouTubeMetadataFetcher.cpp:



Typedefs

- using `json` = `nlohmann::json`

7.121.1 Detailed Description

Implementation for [YouTubeMetadataFetcher](#).

Author

Jasmine Kumar

Definition in file [YouTubeMetadataFetcher.cpp](#).

7.121.2 Typedef Documentation

7.121.2.1 json

```
using json = nlohmann::json
```

Definition at line 19 of file [YouTubeMetadataFetcher.cpp](#).

7.122 YouTubeMetadataFetcher.cpp

[Go to the documentation of this file.](#)

```
00001
00006
00007 // =====
00008 // YouTubeMetadataFetcher.cpp
00009 // =====
00010
00011 #include "YouTubeMetadataFetcher.hpp"
00012
00013 #include <iostream>
00014 #include <stdexcept>
00015 #include <sstream>
00016
00017 #include <nlohmann/json.hpp>
00018
00019 using json = nlohmann::json;
00020
00021 // -- Constructor -----
00022
00023 YouTubeMetadataFetcher::YouTubeMetadataFetcher(std::string apiKey)
00024     : apiKey(std::move(apiKey)) {}
00025
00026 // -- Public: enrich -----
00027
00028 std::list<Video> YouTubeMetadataFetcher::enrich(const std::list<Video>& videos) {
00029     if (apiKey.empty()) {
00030         std::cerr << "[YouTubeMetadataFetcher] No API key - returning videos unchanged\n";
00031         return videos;
00032     }
00033
00034     // Collect all video IDs in order
00035     std::vector<std::string> allIds;
00036     allIds.reserve(videos.size());
00037     for (const auto& v : videos) allIds.push_back(v.getVideoID());
00038
00039     // Fetch video metadata in batches of kBatchSize
00040     std::unordered_map<std::string, VideoMeta> metaMap;
00041     for (int i = 0; i < static_cast<int>(allIds.size()); i += kBatchSize) {
00042         int end = std::min(i + kBatchSize, static_cast<int>(allIds.size()));
```

```

00043         std::vector<std::string> batch(allIds.begin() + i, allIds.begin() + end);
00044     try {
00045         auto batchResult = fetchVideoBatch(batch);
00046         metaMap.insert(batchResult.begin(), batchResult.end());
00047     } catch (const std::exception& ex) {
00048         std::cerr << "[YouTubeMetadataFetcher] fetchVideoBatch failed: "
00049                 << ex.what() << "\n";
00050     }
00051 }
00052
00053 // Collect unique channel IDs from the fetched metadata
00054 std::vector<std::string> channelIdList;
00055 {
00056     std::unordered_map<std::string, bool> seen;
00057     for (const auto& [vid, meta] : metaMap) {
00058         if (!meta.channelId.empty() && seen.find(meta.channelId) == seen.end()) {
00059             channelIdList.push_back(meta.channelId);
00060             seen[meta.channelId] = true;
00061         }
00062     }
00063 }
00064
00065 // Fetch channel logos in batches of kBatchSize
00066 std::unordered_map<std::string, std::string> logoMap;
00067 for (int i = 0; i < static_cast<int>(channelIdList.size()); i += kBatchSize) {
00068     int end = std::min(i + kBatchSize, static_cast<int>(channelIdList.size()));
00069     std::vector<std::string> batch(channelIdList.begin() + i,
00070                                   channelIdList.begin() + end);
00071     try {
00072         auto batchLogos = fetchChannelLogos(batch);
00073         logoMap.insert(batchLogos.begin(), batchLogos.end());
00074     } catch (const std::exception& ex) {
00075         std::cerr << "[YouTubeMetadataFetcher] fetchChannelLogos failed: "
00076                 << ex.what() << "\n";
00077     }
00078 }
00079
00080 // Rebuild the video list with enriched data
00081 std::list<Video> enriched;
00082 for (const auto& v : videos) {
00083     auto it = metaMap.find(v.getVideoID());
00084     if (it == metaMap.end()) {
00085         // API didn't return this video (private/deleted) - keep as-is
00086         enriched.push_back(v);
00087         continue;
00088     }
00089     const VideoMeta& m = it->second;
00090
00091     std::string logoUrl;
00092     auto logoIt = logoMap.find(m.channelId);
00093     if (logoIt != logoMap.end()) logoUrl = logoIt->second;
00094
00095     enriched.emplace_back(
00096         v.getVideoID(),
00097         m.title,
00098         m.channelId,
00099         m.thumbnailUrl,
00100         v.getDuration(),
00101         v.getTags(),
00102         m.channelName,
00103         logoUrl
00104     );
00105 }
00106 return enriched;
00107 }
00108
00109 // -- Private: fetchVideoBatch -----
00110
00111 std::unordered_map<std::string, YouTubeMetadataFetcher::VideoMeta>
00112 YouTubeMetadataFetcher::fetchVideoBatch(const std::vector<std::string>& ids) {
00113     // Build comma-separated ID list
00114     std::ostringstream idParam;
00115     for (size_t i = 0; i < ids.size(); ++i) {
00116         if (i) idParam << ',';
00117         idParam << ids[i];
00118     }
00119
00120     std::string url =
00121         "https://www.googleapis.com/youtube/v3/videos"
00122         "?part=snippet"
00123         "&id=" + idParam.str() +
00124         "&key=" + apiKey;
00125
00126     std::string body = httpGet(url);
00127
00128     std::unordered_map<std::string, VideoMeta> result;
00129     try {

```

```

00130     auto root = json::parse(body);
00131     for (const auto& item : root.value("items", json::array())) {
00132         std::string videoId = item.value("id", "");
00133         if (videoId.empty()) continue;
00134
00135         const auto& snippet = item.value("snippet", json::object());
00136
00137         VideoMeta m;
00138         m.title = snippet.value("title", "");
00139         m.channelId = snippet.value("channelId", "");
00140         m.channelName = snippet.value("channelTitle", "");
00141
00142         // Prefer highest-resolution thumbnail available
00143         const auto& thumbs = snippet.value("thumbnails", json::object());
00144         for (const char* key : {"maxres", "standard", "high", "medium", "default"}) {
00145             if (thumbs.contains(key)) {
00146                 m.thumbnailUrl = thumbs[key].value("url", "");
00147                 if (!m.thumbnailUrl.empty()) break;
00148             }
00149         }
00150
00151         result[videoId] = std::move(m);
00152     }
00153 } catch (const json::parse_error& ex) {
00154     throw std::runtime_error(
00155         std::string("YouTubeMetadataFetcher: videos JSON parse error: ") + ex.what());
00156 }
00157 return result;
00158 }
00159
00160 // -- Private: fetchChannelLogos -----
00161
00162 std::unordered_map<std::string, std::string>
00163 YouTubeMetadataFetcher::fetchChannelLogos(const std::vector<std::string>& channelIds) {
00164     std::ostringstream idParam;
00165     for (size_t i = 0; i < channelIds.size(); ++i) {
00166         if (i) idParam << ',';
00167         idParam << channelIds[i];
00168     }
00169
00170     std::string url =
00171         "https://www.googleapis.com/youtube/v3/channels"
00172         "?part=snippet"
00173         "&id=" + idParam.str() +
00174         "&key=" + apiKey;
00175
00176     std::string body = httpGet(url);
00177
00178     std::unordered_map<std::string, std::string> result;
00179     try {
00180         auto root = json::parse(body);
00181         for (const auto& item : root.value("items", json::array())) {
00182             std::string channelId = item.value("id", "");
00183             if (channelId.empty()) continue;
00184
00185             const auto& snippet = item.value("snippet", json::object());
00186             const auto& thumbs = snippet.value("thumbnails", json::object());
00187
00188             // Channel logos: prefer medium (240x240) over default (88x88)
00189             std::string logoUrl;
00190             for (const char* key : {"medium", "default"}) {
00191                 if (thumbs.contains(key)) {
00192                     logoUrl = thumbs[key].value("url", "");
00193                     if (!logoUrl.empty()) break;
00194                 }
00195             }
00196             if (!logoUrl.empty()) result[channelId] = std::move(logoUrl);
00197         }
00198     } catch (const json::parse_error& ex) {
00199         throw std::runtime_error(
00200             std::string("YouTubeMetadataFetcher: channels JSON parse error: ") + ex.what());
00201     }
00202     return result;
00203 }
00204
00205 // -- Private: httpGet -----
00206
00207 std::string YouTubeMetadataFetcher::httpGet(const std::string& url) {
00208     CURL* curl = curl_easy_init();
00209     if (!curl) throw std::runtime_error("YouTubeMetadataFetcher: curl_easy_init failed");
00210
00211     std::string response;
00212     curl_easy_setopt(curl, CURLOPT_URL, url.c_str());
00213     curl_easy_setopt(curl, CURLOPT_WRITEFUNCTION, writeCallback);
00214     curl_easy_setopt(curl, CURLOPT_WRITEDATA, &response);
00215     curl_easy_setopt(curl, CURLOPT_FOLLOWLOCATION, 1L);
00216     curl_easy_setopt(curl, CURLOPT_HTTPGET, 1L);

```

```

00217 // Aggressive timeouts to prevent hanging on slow/unavailable API endpoints.
00218 // 3s connect timeout ensures socket is reaching the server quickly.
00219 // 10s total timeout prevents long waits for slow API responses.
00220 curl_easy_setopt(curl, CURLOPT_CONNECTTIMEOUT, 3L);
00221 curl_easy_setopt(curl, CURLOPT_TIMEOUT, 10L);
00222
00223 CURLcode res = curl_easy_perform(curl);
00224 curl_easy_cleanup(curl);
00225
00226 if (res != CURLE_OK) {
00227     throw std::runtime_error(
00228         std::string("YouTubeMetadataFetcher: HTTP GET failed: ") +
00229         curl_easy_strerror(res));
00230 }
00231 return response;
00232 }
00233
00234 size_t YouTubeMetadataFetcher::writeCallback(void* contents, size_t size,
00235                                             size_t nmemb, std::string* out) {
00236     out->append(static_cast<char*>(contents), size * nmemb);
00237     return size * nmemb;
00238 }

```

7.123 YouTubeMetadataFetcher.hpp File Reference

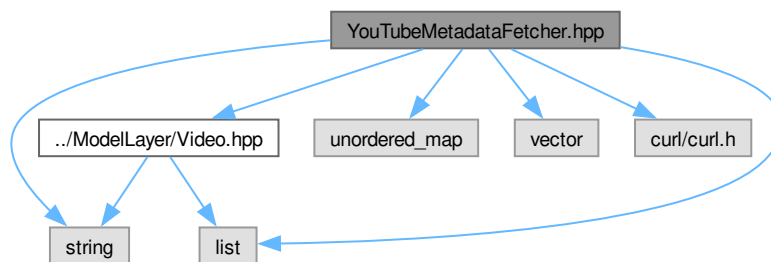
Enriches a list of [Video](#) objects (video-ID only) with full snippet metadata from the YouTube Data API v3.

```

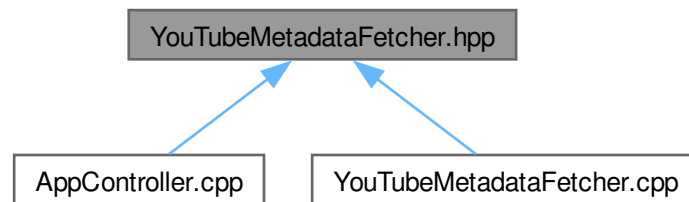
#include <string>
#include <list>
#include <unordered_map>
#include <vector>
#include <curl/curl.h>
#include "../ModelLayer/Video.hpp"

```

Include dependency graph for YouTubeMetadataFetcher.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [YouTubeMetadataFetcher](#)
YouTubeMetadataFetcher class declaration.
- struct [YouTubeMetadataFetcher::VideoMeta](#)
VideoMeta class declaration.

7.123.1 Detailed Description

Enriches a list of [Video](#) objects (video-ID only) with full snippet metadata from the YouTube Data API v3.

Author

Jasmine Kumar

videos.list?part=snippet&id=<ids>&key=<apiKey> -> title, channelId, channelTitle, thumbnailUrl channels.↵
list?part=snippet&id=<channelIds>&key=<apiKey> -> channel logo URL (snippet.thumbnails.default.url)

IDs are batched at most 50 per request (YouTube API limit). Videos not returned by the API (private, deleted) are kept unchanged. Any network or parse error is logged to stderr; the remaining videos are still returned with whatever data was successfully retrieved.

Definition in file [YouTubeMetadataFetcher.hpp](#).

7.124 YouTubeMetadataFetcher.hpp

[Go to the documentation of this file.](#)

```

00001 // =====
00002 // YouTubeMetadataFetcher.hpp
00003 //
00004 // Enriches a list of Video objects (video-ID only) with full snippet metadata
00005 // from the YouTube Data API v3:
00006 //
00007 //   videos.list?part=snippet&id=<ids>&key=<apiKey>
00008 //       -> title, channelId, channelTitle, thumbnailUrl
00009 //
00010 //   channels.list?part=snippet&id=<channelIds>&key=<apiKey>
00011 //       -> channel logo URL (snippet.thumbnails.default.url)
  
```

```

00012 //
00013 // IDs are batched at most 50 per request (YouTube API limit).
00014 // Videos not returned by the API (private, deleted) are kept unchanged.
00015 // Any network or parse error is logged to stderr; the remaining videos are
00016 // still returned with whatever data was successfully retrieved.
00017 // =====
00018
00019 #pragma once
00020
00021 #include <string>
00022 #include <list>
00023 #include <unordered_map>
00024 #include <vector>
00025
00026 #include <curl/curl.h>
00027
00028 #include "../ModelLayer/Video.hpp"
00029
00045
00050 class YouTubeMetadataFetcher {
00051 public:
00052     explicit YouTubeMetadataFetcher(std::string apiKey);
00053
00059     std::list<Video> enrich(const std::list<Video>& videos);
00060
00061 private:
00062     std::string apiKey;
00063
00064     static constexpr int kBatchSize = 50;
00065
00070     struct VideoMeta {
00071         std::string title;
00072         std::string channelId;
00073         std::string channelName;
00074         std::string thumbnailUrl;
00075     };
00076
00077     // Fetches snippet data for up to kBatchSize video IDs.
00078     // Returns a map of videoId -> VideoMeta.
00079     std::unordered_map<std::string, VideoMeta>
00080     fetchVideoBatch(const std::vector<std::string>& ids);
00081
00082     // Fetches the default thumbnail URL for each channel ID.
00083     // Returns a map of channelId -> logoUrl.
00084     std::unordered_map<std::string, std::string>
00085     fetchChannelLogos(const std::vector<std::string>& channelIds);
00086
00087     // Performs a GET request and returns the response body.
00088     // Throws std::runtime_error on curl failure.
00089     std::string httpGet(const std::string& url);
00090
00091     static size_t writeCallback(void* contents, size_t size, size_t nmemb,
00092                                std::string* out);
00093 };

```

7.125 ActiveBlendsPanel.cpp File Reference

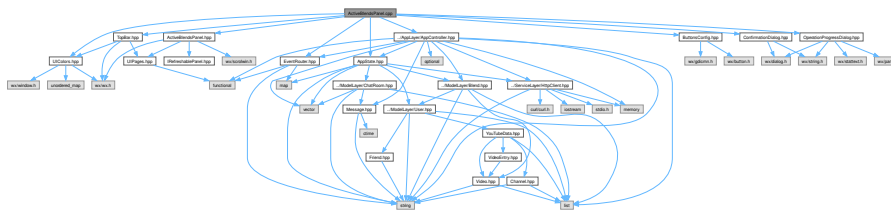
Implementation for [ActiveBlendsPanel](#).

```

#include "ActiveBlendsPanel.hpp"
#include "UIColors.hpp"
#include "ButtonsConfig.hpp"
#include "TopBar.hpp"
#include "ConfirmationDialog.hpp"
#include "OperationProgressDialog.hpp"
#include "../AppLayer/AppController.hpp"
#include "../AppLayer/AppState.hpp"
#include "../AppLayer/EventRouter.hpp"

```

Include dependency graph for ActiveBlendsPanel.cpp:



7.125.1 Detailed Description

Implementation for [ActiveBlendsPanel](#).

Author

Jasmine Kumar

Definition in file [ActiveBlendsPanel.cpp](#).

7.126 ActiveBlendsPanel.cpp

[Go to the documentation of this file.](#)

```
00001
00006
00007 #include "ActiveBlendsPanel.hpp"
00008
00009 #include "UIColors.hpp"
00010 #include "ButtonsConfig.hpp"
00011 #include "TopBar.hpp"
00012 #include "ConfirmationDialog.hpp"
00013 #include "OperationProgressDialog.hpp"
00014 #include "../AppLayer/AppController.hpp"
00015 #include "../AppLayer/AppState.hpp"
00016 #include "../AppLayer/EventRouter.hpp"
00017
00018 ActiveBlendsPanel::ActiveBlendsPanel(wxWindow* parent,
00019                                     ApplicationController& controller,
00020                                     NavigateFn nav)
00021     : wxPanel(parent, wxID_ANY)
00022     , m_controller(controller)
00023     , m_nav(std::move(nav))
00024 {
00025     SetBackgroundColour(UIColors::Background());
00026
00027     auto* root = new wxBoxSizer(wxVERTICAL);
00028
00029     auto* topBar = new TopBar(this, "My Blends", m_nav, Page::HOME);
00030     root->Add(topBar, 0, wxEXPAND);
00031
00032     // Scrollable blend list
00033     m_listScroll = new wxScrolledWindow(this, wxID_ANY,
00034                                         wxDefaultPosition, wxDefaultSize,
00035                                         wxVSCROLL | wxBORDER_NONE);
00036     m_listScroll->SetBackgroundColour(UIColors::Background());
00037     m_listScroll->SetScrollRate(0, 12);
00038
00039     m_listInner = new wxPanel(m_listScroll, wxID_ANY);
00040     m_listInner->SetBackgroundColour(UIColors::Background());
00041     m_listSizer = new wxBoxSizer(wxVERTICAL);
00042     m_listInner->SetSizer(m_listSizer);
00043
00044     auto* scrollSizer = new wxBoxSizer(wxVERTICAL);
00045     scrollSizer->Add(m_listInner, 1, wxEXPAND);
00046     m_listScroll->SetSizer(scrollSizer);
```

```

00047
00048     root->Add(m_listScroll, 1, wxEXPAND | wxALL, 8);
00049
00050     // Create New Blend button at the bottom
00051     auto* newBlendBtn = new wxButton(this, wxID_ANY, "Create New Blend");
00052     UIButtons::ApplySizeBounds(newBlendBtn, ButtonType::FullWidthPrimary);
00053     newBlendBtn->SetBackgroundColour(UIColors::Accent());
00054     newBlendBtn->SetForegroundColour(UIColors::TextPrimary());
00055     newBlendBtn->Bind(wxEVT_BUTTON, [this](wxCommandEvent&) {
00056         m_nav(Page::BLEND_CREATION);
00057     });
00058     root->Add(newBlendBtn, 0, wxEXPAND | wxLEFT | wxRIGHT | wxBOTTOM, 16);
00059
00060     // Empty state label
00061     m_emptyLabel = new wxStaticText(m_listInner, wxID_ANY,
00062                                     "No active blends.",
00063                                     wxDefaultPosition, wxDefaultSize,
00064                                     wxALIGN_CENTER_HORIZONTAL);
00065     m_emptyLabel->SetForegroundColour(UIColors::TextMuted());
00066     m_listSizer->Add(m_emptyLabel, 0, wxALIGN_CENTER_HORIZONTAL | wxTOP, 24);
00067
00068     SetSizer(root);
00069 }
00070
00071 void ActiveBlendsPanel::Refresh()
00072 {
00073     RebuildList();
00074 }
00075
00076 void ActiveBlendsPanel::RebuildList()
00077 {
00078     m_listSizer->Clear(true);
00079
00080     // Re-create empty label (cleared above)
00081     m_emptyLabel = new wxStaticText(m_listInner, wxID_ANY,
00082                                     "No active blends.",
00083                                     wxDefaultPosition, wxDefaultSize,
00084                                     wxALIGN_CENTER_HORIZONTAL);
00085     m_emptyLabel->SetForegroundColour(UIColors::TextMuted());
00086
00087     auto blends = m_controller.fetchUserBlends();
00088
00089     if (blends.empty()) {
00090         m_emptyLabel->Show(true);
00091         m_listSizer->Add(m_emptyLabel, 0, wxALIGN_CENTER_HORIZONTAL | wxTOP, 24);
00092     } else {
00093         m_emptyLabel->Show(false);
00094         m_listSizer->Add(m_emptyLabel, 0);
00095
00096         for (const auto& blend : blends) {
00097             wxColor c_to_use; // holds the background color to use for each list item
00098
00099             // if the the current blend iteration matches the current blend in the app controller,
00100             // change the background so it stands out from other blend items
00101             if (blend.blendID == m_controller.get_current_blend_id()) {
00102                 c_to_use = UIColors::Accent();
00103             } else { // otherwise, set as a basic color for contrast
00104                 c_to_use = UIColors::SurfaceRaised();
00105             }
00106
00107             auto* row = new wxPanel(m_listInner, wxID_ANY);
00108             row->SetBackgroundColour(c_to_use);
00109             row->SetMinSize(wxSize(-1, 52));
00110
00111             auto* rowSizer = new wxBoxSizer(wxHORIZONTAL);
00112
00113             // Blend title label
00114             auto* titleLabel = new wxStaticText(row, wxID_ANY,
00115                                                 wxString::FromUTF8(blend.title));
00116             titleLabel->SetForegroundColour(UIColors::TextPrimary());
00117             wxFont f = titleLabel->GetFont();
00118             f.SetPointSize(12);
00119             f.SetWeight(wxFONTWEIGHT_BOLD);
00120             titleLabel->SetFont(f);
00121
00122             // Open button
00123             auto* openBtn = new wxButton(row, wxID_ANY, "Open");
00124             UIButtons::ApplySizeBounds(openBtn, ButtonType::Compact);
00125             openBtn->SetBackgroundColour(UIColors::Accent());
00126             openBtn->SetForegroundColour(UIColors::TextPrimary());
00127
00128             // Leave button
00129             auto* leaveBtn = new wxButton(row, wxID_ANY, "Leave");
00130             UIButtons::ApplySizeBounds(leaveBtn, ButtonType::Compact);
00131             leaveBtn->SetBackgroundColour(UIColors::Danger());
00132             leaveBtn->SetForegroundColour(UIColors::TextPrimary());
00133

```

```

00134         std::string capID = blend.blendID;
00135         std::string capTitle = blend.title;
00136         openBtn->Bind(wx.EVT_BUTTON, [this, capID, capTitle](wxCommandEvent&) {
00137             OnSelectBlend(capID, capTitle);
00138         });
00139         leaveBtn->Bind(wx.EVT_BUTTON, [this, capID, capTitle](wxCommandEvent&) {
00140             OnLeaveBlend(capID, capTitle);
00141         });
00142
00143         rowSizer->Add(titleLabel, 1, wxALIGN_CENTER_VERTICAL | wxLEFT, 16);
00144         rowSizer->Add(openBtn, 0, wxALIGN_CENTER_VERTICAL | wxRIGHT, 8);
00145         rowSizer->Add(leaveBtn, 0, wxALIGN_CENTER_VERTICAL | wxRIGHT, 16);
00146         row->SetSizer(rowSizer);
00147
00148         m_listSizer->Add(row, 0, wxEXPAND | wxLEFT | wxRIGHT | wxTOP, 6);
00149     }
00150 }
00151
00152 m_listInner->Layout();
00153 m_listScroll->FitInside();
00154 m_listScroll->Layout();
00155 Layout();
00156 }
00157
00158 void ActiveBlendsPanel::OnSelectBlend(const std::string& blendID,
00159                                     const std::string& title)
00160 {
00161     OperationProgressDialog progress(this, "Loading Blend");
00162     progress.ShowModal();
00163     progress.UpdateProgress(0.02, "Preparing...");
00164
00165     // Wire up the progress reporter so ApplicationController updates the bar as it works
00166     m_controller.setProgressReporter([&progress](double ratio, const std::string& message) {
00167         progress.UpdateProgress(ratio, wxString::FromUTF8(message));
00168     });
00169
00170     m_controller.getEventRouter().dispatch("selectBlend",
00171     {"blendID", blendID}, {"blendTitle", title});
00172     m_controller.clearProgressReporter();
00173
00174     progress.UpdateProgress(1.0, "Blend ready");
00175     progress.Destroy();
00176     m_nav(Page::HOME);
00177 }
00178
00179 void ActiveBlendsPanel::OnLeaveBlend(const std::string& blendID,
00180                                    const std::string& title)
00181 {
00182     bool confirmed = ConfirmationDialog::Confirm(
00183         this, "Leave Blend",
00184         "Are you sure you want to leave \"" + title + "\"?");
00185     if (!confirmed) return;
00186
00187     m_controller.getEventRouter().dispatch("leaveBlend",
00188     {"blendID", blendID}, {"blendTitle", title});
00189
00190     RebuildList();
00191 }

```

7.127 ActiveBlendsPanel.hpp File Reference

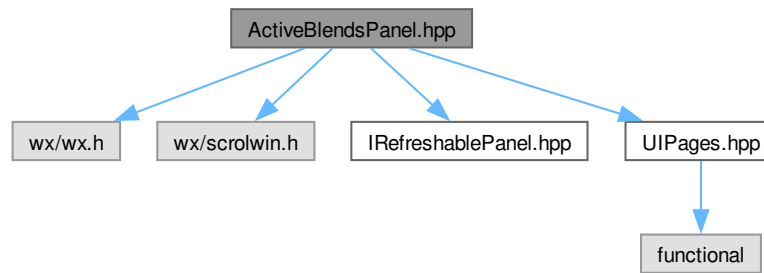
Panel for viewing and managing active blends.

```

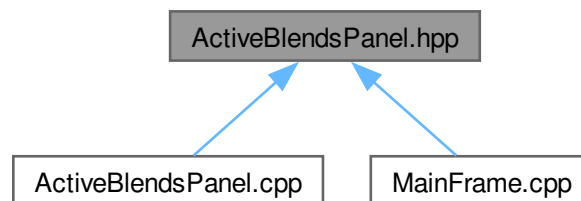
#include <wx/wx.h>
#include <wx/scrolwin.h>
#include "IRefreshablePanel.hpp"
#include "UIPages.hpp"

```

Include dependency graph for `ActiveBlendsPanel.hpp`:



This graph shows which files directly or indirectly include this file:



Classes

- class [ActiveBlendsPanel](#)
[ActiveBlendsPanel](#) class declaration.

7.127.1 Detailed Description

Panel for viewing and managing active blends.

Author

Jasmine Kumar

Definition in file [ActiveBlendsPanel.hpp](#).

7.128 ActiveBlendsPanel.hpp

[Go to the documentation of this file.](#)

```

00001
00006
00007 #pragma once
00008
00009 #include <wx/wx.h>
00010 #include <wx/scrolwin.h>
00011
00012 #include "IRefreshablePanel.hpp"
00013 #include "UIPages.hpp"
00014
00015 class AppController;
00016
00021 class ActiveBlendsPanel : public wxPanel, public IRefreshablePanel {
00022 public:
00023     ActiveBlendsPanel(wxWindow* parent, AppController& controller, NavigateFn nav);
00024     void Refresh() override;
00025 private:
00026     AppController& m_controller;
00027     NavigateFn m_nav;
00028
00029     wxScrolledWindow* m_listScroll;
00030     wxPanel* m_listInner;
00031     wxBoxSizer* m_listSizer;
00032     wxStaticText* m_emptyLabel;
00033
00034     void RebuildList();
00035     void OnSelectBlend(const std::string& blendID, const std::string& title);
00036     void OnLeaveBlend(const std::string& blendID, const std::string& title);
00037 };
00038 
```

7.129 BlendChatPanel.cpp File Reference

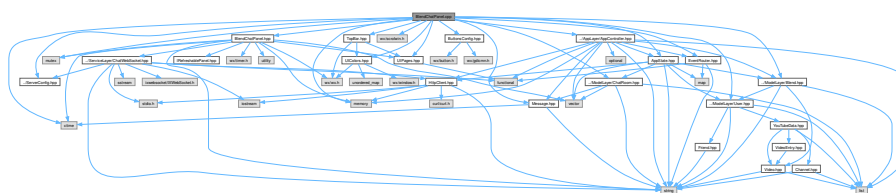
Implementation for [BlendChatPanel](#).

```

#include "BlendChatPanel.hpp"
#include "../ServerConfig.hpp"
#include <wx/scrolwin.h>
#include <ctime>
#include <memory>
#include <mutex>
#include "UIColors.hpp"
#include "ButtonsConfig.hpp"
#include "UIPages.hpp"
#include "TopBar.hpp"
#include "../AppLayer/AppController.hpp"
#include "../AppLayer/AppState.hpp"
#include "../AppLayer/EventRouter.hpp"
#include "../ModelLayer/ChatRoom.hpp"
#include "../ModelLayer/Message.hpp"
#include "../ModelLayer/Blend.hpp"
#include "../ModelLayer/User.hpp"

```

Include dependency graph for BlendChatPanel.cpp:



Functions

- static bool [GetLocalTime](#) (std::time_t ts, std::tm &outLocal)
- static bool [IsSameLocalDay](#) (const std::tm &a, const std::tm &b)

7.129.1 Detailed Description

Implementation for [BlendChatPanel](#).

Author

Jasmine Kumar

Definition in file [BlendChatPanel.cpp](#).

7.129.2 Function Documentation

7.129.2.1 GetLocalTime()

```
bool GetLocalTime (
    std::time_t ts,
    std::tm & outLocal) [static]
```

Definition at line 33 of file [BlendChatPanel.cpp](#).

7.129.2.2 IsSameLocalDay()

```
bool IsSameLocalDay (
    const std::tm & a,
    const std::tm & b) [static]
```

Definition at line 43 of file [BlendChatPanel.cpp](#).

7.130 BlendChatPanel.cpp

[Go to the documentation of this file.](#)

```
00001
00006
00007 // =====
00008 // BlendChatPanel.cpp - Blend chat screen implementation
00009 // =====
00010
00011 #include "BlendChatPanel.hpp"
00012
00013 #include "../ServerConfig.hpp"
00014 #include <wx/scrolwin.h>
00015 #include <ctime>
00016 #include <memory>
00017 #include <mutex>
00018
00019 #include "UIColors.hpp"
00020 #include "ButtonsConfig.hpp"
00021 #include "UIPages.hpp"
00022 #include "TopBar.hpp"
00023 #include "../AppLayer/AppController.hpp"
```



```

00024 #include "../AppLayer/AppState.hpp"
00025 #include "../AppLayer/EventRouter.hpp"
00026 #include "../ModelLayer/ChatRoom.hpp"
00027 #include "../ModelLayer/Message.hpp"
00028 #include "../ModelLayer/Blend.hpp"
00029 #include "../ModelLayer/User.hpp"
00030
00031 // -- Helper: format a Unix timestamp with relative local-time labels ---
00032
00033 static bool GetLocalTime(std::time_t ts, std::tm& outLocal)
00034 {
00035     // Keep conversion thread-safe across platforms.
00036     #if defined(_WIN32)
00037         return localtime_s(&outLocal, &ts) == 0;
00038     #else
00039         return localtime_r(&ts, &outLocal) != nullptr;
00040     #endif
00041 }
00042
00043 static bool IsSameLocalDay(const std::tm& a, const std::tm& b)
00044 {
00045     return a.tm_year == b.tm_year && a.tm_yday == b.tm_yday;
00046 }
00047
00048 wxString BlendChatPanel::FormatTimestamp(std::time_t ts, std::time_t now)
00049 {
00050     std::tm msgLocal{};
00051     std::tm nowLocal{};
00052     if (!GetLocalTime(ts, msgLocal) || !GetLocalTime(now, nowLocal)) {
00053         return "unknown-time";
00054     }
00055
00056     char timeBuf[12]; // "HH:MM\0"
00057     std::strftime(timeBuf, sizeof(timeBuf), "%H:%M", &msgLocal);
00058
00059     if (IsSameLocalDay(msgLocal, nowLocal)) {
00060         return wxString::Format("Today at %s", wxString::FromAscii(timeBuf));
00061     }
00062
00063     std::tm yesterdayLocal{};
00064     if (GetLocalTime(now - 24 * 60 * 60, yesterdayLocal) && IsSameLocalDay(msgLocal, yesterdayLocal))
00065     {
00066         return wxString::Format("Yesterday at %s", wxString::FromAscii(timeBuf));
00067     }
00068
00069     const int monthDiff = (nowLocal.tm_year - msgLocal.tm_year) * 12 + (nowLocal.tm_mon -
00070 msgLocal.tm_mon);
00071     if (monthDiff >= 12) {
00072         char fullBuf[40]; // "Mon DD, YYYY at HH:MM\0"
00073         std::strftime(fullBuf, sizeof(fullBuf), "%b %d, %Y at %H:%M", &msgLocal);
00074         return wxString::FromAscii(fullBuf);
00075     }
00076
00077     char recentBuf[32]; // "Mon DD at HH:MM\0"
00078     std::strftime(recentBuf, sizeof(recentBuf), "%b %d at %H:%M", &msgLocal);
00079     return wxString::FromAscii(recentBuf);
00080 }
00081
00082 // -- Construction -----
00083 BlendChatPanel::BlendChatPanel(wxWindow* parent,
00084                                 AppController& controller,
00085                                 NavigateFn nav)
00086 : wxPanel(parent, wxID_ANY)
00087 , m_controller(controller)
00088 , m_nav(std::move(nav))
00089 {
00090     SetBackgroundColour(UIColors::Background());
00091
00092     auto* root = new wxBoxSizer(wxVERTICAL);
00093
00094     // -- TopBar (title updated in Reload) -----
00095     auto* topBar = new TopBar(this, "Blend Chat", m_nav, Page::HOME);
00096     root->Add(topBar, 0, wxEXPAND);
00097
00098     // -- "No active blend." placeholder -----
00099     m_noBlendLabel = new wxStaticText(this, wxID_ANY, "No active blend.",
00100                                     wxDefaultPosition, wxDefaultSize,
00101                                     wxALIGN_CENTER_HORIZONTAL);
00102     m_noBlendLabel->SetForegroundColour(UIColors::TextMuted());
00103     root->Add(m_noBlendLabel, 1, wxALIGN_CENTER | wxALL, 20);
00104
00105     // -- Participants toggle button + dropdown -----
00106     m_participantsBtn = new wxButton(this, wxID_ANY, "Participants",
00107                                     wxDefaultPosition, wxSize(-1, 48),
00108                                     wxBU_LEFT);
00109     m_participantsBtn->SetBackgroundColour(UIColors::Accent());

```

```

00109     m_participantsBtn->SetForegroundColour(*wxWHITE);
00110     {
00111         wxFont f = m_participantsBtn->GetFont();
00112         f.SetPointSize(14);
00113         f.SetWeight(wxFONTWEIGHT_BOLD);
00114         m_participantsBtn->SetFont(f);
00115     }
00116     m_participantsBtn->Hide(); // shown in Reload() when blend is active
00117     root->Add(m_participantsBtn, 0, wxEXPAND);
00118
00119     m_participantsDropdown = new wxPanel(this, wxID_ANY);
00120     m_participantsDropdown->SetBackgroundColour(UIColors::SurfaceRaised());
00121     m_participantsSizer = new wxBoxSizer(wxVERTICAL);
00122     m_participantsDropdown->SetSizer(m_participantsSizer);
00123     m_participantsDropdown->Hide();
00124     root->Add(m_participantsDropdown, 0, wxEXPAND);
00125
00126     m_participantsBtn->Bind(wxEVT_BUTTON, &BlendChatPanel::OnParticipantsToggle, this);
00127
00128     // -- Message scroll area -----
00129     m_msgScroll = new wxScrolledWindow(this, wxID_ANY,
00130                                     wxDefaultPosition, wxDefaultSize,
00131                                     wxVSCROLL | wxBORDER_NONE);
00132     m_msgScroll->SetBackgroundColour(UIColors::Surface());
00133     m_msgScroll->SetScrollRate(0, 12);
00134
00135     m_msgInner = new wxPanel(m_msgScroll, wxID_ANY);
00136     m_msgInner->SetBackgroundColour(UIColors::Surface());
00137     m_msgSizer = new wxBoxSizer(wxVERTICAL);
00138     m_msgInner->SetSizer(m_msgSizer);
00139
00140     auto* msgScrollSizer = new wxBoxSizer(wxVERTICAL);
00141     msgScrollSizer->Add(m_msgInner, 1, wxEXPAND);
00142     m_msgScroll->SetSizer(msgScrollSizer);
00143
00144     root->Add(m_msgScroll, 1, wxEXPAND);
00145
00146     // -- Input panel -----
00147     m_inputPanel = new wxPanel(this, wxID_ANY);
00148     m_inputPanel->SetBackgroundColour(UIColors::Surface());
00149     m_inputPanel->SetMinSize(wxSize(-1, 60));
00150
00151     auto* inputSizer = new wxBoxSizer(wxHORIZONTAL);
00152
00153     m_inputCtrl = new wxTextCtrl(m_inputPanel, wxID_ANY, wxEmptyString,
00154                                wxDefaultPosition, wxDefaultSize,
00155                                wxTE_PROCESS_ENTER | wxBORDER_SIMPLE);
00156     m_inputCtrl->SetBackgroundColour(UIColors::SurfaceRaised());
00157     m_inputCtrl->SetForegroundColour(UIColors::TextPrimary());
00158     m_inputCtrl->SetHint("Type a message...");
00159
00160     auto* sendBtn = new wxButton(m_inputPanel, wxID_ANY, "Send");
00161     UIButtons::ApplySizeBounds(sendBtn, ButtonType::Nav);
00162     sendBtn->SetBackgroundColour(UIColors::Accent());
00163     sendBtn->SetForegroundColour(UIColors::TextPrimary());
00164
00165     inputSizer->Add(m_inputCtrl, 1, wxALIGN_CENTER_VERTICAL | wxLEFT | wxTOP | wxBOTTOM, 10);
00166     inputSizer->Add(sendBtn, 0, wxALIGN_CENTER_VERTICAL | wxALL, 10);
00167     m_inputPanel->SetSizer(inputSizer);
00168
00169     root->Add(m_inputPanel, 0, wxEXPAND);
00170
00171     SetSizer(root);
00172
00173     // -- Bindings -----
00174     sendBtn->Bind(wxEVT_BUTTON, &BlendChatPanel::OnSend, this);
00175     m_inputCtrl->Bind(wxEVT_TEXT_ENTER, &BlendChatPanel::OnInputEnter, this);
00176
00177     // -- Poll timer - drains incoming WebSocket messages on the main thread ----
00178     m_pollTimer = new wxTimer(this);
00179     Bind(wxEVT_TIMER, &BlendChatPanel::OnPollTimer, this, m_pollTimer->GetId());
00180     m_pollTimer->Start(500 /* ms */);
00181
00182     // -- Logout listener - close WebSocket immediately when user logs out -----
00183     m_controller.getEventRouter().registerListener("logout",
00184         [this](const EventPayload&) {
00185             m_ws.reset();
00186             m_wsBlendID.clear();
00187         });
00188 }
00189
00190 // -- Public -----
00191
00192 void BlendChatPanel::Refresh()
00193 {
00194     Reload();
00195 }

```

```

00196
00197 void BlendChatPanel::Reload()
00198 {
00199     AppState& state = AppState::getInstance();
00200     Blend* blend = state.getActiveBlend();
00201     ChatRoom* room = state.getActiveChatRoom();
00202
00203     bool hasBlend = (blend != nullptr && room != nullptr);
00204
00205     m_noBlendLabel        ->Show(!hasBlend);
00206     m_participantsBtn     ->Show(hasBlend);
00207     m_msgScroll           ->Show(hasBlend);
00208     m_inputPanel          ->Show(hasBlend);
00209
00210     if (!hasBlend) {
00211         m_participantsDropdown->Hide();
00212         m_ws.reset();
00213         m_wsBlendID.clear();
00214         Layout();
00215         return;
00216     }
00217
00218     // -- Connect WebSocket if blend changed or not yet connected -----
00219     User* currentUser = state.getCurrentUser();
00220     if (currentUser && room->getBlendID() != m_wsBlendID) {
00221         // Load history first (synchronous - ChatRoom is populated before UI rebuild)
00222         m_controller.getEventRouter().dispatch("openChat",
00223             {"blendID", room->getBlendID()},
00224             {"userID", currentUser->getUserID()});
00225         ConnectWebSocket(room->getBlendID(), currentUser->getUserID());
00226     }
00227
00228     // -- Populate participants dropdown -----
00229     m_participantsBtn->Show();
00230     m_participantsSizer->Clear(true /* delete_windows */);
00231
00232     for (const User& p : blend->getParticipants()) {
00233         auto* row = new wxPanel(m_participantsDropdown, wxID_ANY);
00234         row->SetBackgroundColour(UIColors::SurfaceRaised());
00235
00236         // Username - bold, slightly larger
00237         auto* nameLabel = new wxStaticText(row, wxID_ANY,
00238             wxString::FromUTF8(p.getUsername()));
00239         nameLabel->SetForegroundColour(UIColors::TextPrimary());
00240         {
00241             wxFont f = nameLabel->GetFont();
00242             f.SetPointSize(11);
00243             f.SetWeight(wxFONTWEIGHT_BOLD);
00244             nameLabel->SetFont(f);
00245         }
00246
00247         // User ID - normal weight, smaller
00248         auto* idLbl = new wxStaticText(row, wxID_ANY,
00249             wxString::FromUTF8(" (" + p.getUserID() + ")"));
00250         idLbl->SetForegroundColour(UIColors::TextSecondary());
00251         {
00252             wxFont f = idLbl->GetFont();
00253             f.SetPointSize(9);
00254             idLbl->SetFont(f);
00255         }
00256
00257         auto* rowSizer = new wxBoxSizer(wxHORIZONTAL);
00258         rowSizer->Add(nameLabel, 0, wxALIGN_CENTER_VERTICAL | wxTOP | wxBOTTOM | wxLEFT, 8);
00259         rowSizer->Add(idLbl, 0, wxALIGN_CENTER_VERTICAL | wxTOP | wxBOTTOM, 8);
00260         row->SetSizer(rowSizer);
00261
00262         m_participantsSizer->Add(row, 0, wxEXPAND | wxBOTTOM, 1);
00263     }
00264
00265     m_participantsDropdown->Layout();
00266
00267     // -- Rebuild message list -----
00268     m_msgSizer->Clear(true /* delete_windows */);
00269
00270     const std::list<Message>& msgs = room->getMessages();
00271
00272     const std::time_t renderNow = std::time(nullptr);
00273
00274     if (msgs.empty()) {
00275         auto* placeholder = new wxStaticText(m_msgInner, wxID_ANY,
00276             "No messages yet.",
00277             wxDefaultPosition, wxDefaultSize,
00278             wxALIGN_CENTER_HORIZONTAL);
00279         placeholder->SetForegroundColour(UIColors::TextMuted());
00280         m_msgSizer->Add(placeholder, 0, wxALIGN_CENTER_HORIZONTAL | wxTOP, 20);
00281     } else {
00282         for (const Message& msg : msgs) {

```

```

00283         // Message row container
00284         auto* rowPanel = new wxPanel(m_msgInner, wxID_ANY);
00285         rowPanel->SetBackgroundColour(UIColors::Surface());
00286
00287         auto* rowSizer = new wxBoxSizer(wxVERTICAL);
00288
00289         // -- Header line: [username bold] [spacer] [UTC timestamp small grey] --
00290         auto* headerLine = new wxBoxSizer(wxHORIZONTAL);
00291
00292         auto* userLabel = new wxStaticText(rowPanel, wxID_ANY,
00293                                           wxString::FromUTF8(msg.userID));
00294         userLabel->SetForegroundColour(UIColors::TextSecondary());
00295         wxFont uf = userLabel->GetFont();
00296         uf.SetWeight(wxFONTWEIGHT_BOLD);
00297         userLabel->SetFont(uf);
00298
00299         auto* tsLabel = new wxStaticText(rowPanel, wxID_ANY,
00300                                         FormatTimestamp(msg.timestamp, renderNow));
00301         tsLabel->SetForegroundColour(UIColors::TextMuted());
00302         wxFont tf = tsLabel->GetFont();
00303         tf.SetPointSize(tf.GetPointSize() - 1);
00304         tsLabel->SetFont(tf);
00305
00306         headerLine->Add(userLabel, 0, wxALIGN_CENTER_VERTICAL);
00307         headerLine->AddStretchSpacer(1);
00308         headerLine->Add(tsLabel, 0, wxALIGN_CENTER_VERTICAL);
00309         rowSizer->Add(headerLine, 0, wxEXPAND | wxLEFT | wxRIGHT | wxTOP, 8);
00310
00311         // -- Message text below the header -----
00312         auto* textLabel = new wxStaticText(rowPanel, wxID_ANY,
00313                                           wxString::FromUTF8(msg.text));
00314         textLabel->SetForegroundColour(UIColors::TextPrimary());
00315         textLabel->Wrap(600);
00316         rowSizer->Add(textLabel, 0, wxEXPAND | wxLEFT | wxRIGHT | wxBOTTOM, 8);
00317
00318         rowPanel->SetSizer(rowSizer);
00319         m_msgSizer->Add(rowPanel, 0, wxEXPAND | wxLEFT | wxRIGHT, 4);
00320
00321         // -- Hover highlight -----
00322         auto applyHover = [=](bool hover) {
00323             wxColour bg = hover ? UIColors::SurfaceRaised() : UIColors::Surface();
00324             rowPanel->SetBackgroundColour(bg);
00325             userLabel->SetBackgroundColour(bg);
00326             tsLabel->SetBackgroundColour(bg);
00327             textLabel->SetBackgroundColour(bg);
00328             rowPanel->Refresh();
00329         };
00330         for (wxWindow* w : std::initializer_list<wxWindow*>{
00331             rowPanel, userLabel, tsLabel, textLabel}) {
00332             w->Bind(wxEVT_ENTER_WINDOW, [applyHover](wxMouseEvent& e){ applyHover(true);
00333             w->Bind(wxEVT_LEAVE_WINDOW, [applyHover](wxMouseEvent& e){ applyHover(false);
00334         e.Skip(); });
00335         }
00336     }
00337
00338     m_msgInner->Layout();
00339     m_msgScroll->FitInside();
00340     m_msgScroll->Layout();
00341
00342     // Scroll to bottom
00343     int xUnit, yUnit;
00344     m_msgScroll->GetScrollPixelsPerUnit(&xUnit, &yUnit);
00345     if (yUnit > 0) {
00346         int vH = m_msgScroll->GetVirtualSize().GetHeight();
00347         m_msgScroll->Scroll(0, vH / yUnit);
00348     }
00349
00350     Layout();
00351 }
00352
00353 // -- Private -----
00354
00355 void BlendChatPanel::DoSend()
00356 {
00357     wxString text = m_inputCtrl->GetValue().Trim().Trim(false);
00358     if (text.IsEmpty()) return;
00359
00360     AppState& state = AppState::getInstance();
00361     User* user = state.getCurrentUser();
00362     ChatRoom* room = state.getActiveChatRoom();
00363
00364     if (!room) return;
00365
00366     std::string userID = user ? user->getUserID() : "anonymous";
00367     std::string textStr = text.ToStdString();

```

```

00368
00369 // Dispatch through ApplicationController - handleSendMessage validates the sender
00370 // and calls room->addMessage(), making the message available for Reload().
00371 m_controller.getEventRouter().dispatch("sendMessage",
00372     {"userID", userID}, {"text", textStr});
00373
00374 // Send to server via WebSocket so other participants receive it.
00375 if (m_ws) {
00376     try {
00377         m_ws->send_message(textStr);
00378     } catch (const std::exception& e) {
00379         std::cerr << "[BlendChatPanel] WebSocket send failed: " << e.what() << "\n";
00380     }
00381 }
00382
00383 m_inputCtrl->Clear();
00384 Reload();
00385 }
00386
00387 void BlendChatPanel::OnSend(wxCommandEvent& /*evt*/)
00388 {
00389     DoSend();
00390 }
00391
00392 void BlendChatPanel::OnInputEnter(wxCommandEvent& /*evt*/)
00393 {
00394     DoSend();
00395 }
00396
00397 void BlendChatPanel::OnParticipantsToggle(wxCommandEvent& /*evt*/)
00398 {
00399     bool nowVisible = !m_participantsDropdown->IsShown();
00400     m_participantsDropdown->Show(nowVisible);
00401     Layout();
00402 }
00403
00404 // -- WebSocket -----
00405
00406 void BlendChatPanel::ConnectWebSocket(const std::string& blendID,
00407     const std::string& userID)
00408 {
00409     // Reset any existing connection first.
00410     m_ws.reset();
00411     m_wsBlendID.clear();
00412
00413     try {
00414         m_ws = std::make_unique<ChatWebSocket>(kAppBackendWsPrefix);
00415         m_ws->set_blendID(blendID);
00416         m_ws->set_userID(userID);
00417
00418         // Register incoming-message callback - runs on the WS background thread.
00419         // Push to m_incoming; OnPollTimer drains it on the main thread.
00420         m_ws->setOnMessage([this](const std::string& sender,
00421             const std::string& content) {
00422             std::lock_guard<std::mutex> lk(m_incomingMx);
00423             m_incoming.emplace_back(sender, content);
00424         });
00425
00426         m_ws->connect();
00427         m_wsBlendID = blendID;
00428     } catch (const std::exception& e) {
00429         std::cerr << "[BlendChatPanel] WebSocket connect failed: " << e.what() << "\n";
00430         m_ws.reset();
00431     }
00432 }
00433
00434 void BlendChatPanel::OnPollTimer(wxTimerEvent& /*evt*/)
00435 {
00436     // Drain incoming messages pushed by the WebSocket background thread.
00437     std::vector<std::pair<std::string, std::string>> batch;
00438     {
00439         std::lock_guard<std::mutex> lk(m_incomingMx);
00440         std::swap(batch, m_incoming);
00441     }
00442
00443     if (batch.empty()) return;
00444
00445     ChatRoom* room = AppState::getInstance().getActiveChatRoom();
00446     if (!room) return;
00447
00448     for (auto& [sender, content] : batch) {
00449         // Skip echoes of our own messages (already added in DoSend).
00450         User* me = AppState::getInstance().getCurrentUser();
00451         if (me && sender == me->getUserID()) continue;
00452         room->addMessage(sender, content);
00453     }
00454 }

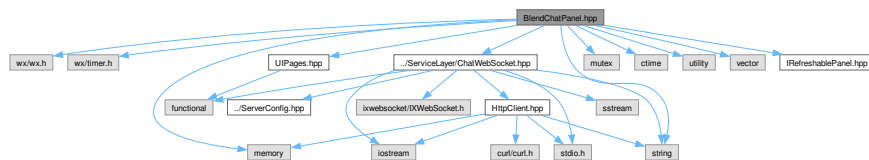
```

```
00455     Reload();
00456 }
```

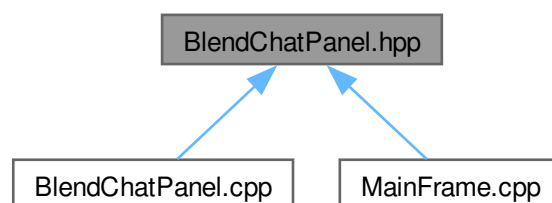
7.131 BlendChatPanel.hpp File Reference

[Blend](#) chat screen.

```
#include <wx/wx.h>
#include <wx/timer.h>
#include <memory>
#include <mutex>
#include <string>
#include <ctime>
#include <utility>
#include <vector>
#include "IRefreshablePanel.hpp"
#include "UIPages.hpp"
#include "../ServiceLayer/ChatWebSocket.hpp"
Include dependency graph for BlendChatPanel.hpp:
```



This graph shows which files directly or indirectly include this file:



Classes

- class [BlendChatPanel](#)
BlendChatPanel class declaration.

7.131.1 Detailed Description

[Blend](#) chat screen.

Author

Jasmine Kumar

Displays the chat room associated with the active blend and lets the user send new messages.

7.131.1.1 LAYOUT

[TopBar](#) ("Blend Chat", back -> HOME) [if no active blend] - "No active blend." label (full height) [if blend active] - scrollable message list + input row

7.131.1.2 MESSAGE LIST

Reload() reads [AppState::getActiveChatRoom\(\)](#)->getMessages() and rebuilds all message rows from scratch. Each row shows: "[userID]: message text" (secondary + primary colour) Relative local timestamp (Today/Yesterday, then month/day, then year after 12 months) Messages are separated by thin wxStaticLine rules. The list auto-scrolls to the bottom after each rebuild.

7.131.1.3 SENDING

DoSend() dispatches "sendMessage" with the current user's ID and the trimmed input text, then calls Reload() so the new message appears. Both the Send button and pressing Enter in the input field call DoSend().

TODO: "sendMessage" is currently a stub in [AppController](#) - messages are added to the in-memory [ChatRoom](#) but may not be persisted to SQLite. Wire up [AppController::handleSendMessage](#) to save to the database. TODO: Real-time updates - currently the list only refreshes on send or manual navigation. Add a polling timer or network event listener to pick up messages from other users. TODO: Show the sender's display name instead of raw userID.

Definition in file [BlendChatPanel.hpp](#).

7.132 BlendChatPanel.hpp

[Go to the documentation of this file.](#)

```
00001
00038
00039 #pragma once
00040
00041 #include <wx/wx.h>
00042 #include <wx/timer.h>
00043
00044 #include <memory>
00045 #include <mutex>
00046 #include <string>
00047 #include <ctime>
00048 #include <utility>
00049 #include <vector>
00050
00051 #include "IRefreshablePanel.hpp"
00052 #include "UIPages.hpp"
00053 #include "../ServiceLayer/ChatWebSocket.hpp"
00054
```

```

00055 class AppController;
00056 class wxScrolledWindow;
00057 class wxTextCtrl;
00058 class wxStaticText;
00059
00064 class BlendChatPanel : public wxPanel, public IRefreshablePanel {
00065 public:
00066     BlendChatPanel(wxWindow* parent, AppController& controller, NavigateFn nav);
00067
00068     void Refresh() override;
00069
00070     // Refreshes message list from AppState's active ChatRoom.
00071     void Reload();
00072
00073 private:
00074     AppController& m_controller;
00075     NavigateFn      m_nav;
00076
00077     // Scrolled window containing the message rows
00078     wxScrolledWindow* m_msgScroll;
00079     // Inner panel inside the scroll window
00080     wxPanel*          m_msgInner;
00081     //Sizer for the inner panel
00082     wxBoxSizer*       m_msgSizer;
00083
00084     // "No active blend." placeholder (shown when no blend is active)
00085     wxStaticText*     m_noBlendLabel;
00086
00087     // Purple toggle button that shows/hides the participants dropdown
00088     wxButton*          m_participantsBtn;
00089     // Dropdown panel containing the participant list (hidden by default)
00090     wxPanel*           m_participantsDropdown;
00091     wxBoxSizer*        m_participantsSizer;
00092
00093     // Text input for typing messages
00094     wxTextCtrl*        m_inputCtrl;
00095
00096     // Panel that wraps the input row (hidden when no blend active)
00097     wxPanel*           m_inputPanel;
00098
00099     // -- WebSocket / incoming message handling -----
00100
00101     // Timer that drains m_incoming on the main thread (~500 ms interval).
00102     wxTimer*           m_pollTimer;
00103
00104     // Blend ID the WebSocket is currently connected to (empty = not connected).
00105     std::string        m_wsBlendID;
00106
00107     // Mutex protecting m_incoming (written by WS background thread, read by timer).
00108     std::mutex          m_incomingMx;
00109
00110     // Incoming messages from other participants waiting to be added to ChatRoom.
00111     // Declared after m_incomingMx; m_ws declared last so it is destroyed first,
00112     // stopping the background thread before the mutex/vector are torn down.
00113     std::vector<std::pair<std::string, std::string>> m_incoming;
00114
00115     // Active WebSocket connection (null when no blend is open).
00116     std::unique_ptr<ChatWebSocket> m_ws;
00117
00118     // Connects (or reconnects) the WebSocket for the given blend/user.
00119     void ConnectWebSocket(const std::string& blendID, const std::string& userID);
00120
00121     // Send logic (shared between button click and Enter key)
00122     void DoSend();
00123
00124     // Event handlers
00125     void OnSend(wxCommandEvent& evt);
00126     void OnInputEnter(wxCommandEvent& evt);
00127     void OnPollTimer(wxTimerEvent& evt);
00128     void OnParticipantsToggle(wxCommandEvent& evt);
00129
00143     static wxString FormatTimestamp(std::time_t ts, std::time_t now);
00144 };

```

7.133 BlendCreationPanel.cpp File Reference

Implementation for [BlendCreationPanel](#).

```

#include "BlendCreationPanel.hpp"
#include <wx/scrolwin.h>

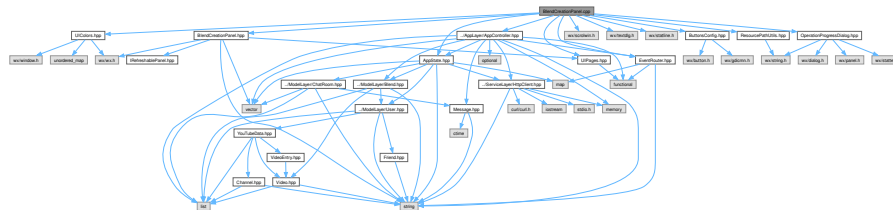
```



```

#include <wx/textdlg.h>
#include <wx/statline.h>
#include "UIColors.hpp"
#include "ButtonsConfig.hpp"
#include "ResourcePathUtils.hpp"
#include "UIPages.hpp"
#include "OperationProgressDialog.hpp"
#include "../AppLayer/AppController.hpp"
#include "../AppLayer/AppState.hpp"
#include "../AppLayer/EventRouter.hpp"
Include dependency graph for BlendCreationPanel.cpp:

```



7.133.1 Detailed Description

Implementation for [BlendCreationPanel](#).

Author

Jasmine Kumar

Definition in file [BlendCreationPanel.cpp](#).

7.134 BlendCreationPanel.cpp

[Go to the documentation of this file.](#)

```

00001
00006
00007 // =====
00008 // BlendCreationPanel.cpp - New-blend setup screen implementation
00009 // =====
00010
00011 #include "BlendCreationPanel.hpp"
00012
00013 #include <wx/scrolwin.h>
00014 #include <wx/textdlg.h>
00015 #include <wx/statline.h>
00016
00017 #include "UIColors.hpp"
00018 #include "ButtonsConfig.hpp"
00019 #include "ResourcePathUtils.hpp"
00020 #include "UIPages.hpp"
00021 #include "OperationProgressDialog.hpp"
00022 #include "../AppLayer/AppController.hpp"
00023 #include "../AppLayer/AppState.hpp"
00024 #include "../AppLayer/EventRouter.hpp"
00025
00026 // -- Construction -----
00027
00028 BlendCreationPanel::BlendCreationPanel(wxWindow* parent,
00029                                       AppController& controller,
00030                                       NavigateFn nav)
00031     : wxPanel(parent, wxID_ANY)

```

```

00032     , m_controller(controller)
00033     , m_nav(std::move(nav))
00034 {
00035     SetBackgroundColour(UIColors::Background());
00036
00037     auto* root = new wxBoxSizer(wxVERTICAL);
00038
00039     // -- TopBar (dynamic back: LOGIN if no blend yet, HOME otherwise) -----
00040     auto* topBar = new wxPanel(this, wxID_ANY);
00041     topBar->SetBackgroundColour(UIColors::Background());
00042     topBar->SetMinSize(wxSize(-1, 40));
00043     {
00044         auto* backBtn = new wxButton(topBar, wxID_ANY, wxEmptyString,
00045                                     wxDefaultPosition, wxSize(36, 32),
00046                                     wxBORDER_NONE);
00047         backBtn->SetBackgroundColour(UIColors::Background());
00048
00049         // Load back icon
00050         wxString themeName = UIColors::Current ? UIColors::Current->Name : wxString("Dark Mode");
00051         wxString folder = themeName.BeforeFirst(' ').Lower();
00052         const wxString iconPath = UIResourcePaths::FindResourcePath("icons/" + folder + "/back.png");
00053         if (!iconPath.empty()) {
00054             wxImage img(iconPath, wxBITMAP_TYPE_PNG);
00055             if (img.IsOk()) {
00056                 img.Rescale(24, 24, wxIMAGE_QUALITY_BILINEAR);
00057                 backBtn->SetBitmap(wxBitmap(img));
00058             }
00059         }
00060
00061         backBtn->Bind(wxEVT_BUTTON, [this](wxCommandEvent&) {
00062             if (AppState::getInstance().getActiveBlend() != nullptr)
00063                 m_nav(Page::HOME);
00064             else
00065                 m_nav(Page::LOGIN);
00066         });
00067         backBtn->Bind(wxEVT_ENTER_WINDOW, [backBtn](wxMouseEvent& e) {
00068             backBtn->SetBackgroundColour(UIColors::SurfaceRaised());
00069             backBtn->Refresh(); e.Skip();
00070         });
00071         backBtn->Bind(wxEVT_LEAVE_WINDOW, [backBtn](wxMouseEvent& e) {
00072             backBtn->SetBackgroundColour(UIColors::Background());
00073             backBtn->Refresh(); e.Skip();
00074         });
00075
00076         auto* titleLabel = new wxStaticText(topBar, wxID_ANY, "New Blend",
00077                                             wxDefaultPosition, wxDefaultSize,
00078                                             wxALIGN_CENTER_HORIZONTAL);
00079         titleLabel->SetForegroundColour(UIColors::TextPrimary());
00080         wxFont tf = titleLabel->GetFont();
00081         tf.SetPointSize(14);
00082         tf.SetWeight(wxFONTWEIGHT_BOLD);
00083         titleLabel->SetFont(tf);
00084
00085         auto* barSizer = new wxBoxSizer(wxHORIZONTAL);
00086         barSizer->Add(backBtn, 0, wxALIGN_CENTER_VERTICAL | wxLEFT, 8);
00087         barSizer->AddStretchSpacer(1);
00088         barSizer->Add(titleLabel, 0, wxALIGN_CENTER_VERTICAL);
00089         barSizer->AddStretchSpacer(1);
00090         barSizer->Add(36 + 8, 0);
00091
00092         auto* outerBar = new wxBoxSizer(wxVERTICAL);
00093         outerBar->Add(barSizer, 1, wxEXPAND | wxTOP | wxBOTTOM, 4);
00094         topBar->SetSizer(outerBar);
00095     }
00096     root->Add(topBar, 0, wxEXPAND);
00097
00098     // -- Blend name row -----
00099     auto* namePanel = new wxPanel(this, wxID_ANY);
00100     namePanel->SetBackgroundColour(UIColors::Surface());
00101     auto* nameSizer = new wxBoxSizer(wxHORIZONTAL);
00102
00103     auto* nameLabel = new wxStaticText(namePanel, wxID_ANY, "Blend Name:");
00104     nameLabel->SetForegroundColour(UIColors::TextPrimary());
00105     wxFont nlf = nameLabel->GetFont();
00106     nlf.SetWeight(wxFONTWEIGHT_BOLD);
00107     nlf.SetPointSize(11);
00108     nameLabel->SetFont(nlf);
00109
00110     m_blendNameCtrl = new wxTextCtrl(namePanel, wxID_ANY, "",
00111                                     wxDefaultPosition, wxSize(-1, 36),
00112                                     wxTE_PROCESS_ENTER);
00113     m_blendNameCtrl->SetHint("Enter a name for your blend...");
00114     m_blendNameCtrl->SetBackgroundColour(UIColors::SurfaceRaised());
00115     m_blendNameCtrl->SetForegroundColour(UIColors::TextPrimary());
00116
00117     nameSizer->Add(nameLabel, 0, wxALIGN_CENTER_VERTICAL | wxLEFT, 16);
00118     nameSizer->Add(m_blendNameCtrl, 1, wxALIGN_CENTER_VERTICAL | wxLEFT | wxRIGHT, 12);

```

```

00119     namePanel->SetSizer(nameSizer);
00120     namePanel->SetMinSize(wxSize(-1, 48));
00121     root->Add(namePanel, 0, wxEXPAND);
00122
00123     // -- Action row -----
00124     auto* actionPanel = new wxPanel(this, wxID_ANY);
00125     actionPanel->SetBackgroundColour(UIColors::Surface());
00126     auto* actionSizer = new wxBoxSizer(wxHORIZONTAL);
00127
00128     auto* blendLabel = new wxStaticText(actionPanel, wxID_ANY, "PARTICIPANTS");
00129     blendLabel->SetForegroundColour(UIColors::TextPrimary());
00130     wxFont lf = blendLabel->GetFont();
00131     lf.SetWeight(wxFONTWEIGHT_BOLD);
00132     lf.SetPointSize(13);
00133     blendLabel->SetFont(lf);
00134
00135     auto* addBtn = new wxButton(actionPanel, wxID_ANY, "Add");
00136     UIButtons::ApplySizeBounds(addBtn, ButtonType::Compact);
00137     addBtn->SetBackgroundColour(UIColors::SurfaceRaised());
00138     addBtn->SetForegroundColour(UIColors::TextPrimary());
00139
00140     m_createBtn = new wxButton(actionPanel, wxID_ANY, "Create Blend");
00141     UIButtons::ApplySizeBounds(m_createBtn, ButtonType::Medium);
00142     m_createBtn->SetBackgroundColour(UIColors::Accent());
00143     m_createBtn->SetForegroundColour(UIColors::TextPrimary());
00144     m_createBtn->Disable();
00145
00146     actionSizer->Add(blendLabel, 0, wxALIGN_CENTER_VERTICAL | wxLEFT, 16);
00147     actionSizer->AddStretchSpacer(1);
00148     actionSizer->Add(addBtn, 0, wxALIGN_CENTER_VERTICAL | wxRIGHT, 8);
00149     actionSizer->Add(m_createBtn, 0, wxALIGN_CENTER_VERTICAL | wxRIGHT, 16);
00150     actionPanel->SetSizer(actionSizer);
00151     actionPanel->SetMinSize(wxSize(-1, 52));
00152
00153     root->Add(actionPanel, 0, wxEXPAND);
00154
00155     // -- User list area -----
00156     m_userListScroll = new wxScrolledWindow(this, wxID_ANY,
00157                                             wxDefaultPosition, wxDefaultSize,
00158                                             wxVSCROLL | wxBORDER_NONE);
00159     m_userListScroll->SetBackgroundColour(UIColors::Background());
00160     m_userListScroll->SetScrollRate(0, 12);
00161
00162     m_userListInner = new wxPanel(m_userListScroll, wxID_ANY);
00163     m_userListInner->SetBackgroundColour(UIColors::Background());
00164     m_userListSizer = new wxBoxSizer(wxVERTICAL);
00165
00166     // "No users added" placeholder
00167     m_emptyLabel = new wxStaticText(m_userListInner, wxID_ANY,
00168                                     "No users added",
00169                                     wxDefaultPosition, wxDefaultSize,
00170                                     wxALIGN_CENTER_HORIZONTAL);
00171     m_emptyLabel->SetForegroundColour(UIColors::TextMuted());
00172     m_userListSizer->Add(m_emptyLabel, 0,
00173                         wxALIGN_CENTER_HORIZONTAL | wxTOP, 24);
00174
00175     m_userListInner->SetSizer(m_userListSizer);
00176
00177     // Wire the inner panel into the scroll window
00178     auto* scrollSizer = new wxBoxSizer(wxVERTICAL);
00179     scrollSizer->Add(m_userListInner, 1, wxEXPAND);
00180     m_userListScroll->SetSizer(scrollSizer);
00181
00182     root->Add(m_userListScroll, 1, wxEXPAND | wxALL, 8);
00183
00184     // -- Count indicator -----
00185     m_countLabel = new wxStaticText(this, wxID_ANY, "0 / 8 users",
00186                                     wxDefaultPosition, wxDefaultSize,
00187                                     wxALIGN_CENTER_HORIZONTAL);
00188     m_countLabel->SetForegroundColour(UIColors::TextSecondary());
00189     root->Add(m_countLabel, 0, wxALIGN_CENTER_HORIZONTAL | wxBOTTOM, 12);
00190
00191     SetSizer(root);
00192
00193     // -- Bindings -----
00194     addBtn->Bind(wxEVT_BUTTON, &BlendCreationPanel::OnAdd, this);
00195     m_createBtn->Bind(wxEVT_BUTTON, &BlendCreationPanel::OnCreate, this);
00196 }
00197
00198 // -- Public -----
00199
00200 void BlendCreationPanel::Refresh()
00201 {
00202     Reload();
00203 }
00204
00205 void BlendCreationPanel::Reload()

```

```

00206 {
00207     m_addedUsers.clear();
00208     m_blendNameCtrl->Clear();
00209
00210     // Pre-populate with the logged-in user so they don't have to add themselves
00211     User* current = AppState::getInstance().getCurrentUser();
00212     if (current)
00213         m_addedUsers.push_back(current->getUserID());
00214
00215     RebuildUserList();
00216     UpdateCountLabel();
00217
00218     // Button enabled once there are >= 2 users (current user + at least one other)
00219     if (m_addedUsers.size() >= 2)
00220         m_createBtn->Enable();
00221     else
00222         m_createBtn->Disable();
00223 }
00224
00225 // -- Private helpers -----
00226
00227 void BlendCreationPanel::RebuildUserList()
00228 {
00229     // Destroy all children of the inner panel except the empty label.
00230     // We recreate the inner sizer from scratch.
00231     m_userListSizer->Clear(true /* delete_windows */);
00232
00233     // Re-create the empty label (it was deleted above)
00234     m_emptyLabel = new wxStaticText(m_userListInner, wxID_ANY,
00235                                     "No users added",
00236                                     wxDefaultPosition, wxDefaultSize,
00237                                     wxALIGN_CENTER_HORIZONTAL);
00238     m_emptyLabel->SetForegroundColour(UIColors::TextMuted());
00239
00240     if (m_addedUsers.empty()) {
00241         m_emptyLabel->Show(true);
00242         m_userListSizer->Add(m_emptyLabel, 0,
00243                             wxALIGN_CENTER_HORIZONTAL | wxTOP, 24);
00244     } else {
00245         m_emptyLabel->Show(false);
00246         m_userListSizer->Add(m_emptyLabel, 0); // keep in sizer but hidden
00247
00248         for (const auto& username : m_addedUsers) {
00249             // Row panel
00250             auto* row = new wxPanel(m_userListInner, wxID_ANY);
00251             row->SetBackgroundColour(UIColors::SurfaceRaised());
00252             row->SetMinSize(wxSize(-1, 44));
00253
00254             auto* rowSizer = new wxBoxSizer(wxHORIZONTAL);
00255
00256             auto* nameLabel = new wxStaticText(row, wxID_ANY,
00257                                                wxString::FromUTF8(username));
00258             nameLabel->SetForegroundColour(UIColors::TextPrimary());
00259
00260             auto* removeBtn = new wxButton(row, wxID_ANY, wxEmptyString,
00261                                            wxDefaultPosition, wxSize(30, 30),
00262                                            wxBORDER_NONE);
00263             removeBtn->SetBackgroundColour(UIColors::SurfaceRaised());
00264             {
00265                 wxString themeName = UIColors::Current ? UIColors::Current->Name : wxString("Dark
Mode");
00266                 wxString folder = themeName.BeforeFirst(' ').Lower();
00267                 for (const auto& c : wxArrayString{
00268                     "YTBLND_src/resources/icons/" + folder + "/remove.png",
00269                     "resources/icons/" + folder + "/remove.png",
00270                     "../resources/icons/" + folder + "/remove.png"}) {
00271                     if (wxFileExists(c)) {
00272                         wxImage img(c, wxBITMAP_TYPE_PNG);
00273                         if (img.IsOk()) {
00274                             img.Rescale(18, 18, wxIMAGE_QUALITY_BILINEAR);
00275                             removeBtn->SetBitmap(wxBitmap(img));
00276                         }
00277                         break;
00278                     }
00279                 }
00280             }
00281             removeBtn->Bind(wxEVT_ENTER_WINDOW, [removeBtn](wxMouseEvent& e) {
00282                 removeBtn->SetBackgroundColour(UIColors::Surface());
00283                 removeBtn->Refresh(); e.Skip();
00284             });
00285             removeBtn->Bind(wxEVT_LEAVE_WINDOW, [removeBtn](wxMouseEvent& e) {
00286                 removeBtn->SetBackgroundColour(UIColors::SurfaceRaised());
00287                 removeBtn->Refresh(); e.Skip();
00288             });
00289
00290             // Capture username by value for the lambda
00291             std::string cap = username;

```

```

00292         removeBtn->Bind(wx.EVT_BUTTON, [this, cap](wxCommandEvent&) {
00293             OnRemoveUser(cap);
00294         });
00295
00296         rowSizer->Add(nameLabel, 0, wxALIGN_CENTER_VERTICAL | wxLEFT, 16);
00297         rowSizer->AddStretchSpacer(1);
00298         rowSizer->Add(removeBtn, 0, wxALIGN_CENTER_VERTICAL | wxRIGHT, 8);
00299         row->SetSizer(rowSizer);
00300
00301         m_userListSizer->Add(row, 0, wxEXPAND | wxLEFT | wxRIGHT | wxTOP,
00302             6);
00303     }
00304 }
00305
00306 m_userListInner->Layout();
00307 m_userListScroll->FitInside();
00308 m_userListScroll->Layout();
00309 }
00310
00311 void BlendCreationPanel::UpdateCountLabel()
00312 {
00313     wxString txt = wxString::Format("%d / 8 users",
00314                                     static_cast<int>(m_addedUsers.size()));
00315     m_countLabel->SetLabel(txt);
00316 }
00317
00318 // -- Event handlers -----
00319
00320 void BlendCreationPanel::OnAdd(wxCommandEvent& /*evt*/)
00321 {
00322     if (static_cast<int>(m_addedUsers.size()) >= 8) {
00323         wxMessageBox("Maximum 8 users reached.", "Limit Reached",
00324                     wxOK | wxICON_INFORMATION, this);
00325         return;
00326     }
00327
00328     wxTextEntryDialog dlg(this, "Enter username:", "Add User");
00329     dlg.SetBackgroundColour(UIColors::Surface());
00330     if (dlg.ShowModal() != wxID_OK) return;
00331
00332     std::string entered = dlg.GetValue().Trim().ToStdString();
00333
00334     if (entered.empty()) return;
00335
00336     // Check for duplicate
00337     for (const auto& u : m_addedUsers) {
00338         if (u == entered) {
00339             wxMessageBox("User \"" + entered + "\" is already in the list.",
00340                         "Duplicate User", wxOK | wxICON_WARNING, this);
00341             return;
00342         }
00343     }
00344
00345     if (!m_controller.lookupUser(entered)) {
00346         wxMessageBox("User \"" + entered + "\" does not exist.\n"
00347                     "Please ask them to create an account first.",
00348                     "User Not Found", wxOK | wxICON_ERROR, this);
00349         return;
00350     }
00351
00352     m_addedUsers.push_back(entered);
00353     RebuildUserList();
00354     UpdateCountLabel();
00355
00356     if (m_addedUsers.size() >= 2)
00357         m_createBtn->Enable();
00358     else
00359         m_createBtn->Disable();
00360 }
00361
00362 void BlendCreationPanel::OnCreate(wxCommandEvent& /*evt*/)
00363 {
00364     if (m_addedUsers.size() < 2) return;
00365
00366     std::string blendName = m_blendNameCtrl->GetValue().ToStdString();
00367     if (blendName.empty()) {
00368         wxMessageBox("Please enter a name for your blend.",
00369                     "Name Required", wxOK | wxICON_INFORMATION, this);
00370         m_blendNameCtrl->SetFocus();
00371         return;
00372     }
00373
00374     // Build payload: {"blendTitle": "...", "userID_0": "alice", "userID_1": "bob", ...}
00375     EventPayload payload;
00376     payload["blendTitle"] = blendName;
00377     for (std::size_t i = 0; i < m_addedUsers.size(); ++i)
00378         payload["userID_" + std::to_string(i)] = m_addedUsers[i];

```

```

00379
00380     OperationProgressDialog progress(this, "Creating Blend");
00381     progress.ShowModal();
00382     progress.UpdateProgress(0.02, "Preparing...");
00383
00384     // Wire up the progress reporter so AppController updates the bar as it works
00385     m_controller.setProgressReporter([&progress](double ratio, const std::string& message) {
00386         progress.UpdateProgress(ratio, wxString::FromUTF8(message));
00387     });
00388
00389     m_controller.getEventRouter().dispatch("createBlend", payload);
00390     m_controller.clearProgressReporter();
00391
00392     progress.UpdateProgress(1.0, "Blend ready");
00393     progress.Destroy();
00394
00395     // Check if any participants were missing data
00396     std::vector<std::string> missing = AppState::getInstance().getUsersWithoutData();
00397     if (!missing.empty()) {
00398         std::string msg = "The following users have no Watch Later data uploaded:\n";
00399         for (const auto& uid : missing)
00400             msg += "    \u2022 " + uid + "\n";
00401
00402         if (AppState::getInstance().getActiveBlend() != nullptr) {
00403             // Blend was still created from whoever had data - warn and continue
00404             msg += "\nTheir videos were not included. The blend was created with the remaining
users.";
00405             wxMessageBox(msg, "Some Users Missing Data", wxOK | wxICON_WARNING, this);
00406             m_nav(Page::HOME);
00407         } else {
00408             // No one had data - cannot create a blend at all
00409             msg += "\nNo blend could be created. Please ensure at least one user has uploaded their
data.";
00410             wxMessageBox(msg, "Cannot Create Blend", wxOK | wxICON_ERROR, this);
00411         }
00412         return;
00413     }
00414
00415     m_nav(Page::HOME);
00416 }
00417
00418 void BlendCreationPanel::OnRemoveUser(const std::string& username)
00419 {
00420     auto it = std::find(m_addedUsers.begin(), m_addedUsers.end(), username);
00421     if (it != m_addedUsers.end())
00422         m_addedUsers.erase(it);
00423
00424     RebuildUserList();
00425     UpdateCountLabel();
00426
00427     if (m_addedUsers.size() >= 2)
00428         m_createBtn->Enable();
00429     else
00430         m_createBtn->Disable();
00431 }

```

7.135 BlendCreationPanel.hpp File Reference

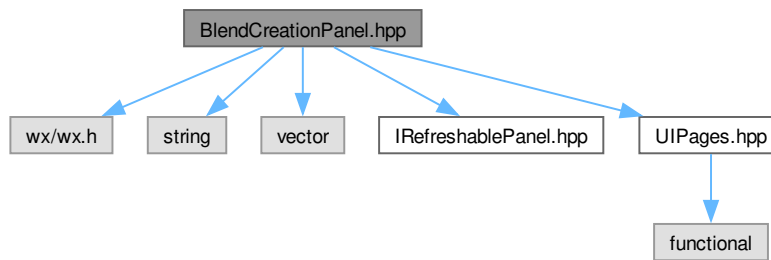
New-blend setup screen.

```

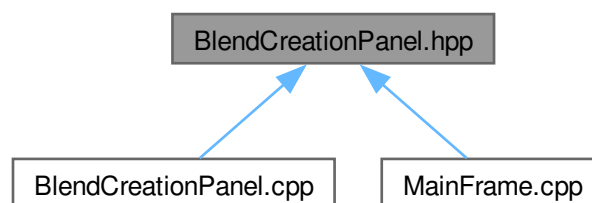
#include <wx/wx.h>
#include <string>
#include <vector>
#include "IRefreshablePanel.hpp"
#include "UIPages.hpp"

```

Include dependency graph for BlendCreationPanel.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [BlendCreationPanel](#)
[BlendCreationPanel](#) class declaration.

7.135.1 Detailed Description

New-blend setup screen.

Author

Jasmine Kumar

Lets the logged-in user assemble a group of up to 8 participants (by username) and then create a blend from their combined YouTube data.

7.135.1.1 LAYOUT

[TopBar](#) ("New Blend", back -> HOME) Action row: "BLEND" label | Add button | Create [Blend](#) button Scrollable user list (wxScrolledWindow) "X / 8 users" count label

7.135.1.2 USER-ADD FLOW

1. [User](#) clicks Add -> wxTextEntryDialog prompts for a username.
2. Duplicate check against m_addedUsers.
3. "lookupUser" event dispatched to [AppController](#) (optimistic - user is added to the list regardless of whether the backend finds them). TODO: Wire up AppController::handleLookupUser to validate that the username exists in the database before adding.
4. "Create Blend" button is enabled once ≥ 2 users are present.

7.135.1.3 BLEND CREATION

OnCreate builds a payload {"userID_0": ..., "userID_1": ..., ...} and dispatches "createBlend". On return it navigates to BLEND_CHAT. TODO: If createBlend fails (e.g. one of the users has no data), display an error instead of navigating away.

7.135.1.4 RELOAD

Reload() is called by MainFrame::NavigateTo(BLEND_CREATION). It clears m_addedUsers so the panel starts fresh each time it is shown. TODO: Pre-populate the current user's own username so they do not have to add themselves manually.

Definition in file [BlendCreationPanel.hpp](#).

7.136 BlendCreationPanel.hpp

[Go to the documentation of this file.](#)

```

00001
00040
00041 #pragma once
00042
00043 #include <wx/wx.h>
00044 #include <string>
00045 #include <vector>
00046
00047 #include "IRefreshablePanel.hpp"
00048 #include "UIPages.hpp"
00049
00050 class AppController;
00051 class wxScrolledWindow;
00052 class wxStaticText;
00053 class wxButton;
00054
00059 class BlendCreationPanel : public wxPanel, public IRefreshablePanel {
00060 public:
00061     BlendCreationPanel(wxWindow* parent, AppController& controller, NavigateFn nav);
00062
00063     void Refresh() override;
00064
00065     // Clears user list and resets to empty state.
00066     void Reload();
00067
00068     // Test-friendly accessors
00069     #ifdef TESTING_MODE
00070     const std::vector<std::string>& GetAddedUsersForTesting() const {
00071         return m_addedUsers;
00072     }
00073     wxButton* GetCreateBtnForTesting() const { return m_createBtn; }
00074     wxStaticText* GetCountLabelForTesting() const { return m_countLabel; }
00075     void AddUserForTesting(const std::string& username) {
00076         m_addedUsers.push_back(username);
00077         RebuildUserList();

```



```

00078         UpdateCountLabel();
00079         if (m_addedUsers.size() >= 2) m_createBtn->Enable();
00080     }
00081     void RemoveUserForTesting(const std::string& username) {
00082         OnRemoveUser(username);
00083     }
00084     void CallOnCreateForTesting() {
00085         wxCommandEvent evt(wxEVT_BUTTON);
00086         OnCreate(evt);
00087     }
00088     #endif
00089
00090 private:
00091     AppController& m_controller;
00092     NavigateFn      m_nav;
00093
00094     // The scrolled container that holds user rows
00095     wxScrolledWindow* m_userListScroll;
00096     // Inner panel inside the scroll window where rows are placed
00097     wxPanel*          m_userListInner;
00098     // Sizer for the inner panel
00099     wxBoxSizer*       m_userListSizer;
00100
00101     // "No users added" placeholder label
00102     wxStaticText*     m_emptyLabel;
00103
00104     // "X / 8 users" count label
00105     wxStaticText*     m_countLabel;
00106
00107     // Blend name input
00108     wxTextCtrl*       m_blendNameCtrl;
00109
00110     // "Create Blend" button - enabled when >= 2 users added
00111     wxButton*         m_createBtn;
00112
00113     // Currently added usernames
00114     std::vector<std::string> m_addedUsers;
00115
00116     // Rebuilds the user-row display from m_addedUsers.
00117     void RebuildUserList();
00118
00119     // Updates the count label text.
00120     void UpdateCountLabel();
00121
00122     // Event handlers
00123     void OnAdd(wxCommandEvent& evt);
00124     void OnCreate(wxCommandEvent& evt);
00125     void OnRemoveUser(const std::string& username);
00126 };

```

7.137 BlendFeedPanel.cpp File Reference

Implementation for [BlendFeedPanel](#).

```

#include "BlendFeedPanel.hpp"
#include <wx/sizer.h>
#include "UIColors.hpp"
#include "../AppLayer/AppState.hpp"

```



```

00038     outer->Add(grid, 1, wxEXPAND | wxALL, 16);
00039     SetSizer(outer);
00040
00041     LoadFromBlend();
00042 }
00043
00044 void BlendFeedPanel::Refresh()
00045 {
00046     m_offset = 0;
00047     LoadFromBlend();
00048 }
00049
00050 // -----
00051 // NextPage
00052 // -----
00053 void BlendFeedPanel::NextPage()
00054 {
00055     Blend* blend = AppState::getInstance().getActiveBlend();
00056     if (!blend || blend->size() == 0) {
00057         m_offset = 0;
00058         LoadFromBlend();
00059         return;
00060     }
00061
00062     int nextOffset = m_offset + kPageSize;
00063     if (blend->size() - nextOffset < kPageSize) {
00064         // Not enough videos remaining to fill a full page - refresh with unseen videos
00065         m_controller.getEventRouter().dispatch("refresh");
00066         m_offset = 0;
00067     } else {
00068         m_offset = nextOffset;
00069     }
00070     LoadFromBlend();
00071 }
00072
00073 // -----
00074 // LoadFromBlend
00075 // -----
00076 void BlendFeedPanel::LoadFromBlend()
00077 {
00078     Blend* blend = AppState::getInstance().getActiveBlend();
00079
00080     if (!blend || blend->size() == 0) {
00081         // No active blend - show all cards in placeholder state
00082         for (int i = 0; i < kPageSize; ++i) {
00083             m_cards[i]->Clear();
00084         }
00085         return;
00086     }
00087
00088     const int total = blend->size();
00089
00090     // Clamp offset in case the blend shrank since last NextPage()
00091     if (m_offset >= total) {
00092         m_offset = 0;
00093     }
00094
00095     for (int i = 0; i < kPageSize; ++i) {
00096         int idx = m_offset + i;
00097         if (idx < total)
00098             m_cards[i]->SetVideo(blend->getVideo(idx));
00099         else
00100             m_cards[i]->Clear();
00101     }
00102 }

```

7.139 BlendFeedPanel.hpp File Reference

Home-page video feed panel that displays paged blend results.

```

#include <wx/wx.h>
#include "VideoCard.hpp"
#include "IRefreshablePanel.hpp"
#include "../AppLayer/AppController.hpp"

```


7.139.1.2 LOADING

LoadFromBlend() reads [AppState](#) directly so it always reflects the current active blend.

Definition in file [BlendFeedPanel.hpp](#).

7.140 BlendFeedPanel.hpp

[Go to the documentation of this file.](#)

```

00001
00020
00021 #pragma once
00022
00023 #include <wx/wx.h>
00024
00025 #include "VideoCard.hpp"
00026 #include "IRefreshablePanel.hpp"
00027 #include "../AppLayer/AppController.hpp"
00028
00033 class BlendFeedPanel : public wxPanel, public IRefreshablePanel {
00034 public:
00040     BlendFeedPanel(wxWindow* parent, AppController& controller);
00041
00045     void Refresh() override;
00046
00050     void NextPage();
00051
00057     void LoadFromBlend();
00058
00059 private:
00060     static constexpr int kPageSize = 6;    // 3 cols x 2 rows
00061
00062     AppController& m_controller;
00063     VideoCard*     m_cards[kPageSize];
00064     int            m_offset;    // index of the first card in the current page
00065 };

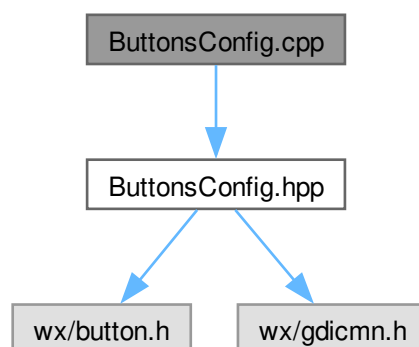
```

7.141 ButtonsConfig.cpp File Reference

Implements shared button sizing presets used across UILayer.

```
#include "ButtonsConfig.hpp"
```

Include dependency graph for ButtonsConfig.cpp:



Functions

- const [BtnConfig](#) & [UIButtons::GetConfig](#) ([ButtonType](#) type)
Returns the raw dimension preset for a button type.
- wxSize [UIButtons::GetSize](#) ([ButtonType](#) type)
Converts a preset into its minimum-size wxSize.
- wxSize [UIButtons::GetMinSize](#) ([ButtonType](#) type)
Returns the configured minimum size for a preset.
- wxSize [UIButtons::GetMaxSize](#) ([ButtonType](#) type)
Returns the configured maximum size for a preset.
- void [UIButtons::ApplySizeBounds](#) (wxButton *button, [ButtonType](#) type)
Applies the selected preset's min and max size bounds.

7.141.1 Detailed Description

Implements shared button sizing presets used across UILayer.

Author

Shamar Pennant

Definition in file [ButtonsConfig.cpp](#).

7.141.2 Function Documentation

7.141.2.1 ApplySizeBounds()

```
void UIButtons::ApplySizeBounds (
    wxButton * button,
    ButtonType type)
```

Applies the selected preset's min and max size bounds.

Parameters

<i>button</i>	Target button.
<i>type</i>	Preset key.

Definition at line 67 of file [ButtonsConfig.cpp](#).

7.141.2.2 GetConfig()

```
const BtnConfig & UIButtons::GetConfig (
    ButtonType type)
```

Returns the raw dimension preset for a button type.

Parameters

<i>type</i>	Preset key.
-------------	-------------

Returns

Constant preset dimensions.

Definition at line 30 of file [ButtonsConfig.cpp](#).

7.141.2.3 GetMaxSize()

```
wxSize UIButtons::GetMaxSize (  
    ButtonType type)
```

Returns the configured maximum size for a preset.

Parameters

<i>type</i>	Preset key.
-------------	-------------

Returns

Maximum size constraints. (-1, -1) means no max bound.

Definition at line 61 of file [ButtonsConfig.cpp](#).

7.141.2.4 GetMinSize()

```
wxSize UIButtons::GetMinSize (  
    ButtonType type)
```

Returns the configured minimum size for a preset.

Parameters

<i>type</i>	Preset key.
-------------	-------------

Returns

Minimum size constraints.

Definition at line 55 of file [ButtonsConfig.cpp](#).

7.141.2.5 GetSize()

```
wxSize UIButtons::GetSize (  
    ButtonType type)
```

Converts a preset into its minimum-size wxSize.

Parameters

<i>type</i>	Preset key.
-------------	-------------

Returns

Equivalent minimum-size wxSize for the selected preset.

Definition at line 50 of file [ButtonsConfig.cpp](#).

7.142 ButtonsConfig.cpp

[Go to the documentation of this file.](#)

```

00001
00006
00007 #include "ButtonsConfig.hpp"
00008
00009 namespace {
00014 constexpr BtnConfig kTopBarBack = {80, 30, 180, 36};
00015 constexpr BtnConfig kNav = {96, 36, 220, 44};
00016 constexpr BtnConfig kCompact = {80, 34, 160, 40};
00017 constexpr BtnConfig kSmall = {100, 34, 180, 40};
00018 constexpr BtnConfig kMedium = {120, 34, 260, 42};
00019 constexpr BtnConfig kLarge = {140, 34, 280, 42};
00020 constexpr BtnConfig kFormTab = {120, 36, 260, 44};
00021 constexpr BtnConfig kFormSubmit = {140, 40, 320, 48};
00022 constexpr BtnConfig kFullWidthPrimary = {160, 40, 560, 48};
00023 constexpr BtnConfig kFullWidthSecondary = {160, 32, 560, 40};
00024 constexpr BtnConfig kFullWidthDanger = {160, 40, 560, 48};
00025 constexpr BtnConfig kIcon = {30, 30, 44, 44};
00026 }
00027
00028 namespace UIButtons {
00029
00030 const BtnConfig& GetConfig(ButtonType type)
00031 {
00032     switch (type) {
00033     case ButtonType::TopBarBack:      return kTopBarBack;
00034     case ButtonType::Nav:              return kNav;
00035     case ButtonType::Compact:         return kCompact;
00036     case ButtonType::Small:           return kSmall;
00037     case ButtonType::Medium:          return kMedium;
00038     case ButtonType::Large:           return kLarge;
00039     case ButtonType::FormTab:         return kFormTab;
00040     case ButtonType::FormSubmit:      return kFormSubmit;
00041     case ButtonType::FullWidthPrimary: return kFullWidthPrimary;
00042     case ButtonType::FullWidthSecondary: return kFullWidthSecondary;
00043     case ButtonType::FullWidthDanger: return kFullWidthDanger;
00044     case ButtonType::Icon:            return kIcon;
00045     }
00046
00047     return kNav;
00048 }
00049
00050 wxSize GetSize(ButtonType type)
00051 {
00052     return GetMinSize(type);
00053 }
00054
00055 wxSize GetMinSize(ButtonType type)
00056 {
00057     const BtnConfig& cfg = GetConfig(type);
00058     return wxSize(cfg.minWidth, cfg.minHeight);
00059 }
00060
00061 wxSize GetMaxSize(ButtonType type)
00062 {
00063     const BtnConfig& cfg = GetConfig(type);
00064     return wxSize(cfg.maxWidth, cfg.maxHeight);
00065 }
00066
00067 void ApplySizeBounds(wxButton* button, ButtonType type)
00068 {
00069     if (!button) {

```



```

00070         return;
00071     }
00072
00073     button->SetMinSize(GetMinSize(type));
00074     button->SetMaxSize(GetMaxSize(type));
00075 }
00076
00077 } // namespace UIButtons

```

7.143 ButtonsConfig.hpp File Reference

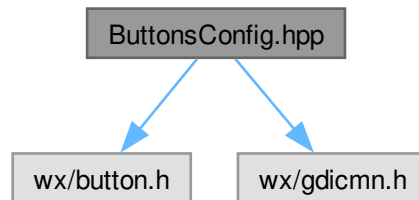
Centralized button size presets for UI layer widgets.

```

#include <wx/button.h>
#include <wx/gdicmn.h>

```

Include dependency graph for ButtonsConfig.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- struct [BtnConfig](#)
Width/height dimensions for a button preset.

Enumerations

- enum class [ButtonType](#) {
[TopBarBack](#) = 0 , [Nav](#) , [Compact](#) , [Small](#) ,
[Medium](#) , [Large](#) , [FormTab](#) , [FormSubmit](#) ,
[FullWidthPrimary](#) , [FullWidthSecondary](#) , [FullWidthDanger](#) , [Icon](#) }
Named presets used to keep button sizing consistent across panels.

Functions

- const [BtnConfig](#) & [UIButtons::GetConfig](#) ([ButtonType](#) type)
Returns the raw dimension preset for a button type.
- wxSize [UIButtons::GetMinSize](#) ([ButtonType](#) type)
Returns the configured minimum size for a preset.
- wxSize [UIButtons::GetMaxSize](#) ([ButtonType](#) type)
Returns the configured maximum size for a preset.
- wxSize [UIButtons::GetSize](#) ([ButtonType](#) type)
Converts a preset into its minimum-size wxSize.
- void [UIButtons::ApplySizeBounds](#) (wxButton *button, [ButtonType](#) type)
Applies the selected preset's min and max size bounds.

7.143.1 Detailed Description

Centralized button size presets for UI layer widgets.

Author

Shamar Pennant

Presets are organized by usage area:

- Navigation: [TopBarBack](#), [Nav](#)
- Forms: [FormTab](#), [FormSubmit](#)
- Full-width actions: [FullWidthPrimary](#), [FullWidthSecondary](#), [FullWidthDanger](#)
- Utility/dialog actions: [Compact](#), [Small](#), [Medium](#), [Icon](#)

Use [UIButtons::ApplySizeBounds\(\)](#) for direct control styling, or [UIButtons::GetSize\(\)](#) when a wxSize is required.

Note: [ApplySizeBounds\(\)](#) also applies an optional max size bound from config.

Definition in file [ButtonsConfig.hpp](#).

7.143.2 Enumeration Type Documentation

7.143.2.1 ButtonType

```
enum class ButtonType [strong]
```

Named presets used to keep button sizing consistent across panels.

Enumerator

TopBarBack	Navigation back button (TopBar).
Nav	Primary nav buttons used in page headers/home top rows.
Compact	Small utility action (e.g., Add button).
Small	Dialog action button size.
Medium	Medium action button size.
FormTab	Login/Register tab buttons.
FormSubmit	Form submit actions (Sign In / Create Account).
FullWidthPrimary	Full-width prominent action.
FullWidthSecondary	Full-width secondary action.
FullWidthDanger	Full-width destructive action (Log Out).
Icon	Square icon-style action (e.g., remove-row button).

Definition at line 28 of file [ButtonsConfig.hpp](#).

7.143.3 Function Documentation

7.143.3.1 ApplySizeBounds()

```
void UIButtons::ApplySizeBounds (
    wxButton * button,
    ButtonType type)
```

Applies the selected preset's min and max size bounds.

Parameters

<i>button</i>	Target button.
<i>type</i>	Preset key.

Definition at line 67 of file [ButtonsConfig.cpp](#).

7.143.3.2 GetConfig()

```
const BtnConfig & UIButtons::GetConfig (
    ButtonType type)
```

Returns the raw dimension preset for a button type.

Parameters

<i>type</i>	Preset key.
-------------	-------------

Returns

Constant preset dimensions.

Definition at line 30 of file [ButtonsConfig.cpp](#).

7.143.3.3 GetMaxSize()

```
wxSize UIButtons::GetMaxSize (
    ButtonType type)
```

Returns the configured maximum size for a preset.

Parameters

<i>type</i>	Preset key.
-------------	-------------

Returns

Maximum size constraints. (-1, -1) means no max bound.

Definition at line 61 of file [ButtonsConfig.cpp](#).

7.143.3.4 GetMinSize()

```
wxSize UIButtons::GetMinSize (
    ButtonType type)
```

Returns the configured minimum size for a preset.

Parameters

<i>type</i>	Preset key.
-------------	-------------

Returns

Minimum size constraints.

Definition at line 55 of file [ButtonsConfig.cpp](#).

7.143.3.5 GetSize()

```
wxSize UIButtons::GetSize (
    ButtonType type)
```

Converts a preset into its minimum-size wxSize.

Parameters

<i>type</i>	Preset key.
-------------	-------------

Returns

Equivalent minimum-size wxSize for the selected preset.

Definition at line 50 of file [ButtonsConfig.cpp](#).

7.144 ButtonsConfig.hpp

[Go to the documentation of this file.](#)

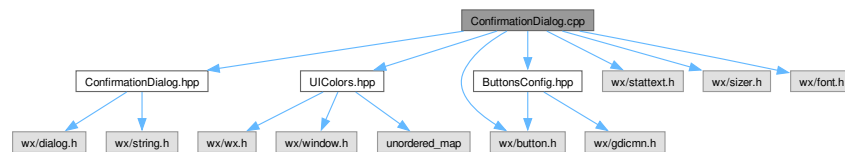
```
00001
00018
00019 #pragma once
00020
00021 #include <wx/button.h>
00022 #include <wx/gdicmn.h>
00023
00028 enum class ButtonType {
00029     TopBarBack = 0,
00030     Nav,
00031     Compact,
00032     Small,
00033     Medium,
00034     Large,
00035     FormTab,
00036     FormSubmit,
00037     FullWidthPrimary,
00038     FullWidthSecondary,
00039     FullWidthDanger,
00040     Icon
00041 };
00042
00050 struct BtnConfig {
00051     int minWidth;
00052     int minHeight;
00053     int maxWidth;
00054     int maxHeight;
00055 };
00056
00057 namespace UIButtons {
00063     const BtnConfig& GetConfig(ButtonType type);
00064
00070     wxSize GetMinSize(ButtonType type);
00071
00077     wxSize GetMaxSize(ButtonType type);
00078
00084     wxSize GetSize(ButtonType type);
00085
00091     void ApplySizeBounds(wxButton* button, ButtonType type);
00092 }
```

7.145 ConfirmationDialog.cpp File Reference

Implementation for [ConfirmationDialog](#).

```
#include "ConfirmationDialog.hpp"
#include "UIColors.hpp"
#include "ButtonsConfig.hpp"
#include <wx/stattext.h>
#include <wx/button.h>
#include <wx/sizer.h>
#include <wx/font.h>
```

Include dependency graph for ConfirmationDialog.cpp:



7.145.1 Detailed Description

Implementation for [ConfirmationDialog](#).

Author

Jasmine Kumar

Definition in file [ConfirmationDialog.cpp](#).

7.146 ConfirmationDialog.cpp

[Go to the documentation of this file.](#)

```
00001
00006
00007 #include "ConfirmationDialog.hpp"
00008 #include "UIColors.hpp"
00009 #include "ButtonsConfig.hpp"
00010 #include <wx/stattext.h>
00011 #include <wx/button.h>
00012 #include <wx/sizer.h>
00013 #include <wx/font.h>
00014
00015 // -----
00016 // Constructor
00017 // -----
00018 ConfirmationDialog::ConfirmationDialog(wxWindow* parent,
00019                                     const wxString& title,
00020                                     const wxString& message,
00021                                     const wxString& okLabel,
00022                                     const wxString& cancelLabel)
00023 : wxDialog(parent, wxID_ANY, title,
00024           wxDefaultPosition, wxDefaultSize,
00025           wxDEFAULT_DIALOG_STYLE | wxSTAY_ON_TOP)
00026 {
00027     // -- Overall panel background -----
00028     SetBackgroundColour(UIColors::Background());
00029
```

```

00030 // -- Message label -----
00031 wxStaticText* msgLabel = new wxStaticText(this, wxID_ANY, message,
00032                                           wxDefaultPosition, wxDefaultSize,
00033                                           wxALIGN_CENTER_HORIZONTAL);
00034 msgLabel->SetForegroundColour(UIColors::TextPrimary());
00035 msgLabel->SetBackgroundColour(UIColors::Background());
00036
00037 wxFont msgFont = msgLabel->GetFont();
00038 msgFont.SetPointSize(11);
00039 msgLabel->SetFont(msgFont);
00040
00041 // -- OK button (Accent colour) -----
00042 wxButton* okBtn = new wxButton(this, wxID_OK, okLabel,
00043                                wxDefaultPosition, wxDefaultSize,
00044                                wxBORDER_NONE);
00045 UIButtons::ApplySizeBounds(okBtn, ButtonType::Small);
00046 okBtn->SetBackgroundColour(UIColors::Accent());
00047 okBtn->SetForegroundColour(UIColors::TextPrimary());
00048
00049 okBtn->Bind(wxEVT_ENTER_WINDOW, [okBtn](wxMouseEvent& e) {
00050     okBtn->SetBackgroundColour(UIColors::AccentHover());
00051     okBtn->Refresh();
00052     e.Skip();
00053 });
00054 okBtn->Bind(wxEVT_LEAVE_WINDOW, [okBtn](wxMouseEvent& e) {
00055     okBtn->SetBackgroundColour(UIColors::Accent());
00056     okBtn->Refresh();
00057     e.Skip();
00058 });
00059
00060 // -- Cancel button (SurfaceRaised) -----
00061 wxButton* cancelBtn = new wxButton(this, wxID_CANCEL, cancelLabel,
00062                                    wxDefaultPosition, wxDefaultSize,
00063                                    wxBORDER_NONE);
00064 UIButtons::ApplySizeBounds(cancelBtn, ButtonType::Small);
00065 cancelBtn->SetBackgroundColour(UIColors::SurfaceRaised());
00066 cancelBtn->SetForegroundColour(UIColors::TextPrimary());
00067
00068 cancelBtn->Bind(wxEVT_ENTER_WINDOW, [cancelBtn](wxMouseEvent& e) {
00069     cancelBtn->SetBackgroundColour(wxColour(55, 55, 55));
00070     cancelBtn->Refresh();
00071     e.Skip();
00072 });
00073 cancelBtn->Bind(wxEVT_LEAVE_WINDOW, [cancelBtn](wxMouseEvent& e) {
00074     cancelBtn->SetBackgroundColour(UIColors::SurfaceRaised());
00075     cancelBtn->Refresh();
00076     e.Skip();
00077 });
00078
00079 // -- Button row -----
00080 wxBoxSizer* btnSizer = new wxBoxSizer(wxHORIZONTAL);
00081 btnSizer->Add(cancelBtn, 0, wxRIGHT, 12);
00082 btnSizer->Add(okBtn, 0);
00083
00084 // -- Root sizer with generous padding for "rounded" feel -----
00085 wxBoxSizer* rootSizer = new wxBoxSizer(wxVERTICAL);
00086 rootSizer->AddSpacer(24);
00087 rootSizer->Add(msgLabel, 0, wxEXPAND | wxLEFT | wxRIGHT, 32);
00088 rootSizer->AddSpacer(28);
00089 rootSizer->Add(btnSizer, 0, wxALIGN_RIGHT | wxRIGHT, 32);
00090 rootSizer->AddSpacer(24);
00091
00092 SetSizer(rootSizer);
00093 Fit();
00094 SetMinSize(wxSize(340, -1));
00095 Fit(); // refit after applying min size
00096
00097 if (parent) {
00098     CentreOnParent();
00099 } else {
00100     CentreOnScreen();
00101 }
00102 }
00103
00104 // -----
00105 // Static convenience helper
00106 // -----
00107 bool ConfirmationDialog::Confirm(wxWindow* parent,
00108                                  const wxString& title,
00109                                  const wxString& message)
00110 {
00111     ConfirmationDialog dlg(parent, title, message);
00112     return dlg.ShowModal() == wxID_OK;
00113 }

```

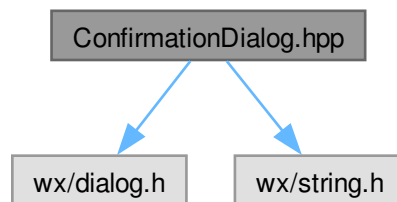
7.147 ConfirmationDialog.hpp File Reference

Dark-themed modal confirmation dialog.

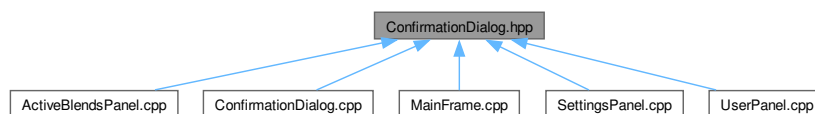
```
#include <wx/dialog.h>
```

```
#include <wx/string.h>
```

Include dependency graph for ConfirmationDialog.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [ConfirmationDialog](#)
[ConfirmationDialog](#) class declaration.

7.147.1 Detailed Description

Dark-themed modal confirmation dialog.

Author

Jasmine Kumar

Definition in file [ConfirmationDialog.hpp](#).

7.148 ConfirmationDialog.hpp

[Go to the documentation of this file.](#)

```

00001
00006
00007 // A simple two-button (OK / Cancel) dialog that matches the YTBIND dark theme.
00008 // Use the static Confirm() helper for the common one-shot pattern, or
00009 // construct manually when you need custom button labels.
00010 //
00011 // Typical usage (static helper):
00012 //   if (ConfirmationDialog::Confirm(this, "Log Out", "Are you sure?")) { ... }
00013 //
00014 // Custom labels:
00015 //   ConfirmationDialog dlg(this, "Delete Blend", "This cannot be undone.",
00016 //                           "Delete", "Keep");
00017 //   if (dlg.ShowModal() == wxID_OK) { ... }
00018 //
00019 // Styling notes:
00020 //   - OK button uses UIColors::Accent() (purple)
00021 //   - Cancel button uses UIColors::SurfaceRaised() (grey)
00022 //   - Both buttons lighten slightly on hover
00023 //
00024 // TODO: Add a wxIcon / warning symbol to the left of the message for
00025 //        destructive-action dialogs (delete, logout, etc.).
00026
00027 #pragma once
00028 #include <wx/dialog.h>
00029 #include <wx/string.h>
00030
00031 // A dark-themed modal confirmation dialog.
00032 //
00033 // Typical usage via the static helper:
00034 //   if (ConfirmationDialog::Confirm(parent, "Delete blend?", "This cannot be undone.)) { ... }
00035 //
00036 // Or construct manually for custom button labels:
00037 //   ConfirmationDialog dlg(parent, "Title", "Message", "Delete", "Keep");
00038 //   if (dlg.ShowModal() == wxID_OK) { ... }
00043 class ConfirmationDialog : public wxDialog {
00044 public:
00045     ConfirmationDialog(wxWindow*      parent,
00046                       const wxString& title,
00047                       const wxString& message,
00048                       const wxString& okLabel    = wxT("OK"),
00049                       const wxString& cancelLabel = wxT("Cancel"));
00050
00051     // Convenience: create, show, destroy - returns true if OK was clicked.
00052     static bool Confirm(wxWindow*      parent,
00053                       const wxString& title,
00054                       const wxString& message);
00055 };

```

7.149 DataInstructionsPanel.cpp File Reference

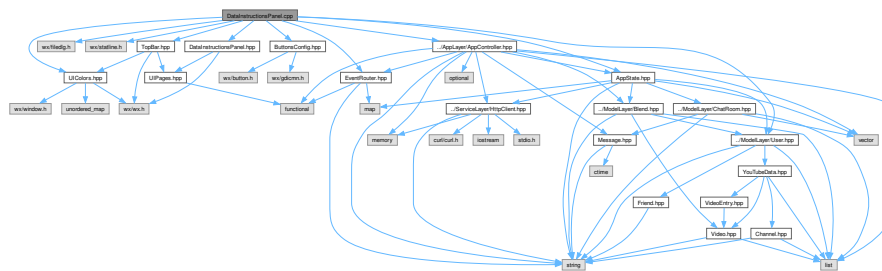
Implementation for [DataInstructionsPanel](#).

```

#include "DataInstructionsPanel.hpp"
#include <wx/filedlg.h>
#include <wx/statline.h>
#include "UIColors.hpp"
#include "ButtonsConfig.hpp"
#include "TopBar.hpp"
#include "../AppLayer/AppController.hpp"
#include "../AppLayer/AppState.hpp"
#include "../AppLayer/EventRouter.hpp"
#include "../ModelLayer/User.hpp"

```


Include dependency graph for DataInstructionsPanel.cpp:



7.149.1 Detailed Description

Implementation for `DataInstructionsPanel`.

Author

Jasmine Kumar

Definition in file [DataInstructionsPanel.cpp](#).

7.150 DataInstructionsPanel.cpp

[Go to the documentation of this file.](#)

[illegible]

```

00044     {
00045         wxFont f = heading->GetFont();
00046         f.SetPointSize(22); f.SetWeight(wxFONTWEIGHT_BOLD);
00047         heading->SetFont(f);
00048     }
00049     heading->SetForegroundColour(UIColors::TextPrimary());
00050
00051     auto* sub = new wxStaticText(this, wxID_ANY,
00052         "YTLND needs your YouTube export data to create a blend.",
00053         wxDefaultPosition, wxDefaultSize, wxALIGN_CENTRE_HORIZONTAL);
00054     sub->SetForegroundColour(UIColors::TextSecondary());
00055
00056     outer->AddStretchSpacer(1);
00057     outer->Add(heading, 0, wxALIGN_CENTER | wxBOTTOM, 8);
00058     outer->Add(sub, 0, wxALIGN_CENTER | wxBOTTOM, 32);
00059
00060     // -- Steps card -----
00061     auto* card = new wxPanel(this, wxID_ANY);
00062     card->SetBackgroundColour(UIColors::Surface());
00063     card->SetMinSize(wxSize(520, -1));
00064     auto* cardSizer = new wxBoxSizer(wxVERTICAL);
00065
00066     const wxString steps =
00067         "1. Go to takeout.google.com\n"
00068         "2. Click \"Deselect all\", then tick YouTube and YouTube Music\n"
00069         "3. Export and download the archive\n"
00070         "4. Inside the zip, choose one of these files:\n"
00071         "    - playlists / <any playlist>.csv (e.g. Watch later videos.csv)\n"
00072         "    - history / watch-history.html\n"
00073         "5. Click \"Browse for Data File...\" below and select that file";
00074
00075     auto* stepsLabel = new wxStaticText(card, wxID_ANY, steps);
00076     stepsLabel->SetForegroundColour(UIColors::TextPrimary());
00077     {
00078         wxFont f = stepsLabel->GetFont(); f.SetPointSize(11);
00079         stepsLabel->SetFont(f);
00080     }
00081
00082     auto* div = new wxStaticLine(card, wxID_ANY);
00083     div->SetBackgroundColour(UIColors::Separator());
00084
00085     auto* browseBtn = new wxButton(card, wxID_ANY, "Browse for Data File...");
00086     UIButtons::ApplySizeBounds(browseBtn, ButtonType::FullWidthPrimary);
00087     browseBtn->SetBackgroundColour(UIColors::Accent());
00088     browseBtn->SetForegroundColour(UIColors::TextPrimary());
00089
00090     cardSizer->Add(stepsLabel, 0, wxALL, 24);
00091     cardSizer->Add(div, 0, wxEXPAND | wxLEFT | wxRIGHT, 0);
00092     cardSizer->Add(browseBtn, 0, wxEXPAND | wxALL, 20);
00093     card->SetSizer(cardSizer);
00094
00095     auto* hCentre = new wxBoxSizer(wxHORIZONTAL);
00096     hCentre->AddStretchSpacer(1);
00097     hCentre->Add(card, 0, wxEXPAND);
00098     hCentre->AddStretchSpacer(1);
00099
00100     outer->Add(hCentre, 0, wxEXPAND);
00101     outer->AddStretchSpacer(2);
00102     SetSizer(outer);
00103
00104     browseBtn->Bind(wx.EVT_BUTTON, &DataInstructionsPanel::OnBrowse, this);
00105 }
00106
00107 void DataInstructionsPanel::OnBrowse(wxCommandEvent& /*evt*/)
00108 {
00109     wxFileDialog dlg(this, "Select YouTube Export Data", "", "",
00110         "Supported files (*.csv;*.html;*.htm)|*.csv;*.html;*.htm|"
00111         "CSV files (*.csv)|*.csv|"
00112         "HTML files (*.html;*.htm)|*.html;*.htm|"
00113         "All files (*.*)|*.*",
00114         wxFD_OPEN | wxFD_FILE_MUST_EXIST);
00115     if (dlg.ShowModal() != wxID_OK) return;
00116
00117     std::string path = dlg.GetPath().ToStdString();
00118     User* user = AppState::getInstance().getCurrentUser();
00119     if (!user) return;
00120
00121     m_controller.getEventRouter().dispatch("uploadData",
00122         {"filePath", path}, {"userID", user->getUserID()});
00123
00124     std::string uploadError = AppState::getInstance().takePendingUploadError();
00125     if (!uploadError.empty()) {
00126         wxMessageBox(uploadError, "Upload Failed", wxOK | wxICON_ERROR, this);
00127         return;
00128     }
00129
00130     if (user->getYouTubeData().getWatchLaterVideos().empty()) {

```

```

00131         wxMessageBox(
00132             "Upload did not produce any usable videos. Please try another file.",
00133             "Upload Incomplete",
00134             wxOK | wxICON_WARNING,
00135             this
00136         );
00137         return;
00138     }
00139     m_nav(Page::BLEND_CREATION);
00140 }
00141
00142

```

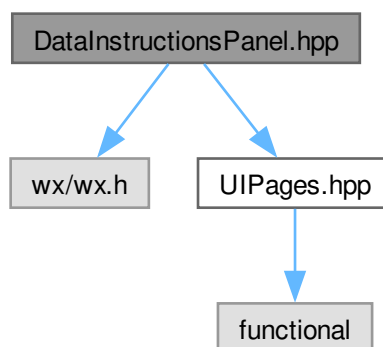
7.151 DataInstructionsPanel.hpp File Reference

Setup instructions panel for importing Google Takeout CSV data.

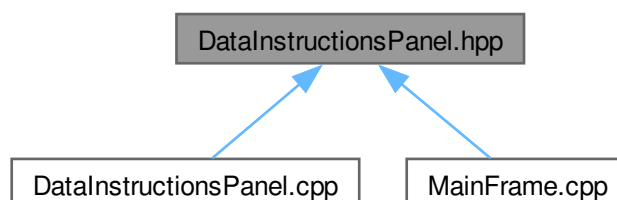
```
#include <wx/wx.h>
```

```
#include "UIPages.hpp"
```

Include dependency graph for DataInstructionsPanel.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [DataInstructionsPanel](#)

[DataInstructionsPanel](#) class declaration.

7.151.1 Detailed Description

Setup instructions panel for importing Google Takeout CSV data.

Author

Jasmine Kumar

Definition in file [DataInstructionsPanel.hpp](#).

7.152 DataInstructionsPanel.hpp

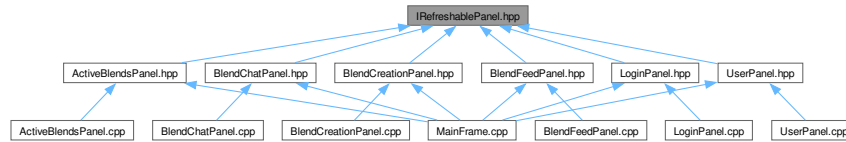
[Go to the documentation of this file.](#)

```
00001
00006
00007 // Shown after login when the user has no YouTube Watch Later data.
00008 // Walks them through exporting from Google Takeout and loading the CSV.
00009 //
00010 // LAYOUT
00011 // -----
00012 // "Set Up Your YouTube Data" heading
00013 // Subtitle explaining why data is needed
00014 // Steps card (Surface, centred):
00015 //     Numbered step-by-step instructions for Google Takeout export
00016 //     Horizontal divider
00017 //     "Browse for CSV..." button (Accent)
00018 //     "Skip for now" button (SurfaceRaised / muted)
00019 //
00020 // CSV UPLOAD
00021 // -----
00022 // OnBrowse: opens wxFileDialog (CSV filter), dispatches "uploadData" with
00023 // {filePath, userID} from AppState, then navigates to BLEND_CREATION.
00024 // TODO: Display a loading indicator while parsing - large CSV files may
00025 // take a noticeable amount of time.
00026 // TODO: Show an error/success message if the upload fails or the CSV is
00027 // malformed, rather than always navigating to BLEND_CREATION.
00028 //
00029 // SKIP
00030 // ----
00031 // OnSkip: navigates directly to BLEND_CREATION without uploading.
00032 // The user can still create a blend (the algorithm will have less/no
00033 // personal data to work with).
00034 // TODO: Remind the user on the blend creation screen that they skipped
00035 // data upload, so the blend may not be personalised.
00036
00037 #pragma once
00038 #include <wx/wx.h>
00039 #include "UIPages.hpp"
00040
00041 class AppController;
00042
00043 // Shown after login when the user has no YouTube data loaded yet.
00044 // Explains how to obtain and place the Watch Later CSV, then lets the
00045 // user browse for the file or skip and go straight to blend creation.
00050 class DataInstructionsPanel : public wxPanel {
00051 public:
00052     DataInstructionsPanel(wxWindow* parent, AppController& controller, NavigateFn nav);
00053
00054 private:
00055     AppController& m_controller;
00056     NavigateFn m_nav;
00057
00058     void OnBrowse(wxCommandEvent&);
00059 };
```

7.153 IRefreshablePanel.hpp File Reference

Interface for panels that support refresh operations.

This graph shows which files directly or indirectly include this file:



Classes

- class [IRefreshablePanel](#)
Interface for panels that support refresh operations.

7.153.1 Detailed Description

Interface for panels that support refresh operations.

Author

Shamar Pennant

[IRefreshablePanel](#) is a mixin interface that enables UI panels to receive refresh notifications when they become visible or when data changes. Implementing panels can update their display to reflect current application state whenever Refresh() is called.

Definition in file [IRefreshablePanel.hpp](#).

7.154 IRefreshablePanel.hpp

[Go to the documentation of this file.](#)

```

00001
00011
00012 #pragma once
00013
00021 class IRefreshablePanel {
00022 public:
00023     virtual ~IRefreshablePanel() = default;
00024
00029     virtual void Refresh() = 0;
00030 };
  
```


7.155.2.2 MakeField()

```
wxTextCtrl * MakeField (
    wxWindow * parent,
    const wxString & hint,
    bool password = false) [static]
```

Definition at line 29 of file [LoginPanel.cpp](#).

7.156 LoginPanel.cpp

[Go to the documentation of this file.](#)

```
00001
00007
00008 // =====
00009 // LoginPanel.cpp - Sign In / Register screen implementation
00010 // =====
00011
00012 #include "LoginPanel.hpp"
00013
00014 #include <wx/dcbuffer.h>
00015 #include <wx/graphics.h>
00016 #include <wx/simplebook.h>
00017 #include <wx/statline.h>
00018 #include <unordered_map>
00019
00020 #include "UIColors.hpp"
00021 #include "ButtonsConfig.hpp"
00022 #include "ResourcePathUtils.hpp"
00023 #include "OperationProgressDialog.hpp"
00024 #include "../AppLayer/AppController.hpp"
00025 #include "../AppLayer/AppState.hpp"
00026 #include "../AppLayer/EventRouter.hpp"
00027 #include "../ModelLayer/User.hpp"
00028
00029 static wxTextCtrl* MakeField(wxWindow* parent,
00030                             const wxString& hint,
00031                             bool password = false)
00032 {
00033     long style = wxTE_PROCESS_ENTER | (password ? wxTE_PASSWORD : 0);
00034     auto* ctrl = new wxTextCtrl(parent, wxID_ANY, "", wxDefaultPosition,
00035                                wxSize(-1, 36), style);
00036     ctrl->SetHint(hint);
00037     ctrl->SetBackgroundColour(UIColors::SurfaceRaised());
00038     ctrl->SetForegroundColour(UIColors::TextPrimary());
00039     return ctrl;
00040 }
00041
00042 static wxStaticText* MakeError(wxWindow* parent)
00043 {
00044     auto* lbl = new wxStaticText(parent, wxID_ANY, "",
00045                                 wxDefaultPosition, wxDefaultSize,
00046                                 wxALIGN_CENTER_HORIZONTAL);
00047     lbl->SetForegroundColour(UIColors::Danger());
00048     lbl->Hide();
00049     return lbl;
00050 }
00051
00052 // -- Construction -----
00053
00054 LoginPanel::LoginPanel(wxWindow* parent, AppController& controller, NavigateFn nav,
00055                       const wxImage& bgImage)
00056 : wxPanel(parent, wxID_ANY)
00057 , m_controller(controller)
00058 , m_nav(std::move(nav))
00059 , m_bgImage(bgImage)
00060 {
00061     SetBackgroundStyle(wxBG_STYLE_PAINT);
00062     Bind(wxEVT_PAINT, &LoginPanel::OnPaint, this);
00063
00064     // -- Outer centering sizer -----
00065     auto* outer = new wxBoxSizer(wxVERTICAL);
00066
00067     // -- YTB LND title -----
00068     auto* titleLabel = new wxStaticText(this, wxID_ANY, "YTB LND",
00069                                         wxDefaultPosition, wxDefaultSize,
```

```

00070                                     wxALIGN_CENTRE_HORIZONTAL);
00071     wxFont tf = titleLabel->GetFont();
00072     tf.SetPointSize(42);
00073     tf.SetWeight(wxFONTWEIGHT_BOLD);
00074     titleLabel->SetFont(tf);
00075     titleLabel->SetForegroundColour(UIColors::Accent());
00076
00077     auto* subtitle = new wxStaticText(this, wxID_ANY, "Your blend, your music.",
00078                                     wxDefaultPosition, wxDefaultSize,
00079                                     wxALIGN_CENTRE_HORIZONTAL);
00080     subtitle->SetForegroundColour(UIColors::TextSecondary());
00081
00082     outer->AddStretchSpacer(2);
00083     outer->Add(titleLabel, 0, wxALIGN_CENTER | wxBOTTOM, 4);
00084     outer->Add(subtitle, 0, wxALIGN_CENTER | wxBOTTOM, 32);
00085
00086     // -- Form card -----
00087     auto* card = new wxPanel(this, wxID_ANY);
00088     card->SetBackgroundColour(UIColors::Surface());
00089     card->SetMinSize(wxSize(400, -1));
00090     auto* cardSizer = new wxBoxSizer(wxVERTICAL);
00091
00092     // Tab strip
00093     auto* tabRow = new wxBoxSizer(wxHORIZONTAL);
00094     m_signinTab = new wxButton(card, wxID_ANY, "Sign In",
00095                               wxDefaultPosition, wxDefaultSize);
00096     m_registerTab = new wxButton(card, wxID_ANY, "Register",
00097                                 wxDefaultPosition, wxDefaultSize);
00098     for (auto* btn : {m_signinTab, m_registerTab}) {
00099         UIButtons::ApplySizeBounds(btn, ButtonType::FormTab);
00100         btn->SetBackgroundColour(UIColors::SurfaceRaised());
00101         btn->SetForegroundColour(UIColors::TextSecondary());
00102     }
00103     tabRow->Add(m_signinTab, 1, wxEXPAND);
00104     tabRow->Add(m_registerTab, 1, wxEXPAND);
00105     cardSizer->Add(tabRow, 0, wxEXPAND);
00106
00107     // Divider below tabs
00108     auto* div = new wxStaticLine(card, wxID_ANY);
00109     div->SetBackgroundColour(UIColors::Separator());
00110     cardSizer->Add(div, 0, wxEXPAND);
00111
00112     // Form pages
00113     m_formBook = new wxSimplebook(card, wxID_ANY);
00114     m_formBook->SetBackgroundColour(UIColors::Surface());
00115
00116     // Page 0: Sign In
00117     {
00118         auto* page = new wxPanel(m_formBook, wxID_ANY);
00119         page->SetBackgroundColour(UIColors::Surface());
00120         auto* sz = new wxBoxSizer(wxVERTICAL);
00121         BuildSignInForm(page, sz);
00122         page->SetSizer(sz);
00123         m_formBook->AddPage(page, "SignIn");
00124     }
00125
00126     // Page 1: Register
00127     {
00128         auto* page = new wxPanel(m_formBook, wxID_ANY);
00129         page->SetBackgroundColour(UIColors::Surface());
00130         auto* sz = new wxBoxSizer(wxVERTICAL);
00131         BuildRegisterForm(page, sz);
00132         page->SetSizer(sz);
00133         m_formBook->AddPage(page, "Register");
00134     }
00135
00136     cardSizer->Add(m_formBook, 0, wxEXPAND);
00137     card->SetSizer(cardSizer);
00138
00139     // Centre the card horizontally
00140     auto* hCentre = new wxBoxSizer(wxHORIZONTAL);
00141     hCentre->AddStretchSpacer(1);
00142     hCentre->Add(card, 0, wxEXPAND);
00143     hCentre->AddStretchSpacer(1);
00144
00145     outer->Add(hCentre, 0, wxEXPAND);
00146     outer->AddStretchSpacer(3);
00147     SetSizer(outer);
00148
00149     // -- Tab bindings -----
00150     m_signinTab->Bind(wxEVT_BUTTON, [this](wxCommandEvent&){ ShowTab(0); });
00151     m_registerTab->Bind(wxEVT_BUTTON, [this](wxCommandEvent&){ ShowTab(1); });
00152
00153     ShowTab(0); // start on Sign In
00154
00155     // Load the themed background and re-load on theme switch
00156     LoadThemedBackground();

```



```

00157     m_controller.getEventRouter().registerListener("theme_bg_login",
00158     [this](const EventPayload&) { LoadThemedBackground(); Refresh(); });
00159 }
00160
00161 // -- Form builders -----
00162
00163 void LoginPanel::BuildSignInForm(wxWindow* parent, wxSizer* sizer)
00164 {
00165     m_siUsername = MakeField(parent, "Username");
00166     m_siPassword = MakeField(parent, "Password", true);
00167     m_siError = MakeError(parent);
00168
00169     auto* btn = new wxButton(parent, wxID_ANY, "Sign In",
00170                             wxDefaultPosition, wxDefaultSize);
00171     UIButtons::ApplySizeBounds(btn, ButtonType::FormSubmit);
00172     btn->SetBackgroundColour(UIColors::Accent());
00173     btn->SetForegroundColour(UIColors::TextPrimary());
00174
00175     sizer->Add(m_siUsername, 0, wxEXPAND | wxCENTRE | wxALL, 8);
00176     sizer->Add(m_siPassword, 0, wxEXPAND | wxCENTRE | wxALL, 8);
00177     sizer->Add(m_siError, 0, wxEXPAND | wxTOP | wxBOTTOM, 8);
00178     sizer->Add(btn, 0, wxEXPAND | wxALL | wxCENTER, 20);
00179
00180     btn->Bind(wxEVT_BUTTON, &LoginPanel::OnSignIn, this);
00181     // Also allow Enter in password field to submit
00182     m_siPassword->Bind(wxEVT_TEXT_ENTER,
00183     [this](wxCommandEvent& e) { OnSignIn(e); });
00184 }
00185
00186 void LoginPanel::BuildRegisterForm(wxWindow* parent, wxSizer* sizer)
00187 {
00188     m_regUsername = MakeField(parent, "Username (this will be your login ID)");
00189     m_regEmail = MakeField(parent, "Email (optional)");
00190     m_regPassword = MakeField(parent, "Password", true);
00191     m_regConfirm = MakeField(parent, "Confirm password", true);
00192     m_regError = MakeError(parent);
00193
00194     auto* btn = new wxButton(parent, wxID_ANY, "Create Account",
00195                             wxDefaultPosition, wxDefaultSize);
00196     UIButtons::ApplySizeBounds(btn, ButtonType::FormSubmit);
00197     btn->SetBackgroundColour(UIColors::Accent());
00198     btn->SetForegroundColour(UIColors::TextPrimary());
00199
00200     sizer->Add(m_regUsername, 0, wxEXPAND | wxALL, 8);
00201     sizer->Add(m_regEmail, 0, wxEXPAND | wxALL, 8);
00202     sizer->Add(m_regPassword, 0, wxEXPAND | wxALL, 8);
00203     sizer->Add(m_regConfirm, 0, wxEXPAND | wxALL, 8);
00204     sizer->Add(m_regError, 0, wxEXPAND | wxTOP | wxBOTTOM, 8);
00205     sizer->Add(btn, 0, wxEXPAND | wxALL | wxCENTER, 20);
00206
00207     btn->Bind(wxEVT_BUTTON, &LoginPanel::OnRegister, this);
00208 }
00209
00210 // -- Tab switcher -----
00211
00212 void LoginPanel::ShowTab(int index)
00213 {
00214     m_formBook->SetSelection(index);
00215
00216     // Active tab: accent colour; inactive: muted
00217     auto styleActive = [] (wxButton* btn) {
00218         btn->SetBackgroundColour(UIColors::Accent());
00219         btn->SetForegroundColour(UIColors::TextPrimary());
00220     };
00221     auto styleInactive = [] (wxButton* btn) {
00222         btn->SetBackgroundColour(UIColors::SurfaceRaised());
00223         btn->SetForegroundColour(UIColors::TextSecondary());
00224     };
00225
00226     if (index == 0) { styleActive(m_signinTab); styleInactive(m_registerTab); }
00227     else { styleInactive(m_signinTab); styleActive(m_registerTab); }
00228
00229     m_siError->Hide();
00230     m_regError->Hide();
00231     Layout();
00232 }
00233
00234 // -- Event handlers -----
00235
00236 void LoginPanel::OnSignIn(wxCommandEvent& /*evt*/)
00237 {
00238     std::string username = m_siUsername->GetValue().Trim().ToString();
00239     std::string password = m_siPassword->GetValue().ToString();
00240
00241     if (username.empty() || password.empty()) {
00242         m_siError->SetLabel("Please enter your username and password.");
00243         m_siError->Show();
00244     }

```

```

00244         m_siError->GetParent()->Layout();
00245         Layout();
00246         return;
00247     }
00248
00249     OperationProgressDialog progress(this, "Signing In");
00250     progress.ShowModal();
00251     progress.UpdateProgress(0.02, "Preparing login...");
00252
00253     m_controller.setProgressReporter([&progress](double ratio, const std::string& message) {
00254         progress.UpdateProgress(ratio, wxString::FromUTF8(message));
00255     });
00256
00257     // Dispatch uses userID - username IS the userID in this app
00258     m_controller.getEventRouter().dispatch("login",
00259         {"userID", username}, {"password", password});
00260     m_controller.clearProgressReporter();
00261
00262     progress.UpdateProgress(1.0, "Login finished");
00263     progress.Destroy();
00264
00265     if (AppState::getInstance().getCurrentUser() == nullptr) {
00266         m_siError->SetLabel("Incorrect username or password.");
00267         m_siError->Show();
00268         m_siError->GetParent()->Layout();
00269         m_siPassword->Clear();
00270         Layout();
00271         return;
00272     }
00273
00274     m_siError->Hide();
00275     ProceedAfterLogin();
00276 }
00277
00278 void LoginPanel::OnRegister(wxCommandEvent& /*evt*/)
00279 {
00280     std::string username = m_regUsername->GetValue().Trim().ToString();
00281     std::string email = m_regEmail->GetValue().Trim().ToString();
00282     std::string password = m_regPassword->GetValue().ToString();
00283     std::string confirm = m_regConfirm->GetValue().ToString();
00284
00285     if (username.empty() || password.empty()) {
00286         m_regError->SetLabel("Username and password are required.");
00287         m_regError->Show(); Layout(); return;
00288     }
00289     if (password != confirm) {
00290         m_regError->SetLabel("Passwords do not match.");
00291         m_regError->Show(); Layout(); return;
00292     }
00293     if (password.size() < 6) {
00294         m_regError->SetLabel("Password must be at least 6 characters.");
00295         m_regError->Show(); Layout(); return;
00296     }
00297
00298     // userID == username in this app
00299     m_controller.getEventRouter().dispatch("register",
00300         {"userID", username},
00301         {"username", username},
00302         {"email", email},
00303         {"password", password});
00304
00305     if (AppState::getInstance().getCurrentUser() == nullptr) {
00306         m_regError->SetLabel("Account already exists. Try signing in instead.");
00307         m_regError->Show(); Layout(); return;
00308     }
00309
00310     m_regError->Hide();
00311     ProceedAfterLogin();
00312 }
00313
00314 // -- Post-login routing -----
00315
00316 void LoginPanel::ProceedAfterLogin()
00317 {
00318     User* user = AppState::getInstance().getCurrentUser();
00319     if (!user) return;
00320
00321     // If a blend was restored by handleLogin, go straight to the feed
00322     if (AppState::getInstance().getActiveBlend() != nullptr) {
00323         m_nav(Page::HOME);
00324         return;
00325     }
00326
00327     // If the user has no Watch Later data, send them to the upload screen.
00328     // Show an info dialog first if they were added to a blend by someone else.
00329     bool hasData = !user->getYouTubeData().getWatchLaterVideos().empty();
00330     if (!hasData) {

```

```

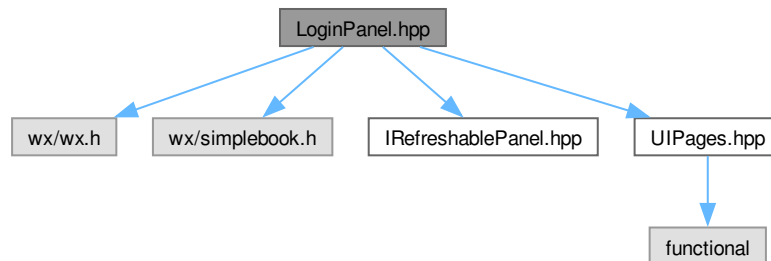
00331         std::string msg = AppState::getInstance().takePendingBlendMessage();
00332         if (!msg.empty())
00333             wxMessageBox(msg, "You're in a Blend!", wxOK | wxICON_INFORMATION, nullptr);
00334         m_nav(Page::DATA_INSTRUCTIONS);
00335         return;
00336     }
00337
00338     // Has data but no blend yet - let them create one
00339     m_nav(Page::BLEND_CREATION);
00340 }
00341
00342 // -- Reset -----
00343
00344 void LoginPanel::Refresh()
00345 {
00346     Reset();
00347 }
00348
00349 void LoginPanel::Reset()
00350 {
00351     m_siUsername->Clear();
00352     m_siPassword->Clear();
00353     m_regUsername->Clear();
00354     m_regEmail->Clear();
00355     m_regPassword->Clear();
00356     m_regConfirm->Clear();
00357     m_siError ->Hide();
00358     m_regError->Hide();
00359     ShowTab(0);
00360 }
00361
00362 // -- Themed background -----
00363
00364 void LoginPanel::LoadThemedBackground() {
00365     wxString themeName = UIColors::Current ? UIColors::Current->Name : wxString("Dark Mode");
00366
00367     static const std::unordered_map<wxString, wxString> bgMap = {
00368         {"Dark Mode",      "dark_purple_stripes.jpg"},
00369         {"Light Mode",     "checkered_wave_background.jpg"},
00370         {"Neon Mode",      "purple_neon_synth.jpg"},
00371         {"Cyber Amber",    "cyber_amber.jpg"},
00372         {"Nordic Frost",   "nordic_frost.jpg"},
00373         {"Forest Floor",   "forest_floor.jpg"},
00374         {"Synthwave 84",   "synthwave_84.jpg"},
00375     };
00376
00377     auto it = bgMap.find(themeName);
00378     wxString imageFile = (it != bgMap.end()) ? it->second : "dark_purple_stripes.jpg";
00379
00380     const wxString imagePath = UIResourcePaths::FindResourcePath(imageFile);
00381     if (!imagePath.empty()) {
00382         wxImage img(imagePath, wxBITMAP_TYPE_JPEG);
00383         if (img.IsOk()) {
00384             m_bgImage = img;
00385             return;
00386         }
00387     }
00388     m_bgImage = wxNullImage;
00389 }
00390
00391 // -- Background painting -----
00392
00393 void LoginPanel::OnPaint(wxPaintEvent&)
00394 {
00395     wxBufferedPaintDC dc(this);
00396     if (m_bgImage.IsOk()) {
00397         wxSize sz = GetClientSize();
00398         if (sz.x > 0 && sz.y > 0) {
00399             wxBitmap scaled(m_bgImage.Scale(sz.x, sz.y, wxIMAGE_QUALITY_HIGH));
00400             dc.DrawBitmap(scaled, 0, 0, false);
00401             return;
00402         }
00403     }
00404     dc.SetBackground(wxBrush(UIColors::Background()));
00405     dc.Clear();
00406 }

```

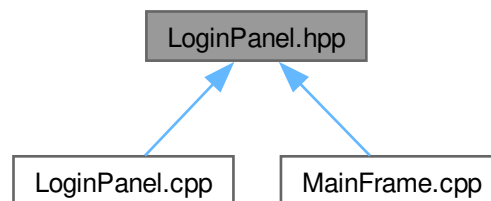
7.157 LoginPanel.hpp File Reference

Sign In / Register screen.

```
#include <wx/wx.h>
#include <wx/simplebook.h>
#include "IRefreshablePanel.hpp"
#include "UIPages.hpp"
Include dependency graph for LoginPanel.hpp:
```



This graph shows which files directly or indirectly include this file:



Classes

- class [LoginPanel](#)
[LoginPanel](#) class declaration.

7.157.1 Detailed Description

Sign In / Register screen.

Author

Jasmine Kumar
Shamar Pennant

The first screen the user sees. Contains a two-tab form card (Sign In and Register) rendered on top of the dark background with the YTBLND title.

7.157.1.1 SIGN IN

Dispatches "login" with {userID, password}. userID == username in this app. Checks [AppState::getCurrentUser\(\)](#) immediately after dispatch (the call is synchronous). On success, calls [ProceedAfterLogin\(\)](#). Displays an inline error label on failure (wrong credentials).

7.157.1.2 REGISTER

Validates locally (non-empty, passwords match, >=6 chars), then dispatches "register" with {userID, username, email, password}. On success, calls [ProceedAfterLogin\(\)](#). Displays an inline error label if the username is already taken.

7.157.1.3 POST-LOGIN ROUTING

[ProceedAfterLogin\(\)](#) checks whether the user's Watch Later list is empty. Empty -> DATA_INSTRUCTIONS (upload CSV first). Non-empty -> BLEND_CREATION (data already loaded, skip upload). TODO: This check queries the in-memory [YouTubeData](#) object. If data is stored in SQLite and lazy-loaded, the list may appear empty even for returning users. Verify that [SqliteDataManager](#) loads data into the [User](#) object before this check runs.

7.157.1.4 RESET

[Reset\(\)](#) is called by [MainFrame::NavigateTo\(LOGIN\)](#) (i.e. on logout). It clears all fields and returns to the Sign In tab.

Definition in file [LoginPanel.hpp](#).

7.158 LoginPanel.hpp

[Go to the documentation of this file.](#)

```
00001
00039
00040 #pragma once
00041 #include <wx/wx.h>
00042 #include <wx/simplebook.h>
00043 #include "IRefreshablePanel.hpp"
00044 #include "UIPages.hpp"
00045
00046 class AppController;
00047
00052 class LoginPanel : public wxPanel, public IRefreshablePanel {
00053 public:
00054     LoginPanel(wxWindow* parent, AppController& controller, NavigateFn nav,
00055               const wxImage& bgImage = wxNullImage);
00056
00057     void Refresh() override;
00058
00059     // Clear all fields and reset to Sign In tab. Called by MainFrame on logout.
00060     void Reset();
00061
00062 private:
00063     AppController& m_controller;
00064     NavigateFn m_nav;
00065
00066     // Tab toggle buttons
00067     wxButton* m_signinTab;
00068     wxButton* m_registerTab;
00069
00070     // Inner simplebook switches between sign-in and register forms
00071     wxSimplebook* m_formBook;
00072
```

```

00073 // Sign-in form widgets
00074 wxTextCtrl* m_siUsername;
00075 wxTextCtrl* m_siPassword;
00076 wxStaticText* m_siError;
00077
00078 // Register form widgets
00079 wxTextCtrl* m_regUsername;
00080 wxTextCtrl* m_regEmail;
00081 wxTextCtrl* m_regPassword;
00082 wxTextCtrl* m_regConfirm;
00083 wxStaticText* m_regError;
00084
00085 wxImage m_bgImage;
00086
00087 void BuildSignInForm (wxWindow* parent, wxSizer* sizer);
00088 void BuildRegisterForm(wxWindow* parent, wxSizer* sizer);
00089 void ShowTab(int index); // 0 = sign in, 1 = register
00090
00091 void OnSignIn (wxCommandEvent&);
00092 void OnRegister(wxCommandEvent&);
00093 void OnPaint (wxPaintEvent&);
00094
00095 void LoadThemedBackground();
00096
00097 // After a successful login, decide which page comes next.
00098 void ProceedAfterLogin();
00099
00100 };

```

7.159 MainFrame.cpp File Reference

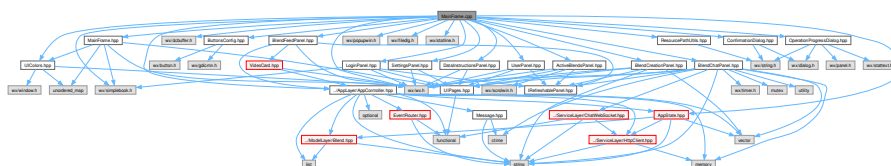
Implementation for [MainFrame](#).

```

#include "MainFrame.hpp"
#include <wx/wx.h>
#include <wx/simplebook.h>
#include <wx/dcbuffer.h>
#include "UIColors.hpp"
#include "ButtonsConfig.hpp"
#include "ResourcePathUtils.hpp"
#include "ConfirmationDialog.hpp"
#include <wx/popupwin.h>
#include <wx/filedlg.h>
#include <wx/statline.h>
#include "LoginPanel.hpp"
#include "DataInstructionsPanel.hpp"
#include "BlendFeedPanel.hpp"
#include "UserPanel.hpp"
#include "SettingsPanel.hpp"
#include "BlendCreationPanel.hpp"
#include "BlendChatPanel.hpp"
#include "ActiveBlendsPanel.hpp"
#include "OperationProgressDialog.hpp"
#include "../AppLayer/AppController.hpp"
#include "../AppLayer/AppState.hpp"

```

Include dependency graph for MainFrame.cpp:



Classes

- class [ImageBackgroundPanel](#)

7.159.1 Detailed Description

Implementation for [MainFrame](#).

Author

Jasmine Kumar
 Shamar Pennant
 Xavier Lusetti

Definition in file [MainFrame.cpp](#).

7.160 MainFrame.cpp

[Go to the documentation of this file.](#)

```

00001
00008
00009 #include "MainFrame.hpp"
00010 #include <wx/wx.h>
00011 #include <wx/simplebook.h>
00012 #include <wx/dcbuffer.h>
00013
00014 #include "UIColors.hpp"
00015 #include "ButtonsConfig.hpp"
00016 #include "ResourcePathUtils.hpp"
00017 #include "ConfirmationDialog.hpp"
00018
00019 #include <wx/popupwin.h>
00020 #include <wx/filedlg.h>
00021 #include <wx/statline.h>
00022 #include "LoginPanel.hpp"
00023 #include "DataInstructionsPanel.hpp"
00024 #include "BlendFeedPanel.hpp"
00025 #include "UserPanel.hpp"
00026 #include "SettingsPanel.hpp"
00027 #include "BlendCreationPanel.hpp"
00028 #include "BlendChatPanel.hpp"
00029 #include "ActiveBlendsPanel.hpp"
00030 #include "OperationProgressDialog.hpp"
00031 #include "../AppLayer/AppController.hpp"
00032 #include "../AppLayer/AppState.hpp"
00033
00034 void MainFrame::ApplyAppIcons()
00035 {
00036     const wxArrayString iconCandidates = {
00037         "ytblnd-framev2.png",
00038         "ytblnd-starv2.png",
00039         "ytblnd-star.png",
00040     };
00041
00042     wxIconBundle iconBundle;
00043     for (const auto& fileName : iconCandidates) {
00044         const wxString iconPath = UIResourcePaths::FindResourcePath(fileName);
00045         if (iconPath.empty()) {
00046             continue;
00047         }
00048
00049         wxIcon icon;
00050         if (icon.LoadFile(iconPath, wxBITMAP_TYPE_PNG) && icon.IsOk()) {
00051             iconBundle.AddIcon(icon);
00052         }
00053     }
00054
00055     if (!iconBundle.IsEmpty()) {
00056         SetIcons(iconBundle);
00057     }

```

```

00057     } else {
00058         wxLogWarning("App icon files not found in resources.");
00059     }
00060 }
00061
00062 // -- Background Panel -----
00063 class ImageBackgroundPanel : public wxPanel {
00064 public:
00065     ImageBackgroundPanel(wxWindow* parent,
00066                         const std::unordered_map<int, wxImage>& imageStore,
00067                         int key)
00068         : wxPanel(parent)
00069         , m_imageStore(imageStore)
00070         , m_key(key)
00071     {
00072         SetBackgroundStyle(wxBG_STYLE_PAINT);
00073         Bind(wxEVT_PAINT, &ImageBackgroundPanel::OnPaint, this);
00074     }
00075
00076 private:
00077     const std::unordered_map<int, wxImage>& m_imageStore;
00078     int m_key;
00079
00080     void OnPaint(wxPaintEvent&) {
00081         wxBufferedPaintDC dc(this);
00082         dc.Clear();
00083         auto it = m_imageStore.find(m_key);
00084         if (it == m_imageStore.end() || !it->second.IsOk()) {
00085             return;
00086         }
00087
00088         wxSize sz = GetClientSize();
00089         if (sz.x <= 0 || sz.y <= 0) {
00090             return;
00091         }
00092
00093         wxBitmap scaled(it->second.Scale(sz.x, sz.y, wxIMAGE_QUALITY_HIGH));
00094         dc.DrawBitmap(scaled, 0, 0, true);
00095     }
00096 };
00097
00098 // -- Construction -----
00099 MainFrame::MainFrame(AppController& controller)
00100     : wxFrame(nullptr, wxID_ANY, "YTBLND",
00101               wxDefaultPosition, wxSize(1280, 800),
00102               wxDEFAULT_FRAME_STYLE | wxMAXIMIZE),
00103     controller(controller),
00104     refreshBtn(nullptr),
00105     refreshInProgress(false)
00106 {
00107     // Resolve the path to the theme.txt containing the UI colour themes
00108     UIColors::LoadThemesFromFile(UIResourcePaths::FindResourcePath("theme.txt"));
00109
00110     ApplyAppIcons();
00111
00112     // resolve the path to the background image and load it as a wxImage
00113     const wxString bgPath = UIResourcePaths::FindResourcePath("checked_wave_background.jpg");
00114     if (bgPath.empty() || !LoadImage(BG_MAIN, bgPath)) {
00115         wxLogWarning("Background image not found. Checked ../resources and resources.");
00116     }
00117
00118     // create the outer container panel with the app background colour
00119     auto* bgPanel = new wxPanel(this);
00120     bgPanel->SetBackgroundColour(UIColors::Background());
00121
00122     // placing our new background panel in the book
00123     book = new wxSimplebook(bgPanel, wxID_ANY, wxDefaultPosition, wxDefaultSize, wxBORDER_NONE);
00124
00125     NavigateFn nav = [this](Page p) { NavigateTo(p); };
00126
00127     // --- Initialize pages and wrap each in a panel that paints the background image ---
00128
00129     // the login panel - paints its own background image
00130     loginPanel = new LoginPanel(book, controller, nav, images[BG_MAIN]);
00131     book->AddPage(loginPanel, "Login");
00132
00133     auto* dataPageBg = new wxPanel(book);
00134     dataPageBg->SetBackgroundColour(UIColors::Background());
00135     dataInstrPanel = new DataInstructionsPanel(dataPageBg, controller, nav);
00136     {
00137         auto* s = new wxBoxSizer(wxVERTICAL);
00138         s->Add(dataInstrPanel, 1, wxEXPAND);
00139         dataPageBg->SetSizer(s);
00140     }
00141     book->AddPage(dataPageBg, "DataInstructions");
00142
00143     auto* creationPageBg = new wxPanel(book);

```



```

00144     creationPageBg->SetBackgroundColour(UIColors::Background());
00145     creationPanel = new BlendCreationPanel(creationPageBg, controller, nav);
00146     {
00147         auto* s = new wxBoxSizer(wxVERTICAL);
00148         s->Add(creationPanel, 1, wxEXPAND);
00149         creationPageBg->SetSizer(s);
00150     }
00151     book->AddPage(creationPageBg, "BlendCreation");
00152
00153     auto* homePageBg = new wxPanel(book);
00154     homePageBg->SetBackgroundColour(UIColors::Background());
00155     feedPanel = new BlendFeedPanel(homePageBg, controller);
00156     {
00157         auto* homePage = BuildHomePage(homePageBg);
00158         auto* s = new wxBoxSizer(wxVERTICAL);
00159         s->Add(homePage, 1, wxEXPAND);
00160         homePageBg->SetSizer(s);
00161     }
00162     book->AddPage(homePageBg, "Home");
00163
00164     auto* userPageBg = new wxPanel(book);
00165     userPageBg->SetBackgroundColour(UIColors::Background());
00166     userPanel = new UserPanel(userPageBg, controller, nav);
00167     {
00168         auto* s = new wxBoxSizer(wxVERTICAL);
00169         s->Add(userPanel, 1, wxEXPAND);
00170         userPageBg->SetSizer(s);
00171     }
00172     book->AddPage(userPageBg, "User");
00173
00174     auto* settingsPageBg = new wxPanel(book);
00175     settingsPageBg->SetBackgroundColour(UIColors::Background());
00176     settingsPanel = new SettingsPanel(settingsPageBg, controller, nav);
00177     {
00178         auto* s = new wxBoxSizer(wxVERTICAL);
00179         s->Add(settingsPanel, 1, wxEXPAND);
00180         settingsPageBg->SetSizer(s);
00181     }
00182     book->AddPage(settingsPageBg, "Settings");
00183
00184     auto* chatPageBg = new wxPanel(book);
00185     chatPageBg->SetBackgroundColour(UIColors::Background());
00186     chatPanel = new BlendChatPanel(chatPageBg, controller, nav);
00187     {
00188         auto* s = new wxBoxSizer(wxVERTICAL);
00189         s->Add(chatPanel, 1, wxEXPAND);
00190         chatPageBg->SetSizer(s);
00191     }
00192     book->AddPage(chatPageBg, "BlendChat");
00193
00194     auto* activeBlendsPageBg = new wxPanel(book);
00195     activeBlendsPageBg->SetBackgroundColour(UIColors::Background());
00196     activeBlendsPanel = new ActiveBlendsPanel(activeBlendsPageBg, controller, nav);
00197     {
00198         auto* s = new wxBoxSizer(wxVERTICAL);
00199         s->Add(activeBlendsPanel, 1, wxEXPAND);
00200         activeBlendsPageBg->SetSizer(s);
00201     }
00202     book->AddPage(activeBlendsPageBg, "ActiveBlends");
00203
00204     // 6. Layout: Book expands inside BackgroundPanel
00205     auto* rootSizer = new wxBoxSizer(wxVERTICAL);
00206     rootSizer->Add(book, 1, wxEXPAND);
00207     bgPanel->SetSizer(rootSizer);
00208
00209     // Layout: BackgroundPanel expands inside the Frame
00210     auto* frameSizer = new wxBoxSizer(wxVERTICAL);
00211     frameSizer->Add(bgPanel, 1, wxEXPAND);
00212     SetSizer(frameSizer);
00213
00214     // Theme change listener
00215     controller.getEventRouter().registerListener("theme_updated",
00216         [this, &controller](const EventPayload& /*payload*/) {
00217             UIColors::UpdateUI(this);
00218             ReloadThemedIcons();
00219             controller.getEventRouter().dispatch("theme_bg_login", {});
00220         });
00221
00222     Maximize(true);
00223     book->SetSelection(static_cast<int>(Page::HOME));
00224     Layout();
00225     book->SetSelection(static_cast<int>(Page::LOGIN));
00226 }
00227
00228 // -- Navigation -----
00229 void MainFrame::NavigateTo(Page page) {
00230     if (page == Page::LOGIN) loginPanel->Refresh();

```

```

00231     if (page == Page::BLEND_CREATION)    creationPanel->Refresh();
00232     if (page == Page::HOME) {
00233         feedPanel->Refresh();
00234         UpdateBlendIndicatorLabel();
00235     }
00236     if (page == Page::USER)                userPanel->Refresh();
00237     if (page == Page::BLEND_CHAT)          chatPanel->Refresh();
00238     if (page == Page::ACTIVE_BLENDS)       activeBlendsPanel->Refresh();
00239     book->SetSelection(static_cast<int>(page));
00240 }
00241
00242 void MainFrame::TriggerFeedRefresh() {
00243     if (refreshInProgress) {
00244         return;
00245     }
00246     refreshInProgress = true;
00247     if (refreshBtn != nullptr) {
00248         refreshBtn->Disable();
00249     }
00250 }
00251
00252 OperationProgressDialog progress(this, "Refreshing Feed");
00253 progress.ShowModal();
00254 progress.UpdateProgress(0.08, "Preparing refresh...");
00255
00256 controller.setProgressReporter([&progress](double ratio, const std::string& message) {
00257     progress.UpdateProgress(ratio, wxString::FromUTF8(message));
00258 });
00259
00260 // NextPage may either advance the local page offset immediately or trigger
00261 // controller-level refresh work when a new blend sample is required.
00262 progress.UpdateProgress(0.2, "Updating feed...");
00263 try {
00264     feedPanel->NextPage();
00265     progress.UpdateProgress(1.0, "Feed refreshed");
00266 } catch (...) {
00267     controller.clearProgressReporter();
00268     if (refreshBtn != nullptr) {
00269         refreshBtn->Enable();
00270     }
00271     refreshInProgress = false;
00272     throw;
00273 }
00274
00275 controller.clearProgressReporter();
00276 progress.Destroy();
00277 if (refreshBtn != nullptr) {
00278     refreshBtn->Enable();
00279 }
00280 refreshInProgress = false;
00281 }
00282
00283 void MainFrame::UpdateBlendIndicatorLabel() {
00284     if (blendIndicatorLabel == nullptr) {
00285         return;
00286     }
00287
00288     const std::string title = controller.get_current_blend_title();
00289     const wxString uiText = title.empty()
00290         ? wxString("No active blend selected")
00291         : wxString::FromUTF8(title);
00292
00293     blendIndicatorLabel->SetLabel(uiText);
00294     blendIndicatorLabel->SetForegroundColour(UIColors::Accent());
00295     blendIndicatorLabel->Wrap(-1);
00296     blendIndicatorLabel->Refresh();
00297 }
00298
00299 // -- Icon helpers -----
00300
00301 wxBitmap MainFrame::LoadThemedIcon(const wxString& name, int size) {
00302     wxString folder = UIResourcePaths::GetIconFolder();
00303     wxString path = UIResourcePaths::FindResourcePath("icons/" + folder + "/" + name + ".png");
00304     if (!path.empty()) {
00305         wxImage img(path, wxBITMAP_TYPE_PNG);
00306         if (img.IsOk()) {
00307             img.Rescale(size, size, wxIMAGE_QUALITY_BILINEAR);
00308             return wxBitmap(img);
00309         }
00310     }
00311     return wxNullBitmap;
00312 }
00313
00314 wxButton* MainFrame::MakeIconButton(wxWindow* parent, const wxString& iconName,
00315                                     IconBase base, int iconSize) {
00316     wxColour initBg = (base == IconBase::Surface) ? UIColors::Surface() : UIColors::Background();
00317     auto* btn = new wxButton(parent, wxID_ANY, wxEmptyString,

```

```

00318                                     wxDefaultPosition, wxSize(iconSize + 16, iconSize + 12),
00319                                     wxBORDER_NONE);
00320 btn->SetBackgroundColour(initBg);
00321 wxBitmap bmp = LoadThemedIcon(iconName, iconSize);
00322 if (bmp.IsOk()) btn->SetBitmap(bmp);
00323
00324 // Capitalize first letter for display
00325 wxString tipText = iconName;
00326 if (!tipText.empty())
00327     tipText[0] = wxToUpper(tipText[0]);
00328
00329 // Shared tooltip popup pointer (one per button, created on first hover)
00330 auto* tipPtr = new wxPopupWindow*{nullptr};
00331 auto hideTip = [tipPtr]() {
00332     if (*tipPtr) {
00333         (*tipPtr)->Hide();
00334         (*tipPtr)->Destroy();
00335         *tipPtr = nullptr;
00336     }
00337 };
00338
00339 btn->Bind(wxEVT_LEFT_DOWN, [hideTip](wxMouseEvent& e) {
00340     hideTip();
00341     e.Skip();
00342 });
00343 btn->Bind(wxEVT_KILL_FOCUS, [hideTip](wxFocusEvent& e) {
00344     hideTip();
00345     e.Skip();
00346 });
00347 btn->Bind(wxEVT_DESTROY, [tipPtr, hideTip](wxWindowDestroyEvent& e) {
00348     hideTip();
00349     delete tipPtr;
00350     e.Skip();
00351 });
00352
00353 btn->Bind(wxEVT_ENTER_WINDOW, [btn, tipText, tipPtr](wxMouseEvent& e) {
00354     btn->SetBackgroundColour(UIColors::SurfaceRaised());
00355     btn->Refresh();
00356
00357     // Create themed tooltip near the cursor
00358     if (!*tipPtr) {
00359         auto* tip = new wxPopupWindow(btn->GetParent());
00360         auto* tipPanel = new wxPanel(tip, wxID_ANY);
00361         tipPanel->SetBackgroundColour(UIColors::SurfaceRaised());
00362         auto* lbl = new wxStaticText(tipPanel, wxID_ANY, tipText);
00363         lbl->SetForegroundColour(UIColors::AccentHover());
00364         lbl->SetBackgroundColour(UIColors::SurfaceRaised());
00365         auto* sz = new wxBoxSizer(wxHORIZONTAL);
00366         sz->Add(lbl, 0, wxALL, 6);
00367         tipPanel->SetSizer(sz);
00368         tipPanel->Fit();
00369         tip->SetClientSize(tipPanel->GetBestSize());
00370         *tipPtr = tip;
00371     }
00372     // Position centered below the button
00373     wxPoint btnScreen = btn->ClientToScreen(wxPoint(0, btn->GetSize().GetHeight()));
00374     int tipW = (*tipPtr)->GetSize().GetWidth();
00375     int btnW = btn->GetSize().GetWidth();
00376     (*tipPtr)->SetPosition(wxPoint(btnScreen.x + (btnW - tipW) / 2, btnScreen.y + 4));
00377     (*tipPtr)->Show();
00378     e.Skip();
00379 });
00380 btn->Bind(wxEVT_LEAVE_WINDOW, [btn, base, tipPtr](wxMouseEvent& e) {
00381     btn->SetBackgroundColour(
00382         (base == IconBase::Surface) ? UIColors::Surface() : UIColors::Background());
00383     btn->Refresh();
00384     if (*tipPtr) {
00385         (*tipPtr)->Hide();
00386         (*tipPtr)->Destroy();
00387         *tipPtr = nullptr;
00388     }
00389     e.Skip();
00390 });
00391 return btn;
00392 }
00393
00394 // -- Build Home Page -----
00395 wxPanel* MainFrame::BuildHomePage(wxWindow* parent) {
00396     auto* page = new wxPanel(parent, wxID_ANY);
00397     auto* vbox = new wxBoxSizer(wxVERTICAL);
00398
00399     // -- Row 1: Top ribbon (Surface bg, left-aligned: settings + user icons) --
00400     auto* ribbon = new wxPanel(page, wxID_ANY);
00401     ribbon->SetBackgroundColour(UIColors::Surface());
00402     ribbon->SetMinSize(wxSize(-1, 32));
00403     auto* ribbonSizer = new wxBoxSizer(wxHORIZONTAL);
00404

```

```

00405 settingsIconBtn = MakeIconButton(ribbon, "settings", IconBase::Surface);
00406 userIconBtn     = MakeIconButton(ribbon, "user",      IconBase::Surface);
00407 auto* settingsBtn = settingsIconBtn;
00408 auto* userBtn     = userIconBtn;
00409
00410 settingsBtn->Bind(wxEVT_BUTTON, [this, settingsBtn](wxCommandEvent&){
00411     CallAfter([this, settingsBtn]() {
00412         if (settingsBtn->IsShownOnScreen()) {
00413             ShowSettingsDropdown(settingsBtn);
00414         }
00415     });
00416 });
00417 userBtn     ->Bind(wxEVT_BUTTON, [this, userBtn](wxCommandEvent&){
00418     CallAfter([this, userBtn]() {
00419         if (userBtn->IsShownOnScreen()) {
00420             ShowUserDropdown(userBtn);
00421         }
00422     });
00423 });
00424
00425 ribbonSizer->Add(settingsBtn, 0, wxALIGN_CENTER_VERTICAL | wxLEFT, 8);
00426 ribbonSizer->Add(userBtn,     0, wxALIGN_CENTER_VERTICAL | wxLEFT, 4);
00427 ribbonSizer->AddStretchSpacer(1);
00428 ribbon->SetSizer(ribbonSizer);
00429
00430 // -- Row 2: Title row (YTBLND centered-left, blends + chat icons right) ---
00431 auto* titlePanel = new wxPanel(page, wxID_ANY);
00432 auto* titleGrid = new wxGridSizer(1, 3, 0, 0);
00433
00434 auto* titleLabel = new wxStaticText(titlePanel, wxID_ANY, "YTBLND");
00435 wxFont titleFont = titleLabel->GetFont();
00436 titleFont.SetPointSize(38);
00437 titleFont.SetWeight(wxFONTWEIGHT_BOLD);
00438 titleLabel->SetFont(titleFont);
00439 titleLabel->SetForegroundColour(UIColors::Accent());
00440
00441 auto* leftSlot = new wxPanel(titlePanel, wxID_ANY);
00442 leftSlot->SetBackgroundColour(UIColors::Background());
00443
00444 auto* rightSlot = new wxPanel(titlePanel, wxID_ANY);
00445 rightSlot->SetBackgroundColour(UIColors::Background());
00446
00447 blendsIconBtn = MakeIconButton(rightSlot, "blends", IconBase::Background, 32);
00448 chatIconBtn   = MakeIconButton(rightSlot, "chat",   IconBase::Background, 32);
00449 auto* blendsBtn = blendsIconBtn;
00450 auto* chatBtn   = chatIconBtn;
00451
00452 blendsBtn->Bind(wxEVT_BUTTON, [this](wxCommandEvent&){ NavigateTo(Page::ACTIVE_BLENDS); });
00453 chatBtn   ->Bind(wxEVT_BUTTON, [this](wxCommandEvent&){
00454     if (AppState::getInstance().getActiveBlend() != nullptr)
00455         NavigateTo(Page::BLEND_CHAT);
00456     else
00457         NavigateTo(Page::BLEND_CREATION);
00458 });
00459
00460 // Keep actions in the right third and right-align within that column.
00461 auto* rightControls = new wxBoxSizer(wxHORIZONTAL);
00462 rightControls->AddStretchSpacer(1);
00463 rightControls->Add(blendsBtn, 0, wxALIGN_CENTER_VERTICAL | wxRIGHT, 12);
00464 rightControls->Add(chatBtn,   0, wxALIGN_CENTER_VERTICAL | wxRIGHT, 20);
00465 rightSlot->SetSizer(rightControls);
00466
00467 // Equal-width 3-column grid: empty left slot, centered title, right actions.
00468 // This keeps the title centered in the full frame while scaling naturally.
00469 titleGrid->Add(leftSlot, 1, wxEXPAND);
00470 titleGrid->Add(titleLabel, 0, wxALIGN_CENTER);
00471 titleGrid->Add(rightSlot, 1, wxEXPAND);
00472 titlePanel->SetSizer(titleGrid);
00473
00474 // -- Feed grid (unchanged) -----
00475 feedPanel->Reparent(page);
00476
00477 // -- Refresh icon (centered, no text, no container) -----
00478 auto* refreshPanel = new wxPanel(page, wxID_ANY);
00479 auto* rbox = new wxBoxSizer(wxHORIZONTAL);
00480 refreshBtn = MakeIconButton(refreshPanel, "refresh", IconBase::Background, 40);
00481 refreshBtn->Bind(wxEVT_BUTTON, [this](wxCommandEvent&){ TriggerFeedRefresh(); });
00482 rbox->AddStretchSpacer(1);
00483 rbox->Add(refreshBtn, 0, wxALL, 8);
00484 rbox->AddStretchSpacer(1);
00485 refreshPanel->SetSizer(rbox);
00486
00487 // setting up the label to indicate the current blend
00488 wxPanel* bi_panel = new wxPanel(page, wxID_ANY);
00489
00490 bi_panel->SetBackgroundColour(UIColors::Background());
00491 blendIndicatorLabel = new wxStaticText(bi_panel, wxID_ANY, wxEmptyString);

```

```

00492     wxFont indicator_font = blendIndicatorLabel->GetFont();
00493     indicator_font.SetPointSize(16);
00494     indicator_font.SetWeight(wxFONTWEIGHT_LIGHT);
00495     blendIndicatorLabel->SetFont(indicator_font);
00496     blendIndicatorLabel->SetForegroundColour(UIColors::TextPrimary());
00497     UpdateBlendIndicatorLabel();
00498
00499     auto* BI_lbl_box = new wxBoxSizer(wxHORIZONTAL);
00500     BI_lbl_box->AddStretchSpacer(1);
00501     BI_lbl_box->Add(blendIndicatorLabel);
00502     BI_lbl_box->AddStretchSpacer(1);
00503     bi_panel->SetSizer(BI_lbl_box);
00504
00505     vbox->Add(ribbon, 0, wxEXPAND);
00506     vbox->Add(titlePanel, 0, wxEXPAND);
00507     vbox->Add(bi_panel, 0, wxEXPAND | wxCENTER);
00508     vbox->Add(feedPanel, 1, wxEXPAND | wxALL, 16);
00509     vbox->Add(refreshPanel, 0, wxEXPAND);
00510     vbox->AddSpacer(24);
00511     page->SetSizer(vbox);
00512
00513     return page;
00514 }
00515
00516 void MainFrame::ReloadThemedIcons() {
00517     auto reload = [](wxButton* btn, const wxString& name, int size) {
00518         if (!btn) return;
00519         wxBitmap bmp = MainFrame::LoadThemedIcon(name, size);
00520         if (bmp.IsOk()) btn->SetBitmap(bmp);
00521         btn->Refresh();
00522     };
00523     reload(settingsIconBtn, "settings", 24);
00524     reload(userIconBtn, "user", 24);
00525     reload(blendsIconBtn, "blends", 32);
00526     reload(chatIconBtn, "chat", 32);
00527     reload(refreshBtn, "refresh", 40);
00528 }
00529
00530 // -- Helper: create a hoverable row for dropdown menus -----
00531 namespace {
00532 wxPanel* MakeDropDownRow(wxWindow* parent, const wxString& label,
00533     const wxColour& textColor = UIColors::TextPrimary()) {
00534     auto* row = new wxPanel(parent, wxID_ANY);
00535     row->SetBackgroundColour(UIColors::Surface());
00536     row->SetMinSize(wxSize(220, 40));
00537     auto* sizer = new wxBoxSizer(wxHORIZONTAL);
00538     auto* lbl = new wxStaticText(row, wxID_ANY, label);
00539     lbl->SetForegroundColour(textColor);
00540     lbl->SetBackgroundColour(UIColors::Surface());
00541     sizer->Add(lbl, 0, wxALIGN_CENTER_VERTICAL | wxLEFT | wxRIGHT, 16);
00542     row->SetSizer(sizer);
00543
00544     for (wxWindow* w : {static_cast<wxWindow*>(row), static_cast<wxWindow*>(lbl)}) {
00545         w->Bind(wxEVT_ENTER_WINDOW, [row, lbl](wxMouseEvent& e) {
00546             row->SetBackgroundColour(UIColors::SurfaceRaised());
00547             lbl->SetBackgroundColour(UIColors::SurfaceRaised());
00548             row->Refresh(); e.Skip();
00549         });
00550         w->Bind(wxEVT_LEAVE_WINDOW, [row, lbl](wxMouseEvent& e) {
00551             row->SetBackgroundColour(UIColors::Surface());
00552             lbl->SetBackgroundColour(UIColors::Surface());
00553             row->Refresh(); e.Skip();
00554         });
00555     }
00556     return row;
00557 }
00558 }
00559
00560 // -- Settings dropdown -----
00561
00562 void MainFrame::ShowSettingsDropdown(wxWindow* anchor) {
00563     auto* popup = new wxPopupTransientWindow(this, wxBORDER_SIMPLE | wxPU_CONTAINS_CONTROLS);
00564     auto* panel = new wxPanel(popup, wxID_ANY);
00565     panel->SetBackgroundColour(UIColors::Surface());
00566     auto* sizer = new wxBoxSizer(wxVERTICAL);
00567
00568     // "Theme >" row -- toggles the theme list below
00569     auto* themeRow = MakeDropDownRow(panel, "Theme >");
00570     sizer->Add(themeRow, 0, wxEXPAND);
00571
00572     // Theme sub-items (initially hidden)
00573     auto* themeSub = new wxPanel(panel, wxID_ANY);
00574     themeSub->SetBackgroundColour(UIColors::Surface());
00575     auto* themeSubSizer = new wxBoxSizer(wxVERTICAL);
00576
00577     for (const wxString& name : UIColors::ThemeOrder) {
00578         auto* row = new wxPanel(themeSub, wxID_ANY);

```

```

00579         row->SetBackgroundColour(UIColors::Surface());
00580         row->SetMinSize(wxSize(220, 36));
00581         auto* rowSz = new wxBoxSizer(wxHORIZONTAL);
00582         auto* lbl = new wxStaticText(row, wxID_ANY, " " + name);
00583         lbl->SetForegroundColour(UIColors::TextPrimary());
00584         lbl->SetBackgroundColour(UIColors::Surface());
00585         rowSz->Add(lbl, 0, wxALIGN_CENTER_VERTICAL | wxLEFT | wxRIGHT, 16);
00586         row->SetSizer(rowSz);
00587
00588         // Hover
00589         for (wxWindow* w : {static_cast<wxWindow*>(row), static_cast<wxWindow*>(lbl)}) {
00590             w->Bind(wxEVT_ENTER_WINDOW, [row, lbl](wxMouseEvent& e) {
00591                 row->SetBackgroundColour(UIColors::SurfaceRaised());
00592                 lbl->SetBackgroundColour(UIColors::SurfaceRaised());
00593                 row->Refresh(); e.Skip();
00594             });
00595             w->Bind(wxEVT_LEAVE_WINDOW, [row, lbl](wxMouseEvent& e) {
00596                 row->SetBackgroundColour(UIColors::Surface());
00597                 lbl->SetBackgroundColour(UIColors::Surface());
00598                 row->Refresh(); e.Skip();
00599             });
00600         }
00601
00602         // Click -- apply theme
00603         wxString capName = name;
00604         auto themeAction = [this, popup, capName](wxMouseEvent&) {
00605             UIColors::SetTheme(capName);
00606             wxWindow* top = wxGetTopLevelParent(this);
00607             UIColors::UpdateUI(top);
00608             controller.getEventRouter().dispatch("theme_updated", {});
00609             popup->Dismiss();
00610         };
00611         row->Bind(wxEVT_LEFT_UP, themeAction);
00612         lbl->Bind(wxEVT_LEFT_UP, themeAction);
00613
00614         themeSubSizer->Add(row, 0, wxEXPAND);
00615     }
00616     themeSub->SetSizer(themeSubSizer);
00617     themeSub->Hide();
00618     sizer->Add(themeSub, 0, wxEXPAND);
00619
00620     // Toggle theme sub-list on click of "Theme >" row
00621     auto toggleThemes = [themeSub, panel, popup](wxMouseEvent&) {
00622         themeSub->Show(!themeSub->IsShown());
00623         panel->Layout();
00624         panel->Fit();
00625         popup->SetClientSize(panel->GetBestSize());
00626         popup->Layout();
00627     };
00628     themeRow->Bind(wxEVT_LEFT_UP, toggleThemes);
00629     for (wxWindow* w : themeRow->GetChildren())
00630         w->Bind(wxEVT_LEFT_UP, toggleThemes);
00631
00632     panel->SetSizer(sizer);
00633     panel->Layout();
00634     panel->Fit();
00635     popup->SetClientSize(panel->GetBestSize());
00636     popup->Layout();
00637
00638     popup->Position(anchor->ClientToScreen(wxPoint(0, 0)), wxSize(0, anchor->GetSize().GetHeight()));
00639     popup->Popup();
00640 }
00641
00642 // -- User dropdown -----
00643
00644 void MainFrame::ShowUserDropdown(wxWindow* anchor) {
00645     auto* popup = new wxPopupTransientWindow(this, wxBORDER_SIMPLE | wxPU_CONTAINS_CONTROLS);
00646     auto* panel = new wxPanel(popup, wxID_ANY);
00647     panel->SetBackgroundColour(UIColors::Surface());
00648     auto* sizer = new wxBoxSizer(wxVERTICAL);
00649
00650     // Username display (centered, non-clickable)
00651     User* user = AppState::getInstance().getCurrentUser();
00652     wxString username = user ? wxString::FromUTF8(user->getUsername()) : "Not logged in";
00653     auto* nameLabel = new wxStaticText(panel, wxID_ANY, username,
00654                                     wxDefaultPosition, wxDefaultSize,
00655                                     wxALIGN_CENTRE_HORIZONTAL);
00656     nameLabel->SetForegroundColour(UIColors::TextPrimary());
00657     nameLabel->SetBackgroundColour(UIColors::Surface());
00658     nameLabel->SetMinSize(wxSize(220, -1));
00659     wxFont nf = nameLabel->GetFont();
00660     nf.SetWeight(wxFONTWEIGHT_BOLD);
00661     nameLabel->SetFont(nf);
00662     sizer->Add(nameLabel, 0, wxEXPAND | wxALL, 10);
00663
00664     // "Re-upload Data" row
00665     auto* reuploadRow = MakeDropDownRow(panel, "Re-upload Data");

```

```

00666     sizer->Add(reuploadRow, 0, wxEXPAND);
00667
00668     auto reuploadAction = [this, popup](wxMouseEvent&) {
00669         popup->Dismiss();
00670         User* u = AppState::getInstance().getCurrentUser();
00671         if (!u) return;
00672         wxFileDialog dlg(this, "Select Watch Later CSV", "", "",
00673             "Supported files (*.csv;*.html;*.htm)|*.csv;*.html;*.htm|All files
(*.*)|*.*",
00674             wxFD_OPEN | wxFD_FILE_MUST_EXIST);
00675         if (dlg.ShowModal() != wxID_OK) return;
00676         controller.getEventRouter().dispatch("uploadData",
{"filePath", dlg.GetPath().ToStdString()}, {"userID", u->getUserID()});
00677         wxMessageBox("Your data has been re-uploaded.", "Data Updated", wxOK | wxICON_INFORMATION,
this);
00678     };
00679     reuploadRow->Bind(wxEVT_LEFT_UP, reuploadAction);
00680     for (wxWindow* w : reuploadRow->GetChildren())
00681         w->Bind(wxEVT_LEFT_UP, reuploadAction);
00682
00683     // "Delete Account" row (same muted color as others, no red)
00684     auto* deleteRow = MakeDropDownRow(panel, "Delete Account");
00685     sizer->Add(deleteRow, 0, wxEXPAND);
00686
00687     auto deleteAction = [this, popup](wxMouseEvent&) {
00688         popup->Dismiss();
00689         User* u = AppState::getInstance().getCurrentUser();
00690         if (!u) return;
00691         bool confirmed = ConfirmationDialog::Confirm(this, "Delete Account",
"Are you sure you want to delete your account? This cannot be undone.");
00692         if (!confirmed) return;
00693         wxTextEntryDialog pwDlg(this, "Re-enter your password:", "Confirm Delete");
00694         if (pwDlg.ShowModal() != wxID_OK) return;
00695         std::string password = pwDlg.GetValue().ToStdString();
00696         if (password.empty()) return;
00697         controller.getEventRouter().dispatch("deleteAccount",
{"userID", u->getUserID()}, {"password", password});
00698         if (AppState::getInstance().getCurrentUser() == nullptr) {
00699             wxMessageBox("Account deleted.", "Done", wxOK | wxICON_INFORMATION, this);
00700             NavigateTo(Page::LOGIN);
00701         } else {
00702             wxMessageBox("Delete failed. Check password.", "Error", wxOK | wxICON_ERROR, this);
00703         }
00704     };
00705     deleteRow->Bind(wxEVT_LEFT_UP, deleteAction);
00706     for (wxWindow* w : deleteRow->GetChildren())
00707         w->Bind(wxEVT_LEFT_UP, deleteAction);
00708
00709     // "Log Out" row
00710     auto* logoutRow = MakeDropDownRow(panel, "Log Out");
00711     sizer->Add(logoutRow, 0, wxEXPAND);
00712
00713     auto logoutAction = [this, popup](wxMouseEvent&) {
00714         popup->Dismiss();
00715         bool confirmed = ConfirmationDialog::Confirm(this, "Log Out", "Log out?");
00716         if (!confirmed) return;
00717         controller.getEventRouter().dispatch("logout", {});
00718         NavigateTo(Page::LOGIN);
00719     };
00720     logoutRow->Bind(wxEVT_LEFT_UP, logoutAction);
00721     for (wxWindow* w : logoutRow->GetChildren())
00722         w->Bind(wxEVT_LEFT_UP, logoutAction);
00723
00724     panel->SetSizer(sizer);
00725     panel->Layout();
00726     panel->Fit();
00727     popup->SetClientSize(panel->GetBestSize());
00728     popup->Layout();
00729     popup->Position(anchor->ClientToScreen(wxPoint(0, 0)), wxSize(0, anchor->GetSize().GetHeight()));
00730     popup->Popup();
00731 }
00732
00733 bool MainFrame::LoadImage(int key, const wxString& path)
00734 {
00735     wxImage image;
00736     if (!image.LoadFile(path, wxBITMAP_TYPE_ANY) || !image.IsOk()) {
00737         return false;
00738     }
00739     images[key] = image;
00740     return true;
00741 }

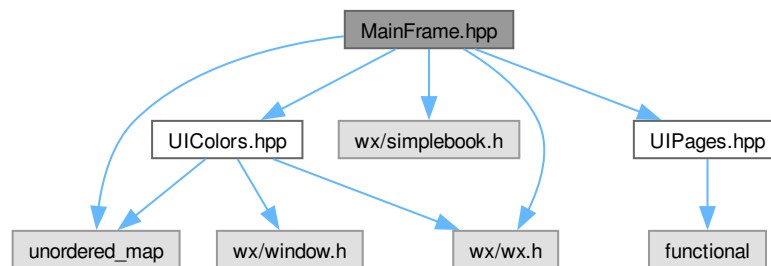
```

7.161 MainFrame.hpp File Reference

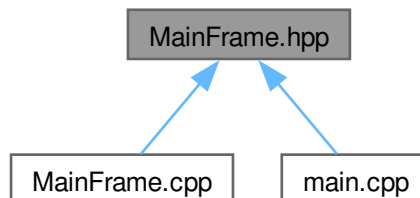
Root application window containing the full-window page switcher.

```
#include <unordered_map>
#include <wx/wx.h>
#include <wx/simplebook.h>
#include "UIPages.hpp"
#include "UIColors.hpp"
```

Include dependency graph for MainFrame.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [MainFrame](#)

Root application window containing the full-window page switcher.

7.161.1 Detailed Description

Root application window containing the full-window page switcher.

Author

Jasmine Kumar
 Shamar Pennant
 Xavier Lusetti

[MainFrame](#) is the single top-level wxFrame. It contains a wxSimplebook that acts as a full-window page switcher (no visible tabs). Every panel is created once at startup and kept alive for the application's lifetime; navigation simply changes which page is visible.

7.161.1.0.1 Page order (must match the Page enum in UIPages.hpp exactly)

Index	Enum	Panel
0	LOGIN	LoginPanel
1	DATA_INSTRUCTIONS	DataInstructionsPanel
2	BLEND_CREATION	BlendCreationPanel
3	HOME	BlendFeedPanel wrapper
4	USER	UserPanel
5	SETTINGS	SettingsPanel
6	BLEND_CHAT	BlendChatPanel

Definition in file [MainFrame.hpp](#).

7.162 MainFrame.hpp

[Go to the documentation of this file.](#)

```

00001
00024
00025 #pragma once
00026 #include <unordered_map>
00027 #include <wx/wx.h>
00028 #include <wx/simplebook.h>
00029 #include "UIPages.hpp"
00030 #include "UIColors.hpp"
00031
00032 class AppController;
00033 class LoginPanel;
00034 class DataInstructionsPanel;
00035 class BlendFeedPanel;
00036 class UserPanel;
00037 class SettingsPanel;
00038 class BlendCreationPanel;
00039 class BlendChatPanel;
00040 class ActiveBlendsPanel;
00041
00049 class MainFrame : public wxFrame {
00050 public:
00056     explicit MainFrame(AppController& controller);
00057
00063     void NavigateTo(Page page);
00064
00069     void TriggerFeedRefresh();
00070
00071 private:
00072     enum class IconBase {
00073         Surface,
00074         Background,
00075     };
00076
00077     enum ImageKey {
00078         BG_MAIN = 0,
00079     };
00080

```

```

00081     ApplicationController& controller;
00082
00083     wxSimplebook*      book;
00084     LoginPanel*        loginPanel;
00085     DataInstructionsPanel* dataInstrPanel;
00086     BlendCreationPanel* creationPanel;
00087     BlendFeedPanel*    feedPanel;
00088     UserPanel*         userPanel;
00089     SettingsPanel*     settingsPanel;
00090     BlendChatPanel*    chatPanel;
00091     ActiveBlendsPanel* activeBlendsPanel;
00092     // Home-page icon button references for theme-switch icon reloading.
00093     wxButton*          refreshBtn;
00094     wxButton*          settingsIconBtn = nullptr;
00095     wxButton*          userIconBtn    = nullptr;
00096     wxButton*          blendsIconBtn  = nullptr;
00097     wxButton*          chatIconBtn    = nullptr;
00098     wxStaticText*      blendIndicatorLabel = nullptr;
00099     // Guards against duplicate refresh dispatches from double-clicks.
00100     bool               refreshInProgress;
00101
00103     void ReloadThemedIcons();
00105     void UpdateBlendIndicatorLabel();
00106
00113     static wxBitmap LoadThemedIcon(const wxString& name, int size = 24);
00114
00123     wxButton* MakeIconButton(wxWindow* parent, const wxString& iconName,
00124                             IconBase base, int iconSize = 24);
00125
00129     void ApplyAppIcons();
00130
00131     std::unordered_map<int, wxImage> images; // images
00132
00137     wxPanel* BuildHomePage(wxWindow* parent);
00138     wxPanel* BuildSettingsPage(wxWindow* parent);
00139
00140     void OnSettings(wxCommandEvent&);
00141     void OnUser     (wxCommandEvent&);
00142     void OnBlend    (wxCommandEvent&);
00143     void OnRefresh  (wxCommandEvent&);
00144
00145     bool LoadImage(int key, const wxString& path);
00146
00148     void ShowSettingsDropdown(wxWindow* anchor);
00150     void ShowUserDropdown(wxWindow* anchor);
00151 };

```

7.163 OperationProgressDialog.cpp File Reference

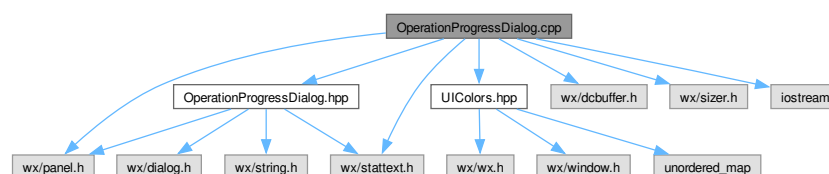
Implementation of non-modal progress dialog.

```

#include "OperationProgressDialog.hpp"
#include "UIColors.hpp"
#include <wx/dcbuffer.h>
#include <wx/sizer.h>
#include <wx/stattext.h>
#include <wx/panel.h>
#include <iostream>

```

Include dependency graph for OperationProgressDialog.cpp:



7.163.1 Detailed Description

Implementation of non-modal progress dialog.

Author

Shamar Pennant

Definition in file [OperationProgressDialog.cpp](#).

7.164 OperationProgressDialog.cpp

[Go to the documentation of this file.](#)

```

00001
00006
00007 #include "OperationProgressDialog.hpp"
00008 #include "UIColors.hpp"
00009
00010 #include <wx/dcbuffer.h>
00011 #include <wx/sizer.h>
00012 #include <wx/stattext.h>
00013 #include <wx/panel.h>
00014 #include <iostream>
00015
00016 namespace {
00017 constexpr int kProgressBorderPx = 1;
00018 }
00019
00020 OperationProgressDialog::OperationProgressDialog(wxWindow* parent, const wxString& title)
00021 : wxDialog(parent, wxID_ANY, title,
00022           wxDefaultPosition, wxSize(420, 180),
00023           wxDEFAULT_DIALOG_STYLE | wxRESIZE_BORDER) {
00024
00025     // Create main vertical sizer
00026     auto* mainSizer = new wxBoxSizer(wxVERTICAL);
00027     mainSizer->AddSpacer(10);
00028
00029     // Status message label
00030     messageLabel = new wxStaticText(this, wxID_ANY, wxT("Initializing..."));
00031     wxFont messageFont = messageLabel->GetFont();
00032     messageFont.SetPointSize(10);
00033     messageLabel->SetFont(messageFont);
00034     messageLabel->SetForegroundColour(UIColors::TextPrimary());
00035     mainSizer->Add(messageLabel, 0, wxEXPAND | wxLEFT | wxRIGHT, 20);
00036     mainSizer->AddSpacer(8);
00037
00038     // Custom progress bar (track + fill) to ensure theme colors render on GTK.
00039     progressTrack = new wxPanel(this, wxID_ANY, wxDefaultPosition, wxSize(-1, 14));
00040     progressTrack->SetMinSize(wxSize(-1, 14));
00041     progressTrack->SetBackgroundStyle(wxBG_STYLE_PAINT);
00042     progressTrack->SetBackgroundColour(UIColors::SurfaceRaised());
00043
00044     // Paint a consistent border for visibility in all themes.
00045     progressTrack->Bind(wxEVT_PAINT, [this](wxPaintEvent&) {
00046         wxAutoBufferedPaintDC dc(progressTrack);
00047         const wxSize sz = progressTrack->GetClientSize();
00048
00049         dc.SetPen(*wxTRANSPARENT_PEN);
00050         dc.SetBrush(wxBrush(UIColors::Separator()));
00051         dc.DrawRectangle(0, 0, sz.GetWidth(), sz.GetHeight());
00052
00053         const int innerW = std::max(0, sz.GetWidth() - (2 * kProgressBorderPx));
00054         const int innerH = std::max(0, sz.GetHeight() - (2 * kProgressBorderPx));
00055         dc.SetBrush(wxBrush(UIColors::SurfaceRaised()));
00056         dc.DrawRectangle(kProgressBorderPx, kProgressBorderPx, innerW, innerH);
00057     });
00058
00059     progressFill = new wxPanel(progressTrack, wxID_ANY, wxDefaultPosition, wxSize(0, 14));
00060     progressFill->SetBackgroundColour(UIColors::Accent());
00061
00062     progressTrack->Bind(wxEVT_SIZE, [this](wxSizeEvent& evt) {
00063         UpdateProgressFill();
00064         evt.Skip();
00065     });
00066

```

```

00067     mainSizer->Add(progressTrack, 0, wxEXPAND | wxLEFT | wxRIGHT, 20);
00068
00069     // Percentage label centred below the progress bar.
00070     percentLabel = new wxStaticText(this, wxID_ANY, wxT("0%"),
00071                                     wxDefaultPosition, wxDefaultSize,
00072                                     wxALIGN_CENTRE_HORIZONTAL);
00073     wxFont percentFont = percentLabel->GetFont();
00074     percentFont.SetPointSize(9);
00075     percentLabel->SetFont(percentFont);
00076     percentLabel->SetForegroundColour(UIColors::TextSecondary());
00077     mainSizer->AddSpacer(6);
00078     mainSizer->Add(percentLabel, 0, wxALIGN_CENTER_HORIZONTAL);
00079     mainSizer->AddStretchSpacer(1);
00080
00081     SetSizer(mainSizer);
00082     SetBackgroundColour(UIColors::Surface());
00083     SetWindowStyle(wxDEFAULT_DIALOG_STYLE | wxRESIZE_BORDER);
00084     UpdateProgressFill();
00085
00086     std::cout << "[OperationProgressDialog] Created progress dialog: "
00087               << std::string(title.mb_str()) << "\n";
00088 }
00089
00090 OperationProgressDialog::~OperationProgressDialog() {
00091     std::cout << "[OperationProgressDialog] Destroyed progress dialog\n";
00092 }
00093
00094 void OperationProgressDialog::UpdateProgress(double progress, const wxString& message) {
00095     // Clamp progress to 0.0-1.0 range
00096     if (progress < 0.0) progress = 0.0;
00097     if (progress > 1.0) progress = 1.0;
00098
00099     // Update custom progress fill
00100     currentProgress = progress;
00101     UpdateProgressFill();
00102
00103     // Update percentage label
00104     int percent = static_cast<int>(progress * 100);
00105     percentLabel->SetLabel(wxString::Format(wxT("%d%%"), percent));
00106
00107     // Update message if provided
00108     if (!message.IsEmpty()) {
00109         messageLabel->SetLabel(message);
00110     }
00111
00112     // Redraw dialog
00113     Layout();
00114     Refresh();
00115     wxYield();
00116 }
00117
00118 int OperationProgressDialog::ShowModal() {
00119     // Display non-modally to keep application responsive
00120     Show(true);
00121     return wxID_OK;
00122 }
00123
00124 void OperationProgressDialog::UpdateProgressFill() {
00125     if (progressTrack == nullptr || progressFill == nullptr) {
00126         return;
00127     }
00128
00129     const wxSize trackSize = progressTrack->GetClientSize();
00130     const int innerW = std::max(0, trackSize.GetWidth() - (2 * kProgressBorderPx));
00131     const int innerH = std::max(0, trackSize.GetHeight() - (2 * kProgressBorderPx));
00132     const int fillWidth = static_cast<int>(innerW * currentProgress);
00133
00134     progressFill->SetSize(kProgressBorderPx,
00135                          kProgressBorderPx,
00136                          std::max(0, fillWidth),
00137                          innerH);
00138
00139     progressTrack->Refresh();
00140     progressFill->Refresh();
00141 }

```

7.165 OperationProgressDialog.hpp File Reference

Dark-themed non-modal progress dialog for file upload/processing.

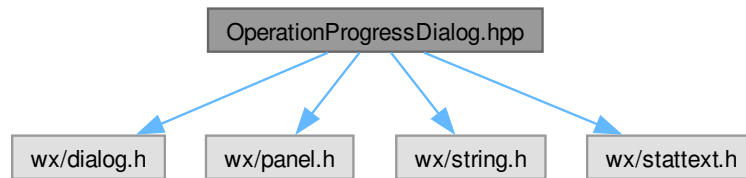
```

#include <wx/dialog.h>
#include <wx/panel.h>

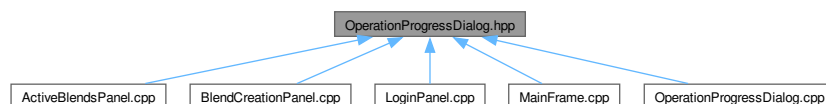
```

```
#include <wx/string.h>
#include <wx/stattext.h>
```

Include dependency graph for OperationProgressDialog.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [OperationProgressDialog](#)
Non-modal progress dialog showing upload/processing progress.

7.165.1 Detailed Description

Dark-themed non-modal progress dialog for file upload/processing.

Author

Shamar Pennant

Displays progress bar, percentage, and status message during long operations. Helping to indicate to the user the progress of file uploading

Definition in file [OperationProgressDialog.hpp](#).

7.166 OperationProgressDialog.hpp

[Go to the documentation of this file.](#)

```

00001
00010
00011 #pragma once
00012 #include <wx/dialog.h>
00013 #include <wx/panel.h>
00014 #include <wx/string.h>
00015 #include <wx/stattext.h>
00016
00021 class OperationProgressDialog : public wxDialog {
00022 public:
00026     OperationProgressDialog(wxWindow*      parent,
00027                             const wxString& title);
00028
00032     ~OperationProgressDialog();
00033
00039     void UpdateProgress(double progress, const wxString& message = wxEmptyString);
00040
00044     virtual int ShowModal();
00045
00046 private:
00048     wxPanel*      progressTrack;
00049     wxPanel*      progressFill;
00050     wxStaticText* percentLabel;
00051     wxStaticText* messageLabel;
00052     double        currentProgress = 0.0;
00053
00054     void UpdateProgressFill();
00055 };
00056
00057

```

7.167 ResourcePathUtils.cpp File Reference

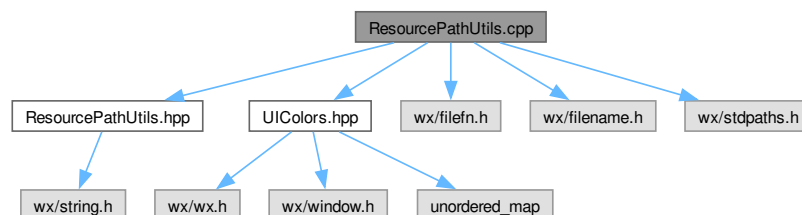
Shared resource path resolution helpers for UI components.

```

#include "ResourcePathUtils.hpp"
#include "UIColors.hpp"
#include <wx/filefn.h>
#include <wx/filename.h>
#include <wx/stdpaths.h>

```

Include dependency graph for ResourcePathUtils.cpp:



Functions

- wxString [UIResourcePaths::FindResourcePath](#) (const wxString &fileName)
Resolve a UI resource path for both source-tree and installed layouts.
- wxString [UIResourcePaths::GetIconFolder](#) ()
Returns the icon folder name for the current theme.

7.167.1 Detailed Description

Shared resource path resolution helpers for UI components.

Definition in file [ResourcePathUtils.cpp](#).

7.167.2 Function Documentation

7.167.2.1 FindResourcePath()

```
wxString UIResourcePaths::FindResourcePath (  
    const wxString & fileName)
```

Resolve a UI resource path for both source-tree and installed layouts.

Parameters

<i>fileName</i>	Resource file path relative to the resources root.
-----------------	--

Returns

Path to the first matching resource, or empty string if not found.

Definition at line 15 of file [ResourcePathUtils.cpp](#).

7.167.2.2 GetIconFolder()

```
wxString UIResourcePaths::GetIconFolder ()
```

Returns the icon folder name for the current theme.

Maps theme names to icon folder names. Themes without their own icon set fall back to the closest visual match (dark or light).

Definition at line 47 of file [ResourcePathUtils.cpp](#).

7.168 ResourcePathUtils.cpp

[Go to the documentation of this file.](#)

```

00001
00005
00006 #include "ResourcePathUtils.hpp"
00007 #include "UIColors.hpp"
00008
00009 #include <wx/filefn.h>
00010 #include <wx/filename.h>
00011 #include <wx/stdpaths.h>
00012
00013 namespace UIResourcePaths {
00014
00015 wxString FindResourcePath(const wxString& fileName)
00016 {
00017     const wxArrayString relativeCandidates = {
00018         "YTBIND_src/resources/" + fileName,
00019         "resources/" + fileName,
00020         "../resources/" + fileName,
00021     };
00022
00023     for (const auto& candidate : relativeCandidates) {
00024         if (wxFileExists(candidate)) {
00025             return candidate;
00026         }
00027     }
00028
00029     wxFileName exePath(wxStandardPaths::Get().GetExecutablePath());
00030     const wxString exeDir = exePath.GetPath();
00031     const wxArrayString exeRelativeCandidates = {
00032         exeDir + "../../../YTBIND_src/resources/" + fileName,
00033         exeDir + "../../../share/ytblnd/resources/" + fileName,
00034         exeDir + "../resources/" + fileName,
00035         exeDir + "/resources/" + fileName,
00036     };
00037
00038     for (const auto& candidate : exeRelativeCandidates) {
00039         if (wxFileExists(candidate)) {
00040             return candidate;
00041         }
00042     }
00043
00044     return wxEmptyString;
00045 }
00046
00047 wxString GetIconFolder()
00048 {
00049     wxString themeName = "Dark Mode";
00050     if (UIColors::Current) {
00051         themeName = UIColors::Current->Name;
00052     }
00053
00054     if (themeName == "Dark Mode") return "dark";
00055     if (themeName == "Light Mode") return "light";
00056     if (themeName == "Neon Mode") return "neon";
00057     if (themeName == "Cyber Amber") return "amber";
00058     if (themeName == "Forest Floor") return "amber";
00059     if (themeName == "Nordic Frost") return "light";
00060     // All remaining dark-background themes use dark icons
00061     return "dark";
00062 }
00063
00064 }

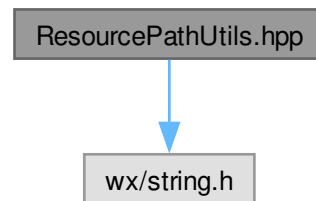
```

7.169 ResourcePathUtils.hpp File Reference

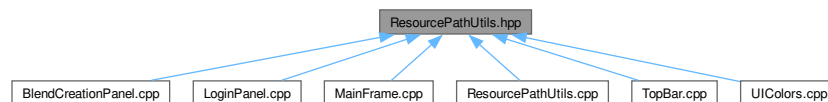
Shared resource path resolution helpers for UI components.


```
#include <wx/string.h>
```

Include dependency graph for ResourcePathUtils.hpp:



This graph shows which files directly or indirectly include this file:



Functions

- wxString [UIResourcePaths::FindResourcePath](#) (const wxString &fileName)
Resolve a UI resource path for both source-tree and installed layouts.
- wxString [UIResourcePaths::GetIconFolder](#) ()
Returns the icon folder name for the current theme.

7.169.1 Detailed Description

Shared resource path resolution helpers for UI components.

Author

Xavier Lusetti
Shamar Pennant

Definition in file [ResourcePathUtils.hpp](#).

7.169.2 Function Documentation

7.169.2.1 FindResourcePath()

```
wxString UIResourcePaths::FindResourcePath (  
    const wxString & fileName)
```

Resolve a UI resource path for both source-tree and installed layouts.

Parameters

<i>fileName</i>	Resource file path relative to the resources root.
-----------------	--

Returns

Path to the first matching resource, or empty string if not found.

Definition at line 15 of file [ResourcePathUtils.cpp](#).

7.169.2.2 GetIconFolder()

```
wxString UIResourcePaths::GetIconFolder ()
```

Returns the icon folder name for the current theme.

Maps theme names to icon folder names. Themes without their own icon set fall back to the closest visual match (dark or light).

Definition at line 47 of file [ResourcePathUtils.cpp](#).

7.170 ResourcePathUtils.hpp

[Go to the documentation of this file.](#)

```
00001
00007
00008 #pragma once
00009
00010 #include <wx/string.h>
00011
00012 namespace UIResourcePaths {
00013
00019 wxString FindResourcePath(const wxString& fileName);
00020
00027 wxString GetIconFolder();
00028
00029 }
```

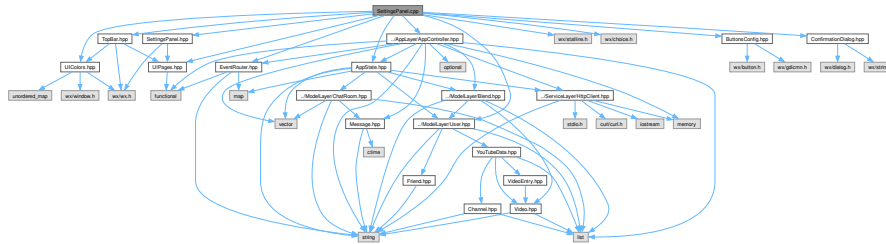
7.171 SettingsPanel.cpp File Reference

Implementation for [SettingsPanel](#).

```
#include "SettingsPanel.hpp"
#include <wx/statline.h>
#include <wx/choice.h>
#include "UIColors.hpp"
#include "UIPages.hpp"
#include "TopBar.hpp"
#include "ButtonsConfig.hpp"
#include "ConfirmationDialog.hpp"
#include "../AppLayer/AppController.hpp"
#include "../AppLayer/AppState.hpp"
#include "../AppLayer/EventRouter.hpp"
```

```
#include "../ModelLayer/User.hpp"
```

Include dependency graph for SettingsPanel.cpp:



7.171.1 Detailed Description

Implementation for `SettingsPanel`.

Author

Jasmine Kumar
Xavier Lusetti

Definition in file [SettingsPanel.cpp](#).

7.172 SettingsPanel.cpp

[Go to the documentation of this file.](#)

```
00001
00002
00003 #include "SettingsPanel.hpp"
00004
00005 #include <wx/statline.h>
00006 #include <wx/choice.h>
00007
00008 #include "UIColors.hpp"
00009 #include "UIPages.hpp"
00010 #include "TopBar.hpp"
00011 #include "ButtonsConfig.hpp"
00012 #include "ConfirmationDialog.hpp"
00013
00014 #include "../AppLayer/AppController.hpp"
00015 #include "../AppLayer/AppState.hpp"
00016 #include "../AppLayer/EventRouter.hpp"
00017 #include "../ModelLayer/User.hpp"
00018
00019 // Constructor
00020
00021 SettingsPanel::SettingsPanel(wxWindow* parent, AppController& controller, NavigateFn nav)
00022 : wxPanel(parent, wxID_ANY)
00023 , m_controller(controller)
00024 , m_nav(std::move(nav))
00025 {
00026     SetBackgroundColour(UIColors::Background());
00027
00028     auto* root = new wxBoxSizer(wxVERTICAL);
00029
00030     // Top bar
00031     auto* topBar = new TopBar(this, "Settings", m_nav, Page::HOME);
00032     root->Add(topBar, 0, wxEXPAND);
00033
00034     // Theme selection section
00035     auto* themePanel = new wxPanel(this, wxID_ANY);
00036     themePanel->SetBackgroundColour(UIColors::Danger());
00037     auto* themeSizer = new wxBoxSizer(wxHORIZONTAL);
```

```

00043
00044     auto* themeLabel = new wxStaticText(themePanel, wxID_ANY, "App Theme:");
00045     themeLabel->SetForegroundColour(UIColors::TextPrimary());
00046
00047     // Theme Options, populated using the ordered theme vector
00048     wxArrayString choices;
00049     for (const wxString& name : UIColors::ThemeOrder) {
00050         choices.Add(name);
00051     }
00052
00053     m_themeChoice = new wxChoice(themePanel, wxID_ANY, wxDefaultPosition, wxDefaultSize, choices);
00054
00055     // Set current selection based on the active palette name
00056     if (UIColors::Current) {
00057         m_themeChoice->SetStringSelection(UIColors::Current->Name);
00058     }
00059
00060     m_themeChoice->Bind(wxEVT_CHOICE, &SettingsPanel::OnThemeChanged, this);
00061
00062     themeSizer->Add(themeLabel, 0, wxALIGN_CENTER_VERTICAL | wxRIGHT, 10);
00063     themeSizer->Add(m_themeChoice, 1, wxEXPAND);
00064
00065     themePanel->SetSizer(themeSizer);
00066
00067     // Add it to the root sizer with padding
00068     root->Add(themePanel, 0, wxEXPAND | wxLEFT | wxRIGHT | wxTOP, 16);
00069
00070     // Divider
00071     auto* divider = new wxStaticLine(this, wxID_ANY);
00072     divider->SetBackgroundColour(UIColors::Separator());
00073     root->Add(divider, 0, wxEXPAND);
00074
00075     // Spacer
00076     root->AddStretchSpacer(1);
00077
00078     // Logout section
00079     auto* logoutBtn = new wxButton(this, wxID_ANY, "Log Out");
00080     UIButtons::ApplySizeBounds(logoutBtn, ButtonType::FullWidthDanger);
00081     logoutBtn->SetBackgroundColour(UIColors::Danger());
00082     logoutBtn->SetForegroundColour(UIColors::TextPrimary());
00083     root->Add(logoutBtn, 0, wxEXPAND | wxLEFT | wxRIGHT | wxBOTTOM, 16);
00084
00085     SetSizer(root);
00086
00087     logoutBtn->Bind(wxEVT_BUTTON, &SettingsPanel::OnLogout, this);
00088     Bind(wxEVT_SHOW, &SettingsPanel::OnShow, this);
00089 }
00090
00091 // Private helpers
00092
00093 void SettingsPanel::RefreshUserInfo()
00094 {
00095     User* user = AppState::getInstance().getCurrentUser();
00096 }
00097
00098 // Event handlers
00099
00100 void SettingsPanel::OnShow(wxShowEvent& evt)
00101 {
00102     if (evt.IsShown()) {
00103         RefreshUserInfo();
00104         // Update selection in case the theme was changed elsewhere
00105         if (m_themeChoice && UIColors::Current) {
00106             m_themeChoice->SetStringSelection(UIColors::Current->Name);
00107         }
00108     }
00109     evt.Skip();
00110 }
00111
00112 void SettingsPanel::OnLogout(wxCommandEvent& /*evt*/)
00113 {
00114     bool confirmed = ConfirmationDialog::Confirm(this, "Log Out", "Log out?");
00115     if (!confirmed) return;
00116
00117     m_controller.getEventRouter().dispatch("logout", {});
00118     m_nav(Page::LOGIN);
00119 }
00120
00121 // When the user changes their desired theme,
00122 // this change needs to be populated throughout the entire app
00123 void SettingsPanel::OnThemeChanged(wxCommandEvent& evt) {
00124     // Get the string name directly from the choice UI
00125     wxString selectedTheme = m_themeChoice->GetStringSelection();
00126
00127     // Update theme selection in the global color controller
00128     UIColors::SetTheme(selectedTheme);
00129 }

```

```

00130     // Refresh the entire app starting from the top frame
00131     wxWindow* top = wxGetTopLevelParent(this);
00132     UIColors::UpdateUI(top);
00133
00134     // BROADCASTER for MainFrame to listen for the theme update
00135     m_controller.getEventRouter().dispatch("theme_updated", {});
00136 }

```

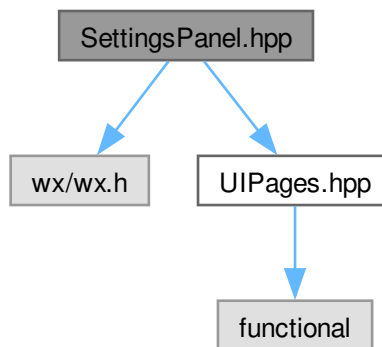
7.173 SettingsPanel.hpp File Reference

Settings Panel controls the theme choice for the user's GUI.

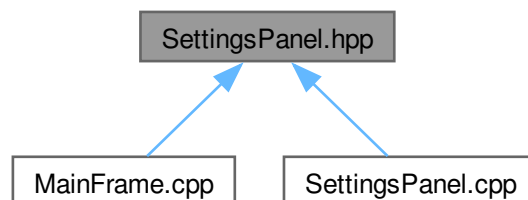
```
#include <wx/wx.h>
```

```
#include "UIPages.hpp"
```

Include dependency graph for SettingsPanel.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [SettingsPanel](#)
SettingsPanel class declaration.

7.173.1 Detailed Description

Settings Panel controls the theme choice for the user's GUI.

Author

Xavier Lusetti

Definition in file [SettingsPanel.hpp](#).

7.174 SettingsPanel.hpp

[Go to the documentation of this file.](#)

```

00001
00007 #pragma once
00008
00009 #include <wx/wx.h>
00010
00011 #include "UIPages.hpp"
00012
00013 class AppController;
00014 class wxChoice;
00015
00020 class SettingsPanel : public wxPanel {
00021 public:
00022     SettingsPanel(wxWindow* parent, AppController& controller, NavigateFn nav);
00023
00024 private:
00025     AppController& m_controller;
00026     NavigateFn m_nav;
00027     wxChoice* m_themeChoice = nullptr;
00028
00029     void RefreshUserInfo();
00030
00031     void OnShow(wxShowEvent& evt);
00032     void OnThemeChanged(wxCommandEvent& evt);
00033     void OnLogout(wxCommandEvent& evt);
00034 };

```

7.175 TopBar.cpp File Reference

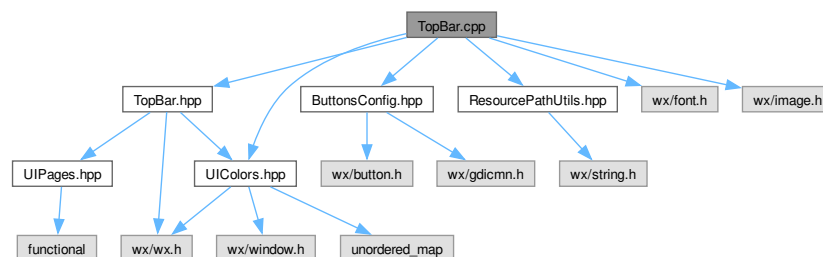
Implementation for [TopBar](#).

```

#include "TopBar.hpp"
#include "UIColors.hpp"
#include "ButtonsConfig.hpp"
#include "ResourcePathUtils.hpp"
#include <wx/font.h>
#include <wx/image.h>

```

Include dependency graph for TopBar.cpp:



7.175.1 Detailed Description

Implementation for [TopBar](#).

Author

Jasmine Kumar

Xavier Lusetti

Definition in file [TopBar.cpp](#).

7.176 TopBar.cpp

[Go to the documentation of this file.](#)

```

00001
00007
00008 // =====
00009 // TopBar.cpp - Reusable interior-page header bar implementation
00010 // =====
00011
00012 #include "TopBar.hpp"
00013 #include "UIColors.hpp"
00014 #include "ButtonsConfig.hpp"
00015 #include "ResourcePathUtils.hpp"
00016 #include <wx/font.h>
00017 #include <wx/image.h>
00018
00019 TopBar::TopBar(wxWindow* parent, const wxString& title, NavigateFn nav, Page backDest)
00020     : wxPanel(parent, wxID_ANY)
00021     , m_nav(std::move(nav))
00022     , m_backDest(backDest)
00023 {
00024     SetBackgroundColour(UIColors::Background());
00025
00026     // -- Back icon button (standalone on Background) -----
00027     m_backBtn = new wxButton(this, wxID_ANY, wxEmptyString,
00028                             wxDefaultPosition, wxSize(32, 32),
00029                             wxBORDER_NONE);
00030     OnThemeUpdate();
00031
00032     // Back button event bindings
00033     m_backBtn->Bind(wxEVT_BUTTON, &TopBar::OnBack, this);
00034     // Hover effects
00035     m_backBtn->Bind(wxEVT_ENTER_WINDOW, [this](wxMouseEvent& e) {
00036         m_backBtn->SetBackgroundColour(UIColors::SurfaceRaised());
00037         m_backBtn->Refresh();
00038         e.Skip();
00039     });
00040     m_backBtn->Bind(wxEVT_LEAVE_WINDOW, [this](wxMouseEvent& e) {
00041         m_backBtn->SetBackgroundColour(UIColors::Background());
00042         m_backBtn->Refresh();
00043         e.Skip();
00044     });
00045
00046     // -- Title label -----
00047     wxStaticText* titleLabel = new wxStaticText(this, wxID_ANY, title,
00048                                                 wxDefaultPosition, wxDefaultSize,
00049                                                 wxALIGN_CENTER_HORIZONTAL);
00050     titleLabel->SetForegroundColour(UIColors::TextPrimary());
00051
00052     wxFont titleFont = titleLabel->GetFont();
00053     titleFont.SetPointSize(14);
00054     titleFont.SetWeight(wxFONTWEIGHT_BOLD);
00055     titleLabel->SetFont(titleFont);
00056
00057     // -- Layout -----
00058     // Three-column sizer: [back btn | title (stretch) | right spacer]
00059     // The right spacer mirrors the back button width so the title stays centred.
00060     wxBoxSizer* sizer = new wxBoxSizer(wxHORIZONTAL);
00061     sizer->Add(m_backBtn, 0, wxALIGN_CENTER_VERTICAL | wxLEFT, 8);
00062     sizer->AddStretchSpacer(1);
00063     sizer->Add(titleLabel, 0, wxALIGN_CENTER_VERTICAL);
00064     sizer->AddStretchSpacer(1);
00065     sizer->Add(36 + 8, 0); // right spacer to balance the back icon

```

```

00066
00067     wxBoxSizer* outerSizer = new wxBoxSizer(wxVERTICAL);
00068     outerSizer->Add(sizer, 1, wxEXPAND | wxTOP | wxBOTTOM, 4);
00069
00070     SetSizer(outerSizer);
00071     SetMinSize(wxSize(-1, 40));
00072 }
00073
00074 void TopBar::OnThemeUpdate() {
00075     if (!m_backBtn) return;
00076
00077     // Update the button background to match the new theme
00078     m_backBtn->SetBackgroundColour(UIColors::Background());
00079
00080     // Fetch the updated SVG icon
00081     wxBitmapBundle icon = UIColors::GetIcon("back.svg", ColorRole::TextPrimary, 32);
00082
00083     if (icon.IsOk()) {
00084         m_backBtn->SetBitmap(icon);
00085     }
00086
00087     // Repaint
00088     m_backBtn->Refresh();
00089 }
00090
00091 void TopBar::OnBack(wxCommandEvent& /*evt*/)
00092 {
00093     if (m_nav) {
00094         m_nav(m_backDest);
00095     }
00096 }

```

7.177 TopBar.hpp File Reference

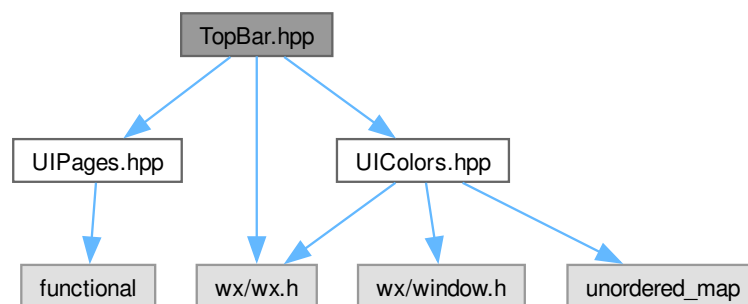
Reusable interior-page header bar.

```
#include <wx/wx.h>
```

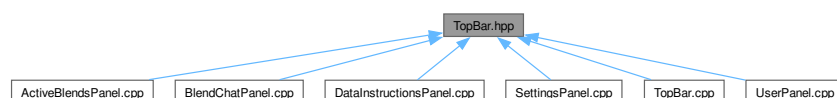
```
#include "UIPages.hpp"
```

```
#include "UIColors.hpp"
```

Include dependency graph for TopBar.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [TopBar](#)

Reusable horizontal top bar for interior pages.

7.177.1 Detailed Description

Reusable interior-page header bar.

Author

Jasmine Kumar

Xavier Lusetti

Displays a left-aligned back button and a centred title label for interior pages. The back button triggers the provided [NavigateFn](#) callback with the configured destination page.

Definition in file [TopBar.hpp](#).

7.178 TopBar.hpp

[Go to the documentation of this file.](#)

```
00001
00011
00012 #pragma once
00013 #include <wx/wx.h>
00014 #include "UIPages.hpp"
00015 #include "UIColors.hpp"
00016
00021 class TopBar : public wxPanel, public IThemedElement {
00022 public:
00030     TopBar(wxWindow* parent, const wxString& title, NavigateFn nav, Page backDest);
00031
00032     // Theme update override required because TopBar uses .svg icons
00033     void OnThemeUpdate() override;
00034
00035 private:
00036
00041     void OnBack(wxCommandEvent& evt);
00042
00044     NavigateFn m_nav;
00046     Page m_backDest;
00048     wxButton* m_backBtn = nullptr;
00049 };
```

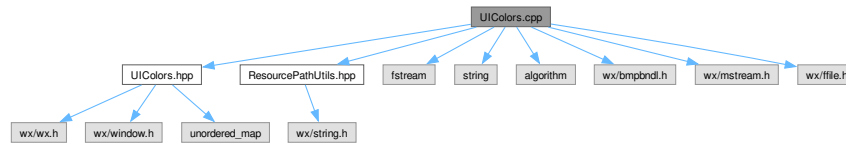
7.179 UIColors.cpp File Reference

Logic for dynamic theme switching and UI recursion.

```
#include "UIColors.hpp"
#include "ResourcePathUtils.hpp"
#include <fstream>
#include <string>
#include <algorithm>
#include <wx/bmpbndl.h>
#include <wx/mstream.h>
```

```
#include <wx/ffile.h>
```

Include dependency graph for UIColors.cpp:



7.179.1 Detailed Description

Logic for dynamic theme switching and UI recursion.

Author

Xavier Lusetti

Definition in file [UIColors.cpp](#).

7.180 UIColors.cpp

[Go to the documentation of this file.](#)

```

00001
00006
00007 #include "UIColors.hpp"
00008 #include "ResourcePathUtils.hpp"
00009
00010 #include <fstream>
00011 #include <string>
00012 #include <algorithm>
00013
00014 #include <wx/bmpbndl.h>
00015 #include <wx/mstream.h>
00016 #include <wx/ffile.h>
00017
00018 // Initialize;
00019 // unordered map of palettes
00020 // vector of ordered themes
00021 // and pointer to the current palette
00022 std::unordered_map<wxString, Palette> UIColors::ThemeMap;
00023 std::vector<wxString> UIColors::ThemeOrder;
00024 Palette* UIColors::Current = nullptr;
00025
00026 bool UIColors::LoadThemesFromFile(const wxString& filepath) {
00027     std::ifstream file(filepath.ToStdString());
00028     if (!file.is_open()) {
00029         return false;
00030     }
00031
00032     std::string line;
00033     while (std::getline(file, line)) {
00034         // Clean up the line
00035         wxString themeName = wxString::FromUTF8(line.c_str());
00036         themeName.Trim(true).Trim(false);
00037         if (themeName.IsEmpty()) continue;
00038
00039         Palette p;
00040         p.Name = themeName;
00041
00042         // Helper to read and trim the next line from the file
00043         auto readColor = [&]() -> wxColour {
00044             std::string hex;
00045             if (std::getline(file, hex)) {
00046                 wxString wxHex = wxString::FromUTF8(hex.c_str());

```

```

00047
00048         // Remove whitespace
00049         wxHex.Replace(" ", "");
00050         wxHex.Replace("\r", "");
00051         wxHex.Replace("\t", "");
00052         wxHex.Trim(true).Trim(false);
00053
00054         if (wxHex.IsEmpty()) return *wxBLACK;
00055
00056         wxColour col(wxHex);
00057         if (!col.IsOk()) {
00058             wxLogError("Failed to parse color: '%s'", wxHex);
00059             return *wxBLACK;
00060         }
00061         return col;
00062     }
00063     return *wxBLACK;
00064 };
00065
00066     // Order must be matching within the .txt file
00067     p.Background = readColor();
00068     p.Surface = readColor();
00069     p.SurfaceRaised = readColor();
00070     p.Accent = readColor();
00071     p.AccentHover = readColor();
00072     p.TextPrimary = readColor();
00073     p.TextSecondary = readColor();
00074     p.TextMuted = readColor();
00075     p.Separator = readColor();
00076     p.Danger = readColor();
00077
00078     ThemeMap[themeName] = p;
00079
00080     // Record the string name in order IF it isn't already there
00081     if (std::find(ThemeOrder.begin(), ThemeOrder.end(), themeName) == ThemeOrder.end()) {
00082         ThemeOrder.push_back(themeName);
00083     }
00084 }
00085
00086     // DEFAULT to the first theme loaded if nothing is set
00087     if (ThemeMap.empty()) {
00088         SetTheme("Dark Mode");
00089     // SUCCESSFUL loading. Use the first loaded theme
00090     } else if (Current == nullptr) {
00091         Current = &ThemeMap[ThemeOrder[0]];
00092     }
00093
00094     return true;
00095 }
00096
00097 void UIColors::SetTheme(const wxString& themeName) {
00098     auto it = ThemeMap.find(themeName);
00099
00100     if (it != ThemeMap.end()) {
00101         Current = it->second;
00102     }
00103     // DEFAULT fail safe
00104     else if (themeName == "Dark Mode") {
00105         // Record the details, update the map, update the pointer, update the ordered vector
00106         Palette dark;
00107         dark.Name = "Dark Mode";
00108         dark.Background = wxColour("#121212");
00109         dark.Surface = wxColour("#1E1E1E");
00110         dark.SurfaceRaised = wxColour("#2C2C2C");
00111         dark.Accent = wxColour("#8A00FF");
00112         dark.AccentHover = wxColour("#A028FF");
00113         dark.TextPrimary = wxColour("FFFFFF");
00114         dark.TextSecondary = wxColour("#BDBDBD");
00115         dark.TextMuted = wxColour("#757575");
00116         dark.Separator = wxColour("#252525");
00117         dark.Danger = wxColour("#FF4C29");
00118
00119         ThemeMap["Dark Mode"] = dark;
00120         Current = &ThemeMap["Dark Mode"];
00121         if (std::find(ThemeOrder.begin(), ThemeOrder.end(), "Dark Mode") == ThemeOrder.end()) {
00122             ThemeOrder.push_back("Dark Mode");
00123         }
00124     }
00125 }
00126
00127 wxButton* UIColors::MakeIconButton(wxWindow* parent, const wxString& iconName,
00128                                     ColorRole bgRole, int iconSize)
00129 {
00130     // Create the button with a size slightly larger than the icon for padding
00131     wxButton* btn = new wxButton(parent, wxID_ANY, wxEmptyString,
00132                                  wxDefaultPosition, wxSize(iconSize + 12, iconSize + 12),
00133                                  wxBORDER_NONE);

```

```

00134
00135 // Tag the button with the icon name so OnThemeUpdate knows which SVG to reload
00136 btn->SetName(iconName);
00137
00138 // Apply initial theme colors
00139 wxColour bg;
00140 switch(bgRole) {
00141     case ColorRole::Surface: bg = Surface(); break;
00142     default:                 bg = Background(); break;
00143 }
00144 btn->SetBackgroundColour(bg);
00145
00146 // Load and set the initial SVG
00147 wxBitmapBundle icon = GetIcon(iconName + ".svg", ColorRole::TextPrimary, iconSize);
00148 if (icon.IsOk()) {
00149     btn->SetBitmap(icon);
00150 }
00151
00152 return btn;
00153 }
00154
00155 wxBitmapBundle UIColors::GetIcon(const wxString& iconName, ColorRole role, int size) {
00156     wxString folder = UIResourcePaths::GetIconFolder();
00157     wxString path = UIResourcePaths::FindResourcePath("icons/" + folder + "/" + iconName);
00158
00159     if (path.empty() || !Current) return wxBitmapBundle();
00160
00161     wxFFFile file(path, "r");
00162     if (!file.IsOpened()) return wxBitmapBundle();
00163
00164     wxString svgContent;
00165     file.ReadAll(&svgContent);
00166
00167     // Map the role to the current palette's color
00168     wxColour targetColor;
00169     switch(role) {
00170         case ColorRole::Background: targetColor = Current->Background; break;
00171         case ColorRole::Surface: targetColor = Current->Surface; break;
00172         case ColorRole::SurfaceRaised: targetColor = Current->SurfaceRaised; break;
00173         case ColorRole::Accent: targetColor = Current->Accent; break;
00174         case ColorRole::AccentHover: targetColor = Current->AccentHover; break;
00175         case ColorRole::TextPrimary: targetColor = Current->TextPrimary; break;
00176         case ColorRole::TextSecondary: targetColor = Current->TextPrimary; break;
00177         case ColorRole::TextMuted: targetColor = Current->TextPrimary; break;
00178         case ColorRole::Separator: targetColor = Current->Separator; break;
00179         case ColorRole::Danger: targetColor = Current->Danger; break;
00180         default: targetColor = Current->TextPrimary;
00181     }
00182
00183     svgContent.Replace("CURRENT_COLOR", targetColor.GetAsString(wxC2S_HTML_SYNTAX));
00184
00185     return wxBitmapBundle::FromSVG(svgContent, wxSize(size, size));
00186 }
00187
00188 ColorRole UIColors::GetRoleFromColour(const wxColour& col) {
00189     // Detect roles for current background and foreground by searching through the theme map
00190     for (auto const& [name, p] : ThemeMap) {
00191         if (col == p.Background) return ColorRole::Background;
00192         if (col == p.Surface) return ColorRole::Surface;
00193         if (col == p.SurfaceRaised) return ColorRole::SurfaceRaised;
00194         if (col == p.Accent) return ColorRole::Accent;
00195         if (col == p.AccentHover) return ColorRole::AccentHover;
00196         if (col == p.TextPrimary) return ColorRole::TextPrimary;
00197         if (col == p.TextSecondary) return ColorRole::TextSecondary;
00198         if (col == p.TextMuted) return ColorRole::TextMuted;
00199         if (col == p.Separator) return ColorRole::Separator;
00200         if (col == p.Danger) return ColorRole::Danger;
00201     }
00202     return ColorRole::None;
00203 }
00204
00205 void UIColors::UpdateUI(wxWindow* win) {
00206     if (!win || !Current) return;
00207
00208     ColorRole bgRole = GetRoleFromColour(win->GetBackgroundColour());
00209     ColorRole fgRole = GetRoleFromColour(win->GetForegroundColour());
00210
00211     // Background
00212     switch (bgRole) {
00213         case ColorRole::Background: win->SetBackgroundColour(Current->Background); break;
00214         case ColorRole::Surface: win->SetBackgroundColour(Current->Surface); break;
00215         case ColorRole::SurfaceRaised: win->SetBackgroundColour(Current->SurfaceRaised); break;
00216         case ColorRole::Accent: win->SetBackgroundColour(Current->Accent); break;
00217         case ColorRole::AccentHover: win->SetBackgroundColour(Current->AccentHover); break;
00218         case ColorRole::Danger: win->SetBackgroundColour(Current->Danger); break;
00219         case ColorRole::Separator: win->SetBackgroundColour(Current->Separator); break;
00220         default: break; // Keep original color if no role matched

```

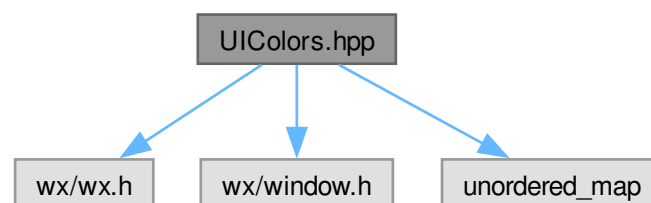
```
00221     }
00222
00223     // Foreground
00224     switch (fgRole) {
00225         case ColorRole::TextPrimary:    win->SetForegroundColour(Current->TextPrimary); break;
00226         case ColorRole::TextSecondary:  win->SetForegroundColour(Current->TextSecondary); break;
00227         case ColorRole::TextMuted:      win->SetForegroundColour(Current->TextMuted); break;
00228         case ColorRole::Accent:         win->SetForegroundColour(Current->Accent); break;
00229         case ColorRole::Danger:         win->SetForegroundColour(Current->Danger); break;
00230         default: break;
00231     }
00232
00233     win->Refresh();
00234     win->Update();
00235
00236     // Generic check for themed elements (.svg elements)
00237     if (auto* themed = dynamic_cast<IThemedElement*>(win)) {
00238         themed->OnThemeUpdate();
00239     }
00240
00241     // Recurse through children
00242     for (auto* child : win->GetChildren()) {
00243         UpdateUI(child);
00244     }
00245 }
```

7.181 UIColors.hpp File Reference

Colour palletes to use throughout the GUI.

```
#include <wx/wx.h>
#include <wx/window.h>
#include <unordered_map>
```

Include dependency graph for UIColors.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class `IThemedElement`
Interface for any UI component that needs custom refresh logic (like icons).
- struct `Palette`
`Palette` class declaration.
- class `UIColors`

Enumerations

- enum class **ColorRole** {
Background , **Surface** , **SurfaceRaised** , **Accent** ,
AccentHover , **TextPrimary** , **TextSecondary** , **TextMuted** ,
Separator , **Danger** , **None** }

7.181.1 Detailed Description

Colour palletes to use throughout the GUI.

Author

Xavier Luseti
 Jasmine Kumar

All wxColour constants live here so the visual style is consistent and easy to retheme.

Palette:

Name / Usage Background, Window / page background Surface / Card / panel backgrounds SurfaceRaised / Inputs, raised elements Accent / Buttons AccentHover / Lighter purple for hover feedback TextPrimary / Body text TextSecondary / Secondary / caption text TextMuted / Placeholder / disabled text Separator / Rule lines Danger / Destructive-action buttons

Definition in file [UIColors.hpp](#).

7.181.2 Enumeration Type Documentation

7.181.2.1 ColorRole

```
enum class ColorRole [strong]
```

Definition at line 31 of file [UIColors.hpp](#).

7.182 UIColors.hpp

[Go to the documentation of this file.](#)

```
00001
00024
00025 #pragma once
00026
00027 #include <wx/wx.h>
00028 #include <wx/window.h>
00029 #include <unordered_map>
00030
00031 enum class ColorRole {
00032     Background,
00033     Surface,
00034     SurfaceRaised,
00035     Accent,
00036     AccentHover,
00037     TextPrimary,
00038     TextSecondary,
00039     TextMuted,
00040     Separator,
```

```

00041     Danger,
00042
00043     // If color doesn't match our palette return nothing
00044     None
00045 };
00046
00050 class IThemedElement {
00051 public:
00052     virtual ~IThemedElement() = default;
00053     virtual void OnThemeUpdate() = 0; // Every class implements its own refresh logic
00054 };
00055
00060 struct Palette {
00061     wxString Name; // The name of the colour theme associated with this palette.
00062
00063     // Palette has attribute ^colour roles^
00064     wxColour Background;
00065     wxColour Surface;
00066     wxColour SurfaceRaised;
00067     wxColour Accent;
00068     wxColour AccentHover;
00069     wxColour TextPrimary;
00070     wxColour TextSecondary;
00071     wxColour TextMuted;
00072     wxColour Separator;
00073     wxColour Danger;
00074 };
00075
00076 class UIColors {
00077 public:
00078     // The dictionary storing all themes loaded from the text file (themes.txt).
00079     static std::unordered_map<wxString, Palette> ThemeMap;
00080
00081     // An ordered list of theme string names
00082     static std::vector<wxString> ThemeOrder;
00083
00084     // The current active palette pointer
00085     static Palette* Current;
00086
00087     // Static Getter Methods used to get colours across the app.
00088     static wxColour Background() { if (!Current) SetTheme("Dark Mode"); return Current->Background; }
00089     static wxColour Surface() { if (!Current) SetTheme("Dark Mode"); return Current->Surface; }
00090     static wxColour SurfaceRaised() { if (!Current) SetTheme("Dark Mode"); return
Current->SurfaceRaised; }
00091     static wxColour Accent() { if (!Current) SetTheme("Dark Mode"); return Current->Accent; }
00092     static wxColour AccentHover() { if (!Current) SetTheme("Dark Mode"); return
Current->AccentHover; }
00093     static wxColour TextPrimary() { if (!Current) SetTheme("Dark Mode"); return
Current->TextPrimary; }
00094     static wxColour TextSecondary() { if (!Current) SetTheme("Dark Mode"); return
Current->TextSecondary; }
00095     static wxColour TextMuted() { if (!Current) SetTheme("Dark Mode"); return Current->TextMuted; }
00096     static wxColour Separator() { if (!Current) SetTheme("Dark Mode"); return Current->Separator; }
00097     static wxColour Danger() { if (!Current) SetTheme("Dark Mode"); return Current->Danger; }
00098
00099
00100
00106     static bool LoadThemesFromFile(const wxString& filepath);
00107
00112     static void SetTheme(const wxString& themeName);
00113
00114
00122     static wxButton* MakeIconButton(wxWindow* parent, const wxString& iconName,
ColorRole bgRole, int iconSize);
00123
00124
00132     static wxBitmapBundle GetIcon(const wxString& iconName, ColorRole role, int size);
00133
00138     static void UpdateUI(wxWindow* win);
00139
00140 private:
00146     static ColorRole GetRoleFromColour(const wxColour& col);
00147 };

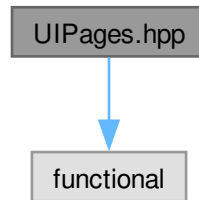
```

7.183 UIPages.hpp File Reference

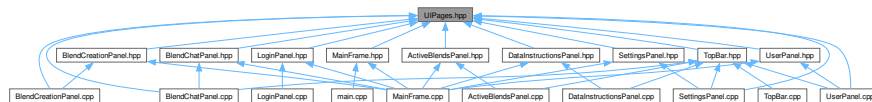
Page enum and [NavigateFn](#) callback type for the [MainFrame](#) page switcher.

```
#include <functional>
```

Include dependency graph for UIPages.hpp:



This graph shows which files directly or indirectly include this file:



Typedefs

- using [NavigateFn](#) = std::function<void([Page](#))>
Callback type passed to every sub-panel for initiating page transitions.

Enumerations

- enum class [Page](#) {
[LOGIN](#) = 0 , [DATA_INSTRUCTIONS](#) = 1 , [BLEND_CREATION](#) = 2 , [HOME](#) = 3 ,
[USER](#) = 4 , [SETTINGS](#) = 5 , [BLEND_CHAT](#) = 6 , [ACTIVE_BLENDS](#) = 7 }
All top-level pages in the [MainFrame](#) wxSimplebook.

7.183.1 Detailed Description

[Page](#) enum and [NavigateFn](#) callback type for the [MainFrame](#) page switcher.

Author

Jasmine Kumar

Defines every top-level page hosted in the [MainFrame](#) wxSimplebook and the [NavigateFn](#) callback type passed to sub-panels for page switching.

Pages must be added to the wxSimplebook in the same order as the [Page](#) enum values (0, 1, 2, ...). Sub-panels call `m_nav(Page::TARGET)` to navigate without depending on [MainFrame](#) directly.

7.183.1.0.1 Adding a new page

1. Add a new enumerator at the end of [Page](#) (keep values contiguous).
2. Create and add the corresponding wxPanel in `MainFrame::BuildPages()` at the matching position.
3. If the page needs reset/reload on arrival, add the call in `MainFrame::NavigateTo()`.

Definition in file [UIPages.hpp](#).

7.183.2 Typedef Documentation

7.183.2.1 NavigateFn

```
using NavigateFn = std::function<void(Page)>
```

Callback type passed to every sub-panel for initiating page transitions.

Call `m_nav(Page : : SOME_PAGE)` to trigger a page switch in [MainFrame](#).

Definition at line 45 of file [UIPages.hpp](#).

7.183.3 Enumeration Type Documentation

7.183.3.1 Page

```
enum class Page [strong]
```

All top-level pages in the [MainFrame](#) wxSimplebook.

Numeric values must match the order panels are added in `MainFrame::BuildPages()`.

Enumerator

LOGIN	Sign-in / register screen.
DATA_INSTRUCTIONS	Shown after login when the user has no YouTube data.
BLEND_CREATION	Add participants and create a new blend.
HOME	Blend feed (3x2 grid + refresh).
USER	Account info and logout.
SETTINGS	Settings (stub - back button only).
BLEND_CHAT	Chat tied to the active blend.
ACTIVE_BLENDS	List of blends the user participates in.

Definition at line 29 of file [UIPages.hpp](#).

7.186 UserPanel.cpp

[Go to the documentation of this file.](#)

```

00001
00006
00007 // =====
00008 // UserPanel.cpp - Account info and logout screen implementation
00009 // =====
00010
00011 #include "UserPanel.hpp"
00012
00013 #include <wx/statline.h>
00014 #include <wx/textctrl.h>
00015 #include <wx/filedlg.h>
00016
00017 #include "UIColors.hpp"
00018 #include "ButtonsConfig.hpp"
00019 #include "UIPages.hpp"
00020 #include "TopBar.hpp"
00021 #include "ConfirmationDialog.hpp"
00022 #include "../AppLayer/AppController.hpp"
00023 #include "../AppLayer/AppState.hpp"
00024 #include "../AppLayer/EventRouter.hpp"
00025 #include "../ModelLayer/User.hpp"
00026
00027 // -- Construction -----
00028
00029 UserPanel::UserPanel(wxWindow* parent, AppController& controller, NavigateFn nav)
00030 : wxPanel(parent, wxID_ANY, wxDefaultPosition, wxDefaultSize, wxTRANSPARENT_WINDOW)
00031 , m_controller(controller)
00032 , m_nav(std::move(nav))
00033 {
00034     auto* root = new wxBoxSizer(wxVERTICAL);
00035
00036     // -- TopBar -----
00037     auto* topBar = new TopBar(this, "My Account", m_nav, Page::HOME);
00038     root->Add(topBar, 0, wxEXPAND);
00039
00040     // -- User info section -----
00041     auto* infoPanel = new wxPanel(this, wxID_ANY);
00042     infoPanel->SetBackgroundColour(UIColors::Surface());
00043     auto* infoSizer = new wxBoxSizer(wxVERTICAL);
00044
00045     m_usernameLabel = new wxStaticText(infoPanel, wxID_ANY, "Username: Not logged in");
00046     m_usernameLabel->SetForegroundColour(UIColors::TextPrimary());
00047     wxFont uf = m_usernameLabel->GetFont(); // grab a font
00048     uf.SetPointSize(24);
00049     m_usernameLabel->SetFont(uf);
00050
00051     m_emailLabel = new wxStaticText(infoPanel, wxID_ANY, "Email: ");
00052     m_emailLabel->SetForegroundColour(UIColors::TextSecondary());
00053     uf = m_emailLabel->GetFont();
00054     uf.SetPointSize(16);
00055     m_emailLabel->SetFont(uf);
00056
00057     // -- Divider -----
00058     auto* divider = new wxStaticLine(this, wxID_ANY);
00059     divider->SetBackgroundColour(UIColors::Separator());
00060     root->Add(divider, 0, wxEXPAND);
00061
00062     // -- Spacer pushes buttons to bottom -----
00063     root->AddStretchSpacer(2);
00064
00065     infoSizer->Add(m_usernameLabel, 0, wxCENTER | wxTOP, 16);
00066     infoSizer->Add(m_emailLabel, 0, wxCENTER | wxBOTTOM, 16);
00067     infoPanel->SetSizer(infoSizer);
00068     root->Add(infoPanel, 0, wxCENTER | wxBOTTOM);
00069
00070     root->AddStretchSpacer(1); // space out the buttons and the labels
00071
00072     // Re-upload YouTube data
00073     auto* reuploadBtn = new wxButton(this, wxID_ANY, "Re-upload YouTube Data");
00074     UIButtons::ApplySizeBounds(reuploadBtn, ButtonType::Large);
00075     reuploadBtn->SetBackgroundColour(UIColors::Accent());
00076     reuploadBtn->SetForegroundColour(UIColors::TextPrimary());
00077     root->Add(reuploadBtn, 0, wxEXPAND | wxCENTER | wxBOTTOM, 16);
00078
00079     // Delete Account
00080     auto* del_acc = new wxButton(this, wxID_ANY, "Delete Account");
00081     UIButtons::ApplySizeBounds(del_acc, ButtonType::Large);
00082     del_acc->SetBackgroundColour(UIColors::Danger());
00083     del_acc->SetForegroundColour(UIColors::TextPrimary());
00084     root->Add(del_acc, 0, wxEXPAND | wxCENTER | wxBOTTOM, 16); // keep the button in the center
00085
00086     m_deletePasswordField = new wxTextCtrl(this, wxID_ANY, "", wxDefaultPosition,

```

```

00087         wxDefaultSize, wxTE_PASSWORD | wxTE_PROCESS_ENTER);
00088     m_deletePasswordField->SetHint("Re-enter password to delete account");
00089     m_deletePasswordField->SetBackgroundColour(UIColors::SurfaceRaised());
00090     m_deletePasswordField->SetForegroundColour(UIColors::TextPrimary());
00091     root->Add(m_deletePasswordField, 0, wxEXPAND | wxCENTER | wxBOTTOM, 8);
00092
00093     m_confirmDeleteBtn = new wxButton(this, wxID_ANY, "Confirm Delete");
00094     UIButtons::ApplySizeBounds(m_confirmDeleteBtn, ButtonType::Large);
00095     m_confirmDeleteBtn->SetBackgroundColour(UIColors::Danger());
00096     m_confirmDeleteBtn->SetForegroundColour(UIColors::TextPrimary());
00097     root->Add(m_confirmDeleteBtn, 0, wxEXPAND | wxCENTER | wxBOTTOM, 8);
00098
00099     m_deleteErrorLabel = new wxStaticText(this, wxID_ANY, "");
00100     m_deleteErrorLabel->SetForegroundColour(UIColors::Danger());
00101     root->Add(m_deleteErrorLabel, 0, wxEXPAND | wxCENTER | wxBOTTOM, 12);
00102
00103     // -- Logout -----
00104     auto* logoutBtn = new wxButton(this, wxID_ANY, "Log Out");
00105     UIButtons::ApplySizeBounds(logoutBtn, ButtonType::Large);
00106     logoutBtn->SetBackgroundColour(UIColors::Danger());
00107     logoutBtn->SetForegroundColour(UIColors::TextPrimary());
00108     root->Add(logoutBtn, 0, wxEXPAND | wxCENTER | wxBOTTOM, 16); // keep the button in the bottom
center as well
00109
00110     // add some extra padding space beneath the buttons
00111     root->AddStretchSpacer(1);
00112
00113     SetSizer(root);
00114
00115     reuploadBtn->Bind(wxEVT_BUTTON, &UserPanel::OnReuploadCSV, this);
00116     del_acc->Bind(wxEVT_BUTTON, &UserPanel::OnDeleteRequest, this);
00117     m_confirmDeleteBtn->Bind(wxEVT_BUTTON, &UserPanel::OnConfirmDelete, this);
00118     m_deletePasswordField->Bind(wxEVT_TEXT_ENTER,
00119         &UserPanel::OnConfirmDelete, this);
00120     logoutBtn->Bind(wxEVT_BUTTON, &UserPanel::OnLogout, this);
00121     Bind(wxEVT_SHOW, &UserPanel::OnShow, this);
00122
00123     SetDeleteReauthVisible(false);
00124 }
00125
00126 // -- Private helpers -----
00127
00128 void UserPanel::Refresh()
00129 {
00130     RefreshUserInfo();
00131 }
00132
00133 void UserPanel::RefreshUserInfo()
00134 {
00135     SetDeleteReauthVisible(false);
00136
00137     User* user = AppState::getInstance().getCurrentUser();
00138     if (user) {
00139         m_usernameLabel->SetLabel("Username: " +
00140             wxString::FromUTF8(user->getUsername()));
00141
00142         // verify email presence
00143         std::string tmp = user->getEmail();
00144         std::string email_str;
00145         if (tmp == "") {email_str = "no provided email at registration...";}
00146         else {email_str = tmp;}
00147
00148         m_emailLabel->SetLabel("Email: " +
00149             wxString::FromUTF8(email_str));
00150     } else {
00151         m_usernameLabel->SetLabel("Not logged in");
00152         m_emailLabel->SetLabel("");
00153     }
00154     Layout();
00155 }
00156
00157 void UserPanel::SetDeleteReauthVisible(bool isVisible)
00158 {
00159     if (!isVisible) {
00160         m_deletePasswordField->Clear();
00161         m_deleteErrorLabel->SetLabel("");
00162     }
00163
00164     m_deletePasswordField->Show(isVisible);
00165     m_confirmDeleteBtn->Show(isVisible);
00166     m_deleteErrorLabel->Show(isVisible && !m_deleteErrorLabel->GetLabel().IsEmpty());
00167     Layout();
00168 }
00169
00170 // -- Event handlers -----
00171
00172 void UserPanel::OnShow(wxShowEvent& evt)

```

```

00173 {
00174     if (evt.IsShown())
00175         Refresh();
00176     evt.Skip();
00177 }
00178
00179 void UserPanel::OnDeleteRequest(wxCommandEvent& /*evt*/)
00180 {
00181     ConfirmationDialog dlg(this,
00182         "Delete Account",
00183         "Are you sure you want to delete your account? This cannot be undone.",
00184         "Yes",
00185         "Nevermind");
00186     if (dlg.ShowModal() != wxID_OK) {
00187         SetDeleteReauthVisible(false);
00188         return;
00189     }
00190
00191     SetDeleteReauthVisible(true);
00192     m_deletePasswordField->SetFocus();
00193 }
00194
00195 void UserPanel::OnConfirmDelete(wxCommandEvent& /*evt*/)
00196 {
00197     User* user = AppState::getInstance().getCurrentUser();
00198     if (user == nullptr) {
00199         m_deleteErrorLabel->SetLabel("No active user session.");
00200         m_deleteErrorLabel->Show();
00201         Layout();
00202         return;
00203     }
00204
00205     const std::string password = m_deletePasswordField->GetValue().ToStdString();
00206     if (password.empty()) {
00207         m_deleteErrorLabel->SetLabel("Password is required to delete your account.");
00208         m_deleteErrorLabel->Show();
00209         Layout();
00210         return;
00211     }
00212
00213     m_controller.getEventRouter().dispatch(
00214         "deleteAccount",
00215         {{ "userID", user->getUserID() }, { "password", password }}
00216     );
00217
00218     if (AppState::getInstance().getCurrentUser() == nullptr) {
00219         SetDeleteReauthVisible(false);
00220         wxMessageBox("Your account has been deleted successfully.",
00221             "Account Deleted",
00222             wxOK | wxICON_INFORMATION,
00223             this);
00224         m_nav(Page::LOGIN);
00225         return;
00226     }
00227
00228     m_deletePasswordField->Clear();
00229     m_deleteErrorLabel->SetLabel("Delete failed. Check password and try again.");
00230     m_deleteErrorLabel->Show();
00231     Layout();
00232 }
00233
00234 void UserPanel::OnReuploadCSV(wxCommandEvent& /*evt*/)
00235 {
00236     User* user = AppState::getInstance().getCurrentUser();
00237     if (!user) {
00238         wxMessageBox("No active user session.", "Error", wxOK | wxICON_ERROR, this);
00239         return;
00240     }
00241
00242     wxFileDialog dlg(this, "Select YouTube Export Data", "", "",
00243         "Supported files (*.csv;*.html;*.htm)|*.csv;*.html;*.htm|"
00244         "CSV files (*.csv)|*.csv|"
00245         "HTML files (*.html;*.htm)|*.html;*.htm|"
00246         "All files (*.*)|*.*",
00247         wxFD_OPEN | wxFD_FILE_MUST_EXIST);
00248     if (dlg.ShowModal() != wxID_OK) return;
00249
00250     std::string filePath = dlg.GetPath().ToStdString();
00251
00252     m_controller.getEventRouter().dispatch("uploadData",
00253         {{ "filePath", filePath }, { "userID", user->getUserID() }});
00254
00255     std::string uploadError = AppState::getInstance().takePendingUploadError();
00256     if (!uploadError.empty()) {
00257         wxMessageBox(uploadError, "Upload Failed", wxOK | wxICON_ERROR, this);
00258         return;
00259     }

```

```

00260
00261     wxMessageBox("Your YouTube data has been re-uploaded.\n"
00262                 "Active blends will be updated with your new data.",
00263                 "Data Updated", wxOK | wxICON_INFORMATION, this);
00264 }
00265
00266 void UserPanel::OnLogout(wxCommandEvent& /*evt*/)
00267 {
00268     bool confirmed = ConfirmationDialog::Confirm(this, "Log Out", "Log out?");
00269     if (!confirmed) return;
00270
00271     m_controller.getEventRouter().dispatch("logout", {});
00272     m_nav(Page::LOGIN);
00273 }

```

7.187 UserPanel.hpp File Reference

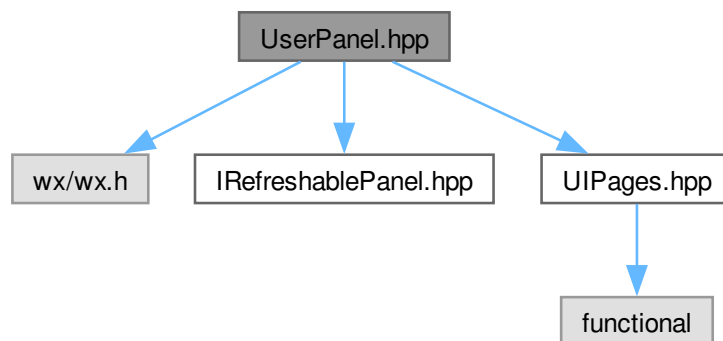
Account info and logout panel.

```

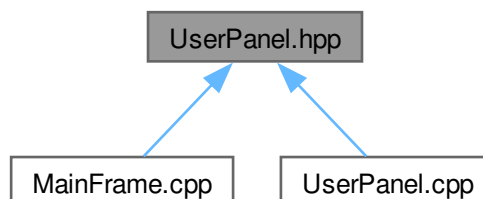
#include <wx/wx.h>
#include "IRefreshablePanel.hpp"
#include "UIPages.hpp"

```

Include dependency graph for UserPanel.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [UserPanel](#)

Panel for user account display and account actions.

7.187.1 Detailed Description

Account info and logout panel.

Author

Jasmine Kumar

A panel that shows the current user's username and email, plus logout and account-deletion controls.

7.187.1.1 LAYOUT

[TopBar](#) ("My Account", back -> HOME) Info panel with username and email labels Separator and spacer Action buttons (logout and delete-account flow)

7.187.1.2 REFRESH ON SHOW

RefreshUserInfo() runs whenever the panel is shown (wxEVT_SHOW) so labels always reflect current [AppState](#).

7.187.1.3 LOGOUT

OnLogout: [ConfirmationDialog](#) -> dispatch("logout") -> nav(LOGIN).

7.187.1.4 FUTURE FEATURES

TODO: Add a "Change Password" form. TODO: Show YouTube data status and a CSV re-upload action. TODO: Display blend history.

Definition in file [UserPanel.hpp](#).

7.188 UserPanel.hpp

[Go to the documentation of this file.](#)

```

00001
00031
00032 #pragma once
00033
00034 #include <wx/wx.h>
00035
00036 #include "IRefreshablePanel.hpp"
00037 #include "UIPages.hpp"
00038
00039 class AppController;
00040 class wxStaticText;
00041 class wxTextCtrl;
00042 class wxButton;
00043
00048 class UserPanel : public wxPanel, public IRefreshablePanel {
00049 public:
00056     UserPanel(wxWindow* parent, AppController& controller, NavigateFn nav);
00057
00061     void Refresh() override;
00062
00063 private:
00064     AppController& m_controller;
00065     NavigateFn      m_nav;
00066
00068     wxStaticText* m_usernameLabel;
00069     wxStaticText* m_emailLabel;
00070     wxTextCtrl*   m_deletePasswordField;
00071     wxStaticText* m_deleteErrorLabel;
00072     wxButton*     m_confirmDeleteBtn;
00073
00075     void RefreshUserInfo();
00076
00078     void SetDeleteReauthVisible(bool isVisible);
00079
00081     void OnDeleteRequest(wxCommandEvent& evt);
00083     void OnConfirmDelete(wxCommandEvent& evt);
00085     void OnReuploadCSV(wxCommandEvent& evt);
00087     void OnLogout      (wxCommandEvent& evt);
00089     void OnShow        (wxShowEvent&    evt);
00090 };

```

7.189 VideoCard.cpp File Reference

Implementation for [VideoCard](#).

```

#include "VideoCard.hpp"
#include <algorithm>
#include <thread>
#include <wx/dcbuffer.h>
#include <wx/graphics.h>
#include <wx/mstream.h>
#include <wx/image.h>
#include <wx/settings.h>
#include <wx/utils.h>
#include <curl/curl.h>
#include "UIColors.hpp"

```


The graph illustrates the dependencies between various Java classes. The classes are organized into layers, with 'WebContextImpl' at the top. Below it are 'ServletContext' (with sub-nodes 'signature', 'Printer', 'writeAttribute', 'writeMessage', 'writePage', 'writeSettings', 'writeValue') and 'ServletContextImpl'. The middle layer contains 'HttpContextImpl', 'HttpServletResponseImpl', 'HttpServletRequestImpl', 'HttpSessionImpl', 'HttpServletResponseWrapperImpl', 'HttpServletRequestWrapperImpl', 'HttpSessionWrapperImpl', 'HttpServletResponseWrapperImpl', 'HttpServletRequestWrapperImpl', 'HttpSessionWrapperImpl'. The bottom layer contains 'HttpServletResponseImpl', 'HttpServletRequestImpl', 'HttpSessionImpl', 'HttpServletResponseWrapperImpl', 'HttpServletRequestWrapperImpl', 'HttpSessionWrapperImpl'. Arrows indicate dependencies between these classes.

- **wxDEFINE_EVENT** (EVT_THUMBNAIL_LOADED, [ThumbnailEvent](#))
- **wxDEFINE_EVENT** (EVT_CHANNEL_LOGO_LOADED, [ThumbnailEvent](#))
- static double [LineHeight](#) (wxGraphicsContext *gc)

Implementation for VideoCard.

Jasmine Kumar

Definition in file [VideoCard.cpp](#).

7.189.2.1 LineHeight()

```
double LineHeight (
    wxGraphicsContext * gc) [static]
```

Definition at line 218 of file VideoCard.cpp.

7.190 VideoCard.cpp

[Go to the documentation of this file.](#)

```

00001
00006
00007 // =====
00008 // VideoCard.cpp - Custom video preview card widget implementation
00009 // =====
00010
00011 #include "VideoCard.hpp"
00012
00013 #include <algorithm>
00014 #include <thread>
00015
00016 #include <wx/dcbuffer.h>
00017 #include <wx/graphics.h>
00018 #include <wx/mstream.h>
00019 #include <wx/image.h>
00020 #include <wx/settings.h> // wxSystemSettings
00021 #include <wx/utils.h>    // wxLaunchDefaultBrowser
00022
00023 #include <curl/curl.h>
00024
00025 #include "UIColors.hpp"
00026
00027 // -----
00028 // Event definitions
00029 // -----
00030 wxDEFINE_EVENT(EVT_THUMBNAIL_LOADED, ThumbnailEvent);
00031 wxDEFINE_EVENT(EVT_CHANNEL_LOGO_LOADED, ThumbnailEvent);
00032
00033 // -----
00034 // Internal curl helper - downloads a URL into memory and returns a wxImage.
00035 // Called only from background threads.
00036 // -----
00037 namespace {
00038
00039 size_t curlWrite(void* ptr, size_t size, size_t nmemb, std::string* out) {
00040     out->append(static_cast<char*>(ptr), size * nmemb);
00041     return size * nmemb;
00042 }
00043
00044 wxImage fetchImageViaCurl(const std::string& url) {
00045     CURL* curl = curl_easy_init();
00046     if (!curl) return wxImage();
00047
00048     std::string buf;
00049     curl_easy_setopt(curl, CURLOPT_URL, url.c_str());
00050     curl_easy_setopt(curl, CURLOPT_WRITEFUNCTION, curlWrite);
00051     curl_easy_setopt(curl, CURLOPT_WRITEDATA, &buf);
00052     curl_easy_setopt(curl, CURLOPT_FOLLOWLOCATION, 1L);
00053     curl_easy_setopt(curl, CURLOPT_TIMEOUT, 15L);
00054
00055     CURLcode res = curl_easy_perform(curl);
00056     curl_easy_cleanup(curl);
00057
00058     if (res != CURLE_OK || buf.empty()) return wxImage();
00059
00060     wxMemoryInputStream stream(buf.data(), buf.size());
00061     wxImage img;
00062     // wxBITMAP_TYPE_ANY lets wxWidgets detect JPEG / PNG / WebP automatically
00063     img.LoadFile(stream, wxBITMAP_TYPE_ANY);
00064     return img;
00065 }
00066
00067 } // namespace
00068
00069 // -----
00070 // Constructor
00071 // -----
00072 VideoCard::VideoCard(wxWindow* parent, AppController& controller)
00073 : wxPanel(parent, wxID_ANY, wxDefaultPosition, wxSize(260, 250))
00074 , m_controller(controller)
00075 {
00076     SetBackgroundColour(UIColors::Surface());
00077     SetBackgroundStyle(wxBG_STYLE_PAINT);
00078
00079     Bind(wxEVT_PAINT, &VideoCard::OnPaint, this);
00080     Bind(wxEVT_ENTER_WINDOW, &VideoCard::OnMouseEnter, this);
00081     Bind(wxEVT_LEAVE_WINDOW, &VideoCard::OnMouseLeave, this);
00082     Bind(wxEVT_LEFT_UP, &VideoCard::OnClick, this);
00083     Bind(EVT_THUMBNAIL_LOADED, &VideoCard::OnThumbnailLoaded, this);
00084     Bind(EVT_CHANNEL_LOGO_LOADED, &VideoCard::OnChannelLogoLoaded, this);
00085
00086     if (!wxImage::FindHandler(wxBITMAP_TYPE_JPEG))

```

```

00087         wxImage::AddHandler(new wxJPEGHandler());
00088         if (!wxImage::FindHandler(wxBITMAP_TYPE_PNG))
00089             wxImage::AddHandler(new wxPNGHandler());
00090     }
00091
00092     // -----
00093     // Public API
00094     // -----
00095     void VideoCard::SetVideo(const Video& video) {
00096         m_videoID = wxString::FromUTF8(video.getVideoID());
00097         m_title = wxString::FromUTF8(video.getTitle());
00098         m_channelID = wxString::FromUTF8(video.getChannelID());
00099         m_channelName = wxString::FromUTF8(video.getChannelName());
00100         m_thumbnailURL = wxString::FromUTF8(video.getThumbnailURL());
00101         m_channelLogoURL = wxString::FromUTF8(video.getChannelLogoURL());
00102
00103         m_thumbnail = wxBitmap();
00104         m_thumbLoading = false; m_thumbFailed = false;
00105
00106         m_channelLogo = wxBitmap();
00107         m_logoLoading = false; m_logoFailed = false;
00108
00109         FetchThumbnailAsync();
00110         FetchChannelLogoAsync();
00111         Refresh();
00112     }
00113
00114     void VideoCard::Clear() {
00115         m_videoID = m_title = m_channelID = m_channelName
00116             = m_thumbnailURL = m_channelLogoURL = wxEmptyString;
00117
00118         m_thumbnail = wxBitmap();
00119         m_thumbLoading = m_thumbFailed = false;
00120
00121         m_channelLogo = wxBitmap();
00122         m_logoLoading = m_logoFailed = false;
00123
00124         Refresh();
00125     }
00126
00127     // -----
00128     // Async fetches
00129     // -----
00130     void VideoCard::FetchThumbnailAsync() {
00131         if (m_videoID.IsEmpty()) return;
00132
00133         m_thumbLoading = true;
00134         ++m_thumbGen;
00135
00136         // Build the URL: prefer the stored thumbnailURL, fall back to YouTube CDN
00137         std::string url;
00138         if (!m_thumbnailURL.IsEmpty()) {
00139             url = std::string(m_thumbnailURL.ToUTF8());
00140         } else {
00141             url = "https://img.youtube.com/vi/" +
00142                 std::string(m_videoID.ToUTF8()) + "/hqdefault.jpg";
00143         }
00144
00145         VideoCard* card = this;
00146         int gen = m_thumbGen;
00147
00148         std::thread([url, card, gen]() {
00149             wxImage img = fetchImageViaCurl(url);
00150             bool ok = img.IsOk();
00151             auto* evt = new ThumbnailEvent(EVT_THUMBNAIL_LOADED, wxID_ANY,
00152                 std::move(img), ok, gen);
00153             wxQueueEvent(card, evt);
00154         }).detach();
00155     }
00156
00157     void VideoCard::FetchChannelLogoAsync() {
00158         if (m_channelLogoURL.IsEmpty()) return;
00159
00160         m_logoLoading = true;
00161         ++m_logoGen;
00162
00163         std::string url = std::string(m_channelLogoURL.ToUTF8());
00164         VideoCard* card = this;
00165         int gen = m_logoGen;
00166
00167         std::thread([url, card, gen]() {
00168             wxImage img = fetchImageViaCurl(url);
00169             bool ok = img.IsOk();
00170             auto* evt = new ThumbnailEvent(EVT_CHANNEL_LOGO_LOADED, wxID_ANY,
00171                 std::move(img), ok, gen);
00172             wxQueueEvent(card, evt);
00173         }).detach();

```

```

00174 }
00175
00176 // -----
00177 // Event handlers
00178 // -----
00179 void VideoCard::OnThumbnailLoaded(ThumbnailEvent& evt) {
00180     if (evt.generation != m_thumbGen) return;
00181     m_thumbLoading = false;
00182     if (evt.success && evt.image.IsOk()) {
00183         m_thumbnail = wxBitmap(evt.image);
00184         m_thumbFailed = false;
00185     } else {
00186         m_thumbFailed = true;
00187     }
00188     Refresh();
00189 }
00190
00191 void VideoCard::OnChannelLogoLoaded(ThumbnailEvent& evt) {
00192     if (evt.generation != m_logoGen) return;
00193     m_logoLoading = false;
00194     if (evt.success && evt.image.IsOk()) {
00195         m_channelLogo = wxBitmap(evt.image);
00196         m_logoFailed = false;
00197     } else {
00198         m_logoFailed = true;
00199     }
00200     Refresh();
00201 }
00202
00203 void VideoCard::OnMouseEnter(wxMouseEvent& /*evt*/) { m_hovered = true; Refresh(); }
00204 void VideoCard::OnMouseLeave(wxMouseEvent& /*evt*/) { m_hovered = false; Refresh(); }
00205
00206 void VideoCard::OnClick(wxMouseEvent& /*evt*/) {
00207     if (m_videoID.IsEmpty()) return;
00208     wxLaunchDefaultBrowser(
00209         wxString::Format("https://www.youtube.com/watch?v=%s", m_videoID));
00210     EventPayload payload;
00211     payload["videoID"] = std::string(m_videoID.ToUTF8());
00212     m_controller.getEventRouter().dispatch("playVideo", payload);
00213 }
00214
00215 // -----
00216 // Painting helpers
00217 // -----
00218 static double LineHeight(wxGraphicsContext* gc) {
00219     double tw, th, td, tex;
00220     gc->GetTextExtent("Wg", &tw, &th, &td, &tex);
00221     return th;
00222 }
00223
00224 int VideoCard::DrawWrappedText(wxGraphicsContext* gc,
00225                                const wxString& text,
00226                                int maxLines,
00227                                double x, double y,
00228                                double maxWidth) const {
00229     if (text.IsEmpty()) return static_cast<int>(y);
00230
00231     double tw, th, td, tex;
00232     gc->GetTextExtent("Wg", &tw, &th, &td, &tex);
00233     double lineH = th + 2.0;
00234
00235     wxArrayString words = wxSplit(text, ' ');
00236     wxString currentLine;
00237     int lineCount = 0;
00238
00239     auto flush = [&](const wxString& line, bool isLast) {
00240         if (lineCount >= maxLines) return;
00241         wxString toDraw = line;
00242         gc->GetTextExtent(toDraw, &tw, &th, &td, &tex);
00243         if (!isLast || tw > maxWidth) {
00244             while (!toDraw.IsEmpty()) {
00245                 wxString candidate = toDraw + L"\u2026";
00246                 gc->GetTextExtent(candidate, &tw, &th, &td, &tex);
00247                 if (tw <= maxWidth) { toDraw = candidate; break; }
00248                 toDraw.RemoveLast();
00249             }
00250         }
00251         gc->DrawText(toDraw, x, y + lineCount * lineH);
00252         ++lineCount;
00253     };
00254
00255     for (size_t i = 0; i < words.GetCount(); ++i) {
00256         wxString candidate = currentLine.IsEmpty()
00257             ? words[i]
00258             : currentLine + " " + words[i];
00259         gc->GetTextExtent(candidate, &tw, &th, &td, &tex);
00260         if (tw > maxWidth && !currentLine.IsEmpty()) {

```

```

00261         flush(currentLine, false);
00262         if (lineCount >= maxLines) goto done;
00263         currentLine = words[i];
00264     } else {
00265         currentLine = candidate;
00266     }
00267 }
00268 if (!currentLine.IsEmpty()) flush(currentLine, true);
00269 done:
00270 return static_cast<int>(y + lineCount * lineH);
00271 }
00272
00273 // -----
00274 // OnPaint
00275 // -----
00276 void VideoCard::OnPaint(wxPaintEvent& /*evt*/) {
00277     wxAutoBufferedPaintDC dc(this);
00278     wxGraphicsContext* gc = wxGraphicsContext::Create(dc);
00279     if (!gc) return;
00280
00281     const wxSize sz = GetClientSize();
00282     const double W = sz.GetWidth();
00283     const double H = sz.GetHeight();
00284
00285     // -- Card background -----
00286     gc->SetBrush(wxBrush(UIColors::Surface()));
00287     gc->SetPen(*wxTRANSPARENT_PEN);
00288     gc->DrawRoundedRectangle(0, 0, W, H, 10.0);
00289
00290     // -- Thumbnail area - 16:9 capped at 145 px so the text area never
00291     // gets squeezed out when the grid sizer stretches cards wider.
00292     const double thumbH = std::min(W * 9.0 / 16.0, 300.0);
00293
00294     if (m_thumbnail.IsOk()) {
00295         wxImage scaled = m_thumbnail.ConvertToImage()
00296             .Scale(static_cast<int>(W),
00297                 static_cast<int>(thumbH),
00298                 wxIMAGE_QUALITY_BILINEAR);
00299         gc->DrawBitmap(wxBitmap(scaled), 0, 0, W, thumbH);
00300     } else {
00301         gc->SetBrush(wxBrush(UIColors::SurfaceRaised()));
00302         gc->SetPen(*wxTRANSPARENT_PEN);
00303         gc->DrawRectangle(0, 0, W, thumbH);
00304
00305         wxString label;
00306         if (m_thumbLoading) label = "Loading...";
00307         else if (m_thumbFailed) label = L"\u25B6";
00308         // else: no video set - blank
00309
00310         if (!label.IsEmpty()) {
00311             wxFont lf = wxSystemSettings::GetFont(wxSYS_DEFAULT_GUI_FONT);
00312             lf.SetPointSize(m_thumbFailed ? 26 : 11);
00313             gc->SetFont(lf, UIColors::TextMuted());
00314             double tw, th, td, tex;
00315             gc->GetTextExtent(label, &tw, &th, &td, &tex);
00316             gc->DrawText(label, (W - tw) / 2.0, (thumbH - th) / 2.0);
00317         }
00318     }
00319
00320     // -- Title -----
00321     const double padX = 8.0;
00322     const double textW = W - padX * 2.0;
00323     double curY = thumbH + 8.0;
00324
00325     if (!m_title.IsEmpty()) {
00326         wxFont titleFont = wxSystemSettings::GetFont(wxSYS_DEFAULT_GUI_FONT);
00327         titleFont.SetPointSize(10);
00328         titleFont.SetWeight(wxFONTWEIGHT_SEMIBOLD);
00329         gc->SetFont(titleFont, UIColors::TextPrimary());
00330         curY = DrawWrappedText(gc, m_title, 2, padX, curY, textW);
00331         curY += 4.0;
00332     }
00333
00334     // -- Channel row: round logo + channel name -----
00335     // Only render when we have at least a channel name or ID to show.
00336     // Avoids an orphaned grey circle for videos not yet enriched by the API.
00337     wxString chanLabel = m_channelName.IsEmpty() ? m_channelID : m_channelName;
00338     const bool hasChannelData = !chanLabel.IsEmpty() || m_channelLogo.IsOk()
00339         || m_logoLoading;
00340
00341     if (hasChannelData) {
00342         const double logoSize = 24.0;
00343         const double rowH = logoSize;
00344
00345         if (m_channelLogo.IsOk()) {
00346             // Punch a circle into the alpha channel - renders as a disc.
00347             int sz = static_cast<int>(logoSize);

```

```

00348         wxImage scaled = m_channelLogo.ConvertToImage()
00349                     .Scale(sz, sz, wxIMAGE_QUALITY_BILINEAR);
00350         if (!scaled.HasAlpha()) scaled.InitAlpha();
00351
00352         const double cx = sz / 2.0, cy = sz / 2.0, r = sz / 2.0;
00353         for (int py = 0; py < sz; ++py)
00354             for (int px = 0; px < sz; ++px) {
00355                 double dx = px + 0.5 - cx, dy = py + 0.5 - cy;
00356                 if (dx * dx + dy * dy > r * r)
00357                     scaled.SetAlpha(px, py, wxALPHA_TRANSPARENT);
00358             }
00359         gc->DrawBitmap(wxBitmap(scaled), padX, curY, logoSize, logoSize);
00360     } else {
00361         // Placeholder circle (shown while logo is loading)
00362         gc->SetBrush(wxBrush(UIColors::SurfaceRaised()));
00363         gc->SetPen(*wxTRANSPARENT_PEN);
00364         gc->DrawEllipse(padX, curY, logoSize, logoSize);
00365     }
00366
00367     if (!chanLabel.IsEmpty()) {
00368         wxFont chanFont = wxSystemSettings::GetFont(wxSYS_DEFAULT_GUI_FONT);
00369         chanFont.SetPointSize(9);
00370         gc->SetFont(chanFont, UIColors::TextSecondary());
00371
00372         const double textX = padX + logoSize + 5.0;
00373         const double chanTextW = W - textX - padX;
00374
00375         double tw, th, td, tex;
00376         gc->GetTextExtent(chanLabel, &tw, &th, &td, &tex);
00377         if (tw > chanTextW) {
00378             while (!chanLabel.IsEmpty()) {
00379                 wxString cand = chanLabel + L"\u2026";
00380                 gc->GetTextExtent(cand, &tw, &th, &td, &tex);
00381                 if (tw <= chanTextW) { chanLabel = cand; break; }
00382                 chanLabel.RemoveLast();
00383             }
00384         }
00385         gc->DrawText(chanLabel, textX, curY + (rowH - th) / 2.0);
00386     }
00387 }
00388
00389 // -- Hover border -----
00390 if (m_hovered) {
00391     gc->SetBrush(*wxTRANSPARENT_BRUSH);
00392     gc->SetPen(wxPen(UIColors::Accent(), 3));
00393     gc->DrawRoundedRectangle(1.5, 1.5, W - 3.0, H - 3.0, 14.0);
00394 }
00395
00396 delete gc;
00397 }

```

7.191 VideoCard.hpp File Reference

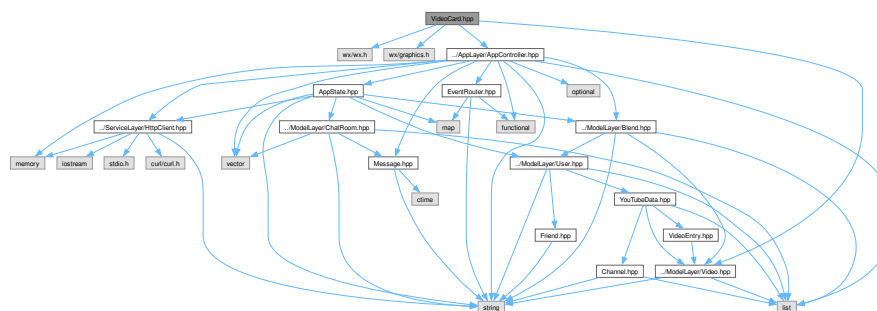
Custom wxPanel widget that displays a single YouTube video preview.

```

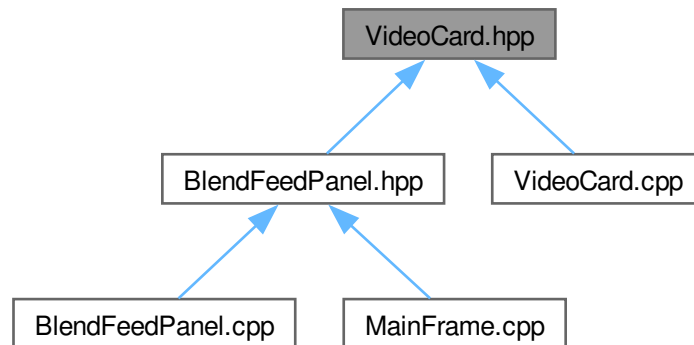
#include <wx/wx.h>
#include <wx/graphics.h>
#include "../ModelLayer/Video.hpp"
#include "../AppLayer/AppController.hpp"

```

Include dependency graph for VideoCard.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [ThumbnailEvent](#)
Event fired on the [VideoCard](#) when an async image fetch completes.
- class [VideoCard](#)
Custom wxPanel widget that displays a single YouTube video preview.

Functions

- **wxDECLARE_EVENT** (EVT_THUMBNAIL_LOADED, [ThumbnailEvent](#))
Fired when the video thumbnail is ready.
- **wxDECLARE_EVENT** (EVT_CHANNEL_LOGO_LOADED, [ThumbnailEvent](#))
Fired when the channel logo is ready.

7.191.1 Detailed Description

Custom wxPanel widget that displays a single YouTube video preview.

Author

Jasmine Kumar

Layout (top -> bottom):

- 16:9 thumbnail - fetched asynchronously via libcurl from thumbnailURL, falling back to [img.youtube.com/vi/<id>/hqdefault.jpg](#) if the URL is empty.
- [Video](#) title (up to 2 wrapped lines).
- [Channel](#) row: 24 px round channel logo + channel name. The logo is fetched asynchronously from [channelLogoURL](#).

Both async fetches use a generation counter so stale results from an earlier `SetVideo()` call are silently discarded. Clicking the card opens the video in the system browser and dispatches a "playVideo" event.

Definition in file [VideoCard.hpp](#).

7.192 VideoCard.hpp

[Go to the documentation of this file.](#)

```

00001
00017
00018 #pragma once
00019
00020 #include <wx/wx.h>
00021 #include <wx/graphics.h>
00022
00023 #include "../ModelLayer/Video.hpp"
00024 #include "../AppLayer/AppController.hpp"
00025
00037 class ThumbnailEvent : public wxEvent {
00038 public:
00039     wxImage image;
00040     bool success;
00041     int generation;
00042
00043     ThumbnailEvent(wxEventType type, int id, wxImage img, bool ok, int gen)
00044         : wxEvent(id, type), image(std::move(img)), success(ok), generation(gen) {}
00045
00046     wxEvent* Clone() const override { return new ThumbnailEvent(*this); }
00047 };
00048
00049 wxDECLARE_EVENT(EVT_THUMBNAIL_LOADED, ThumbnailEvent);
00050 wxDECLARE_EVENT(EVT_CHANNEL_LOGO_LOADED, ThumbnailEvent);
00051
00059 class VideoCard : public wxPanel {
00060 public:
00066     VideoCard(wxWindow* parent, AppController& controller);
00067
00072     void SetVideo(const Video& video);
00073
00075     void Clear();
00076
00077     wxSize DoGetBestSize() const override { return wxSize(260, 250); }
00078
00079 private:
00080     AppController& m_controller;
00081
00082     wxString m_videoID;
00083     wxString m_title;
00084     wxString m_channelID;
00085     wxString m_channelName;
00086     wxString m_thumbnailURL;
00087     wxString m_channelLogoURL;
00088
00089     wxBitmap m_thumbnail;
00090     bool m_thumbLoading = false;
00091     bool m_thumbFailed = false;
00092     int m_thumbGen = 0;
00093
00094     wxBitmap m_channelLogo;
00095     bool m_logoLoading = false;
00096     bool m_logoFailed = false;
00097     int m_logoGen = 0;
00098
00099     bool m_hovered = false;
00100
00102     void FetchThumbnailAsync();
00104     void FetchChannelLogoAsync();
00105
00106     void OnThumbnailLoaded (ThumbnailEvent& evt);
00107     void OnChannelLogoLoaded(ThumbnailEvent& evt);
00108     void OnPaint (wxPaintEvent& evt);
00109     void OnMouseEnter(wxMouseEvent& evt);
00110     void OnMouseLeave(wxMouseEvent& evt);
00111     void OnClick (wxMouseEvent& evt);
00112
00123     int DrawWrappedText (wxGraphicsContext* gc,
00124                         const wxString& text,
00125                         int maxLines,
00126                         double x, double y,
00127                         double maxWidth) const;
00128 };

```


Index

- ~AppController
 - AppController, [21](#)
- ~ChatWebSocket
 - ChatWebSocket, [68](#)
- ~CsvParser
 - CsvParser, [74](#)
- ~DataExtractor
 - DataExtractor, [79](#)
- ~FileBin
 - FileBin, [91](#)
- ~FileFormatter
 - FileFormatter, [93](#)
- ~OperationProgressDialog
 - OperationProgressDialog, [136](#)
- ~SqliteDataManager
 - SqliteDataManager, [148](#)
- Accent
 - Palette, [138](#)
 - UIColors, [157](#)
- AccentHover
 - Palette, [138](#)
 - UIColors, [157](#)
- ACTIVE_BLENDS
 - UIPages.hpp, [433](#)
- activeBlend
 - AppState, [39](#)
- ActiveBlendsPanel, [15](#)
 - ActiveBlendsPanel, [17](#)
 - m_controller, [18](#)
 - m_emptyLabel, [18](#)
 - m_listInner, [18](#)
 - m_listScroll, [18](#)
 - m_listSizer, [18](#)
 - m_nav, [18](#)
 - OnLeaveBlend, [17](#)
 - OnSelectBlend, [17](#)
 - RebuildList, [17](#)
 - Refresh, [17](#)
- activeBlendsPanel
 - MainFrame, [130](#)
- ActiveBlendsPanel.cpp, [346](#), [347](#)
- ActiveBlendsPanel.hpp, [349](#), [351](#)
- addFriend
 - User, [163](#)
- addMessage
 - ChatRoom, [64](#)
- addSessionUser
 - AppState, [34](#)
- algorithmUsed
 - Blend, [44](#)
- ALT_SUCC
 - HttpClient, [112](#)
- apiKey
 - YouTubeMetadataFetcher, [203](#)
- AppController, [19](#)
 - ~AppController, [21](#)
 - AppController, [21](#)
 - appState, [31](#)
 - clearProgressReporter, [22](#)
 - currentBlend, [31](#)
 - enrichActiveBlendFeedIfMissingMetadata, [22](#)
 - enrichIfMissingMetadata, [22](#)
 - enrichMissingMetadataSubsetMultithreaded, [22](#)
 - eventRouter, [31](#)
 - fetchUserBlends, [23](#)
 - get_current_blend_id, [23](#)
 - get_current_blend_title, [23](#)
 - get_current_email, [23](#)
 - get_current_user, [24](#)
 - get_current_username, [24](#)
 - getEventRouter, [24](#)
 - handleCreateBlend, [24](#)
 - handleDeleteAccount, [25](#)
 - handleLeaveBlend, [25](#)
 - handleLogin, [25](#)
 - handleLogout, [25](#)
 - handleOpenChat, [26](#)
 - handlePlayVideo, [26](#)
 - handleRefresh, [26](#)
 - handleRegister, [26](#)
 - handleSelectBlend, [27](#)
 - handleSendMessage, [27](#)
 - handleUploadData, [27](#)
 - http, [31](#)
 - isConnected, [31](#)
 - loadParticipantsWithWatchLater, [27](#)
 - loadParticipantsWithWatchLaterParallel, [28](#)
 - lookupUser, [28](#)
 - parseBlendID, [28](#)
 - parseChatHistory, [29](#)
 - parseParticipantIDs, [29](#)
 - parseUserFromAuthResponse, [29](#)
 - parseWatchLaterVideos, [30](#)
 - pendingAccountCreated_, [32](#)
 - pendingRegistration_, [32](#)
 - progressReporter, [32](#)
 - registerBlendOnServer, [30](#)
 - registerEvents, [30](#)

- reportProgress, 30
- server_url, 32
- setProgressReporter, 31
- AppController.cpp, 247, 248
- Json, 248
- kYouTubeAPIKey, 248
- AppController.hpp, 267, 269
- AppController::BlendSummary, 59
 - blendID, 60
 - title, 60
- ApplyAppIcons
 - MainFrame, 128
- ApplySizeBounds
 - ButtonsConfig.cpp, 374
 - ButtonsConfig.hpp, 379
- AppState, 33
 - activeBlend, 39
 - addSessionUser, 34
 - AppState, 34
 - chatRooms, 39
 - clearSession, 34
 - clearSessionUsers, 35
 - createChatRoom, 35
 - currentUser, 39
 - getActiveBlend, 35
 - getActiveChatRoom, 35
 - getChatRoom, 35
 - getCurrentUser, 36
 - getInstance, 36
 - getSessionUsers, 36
 - getUsersWithoutData, 36
 - isBlendGenerating, 39
 - isGeneratingBlend, 36
 - pendingBlendMessage, 39
 - pendingUploadError, 40
 - sessionUsers, 40
 - setActiveBlend, 37
 - setCurrentUser, 37
 - setIsBlendGenerating, 37
 - setPendingBlendMessage, 37
 - setPendingUploadError, 38
 - setUsersWithoutData, 38
 - takePendingBlendMessage, 38
 - takePendingUploadError, 38
 - usersWithoutData, 40
- appState
 - AppController, 31
- AppState.cpp, 270, 271
- AppState.hpp, 273, 274
- AUTH_USER_PREFIX
 - HttpClient, 112
- Background
 - Palette, 139
 - UIColors, 158
- baseUrl
 - HttpClient, 112
- bins
 - DataSaver, 88
- BLEND
 - HttpClient, 112
- Blend, 40
 - algorithmUsed, 44
 - Blend, 41
 - blendID, 44
 - getAlgorithmUsed, 42
 - getBlendID, 42
 - getParticipants, 42
 - getTitle, 42
 - getVideo, 42
 - getVideoList, 43
 - participants, 44
 - setBlendID, 43
 - setVideoList, 43
 - size, 43
 - title, 44
 - toString, 44
 - videoList, 44
- Blend.cpp, 280
- Blend.hpp, 282, 283
- BLEND_CHAT
 - UIPages.hpp, 433
- BLEND_CREATION
 - UIPages.hpp, 433
- BlendChatPanel, 45
 - BlendChatPanel, 47
 - ConnectWebSocket, 47
 - DoSend, 47
 - FormatTimestamp, 47
 - m_controller, 49
 - m_incoming, 49
 - m_incomingMx, 49
 - m_inputCtrl, 49
 - m_inputPanel, 50
 - m_msgInner, 50
 - m_msgScroll, 50
 - m_msgSizer, 50
 - m_nav, 50
 - m_noBlendLabel, 50
 - m_participantsBtn, 50
 - m_participantsDropdown, 50
 - m_participantsSizer, 51
 - m_pollTimer, 51
 - m_ws, 51
 - m_wsBlendID, 51
 - OnInputEnter, 48
 - OnParticipantsToggle, 48
 - OnPollTimer, 48
 - OnSend, 48
 - Refresh, 49
 - Reload, 49
- BlendChatPanel.cpp, 351, 352
 - GetLocalTime, 352
 - IsSameLocalDay, 352
- BlendChatPanel.hpp, 358, 359
- BlendCreationPanel, 52
 - BlendCreationPanel, 53

- m_addedUsers, [54](#)
- m_blendNameCtrl, [54](#)
- m_controller, [55](#)
- m_countLabel, [55](#)
- m_createBtn, [55](#)
- m_emptyLabel, [55](#)
- m_nav, [55](#)
- m_userListInner, [55](#)
- m_userListScroll, [55](#)
- m_userListSizer, [55](#)
- OnAdd, [53](#)
- OnCreate, [53](#)
- OnRemoveUser, [54](#)
- RebuildUserList, [54](#)
- Refresh, [54](#)
- Reload, [54](#)
- UpdateCountLabel, [54](#)
- BlendCreationPanel.cpp, [360](#), [361](#)
- BlendCreationPanel.hpp, [366](#), [368](#)
- BlendFeedPanel, [56](#)
 - BlendFeedPanel, [58](#)
 - kPageSize, [59](#)
 - LoadFromBlend, [58](#)
 - m_cards, [59](#)
 - m_controller, [59](#)
 - m_offset, [59](#)
 - NextPage, [58](#)
 - Refresh, [58](#)
- BlendFeedPanel.cpp, [369](#), [370](#)
- BlendFeedPanel.hpp, [371](#), [373](#)
- blendID
 - AppController::BlendSummary, [60](#)
 - Blend, [44](#)
 - ChatRoom, [66](#)
 - ChatWebSocket, [70](#)
- blendIndicatorLabel
 - MainFrame, [130](#)
- blendsIconBtn
 - MainFrame, [131](#)
- book
 - MainFrame, [131](#)
- BtnConfig, [60](#)
 - maxHeight, [60](#)
 - maxWidth, [60](#)
 - minHeight, [61](#)
 - minWidth, [61](#)
- build_auth_user_lookup_endpoint
 - HttpClient, [106](#)
- build_blend_participant_endpoint
 - HttpClient, [106](#)
- build_chat_history_endpoint
 - HttpClient, [106](#)
- build_chatroom_detail_endpoint
 - HttpClient, [107](#)
- build_delete_user_endpoint
 - HttpClient, [107](#)
- build_fullUrl
 - ChatWebSocket, [68](#)
- build_latest_blend_endpoint
 - HttpClient, [107](#)
- build_leave_blend_endpoint
 - HttpClient, [108](#)
- build_user_blends_endpoint
 - HttpClient, [108](#)
- build_watch_later_endpoint
 - HttpClient, [108](#)
- buildBlendJson
 - RequestJsonBuilder.cpp, [325](#)
 - RequestJsonBuilder.hpp, [330](#)
- buildDeleteAccountJson
 - RequestJsonBuilder.cpp, [326](#)
 - RequestJsonBuilder.hpp, [331](#)
- BuildHomePage
 - MainFrame, [128](#)
- buildLoginJson
 - RequestJsonBuilder.cpp, [326](#)
 - RequestJsonBuilder.hpp, [331](#)
- BuildRegisterForm
 - LoginPanel, [122](#)
- buildRegisterJson
 - RequestJsonBuilder.cpp, [327](#)
 - RequestJsonBuilder.hpp, [332](#)
- BuildSignInForm
 - LoginPanel, [122](#)
- buildWatchLaterJson
 - RequestJsonBuilder.cpp, [327](#)
 - RequestJsonBuilder.hpp, [332](#)
- ButtonsConfig.cpp, [373](#), [376](#)
 - ApplySizeBounds, [374](#)
 - GetConfig, [374](#)
 - GetMaxSize, [375](#)
 - GetMinSize, [375](#)
 - GetSize, [375](#)
- ButtonsConfig.hpp, [377](#), [380](#)
 - ApplySizeBounds, [379](#)
 - ButtonType, [378](#)
 - Compact, [378](#)
 - FormSubmit, [378](#)
 - FormTab, [378](#)
 - FullWidthDanger, [378](#)
 - FullWidthPrimary, [378](#)
 - FullWidthSecondary, [378](#)
 - GetConfig, [379](#)
 - GetMaxSize, [379](#)
 - GetMinSize, [379](#)
 - GetSize, [380](#)
 - Icon, [378](#)
 - Medium, [378](#)
 - Nav, [378](#)
 - Small, [378](#)
 - TopBarBack, [378](#)
- ButtonType
 - ButtonsConfig.hpp, [378](#)
- categories
 - Channel, [63](#)
- Channel, [61](#)

- categories, 63
- Channel, 61
- channelID, 63
- displayName, 63
- getCategories, 62
- getChannelID, 62
- getDisplayName, 62
- toString, 62
- Channel.cpp, 283, 284
- Channel.hpp, 285, 286
- channelID
 - Channel, 63
 - Video, 175
- channelId
 - YouTubeMetadataFetcher::VideoMeta, 187
- channelLogoURL
 - Video, 175
- channelName
 - Video, 175
 - YouTubeMetadataFetcher::VideoMeta, 187
- chatIconBtn
 - MainFrame, 131
- chatPanel
 - MainFrame, 131
- ChatRoom, 63
 - addMessage, 64
 - blendID, 66
 - ChatRoom, 64
 - clearMessages, 65
 - getBlendID, 65
 - getMessages, 65
 - getParticipantIDs, 65
 - isParticipant, 65
 - messages, 66
 - participantIDs, 66
 - toString, 66
- ChatRoom.cpp, 286, 287
- ChatRoom.hpp, 288, 289
- chatRooms
 - AppState, 39
- ChatWebSocket, 67
 - ~ChatWebSocket, 68
 - blendID, 70
 - build_fullUrl, 68
 - ChatWebSocket, 67
 - connect, 68
 - content_formatter, 68
 - fullUrl, 70
 - get_fullUrl, 68
 - isStarted, 70
 - m_onMessage, 70
 - send_message, 69
 - set_blendID, 69
 - set_userID, 69
 - setOnMessage, 69
 - url, 70
 - userID, 70
 - ws, 70
- ChatWebSocket.cpp, 313, 314
 - urlEncode, 314
- ChatWebSocket.hpp, 315, 317
- Clear
 - VideoCard, 179
- clearMessages
 - ChatRoom, 65
- clearProgressReporter
 - AppController, 22
- clearSession
 - AppState, 34
- clearSessionUsers
 - AppState, 35
- Clone
 - ThumbnailEvent, 153
- ColorRole
 - UIColors.hpp, 430
- Compact
 - ButtonsConfig.hpp, 378
- Confirm
 - ConfirmationDialog, 72
- ConfirmationDialog, 71
 - Confirm, 72
 - ConfirmationDialog, 72
- ConfirmationDialog.cpp, 381
- ConfirmationDialog.hpp, 383, 384
- connect
 - ChatWebSocket, 68
- ConnectWebSocket
 - BlendChatPanel, 47
- content_formatter
 - ChatWebSocket, 68
- controller
 - MainFrame, 131
 - YTBLNDApp, 205
- createChatRoom
 - AppState, 35
- createUser
 - DataManager, 84
 - SqliteDataManager, 148
- creationPanel
 - MainFrame, 131
- CSV
 - File_ID, 90
- CsvParser, 73
 - ~CsvParser, 74
 - CsvParser, 74
 - getParseld, 74
 - parse, 74
 - parse_id, 75
 - raw_data, 75
 - setData, 75
- CsvParser.cpp, 213, 214
- CsvParser.hpp, 215, 216
- CsvSource, 76
 - CsvSource, 77
 - getSourceId, 77
 - read, 77

- setSource, 77
 - source_id, 78
 - src, 78
- CsvSource.cpp, 217
- CsvSource.hpp, 218, 219
- Current
 - UIColors, 161
- currentBlend
 - AppController, 31
- currentProgress
 - OperationProgressDialog, 137
- currentUser
 - AppState, 39
- D
 - HttpClient, 113
- Danger
 - Palette, 139
 - UIColors, 158
- DATA_INSTRUCTIONS
 - UIPages.hpp, 433
- DataExtractor, 78
 - ~DataExtractor, 79
 - DataExtractor, 79
 - extract, 79
 - par_format, 80
 - setParser, 80
 - setSource, 80
 - src_format, 80
- DataExtractor.cpp, 219, 220
- DataExtractor.hpp, 221, 222
- dataInstrPanel
 - MainFrame, 131
- DataInstructionsPanel, 81
 - DataInstructionsPanel, 82
 - m_controller, 82
 - m_nav, 82
 - OnBrowse, 82
- DataInstructionsPanel.cpp, 384, 385
- DataInstructionsPanel.hpp, 387, 388
- DataManager, 83
 - createUser, 84
 - findBlendForUser, 84
 - findUserById, 84
 - loadBlendParticipants, 84
 - loadWatchLater, 85
 - saveBlend, 85
 - saveWatchLater, 85
 - validatePassword, 85
- DataManager.hpp, 317, 319
- DataSaver, 86
 - bins, 88
 - DataSaver, 86
 - formatters, 88
 - save, 87
 - setFileBin, 87
 - setFileFormatter, 87
- DataSaver.hpp, 222, 223
- db
 - SqliteDataManager, 151
- del
 - HttpClient, 109
- DELETE_USER_PREFIX
 - HttpClient, 113
- deregisterAll
 - EventRouter, 89
- dispatch
 - EventRouter, 89
- displayName
 - Channel, 63
 - Friend, 98
- DoGetBestSize
 - VideoCard, 179
- DoSend
 - BlendChatPanel, 47
- DrawWrappedText
 - VideoCard, 179
- DUPLICATE
 - HttpClient, 113
- duration
 - Video, 175
- email
 - Friend, 98
 - User, 166
- enrich
 - YouTubeMetadataFetcher, 202
- enrichActiveBlendFeedIfMissingMetadata
 - AppController, 22
- enrichIfMissingMetadata
 - AppController, 22
- enrichMissingMetadataSubsetMultithreaded
 - AppController, 22
- escape
 - ModelJson, 13
- EventHandler
 - EventRouter.hpp, 277
- EventPayload
 - EventRouter.hpp, 277
- EventRouter, 88
 - deregisterAll, 89
 - dispatch, 89
 - EventRouter, 89
 - listeners, 90
 - registerListener, 89
- eventRouter
 - AppController, 31
- EventRouter.cpp, 275, 276
- EventRouter.hpp, 276, 278
 - EventHandler, 277
 - EventPayload, 277
- extract
 - DataExtractor, 79
- extract_links
 - HtmlParser, 100
- extractVideoIdFromHref
 - WatchHistoryParser, 188

- feedPanel
 - MainFrame, 131
- FetchChannelLogoAsync
 - VideoCard, 179
- fetchChannelLogos
 - YouTubeMetadataFetcher, 202
- FetchThumbnailAsync
 - VideoCard, 179
- fetchUserBlends
 - AppController, 23
- fetchVideoBatch
 - YouTubeMetadataFetcher, 202
- File_ID, 90
 - CSV, 90
 - HTML, 90
 - JSON, 90
- File_ID.hpp, 224
- FileBin, 91
 - ~FileBin, 91
 - setFileName, 91
 - setOutputFile, 92
 - setRawData, 92
 - write, 92
- FileBin.hpp, 225, 226
- FileFormatter, 92
 - ~FileFormatter, 93
 - format, 93
 - setData, 93
- FileFormatter.hpp, 226, 227
- filePath
 - WatchHistoryParser, 190
 - WatchLaterParser, 192
- FileSource, 94
 - getSourceId, 94
 - read, 94
 - setSource, 95
- FileSource.hpp, 228, 229
- findBlendForUser
 - DataManager, 84
 - SqliteDataManager, 148
- FindResourcePath
 - ResourcePathUtils.cpp, 415
 - ResourcePathUtils.hpp, 417
- findUserById
 - DataManager, 84
 - SqliteDataManager, 149
- format
 - FileFormatter, 93
- formatters
 - DataSaver, 88
- FormatTimestamp
 - BlendChatPanel, 47
- FormSubmit
 - ButtonsConfig.hpp, 378
- FormTab
 - ButtonsConfig.hpp, 378
- Friend, 95
 - displayName, 98
 - email, 98
 - Friend, 96
 - getDisplayName, 96
 - getEmail, 96
 - getUserID, 97
 - setDisplayName, 97
 - setEmail, 97
 - toString, 97
 - userID, 98
- Friend.cpp, 290
- Friend.hpp, 291, 292
- friends
 - User, 166
- fullUrl
 - ChatWebSocket, 70
- FullWidthDanger
 - ButtonsConfig.hpp, 378
- FullWidthPrimary
 - ButtonsConfig.hpp, 378
- FullWidthSecondary
 - ButtonsConfig.hpp, 378
- G
 - HttpClient, 113
- generateBlend
 - IBlendAlgorithm, 115
 - RandomBlendAlgorithm, 143
 - WeightedBlendAlgorithm, 194
- generation
 - ThumbnailEvent, 153
- get
 - HttpClient, 109
- get_current_blend_id
 - AppController, 23
- get_current_blend_title
 - AppController, 23
- get_current_email
 - AppController, 23
- get_current_user
 - AppController, 24
- get_current_username
 - AppController, 24
- get_fullUrl
 - ChatWebSocket, 68
- get_text
 - HtmlParser, 100
- getActiveBlend
 - AppState, 35
- getActiveChatRoom
 - AppState, 35
- getAlgorithmUsed
 - Blend, 42
- getBlendID
 - Blend, 42
 - ChatRoom, 65
- getCategories
 - Channel, 62
- getChannelID
 - Channel, 62

- Video, 173
- getChannelLogoURL
 - Video, 173
- getChannelName
 - Video, 173
- getChatRoom
 - AppState, 35
- GetConfig
 - ButtonsConfig.cpp, 374
 - ButtonsConfig.hpp, 379
- getCurrentUser
 - AppState, 36
- getDisplayName
 - Channel, 62
 - Friend, 96
- getDuration
 - Video, 174
- getEmail
 - Friend, 96
 - User, 163
- getEventRouter
 - AppController, 24
- getFriends
 - User, 163
- GetIcon
 - UIColors, 158
- GetIconFolder
 - ResourcePathUtils.cpp, 415
 - ResourcePathUtils.hpp, 418
- getInstance
 - AppState, 36
- getLastStatusCode
 - HttpClient, 109
- getLastWatched
 - VideoEntry, 184
- getLikedVideos
 - YouTubeData, 196
- GetLocalTime
 - BlendChatPanel.cpp, 352
- GetMaxSize
 - ButtonsConfig.cpp, 375
 - ButtonsConfig.hpp, 379
- getMessages
 - ChatRoom, 65
- GetMinSize
 - ButtonsConfig.cpp, 375
 - ButtonsConfig.hpp, 379
- getMostWatchedVideos
 - YouTubeData, 196
- getParseld
 - CsvParser, 74
 - HtmlParser, 100
 - Parser, 141
- getParserId
 - WatchHistoryParser, 188
 - WatchLaterParser, 191
- getParticipantIDs
 - ChatRoom, 65
- getParticipants
 - Blend, 42
- getPassword
 - User, 163
- GetRoleFromColour
 - UIColors, 158
- getSessionUsers
 - AppState, 36
- GetSize
 - ButtonsConfig.cpp, 375
 - ButtonsConfig.hpp, 380
- getSourceId
 - CsvSource, 77
 - FileSource, 94
 - HtmlSource, 103
- getSubscribedChannels
 - YouTubeData, 196
- getTags
 - Video, 174
- getThumbnailURL
 - Video, 174
- getTitle
 - Blend, 42
 - Video, 174
- getTopChannels
 - YouTubeData, 197
- getUserID
 - Friend, 97
 - User, 164
- getUsername
 - User, 164
- getUsersWithoutData
 - AppState, 36
- getVideo
 - Blend, 42
 - VideoEntry, 184
- getVideoID
 - Video, 174
- getVideoList
 - Blend, 43
- getWatchCount
 - VideoEntry, 185
- getWatchHistory
 - YouTubeData, 197
- getWatchLaterVideos
 - YouTubeData, 197
- getYouTubeData
 - User, 164
- handleCreateBlend
 - AppController, 24
- handleDeleteAccount
 - AppController, 25
- handleLeaveBlend
 - AppController, 25
- handleLogin
 - AppController, 25
- handleLogout
 - AppController, 25

- handleOpenChat
 - AppController, 26
- handlePlayVideo
 - AppController, 26
- handleRefresh
 - AppController, 26
- handleRegister
 - AppController, 26
- handleSelectBlend
 - AppController, 27
- handleSendMessage
 - AppController, 27
- handleUploadData
 - AppController, 27
- hdr
 - HttpClient, 113
- HOME
 - UIPages.hpp, 433
- HTML
 - File_ID, 90
- HtmlParser, 99
 - extract_links, 100
 - get_text, 100
 - getParseld, 100
 - HtmlParser, 100
 - parse, 101
 - parse_id, 101
 - raw_data, 101
 - setData, 101
- HtmlParser.cpp, 229, 230
- HtmlSource, 102
 - getSourceId, 103
 - HtmlSource, 103
 - read, 103
 - setSource, 103
 - source_id, 104
 - src_path, 104
- HtmlSource.cpp, 231, 232
- http
 - AppController, 31
- HttpClient, 104
 - ALT_SUCC, 112
 - AUTH_USER_PREFIX, 112
 - baseUrl, 112
 - BLEND, 112
 - build_auth_user_lookup_endpoint, 106
 - build_blend_participant_endpoint, 106
 - build_chat_history_endpoint, 106
 - build_chatroom_detail_endpoint, 107
 - build_delete_user_endpoint, 107
 - build_latest_blend_endpoint, 107
 - build_leave_blend_endpoint, 108
 - build_user_blends_endpoint, 108
 - build_watch_later_endpoint, 108
 - D, 113
 - del, 109
 - DELETE_USER_PREFIX, 113
 - DUPLICATE, 113
 - G, 113
 - get, 109
 - getLastStatusCode, 109
 - hdr, 113
 - HttpClient, 105
 - INV_USR_LOG_ERR, 113
 - isRequestSuccessful, 110
 - lastStatusCode, 113
 - LOGIN, 113
 - MAL_REQ_ERR, 114
 - MISS_RM_CNTXT, 114
 - P, 114
 - post, 110
 - REG_USER, 114
 - request, 111
 - SERVER_ERR, 114
 - SUCC, 114
 - UNAUTH_RES_REQ_ERR, 114
 - wasLastRequestSuccessful, 111
 - write_callback, 112
- HttpClient.cpp, 319, 320
- HttpClient.hpp, 322, 323
- httpGet
 - YouTubeMetadataFetcher, 202
- IBlendAlgorithm, 115
 - generateBlend, 115
- IBlendAlgorithm.hpp, 207, 208
- Icon
 - ButtonsConfig.hpp, 378
- IconBase
 - MainFrame, 127
- image
 - ThumbnailEvent, 153
- ImageBackgroundPanel, 116
 - ImageBackgroundPanel, 117
 - m_imageStore, 117
 - m_key, 117
 - OnPaint, 117
- ImageKey
 - MainFrame, 127
- images
 - MainFrame, 132
- import
 - YouTubeDataImporter, 200
- importMultiThreaded
 - YouTubeDataImporter, 200
- initSchema
 - SqliteDataManager, 149
- INV_USR_LOG_ERR
 - HttpClient, 113
- IRefreshablePanel, 118
 - Refresh, 119
- IRefreshablePanel.hpp, 389
- isBlendGenerating
 - AppState, 39
- isConnected
 - AppController, 31
- isGeneratingBlend

- AppState, 36
- isParticipant
 - ChatRoom, 65
- isRequestSuccessful
 - HttpClient, 110
- IsSameLocalDay
 - BlendChatPanel.cpp, 352
- isStarted
 - ChatWebSocket, 70
- IThemedElement, 119
- JSON
 - File_ID, 90
- Json
 - AppController.cpp, 248
- json
 - YouTubeMetadataFetcher.cpp, 341
- JsonUtils.hpp, 292, 294
- kAppBackendBaseUrl
 - ServerConfig.hpp, 312
- kAppBackendWsPrefix
 - ServerConfig.hpp, 312
- kBatchSize
 - YouTubeMetadataFetcher, 203
- kPageSize
 - BlendFeedPanel, 59
- kTestBackendBaseUrl
 - ServerConfig.hpp, 312
- kTestBackendWsPrefix
 - ServerConfig.hpp, 313
- kYouTubeAPIKey
 - AppController.cpp, 248
- lastStatusCode
 - HttpClient, 113
- lastWatched
 - VideoEntry, 186
- likedVideos
 - YouTubeData, 199
- LineHeight
 - VideoCard.cpp, 441
- listeners
 - EventRouter, 90
- loadBlendParticipants
 - DataManager, 84
 - SqliteDataManager, 149
- LoadFromBlend
 - BlendFeedPanel, 58
- LoadImage
 - MainFrame, 128
- loadParticipantsWithWatchLater
 - AppController, 27
- loadParticipantsWithWatchLaterParallel
 - AppController, 28
- LoadThemedBackground
 - LoginPanel, 122
- LoadThemedIcon
 - MainFrame, 128
- LoadThemesFromFile
 - UIColors, 158
- loadWatchLater
 - DataManager, 85
 - SqliteDataManager, 149
- LOGIN
 - HttpClient, 113
 - UIPages.hpp, 433
- LoginPanel, 120
 - BuildRegisterForm, 122
 - BuildSignInForm, 122
 - LoadThemedBackground, 122
 - LoginPanel, 121
 - m_bgImage, 123
 - m_controller, 123
 - m_formBook, 123
 - m_nav, 124
 - m_regConfirm, 124
 - m_regEmail, 124
 - m_regError, 124
 - m_registerTab, 124
 - m_regPassword, 124
 - m_regUsername, 124
 - m_siError, 124
 - m_signinTab, 125
 - m_siPassword, 125
 - m_siUsername, 125
 - OnPaint, 122
 - OnRegister, 122
 - OnSignIn, 122
 - ProceedAfterLogin, 122
 - Refresh, 123
 - Reset, 123
 - ShowTab, 123
- loginPanel
 - MainFrame, 132
- LoginPanel.cpp, 390, 391
 - MakeError, 390
 - MakeField, 390
- LoginPanel.hpp, 395, 397
- lookupUser
 - AppController, 28
- lowerExtension
 - YouTubeDataImporter, 200
- m_addedUsers
 - BlendCreationPanel, 54
- m_backBtn
 - TopBar, 156
- m_backDest
 - TopBar, 156
- m_bgImage
 - LoginPanel, 123
- m_blendNameCtrl
 - BlendCreationPanel, 54
- m_cards
 - BlendFeedPanel, 59
- m_channelID
 - VideoCard, 181

- m_channelLogo
 - VideoCard, [181](#)
- m_channelLogoURL
 - VideoCard, [181](#)
- m_channelName
 - VideoCard, [181](#)
- m_confirmDeleteBtn
 - UserPanel, [171](#)
- m_controller
 - ActiveBlendsPanel, [18](#)
 - BlendChatPanel, [49](#)
 - BlendCreationPanel, [55](#)
 - BlendFeedPanel, [59](#)
 - DataInstructionsPanel, [82](#)
 - LoginPanel, [123](#)
 - SettingsPanel, [146](#)
 - UserPanel, [171](#)
 - VideoCard, [181](#)
- m_countLabel
 - BlendCreationPanel, [55](#)
- m_createBtn
 - BlendCreationPanel, [55](#)
- m_deleteErrorLabel
 - UserPanel, [171](#)
- m_deletePasswordField
 - UserPanel, [171](#)
- m_emailLabel
 - UserPanel, [171](#)
- m_emptyLabel
 - ActiveBlendsPanel, [18](#)
 - BlendCreationPanel, [55](#)
- m_formBook
 - LoginPanel, [123](#)
- m_hovered
 - VideoCard, [181](#)
- m_imageStore
 - ImageBackgroundPanel, [117](#)
- m_incoming
 - BlendChatPanel, [49](#)
- m_incomingMx
 - BlendChatPanel, [49](#)
- m_inputCtrl
 - BlendChatPanel, [49](#)
- m_inputPanel
 - BlendChatPanel, [50](#)
- m_key
 - ImageBackgroundPanel, [117](#)
- m_listInner
 - ActiveBlendsPanel, [18](#)
- m_listScroll
 - ActiveBlendsPanel, [18](#)
- m_listSizer
 - ActiveBlendsPanel, [18](#)
- m_logoFailed
 - VideoCard, [181](#)
- m_logoGen
 - VideoCard, [182](#)
- m_logoLoading
 - VideoCard, [182](#)
- m_msgInner
 - BlendChatPanel, [50](#)
- m_msgScroll
 - BlendChatPanel, [50](#)
- m_msgSizer
 - BlendChatPanel, [50](#)
- m_nav
 - ActiveBlendsPanel, [18](#)
 - BlendChatPanel, [50](#)
 - BlendCreationPanel, [55](#)
 - DataInstructionsPanel, [82](#)
 - LoginPanel, [124](#)
 - SettingsPanel, [146](#)
 - TopBar, [156](#)
 - UserPanel, [171](#)
- m_noBlendLabel
 - BlendChatPanel, [50](#)
- m_offset
 - BlendFeedPanel, [59](#)
- m_onMessage
 - ChatWebSocket, [70](#)
- m_participantsBtn
 - BlendChatPanel, [50](#)
- m_participantsDropdown
 - BlendChatPanel, [50](#)
- m_participantsSizer
 - BlendChatPanel, [51](#)
- m_pollTimer
 - BlendChatPanel, [51](#)
- m_regConfirm
 - LoginPanel, [124](#)
- m_regEmail
 - LoginPanel, [124](#)
- m_regError
 - LoginPanel, [124](#)
- m_registerTab
 - LoginPanel, [124](#)
- m_regPassword
 - LoginPanel, [124](#)
- m_regUsername
 - LoginPanel, [124](#)
- m_siError
 - LoginPanel, [124](#)
- m_signinTab
 - LoginPanel, [125](#)
- m_siPassword
 - LoginPanel, [125](#)
- m_siUsername
 - LoginPanel, [125](#)
- m_themeChoice
 - SettingsPanel, [146](#)
- m_thumbFailed
 - VideoCard, [182](#)
- m_thumbGen
 - VideoCard, [182](#)
- m_thumbLoading
 - VideoCard, [182](#)

- m_thumbnail
 - VideoCard, [182](#)
- m_thumbnailURL
 - VideoCard, [182](#)
- m_title
 - VideoCard, [183](#)
- m_userListInner
 - BlendCreationPanel, [55](#)
- m_userListScroll
 - BlendCreationPanel, [55](#)
- m_userListSizer
 - BlendCreationPanel, [55](#)
- m_usernameLabel
 - UserPanel, [171](#)
- m_videoID
 - VideoCard, [183](#)
- m_ws
 - BlendChatPanel, [51](#)
- m_wsBlendID
 - BlendChatPanel, [51](#)
- main.cpp, [278](#)
- MainFrame, [125](#)
 - activeBlendsPanel, [130](#)
 - ApplyAppIcons, [128](#)
 - blendIndicatorLabel, [130](#)
 - blendsIconBtn, [131](#)
 - book, [131](#)
 - BuildHomePage, [128](#)
 - chatIconBtn, [131](#)
 - chatPanel, [131](#)
 - controller, [131](#)
 - creationPanel, [131](#)
 - dataInstrPanel, [131](#)
 - feedPanel, [131](#)
 - iconBase, [127](#)
 - ImageKey, [127](#)
 - images, [132](#)
 - LoadImage, [128](#)
 - LoadThemedIcon, [128](#)
 - loginPanel, [132](#)
 - MainFrame, [128](#)
 - MakeIconButton, [129](#)
 - NavigateTo, [129](#)
 - refreshBtn, [132](#)
 - refreshInProgress, [132](#)
 - ReloadThemedIcons, [129](#)
 - settingsIconBtn, [132](#)
 - settingsPanel, [132](#)
 - ShowSettingsDropdown, [130](#)
 - ShowUserDropdown, [130](#)
 - TriggerFeedRefresh, [130](#)
 - UpdateBlendIndicatorLabel, [130](#)
 - userIconBtn, [132](#)
 - userPanel, [132](#)
- MainFrame.cpp, [398](#), [399](#)
- MainFrame.hpp, [408](#), [409](#)
- MakeError
 - LoginPanel.cpp, [390](#)
- MakeField
 - LoginPanel.cpp, [390](#)
- MakeIconButton
 - MainFrame, [129](#)
 - UIColors, [159](#)
- MAL_REQ_ERR
 - HttpClient, [114](#)
- maxHeight
 - BtnConfig, [60](#)
- maxWidth
 - BtnConfig, [60](#)
- Medium
 - ButtonsConfig.hpp, [378](#)
- Message, [133](#)
 - Message, [133](#)
 - text, [134](#)
 - timestamp, [134](#)
 - toString, [134](#)
 - userID, [134](#)
- Message.cpp, [294](#), [295](#)
- Message.hpp, [295](#), [297](#)
- messageLabel
 - OperationProgressDialog, [137](#)
- messages
 - ChatRoom, [66](#)
- minHeight
 - BtnConfig, [61](#)
- minWidth
 - BtnConfig, [61](#)
- MISS_RM_CNTXT
 - HttpClient, [114](#)
- ModelJson, [13](#)
 - escape, [13](#)
 - quote, [13](#)
 - toIso8601, [14](#)
- Name
 - Palette, [139](#)
- Nav
 - ButtonsConfig.hpp, [378](#)
- NavigateFn
 - UIPages.hpp, [433](#)
- NavigateTo
 - MainFrame, [129](#)
- NextPage
 - BlendFeedPanel, [58](#)
- normalizeWeights
 - WeightedBlendAlgorithm, [194](#)
- OnAdd
 - BlendCreationPanel, [53](#)
- OnBack
 - TopBar, [155](#)
- OnBrowse
 - DataInstructionsPanel, [82](#)
- OnChannelLogoLoaded
 - VideoCard, [180](#)
- OnClick
 - VideoCard, [180](#)

- OnConfirmDelete
 - UserPanel, 169
- OnCreate
 - BlendCreationPanel, 53
- OnDeleteRequest
 - UserPanel, 169
- OnExit
 - YTBLNDApp, 205
- OnInit
 - YTBLNDApp, 205
- OnInputEnter
 - BlendChatPanel, 48
- OnLeaveBlend
 - ActiveBlendsPanel, 17
- OnLogout
 - SettingsPanel, 145
 - UserPanel, 169
- OnMouseEnter
 - VideoCard, 180
- OnMouseLeave
 - VideoCard, 180
- OnPaint
 - ImageBackgroundPanel, 117
 - LoginPanel, 122
 - VideoCard, 180
- OnParticipantsToggle
 - BlendChatPanel, 48
- OnPollTimer
 - BlendChatPanel, 48
- OnRegister
 - LoginPanel, 122
- OnRemoveUser
 - BlendCreationPanel, 54
- OnReuploadCSV
 - UserPanel, 170
- OnSelectBlend
 - ActiveBlendsPanel, 17
- OnSend
 - BlendChatPanel, 48
- OnShow
 - SettingsPanel, 145
 - UserPanel, 170
- OnSignIn
 - LoginPanel, 122
- OnThemeChanged
 - SettingsPanel, 145
- OnThemeUpdate
 - TopBar, 155
- OnThumbnailLoaded
 - VideoCard, 180
- OperationProgressDialog, 135
 - ~OperationProgressDialog, 136
 - currentProgress, 137
 - messageLabel, 137
 - OperationProgressDialog, 136
 - percentLabel, 137
 - progressFill, 137
 - progressTrack, 137
 - ShowModal, 136
 - UpdateProgress, 136
 - UpdateProgressFill, 137
- OperationProgressDialog.cpp, 410, 411
- OperationProgressDialog.hpp, 412, 414
- P
 - HttpClient, 114
- Page
 - UIPages.hpp, 433
- Palette, 138
 - Accent, 138
 - AccentHover, 138
 - Background, 139
 - Danger, 139
 - Name, 139
 - Separator, 139
 - Surface, 139
 - SurfaceRaised, 139
 - TextMuted, 139
 - TextPrimary, 139
 - TextSecondary, 140
- par_format
 - DataExtractor, 80
- parse
 - CsvParser, 74
 - HtmlParser, 101
 - Parser, 141
 - WatchHistoryParser, 189
 - WatchLaterParser, 191
- parse_id
 - CsvParser, 75
 - HtmlParser, 101
- parseBlendID
 - AppController, 28
- parseChatHistory
 - AppController, 29
- parseMultiThreaded
 - WatchHistoryParser, 189
 - WatchLaterParser, 191
- parseParticipantIDs
 - AppController, 29
- Parser, 140
 - getParseId, 141
 - parse, 141
 - setData, 141
- Parser.hpp, 232, 234
- parser_id
 - WatchHistoryParser, 190
 - WatchLaterParser, 192
- parseRange
 - WatchHistoryParser, 189
- parseUserFromAuthResponse
 - AppController, 29
- parseWatchLaterVideos
 - AppController, 30
- participantIDs
 - ChatRoom, 66
- participants

- Blend, 44
- participantWeights
 - WeightedBlendAlgorithm, 195
- password
 - User, 166
- pendingAccountCreated_
 - AppController, 32
- pendingBlendMessage
 - AppState, 39
- pendingRegistration_
 - AppController, 32
- pendingUploadError
 - AppState, 40
- percentLabel
 - OperationProgressDialog, 137
- post
 - HttpClient, 110
- ProceedAfterLogin
 - LoginPanel, 122
- progressFill
 - OperationProgressDialog, 137
- progressReporter
 - AppController, 32
- progressTrack
 - OperationProgressDialog, 137
- quote
 - ModelJson, 13
- RandomBlendAlgorithm, 142
 - generateBlend, 143
 - RandomBlendAlgorithm, 143
 - videosPerUser, 143
- RandomBlendAlgorithm.cpp, 209, 210
- RandomBlendAlgorithm.hpp, 210, 212
- raw_data
 - CsvParser, 75
 - HtmlParser, 101
- read
 - CsvSource, 77
 - FileSource, 94
 - HtmlSource, 103
- RebuildList
 - ActiveBlendsPanel, 17
- RebuildUserList
 - BlendCreationPanel, 54
- Refresh
 - ActiveBlendsPanel, 17
 - BlendChatPanel, 49
 - BlendCreationPanel, 54
 - BlendFeedPanel, 58
 - IRefreshablePanel, 119
 - LoginPanel, 123
 - UserPanel, 170
- refreshBtn
 - MainFrame, 132
- refreshInProgress
 - MainFrame, 132
- RefreshUserInfo
 - SettingsPanel, 146
 - UserPanel, 170
- REG_USER
 - HttpClient, 114
- registerBlendOnServer
 - AppController, 30
- registerEvents
 - AppController, 30
- registerListener
 - EventRouter, 89
- Reload
 - BlendChatPanel, 49
 - BlendCreationPanel, 54
- ReloadThemedIcons
 - MainFrame, 129
- removeFriend
 - User, 164
- reportProgress
 - AppController, 30
- request
 - HttpClient, 111
- RequestJsonBuilder.cpp, 324, 328
 - buildBlendJson, 325
 - buildDeleteAccountJson, 326
 - buildLoginJson, 326
 - buildRegisterJson, 327
 - buildWatchLaterJson, 327
- RequestJsonBuilder.hpp, 329, 333
 - buildBlendJson, 330
 - buildDeleteAccountJson, 331
 - buildLoginJson, 331
 - buildRegisterJson, 332
 - buildWatchLaterJson, 332
- Reset
 - LoginPanel, 123
- ResourcePathUtils.cpp, 414, 416
 - FindResourcePath, 415
 - GetIconFolder, 415
- ResourcePathUtils.hpp, 416, 418
 - FindResourcePath, 417
 - GetIconFolder, 418
- save
 - DataSaver, 87
- saveBlend
 - DataManager, 85
 - SqliteDataManager, 150
- saveWatchLater
 - DataManager, 85
 - SqliteDataManager, 150
- selectWeightedVideos
 - WeightedBlendAlgorithm, 194
- send_message
 - ChatWebSocket, 69
- Separator
 - Palette, 139
 - UIColors, 159
- SERVER_ERR
 - HttpClient, 114

- server_url
 - AppController, 32
- ServerConfig.hpp, 311
 - kAppBackendBaseUrl, 312
 - kAppBackendWsPrefix, 312
 - kTestBackendBaseUrl, 312
 - kTestBackendWsPrefix, 313
- sessionUsers
 - AppState, 40
- set_blendID
 - ChatWebSocket, 69
- set_userID
 - ChatWebSocket, 69
- setActiveBlend
 - AppState, 37
- setBlendID
 - Blend, 43
- setCurrentUser
 - AppState, 37
- setData
 - CsvParser, 75
 - FileFormatter, 93
 - HtmlParser, 101
 - Parser, 141
- SetDeleteReauthVisible
 - UserPanel, 170
- setDisplay_name
 - Friend, 97
- setEmail
 - Friend, 97
 - User, 165
- setFileBin
 - DataSaver, 87
- setFileFormatter
 - DataSaver, 87
- setFileName
 - FileBin, 91
- setIsBlendGenerating
 - AppState, 37
- setLastWatched
 - VideoEntry, 185
- setLikedVideos
 - YouTubeData, 197
- setOnMessage
 - ChatWebSocket, 69
- setOutputFile
 - FileBin, 92
- setParser
 - DataExtractor, 80
- setPassword
 - User, 165
- setPendingBlendMessage
 - AppState, 37
- setPendingUploadError
 - AppState, 38
- setProgressReporter
 - AppController, 31
- setRawData
 - FileBin, 92
- setSource
 - CsvSource, 77
 - DataExtractor, 80
 - FileSource, 95
 - HtmlSource, 103
- setSubscribedChannels
 - YouTubeData, 198
- SetTheme
 - UIColors, 159
- SETTINGS
 - UIPages.hpp, 433
- settingsIconBtn
 - MainFrame, 132
- SettingsPanel, 144
 - m_controller, 146
 - m_nav, 146
 - m_themeChoice, 146
 - OnLogout, 145
 - OnShow, 145
 - OnThemeChanged, 145
 - RefreshUserInfo, 146
 - SettingsPanel, 145
- settingsPanel
 - MainFrame, 132
- SettingsPanel.cpp, 418, 419
- SettingsPanel.hpp, 421, 422
- setUsername
 - User, 165
- setUsersWithoutData
 - AppState, 38
- SetVideo
 - VideoCard, 180
- setVideoList
 - Blend, 43
- setWatchCount
 - VideoEntry, 185
- setWatchHistory
 - YouTubeData, 198
- setWatchLaterVideos
 - YouTubeData, 198
- setYouTubeData
 - User, 165
- ShowModal
 - OperationProgressDialog, 136
- ShowSettingsDropdown
 - MainFrame, 130
- ShowTab
 - LoginPanel, 123
- ShowUserDropdown
 - MainFrame, 130
- size
 - Blend, 43
- Small
 - ButtonsConfig.hpp, 378
- source_id
 - CsvSource, 78
 - HtmlSource, 104

- SqliteDataManager, 147
 - ~SqliteDataManager, 148
 - createUser, 148
 - db, 151
 - findBlendForUser, 148
 - findUserByID, 149
 - initSchema, 149
 - loadBlendParticipants, 149
 - loadWatchLater, 149
 - saveBlend, 150
 - saveWatchLater, 150
 - SqliteDataManager, 148
 - validatePassword, 150
- SqliteDataManager.cpp, 333, 334
- SqliteDataManager.hpp, 338, 340
- src
 - CsvSource, 78
- src_format
 - DataExtractor, 80
- src_path
 - HtmlSource, 104
- subscribedChannels
 - YouTubeData, 199
- SUCC
 - HttpClient, 114
- success
 - ThumbnailEvent, 153
- Surface
 - Palette, 139
 - UIColors, 159
- SurfaceRaised
 - Palette, 139
 - UIColors, 160
- tags
 - Video, 175
- takePendingBlendMessage
 - AppState, 38
- takePendingUploadError
 - AppState, 38
- text
 - Message, 134
- TextMuted
 - Palette, 139
 - UIColors, 160
- TextPrimary
 - Palette, 139
 - UIColors, 160
- TextSecondary
 - Palette, 140
 - UIColors, 160
- ThemeMap
 - UIColors, 161
- ThemeOrder
 - UIColors, 161
- ThumbnailEvent, 151
 - Clone, 153
 - generation, 153
 - image, 153
 - success, 153
 - ThumbnailEvent, 152
- thumbnailURL
 - Video, 176
- thumbnailUrl
 - YouTubeMetadataFetcher::VideoMeta, 187
- timestamp
 - Message, 134
- title
 - AppController::BlendSummary, 60
 - Blend, 44
 - Video, 176
 - YouTubeMetadataFetcher::VideoMeta, 187
- tolso8601
 - ModelJson, 14
- TopBar, 154
 - m_backBtn, 156
 - m_backDest, 156
 - m_nav, 156
 - OnBack, 155
 - OnThemeUpdate, 155
 - TopBar, 155
- TopBar.cpp, 422, 423
- TopBar.hpp, 424, 425
- TopBarBack
 - ButtonsConfig.hpp, 378
- toString
 - Blend, 44
 - Channel, 62
 - ChatRoom, 66
 - Friend, 97
 - Message, 134
 - User, 166
 - Video, 175
 - VideoEntry, 185
 - YouTubeData, 198
- TriggerFeedRefresh
 - MainFrame, 130
- UIColors, 156
 - Accent, 157
 - AccentHover, 157
 - Background, 158
 - Current, 161
 - Danger, 158
 - GetIcon, 158
 - GetRoleFromColour, 158
 - LoadThemesFromFile, 158
 - MakelconButton, 159
 - Separator, 159
 - SetTheme, 159
 - Surface, 159
 - SurfaceRaised, 160
 - TextMuted, 160
 - TextPrimary, 160
 - TextSecondary, 160
 - ThemeMap, 161
 - ThemeOrder, 161
 - UpdateUI, 160

- UIColors.cpp, 425, 426
- UIColors.hpp, 429, 430
 - ColorRole, 430
- UIPages.hpp, 431, 434
 - ACTIVE_BLENDS, 433
 - BLEND_CHAT, 433
 - BLEND_CREATION, 433
 - DATA_INSTRUCTIONS, 433
 - HOME, 433
 - LOGIN, 433
 - NavigateFn, 433
 - Page, 433
 - SETTINGS, 433
 - USER, 433
- UNAUTH_RES_REQ_ERR
 - HttpClient, 114
- UpdateBlendIndicatorLabel
 - MainFrame, 130
- UpdateCountLabel
 - BlendCreationPanel, 54
- UpdateProgress
 - OperationProgressDialog, 136
- UpdateProgressFill
 - OperationProgressDialog, 137
- UpdateUI
 - UIColors, 160
- url
 - ChatWebSocket, 70
- urlencode
 - ChatWebSocket.cpp, 314
- USER
 - UIPages.hpp, 433
- User, 161
 - addFriend, 163
 - email, 166
 - friends, 166
 - getEmail, 163
 - getFriends, 163
 - getPassword, 163
 - getUserID, 164
 - getUsername, 164
 - getYouTubeData, 164
 - password, 166
 - removeFriend, 164
 - setEmail, 165
 - setPassword, 165
 - setUsername, 165
 - setYouTubeData, 165
 - toString, 166
 - User, 162
 - userID, 166
 - username, 166
 - youtubeData, 167
- User.cpp, 297, 298
- User.hpp, 298, 300
- userIconBtn
 - MainFrame, 132
- userID
 - ChatWebSocket, 70
 - Friend, 98
 - Message, 134
 - User, 166
- username
 - User, 166
- UserPanel, 167
 - m_confirmDeleteBtn, 171
 - m_controller, 171
 - m_deleteErrorLabel, 171
 - m_deletePasswordField, 171
 - m_emailLabel, 171
 - m_nav, 171
 - m_usernameLabel, 171
 - OnConfirmDelete, 169
 - OnDeleteRequest, 169
 - OnLogout, 169
 - OnReuploadCSV, 170
 - OnShow, 170
 - Refresh, 170
 - RefreshUserInfo, 170
 - SetDeleteReauthVisible, 170
 - UserPanel, 169
- userPanel
 - MainFrame, 132
- UserPanel.cpp, 434, 435
- UserPanel.hpp, 438, 440
- usersWithoutData
 - AppState, 40
- validatePassword
 - DataManager, 85
 - SqliteDataManager, 150
- Video, 172
 - channelID, 175
 - channelLogoURL, 175
 - channelName, 175
 - duration, 175
 - getChannelID, 173
 - getChannelLogoURL, 173
 - getChannelName, 173
 - getDuration, 174
 - getTags, 174
 - getThumbnailURL, 174
 - getTitle, 174
 - getVideoID, 174
 - tags, 175
 - thumbnailURL, 176
 - title, 176
 - toString, 175
 - Video, 173
 - videoID, 176
- video
 - VideoEntry, 186
- Video.cpp, 300, 301
- Video.hpp, 302, 303
- VideoCard, 176
 - Clear, 179
 - DoGetBestSize, 179

- DrawWrappedText, 179
- FetchChannelLogoAsync, 179
- FetchThumbnailAsync, 179
- m_channelID, 181
- m_channelLogo, 181
- m_channelLogoURL, 181
- m_channelName, 181
- m_controller, 181
- m_hovered, 181
- m_logoFailed, 181
- m_logoGen, 182
- m_logoLoading, 182
- m_thumbFailed, 182
- m_thumbGen, 182
- m_thumbLoading, 182
- m_thumbnail, 182
- m_thumbnailURL, 182
- m_title, 183
- m_videoID, 183
- OnChannelLogoLoaded, 180
- OnClick, 180
- OnMouseEnter, 180
- OnMouseLeave, 180
- OnPaint, 180
- OnThumbnailLoaded, 180
- SetVideo, 180
- VideoCard, 178
- VideoCard.cpp, 440, 442
 - LineHeight, 441
- VideoCard.hpp, 446, 448
- VideoEntry, 183
 - getLastWatched, 184
 - getVideo, 184
 - getWatchCount, 185
 - lastWatched, 186
 - setLastWatched, 185
 - setWatchCount, 185
 - toString, 185
 - video, 186
 - VideoEntry, 184
 - watchCount, 186
- VideoEntry.cpp, 304, 305
- VideoEntry.hpp, 305, 306
- videoID
 - Video, 176
- videoList
 - Blend, 44
- videosPerUser
 - RandomBlendAlgorithm, 143
 - WeightedBlendAlgorithm, 195
- wasLastRequestSuccessful
 - HttpClient, 111
- watchCount
 - VideoEntry, 186
- watchHistory
 - YouTubeData, 199
- WatchHistoryParser, 187
 - extractVideoIDFromHref, 188
 - filePath, 190
 - getParserId, 188
 - parse, 189
 - parseMultiThreaded, 189
 - parser_id, 190
 - parseRange, 189
 - WatchHistoryParser, 188
- WatchHistoryParser.cpp, 234, 235
- WatchHistoryParser.hpp, 237, 239
- WatchLaterParser, 190
 - filePath, 192
 - getParserId, 191
 - parse, 191
 - parseMultiThreaded, 191
 - parser_id, 192
 - WatchLaterParser, 190
- WatchLaterParser.cpp, 239, 240
- WatchLaterParser.hpp, 242, 243
- watchLaterVideos
 - YouTubeData, 199
- WeightedBlendAlgorithm, 192
 - generateBlend, 194
 - normalizeWeights, 194
 - participantWeights, 195
 - selectWeightedVideos, 194
 - videosPerUser, 195
 - WeightedBlendAlgorithm, 193
- WeightedBlendAlgorithm.hpp, 212, 213
- write
 - FileBin, 92
- write_callback
 - HttpClient, 112
- writeCallback
 - YouTubeMetadataFetcher, 203
- ws
 - ChatWebSocket, 70
- YouTubeData, 195
 - getLikedVideos, 196
 - getMostWatchedVideos, 196
 - getSubscribedChannels, 196
 - getTopChannels, 197
 - getWatchHistory, 197
 - getWatchLaterVideos, 197
 - likedVideos, 199
 - setLikedVideos, 197
 - setSubscribedChannels, 198
 - setWatchHistory, 198
 - setWatchLaterVideos, 198
 - subscribedChannels, 199
 - toString, 198
 - watchHistory, 199
 - watchLaterVideos, 199
 - YouTubeData, 196
- youTubeData
 - User, 167
- YouTubeData.cpp, 307
- YouTubeData.hpp, 309, 311
- YouTubeDataImporter, 199

- import, [200](#)
- importMultiThreaded, [200](#)
- lowerExtension, [200](#)
- YouTubeDataImporter.cpp, [243](#), [244](#)
- YouTubeDataImporter.hpp, [245](#), [246](#)
- YouTubeMetadataFetcher, [201](#)
 - apiKey, [203](#)
 - enrich, [202](#)
 - fetchChannelLogos, [202](#)
 - fetchVideoBatch, [202](#)
 - httpGet, [202](#)
 - kBatchSize, [203](#)
 - writeCallback, [203](#)
 - YouTubeMetadataFetcher, [202](#)
- YouTubeMetadataFetcher.cpp, [340](#), [341](#)
 - json, [341](#)
- YouTubeMetadataFetcher.hpp, [344](#), [345](#)
- YouTubeMetadataFetcher::VideoMeta, [186](#)
 - channelId, [187](#)
 - channelName, [187](#)
 - thumbnailUrl, [187](#)
 - title, [187](#)
- YTBLNDApp, [204](#)
 - controller, [205](#)
 - OnExit, [205](#)
 - OnInit, [205](#)